



Open Chiplet Atlas Harness (OCAH)

Technical Reference Manual

Version 0.5.12, 2026-06-17

Table of Contents

Revision History	2
1. About OCAH	4
1.1. Motivation	4
1.2. OCA Standard Compliance	4
1.3. Use Cases	4
1.3.1. Secure Chiplet Boot and Chain of Trust	5
1.3.2. System Bring-up and Operational Management	5
1.3.3. Interoperable Chiplet Integration	5
1.3.4. Firmware Distribution and Control-Plane Coordination	5
1.3.5. Cryptographic Services	5
1.3.6. Debug, Test, and Serviceability	6
1.3.7. Reference Platform for Ecosystem Adoption	6
2. Getting Started	7
2.1. Dependencies	7
2.1.1. System Requirements	7
2.1.2. Python Dependencies	7
2.1.3. Hardware Requirements	7
2.1.4. License Requirements	7
2.1.5. Environment Setup	7
2.2. Tools	8
2.2.1. General Development Tools	8
2.2.2. Documentation Tools	8
2.2.3. Hardware Development Tools	8
2.2.4. Software Development Tools	9
2.2.5. Software Runtime	9
2.3. Methodology	9
2.3.1. Standards	9
2.3.2. Third Party Code	10
2.3.3. Development Tools	10
2.3.4. Document Processing	11
2.3.5. Asciidoctor Setup	11
2.3.6. Build Process	11
2.3.7. GitHub Source Code Control and Development	12
2.3.8. Tracking Tasks and Bugs with GitHub	13
2.3.9. Pull Requests	14
2.3.10. Generated Content	14
2.3.11. Development Flow	14
2.3.12. Main/release branch proposal	14
2.3.13. Versioning	15
2.3.14. Build with Bender and Make	15
2.3.15. Simulation with Verilator	15
2.3.16. Testbench and Test Sequence	16

2.3.17. Visual Debugging and Waveform Viewer	16
2.3.18. Verilog Style and Lint checking with Verible	17
2.3.19. Component Design	17
2.3.20. CSRs : Configuration and Status Registers	17
2.3.21. Special Considerations for Cryptographic Components	18
2.3.22. Unit Testbenches	19
2.3.23. Fuse Map	19
2.3.24. Interconnect Crossbar Generators	19
2.3.25. Component Directory Organization	19
2.3.26. SystemVerilog Coding Guidelines	20
2.3.27. RTL Code Comments	20
2.3.28. Configuration	20
OCAH Architecture & Design	21
3. OCAH Architecture	22
3.1. OCAH High-Level Features	22
3.2. Top Block Diagram	22
3.2.1. SMU AXI Crossbar	23
3.3. OCAH Memory Map	24
3.4. Power Domains	24
3.5. Clock Domains	25
3.6. Reset	25
4. Security Processor (SEP)	27
4.1. Introduction	27
4.1.1. Threat Model	27
4.1.2. Attack Countermeasures	28
4.1.3. Debug and DFT Security Considerations	28
4.2. Hardware Overview	30
4.3. SEP Interconnect	32
4.3.1. Top Level AXI4	32
4.3.2. Subsystem External Interfaces	33
4.3.3. Local and External AXI4 Traffic Remapping	33
4.4. Memory Map	36
4.5. Detailed Register Map	38
4.5.1. CPU Control and Interrupts	38
4.5.2. Reset and Lifecycle	45
4.5.3. Scratch and eFuse	46
4.5.4. Timers	57
4.5.5. Cryptographic Accelerators	59
4.6. CPU Subsystem	76
4.6.1. CPU Feature Summary	76
4.6.2. Configuration Straps	76
4.6.3. Reset Vector	77
4.6.4. Boot ROM	77
4.6.5. BL0 Secure Boot and BL1 Execution Unlock	77

4.6.6. Dual Scratch Register Banks	78
4.6.7. Direct Memory Access Controller (DMA)	79
4.6.8. Alarm Aggregation Processor (AGP)	80
4.7. Cryptographic Subsystem	81
4.7.1. HMAC-SHA2	82
4.7.2. OTBN PKA Public Key Accelerator	82
4.7.3. AES Cipher	83
4.7.4. KMAC-SHA3	84
4.7.5. True Random Number Generator (TRNG)	84
4.7.6. Entropy Source	86
4.7.7. DRBG Deterministic Random Bit Generator	88
4.7.8. Key Manager (KM)	89
4.7.9. Fuse Controller (OTP)	156
4.8. Peripheral Subsystem	162
4.8.1. Serial Peripheral Interface (SPI)	163
4.8.2. Watchdog Timer (WDT)	163
4.8.3. SEP Reset Controller	164
4.8.4. Token Processing	164
4.8.5. Security Disable (SEC_DIS)	167
4.8.6. Life Cycle Controller (LCC)	168
4.9. Test Mode Entry	181
5. System Management Controller (SMC)	182
5.1. Overview	182
5.2. Memory Map	183
5.2.1. Address Space Organization	184
5.2.2. Detailed Address Map	184
5.2.3. AXI-Lite External Window	186
5.2.4. Register Interface Standards	187
5.2.5. Access Control and Security	187
5.2.6. Component-Specific Register Documentation	187
5.3. Clock and Reset Management	187
5.3.1. Clock Architecture	188
5.3.2. Reset Architecture	189
5.3.3. Clock and Reset-Based Power Management Integration	195
5.4. CPU & Memory System	196
5.4.1. Processor Core Overview	196
5.4.2. Cache and Memory Hierarchy	196
5.4.3. Reset, Boot, and Initialization	196
5.4.4. Memory Repair	197
5.4.5. System Interfaces	197
5.4.6. Cluster Boundary Isolation	198
5.4.7. Interrupt and Control Integration	198
5.4.8. Watchdog Timer System	199
5.4.9. Advanced Bus Infrastructure Monitoring and Error Management	199

5.4.10. Debug	199
5.4.11. Detailed Register Map	199
5.5. Boot ROM	209
5.5.1. ROM Architecture	209
5.5.2. ROM Hardware Configuration	209
5.5.3. ROM Boot Code	210
5.5.4. Integration and Operation	211
5.6. Interrupts	211
5.6.1. Interrupt controllers (MMIO)	211
5.6.2. SMC interrupt vector (<code>cpu_interrupts_o</code>)	212
5.6.3. SMC CPU Interrupt Vector Map	212
5.6.4. RISC-V PLIC	215
5.6.5. System Management Interrupt Source Integration	215
5.6.6. Multi-Tier Interrupt Classification and Response Strategy	216
5.6.7. Multi-Core Interrupt Distribution and Load Management	216
5.6.8. RISC-V CLINT and Precise System Management Timing	216
5.6.9. Fault Detection and Recovery Architecture	216
5.6.10. Advanced PLIC Management and Adaptive Priority Control	216
5.6.11. Multi-Context Architecture and Privilege Level Integration	217
5.6.12. Precision Timer Management and Inter-Core Coordination	217
5.6.13. Register Interfaces	217
5.7. Fabric	222
5.7.1. Configuration Parameters	222
5.7.2. Dual-Network Architecture	223
5.7.3. AXI Interface Widths	223
5.7.4. Traffic Managers and Subordinates	224
5.7.5. Address Remapping	225
5.7.6. Alias Remapping	225
5.7.7. Privilege-Controlled Remapping	225
5.7.8. Inbound Filtering	226
5.7.9. Outbound Filtering	226
5.7.10. Protection Requirements	226
5.7.11. Traffic Routing and Flow	228
5.7.12. Detailed Register Maps	229
5.8. Direct Memory Access (DMA) Engine	231
5.8.1. Architecture Overview	231
5.8.2. Configuration Parameters	232
5.8.3. DMA Frontend	232
5.8.4. DMA Backend	233
5.8.5. DMA Configuration and Control	234
5.8.6. System Integration	235
5.8.7. Detailed Register Map	235
5.9. Zeroer	244
5.9.1. Use Cases	245

5.9.2. Architecture	245
5.9.3. Operation Details	247
5.9.4. Detailed Register Map	247
5.10. Peripherals	247
5.10.1. Mailbox Unit	248
5.10.2. System Timer OCTS	257
5.10.3. eFuse	271
5.10.4. UART 16550	288
5.10.5. Log Engine	294
5.10.6. I2C Controller	301
5.10.7. AVSBus Controller	345
5.10.8. GPIO Peripheral	363
5.10.9. Telemetry Receiver	372
5.11. eFuse Shadow Register Scan Protection	378
5.11.1. SMC eFuse Asset Classification	379
5.11.2. Implementation	379
5.11.3. Downstream Lock Path	380
5.11.4. Notes for DFT	380
5.11.5. Verification	380
6. Debug and Test Ports (DTP)	381
6.1. Overview	381
6.1.1. Purpose	381
6.1.2. Key Features	382
6.1.3. Specifications	382
6.2. JTAG Interface Unit	383
6.2.1. Overview	383
6.2.2. Configuration Parameters	383
6.2.3. Architecture	383
6.2.4. Interface	386
6.3. JTAG PTAP	388
6.3.1. Overview	388
6.3.2. Configuration Parameters	388
6.3.3. Architecture	389
6.3.4. Interface	405
6.4. JTAG STAP	406
6.4.1. Overview	407
6.4.2. Configuration Parameters	407
6.4.3. Architecture	407
6.4.4. Interface	410
6.5. Cross Trigger Network	410
6.5.1. Overview	411
6.5.2. Purpose	411
6.5.3. Configuration Parameters	411
6.5.4. Architecture	411

6.5.5. Interface	420
6.5.6. Memory Map	422
6.6. Cross Trigger Port	422
6.6.1. Overview	422
6.6.2. Configuration	423
6.6.3. Architecture	423
6.6.4. Interface	428
6.7. Cross Trigger Matrix	429
6.7.1. Overview	429
6.7.2. Configuration Parameters	429
6.7.3. Architecture	430
6.7.4. Interface	431
6.8. DTP JTAG Integration	432
6.8.1. DTP JTAG Topology	432
6.8.2. Clock Domains	432
6.8.3. Reset Architecture	432
6.8.4. Security Integration	433
6.9. Clock Stop	433
6.9.1. Architecture	433
7. System Management Network	434
8. AXI-Over-UCIe Bridge	435

The Open Chiplet Atlas Harness (OCAH) is a chiplet design IP where the non-application-specific functions are integrated to form a chiplet design framework. These non-application-specific functions include common chiplet subsystems like Security Processor (SEP), System Management Controller (SMC), System Management Network (SMN), AXI-over-UCIe bridge (AoU), Debug & Test Ports (DTP), etc.

Revision History

This document tracks major revisions and updates to the OCAH documentation.

Table 1. Revision History

Version	Date	Author	Description
0.0.1	2025-10-23	Alessandro Ottaviano	Initial documentation structure setup. Added methodology, microarchitecture, dependencies, tools, and history sections. Reorganized documentation system with distributed IP documentation approach. Added SMC documentation draft.
0.0.2	2026-01-08	Alessandro Ottaviano	Added SMC Integration guideline. Added SMC eFuse shadow register address mapping. Fixed eFuse memory map to include detailed register field mappings (I2C/I3C_ID, CHIPLET_ID, etc.).
0.0.3	2026-01-28	Alessandro Ottaviano	Clarified OCTS timer architecture: runtime primary/secondary distinction via strap.
0.1.0	2026-01-29	Alessandro Ottaviano	Added DTP architecture and microarchitecture specs.
0.2.0	2026-02-12	Alessandro Ottaviano	Added OCAH SMC FuSa specs. Clarified SMC fabric protocols in CPU cluster and memory system size.
0.3.0	2026-02-20	Alessandro Ottaviano	Added OCAH SEP specs. Added OCAH top-level figure and brief description.
0.3.1	2026-03-10	Alessandro Ottaviano	Fixed SMC memory map doc (memmap.adoc) to match RDL: corrected ECAM, M-mode, Xvisor, PLIC, CLINT, BEU addresses/sizes; added missing components (DTP, DFT, CLA, I3C, Mailbox).
0.3.2	2026-03-18	Alessandro Ottaviano	Added FuSa requirement mapping to FuSa specs; Added SMC peripheral clock specs; updated SMC top figure with highlighted clock domains; updated SEP specs.
0.3.3	2026-03-24	Alessandro Ottaviano	Clarified bus and register parity generation and checking; Clarified strobe gating for masked (partial) writes.
0.3.4	2026-03-24	Alessandro Ottaviano	Add detailed breakdown of register types for register parity checking.
0.3.5	2026-03-31	Alessandro Ottaviano	Added parity handling for protocol converters (ID remap, burst splitter, ID prepend, address fixer).
0.4.0	2026-04-09	Alessandro Ottaviano	Expanded DTP and SEP documentation with microarchitecture details. Expanded Scrambler, Key Manager, Entropy Source, and DRBG DTP architecture specs with RTL-derived parameter tables, algorithm details, and integration descriptions. Enriched SEP crypto accelerator sections (OTBN, AES, KMAC, HMAC) and peripheral sections (SPI, WDT, eFuse) with wrapper integration details and OpenTitan cross-references.
0.4.1	2026-04-11	Alessandro Ottaviano	Refactored OCAH Architecture chapter with Power Domains, Clock Domains, Reset, and Address Map sections. Added RTL-backed port tables for SMU, SMC, SEP, and DTP. Renamed and streamlined Integration Guide. Replaced placeholder files with proper content (Motivation, OCA Compliance, Use Cases). Added cover page version and date.
0.5.0	2026-04-11	Alessandro Ottaviano	Open to internal review.
0.5.1	2026-04-22	Alessandro Ottaviano	Refreshed SEP crypto entropy routing description: AES, KMAC, and OTBN EDN ports now fed from post-CSRNG DRBG bits; generic external-entropy-stream option documented.
0.5.2	2026-05-12	Alessandro Ottaviano	Corrected SIB acronym expansion in DTP/JTAG architecture docs.
0.5.3	2026-05-25	Haohao Liao	Updated SEP Lifecycle Controller chapter with feedback from F2F meeting.
0.5.4	2026-05-29	Alessandro Ottaviano	Removed legacy-design references from OCAH AsciiDoc documentation and rewrote provenance-only third-party references as OCAH-owned requirements and behavior.
0.5.5	2026-06-06	Haohao Liao	Added Key Manager key-management architecture: per-UID software/hardware root keys, deterministic UID-derived key hierarchy, requester-selected chip configuration, configuration-tied key revocation, ECC key metadata in signatures, and user input keys.

Version	Date	Author	Description
0.5.6	2026-06-06	Haohao Liao	Refined the SEP Lifecycle Controller feature-control section per review: reordered it to define the disable vector (SIP_DIS / SYS_DIS) before the per-LC-state feature-control result, added an Overrides and Final Gates description (SEC_DIS override and the SECURE_TM final gate on the test group), and removed the obsolete screening/diagnostic test definitions.
0.5.7	2026-06-07	Haohao Liao	Key Manager: moved UID and hardware-root-key read-locks to the warm reset domain (re-applied by ROM on every boot), matching the warm-reset KM CPU.
0.5.8	2026-06-09	Haohao Liao	Updated Key Manager spec per review feedback: clarified OTP inputs (UIDs and life-cycle hardwired; demotion from LCC), the eFuse access path, KDF concatenation notation, 512-bit user_input, and mailbox-only user-input-key injection.
0.5.9	2026-06-10	Haohao Liao	SEP Lifecycle Controller per review: corrected DEMOTE feature-control scope (TEST_DEV honours SIP_DIS/SYS_DIS; RMA_CHIPLET already opens debug), unified the T/F convention (T = feature enabled), and reframed scan protection (removed the secure_tm shadow-register disconnect; Class 1 non-scannable, Class 2 scandump-gated).
0.5.10	2026-06-11	Alessandro Ottaviano	Documented PLIC and mie CSR software initialization requirements before enabling external interrupts.
0.5.11	2026-06-12	Haohao Liao	Fixed contradictory descriptions across the SEP spec following the Lifecycle Controller update.
0.5.12	2026-06-17	Haohao Liao	Aligned SEP scan / scan-dump asset protection in the Lifecycle Controller and Key Manager chapters: crypto / KM secret-bearing storage (including the KPV key vault) is scannable but never scan-dumpable (secure_tm-gated), and reconciled the Attack Countermeasures chapter to match.

Chapter 1. About OCAH

1.1. Motivation

The Open Chiplet Atlas Harness (OCAH) exists to bridge the gap between a chiplet interoperability architecture and a deployable chiplet design. The OCA architecture defines the rules needed to make chiplets from different vendors plug together at the software, system, protocol, transport, and physical layers, but real adoption depends on turning those rules into reusable implementation collateral. OCAH is that implementation vehicle: an OCA-compliant, open-source chiplet framework intended to make interoperability practical rather than purely conceptual.

The motivation for OCAH is rooted in the cost and complexity of modern silicon development. Advanced chips require a large amount of non-differentiating but essential infrastructure, including security, boot, system management, debug, test, and die-to-die integration. Recreating or licensing these subsystems for each chiplet raises cost, extends schedules, and increases integration risk, which in turn limits who can participate in advanced silicon design. OCAH addresses this by packaging the common chiplet-management and infrastructure logic into a reusable framework so that adopters can spend their effort on their own value-differentiating application logic.

A key design premise behind OCAH is that most interoperability requirements in OCA land in the chiplet framework rather than in the application subsystem itself. For that reason, OCAH integrates the non-application-specific functions that repeatedly appear across chiplets, such as the Security Processor (SEP), System Management Controller (SMC), System Management Network (SMN), debug/test access, and, in the broader roadmap, bridges such as AXI-over-UCIe and die-to-die support. By reusing this framework across chiplets, designers gain OCA-aligned boot, security, management, and communication behavior by construction, instead of rebuilding those capabilities from scratch in every project.

The open-source release of OCAH is therefore motivated not only by reuse inside Tenstorrent, but by ecosystem growth. Tenstorrent's stated goal is to enable broader adoption of an open chiplet ecosystem by removing objections to integration, improving out-of-box usability, and making high-quality common infrastructure available under a permissive license. In that sense, OCAH is both a technical framework and an adoption mechanism: it reduces duplicated engineering work, lowers the barrier to interoperable chiplet design, and gives partners a concrete foundation on which to build OCA-compliant chiplets and systems.

1.2. OCA Standard Compliance

OCAH is compliant with the latest [OCA](#) specifications.

WARNING

This section is currently under development. More content will be added as the project matures.

1.3. Use Cases

The following use cases illustrate how OCAH provides value as a system-level framework for reusable chiplet infrastructure. Rather than focusing on individual internal blocks, these use cases highlight the recurring integration, boot, security, management, and serviceability problems that OCAH is intended to solve across a broad range of chiplet-based systems.

1.3.1. Secure Chiplet Boot and Chain of Trust

OCAH provides a reusable foundation for secure chiplet initialization, including boot sequencing, authentication, firmware loading, and the establishment of a trusted execution context before application logic is released.

The SEP ROM BL0 image uses non-volatile public keys to conduct authentication verification on loaded BL1 software images, and feature control tied to the authentication result ensures the chain of trust cannot be bypassed. Device attestation is also supported by recording the boot process and device configuration and performing attestation using public key infrastructure.

This is one of the most important adoption use cases for OCAH because secure boot is required in nearly every deployable chiplet system, yet it is also one of the most repetitive and integration-sensitive pieces of infrastructure.

1.3.2. System Bring-up and Operational Management

OCAH supports the non-application control functions needed to bring a chiplet or multi-chiplet package into operation and keep it there. This includes initialization of platform resources, configuration of management interconnects, power and reset coordination, telemetry collection, lifecycle control, and the handling of service operations that must exist regardless of the application implemented on top of the chiplet.

Runtime voltage rail management is achieved by communicating with Voltage Regulators via the standard PMBus and AVSBus protocols, while Performance State (P-State) management employs distributed sensors and firmware to enforce power and thermal constraints through mechanisms such as dynamic frequency scaling (DFS) and dynamic voltage and frequency scaling (DVFS).

Fuse-based on-chip storage provides configuration values for SRAM repair, sensor trimming, and fabric setup, with the SMC responsible for fuse sensing and distribution.

1.3.3. Interoperable Chiplet Integration

A central OCAH use case is to let adopters integrate application chiplets into an OCA-based system without having to build all surrounding framework logic themselves. By providing common infrastructure for management, security, debug, and communication, OCAH reduces the amount of custom glue logic needed at integration time and makes it easier to combine chiplets from different teams or vendors within a consistent system architecture.

Inter-chiplet coordination relies on standard protocols and transport mechanisms mandated by OCA, including I3C for early-stage boot image and configuration delivery.

1.3.4. Firmware Distribution and Control-Plane Coordination

In many chiplet systems, not every chiplet owns an independent external boot source or complete standalone management stack. OCAH enables a system-level model in which one chiplet or management endpoint can provision firmware, coordinate initialization, and establish runtime policy across other chiplets.

This makes OCAH especially useful in asymmetric systems where different chiplets have different boot roles, capabilities, and trust responsibilities.

1.3.5. Cryptographic Services

OCAH provides a physically isolated execution environment for cryptographic operations through the SEP subsystem. Secure key services, hardware-accelerated symmetric and asymmetric cryptography, and a True Random Number

Generator (TRNG) are available to both SEP firmware and the rest of the system.

User authentication is another service the framework offers: the authentication result can be tied to the secret key derivation so that bypassing authentication renders the system unusable. These capabilities allow adopters to build security features on top of OCAH without provisioning their own cryptographic infrastructure.

1.3.6. Debug, Test, and Serviceability

OCAH is also intended to support the practical needs of development and production beyond nominal runtime operation. Common debug and test access, manufacturing-oriented initialization flows, and service interfaces for fault handling and recovery are all use cases that benefit from a shared framework.

Chiplet status reporting is provided through a dedicated UART-based log engine: internally generated log information from different in-package agents is forwarded to the primary SMC and sent out via its UART port. Providing these mechanisms as part of OCAH improves consistency across chiplets and reduces the effort needed to make a design usable in validation, bring-up, and field support.

1.3.7. Reference Platform for Ecosystem Adoption

Beyond any single chiplet implementation, OCAH serves as a reference framework for organizations adopting OCA. It gives designers a concrete starting point for how boot, security, management, and interoperability infrastructure can be realized in practice, which lowers the barrier to entry for new participants and accelerates ecosystem convergence around common implementation patterns.

Chapter 2. Getting Started

2.1. Dependencies

WARNING

This section is currently under development. **Content has been taken verbatim from existing documentation.** More content will be added as the project matures.

This section outlines the system requirements and dependencies for the OCAH project.

2.1.1. System Requirements

- **Linux Distribution:** Ubuntu 20.04+ or CentOS 8+
- **Python:** Version 3.9.18 or later
- **Memory:** Minimum 16GB RAM recommended for full builds
- **Storage:** Minimum 50GB available disk space

2.1.2. Python Dependencies

The project uses a Python virtual environment for SystemRDL and documentation tools:

- **PeakRDL:** SystemRDL processing and RTL generation
- **Asciidoctor:** AsciiDoc documentation processing and PDF generation
- **Antora:** Multi-repository documentation site generator for AsciiDoc
- **Python packages:** Managed by the repo-level `pyproject.toml` and `uv.lock`; run `make uv-sync` or `uv sync` from the repository root.

2.1.3. Hardware Requirements

For simulation and synthesis:

- **CPU:** Multi-core processor (8+ cores recommended)
- **RAM:** 16GB minimum, 32GB recommended for large designs
- **Storage:** SSD recommended for build performance

2.1.4. License Requirements

- **Open Source Tools:** No additional licenses required
- **Commercial Tools:** Valid licenses required for VCS, Design Compiler, etc.
- **Dependencies:** Some external IPs may have specific license requirements

2.1.5. Environment Setup

Refer to the project setup documentation for detailed installation instructions of these dependencies.

2.2. Tools

WARNING

This section is currently under development. **Content has been taken verbatim from existing documentation.** More content will be added as the project matures.

This section outlines the development tools used in the OCAH project, categorized by purpose and licensing model.

FOSS identifies Free Open Source Software tools. If the tool doesn't have such a tag, then by default it is a commercial tool.

We will also develop a set of custom tools for OCAH for tasks such as AXI/APB crossbar generation, SystemRDL processing and so on. These will be documented here.

2.2.1. General Development Tools

Required Tools

- **Git**: Source code repository (FOSS)
- **Python 3.9+**: For build scripts and SystemRDL tools (FOSS)
- **Bender**: Manage IP dependencies and build automation (FOSS)
- **Make**: Build HW and SW (FOSS)
- **PeakRDL**: SystemRDL processing and RTL generation (FOSS)

Optional Tools

- **Bazel**: Alternative build system (FOSS)

2.2.2. Documentation Tools

- **Asciidoctor**: AsciiDoc documentation processing and PDF generation (FOSS)
- **Antora**: Multi-repository documentation site generator for AsciiDoc (FOSS)
- **DrawIO**: Create diagrams and generate SVG figures (FOSS)
- **Wavedrom/Wavedrompy**: Timing diagrams (FOSS)

2.2.3. Hardware Development Tools

Open Source Tools (FOSS)

- **Verilator**: HDL simulator
- **Verible**: Linting and style checker
- **Svlint**: SystemVerilog linter
- **Surfer**: Waveform viewer
- **Yosys**: Logic synthesis tool

Commercial Tools

- **VCS:** Simulator (requires license)
- **Verdi:** Waveform viewer (requires license)
- **AscentLint:** Advanced linting (requires license)
- **Design Compiler:** Logic synthesis (requires license)
- **PrimeTime:** Static timing analysis (requires license)
- **Vivado:** FPGA synthesis and implementation (requires license)

2.2.4. Software Development Tools

- **GCC:** C/C++ compiler suite (FOSS)
- **Clang:** Alternative C/C++ compiler (FOSS)
- **Rust:** Rust compiler suite (FOSS)
- **Valgrind:** Memory debugging and profiling (FOSS)
- **Renode:** System emulation (FOSS)

2.2.5. Software Runtime

- **TockOS:** Real-time OS for SEP (FOSS)
- **Zephyr OS:** RTOS for SMC (FOSS)

2.3. Methodology

WARNING

This section is currently under development. **Content has been taken verbatim from existing documentation.** More content will be added as the project matures.

This document specifies the development methodology for this project.

2.3.1. Standards

The OCAH design will follow industry or FOSS standards where appropriate:

- SystemVerilog RTL
- AMBA system buses
- SystemRDL memory map
- AsciiDoc documentation with Asciidoctor

The hardware design is written in a synthesizable subset of SystemVerilog RTL language. The coding standards are adopted from the lowRISC organization which are also used in OpenTitan.

System interconnect is implemented with AMBA standard buses. AXI4 and AXI4-lite is used for data interfaces and is generally configured with 64-bit data and address. Control buses for accessing CSRs and other low speed sources are implemented with 32-bit APB.

The system memory map and component CSRs (Configuration and Status Registers) are defined in SystemRDL. This is the single source of truth and all downstream representations are rendered from this source. Examples are documentation (HTML), C headers, SystemVerilog UVM DV and SystemVerilog RTL.

2.3.2. Third Party Code

The OCAH project leverages third party open source RTL code whenever practical in order to boost productivity and not "reinvent the wheel". The primary projects we reference are:

- [ETH PULP](#)
- [lowRISC OpenTitan](#)
- [OCP Caliptra](#)

This is not an exclusive list and there will inevitably be other open source projects added. Furthermore, we will leverage open source tools project which are described in more detail below.

There are strict rules around the use of open source RTL. OCAH must not incorporate any code governed under a GPL license. We will only permit incorporation of code governed under the liberal Apache 2.0 or equivalent licenses such as Solderpad. The reason is that we do not want to be encumbered by any GPL-like restrictions as we intend to ultimately release the OCAH design with a liberal Apache 2.0 type licence. If you have any doubts or questions, contact OCAH project managers for guidance.

The licences discussed above are documented here. We may add some others to this list.

- <https://www.apache.org/licenses/LICENSE-2.0>
- <https://solderpad.org/licenses/SHL-2.1/>

Third party components will be forked into the OCAH repository to decouple them from the originating project. We may still track the original project and update these forked versions if there is a good reason. Some reasons include incorporating bug fixes and feature enhancements.

We fully expect that third party components will be modified to some extent. This would be done by adding a wrapper around the design or portions of it, attempting to leave the core functionality unmodified. This may not always be possible, so the exact method of modification will vary.

An example modification would be the CSR (Configuration and Status Register) design. As described further below, we intend to use SystemRDL for this purpose and generate those implementations. OpenTitan components for example, use a different CSR mechanism which will be replaced so that requires modifications. Another example is that OpenTitan components use TileLink for their system bus interfaces. OCAH will use AMBA APB and AXI4, so again, these will need to be replaced.

When adopting and potentially modifying third party modules, make sure to follow all of the license rules such as keeping all attribution comments in the file headers.

2.3.3. Development Tools

We aim to primarily use open source tools for the development of OCAH in order to eliminate commercial dependencies and enabling open-sourcing not only the design, but also the associated tools.

The goal is to enable new adopters of OCAH to be able to immediately perform simulations and other analyses of the design using readily available free tools.

For internal development and testing we will employ commercial tools to validate the OCAH design, proving readiness for chip deployment i.e. tapeout. This include:

- UVM verification
- lint tools for code quality
- logic synthesis

2.3.4. Document Processing

OCAH uses AsciiDoc for documentation:

- **Component README files:** Written in Markdown (.md) format for GitHub compatibility
- **Detailed documentation:** Written in AsciiDoc (.adoc) format for structured documentation
- **Auto-generated registers:** Generated as AsciiDoc (.adoc) files from SystemRDL sources

This approach provides:

- GitHub-native rendering for README files
- Professional documentation rendering with Asciidoctor for detailed specs
- Human readable document source text that can be revision controlled
- Multiple output formats (PDF, HTML) with cross-referencing capabilities

2.3.5. Asciidoctor Setup

Asciidoctor processes the AsciiDoc documentation. Install the documentation toolchain:

For Asciidoctor (Ruby gem):

```
gem install asciidoctor asciidoctor-pdf
```

For Antora (Node.js):

```
npm install -g @antora/cli @antora/site-generator-default
```

Asciidoctor is also available via the system environment:

```
gem install asciidoctor asciidoctor-pdf
```

2.3.6. Build Process

Generate HTML documentation:

```
cd doc && make html
```

Generate PDF documentation:

```
cd doc && make pdf
```

Diagrams are most easily created using the DrawIO program. Generate SVG files and embed them directly in the AsciiDoc document.

Timing diagrams are created with the Wavedrom program. The interactive application makes it easy to create the JSON source files and edit them while the CLI version of the application is used to create the output SVG files that are embedded in the AsciiDoc document.

Install Wavedrom waveform generator and viewer:

```
pip3 install wavedrom  
  
sudo npm install -g wavedrom # for the interactive editor  
  
sudo npm install wavedrom-cli -g # for the CLI application
```

The editor is launched with `wavedrom-editor` or directly from the website. Save the JSON source file for the figure and generate the SVG rendering as follows:

```
wavedrom-cli -i mywaveform.json5 > mywaveform.svg
```

Then embed the SVG figure into the AsciiDoc document.

References:

- [Asciidoctor](#)
- [drawio](#)
- [wavedrom](#)

2.3.7. GitHub Source Code Control and Development

OCAH development is hosted on GitHub under the Tenstorrent organization. Clone the public repository and use standard GitHub workflows for issues, pull requests, and code review.

Development can proceed on either macOS or Linux. Open-source tools work on both platforms; commercial EDA tools such as VCS and Design Compiler are typically available on shared Linux build hosts when needed.

Follow these steps to get set up and started with development.

\1. Create a GitHub account if you do not already have one.

\2. Configure Git credential management on your machine:

+

```
brew install git-credential-manager
```

\3. Generate an SSH key pair:

+

```
ssh-keygen -t ed25519
```

\4. Start the SSH agent and add your key. You may want to add this to your shell startup script:

+

```
eval "$(ssh-agent -s)"  
ssh-add <name of your key>
```

\5. Add the public key to your GitHub account under <https://github.com/settings/keys>. Verify the key with:

+

```
ssh-add -l
```

\6. Clone the repository and create a feature branch for your work.

+

```
git clone git@github.com:tenstorrent/tt-oca.git  
cd tt-oca  
git checkout -b <your-branch>
```

2.3.8. Tracking Tasks and Bugs with GitHub

Development tasks are managed with GitHub issues. Issues may have parent and child issues, documenting task dependencies throughout the design. Issues are assigned to milestones to track overall development progress, and completion of milestones may trigger design reviews or progress updates as necessary. Tags help filter and sort issues that have certain properties.

Issues can also track bugs. Design, verification, and firmware engineers may file bug issues against the design. Bug issues should be assigned to the owner of the affected code, who may then reassign the issue as appropriate. Bug issues should have a thorough explanation of the observed problem including:

- A copy of, or link to, the program or simulator output.
- Any suggested resolution to the problem.
- Most importantly, information on how to reproduce the issue.
 - Git hash
 - Executed command
 - Program input
 - Simulation seed
 - etc.

An issue template will assist users in filling out all of the relevant information.

2.3.9. Pull Requests

A pull request (PR in GitHub terminology or MR, merge request, in GitLab terminology) is GitHub's process of controlled merging between branches. We make use of this to document major changes to the codebase and facilitate code review.

2.3.10. Generated Content

Content automatically generated from source should *not* be tracked with Git. Makefiles and scripts should generate this content on-demand as part of build and simulation flows. Tracking generated content has many downsides including:

- Repository bloat from constantly changing generated content (especially from binary files).
- Unintended inconsistencies between source and generated content (e.g. bad code goes to tapeout because a designer forgot to run the RTL code generation script before checking in critical source script changes.) Missing content is always preferable to incorrect content.
- Unnecessary and confusing merge conflicts that may not be resolved properly, potentially leading to the same sort of design errors mentioned above.

The following list contains exceptions for tracking generated content:

- Published specs and documentation generated from source for viewing directly from GitHub.

We can consider setting up [Git LFS](#) to track large binary content if absolutely necessary.

2.3.11. Development Flow

We strive for a development process that balances agility and sanity. To that end, we use the following conventions and processes:

2.3.12. Main/release branch proposal

- Development of each feature should happen on its own, dedicated branch, preferably attached to a GitHub issue to facilitate discussion and review.
- The `main` branch is the target for all feature PRs.
 - Direct commits to `main` are not allowed. All changes must come through a PR. The main branch will be protected and no direct pushes are allowed.

- All PRs targeting `main` must pass a battery of basic smoke tests automatically run by the GitHub CI before a merge is allowed. (Note: early in the project, there may not be meaningful tests, but these will be added as the project matures.)
- The `main` branch should be kept 'working' at all times, meaning that every version should be useful for continued development and testing. Developers should be able to pull the latest `main` branch and run the code without major issues.
- When a release is close to ready a release branch is created from `main` and named with the release major.patch version. This branch is used to finalize the release, e.g. by updating documentation, fixing any last minute bugs, etc.
 - All changes to the release branch are based on PRs with GitHub issues targeting the release branch. If these changes also apply to `main`, then a PR should be created against those branches as well.
 - The release branch is also protected and no direct pushes are allowed.
 - All changes targeting the release branch must pass a battery of basic smoke tests automatically run by the GitHub CI before a merge is allowed as well as additional release specific tests.
 - Since changes to the release branch are also applied to `main`, the release branch is not merged back into `main` when the release is finalized. A trial merge can be done to ensure that all relevant changes are applied to `main`, but this is not strictly necessary.
 - When the release is finalized, the release branch is tagged with the release version.
 - Minor (point) releases are made on the same major release branch, e.g. `v1.0.x`, where `x` is the minor version number. This allows users to pull from the 'v1.0.x' branch to get the latest bug fixes and minor enhancements without having to switch to a new major release branch.

2.3.13. Versioning

The OCAH project will use semantic versioning (semver) version 2 for releases. The version format is `MAJOR.MINOR.PATCH`, where:

- MAJOR version when you make incompatible API changes
- MINOR version when you add functionality in a backward compatible manner
- PATCH version when you make backward compatible bug fixes

Additional labels for pre-release and build metadata are available as extensions to the MAJOR.MINOR.PATCH format.

See [semver](#) for more details.

OPEN: We need to adapt the idea of "API changes" to hardware components and document here.

2.3.14. Build with Bender and Make

Bender is used for managing IP dependencies and managing builds.

Makefiles will be used for build automation. We have previously considered other flow such as Bazel but are hesitant to take on extra complexity.

2.3.15. Simulation with Verilator

We will use open source tools as much as possible and hence will focus on using Verilator for unit tests. Verilator has comprehensive SystemVerilog (SV) support and is widely used in industry.

Verilator ingests the RTL code along with the test code written in C/C to generate a program which is compiled and run with gcc. Writing tests in C/C has many benefits:

- high performance, typically better than commercial simulators
- incorporate third party reference code for ciphers as an example
- incorporate code for analysis
- enable development of emulation systems for firmware developers
- can be migrated to DPI based DV environments for sign-off verification

References:

- [Verilator documents](#)

2.3.16. Testbench and Test Sequence

Verilator is our main simulation engine, and we have several choices on how to create unit test benches and sequences for IP components and the SEP subsystem:

- Plain *vanilla* SystemVerilog
- C/C++
- Python with cocotb

The VeeR EL2 CPU which is used in SEP has cocotb based testbenches, while Caliptra uses mostly C with Verilator. Either choice will be fine and it is *TBD* which one we shall emphasize.

We expect that chip integrators and TT verification engineers will use UVM in their verification flows. The UVM flow is however not in scope for the open source release of the OCAH project currently. We *may* release the UVM verification infrastructure to assist teams in full chip integration efforts, but that remains TBD.

Antmicro has recently released a code coverage dashboard tool which works with Verilator. This *may* be used in the verification flows, but is TBD based on future evaluations.

References:

- [cocotb](#)
- [Coverview](#)

2.3.17. Visual Debugging and Waveform Viewer

We have two open source choices: the older GTKWave and Surfer. Surfer is the more modern of the two and runs in either the browser or standalone. It is under active development and has many nice features such as facilities for writing protocol interpreters so that we have mnemonic displays for things like opcodes instead of just raw binary or hex.

Surfer binaries are available on MacOS, Linux and Windows, but can also run in the browser which is very convenient.

References:

- [Surfer project website](#)

- [RISC-V Summit paper on Surfer](#)

2.3.18. Verilog Style and Lint checking with Verible

References:

- [Verible GitHub](#)
- [Verible LSP guide](#)

2.3.19. Component Design

Individual components of the OCAH are packaged in a manner that allows them to also be used as standalone IP components and be easily integrated in other designs. Each component directory includes everything necessary for its use:

- AsciiDoc documentation
- SystemVerilog RTL implementation
- SystemRDL CSR specification
- Unit tests may be one or more of the following
 - Verilator unit tests written in C/C++
 - Cocotb unit tests written in Python
- Lint tool waivers
- Synthesis constraints
- Software driver code in C/Rust
 - Firmware unit tests can be running on the connected host

The design philosophy is to aim to keep the design easy to understand and easy to integrate in OCAH or elsewhere. That means using consistent hardware interfaces and consistent naming schemes. For example, if we decide to use active low resets throughout OCAH, then we should have an input named `rstn_i` on all components. All RTL modules should have a common name prefix. For example, an AES cipher block should have all submodules named `aes*.sv`.

The components themselves should have minimal dependencies. If there are subcomponent and package library dependencies then they should be clearly identified and easy to extract and bundle with a component for use in other non-OCAH systems. We want to avoid obfuscated, complex dependencies within the design that are difficult to unravel.

2.3.20. CSRs : Configuration and Status Registers

Register and memory map descriptions are written in SystemRDL. Output renderings include:

- synthesizable Verilog RTL
- HTML documentation
- C header
- IP-XACT XML

We use the PeakRDL tool to generate the various renderings. The tool is open-source and written in Python. It is possible to write custom Exporter features using Python plugins.

The Verilog generator will create the CSR implementation as well as a matching AMBA bus slave interface, either APB or AXI-lite. Every component will have a CPU software accessible memory mapped CSR block accessible via a 32-bit APB link to enable programming control and configuration register and reading status registers. Interrupt are also defined here.

Cryptographic components will have an additional private key block accessible via a 64-bit AXI4-lite link. Only the key manager will be able to write to individual component's key blocks. The separate key bus enables a design in which the CPU software never has direct access to the secret key data, whether intentionally or inadvertently. As this is a write only interface, we will not implement the read address and read data channels. This will be done by tying off the applicable pins. for key distribution from the key manager to various cipher blocks and is therefore a write-only interface. We can consider tying off the Read Address and Read Data Channel interface pins on the component to remove any read capability.

OpenTitan components have their CSRs described in a Python DSL and use reggen to generate outputs. We'll need to convert those into SystemRDL. Many have already been converted by the Caliptra team. Others like OTBN, we'll need to handle ourselves. Fortunately there is a tool called PeakRDL-opentitan which should manage most of this process, semi-automatically i.e. I expect we'll make some tweaks to the output, but it's a great starting point.

```
git clone https://github.com/Risto97/PeakRDL-opentitan.git
cd PeakRDL-opentitan
pip install -e .
```

TODO: Use a Python virtual environment (for example via `uv sync`) to install the PeakRDL and register-generation tooling used by this repository.

PeakRDL resources are located here:

- [PeakRDL documents](#)
- [PeakRDL GitHub](#)

SystemRDL resources are located here:

- [SystemRDL documents](#)

PeakRDL-opentitan - Converts OT reggen descriptions into SystemRDL

- <https://github.com/Risto97/PeakRDL-opentitan>

2.3.21. Special Considerations for Cryptographic Components

As mentioned above, crypto components will have an AXI4-Lite private key bus interface mastered by the Key Manager.

In addition, there will be some facilities for physical attack countermeasures such as side channel analysis (SCA) and fault injection (FI):

- Entropy noise input used for masking, and random stall cycle or decoy cycle insertion
- Random clock stall or decoy instructions
- Wipe input command for rapid clearing or randomization of sensitive state to prevent data exfiltration

2.3.22. Unit Testbenches

Component functionality will be mostly verified at the individual unit level. This means that we will have at least one unit testbench and test sequence for each. Unit tests are run on Verilator and primarily written in C/C++, although portions may be in standard SystemVerilog. We will not use UVM for unit test. UVM is used by the DV team for chip level sign off tests, but that is beyond the scope of OCAH as a standalone project.

Unit tests live under the directory <component_name>/sim. Each component's sim directory will have a <component_name>_tests.adoc. This document will have an initial section describing how to run any of the tests followed by a description of each test.

Tests must cover at least the following:

- Data transfer over its AMBA bus interfaces (ABP, AXI4 etc.)
- CSR features for control, status and interrupts
- Mission mode functions

All testbenches and test sequence files will have the <component_name> as a prefix.

2.3.23. Fuse Map

The fuse map will be described in an abstract format, similar to CSRs and from which all downstream representations are generated.

2.3.24. Interconnect Crossbar Generators

The OCAH project will be using industry standard AMBA bus protocol for physical interconnect between components as mentioned earlier. To support this, we will have tools to generate interconnection crossbars for both AXI4 and APB4. The interconnect matrix is specified in YAML.

The OpenTitan components are using TL-UL (TileLink Ultralight) for interconnect and we will either add shim logic to translate to AMBA or replace those interfaces with AMBA versions.

2.3.25. Component Directory Organization

All components will have a uniform directory structure which has the following elements:

- README.md : component overview and quick start guide in Markdown format
- doc/ : detailed AsciiDoc documentation (architecture.adoc, interface.adoc, memmap.adoc, index.adoc)
- rtl : all SystemVerilog modules <component>_*.sv
- tb : unit level Verilator tests
- regs/ : <component>.rdl SystemRDL CSR memory map definition and generated collateral
- syn : logic synthesis constraints (optional)
- lint : lint waiver files

2.3.26. SystemVerilog Coding Guidelines

The high level goals are:

- avoid SystemVerilog constructs that are error prone or not consistently supported, particularly across open source tools
- ensure that the code is tidy and easy to read
- keep the code style consistent across the project's code base regardless of who authored it

OCAH will *mostly* follow the OpenTitan SystemVerilog coding rules defined by lowRISC group in [Verilog Coding Style](#). That document is locally copied for convenience.

This section highlights a few of the most important aspects as well as a few deviations from the OpenTitan rules. If there is a conflict between the two, the specification here takes precedence.

- Use four spaces for indentation, rather than two for new files
- Use all upper case letters to name enums, constants (localparam), and parameters instead of UpperCamelCase.
- As stated in the lowRISC guidelines, SystemVerilog Interfaces can be limited in terms of tool support, so we *highly discourage* their use as they are not uniformly supported by open source tools. As an alternative, struct types can be used instead to bundle groups of input or output signals to make ports more abstract.
- Finite State Machines (FSMs) are powerful sequence control constructs but can spiral out of control quickly. Consequences are difficult to understand "next state" logic and long timing paths. Separate the combinatorial "next state" logic from the sequential state update into distinct always blocks. **Do not** combine the two into a single clocked always block. Opt for a [Moore machine](#) implementation when possible to aid in timing closure. If you have complexity of more than ten states or so, use a microsequencer approach. This is more scalable and maintainable.

There will be additional rules that apply to security sensitive components such as ciphers. These will be documented in more detail, but here is a summary for now:

- FSM state encodings are one-hot with error checking
- Error detection is implemented on storage elements and some bus and computational elements
- Secret data is transferred and stored in secret-split fashion

2.3.27. RTL Code Comments

RTL code comments have a tendency to get out of sync with the code or simply be redundant, adding little or negative value. Hence, keep comments in the RTL as minimal as possible and prefer to make the code self explanatory. Include a description of the purpose and overall function of the module in the header. If there is a tricky bit of code with some nuance, it's fine to add a comment describing the overall goal or strategy of the code. However do not just add an English translation of what the Verilog code is doing as that is in best case redundant.

2.3.28. Configuration

Configuration files are used for specifying structure or behavior in applications like interconnect crossbars, fuse maps, memory maps etc. We have several different flavors of configuration files in the codebase today but ideally would like to standardize on one such as HJSON or JSON.

As described earlier, memory maps and CSRs we will be declared in SystemRDL.

OCAH Architecture & Design

This section provides an overview of the OCAH architecture including its subsystem composition, power and clock domains, reset hierarchy, and memory map. Subsequent chapters provide detailed microarchitectural documentation for each subsystem.

Chapter 3. OCAH Architecture

This chapter describes the OCAH subsystem composition, power and clock domains, reset hierarchy, and memory map. Subsequent chapters provide detailed microarchitectural documentation for each subsystem.

3.1. OCAH High-Level Features

- **Security Enclave Processor (SEP)** — Root of Trust (RoT) with the highest security privilege. Provides a physically isolated RISC-V execution environment for secure boot, lifecycle control, device attestation, and hardware-accelerated cryptography (OTBN PKA, AES, HMAC-SHA2, KMAC-SHA3, TRNG, DRBG, Key Manager). A security hierarchy across chiplets enables SiP-wide security management.
- **System Management Controller (SMC)** — Firmware-driven orchestration engine responsible for clock, voltage, and reset management, platform bootstrapping, system initialisation, and power/thermal management. Built around a RISC-V micro-controller with hardware accelerators (DMA, memory zeroer) where firmware alone cannot meet timing requirements. Inter-chiplet management hierarchy via I2C/I3C and OCMI channels.
- **Debug & Test Ports (DTP)** — Standardised debug and test access across heterogeneous, multi-vendor chiplet systems. Layered architecture separating physical transport from logical access, with cross-chiplet event triggering, synchronised clock control for trace, and coordinated resets.
- **System Management Unit (SMU)** — The composite module combining SEP, SMC, and DTP into a single instantiable subsystem.
- **System Management Network (SMN)** — Internal communication fabric (point-to-point or packetised) providing the management backbone for configuration, status reporting, and firmware-driven coordination between SMU, user logic, and D2D interfaces.
- **AXI-over-UCIe Bridge (AoU)** — Protocol bridge between the AXI-based internal fabric and the UCIe die-to-die physical interface, enabling inter-chiplet system management traffic and application-level transaction forwarding.
- **Application-Specific Logic** — OCAH treats the application layer as a black box: the harness services are agnostic to whether the application is a CPU complex, NoC, DSA, memory controller, or high-speed IO. The application logic is outside the scope of OCAH.
- **Safety Monitor (SMO)** — External entity that collects error signals and diagnostic logging from OCAH subsystems to implement fault reaction policies. The SMO is outside the scope of OCAH; its interface requirements are specified separately as part of OCAH's functional safety documentation.

3.2. Top Block Diagram

The figure below presents the OCAH top-level block diagram showing the relationship between the subsystems described above.

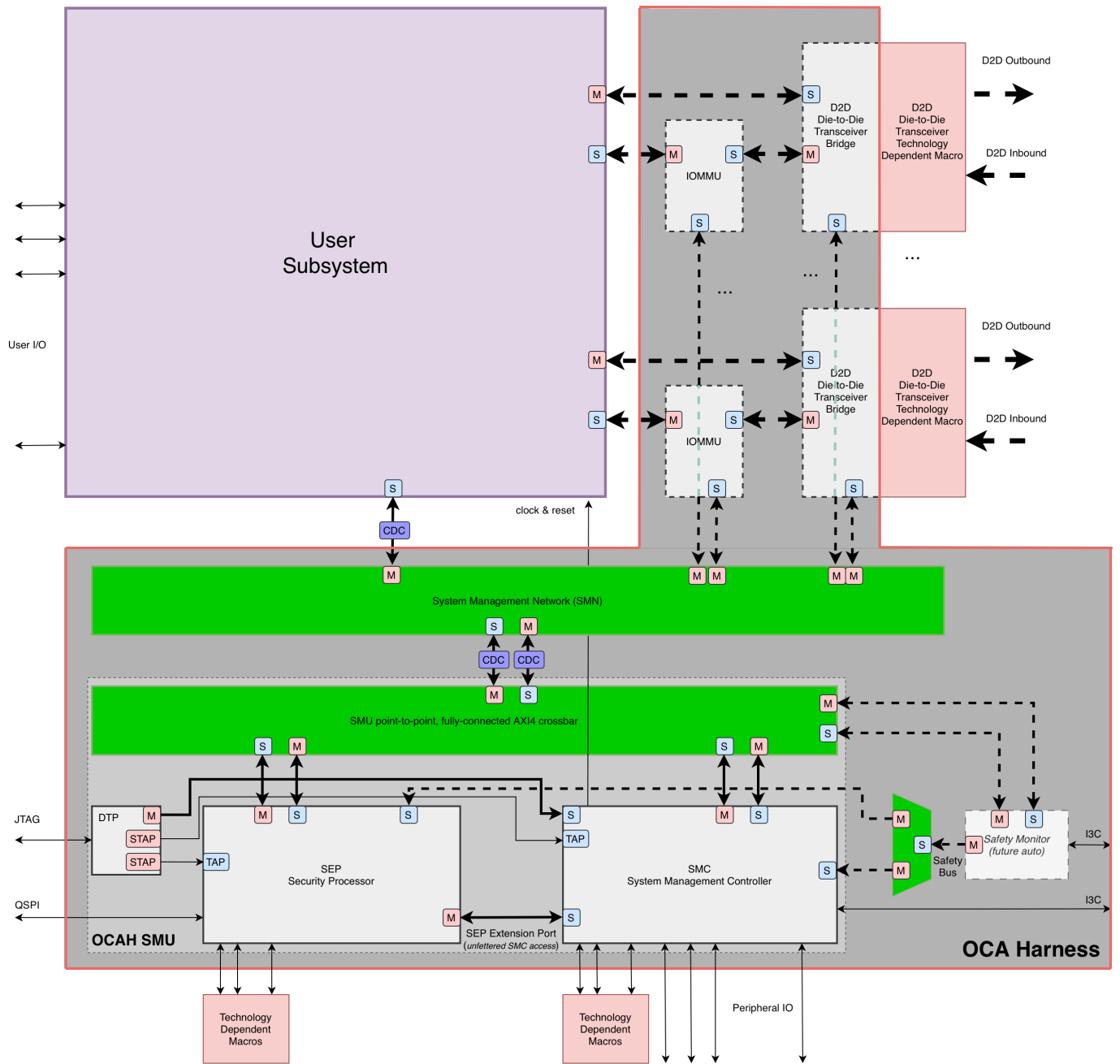


Figure 1. OCAH Top-Level Block Diagram

3.2.1. SMU AXI Crossbar

Inside the SMU, the `smu_axi_xbar` module is the system-level arbitration fabric between SEP, SMC, and the rest of the chiplet (reached through the `smu_axi_in` / `smu_axi_out` boundary pair). It is a 3x3 AXI4 crossbar with non-reflexive connectivity: each master can reach the other two subordinates but never its own inbound port.

Table 2. SMU AXI crossbar ports

Master (from)	Subordinate (to)	Purpose
<code>sep_out</code>	<code>smc_in</code>	SEP-initiated transactions into SMC resources
<code>sep_out</code>	<code>ext_out</code>	SEP-initiated transactions leaving the SMU
<code>smc_out</code>	<code>sep_in</code>	SMC-initiated transactions into SEP resources

Master (from)	Subordinate (to)	Purpose
smc_out	ext_out	SMC-initiated transactions leaving the SMU
ext_in	sep_in	External transactions targeting SEP
ext_in	smc_in	External transactions targeting SMC

Routing decisions use a runtime address map with two programmable rules (SEP and SMC apertures) and one static catch-all (`ext_out`). See [OCAH Memory Map](#) for the fixed-vs-programmable split and the reference envelope convention, and see the SMU AXI Crossbar Address Map section in the Integrator Guide for firmware programming order.

3.3. OCAH Memory Map

OCAH separates the chiplet-visible SMU aperture from the internal address maps used by SEP and SMC firmware. The chiplet integrator decides which system address range reaches the SMU external AXI input. Firmware then programs the SEP and SMC global base/size CSRs so the SMU crossbar can route incoming traffic to the correct inbound port.

Concept	Configured by	Purpose	Example
Chiplet NoC envelope	Integrator / chiplet address map	Controls which external-master addresses reach SMU <code>ext_in</code>	Example only: 64 MiB total SMU aperture
SEP global aperture	SEP firmware via <code>SEP_GLOBAL_BASE_ADDR</code> / <code>SEP_REGION_SIZE</code>	Controls SMU <code>ext_in</code> to <code>sep_in</code> routing and SEP inbound global-to-local remap	Example only: 32 MiB SEP slot in a 32 MiB + 32 MiB SEP/SMC partition
SMC global aperture	SMC firmware via <code>GLOBAL_BASE</code> / <code>REGION_SIZE</code>	Controls SMU <code>ext_in</code> to <code>smc_in</code> routing and SEP-to-SMC decode	Example only: 32 MiB SMC slot in a 32 MiB + 32 MiB SEP/SMC partition

External masters use chiplet-global addresses and are bounded first by the chiplet NoC decode. The SMU crossbar applies the firmware-programmed SEP and SMC global apertures to route those transactions to `sep_in` or `smc_in`. Subsystem-internal CPU maps and local aliases are described in the SEP and SMC chapters.

The current reset defaults are conservative boot values: `SEP_REGION_SIZE` resets to `0x0100_0000` (16 MiB) and SMC `REGION_SIZE` resets to `0x0100_0000` (16 MiB). Firmware may reprogram these values during early boot to match the chiplet address assignment. If the reset values are changed to a 32 MiB SEP/SMC split, this section should be updated accordingly.

3.4. Power Domains

The SMU, SMN, and AXI-over-UCIe bridge (AoU) reside in the same always-on power domain. This domain remains powered whenever the chiplet has supply, ensuring that platform management, inter-chiplet communication, and security services are available even when application logic is powered down.

Vendor-specific macros and subsystems that are outside the scope of OCAH—such as OTP/eFuse controllers, PLLs, die-to-die PHYs (e.g. UCIe PHY), analogue sensors, and application-specific tiles—typically reside in their own dedicated power and voltage domains defined by the adopter’s power architecture. OCAH provides the isolation and sequencing infrastructure (`isolate_req_o[31:0]`, `ss_reset_ctrl_o[31:0]`) that the SMC firmware uses to coordinate power transitions for these external domains, but their supply rail definitions, power gating decisions, and leakage budgets are adopter-specific.

The SMC provides runtime power management services for the application logic, including dynamic voltage and frequency scaling (DVFS) through AVSBus-based voltage regulator control and firmware-driven Performance State (P-State) management. At the RTL level, the SMU accepts a `powergood_i` input that indicates supply stability and exposes

`isolate_req_o[31:0]` to request isolation of individual subsystem domains before power transitions.

NOTE

The figure below will be provided as an annotated version of the top-level block diagram highlighting the power domains.

3.5. Clock Domains

OCAH distinguishes low-level management timing from functional application clocks. The SMC receives an always-on reference clock (`clk_ref_i`) that is available from power-on, before any PLL is locked. Using this reference clock, the SMC controls the PLLs during early bring-up and generates the primary system clock (`clk_smu_i`) that is then distributed to the SMC itself, the SEP, and the DTP.

The SMN lives in its own clock domain with respect to the SMU --- as highlighted by the clock domain crossing in the top level diagram --- and with respect to the application logic.

The following table summarises the clock domains present in OCAH.

Table 3. OCAH Clock Domains

Clock	Subsystem	Description
<code>clk_smu_i</code>	SMU (SMC, SEP, DTP)	Primary system clock, generated by the PLL under SMC control. Distributed internally to the SMC, SEP, and DTP subsystems. Drives the CPU cluster, fabric interconnect, peripheral controllers, and DTP cross-trigger / AXI bridges.
<code>clk_smn_i</code>	SMN	System Management Network clock. Independent of <code>clk_smu_i</code> ; clock-domain crossings (CDCs) are used at the SMU-SMN boundary to isolate the management fabric timing from the SMU subsystem clock.
<code>clk_ref_i</code>	SMC	Always-on reference clock, available before PLL lock. Used by the SMC for early bring-up, PLL control, reset synchronisation, and peripherals requiring a stable, lower-frequency source.
<code>clk_periph_i</code>	SMC	Peripheral clock domain. Drives SMC peripheral subsystems. The reset unit generates a dedicated synchronised reset (<code>rst_primary_periph_clk</code>) in this domain.
<code>clk_telemetry_i</code>	SMC	ATB telemetry clock domain. Separate clock for Advanced Trace Bus data collection, with its own reset (<code>rst_telemetry_ni</code>).
<code>clk_sep_wdt_i</code>	SEP	Low-frequency watchdog timer clock. Must be independent of the system clock so the WDT can detect system clock failures.
<code>jtag_tck</code>	DTP / SEP	JTAG Test Clock. Drives PTAP, STAP state machines, and all JTAG-domain scan logic inside the DTP/SMC/SEP.
<code>entropy_rosc_sample_clk_i</code>	SEP	Ring-oscillator sample clock for the entropy source (TRNG). Must be asynchronous to <code>clk_i</code> and typically runs between 100 and 400 MHz.

NOTE

The figure below will be provided as an annotated version of the top-level block diagram highlighting the clock domains.

3.6. Reset

Reset is handled hierarchically so that the management infrastructure comes out of reset first and then releases dependent domains in a controlled order. A global power-on reset (`rst_cold_ni`) establishes a known state for the entire chiplet, but the always-on domain is intended to be released first so that early control logic is live immediately. From there, the SMC sequences reset deassertion for the SMN and for downstream managed blocks according to platform

dependencies, boot policy, and fault conditions. This staged reset model also supports partial and warm-reset behaviours in which application domains can be reset or recovered without fully tearing down the always-on management infrastructure.

The reset hierarchy within the SMU proceeds as follows:

1. **Power-on / cold reset.** The external `rst_cold_ni` is asserted during power-up and deasserted synchronously after supply voltages stabilise. The `powergood_i` input indicates that supplies are stable.
2. **SMC reset synchronisation.** The SMC reset unit (`smc_reset_unit`) generates domain-specific synchronised resets from the single cold reset input:
 - `rst_primary_smc_clk_no` — synchronised to the primary SMC clock domain
 - `rst_primary_ref_clk_no` — synchronised to the reference clock domain
 - `rst_primary_periph_clk_no` — synchronised to the peripheral clock domain
 - `rst_cold_stable_ref_clk_no` — stable cold reset in the reference domain
3. **DTP reset.** The DTP is held in reset by the SMC's primary synchronised reset (`rst_primary_smc_clk_no`). In addition, the JTAG state machine requires a power-on reset that remains asserted until supplies are stable; at the SMU level this is the `powergood_stable` signal generated by the SMC.
4. **SEP reset.** The SEP is held in reset by the same SMC primary reset (`rst_primary_smc_clk_no`). A separate watchdog reset path (`rst_wdt_n` at the SMU level, generated by the SMC reset unit) allows the SEP to be independently reset by its watchdog timer.
5. **Downstream subsystem sequencing.** The SMC sequences reset deassertion for up to 32 managed subsystems through `ss_reset_ctrl_o[31:0]` and isolation through `isolate_req_o[31:0]`. Subsystems report completion via `ss_reset_complete_i[31:0]`.
6. **Function Level Reset (FLR).** Asserting `cfg_flr_pf_active_i` triggers a warm reset sequence that isolates subsystems through `isolate_req_o` and performs controlled reset propagation without tearing down the always-on infrastructure.
7. **Debug reset overrides.** The DTP provides JTAG IC_RESET controls that can override cold, cool, and core reset domains independently for debug scenarios. These are wired through the SMU to the SMC's `rst_{cold,cool,core}_override_ni` / `rst_{cold,cool,core}_override_en_i` ports.

For the detailed reset controller design, sequencing FSM, reset register interface, and clock-reset interaction, refer to the [Clock and Reset Management](#) chapter. For integration-level reset procedures, refer to the Integrator Guide.

Chapter 4. Security Processor (SEP)

4.1. Introduction

SEP is the Root of Trust (RoT) of the chiplet with the highest security privilege. SEP provides a physically isolated execution environment for highly sensitive security services. The following are some examples:

- **Secure boot:** The SEP ROM BL0 image uses non-volatile public keys to conduct authentication verification on the loaded BL1 software images.
- **Secure lifecycle control:** SEP implements a secure lifecycle controller to manage function, debug and test access to the chiplet according to fuse configuration, in-field demotion, and secret token inputs.
- **Device attestation:** SEP offers support to device attestation by recording the boot process and device configuration followed attestation using public key infrastructure.
- **Cryptography service:** SEP offers secure key services to the rest of the system together with cryptography acceleration. SEP also implements True Random Number Generator (TRNG) to provide random numbers for SEP itself and the rest of the system.
- **User authentication:** Similar to secure boot, user authentication is another authentication service SEP offers. Besides feature control that can be tied to the authentication result, SEP software can also tie the authentication result to the secret key derivation to ensure that bypass of the user authentication leads to a useless system.

The SEP is responsible for ensuring that the chiplet and overall packaged system composed of multiple chiplets remains secure from adversaries. This means that SEP ensures the system

- Runs correctly with the intended firmware and data protected by digital signatures
- Guards against malicious data input to SEP, SMC and the debug/test infrastructure
- Prevents exfiltration of private or secret data on all exits, including side channels
- Prevents privilege escalation which would enable illegal operations

A consolidated description of OCAH use cases is available in the top-level [Use Cases](#) section.

4.1.1. Threat Model

The threat model covers remote software attacks, physical attacks and supply chain attacks. Software attacks are universal and the primary vector against data center implementations while physical attacks are localized and require close proximity. They are relevant for automotive and robotics applications. Supply chain attacks involve modification of chips in production and provisioning.

Software attacks include:

- Malicious firmware modification in boot, overlay or application loading or updating
- Modifying or exfiltrating private data e.g. ML model weights, cryptographic keys
- Providing forged attestation to mimic legal device behavior

Physical attacks include:

- Fault injection to bypass security checks and differential cryptanalysis for key recovery. This can include clock and

voltage glitching, laser and EM energy injection, and thermal gradients. Logically induced faults such as Row Hammer DRAM attacks are also included in this category.

- Side channel for key and other secret data extraction
 - We assume that the attacker has full view of SRAM and OTP data state
- Hardware probing to observe or control data within a chiplet or communication between chiplets

Supply chain attacks include:

- Insider modification of design to disable or weaken security features and insert backdoors. Theft of hardware secrets such as security tokens
- Modification of IC design masks to modify behavior and insert trojans. For example the BL0 boot ROM contents may be modified to disable secure boot.
- Corruption of fuse provisioning to enable or disable features illegally and leak device unique secrets

We however limit the scope of physical attacks we defend against and do not explicitly protect against the following:

- Package internal inter-chiplet communications secure from hardware probing
- Package internal chiplet hardware probing and certain physical attacks such as bias fault injection
- Thermal gradients to influence entropy source and induce timing faults
- Supply chain attacks

4.1.2. Attack Countermeasures

Countermeasures include:

- Protection of BL0 boot ROM and subsequent secure boot via digital signature verification
- Glitch and error detection
 - SRAMs and other memory structures such as FIFOs and register files are protected by parity (byte-wise) or SECCDED
- Temporal noise injection, masking, secret splitting and obfuscation on sensitive operations
- Transaction filtering and firewalling
- Functional and physical isolation of private data assets e.g. cryptographic keys
- Anti-replay facilities e.g. non-volatile counterj in OTP
- Obfuscate data stored in SRAM and OTP memories

4.1.3. Debug and DFT Security Considerations

The debug facilities and DFT share the scan chain infrastructure for their respective tasks. This is a security hazard as certain secret values will be resident in the registers: tokens, secrets, cryptographic keys. Unless prevented, an attacker could halt the chip execution and extract the register state containing these secrets. This section describes the countermeasures we take to prevent this from happening.

One debug feature is scandump which provides access to all internal register states. We must ensure that a certain subset of registers holding secrets are never dumped out in this mode.

There are three scenarios of how registers are handled for DFT scan test and debug scandump:

1. Full DFT and Debug access
2. DFT access but no Debug access
3. No DFT and no Debug access

Consequently there will be two discrete scan chains to handle the registers in groups 1 and 2. The register instances (or module instances) will be labelled as belonging to one of the three groups and the DFT scan insertion tool will group the registers into the respective scan chains which will be managed for security in the TAP controller.

These three groups correspond to the SEP asset classes defined authoritatively in the Lifecycle Controller chapter (*Asset and Scan-Chain Protection*): group 1 = Class 3 (general), group 2 = Class 2 (crypto / KM secret-bearing), group 3 = Class 1 (secrets and integrity-critical control fields).

Debug and Scan Dump

Debug mode is enabled when in-field debug is granted, via demotion (DEMOTED_1 / DEMOTED_2) or an RMA debug state (RMA_SIP / RMA_CHIPLET). In this mode, the user has near full control and observability of the chip state, but we must enforce security restrictions.

General logic (Class 3) scan dump is gated by the `*_DBG` feature-control bits (sourced from the OTP DIS vectors; `10b0` = enabled, `10b1` = disabled, irreversible) and the LC state — a scope's general logic is dumpable only when its local gate (`SEP_DBG` / `CHIPLET_DBG`), the `SIP_DBG` debug-access path into the chiplet, and the LC state permit it.

Secret-bearing logic (Class 2) — crypto-engine key / round-key storage and the Key Manager key vault / shared key storage — is placed on a **dedicated secret-bearing scan chain** so DFT keeps coverage, but is **never scan-dumpable**, for any owner or LC state. It is governed by `secure_tm`, not the `*_DBG` bits: under `secure_tm = 1` the secret sources (UID / `CLASS_KEY`) are disconnected from these consumers, and under `secure_tm = 0` the chain is tied off.

Secrets and integrity-critical fields (Class 1) are excluded from every scan chain (never scannable):

- Class 1a (confidentiality): UIDs, `CLASS_KEY`, token-plaintext input registers (`*_TOKEN_I`).
- Class 1b (integrity): `LOCKS`, `LC_STATE`, `SIP_DIS`, `SYS_DIS`, `*_TOKEN_DIGEST` shadow registers.

Class 2 Asset Manifest and Naming

The authoritative class definitions, scan-chain policy, and gating rules are in the Lifecycle Controller chapter (*Asset and Scan-Chain Protection*); Key Manager specifics are in the Key Manager chapter (*Scan and Scan-Dump Protection*). This section records the implementation manifest. A complete module instance is classified as Class 2 when it is the Key Manager, a device-secret-consuming cryptographic engine, an entropy engine, or sequential distribution logic that retains key, seed, random, digest-working, or round-key state. Classifying complete instances deliberately includes their control flops and avoids fragile per-register marking.

The current SEP implementation has exactly 10 Class 2 RTL scan roots:

- `sep_crypto.hmac_wrapper_s3c_scan`
- `sep_crypto.sep_crypto_otbn_wrapper_s3c_scan`
- `sep_crypto.aes_wrapper_s3c_scan`
- `sep_crypto.kmac_wrapper_s3c_scan`
- `sep_crypto.u_sep_crypto_abr_wrapper_s3c_scan`

- `sep_crypto.u_entropy_source_s3c_scan`
- `sep_crypto.u_drbg_s3c_scan`
- `sep_crypto.u_axis_edn_crypto_s3c_scan`
- `sep_crypto.u_key_manager_s3c_scan`
- `u_trng_wrapper_s3c_scan`

The `_s3c_scan` suffix is reserved for Class 2 instance and generate-scope names. It is not a module-name or per-flop suffix. A nested `_n0_scan` name remains Class 1 and takes precedence over an enclosing `_s3c_scan` hierarchy.

Three writable companion SRAMs are also Class 2 assets: `u_km_sram`, `u_otbn_imem_sram`, and `u_otbn_dmem_sram`. Their macro instance names remain unchanged because `_s3c_scan` identifies RTL scan hierarchies; the owning memory-DFT/MBIST policy must provide the same never-dump outcome. `u_km_rom` is explicitly excluded because immutable executable contents cannot retain runtime secrets. The vendor TRNG core keeps its delivered DFT view beneath `u_trng_wrapper_s3c_scan`.

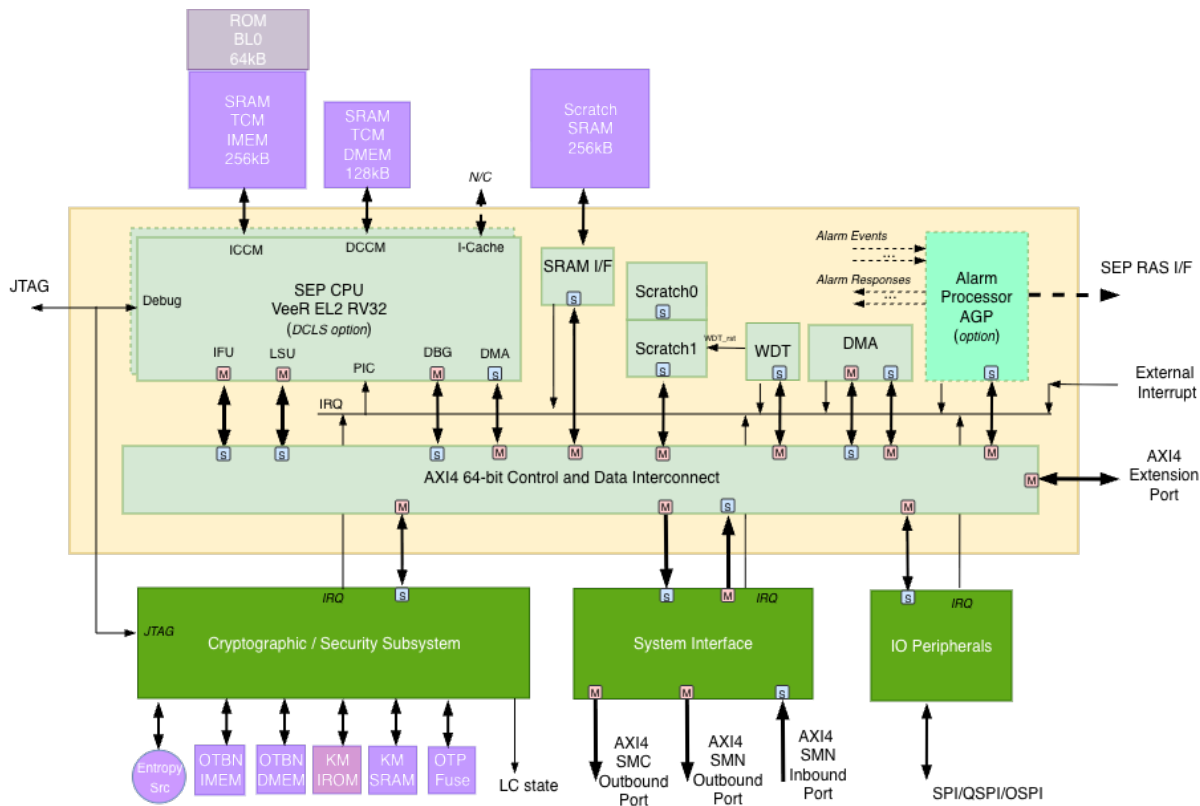
Secure Test Mode (SECURE_TM)

We assert a GPIO pin strap to enable secure test mode. This generates the SECURE_TM signal which is consumed by

- TAP to enable scan test
 - Shadow registers of LOCKS, LC_STATE, SIP_DIS, and SYS_DIS are non-scannable registers
 - Any downstream registers in the path from these registers to the test enablement signals for DTAP or fuse write enablement signals are also non-scannable
 - Dedicated test registers may be inserted in the path while in SECURE_TM mode to enable testing of downstream logic
 - During scan test (`secure_tm = 1`) the secret-source outputs (UID / `CLASS_KEY`) are disconnected from the Class 2 secret-bearing consumers, so no real secret is shifted; during scan dump (`secure_tm = 0`) the Class 2 secret-bearing chain is tied off (never dumpable). See the Lifecycle Controller chapter, '`secure_tm`' and *Scan Dump*.
 - Disable writing to any OTP fuse state to prevent updates.
- OTP controller
 - Disable *writing* of shadow registers: LOCKS, LC_STATE, SIP_DIS and SYS_DIS
 - Other shadow register fields may optionally also have write disabled

4.2. Hardware Overview

The diagram below shows the top level SEP architecture.



SEP is hierarchically organized as follows:

- SEP interconnect fabric
 - AXI4 top level for CPU subsystem, its base platform components, System peripherals, Cryptographic/Security peripherals and IO peripherals subsystems
- External IO ports
 - System Interface to SMN NoC
 - Outbound AXI4 system bus master
 - Inbound AXI4 system bus slave
 - Low speed IO peripherals AXI4 extension port with peripheral interrupt
 - SPI controller interface (single/quad/octal)
 - *Remote peripherals shared with SMC: UART, GPIO*
 - JTAG for debug and test
- External Macro Interfaces
 - Memories: BL0 ROM, DCCM, ICCM, Scratch SRAM, KM SRAM, KM ROM, OTBN IMEM, OTBN DMEM
 - OTP eFuse
 - Entropy source
- CPU subsystem
 - JTAG interface to CPU debugger and system debug facilities
 - Tightly coupled instruction SRAMs and ROM
- Scratch Register Bank x 2

- DMA Engine
- Watchdog Timer (WDT)
- *Alarm aggregation processor (AGP) with Reliability and Service (RAS) port - deferred to the next generation*
- Cryptographic subsystem
 - Cryptographic accelerators: Public Key Engine, AES, HMAC-SHA2, KMAC-SHA3
 - Security components: Life Cycle Controller, Key Manager, OTP Fuse Controller
 - Entropy source, DRBG, Entropy Distribution
 - Key Manager (KM)
 - Key and Policy Vault (KPV)
 - Crypto engine key sideload interfaces

The cryptographic, peripheral IO and systems IO subsystems reside in their own respective hierarchies. These subsystems are connected with the main SEP AXI4 crossconnect fabric. Individual peripherals in the subsystems have AXI4-lite interfaces.

Technology and vendor specific macro blocks such as SRAM, ROM etc. are instantiated outside of SEP and have dedicated SEP interfaces. The goal is to cleanly separate the generic SEP IP from technology specific components in order to make integration and migration easier.

4.3. SEP Interconnect

The interconnect of SEP components and subsystems is done with a hierarchical AXI4 crossbar fabric. Masters drive traffic onto the interconnect fabric and receive responses from the addressed slave. Slave components are directly connected to the AXI4 fabric or via a bridging shim. The shim translates either AXI4-lite or the legacy Tilelink TL-UL used in OpenTitan. The goal is to eventually convert all TL-UL slaves into AXI4-lite slaves.

AXI4-lite slave bridges will need to perform the following tasks in order to mate with AXI4:

- Bursts are not supported: convert burst transactions to single transactions
- Write strobe (WSTRB) for partial write is not supported: TBD how to handle
- Transaction IDs (AxID) handling

4.3.1. Top Level AXI4

C = Connected, N/C = Not Connected.

The table below reflects the generated crossbar connectivity defined in `meta/crossbars/configs/sep_local_axi_xbar.yaml`, along with the dedicated Boot ROM path that lies outside the crossbar. The crossbar has five initiators (CPU IFI, CPU LSU, CPU DBG, DMA, and the System Interface) and ten targets.

A few access paths are intentionally outside this crossbar:

- **Boot ROM** is accessed by the CPU IFI and LSU through a dedicated point-to-point path, not through this crossbar.
- The CPU's own **ICCM/DCCM** are tightly coupled to the core and are not reached through this crossbar by the IFI/LSU. The **CPU TCM** target below is a backdoor AXI path into the ICCM/DCCM, used by the DMA engine to preload code/data.

- The *Alarm AGP* is a feature deferred to the next generation and is not present.

The **System Periph** target aggregates the System Bus interface, the dual scratch register bank, the AP/STEE remap regions, and the outbound routing to SMC/SMN resources.

The **Entropy Pool** target is `sep_entropy_fifo` (0x1095_0000): a 64-bit read-only entropy drain aperture, filled autonomously by a native EDN endpoint on a separate hardware path (not this crossbar). Writes to it terminate with `BRESP=SLVERR`.

The **System Interface** initiator is the external inbound master. It does not attach to this crossbar directly; it enters SEP through the dedicated `smn_inbound` port and the inbound traffic filter (block-by-default), then through the `sep_system_peripherals` inner crossbar. That inner crossbar reaches the **mailboxes** and **system CSRs** directly, and loops everything else back into this crossbar's System Interface row. The inbound filter controls which addresses the external master may reach; it is block-by-default after POR and configured by SEP firmware.

Masters	Crossbar										Dedicated Path
	CPU TCM	Scratch SRAM	DMA CSR	WDT	Reset Ctrl	Crypto Periph	System Periph	IO Periph	Entropy Pool	AXI Extension	Boot ROM
CPU IFI	N/C	C	N/C	N/C	N/C	N/C	N/C	N/C	N/C	N/C	C
CPU LSU	N/C	C	C	C	C	C	C	C	C	C	C
CPU DBG	N/C	C	C	C	C	C	C	C	C	C	N/C
DMA	C	C	N/C	C	C	C	C	C	N/C	C	N/C
System I/F	N/C	C	C	C	N/C	C	N/C	C	C	C	N/C

4.3.2. Subsystem External Interfaces

- Memory interfaces: BL0 ROM, DCCM, ICCM, Scratch SRAM, KM SRAM, KM ROM, OTBN IMEM, OTBN DMEM
- Fuse controller interface
- Entropy source interface
- JTAG interfaces
- *RAS Interface - Alarm Processor is deferred to next generation*

4.3.3. Local and External AXI4 Traffic Remapping

The figure below illustrates the AXI4 traffic management performed by the Systems Peripheral block. The AXI4 slave port is driven by one of the local masters from the SEP i.e. the CPU or the DMA controller. Traffic may be local to the SEP or outbound to external resources either within the SMU or beyond the SMU. Traffic beyond the SMU to the rest of the chiplet or remote chiplets will bridge to the system NOC also known as the SMN.

Transaction Routing

Local resources can be addressed by SEP CPU direct internal addresses or by the SEP CPU local-alias view. Local resources include CSRs in SEP components and subsystems, SRAM, and the AP- and STEE-controlled remappers. The local-alias view is an internal CPU-side naming convention: for example, addresses in the high alias range can be translated back to the direct internal `0x1000_0000` and `0x2000_0000` regions. This high local-alias window has a fixed SEP-local size of 768 MiB.

External masters use chiplet-global addresses. The `SEP_GLOBAL_BASE_ADDR` and `SEP_REGION_SIZE` CSRs define the global SEP aperture used by the SMU crossbar and by SEP inbound global-to-local remap. The current `SEP_REGION_SIZE` reset is `0x0100_0000` (16 MiB); firmware may program a larger or different aperture during early boot to match the chiplet address assignment.

Neighboring SMC resources are accessed through the SMC global aperture programmed by SMC firmware and supplied to SEP/SMU integration logic. That aperture is independent from the SEP CPU local-alias range.

As stated earlier, any transaction which is not routed to local SEP resources or to neighboring SMC resources is directly routed out to the SMN system NoC.

SEP's outbound traffic serves to:

1. Provide security services and
2. extend its protection to the reset of the system

For 1 the outbound traffic traverses the AP controlled remappers and for 2 the traffic goes directly to the SMN system fabric. SEP differentiates between the two scenarios 1 and 2 and enforces that the correct path is used.

When SEP accesses shared resources such as shared memory for example, the traffic goes through the AP or STEE controlled remappers to get the address translation service. These remappers are controlled by external resources such as Ascalon AP. The inbound traffic filter ensures that only STEE has access to the STEE address remapper.

When SEP accesses external private memory or the system's memory mapped IO, the SEP traffic is directly routed to SMN.

SEP's outbound traffic carries a source ID. Traffic destined to the SMN uses SEP's unique source ID. Traffic that traverses through the AP controlled remapper uses the regular source ID termed `others''`. Traffic then traverses the outbound filter after traversing the remapper. SEP works with M-mode software to program the outbound filters so that traffic with source ID `others''` cannot access M-mode assets. The outbound filter enables filtering based on address, NS and source ID.

The SEP's inbound traffic filter is configured by default to block all traffic from entering the SEP. Only the SEP CPU can program these filters. Again, these filters enable filtering based on address, NS and source ID.

Address Remapping

Sixteen remap regions are available, each supported by an address remapper. Remappers define the following attributes:

- Input address range
- Remapped output address offset
- Cacheable flag

The programmable remap regions accomplish several jobs. First, they expand local 32-bit addresses to the global 56-bit address space. Second, they provide software-programmed translation and cacheability attributes for locally initiated transactions. Out of reset, these programmable remap entries are invalid and therefore behave as transparent pass-through entries until firmware programs them.

After the alias remap, traffic routing is done as follows:

- External or cross-subsystem traffic reaches SEP through the global aperture described by `SEP_GLOBAL_BASE_ADDR` and

SEP_REGION_SIZE. The current **SEP_REGION_SIZE** reset is **0x0100_0000** (16 MiB), and firmware programs the runtime value during early boot to match the chiplet map. Barrier is recommended between programming these routing CSRs and issuing later MMIO accesses.

- SEP CPU local-alias traffic is an internal CPU-side view of the SEP map. It exists so software can use high alias addresses such as **0xD000_0000** for resources whose direct internal addresses begin at **0x1000_0000**. This high local alias span has a fixed SEP-local size of 768 MiB.
- Any transaction that doesn't get routed to local resources is directly routed out to the SMN.

To compute global addresses for external agents, firmware needs the chiplet ID or chiplet BAR for the integration. Both the chiplet ID and chiplet BAR are available in SMC as MMRs.

SEPs outbound traffic to the main fabric serves the purposes of 1) providing services and 2) extending its protection to the rest of the system. For the former purpose, SEP's outbound traffic goes through an AP-controlled remap logic. For the latter purpose, SEP's outbound traffic directly goes to the NoC. SEP software shall differentiate these two purposes and choose/enforce the correct traffic path to be used.

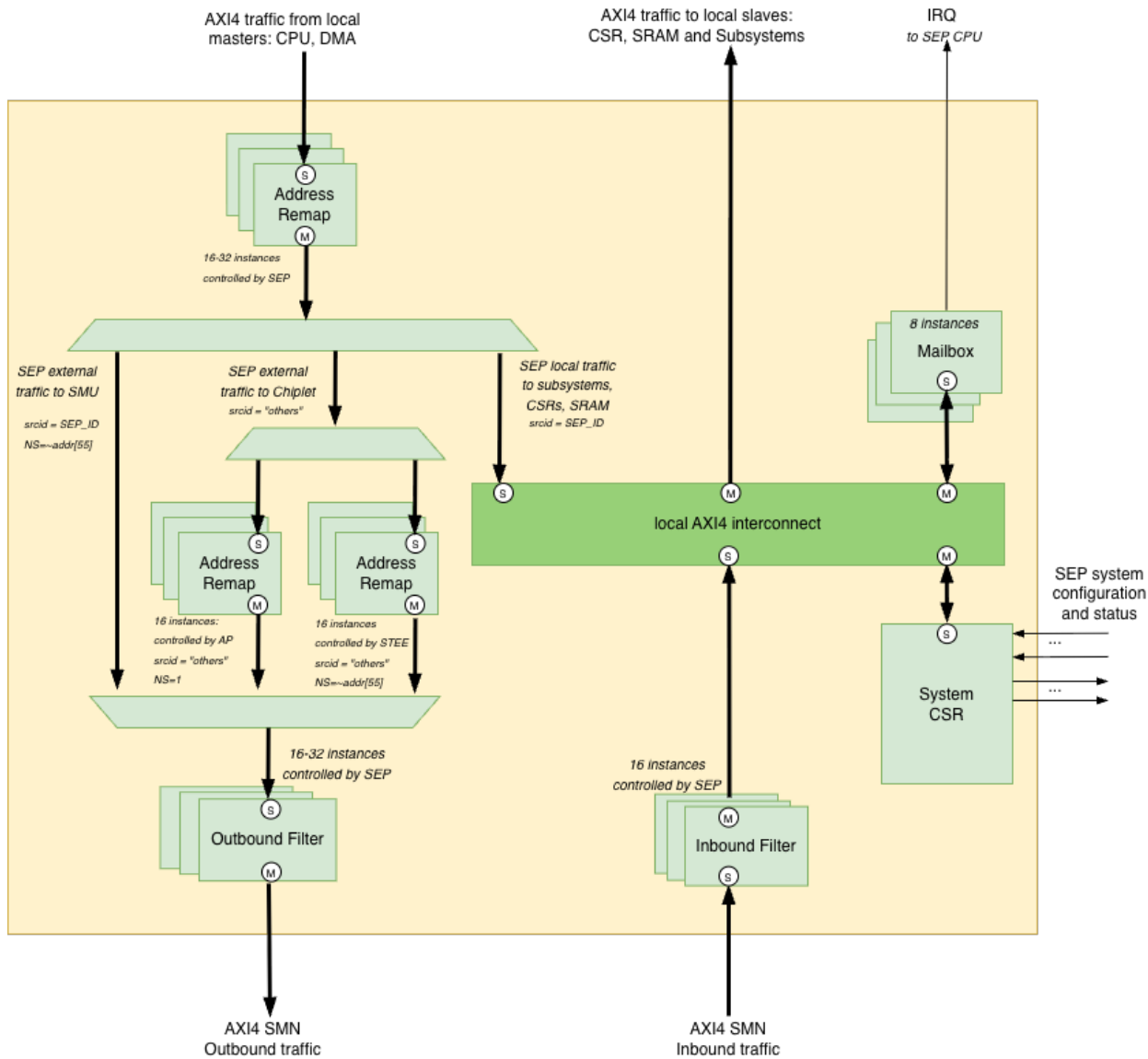
- When SEP accesses shared resources (e.g. shared memory) with the rest of the system, SEP's traffic goes through the AP-controlled or STEE-controlled remap logic to get address translation. Such remap logic is supposed to be controlled by the rest of the system, like Ascalon. The inbound traffic filter ensures only STEE has access to the STEE-controlled remap logic.
- When SEP accesses its external private memory or the system's memory mapped IO, SEP's traffic directly goes to the system fabric.

Mailboxes

SEP uses mailboxes to communicate with the rest of the system such as AP cores for example. Each different agent has its own private mailbox in order to separate the contexts. Each mailbox resides in its own 4 kB address space. Each mailbox is configured according to the inbound traffic filter and has an inbox and outbox pair. When the inbox is written, an interrupt is sent to the SEP CPU to notify it of an incoming service request. Similarly, when the outbox is written by the SEP, an interrupt is sent out of the SMU to the system's interrupt controller (APLIC). The outbox is only used for communication with AP cores. For all other resources we only use inboxes on both ends of the transaction.

The SEP mailboxes have 64-bit data inputs while SEP is a 32-bit system. This means that each 64-bit input word is mapped to two 32-bit words. The SEP host can access both the low-order and high-order words.

Mailbox Type	Description
ISS	Uses fabric source ID to qualify incoming transactions. Legal source ID is programmable, lockable and reset by SEP reset.
STEE	Relies on STEE indicator in the transactions (NS bit) to qualify incoming transactions.
DTEE	Reserved. Relies on the DTEE indicator in the transactions (confidential bit) to qualify incoming transactions.
Regular	Relies on PMA/PMP or page table for protection. No special action required by SEP.



4.4. Memory Map

SEP uses 32-bit addressing. The following table outlines the SEP CPU's logical view of the SEP-local address space. External masters use the chiplet-global base programmed in `SEP_GLOBAL_BASE_ADDR`; SEP CPU software uses either direct internal addresses (`0x1000_0000`, `0x1080_0000`, and so on) or the local alias view described below.

Base Address	End Address	Size	Unit	Description
0x0000_0000	0x1000_0000	256 MB	External	External to chiplet, but not in SMU (SMC, DTP, etc).
0x1000_0000	0x2000_0000	256 MB	SEP Local	SEP Local resources (DMA, Crypto, etc)
0x2000_0000	0x4000_0000	512 MB	SEP AXI Extension	Reserved for adopter extension IP
0x4000_0000	0x8000_0000	1 GB	SMC	SMC Resources
0x8000_0000	0xC000_0000	1 GB	SMU	SMU Resources (DTP, etc)
0xC000_0000	0xD000_0000	256 MB	SEP CPU Resources	EL2 Veer Resources (DCCM, ICCM, PIC)

Base Address	End Address	Size	Unit	Description
0xD000_0000	0x1_0000_0000	768 MB	SEP Local Alias	Corresponds to 0x1000_0000 - 0x4000_0000

The **SEP Local Alias** row is an alternate CPU-side name for the same hardware listed in the **SEP Local** and **SEP AXI Extension** rows. For example, **0xD000_0000** aliases **0x1000_0000** and **0xD080_2000** aliases **0x1080_2000**. This alias range is used by SEP-local masters, while external masters use chiplet-global addresses.

The EL2 CPU resources are mapped into a separate segment than the SEP Local resources due to constraints on the CPU memory map. The ICCM, DCCM, and PIC addresses in the **0xC000_0000** region are statically configured in the CPU core module and are reached through the CPU's internal wiring. The high local-alias mapping covers the **0xD000_0000** and above alias view. CPU CSRs are accessed with dedicated RISC-V instructions and do not appear in the address map.

The high local alias span has a fixed SEP-local size of 768 MiB (**0xD000_0000** through **0xFFFF_FFFF**), corresponding to the direct internal range **0x1000_0000** through **0x3FFF_FFFF**. The chiplet-global SEP aperture configured by **SEP_GLOBAL_BASE_ADDR** / **SEP_REGION_SIZE** controls the SMU xbar and inbound global-to-local remap.

Base Address	End Address	Size	Unit	Description
0xC000_0000	0xC003_FFFF	256 kB	ICCM	Tightly Coupled Instruction SRAM
0xC004_0000	0xC005_FFFF	128 kB	DCCM	Tightly Coupled Data SRAM
0xC006_0000	0xC007_FFFF			Reserved
0xC008_0000	0xC008_7FFF	32 kB	PIC	Interrupt controller for EL2 CPU
0xC008_8000	0xCFFF_FFFF			Reserved

The remaining subsystems are mapped into regions 1 and 2 for the EL2 CPU.

Base Address	End Address	Size	Unit	Description
0x1000_0000	0x1000_FFFF	64 kB	SRAM	Scratch SRAM
0x1001_0000	0x1003_FFFF	192 kB	RSV	Reserved Scratch SRAM expansion
0x1004_0000	0x1004_FFFF	64 kB	ROM	BL0 immutable instruction memory
0x1005_0000	0x107F_FFFF	7,872 KB	RSV	reserved
0x1080_0000	0x1080_0FFF	4 kB	DMA	DMA CSR
0x1080_1000	0x1080_1FFF	4 kB	WDT	Watchdog Timer
0x1080_2000	0x1080_2FFF	4 kB	SRB	Dual Scratch Register Banks
0x1080_3000	0x1080_3007	8 B	RST_CTRL	SEP Reset Controller (software reset for KM and crypto accelerators)
0x1080_3008	0x108F_FFFF	1,043 kB	RSV	reserved
0x1090_0000	0x1090_FFFF	64kB	OTBN	Public Key processor memory and CSR
0x1091_0000	0x1091_0FFF	4 kB	AES	Block Cipher
0x1091_1000	0x1091_2FFF	8 kB	HMAC-SHA2	Hash / MAC
0x1091_3000	0x1091_3FFF	4 kB	KMAC-SHA3	Hash / MAC
0x1091_4000	0x1091_4FFF	4 kB	RSV	Reserved

Base Address	End Address	Size	Unit	Description
0x1091_5000	0x1091_5FFF	4 kB	DRBG	Deterministic Random Bit Generator
0x1091_6000	0x1091_6FFF	4 kB	ESRC	Entropy Source
0x1091_7000	0x1091_7FFF	4 kB	TRNG	Random Num Gen
0x1091_8000	0x1091_FFFF	32 kB	LC	Life Cycle Controller
0x1092_0000	0x1092_0FFF	4 kB	KM	Key Manager
0x1092_1000	0x1092_FFFF		RSV	Reserved
0x1093_0000	0x1093_7FFF	32 kB	OTP	Fuse Control and Shadow Memory
0x1093_8000	0x1094_FFFF		RSV	Reserved
0x1095_0000	0x1095_FFFF	64 kB	EPOOL	Entropy Pool FIFO (read-only drain; writes return SLVERR). Filled by native EDN.
0x1096_0000	0x109F_FFFF		RSV	Reserved
0x10A0_0000	0x10A3_FFFF	256 kB	SYS	System Bus I/F
0x10A4_0000	0x10AF_FFFF		RSV	Reserved
0x10B0_0000	0x10BF_FFFF	1 MB	IO	External IO Peripherals Bridge: UART, GPIO etc.
0x1100_0000	0x117F_FFFF	8 MB	AP Remap Region	Write to this region to be routed to the AP remapper
0x1180_0000	0x11FF_FFFF	8 MB	STEE Remap Region	Write to this region to be routed to the STEE remapper
0x1200_0000	0x1FFF_FFFF	20 MB	RSV	Reserved
0x2000_0000	0x4000_0000	512 MB	AXI Extension Region	Reserved for adopter extension IP

4.5. Detailed Register Map

The following register tables are generated from the SEP register description (RDL) sub-blocks under [hw/sys/sep/regs/](#). Each entry corresponds to a unit in the SEP memory map above.

4.5.1. CPU Control and Interrupts

Table 4. Register Map

Address	Name	Access	Description
0x8	CLOCK_GATE_CTRL	RW	TODO: clock-gate control - placeholder fields, not yet implemented.
0x10	REFERENCE_COUNTER	RW	-
0x18	TIMEOUT_INTERRUPT	R	Reserved - per-block timeout interrupt not yet implemented.
0x20	PKA_CTRL	RW	TODO: PKA control - placeholder fields, not yet implemented.
0x28	TIMEOUT_COUNT_DMA	RW	Reserved - per-block timeout count/threshold not yet implemented.
0x30	TIMEOUT_COUNT_SYS_I N	RW	Reserved - per-block timeout count/threshold not yet implemented.
0x38	TIMEOUT_COUNT_MAI LBOX_INBOUND	RW	Reserved - per-block timeout count/threshold not yet implemented.

Address	Name	Access	Description
0x40	TIMEOUT_COUNT_MAILBOX_OUTBOUND	RW	Reserved - per-block timeout count/threshold not yet implemented.
0x48	TIMEOUT_COUNT_ENTROPY_WRITE	RW	Reserved - per-block timeout count/threshold not yet implemented.
0x50	TIMEOUT_COUNT_ENTROPY_READ	RW	Reserved - per-block timeout count/threshold not yet implemented.
0x58	TIMEOUT_COUNT_FILTER_OUT	RW	Reserved - per-block timeout count/threshold not yet implemented.
0x60	TIMEOUT_COUNT_ALIAS_REMAP	RW	Reserved - per-block timeout count/threshold not yet implemented.
0x68	TIMEOUT_ENABLE	RW	Reserved - per-block timeout enable not yet implemented.
0x70	TIMEOUT_CLEAR	W	Reserved - per-block timeout interrupt-clear not yet implemented.
0x78	TIMEOUT_MODE	W	Reserved - per-block timeout mode not yet implemented.
0xB0	SEP_TEST_CTRL	R	-
0xC0	SEP_GLOBAL_BASE_ADDR	RW	-
0xC8	SEP_LOCAL_BASE_ADDR	RW	-
0xD0	SEP_REGION_SIZE	RW	-
0x100	SMU_GLOBAL_BASE_ADDR	RW	-
0x110	SMU_REGION_SIZE	RW	-
0x140	SMC_FUSE_SENSE_STATUS	R	-
0x150	SEP_FUSE_SENSE_STATUS	R	-
0x160	SEP_STRAPS	R	-
0x170	RAS_BANK_INFO	RW	-
0x178	SEP_SW_DEBUG	RW	-
0x180	SEP_NMI_VEC	RW	NMI vector address for VeeR-EL2 CPU. The default value is 0xC000_1000. The NMI interrupt is used for the WDT Bark interrupt.
0x188	SEP_NMI_VEC_LOCK	RW	Lock the NMI vector address. Once locked, the NMI vector address cannot be changed. The NMI interrupt is used for the WDT Bark interrupt.
0x190	EXT_TRNG_SRC_SEL	RW	Per-stream external TRNG source selection for crypto subsystem entropy muxes.
0x198	EXT_TRNG_SRC_SEL_LOCK	RW	Lock the TRNG source selection. Once locked, EXT_TRNG_SRC_SEL cannot be changed until reset.
0x1A0	KM_WIPE_CTRL	RW	Key Manager emergency wipe control.
0x1000	SEP_VERSION_ID	R	-

Table 5. CLOCK_GATE_CTRL Register

Bits	Field	Access	Reset	Description
63:1	Reserved	-	-	Reserved
0	pkc_cg_enable	RW	0x0	Reserved - PKA clock-gate enable, not yet implemented.

Table 6. REFERENCE_COUNTER Register

Bits	Field	Access	Reset	Description
63:0	rc	RW	0x0	SEP REFERENCE COUNTER, counting after deassert of cold reset

Table 7. TIMEOUT_INTERRUPT Register

Bits	Field	Access	Reset	Description
63:1	Reserved	-	-	Reserved
0	reserved	R	0x0	Reserved - timeout interrupt status not yet implemented.

Table 8. PKA_CTRL Register

Bits	Field	Access	Reset	Description
63:3	Reserved	-	-	Reserved
2	pka_noise_src_valid	RW	0x0	Reserved - PKA noise source valid, not yet implemented.
1	pka_noise_src	RW	0x0	Reserved - PKA noise source select, not yet implemented.
0	pka_dpa_disable	RW	0x0	Reserved - disable DPA countermeasures in PKA, not yet implemented.

Table 9. TIMEOUT_COUNT_DMA Register

Bits	Field	Access	Reset	Description
63:1	Reserved	-	-	Reserved
0	reserved	RW	0x0	Reserved - timeout count not yet implemented.

Table 10. TIMEOUT_COUNT_SYS_IN Register

Bits	Field	Access	Reset	Description
63:1	Reserved	-	-	Reserved
0	reserved	RW	0x0	Reserved - timeout count not yet implemented.

Table 11. TIMEOUT_COUNT_MAILBOX_INBOUND Register

Bits	Field	Access	Reset	Description
63:1	Reserved	-	-	Reserved
0	reserved	RW	0x0	Reserved - timeout count not yet implemented.

Table 12. TIMEOUT_COUNT_MAILBOX_OUTBOUND Register

Bits	Field	Access	Reset	Description
63:1	Reserved	-	-	Reserved
0	reserved	RW	0x0	Reserved - timeout count not yet implemented.

Table 13. TIMEOUT_COUNT_ENTROPY_WRITE Register

Bits	Field	Access	Reset	Description
63:1	Reserved	-	-	Reserved
0	reserved	RW	0x0	Reserved - timeout count not yet implemented.

Table 14. TIMEOUT_COUNT_ENTROPY_READ Register

Bits	Field	Access	Reset	Description
63:1	Reserved	-	-	Reserved

Bits	Field	Access	Reset	Description
0	reserved	RW	0x0	Reserved - timeout count not yet implemented.

Table 15. TIMEOUT_COUNT_FILTER_OUT Register

Bits	Field	Access	Reset	Description
63:1	Reserved	-	-	Reserved
0	reserved	RW	0x0	Reserved - timeout count not yet implemented.

Table 16. TIMEOUT_COUNT_ALIAS_REMAP Register

Bits	Field	Access	Reset	Description
63:1	Reserved	-	-	Reserved
0	reserved	RW	0x0	Reserved - timeout count not yet implemented.

Table 17. TIMEOUT_ENABLE Register

Bits	Field	Access	Reset	Description
63:1	Reserved	-	-	Reserved
0	reserved	RW	0x0	Reserved - timeout monitors not yet implemented.

Table 18. TIMEOUT_CLEAR Register

Bits	Field	Access	Reset	Description
63:1	Reserved	-	-	Reserved
0	reserved	W	0x0	Reserved - timeout monitors not yet implemented.

Table 19. TIMEOUT_MODE Register

Bits	Field	Access	Reset	Description
63:1	Reserved	-	-	Reserved
0	reserved	W	0x0	Reserved - timeout monitors not yet implemented.

Table 20. SEP_TEST_CTRL Register

Bits	Field	Access	Reset	Description
31	fast_spi_en	R	0x0	Fast SPI Enable
30	fast_iccm_en	R	0x0	Fast ICCM Enable
29	fast_dccm_en	R	0x0	Fast DCCM Enable
28	fast_sram_en	R	0x0	Fast SRAM Enable
27	fast_pka_en	R	0x0	Fast PKA Enable
26	sep_standalone	R	0x0	SEP is standalone (not part of eg SMU)
25:0	Reserved	-	-	Reserved

Table 21. SEP_GLOBAL_BASE_ADDR Register

Bits	Field	Access	Reset	Description
63:56	Reserved	-	-	Reserved
55:0	addr	RW	0x0	-

Table 22. SEP_LOCAL_BASE_ADDR Register

Bits	Field	Access	Reset	Description
63:56	Reserved	-	-	Reserved
55:0	addr	RW	0xD0000000	-

Table 23. SEP_REGION_SIZE Register

Bits	Field	Access	Reset	Description
63:32	Reserved	-	-	Reserved
31:0	size	RW	0x1000000	-

Table 24. SMU_GLOBAL_BASE_ADDR Register

Bits	Field	Access	Reset	Description
63:56	Reserved	-	-	Reserved
55:0	addr	RW	0x80000000	-

Table 25. SMU_REGION_SIZE Register

Bits	Field	Access	Reset	Description
63:32	Reserved	-	-	Reserved
31:0	size	RW	0x40000000	-

Table 26. SMC_FUSE_SENSE_STATUS Register

Bits	Field	Access	Reset	Description
63:1	Reserved	-	-	Reserved
0	smc_fuse_sense_done	R	0x0	This bit indicates when SMC's fuse sense is completed. While fuse sense is ongoing, SMC fabric will be in reset. Any requests to the SMC will hang. When this is 1, it is safe to send requests to the SMC.

Table 27. SEP_FUSE_SENSE_STATUS Register

Bits	Field	Access	Reset	Description
63:1	Reserved	-	-	Reserved
0	sep_fuse_sense_done	R	0x0	This reg is used to indicate the fuse sense status of SEP.

Table 28. SEP_STRAPS Register

Bits	Field	Access	Reset	Description
31:2	Reserved	-	-	Reserved
1	bypass_mem_repair	R	0x0	straps for bypass_mem_repair → Used to indicate if the memory repair should be bypassed
0	test_en	R	0x0	straps for test_en → Used to be latched by SEP and generate the secure_tm

Table 29. RAS_BANK_INFO Register

Bits	Field	Access	Reset	Description
31:8	Reserved	-	-	Reserved
7:4	bank_instance	RW	0x0	Chiplet instance ID
3:0	bank_chip	RW	0x0	Chiplet type ID

Table 30. SEP_SW_DEBUG Register

Bits	Field	Access	Reset	Description
63:32	Reserved	-	-	Reserved
31:0	sep_sw_debug	RW	0x0	software writable debug information. This reg is routed to CLA

Table 31. SEP_NMI_VEC Register

Bits	Field	Access	Reset	Description
63:32	Reserved	-	-	Reserved
31:1	nmi_vec	RW	0x60000080	NMI vector address [31:1] for VeeR-EL2 CPU. Bit 0 is always 0 (aligned). Address where CPU jumps on Non-Maskable Interrupt. This is used for the WDT Bark interrupt.
0	rsvd	R	0x0	Reserved, bit 0 is always 0 (aligned)

Table 32. SEP_NMI_VEC_LOCK Register

Bits	Field	Access	Reset	Description
63:1	Reserved	-	-	Reserved
0	lock	RW	0x0	Lock the NMI vector address. Once locked, the NMI vector address cannot be changed. The NMI interrupt is used for the WDT Bark interrupt.

Table 33. EXT_TRNG_SRC_SEL Register

Bits	Field	Access	Reset	Description
63:3	Reserved	-	-	Reserved
2:0	sel	RW	0x7	TRNG source select per stream. Bit N selects the source for stream N (bit0 = Key Manager, bit1 = crypto blocks, bit2 = entropy pool). 0 = internal DRBG output, 1 = external TRNG (default).

Table 34. EXT_TRNG_SRC_SEL_LOCK Register

Bits	Field	Access	Reset	Description
63:1	Reserved	-	-	Reserved
0	lock	RW	0x0	Lock TRNG source selection. Once set, cannot be cleared by software.

Table 35. KM_WIPE_CTRL Register

Bits	Field	Access	Reset	Description
63:1	Reserved	-	-	Reserved
0	wipe_state	RW	0x0	Key Manager wipe state. A rising edge zeroes all KPV entries and sets the WIPE_STATE interrupt status.

Table 36. SEP_VERSION_ID Register

Bits	Field	Access	Reset	Description
63:32	Reserved	-	-	Reserved
31:0	version_id	R	0xDEADBEEF	Version ID for the CPU

NOTE

Register Arrays: This register map contains the following register arrays:

- **meipl[32]**: 32 registers
 - Base Address: 0x4
 - Address Increment: 0x4 per register
- **meip[2]**: 2 registers
 - Base Address: 0x1000
 - Address Increment: 0x4 per register
- **meie[32]**: 32 registers
 - Base Address: 0x2004
 - Address Increment: 0x4 per register
- **meigwctrl[32]**: 32 registers
 - Base Address: 0x4004
 - Address Increment: 0x4 per register
- **meigwclr[32]**: 32 registers
 - Base Address: 0x5004
 - Address Increment: 0x4 per register

Table 37. Register Map

Address	Name	Access	Description
0x4 - 0x80	meipl[32]	RW	External Interrupt Priority Level Register. Sets the priority level for an interrupt source.
0x1000 - 0x1004	meip[2]	R	External Interrupt Pending Register. Reports pending status of up to 32 interrupt sources.
0x2004 - 0x2080	meie[32]	RW	External Interrupt Enable Register. Controls interrupt enable for a single source.
0x3000	mpiccfg	RW	PIC Configuration Register. Used to select the operational parameters of the PIC.
0x4004 - 0x4080	meigwctrl[32]	RW	External Interrupt Gateway Configuration Register. Controls interrupt type and polarity.
0x5004 - 0x5080	meigwclr[32]	W	External Interrupt Gateway Clear Register. Writing any value clears the gateway's interrupt pending (IP) bit for edge-triggered interrupts.

Table 38. meipl[32] Register

Bits	Field	Access	Reset	Description
31:4	Reserved	-	-	Reserved
3:0	intpriority	RW	0x0	External interrupt priority level: RISC-V standard order: 0=Never interrupt, 1..15=Priority (1 lowest, 15 highest) Reverse order: 15=Never interrupt, 14..0=Priority (14 lowest, 0 highest)

Table 39. meip[2] Register

Bits	Field	Access	Reset	Description
31:0	intpend	R	0x0	External interrupt pending bits. Bit Y indicates pending status for source X*32+Y. 0: Interrupt not pending 1: Interrupt pending

Table 40. meie[32] Register

Bits	Field	Access	Reset	Description
31:1	Reserved	-	-	Reserved
0	inten	RW	0x0	External interrupt enable: 0: Interrupt disabled 1: Interrupt enabled

Table 41. mpiccfg Register

Bits	Field	Access	Reset	Description
31:1	Reserved	-	-	Reserved
0	priord	RW	0x0	Priority order: 0: RISC-V standard compliant priority order (0=lowest to 15=highest) 1: Reverse priority order (15=lowest to 0=highest)

Table 42. meigwctrl[32] Register

Bits	Field	Access	Reset	Description
31:2	Reserved	-	-	Reserved
1	irq_type	RW	0x0	External interrupt type: 0: Level-triggered interrupt 1: Edge-triggered interrupt
0	polarity	RW	0x0	External interrupt polarity: 0: Active-high interrupt 1: Active-low interrupt

Table 43. meigwclr[32] Register

Bits	Field	Access	Reset	Description
31:1	Reserved	-	-	Reserved
0	clear	W	0x0	Write any value to clear the interrupt pending bit in the gateway. This register is write-only; reads return 0.

4.5.2. Reset and Lifecycle

Table 44. Register Map

Address	Name	Access	Description
0x0	SW_RESET_N	RW	-

Table 45. SW_RESET_N Register

Bits	Field	Access	Reset	Description
63:5	Reserved	-	-	Reserved
4	kmac_sw_rst_n	RW	0x1	KMAC reset (0=assert, 1=release)
3	hmac_sw_rst_n	RW	0x1	HMAC reset (0=assert, 1=release)
2	aes_sw_rst_n	RW	0x1	AES reset (0=assert, 1=release)
1	otbn_sw_rst_n	RW	0x1	OTBN reset (0=assert, 1=release)
0	km_sw_rst_n	RW	0x0	KM reset (0=assert, 1=release)

Table 46. Register Map

Address	Name	Access	Description
0x0	FEAT_CTRL	R	-
0x8	DEMOTEL1	RW	-

Address	Name	Access	Description
0x10	DEMOTE_2	RW	-

Table 47. FEAT_CTRL Register

Bits	Field	Access	Reset	Description
63:0	feature_control	R	0x0	Lifecycle feature control bits,

Table 48. DEMOTE_1 Register

Bits	Field	Access	Reset	Description
63:2	rsvd	RW	0x0	Not used
1	lock	RW	0x0	Demote lock bit. Once locked, demote can no longer be changed
0	demote	RW	0x0	Demote bit to enter PROD_DBG state

Table 49. DEMOTE_2 Register

Bits	Field	Access	Reset	Description
63:2	rsvd	RW	0x0	Not used
1	lock	RW	0x0	Demote lock bit. Once locked, demote can no longer be changed
0	demote	RW	0x0	Demote bit to enter PROD_DBG state

4.5.3. Scratch and eFuse

NOTE

Register Arrays: This register map contains the following register arrays:

- **SCRATCH[8]:** 8 registers
 - Base Address: 0x0
 - Address Increment: 0x8 per register

Table 50. Register Map

Address	Name	Access	Description
0x0 - 0x38	SCRATCH[8]	RW	-

Table 51. SCRATCH[8] Register

Bits	Field	Access	Reset	Description
63:32	Reserved	-	-	Reserved
31:0	data	RW	0x0	Scratch register for the CPU

Table 52. Register Map

Address	Name	Access	Description
0x0	LOCKS	RW	-
0x8	LC_STATE	R	-
0xC	SBOOT_DIS	R	-
0x10	TRANSIENT_RMA_EN	R	-

Address	Name	Access	Description
0x14	SiP_DIS	RW	An owner's (SiP or System) feature control vector. When set, the corresponding feature is disabled
0x1C	SYS_DIS	RW	An owner's (SiP or System) feature control vector. When set, the corresponding feature is disabled
0x24	RMA_SIP_TOKEN_DIGEST	R	-
0x44	RMA_CHIPLET_TOKEN_DIGEST	R	-
0x64	CLASS_KEY	R	-
0x84	CHIPLET_PUBK_REVOKED	R	-
0x88	BL1_VERSION	R	-
0xA8	BL2_VERSION	R	-
0xC8	CHIPLET_UID	R	-
0xE8	SiP_PUBK_DIGEST	R	-
0x108	SiP_UID	R	-
0x128	SYS_PUBK_DIGEST	R	-
0x148	SYS_UID	R	-
0x168	STATUS_RPT	R	-
0x16C	SEP_ROM_CTRL	R	-
0x170	SEP_SPI_CTRL_FIELD_EN	R	-
0x174	SPI_DISCOVERY_CTRL	R	-
0x178	SPI_PHY_DQ_TIMING	R	-
0x17C	SPI_PHY_DQS_TIMING	R	-
0x180	SPI_PHY_GATE_LPBK	R	-
0x184	SPI_PHY_DLL_SLAVE	R	-
0x188	SPI_PHY_DLL_MASTER	R	-
0x18C	SPI_PHY_MISC	R	-
0x190	SPI_RB_VALID_TIME	R	-
0x194	PUBLIC_KEY_0	R	-
0x1B4	PUBLIC_KEY_1	R	-
0x1D4	RESERVED_0	R	-
0x214	RESERVED_1	R	-
0x254	RESERVED_2	R	-
0x294	RESERVED_3	R	-
0x2D4	RESERVED_4	R	-
0x314	RESERVED_5	R	-
0x354	RESERVED_6	R	-
0x394	RESERVED_7	R	-
0x3D4	RESERVED_LAST_256	R	-
0x3F4	RESERVED_LAST_64	R	-
0x3FC	RESERVED_LAST_32	R	-

Table 53. LOCKS Register

Bits	Field	Access	Reset	Description
63	RESERVED_LAST_32_READ_LOCK	RW	0x0	RESERVED_LAST_32_READ_LOCK, when set, this field is not read-able (neither the eFuse nor the shadow register)
62	RESERVED_LAST_32_WRITE_LOCK	RW	0x0	RESERVED_LAST_32_WRITE_LOCK, when set, this field is not write-able (neither the eFuse nor the shadow register)
61	RESERVED_LAST_64_READ_LOCK	RW	0x0	RESERVED_LAST_64_READ_LOCK, when set, this field is not read-able (neither the eFuse nor the shadow register)
60	RESERVED_LAST_64_WRITE_LOCK	RW	0x0	RESERVED_LAST_64_WRITE_LOCK, when set, this field is not write-able (neither the eFuse nor the shadow register)
59	RESERVED_LAST_256_READ_LOCK	RW	0x0	RESERVED_LAST_256_READ_LOCK, when set, this field is not read-able (neither the eFuse nor the shadow register)
58	RESERVED_LAST_256_WRITE_LOCK	RW	0x0	RESERVED_LAST_256_WRITE_LOCK, when set, this field is not write-able (neither the eFuse nor the shadow register)
57	RESERVED_7_READ_LOCK	RW	0x0	RESERVED_7_READ_LOCK, when set, this field is not read-able (neither the eFuse nor the shadow register)
56	RESERVED_7_WRITE_LOCK	RW	0x0	RESERVED_7_WRITE_LOCK, when set, this field is not write-able (neither the eFuse nor the shadow register)
55	RESERVED_6_READ_LOCK	RW	0x0	RESERVED_6_READ_LOCK, when set, this field is not read-able (neither the eFuse nor the shadow register)
54	RESERVED_6_WRITE_LOCK	RW	0x0	RESERVED_6_WRITE_LOCK, when set, this field is not write-able (neither the eFuse nor the shadow register)
53	RESERVED_5_READ_LOCK	RW	0x0	RESERVED_5_READ_LOCK, when set, this field is not read-able (neither the eFuse nor the shadow register)
52	RESERVED_5_WRITE_LOCK	RW	0x0	RESERVED_5_WRITE_LOCK, when set, this field is not write-able (neither the eFuse nor the shadow register)
51	RESERVED_4_READ_LOCK	RW	0x0	RESERVED_4_READ_LOCK, when set, this field is not read-able (neither the eFuse nor the shadow register)
50	RESERVED_4_WRITE_LOCK	RW	0x0	RESERVED_4_WRITE_LOCK, when set, this field is not write-able (neither the eFuse nor the shadow register)
49	RESERVED_3_READ_LOCK	RW	0x0	RESERVED_3_READ_LOCK, when set, this field is not read-able (neither the eFuse nor the shadow register)
48	RESERVED_3_WRITE_LOCK	RW	0x0	RESERVED_3_WRITE_LOCK, when set, this field is not write-able (neither the eFuse nor the shadow register)
47	RESERVED_2_READ_LOCK	RW	0x0	RESERVED_2_READ_LOCK, when set, this field is not read-able (neither the eFuse nor the shadow register)
46	RESERVED_2_WRITE_LOCK	RW	0x0	RESERVED_2_WRITE_LOCK, when set, this field is not write-able (neither the eFuse nor the shadow register)
45	RESERVED_1_READ_LOCK	RW	0x0	RESERVED_1_READ_LOCK, when set, this field is not read-able (neither the eFuse nor the shadow register)
44	RESERVED_1_WRITE_LOCK	RW	0x0	RESERVED_1_WRITE_LOCK, when set, this field is not write-able (neither the eFuse nor the shadow register)
43	RESERVED_0_READ_LOCK	RW	0x0	RESERVED_0_READ_LOCK, when set, this field is not read-able (neither the eFuse nor the shadow register)
42	RESERVED_0_WRITE_LOCK	RW	0x0	RESERVED_0_WRITE_LOCK, when set, this field is not write-able (neither the eFuse nor the shadow register)
41	SEP_PUBLIC_KEY_HASH_1_READ_LOCK	RW	0x0	SEP_PUBLIC_KEY_HASH_1_READ_LOCK, when set, this field is not read-able (neither the eFuse nor the shadow register)
40	SEP_PUBLIC_KEY_HASH_1_WRITE_LOCK	RW	0x0	SEP_PUBLIC_KEY_HASH_1_WRITE_LOCK, when set, this field is not write-able (neither the eFuse nor the shadow register)

Bits	Field	Access	Reset	Description
39	SEP_PUBLIC_KEY_HASH_0_READ_LOCK	RW	0x0	SEP_PUBLIC_KEY_HASH_0_READ_LOCK, when set, this field is not read-able (neither the eFuse nor the shadow register)
38	SEP_PUBLIC_KEY_HASH_0_WRITE_LOCK	RW	0x0	SEP_PUBLIC_KEY_HASH_0_WRITE_LOCK, when set, this field is not write-able (neither the eFuse nor the shadow register)
37	SEP_SPI_CTRL_READ_LOCK	RW	0x0	sep_spi_ctrl_read_lock, when set, this field is not read-able (neither the eFuse nor the shadow register)
36	SEP_SPI_CTRL_WRITE_LOCK	RW	0x0	sep_spi_ctrl_write_lock, when set, this field is not write-able (neither the eFuse nor the shadow register)
35	SEP_ROM_CTRL_READ_LOCK	RW	0x0	sep_rom_ctrl_read_lock, when set, this field is not read-able (neither the eFuse nor the shadow register)
34	SEP_ROM_CTRL_WRITE_LOCK	RW	0x0	sep_rom_ctrl_write_lock, when set, this field is not write-able (neither the eFuse nor the shadow register)
33	STATUS_RPT_READ_LOCK	RW	0x0	status_rpt_read_lock, when set, this field is not read-able (neither the eFuse nor the shadow register)
32	STATUS_RPT_WRITE_LOCK	RW	0x0	status_rpt_write_lock, when set, this field is not write-able (neither the eFuse nor the shadow register)
31	SYS_UID_READ_LOCK	RW	0x0	sys_uid_read_lock, when set, this field is not read-able (neither the eFuse nor the shadow register)
30	SYS_UID_WRITE_LOCK	RW	0x0	sys_uid_write_lock, when set, this field is not write-able (neither the eFuse nor the shadow register)
29	SYS_PUBK_DIGEST_READ_LOCK	RW	0x0	sys_pubk_digest_read_lock, when set, this field is not read-able (neither the eFuse nor the shadow register)
28	SYS_PUBK_DIGEST_WRITE_LOCK	RW	0x0	sys_pubk_digest_write_lock, when set, this field is not write-able (neither the eFuse nor the shadow register)
27	SiP_UID_READ_LOCK	RW	0x0	sip_uid_read_lock, when set, this field is not read-able (neither the eFuse nor the shadow register)
26	SiP_UID_WRITE_LOCK	RW	0x0	sip_uid_write_lock, when set, this field is not write-able (neither the eFuse nor the shadow register)
25	SiP_PUBK_DIGEST_READ_LOCK	RW	0x0	sip_pubk_digest_read_lock, when set, this field is not read-able (neither the eFuse nor the shadow register)
24	SiP_PUBK_DIGEST_WRITE_LOCK	RW	0x0	sip_pubk_digest_write_lock, when set, this field is not write-able (neither the eFuse nor the shadow register)
23	CHIPLET_UID_READ_LOCK	RW	0x0	chiptlet_uid_read_lock, when set, this field is not read-able (neither the eFuse nor the shadow register)
22	CHIPLET_UID_WRITE_LOCK	RW	0x0	chiptlet_uid_write_lock, when set, this field is not write-able (neither the eFuse nor the shadow register)
21	BL2_VERSION_READ_LOCK	RW	0x0	bl2_version_read_lock, when set, this field is not read-able (neither the eFuse nor the shadow register)
20	BL2_VERSION_WRITE_LOCK	RW	0x0	bl2_version_write_lock, when set, this field is not write-able (neither the eFuse nor the shadow register)
19	BL1_VERSION_READ_LOCK	RW	0x0	bl1_version_read_lock, when set, this field is not read-able (neither the eFuse nor the shadow register)
18	BL1_VERSION_WRITE_LOCK	RW	0x0	bl1_version_write_lock, when set, this field is not write-able (neither the eFuse nor the shadow register)
17	CHIPLET_PUBK_REVOKE_READ_LOCK	RW	0x0	chiptlet_pubk_revoke_sel_read_lock, when set, this field is not read-able (neither the eFuse nor the shadow register)
16	CHIPLET_PUBK_REVOKE_WRITE_LOCK	RW	0x0	chiptlet_pubk_revoke_sel_write_lock, when set, this field is not write-able (neither the eFuse nor the shadow register)
15	CLASS_KEY_READ_LOCK	RW	0x0	class_key_read_lock, when set, this field is not read-able (neither the eFuse nor the shadow register)

Bits	Field	Access	Reset	Description
14	CLASS_KEY_WRITE_LOCK	RW	0x0	class_key_write_lock, when set, this field is not write-able (neither the eFuse nor the shadow register)
13	RMA_CHIPLET_TOKEN_DIGEST_READ_LOCK	RW	0x0	rma_chiplet_token_digest_read_lock, when set, this field is not read-able (neither the eFuse nor the shadow register)
12	RMA_CHIPLET_TOKEN_DIGEST_WRITE_LOCK	RW	0x0	rma_chiplet_token_digest_write_lock, when set, this field is not write-able (neither the eFuse nor the shadow register)
11	RMA_SIP_TOKEN_DIGEST_READ_LOCK	RW	0x0	rma_sip_token_digest_read_lock, when set, this field is not read-able (neither the eFuse nor the shadow register)
10	RMA_SIP_TOKEN_DIGEST_WRITE_LOCK	RW	0x0	rma_sip_token_digest_write_lock, when set, this field is not write-able (neither the eFuse nor the shadow register)
9	SYS_DIS_READ_LOCK	RW	0x0	sys_dis_read_lock, when set, this field is not read-able (neither the eFuse nor the shadow register)
8	SYS_DIS_WRITE_LOCK	RW	0x0	sys_dis_write_lock, when set, this field is not write-able (neither the eFuse nor the shadow register)
7	SIP_DIS_READ_LOCK	RW	0x0	sip_dis_read_lock, when set, this field is not read-able (neither the eFuse nor the shadow register)
6	SIP_DIS_WRITE_LOCK	RW	0x0	sip_dis_write_lock, when set, this field is not write-able (neither the eFuse nor the shadow register)
5	TRANSIENT_RMA_EN_READ_LOCK	RW	0x0	TRANSIENT_RMA_EN_READ_LOCK, when set, this field is not read-able (neither the eFuse nor the shadow register)
4	TRANSIENT_RMA_EN_WRITE_LOCK	RW	0x0	TRANSIENT_RMA_EN_WRITE_LOCK, when set, this field is not write-able (neither the eFuse nor the shadow register)
3	SBOOT_DIS_READ_LOCK	RW	0x0	sboot_dis_read_lock, when set, this field is not read-able (neither the eFuse nor the shadow register)
2	SBOOT_DIS_WRITE_LOCK	RW	0x0	sboot_dis_write_lock, when set, this field is not write-able (neither the eFuse nor the shadow register)
1	LC_STATE_READ_LOCK	RW	0x0	lc_state_read_lock, when set, this field is not read-able (neither the eFuse nor the shadow register)
0	LC_STATE_WRITE_LOCK	RW	0x0	lc_state_write_lock, when set, this field is not write-able (neither the eFuse nor the shadow register)

Table 54. LC_STATE Register

Bits	Field	Access	Reset	Description
31:8	rsvd	R	0x0	Unused
7:0	lc_state	R	0xF0	Life cycle manufacturing state, differentially encoded: {~raw[3:0], raw[3:0]}

Table 55. SBOOT_DIS Register

Bits	Field	Access	Reset	Description
31:1	rsvd	R	0x0	Unused
0	disable_secure_boot	R	0x0	Chicken bit to disable secure boot in SEP ROM.

Table 56. TRANSIENT_RMA_EN Register

Bits	Field	Access	Reset	Description
31:1	rsvd	R	0x0	Unused
0	transient_rma_en	R	0x0	When Transient RMA EN is 1, matching on corresponding *_Tokens will automatically transfer the chip to the corresponding RMA state by setting the LC_STATE reg

Table 57. SiP_DIS Register

Bits	Field	Access	Reset	Description
63:48	func_reserved	RW	0x0	Reserved
47:37	test_reserved	RW	0x0	Reserved
36	ap_dtest	RW	0x0	When true, diagnostic tests are disabled for non-SEP logic; When false, diagnostic tests are enabled for non-SEP logic.
35	ap_stest	RW	0x0	When true, screening tests are disabled for non-SEP logic; When false, screening tests are enabled for non-SEP logic.
34	sep_dtest	RW	0x0	When true, diagnostic tests are disabled for SEP; When false, diagnostic tests are enabled for SEP.
33	sep_stest	RW	0x0	When true, screening tests are disabled for SEP; When false, screening tests are enabled for SEP.
32	fuse_test	RW	0x0	When true, fuse controller and fuse block interface are non-scannable; When false, fuse controller and fuse block interface are scannable.
31:5	debug_reserved	RW	0x0	Reserved
4	sip_debug	RW	0x0	When true, external debug traffic is forwarded to downstream chiplets via outbound JTAG PTAP, is disabled; When false, external debug traffic is not forwarded to outbound JTAG PTAP.
3	ap_trace	RW	0x0	When true, trace logic across the chiplet, e.g. Ascalon cluster, is disabled; When false, trace logic is enabled.
2	ap_debug	RW	0x0	When true, debug module across the chiplet, e.g. Ascalon cluster, is disabled; When false, debug logic is enabled.
1	soc_debug	RW	0x0	Silicon overall debug control; When true, external traffics can only get access to silicon's peripheral states, but not internal state. Scan dump is disabled; When false, external traffics can get access to silicon's internal states. Scan dump is enabled.
0	sep_debug	RW	0x0	SEP's overall debug control; When true, external traffics need to go through SEP inbound filter to get access to SEP's internal states, scan dump of SEP is disabled; When false, SEP inbound traffic filter let all transactions pass, scan dump of SEP is enabled.

Table 58. SYS_DIS Register

Bits	Field	Access	Reset	Description
63:48	func_reserved	RW	0x0	Reserved
47:37	test_reserved	RW	0x0	Reserved
36	ap_dtest	RW	0x0	When true, diagnostic tests are disabled for non-SEP logic; When false, diagnostic tests are enabled for non-SEP logic.
35	ap_stest	RW	0x0	When true, screening tests are disabled for non-SEP logic; When false, screening tests are enabled for non-SEP logic.
34	sep_dtest	RW	0x0	When true, diagnostic tests are disabled for SEP; When false, diagnostic tests are enabled for SEP.
33	sep_stest	RW	0x0	When true, screening tests are disabled for SEP; When false, screening tests are enabled for SEP.
32	fuse_test	RW	0x0	When true, fuse controller and fuse block interface are non-scannable; When false, fuse controller and fuse block interface are scannable.
31:5	debug_reserved	RW	0x0	Reserved

Bits	Field	Access	Reset	Description
4	sip_debug	RW	0x0	When true, external debug traffic is forwarded to downstream chiplets via outbound JTAG PTAP, is disabled; When false, external debug traffic is not forwarded to outbound JTAG PTAP.
3	ap_trace	RW	0x0	When true, trace logic across the chiplet, e.g. Ascalon cluster, is disabled; When false, trace logic is enabled.
2	ap_debug	RW	0x0	When true, debug module across the chiplet, e.g. Ascalon cluster, is disabled; When false, debug logic is enabled.
1	soc_debug	RW	0x0	Silicon overall debug control; When true, external traffics can only get access to silicon's peripheral states, but not internal state. Scan dump is disabled; When false, external traffics can get access to silicon's internal states. Scan dump is enabled.
0	sep_debug	RW	0x0	SEP's overall debug control; When true, external traffics need to go through SEP inbound filter to get access to SEP's internal states, scan dump of SEP is disabled; When false, SEP inbound traffic filter let all transactions pass, scan dump of SEP is enabled.

Table 59. RMA_SIP_TOKEN_DIGEST Register

Bits	Field	Access	Reset	Description
255:0	token_digest	R	0x0	Hash digest of life cycle secret that allows entering RMA_SIP state / Note this is a 256 bit field

Table 60. RMA_CHIPLET_TOKEN_DIGEST Register

Bits	Field	Access	Reset	Description
255:0	token_digest	R	0x0	Hash digest of life cycle secret that allows entering RMA_CHIPLET state/ Note this is a 256 bit field

Table 61. CLASS_KEY Register

Bits	Field	Access	Reset	Description
255:0	key	R	0x0	Class key that could be used for FW encryption/decryption

Table 62. CHIPLET_PUBK_REVOKE Register

Bits	Field	Access	Reset	Description
31:0	select	R	0x0	Revoke public key used to verify BL1/ SEP ROM Should reject using the corresponding public key when associated bit is set to 1

Table 63. BL1_VERSION Register

Bits	Field	Access	Reset	Description
255:0	version	R	0x0	Minimum BL1 version that can be loaded by SEEPROM

Table 64. BL2_VERSION Register

Bits	Field	Access	Reset	Description
255:0	version	R	0x0	Minimum BL2 version that can be loaded by BL1. If more boot stages exist, rollback protection can be implemented by BL2.

Table 65. CHIPLET_UID Register

Bits	Field	Access	Reset	Description
255:0	uid	R	0x0	Silicon creator's confidential device unique ID.

Table 66. SiP_PUBK_DIGEST Register

Bits	Field	Access	Reset	Description
255:0	key_digest	R	0x0	Hash digest of SiP owner's root public key for secure boot.

Table 67. SiP_UID Register

Bits	Field	Access	Reset	Description
255:0	uid	R	0x0	SiP owner's confidential device unique ID.

Table 68. SYS_PUBK_DIGEST Register

Bits	Field	Access	Reset	Description
255:0	key_digest	R	0x0	Hash digest of system owner's root public key for secure boot.

Table 69. SYS_UID Register

Bits	Field	Access	Reset	Description
255:0	uid	R	0x0	System owner's confidential device unique ID.

Table 70. STATUS_RPT Register

Bits	Field	Access	Reset	Description
31:2	reserved	R	0x0	reserved
1:0	rpt	R	0x0	BOOTROM report status based on the bit. 2'b00 → bootrom report status based on GPIO; 2'b01 → bootrom report status; otherwise, bootrom does not report status

Table 71. SEP_ROM_CTRL Register

Bits	Field	Access	Reset	Description
31:6	reserved	R	0x0	reserved
5:1	rom_swap_ctrl	R	0x0	controls how the rom slices are swapped
0	rom_endianness_ctrl	R	0x0	controls the endianness of the rom data out. 0 → little endian; 1 → big endian

Table 72. SEP_SPI_CTRL_FIELD_EN Register

Bits	Field	Access	Reset	Description
31:19	spi_control_field_en_rsvd	R	0x0	Reserved 13 bits
18:8	smu_pll_sysclk	R	0x0	11 bits that indicates the configured sysclk PLL frequency in MHz; ignored if 0
7:0	spi_control_field_en	R	0x0	8 bit which indicates the following 8 spi config regs are enabled or not. 1 → enabled; 0 → disabled

Table 73. SPI_DISCOVERY_CTRL Register

Bits	Field	Access	Reset	Description
31:0	discovery	R	0x0	spi discovery control reg

Table 74. SPI_PHY_DQ_TIMING Register

Bits	Field	Access	Reset	Description
31:0	dq_timing	R	0x0	spi phy dq timing reg

Table 75. SPI_PHY_DQS_TIMING Register

Bits	Field	Access	Reset	Description
31:0	dqs_timing	R	0x0	spi phy dqs timing reg

Table 76. SPI_PHY_GATE_LPBK Register

Bits	Field	Access	Reset	Description
31:0	gate_lpbk	R	0x0	spi phy gate lpbk ctrl reg

Table 77. SPI_PHY_DLL_SLAVE Register

Bits	Field	Access	Reset	Description
31:0	dll_slave	R	0x0	spi phy dll slv ctrl reg

Table 78. SPI_PHY_DLL_MASTER Register

Bits	Field	Access	Reset	Description
31:0	dll_master	R	0x0	spi phy dll mst ctrl reg

Table 79. SPI_PHY_MISC Register

Bits	Field	Access	Reset	Description
31:0	misc	R	0x0	spi init phy misc

Table 80. SPI_RB_VALID_TIME Register

Bits	Field	Access	Reset	Description
31:0	rb_valid_time	R	0x0	spi init rb valid time

Table 81. PUBLIC_KEY_0 Register

Bits	Field	Access	Reset	Description
255:0	key_hash	R	0x0	Extra public key hash digest

Table 82. PUBLIC_KEY_1 Register

Bits	Field	Access	Reset	Description
255:0	key_hash	R	0x0	Extra public key hash digest

Table 83. RESERVED_0 Register

Bits	Field	Access	Reset	Description
511:0	rsvd	R	0x0	Reserved region for future use

Table 84. RESERVED_1 Register

Bits	Field	Access	Reset	Description
511:0	rsvd	R	0x0	Reserved region for future use

Table 85. RESERVED_2 Register

Bits	Field	Access	Reset	Description
511:0	rsvd	R	0x0	Reserved region for future use

Table 86. RESERVED_3 Register

Bits	Field	Access	Reset	Description
511:0	rsvd	R	0x0	Reserved region for future use

Table 87. RESERVED_4 Register

Bits	Field	Access	Reset	Description
511:0	rsvd	R	0x0	Reserved region for future use

Table 88. RESERVED_5 Register

Bits	Field	Access	Reset	Description
511:0	rsvd	R	0x0	Reserved region for future use

Table 89. RESERVED_6 Register

Bits	Field	Access	Reset	Description
511:0	rsvd	R	0x0	Reserved region for future use

Table 90. RESERVED_7 Register

Bits	Field	Access	Reset	Description
511:0	rsvd	R	0x0	Reserved region for future use

Table 91. RESERVED_LAST_256 Register

Bits	Field	Access	Reset	Description
255:0	rsvd	R	0x0	Reserved region for future use

Table 92. RESERVED_LAST_64 Register

Bits	Field	Access	Reset	Description
63:0	rsvd	R	0x0	Reserved region for future use

Table 93. RESERVED_LAST_32 Register

Bits	Field	Access	Reset	Description
31:0	rsvd	R	0x0	Reserved region for future use

NOTE

Register Arrays: This register map contains the following register arrays:

- **RMA_SIP_TOKEN_I[8]:** 8 registers
 - Base Address: 0x0
 - Address Increment: 0x4 per register
- **RMA_CHIPLET_TOKEN_I[8]:** 8 registers
 - Base Address: 0x20

- Address Increment: 0x4 per register
- **SEC_DISABLE_TOKEN_I[8]**: 8 registers
 - Base Address: 0x40
 - Address Increment: 0x4 per register

Table 94. Register Map

Address	Name	Access	Description
0x0 - 0x1C	RMA_SIP_TOKEN_I[8]	RW	Software-writeable RMA token word. The assembled token is hashed and compared against the fused value to authorize lifecycle (LC) state transitions.
0x20 - 0x3C	RMA_CHIPLET_TOKEN_I[8]	RW	Software-writeable RMA token word. The assembled token is hashed and compared against the fused value to authorize lifecycle (LC) state transitions.
0x40 - 0x5C	SEC_DISABLE_TOKEN_I[8]	RW	Software-writeable security-disable token word. The assembled token is hashed and compared against the post-silicon injected value to authorize security_disable.
0x60	TOKEN_EOP	W	Token end-of-packet control. Writing the per-token go bit signals that all token words have been written and triggers a pulse to begin token hashing.
0x64	RMA_SIP_TOKEN_MATCH	R	Token match status. Reports the result of the token hash comparison (match, mismatch, or error).
0x68	RMA_CHIPLET_TOKEN_MATCH	R	Token match status. Reports the result of the token hash comparison (match, mismatch, or error).
0x6C	SEC_DISABLE_TOKEN_MATCH	R	Token match status. Reports the result of the token hash comparison (match, mismatch, or error).

Table 95. RMA_SIP_TOKEN_I[8] Register

Bits	Field	Access	Reset	Description
31:0	token	RW	-	Portion of writeable version of RMA_*_TOKEN, hashed, and compared with fused value to allow LC to change states.

Table 96. RMA_CHIPLET_TOKEN_I[8] Register

Bits	Field	Access	Reset	Description
31:0	token	RW	-	Portion of writeable version of RMA_*_TOKEN, hashed, and compared with fused value to allow LC to change states.

Table 97. SEC_DISABLE_TOKEN_I[8] Register

Bits	Field	Access	Reset	Description
31:0	token	RW	-	Portion of writeable version of SEC_DISABLE_TOKEN, hashed, and compared with the post silicon injected values to allow security_disable.

Table 98. TOKEN_EOP Register

Bits	Field	Access	Reset	Description
31:17	Reserved	-	-	Reserved
16	secure_disable_token_go	W	0x0	Write 1 to indicate that all token data has been written; this generates a pulse to begin token hashing.
15:9	Reserved	-	-	Reserved

Bits	Field	Access	Reset	Description
8	rma_chiplet_token_g o	W	0x0	Write 1 to indicate that all token data has been written; this generates a pulse to begin token hashing.
7:1	Reserved	-	-	Reserved
0	rma_sip_token_go	W	0x0	Write 1 to indicate that all token data has been written; this generates a pulse to begin token hashing.

Table 99. RMA_SIP_TOKEN_MATCH Register

Bits	Field	Access	Reset	Description
31:6	Reserved	-	-	Reserved
5:0	token_match_status	R	0x0	Store the status of the token match. Match = 6'b010101, Mismatch = 6'b101010, Error = 6'b111111

Table 100. RMA_CHIPLET_TOKEN_MATCH Register

Bits	Field	Access	Reset	Description
31:6	Reserved	-	-	Reserved
5:0	token_match_status	R	0x0	Store the status of the token match. Match = 6'b010101, Mismatch = 6'b101010, Error = 6'b111111

Table 101. SEC_DISABLE_TOKEN_MATCH Register

Bits	Field	Access	Reset	Description
31:6	Reserved	-	-	Reserved
5:0	token_match_status	R	0x0	Store the status of the token match. Match = 6'b010101, Mismatch = 6'b101010, Error = 6'b111111

4.5.4. Timers

Table 102. Register Map

Address	Name	Access	Description
0x0	ALERT_TEST	W	Alert Test Register
0x4	WKUP_CTRL	RW	Wakeup Timer Control register. Each write to the register resets the internal prescaler count.
0x8	WKUP_THOLD_HI	RW	Wakeup Timer Threshold Register (bits 63 - 32)
0xC	WKUP_THOLD_LO	RW	Wakeup Timer Threshold Register (bits 31 - 0)
0x10	WKUP_COUNT_HI	RW	Wakeup Timer Count Register (bits 63 - 32)
0x14	WKUP_COUNT_LO	RW	Wakeup Timer Count Register (bits 31 - 0)
0x18	WDOG_REGWEN	RW	Watchdog Timer Write Enable Register
0x1C	WDOG_CTRL	RW	Watchdog Timer Control register
0x20	WDOG_BARK_THOLD	RW	Watchdog Timer Bark Threshold Register
0x24	WDOG_BITE_THOLD	RW	Watchdog Timer Bite Threshold Register
0x28	WDOG_COUNT	RW	Watchdog Timer Count Register
0x2C	INTR_STATE	RW	Interrupt State Register
0x30	INTR_TEST	W	Interrupt Test Register
0x34	WKUP_CAUSE	RW	Wakeup request status

Table 103. ALERT_TEST Register

Bits	Field	Access	Reset	Description
31:1	Reserved	-	-	Reserved
0	fatal_fault	W	0x0	Write 1 to trigger one alert event of this kind.

Table 104. WKUP_CTRL Register

Bits	Field	Access	Reset	Description
31:13	Reserved	-	-	Reserved
12:1	prescaler	RW	0x0	Pre-scaler value for wakeup timer count.
0	enable	RW	0x0	When set to 1, the wakeup timer will count.

Table 105. WKUP_THOLD_HI Register

Bits	Field	Access	Reset	Description
31:0	threshold_hi	RW	0x0	The count at which a wakeup interrupt should be generated, top 32 bits.

Table 106. WKUP_THOLD_LO Register

Bits	Field	Access	Reset	Description
31:0	threshold_lo	RW	0x0	The count at which a wakeup interrupt should be generated, bottom 32 bits.

Table 107. WKUP_COUNT_HI Register

Bits	Field	Access	Reset	Description
31:0	count_hi	RW	0x0	The current wakeup counter value, top 32 bits.

Table 108. WKUP_COUNT_LO Register

Bits	Field	Access	Reset	Description
31:0	count_lo	RW	0x0	The current wakeup counter value, bottom 32 bits.

Table 109. WDOG_REGWEN Register

Bits	Field	Access	Reset	Description
31:1	Reserved	-	-	Reserved
0	regwen	RW	0x1	Once cleared, the watchdog configuration will be locked until the next reset.

Table 110. WDOG_CTRL Register

Bits	Field	Access	Reset	Description
31:2	Reserved	-	-	Reserved
1	pause_in_sleep	RW	0x0	When set to 1, the watchdog timer will not count during sleep.
0	enable	RW	0x0	When set to 1, the watchdog timer will count.

Table 111. WDOG_BARK_THOLD Register

Bits	Field	Access	Reset	Description
31:0	threshold	RW	0x0	The count at which a watchdog bark interrupt should be generated.

Table 112. WDOG_BITE_THOLD Register

Bits	Field	Access	Reset	Description
31:0	threshold	RW	0x0	The count at which a watchdog bite reset should be generated.

Table 113. WDOG_COUNT Register

Bits	Field	Access	Reset	Description
31:0	count	RW	0x0	The current watchdog counter value.

Table 114. INTR_STATE Register

Bits	Field	Access	Reset	Description
31:2	Reserved	-	-	Reserved
1	wdog_timer_bark	RW	0x0	Raised if the watchdog timer has hit the bark threshold.
0	wkup_timer_expired	RW	0x0	Raised if the wakeup timer has hit the specified threshold.

Table 115. INTR_TEST Register

Bits	Field	Access	Reset	Description
31:2	Reserved	-	-	Reserved
1	wdog_timer_bark	W	0x0	Write 1 to force wdog_timer_bark interrupt.
0	wkup_timer_expired	W	0x0	Write 1 to force wkup_timer_expired interrupt.

Table 116. WKUP_CAUSE Register

Bits	Field	Access	Reset	Description
31:1	Reserved	-	-	Reserved
0	cause	RW	0x0	AON timer requested wakeup, write 0 to clear.

4.5.5. Cryptographic Accelerators

Register Arrays: This register map contains the following register arrays:

- **KEY_SHARE0[8]:** 8 registers
 - Base Address: 0x4
 - Address Increment: 0x4 per register
- **KEY_SHARE1[8]:** 8 registers
 - Base Address: 0x24
 - Address Increment: 0x4 per register
- **IV[4]:** 4 registers
 - Base Address: 0x44
 - Address Increment: 0x4 per register
- **DATA_IN[4]:** 4 registers
 - Base Address: 0x54
 - Address Increment: 0x4 per register

NOTE

- **DATA_OUT[4]:** 4 registers
 - Base Address: 0x64
 - Address Increment: 0x4 per register

Table 117. Register Map

Address	Name	Access	Description
0x4 - 0x20	KEY_SHARE0[8]	W	-
0x24 - 0x40	KEY_SHARE1[8]	W	-
0x44 - 0x50	IV[4]	RW	-
0x54 - 0x60	DATA_IN[4]	W	-
0x64 - 0x70	DATA_OUT[4]	R	-
0x74	CTRL_SHADOWED	RW	-
0x78	CTRL_AUX_SHADOWED	RW	-
0x7C	CTRL_AUX_REGWEN	RW	-
0x80	TRIGGER	W	-
0x84	STATUS	R	-
0x88	CTRL_GCM_SHADOWED	RW	-

Table 118. KEY_SHARE0[8] Register

Bits	Field	Access	Reset	Description
31:0	KEY_SHARE0	W	-	Initial Key Registers Share 0. The actual initial key corresponds to Initial Key Registers Share 0 XORed with Initial Key Registers Share 1. Loaded into the internal Full Key register upon starting encryption/decryption of the next block. All key registers (Share 0 and Share 1) must be written at least once when the key is changed, regardless of key length (write random data for unused bits). The order in which the registers are updated does not matter. Can only be updated when the AES unit is idle. If the AES unit is non-idle, writes to these registers are ignored. Upon reset, these registers are cleared with pseudo-random data.

Table 119. KEY_SHARE1[8] Register

Bits	Field	Access	Reset	Description
31:0	KEY_SHARE1	W	-	Initial Key Registers Share 1. The actual initial key corresponds to Initial Key Registers Share 0 XORed with Initial Key Registers Share 1. Loaded into the internal Full Key register upon starting encryption/decryption of the next block. All key registers (Share 0 and Share 1) must be written at least once when the key is changed, regardless of key length (write random data for unused bits). The order in which the registers are updated does not matter. Can only be updated when the AES unit is idle. If the AES unit is non-idle, writes to these registers are ignored. Upon reset, these registers are cleared with pseudo-random data.

Table 120. IV[4] Register

Bits	Field	Access	Reset	Description
31:0	IV	RW	-	Initialization Vector Registers. The initialization vector (IV) or initial counter value must be written to these registers when starting a new message in CBC or CTR mode (see Control Register), respectively. In CBC and CTR modes, the AES unit does not start encryption/decryption with a partially updated IV. Each register has to be written at least once. The order in which the registers are written does not matter. If the AES unit is non-idle, writes to these registers are ignored. Whenever starting a new message, the corresponding IV value must be provided by the processor. Once started, the AES unit automatically updates the contents of these registers. In ECB mode, the IV registers are not used and do not need to be configured. Upon reset, these registers are cleared with pseudo-random data.

Table 121. DATA_IN[4] Register

Bits	Field	Access	Reset	Description
31:0	DATA_IN	W	-	Input Data Registers. If MANUAL_OPERATION=0 (see Control Register), the AES unit automatically starts encryption/decryption after all Input Data registers have been written. Each register has to be written at least once. The order in which the registers are written does not matter. Loaded into the internal State register upon starting encryption/decryption of the next block. After that, the processor can update the Input Data registers (See INPUT_READY field of Status Register). Upon reset, these registers are cleared with pseudo-random data.

Table 122. DATA_OUT[4] Register

Bits	Field	Access	Reset	Description
31:0	DATA_OUT	R	-	Output Data Register. Holds the output data produced by the AES unit during the last encryption/decryption operation. If MANUAL_OPERATION=0 (see Control Register), the AES unit is stalled when the previous output data has not yet been read and is about to be overwritten. Each register has to be read at least once. The order in which the registers are read does not matter. Upon reset, these registers are cleared with pseudo-random data.

Table 123. CTRL_SHADOWED Register

Bits	Field	Access	Reset	Description
31:16	Reserved	-	-	Reserved
15	MANUAL_OPERATION	RW	-	Controls whether the AES unit is operated in normal/automatic mode (0) or fully manual mode (1). In automatic mode (0), the AES unit automatically i) starts to encrypt/decrypt when it receives new input data, and ii) stalls during the last encryption/decryption cycle if the previous output data has not yet been read. This is the most efficient mode to operate in. Note that the corresponding status tracking is automatically cleared upon a write to the Control Register. In manual mode (1), the AES unit i) only starts to encrypt/decrypt after receiving a start trigger (see Trigger Register), and ii) overwrites previous output data irrespective of whether it has been read out or not. This mode is useful if software needs full control over the AES unit.

Bits	Field	Access	Reset	Description
14:12	PRNG_RESEED_RATE	RW	-	3-bit one-hot field to control the reseeding rate of the internal pseudo-random number generator (PRNG) used for masking. Invalid input values, i.e., values with multiple bits set and value 3'b000 are mapped to the highest reseeding rate PER_1 (3'b001).
11	SIDELoad	RW	-	Controls whether the AES unit uses the key provided by the key manager via key sideload interface (1) or the key provided by software via Initial Key Registers KEY_SHARE1_0 - KEY_SHARE1_7 (0).
10:8	KEY_LEN	RW	-	3-bit one-hot field to select AES key length. Invalid input values, i.e., values with multiple bits set, value 3'b000, and value 3'b010 in case 192-bit keys are not supported (because disabled at compile time) are mapped to AES_256 (3'b100).
7:2	MODE	RW	-	6-bit one-hot field to select AES block cipher mode. Invalid input values, i.e., values with multiple bits set and value 6'b00_0000, are mapped to AES_NONE (6'b11_1111).
1:0	OPERATION	RW	-	2-bit one-hot field to select the operation of AES unit. Invalid input values, i.e., values with multiple bits set and value 2'b00, are mapped to AES_ENC (2'b01).

Table 124. CTRL_AUX_SHADOWED Register

Bits	Field	Access	Reset	Description
31:2	Reserved	-	-	Reserved
1	FORCE_MASKS	RW	-	Allow the internal masking PRNG to advance (0) or force its internal state (1) leading to constant masks. Setting all masks to constant value can be useful when performing SCA. To completely disable the masking, the second key share (KEY_SHARE1_0 - KEY_SHARE1_7) must be zero as well. In addition, a special seed needs to be loaded into the masking PRNG using the EDN interface. Only applicable if both the Masking parameter and the SecAllowForcingMasks parameter are set to one.
0	KEY_TOUCH_FORCES_RESEED	RW	-	Controls whether providing a new key triggers the reseeding of internal pseudo-random number generators used for clearing and masking (1) or not (0).

Table 125. CTRL_AUX_REGWEN Register

Bits	Field	Access	Reset	Description
31:1	Reserved	-	-	Reserved
0	CTRL_AUX_REGWEN	RW	0x1	Auxiliary Control Register configuration enable bit. If this is cleared to 0, the Auxiliary Control Register cannot be written anymore.

Table 126. TRIGGER Register

Bits	Field	Access	Reset	Description
31:4	Reserved	-	-	Reserved
3	PRNG_RESEED	W	-	Keep continuing with the current states of the internal pseudo-random number generators used for register clearing and masking (0) or perform a reseed of the internal states from the connected entropy source (1). If the KEY_TOUCH_FORCES_RESEED bit in the Auxiliary Control Register is set to one, this trigger will automatically get set after providing a new initial key.

Bits	Field	Access	Reset	Description
2	DATA_OUT_CLEAR	W	-	Keep current values in Output Data registers (0) or clear those registers with pseudo-random data (1).
1	KEY_IV_DATA_IN_CLEAR	W	-	Keep current values in Initial Key, internal Full Key and Decryption Key registers, IV registers and Input Data registers (0) or clear all those registers with pseudo-random data (1).
0	START	W	-	Keep AES unit paused (0) or trigger the encryption/decryption of one data block (1). This trigger is cleared to 0 if MANUAL_OPERATION=0 or if MODE=AES_NONE (see Control Register).

Table 127. STATUS Register

Bits	Field	Access	Reset	Description
31:7	Reserved	-	-	Reserved
6	ALERT_FATAL_FAULT	R	-	No fatal fault has occurred inside the AES unit (0). A fatal fault has occurred and the AES unit needs to be reset (1). Examples for fatal faults include i) storage errors in the Control Register, ii) if any internal FSM enters an invalid state, iii) if any sparsely encoded signal takes on an invalid value, iv) errors in the internal round counter, v) escalations triggered by the life cycle controller, and vi) fatal integrity failures on the TL-UL bus.
5	ALERT_RECOV_CTRL_UPDATE_ERR	R	-	An update error has not occurred (0) or has occurred (1) in the shadowed Control Register. AES operation needs to be restarted by re-writing the Control Register.
4	INPUT_READY	R	-	The AES unit is ready (1) or not ready (0) to receive new data input via the DATA_IN registers. If the present values in the DATA_IN registers have not yet been loaded into the module this flag is 0 (not ready).
3	OUTPUT_VALID	R	-	The AES unit has no valid output (0) or has valid output data (1).
2	OUTPUT_LOST	R	-	All previous output data has been fully read by the processor (0) or at least one previous output data block has been lost (1). It has been overwritten by the AES unit before the processor could fully read it. Once set to 1, this flag remains set until AES operation is restarted by re-writing the Control Register. The primary use of this flag is for design verification. This flag is not meaningful if MANUAL_OPERATION=0 (see Control Register).
1	STALL	R	-	The AES unit is not stalled (0) or stalled (1) because there is previous output data that must be read by the processor before the AES unit can overwrite this data. This flag is not meaningful if MANUAL_OPERATION=1 (see Control Register).
0	IDLE	R	-	The AES unit is idle (1) or busy (0). This flag is 0 if one of the following operations is currently running: i) encryption/decryption, ii) register clearing or iii) PRNG reseeding. This flag is also 0 if an encryption/decryption is running but the AES unit is stalled.

Table 128. CTRL_GCM_SHADOWED Register

Bits	Field	Access	Reset	Description
31:11	Reserved	-	-	Reserved
10:6	NUM_VALID_BYTES	RW	-	Number of valid bytes of the current input block. Only the last block in the GCM_AAD and GCM_TEXT phases are expected to have not all bytes marked as valid. For all other blocks, the number of valid bytes should be set to 16. Invalid input values, i.e., the value 5'b0_0000, and all other values different from 5'b1_0000 in case GCM is not supported (because disabled at compile time) are mapped to 5'b1_0000.

Bits	Field	Access	Reset	Description
5:0	PHASE	RW	-	6-bit one-hot field to select the phase of the Galois/Counter Mode (GCM) of operation. Invalid input values, i.e., values with multiple bits set and value 6'b00_0000, are mapped to GCM_INIT (6'b00_0001). In case support for GCM has been disabled at compile time, this field is not writable and always reads as GCM_INIT (6'b00_0001).

NOTE

Register Arrays: This register map contains the following register arrays:

- **KEY[32]:** 32 registers
 - Base Address: 0x24
 - Address Increment: 0x4 per register
- **DIGEST[16]:** 16 registers
 - Base Address: 0xA4
 - Address Increment: 0x4 per register

Table 129. Register Map

Address	Name	Access	Description
0x0	INTR_STATE	RW	-
0x4	INTR_ENABLE	RW	-
0x8	INTR_TEST	W	-
0xC	ALERT_TEST	W	-
0x10	CFG	RW	-
0x14	CMD	W	-
0x18	STATUS	R	-
0x1C	ERR_CODE	R	-
0x20	WIPE_SECRET	W	-
0x24 - 0xA0	KEY[32]	W	-
0xA4 - 0xE0	DIGEST[16]	RW	-
0xE4	MSG_LENGTH_LOWER	RW	-
0xE8	MSG_LENGTH_UPPER	RW	-

Table 130. INTR_STATE Register

Bits	Field	Access	Reset	Description
31:3	Reserved	-	-	Reserved
2	hmac_err	RW	0x0	HMAC error has occurred. ERR_CODE register shows which error occurred.

Bits	Field	Access	Reset	Description
1	<code>fifo_empty</code>	R	0x0	The message FIFO is empty. This interrupt is raised only if the message FIFO is actually writable by software, i.e., if all of the following conditions are met: i) The HMAC block is not running in HMAC mode and performing the second round of computing the final hash of the outer key as well as the result of the first round using the inner key. ii) Software has not yet written the Process or Stop command to finish the hashing operation. For the interrupt to be raised, the message FIFO must also have been full previously. Otherwise, the hardware empties the FIFO faster than software can fill it and there is no point in interrupting the software to inform it about the message FIFO being empty.
0	<code>hmac_done</code>	RW	0x0	HMAC/SHA-2 has completed.

Table 131. INTR_ENABLE Register

Bits	Field	Access	Reset	Description
31:3	<code>Reserved</code>	-	-	Reserved
2	<code>hmac_err</code>	RW	0x0	Enable interrupt when !!INTR_STATE.hmac_err is set.
1	<code>fifo_empty</code>	RW	0x0	Enable interrupt when !!INTR_STATE.fifo_empty is set.
0	<code>hmac_done</code>	RW	0x0	Enable interrupt when !!INTR_STATE.hmac_done is set.

Table 132. INTR_TEST Register

Bits	Field	Access	Reset	Description
31:3	<code>Reserved</code>	-	-	Reserved
2	<code>hmac_err</code>	W	0x0	Write 1 to force !!INTR_STATE.hmac_err to 1.
1	<code>fifo_empty</code>	W	0x0	Write 1 to force !!INTR_STATE.fifo_empty to 1.
0	<code>hmac_done</code>	W	0x0	Write 1 to force !!INTR_STATE.hmac_done to 1.

Table 133. ALERT_TEST Register

Bits	Field	Access	Reset	Description
31:1	<code>Reserved</code>	-	-	Reserved
0	<code>fatal_fault</code>	W	0x0	Write 1 to trigger one alert event of this kind.

Table 134. CFG Register

Bits	Field	Access	Reset	Description
31:15	<code>Reserved</code>	-	-	Reserved

Bits	Field	Access	Reset	Description
14:9	key_length	RW	0x20	Key length configuration. This is a 6-bit one-hot encoded field to configure the key length for HMAC. The HMAC can be programmed with the following key lengths: 128-bit, 256-bit, 384-bit, 512-bit and 1024-bit. But the HMAC supports any arbitrary key length: the software should configure the HMAC with the next largest supported key length and concatenate zeros to reach the programmed key length. The position of these zeros depends on the endianness, thus on the programmed [CFG.key_swap](registers.md#cfg—key_swap). For example, for an 80-bit key, HMAC should be configured with an 128-bit key length, fed with the 80-bit key and with 48 zero-bits. Note that the key length cannot be greater than the block size: up to 1024-bit for SHA-2 384/512 and up to 512-bit for SHA-2 256. The value of this register is irrelevant when only SHA-2 (not keyed HMAC) is configured. However, for HMAC mode (hmac_en == 1), when HMAC is triggered to start while !!KEY_LENGTH holds Key_None or !!KEY_LENGTH holds Key_1024 for !!DIGEST_SIZE = SHA2_256, starting is blocked and an error is signalled to SW.
8:5	digest_size	RW	0x8	Digest size configuration. This is a 4-bit one-hot encoded field to select digest size for either HMAC or SHA-2. Invalid/unsupported values, i.e., values that don't correspond to SHA2_256, SHA2_384, or SHA2_512, are mapped to SHA2_None.
4	key_swap	RW	0x0	Key register byte swap (deprecated: KEY path is for DV only; production uses key manager sideload). If 1 the endianness of each KEY_* register is swapped. Default value (value 0) is big endian representation of the KEY_* CSRs.
3	digest_swap	RW	0x0	Digest register byte swap. If 1 the value in each digest output register is converted to big-endian byte order. This setting does not affect the order of the digest output registers, !!DIGEST_0 still contains the first 4 bytes of the digest.
2	endian_swap	RW	0x0	Endian swap. If 0, each value will be added to the message in little-endian byte order. The value is written to MSG_FIFO same to the SW writes. If 1, then each individual multi-byte value, regardless of its alignment, written to !!MSG_FIFO will be added to the message in big-endian byte order. A message written to !!MSG_FIFO one byte at a time will not be affected by this setting. From a hardware perspective byte swaps are performed on a TL-UL word granularity.
1	sha_en	RW	-	SHA-2 enable. If 0, the SHA engine will not initiate compression, this is used to stop operation of the SHA-2 engine until configuration has been done. When the SHA-2 engine is disabled the digest is cleared.
0	hmac_en	RW	-	HMAC datapath enable. If this bit is 1, HMAC operates when hash_start toggles.

Table 135. CMD Register

Bits	Field	Access	Reset	Description
31:4	Reserved	-	-	Reserved
3	hash_continue	W	-	When 1 is written to this field, SHA-2 or HMAC will continue hashing based on the current hash in the digest registers and the message length, which both have to be restored to switch context.
2	hash_stop	W	-	When 1 is written to this field, SHA-2 or HMAC will afterwards set the hmac_done interrupt as soon as the current block has been hashed. The hash can then be read from the registers !!DIGEST_0 to !!DIGEST_15. Together with the message length in !!MSG_LENGTH_LOWER and !!MSG_LENGTH_UPPER, this forms the information that has to be saved before switching context.
1	hash_process	W	-	If 1 is written to this field, SHA-2 or HMAC calculates the digest or signing based on currently received message.
0	hash_start	W	-	If 1 is written into this field, SHA-2 or HMAC begins its operation. CPU must configure relative information first, such as the digest size, secret key and the key length.

Table 136. STATUS Register

Bits	Field	Access	Reset	Description
31:10	Reserved	-	-	Reserved
9:4	fifo_depth	R	-	FIFO entry count.
3	Reserved	-	-	Reserved
2	fifo_full	R	-	FIFO full. Data written to the FIFO whilst it is full will cause back-pressure on the interconnect
1	fifo_empty	R	0x1	FIFO empty
0	hmac_idle	R	0x1	HMAC idle status. When IDLE, the DIGEST and the MSG_LENGTH_LOWER/MSG_LENGTH_UPPER can be written to from SW which enables restoring context (to support context switching).

Table 137. ERR_CODE Register

Bits	Field	Access	Reset	Description
31:0	err_code	R	0x0	If an error interrupt occurs, this register has information of error cause. Please take a look at `hw/ip/prim/rtl/prim_sha2_pkg.sv:err_code_e` enum type.

Table 138. WIPE_SECRET Register

Bits	Field	Access	Reset	Description
31:0	secret	W	-	Secret value

Table 139. KEY[32] Register

Bits	Field	Access	Reset	Description
31:0	key_0	W	-	32-bit chunk of 1024-bit secret key

Table 140. DIGEST[16] Register

Bits	Field	Access	Reset	Description
31:0	digest_0	RW	-	32-bit chunk of 512-bit digest

Table 141. MSG_LENGTH_LOWER Register

Bits	Field	Access	Reset	Description
31:0	V	RW	-	Message Length [31:0]

Table 142. MSG_LENGTH_UPPER Register

Bits	Field	Access	Reset	Description
31:0	V	RW	-	Message Length [63:32]

NOTE

Register Arrays: This register map contains the following register arrays:

- **KEY_SHARE0[16]:** 16 registers
 - Base Address: 0x30
 - Address Increment: 0x4 per register
- **KEY_SHARE1[16]:** 16 registers
 - Base Address: 0x70
 - Address Increment: 0x4 per register
- **PREFIX[11]:** 11 registers
 - Base Address: 0xB4
 - Address Increment: 0x4 per register

Table 143. Register Map

Address	Name	Access	Description
0x0	INTR_STATE	RW	-
0x4	INTR_ENABLE	RW	-
0x8	INTR_TEST	W	-
0xC	ALERT_TEST	W	-
0x10	CFG_REGWEN	R	-
0x14	CFG_SHADOWED	RW	-
0x18	CMD	W	-
0x1C	STATUS	R	-
0x20	ENTROPY_PERIOD	RW	-
0x24	ENTROPY_REFRESH_HASH_CNT	R	-
0x28	ENTROPY_REFRESH_THRESHOLD_SHADOWED	RW	-
0x2C	ENTROPY_SEED	W	-
0x30 - 0x6C	KEY_SHARE0[16]	W	-
0x70 - 0xAC	KEY_SHARE1[16]	W	-
0xB0	KEY_LEN	W	-
0xB4 - 0xDC	PREFIX[11]	RW	-
0xE0	ERR_CODE	R	-

Table 144. INTR_STATE Register

Bits	Field	Access	Reset	Description
31:3	Reserved	-	-	Reserved
2	kmac_err	RW	0x0	KMAC/SHA3 error occurred. ERR_CODE register shows the details
1	fifo_empty	R	0x0	The message FIFO is empty. This interrupt is raised only if the message FIFO is actually writable by software, i.e., if all of the following conditions are met: i) The KMAC block is not exercised by a hardware application interface. ii) The SHA3 block is in the Absorb state. iii) Software has not yet written the Process command to finish the absorption process. For the interrupt to be raised, the message FIFO must also have been full previously. Otherwise, the hardware empties the FIFO faster than software can fill it and there is no point in interrupting the software to inform it about the message FIFO being empty.
0	kmac_done	RW	0x0	KMAC/SHA3 absorbing has been completed

Table 145. INTR_ENABLE Register

Bits	Field	Access	Reset	Description
31:3	Reserved	-	-	Reserved
2	kmac_err	RW	0x0	Enable interrupt when !!INTR_STATE.kmac_err is set.
1	fifo_empty	RW	0x0	Enable interrupt when !!INTR_STATE.fifo_empty is set.
0	kmac_done	RW	0x0	Enable interrupt when !!INTR_STATE.kmac_done is set.

Table 146. INTR_TEST Register

Bits	Field	Access	Reset	Description
31:3	Reserved	-	-	Reserved
2	kmac_err	W	0x0	Write 1 to force !!INTR_STATE.kmac_err to 1.
1	fifo_empty	W	0x0	Write 1 to force !!INTR_STATE.fifo_empty to 1.
0	kmac_done	W	0x0	Write 1 to force !!INTR_STATE.kmac_done to 1.

Table 147. ALERT_TEST Register

Bits	Field	Access	Reset	Description
31:2	Reserved	-	-	Reserved
1	fatal_fault_err	W	0x0	Write 1 to trigger one alert event of this kind.
0	recov_operation_err	W	0x0	Write 1 to trigger one alert event of this kind.

Table 148. CFG_REGWEN Register

Bits	Field	Access	Reset	Description
31:1	Reserved	-	-	Reserved
0	en	R	0x1	Configuration enable.

Table 149. CFG_SHADOWED Register

Bits	Field	Access	Reset	Description
31:27	Reserved	-	-	Reserved

Bits	Field	Access	Reset	Description
26	en_unsupported_mode_strength	RW	0x0	Enable Unsupported Mode and Strength configs. SW may set this field for KMAC to move forward with unsupported Keccak Mode and Strength configurations, such as cSHAKE512. If not set, KMAC won't propagate the SW command (CmdStart) to the rest of the blocks (AppIntf, KMAC Core, SHA3).
25	Reserved	-	-	Reserved
24	entropy_ready	RW	0x0	Entropy Ready status. Software sets this field to allow the entropy generator in KMAC to fetch the entropy and run.
23:21	Reserved	-	-	Reserved
20	msg_mask	RW	0x0	Message Masking with PRNG. If 1, KMAC applies PRNG to the input messages to the Keccak module when KMAC mode is on.
19	entropy_fast_processes	RW	0x0	Entropy Fast process mode. If 1, entropy logic uses garbage data while not processing the KMAC key block. It will re-use previous entropy value and will not expand the entropy when it is consumed. Only it refreshes the entropy while processing the secret key block. This process should not be used if SCA resistance is required because it may cause side channel leakage.
18	Reserved	-	-	Reserved
17:16	entropy_mode	RW	0x0	Entropy Mode Using this field, software can configure mode of operation of the internal pseudo-random number generator (PRNG). Note that once software has configured a value different from <code>idle_mode</code> after reset, and set the <code>!!CFG_SHADOWED.entropy_ready</code> bit afterwards, the PRNG configuration cannot be changed anymore (unless an EDN request timeout occurs when running in EDN mode). Further writes to this field will change the value received upon reading the register, but they will be ignored by the internal hardware.
15:13	Reserved	-	-	Reserved
12	sideload	RW	0x1	Sideloading Key. Deprecated: defaults to 1 (sideload). Set to 0 for DV testing with SW key path only. If 1, KMAC uses KeyMgr sideloaded key for SW initiated KMAC operation. KMAC uses the sideloaded key regardless of this configuration when KeyMgr initiates the KMAC operation for Key Derivation Function (KDF).
11:10	Reserved	-	-	Reserved
9	state_endianness	RW	0x0	State Endianness. If 1 then each individual word in the <code>!!STATE</code> output register is converted to big-endian byte order. The order of the words in relation to one another is not changed. This setting does not affect how the state is interpreted during computation.

Bits	Field	Access	Reset	Description
8	msg_endianness	RW	0x0	Message Endianness. If 1 then each individual multi-byte value, regardless of its alignment, written to !!MSG_FIFO will be added to the message in big-endian byte order. If 0, each value will be added to the message in little-endian byte order. A message written to !!MSG_FIFO one byte at a time will not be affected by this setting. From a hardware perspective byte swaps are performed on a TL-UL word granularity.
7:6	Reserved	-	-	Reserved
5:4	mode	RW	0x0	Keccak hashing mode. This module supports SHA3 main hashing algorithm and the part of its derived functions, SHAKE and cSHAKE with limitations. This field is to select the mode.
3:1	kstrength	RW	0x0	Hashing Strength 3 bit field to select the security strength of SHA3 hashing engine. If mode field is set to SHAKE or cSHAKE, only 128 and 256 strength can be selected. Other value will result error when hashing starts.
0	kmac_en	RW	0x0	KMAC datapath enable. If this bit is 1, the incoming message is processed in KMAC with the secret key.

Table 150. CMD Register

Bits	Field	Access	Reset	Description
31:11	Reserved	-	-	Reserved
10	err_processed	W	-	When error occurs and one of the state machine stays at Error handling state, SW may process the error based on ERR_CODE, then let FSM back to the reset state
9	hash_cnt_clr	W	-	Software can set this bit to 1 to manually clear !!ENTROPY_REFRESH_HASH_CNT.
8	entropy_req	W	-	Software can set this bit to 1 to manually trigger the reseeding of the internal PRNG when running in EDN mode. This will also clear !!ENTROPY_REFRESH_HASH_CNT. Note that the hardware may miss the trigger pulse if the module is not idle, or if the module is currently performing a reseed operation.
7:6	Reserved	-	-	Reserved
5:0	cmd	W	-	Issue a command to the KMAC/SHA3 IP. The command is sparse encoded. To prevent sw from writing multiple commands at once, the field is defined as enum.

Table 151. STATUS Register

Bits	Field	Access	Reset	Description
31:18	Reserved	-	-	Reserved
17	ALERT_RECOV_CTRL_UP DATE_ERR	R	0x0	An update error has not occurred (0) or has occurred (1) in the shadowed Control Register. KMAC operation needs to be restarted by re-writing the Control Register.

Bits	Field	Access	Reset	Description
16	ALERT_FATAL_FAULT	R	0x0	No fatal fault has occurred inside the KMAC unit (0). A fatal fault has occurred and the KMAC unit needs to be reset (1), Examples for such faults include i) TL-UL bus integrity fault ii) storage errors in the shadow registers iii) errors in the message, round, or key counter iv) any internal FSM entering an invalid state v) an error in the redundant lfsr
15	fifo_full	R	-	Message FIFO Full indicator
14	fifo_empty	R	0x1	Message FIFO Empty indicator. The FIFO's Pass parameter is set to 10b 1. So, by default, if the SHA engine is ready, the write data to FIFO just passes through. In this case, fifo_depth remains 0. fifo_empty, however, lowers the value to 0 for a cycle, then goes back to the empty state, 1. See the "Message FIFO" section in the spec for the reason.
13	Reserved	-	-	Reserved
12:8	fifo_depth	R	-	Count of occupied entries in the message FIFO.
7:3	Reserved	-	-	Reserved
2	sha3_squeeze	R	-	If 1, SHA3 completes sponge absorbing stage. In this stage, SW can manually run the hashing engine.
1	sha3_absorb	R	-	If 1, SHA3 is receiving message stream and processing it
0	sha3_idle	R	0x1	If 1, SHA3 hashing engine is in idle state.

Table 152. ENTROPY_PERIOD Register

Bits	Field	Access	Reset	Description
31:16	wait_timer	RW	0x0	EDN request wait timer. The entropy module in KMAC waits up to this field in the timer pulse after it sends request to EDN module. If the timer expires, the entropy module moves to an error state and notifies to the system. If there is a pending EDN request during wait timer update, then this update is delayed until the EDN request is complete. If 0, the entropy module waits the EDN response always. If EDN does not respond in this configuration, the software shall reset the IP.
15:10	Reserved	-	-	Reserved
9:0	prescaler	RW	0x0	EDN Wait timer prescaler. EDN Wait timer has 16 bit value. The timer value is increased when the timer pulse is generated. Timer pulse is raises when the number of the clock cycles hit this prescaler value. The exact period of the timer pulse is unknown as the KMAC input clock may contain jitters.

Table 153. ENTROPY_REFRESH_HASH_CNT Register

Bits	Field	Access	Reset	Description
31:10	Reserved	-	-	Reserved
9:0	hash_cnt	R	0x0	Hash (KMAC) counter

Table 154. ENTROPY_REFRESH_THRESHOLD_SHADOWED Register

Bits	Field	Access	Reset	Description
31:10	Reserved	-	-	Reserved
9:0	threshold	RW	0x0	Hash Threshold

Table 155. ENTROPY_SEED Register

Bits	Field	Access	Reset	Description
31:0	seed	W	-	32-bit chunk of the entropy generator seed

Table 156. KEY_SHARE0[16] Register

Bits	Field	Access	Reset	Description
31:0	key_0	W	-	32-bit chunk of up-to 512-bit Secret Key

Table 157. KEY_SHARE1[16] Register

Bits	Field	Access	Reset	Description
31:0	key_0	W	-	32-bit chunk of up-to 512-bit Secret Key

Table 158. KEY_LEN Register

Bits	Field	Access	Reset	Description
31:3	Reserved	-	-	Reserved
2:0	len	W	0x0	Key length choice

Table 159. PREFIX[11] Register

Bits	Field	Access	Reset	Description
31:0	prefix_0	RW	0x0	32-bit chunk of Encoded NS Prefix

Table 160. ERR_CODE Register

Bits	Field	Access	Reset	Description
31:0	err_code	R	0x0	If the <code>kmac_err</code> interrupt occurs, this register has information on the error cause. Bits 31:24 contain the error code (please refer to <code>err_code_e</code> in <code>hw/ip/kmac/rtl/kmac_pkg.sv</code>) for the encoding, and bits 23:0 contain additional debug information. This register does not get cleared when the <code>kmac_err</code> interrupt state gets cleared.

Table 161. Register Map

Address	Name	Access	Description
0x0	INTR_STATE	RW	-
0x4	INTR_ENABLE	RW	-
0x8	INTR_TEST	W	-
0xC	ALERT_TEST	W	-
0x10	CMD	W	-
0x14	CTRL	RW	-
0x18	STATUS	R	-
0x1C	ERR_BITS	RW	-

Address	Name	Access	Description
0x20	FATAL_ALERT_CAUSE	R	-
0x24	INSN_CNT	RW	-
0x28	LOAD_CHECKSUM	RW	-

Table 162. INTR_STATE Register

Bits	Field	Access	Reset	Description
31:1	Reserved	-	-	Reserved
0	done	RW	0x0	OTBN has completed the operation.

Table 163. INTR_ENABLE Register

Bits	Field	Access	Reset	Description
31:1	Reserved	-	-	Reserved
0	done	RW	0x0	Enable interrupt when !!INTR_STATE.done is set.

Table 164. INTR_TEST Register

Bits	Field	Access	Reset	Description
31:1	Reserved	-	-	Reserved
0	done	W	0x0	Write 1 to force !!INTR_STATE.done to 1.

Table 165. ALERT_TEST Register

Bits	Field	Access	Reset	Description
31:2	Reserved	-	-	Reserved
1	recov	W	0x0	Write 1 to trigger one alert event of this kind.
0	fatal	W	0x0	Write 1 to trigger one alert event of this kind.

Table 166. CMD Register

Bits	Field	Access	Reset	Description												
31:8	Reserved	-	-	Reserved												
7:0	cmd	W	0x0	The operation to perform. <table><tr><th>Value</th><th>Name</th><th>Description</th></tr><tr><td>0xd8</td><td>EXECUTE</td><td>Starts the execution of the program stored in the instruction memory, starting at address zero.</td></tr><tr><td>0xc3</td><td>SEC_WIPE_DMEM</td><td>Securely removes all contents from the data memory.</td></tr><tr><td>0x1e</td><td>SEC_WIPE_IMEM</td><td>Securely removes all contents from the instruction memory.</td></tr></table>	Value	Name	Description	0xd8	EXECUTE	Starts the execution of the program stored in the instruction memory, starting at address zero.	0xc3	SEC_WIPE_DMEM	Securely removes all contents from the data memory.	0x1e	SEC_WIPE_IMEM	Securely removes all contents from the instruction memory.
Value	Name	Description														
0xd8	EXECUTE	Starts the execution of the program stored in the instruction memory, starting at address zero.														
0xc3	SEC_WIPE_DMEM	Securely removes all contents from the data memory.														
0x1e	SEC_WIPE_IMEM	Securely removes all contents from the instruction memory.														

Table 167. CTRL Register

Bits	Field	Access	Reset	Description
31:1	Reserved	-	-	Reserved
0	software_errs_fatal	RW	0x0	Controls the reaction to software errors. When set software errors produce fatal errors, rather than recoverable errors. Writes are ignored if OTBN is not idle.

Table 168. STATUS Register

Bits	Field	Access	Reset	Description																					
31:8	Reserved	-	-	Reserved																					
7:0	status	R	0x4	Indicates the current operational state OTBN is in. All BUSY values represent an operation started by a write to the !!CMD register. <table><thead><tr><th>Value</th><th>Name</th><th>Description</th></tr></thead><tbody><tr><td>0x00</td><td>IDLE</td><td>OTBN is idle: it is not performing any action.</td></tr><tr><td>0x01</td><td>BUSY_EXECUTE</td><td>OTBN is busy executing software.</td></tr><tr><td>0x02</td><td>BUSY_SEC_WIPE_DMEN</td><td>OTBN is busy securely wiping the data memory.</td></tr><tr><td>0x03</td><td>BUSY_SEC_WIPE_IMEM</td><td>OTBN is busy securely wiping the instruction memory.</td></tr><tr><td>0x04</td><td>BUSY_SEC_WIPE_INT</td><td>OTBN is busy securely wiping the internal state.</td></tr><tr><td>0xFF</td><td>LOCKED</td><td>OTBN is locked as reaction to a fatal error, and must be reset to unlock it again. See also the section "Reaction to Fatal Errors".</td></tr></tbody></table>	Value	Name	Description	0x00	IDLE	OTBN is idle: it is not performing any action.	0x01	BUSY_EXECUTE	OTBN is busy executing software.	0x02	BUSY_SEC_WIPE_DMEN	OTBN is busy securely wiping the data memory.	0x03	BUSY_SEC_WIPE_IMEM	OTBN is busy securely wiping the instruction memory.	0x04	BUSY_SEC_WIPE_INT	OTBN is busy securely wiping the internal state.	0xFF	LOCKED	OTBN is locked as reaction to a fatal error, and must be reset to unlock it again. See also the section "Reaction to Fatal Errors".
Value	Name	Description																							
0x00	IDLE	OTBN is idle: it is not performing any action.																							
0x01	BUSY_EXECUTE	OTBN is busy executing software.																							
0x02	BUSY_SEC_WIPE_DMEN	OTBN is busy securely wiping the data memory.																							
0x03	BUSY_SEC_WIPE_IMEM	OTBN is busy securely wiping the instruction memory.																							
0x04	BUSY_SEC_WIPE_INT	OTBN is busy securely wiping the internal state.																							
0xFF	LOCKED	OTBN is locked as reaction to a fatal error, and must be reset to unlock it again. See also the section "Reaction to Fatal Errors".																							

Table 169. ERR_BITS Register

Bits	Field	Access	Reset	Description
31:24	Reserved	-	-	Reserved
23	fatal_software	RW	0x0	A FATAL_SOFTWARE error was observed.
22	lifecycle_escalation	RW	0x0	A LIFECYCLE_ESCALATION error was observed.
21	illegal_bus_access	RW	0x0	An ILLEGAL_BUS_ACCESS error was observed.
20	bad_internal_state	RW	0x0	A BAD_INTERNAL_STATE error was observed.
19	bus_intg_violation	RW	0x0	A BUS_INTG_VIOLATION error was observed.
18	reg_intg_violation	RW	0x0	A REG_INTG_VIOLATION error was observed.
17	dmem_intg_violation	RW	0x0	A DMEM_INTG_VIOLATION error was observed.
16	imem_intg_violation	RW	0x0	A IMEM_INTG_VIOLATION error was observed.
15:8	Reserved	-	-	Reserved
7	rnd_fips_chk_fail	RW	0x0	An RND_FIPS_CHK_FAIL error was observed.
6	rnd_rep_chk_fail	RW	0x0	An RND_REP_CHK_FAIL error was observed.
5	key_invalid	RW	0x0	A KEY_INVALID error was observed.
4	loop	RW	0x0	A LOOP error was observed.
3	illegal_insn	RW	0x0	An ILLEGAL_INSN error was observed.
2	call_stack	RW	0x0	A CALL_STACK error was observed.
1	bad_insn_addr	RW	0x0	A BAD_INSN_ADDR error was observed.
0	bad_data_addr	RW	0x0	A BAD_DATA_ADDR error was observed.

Table 170. FATAL_ALERT_CAUSE Register

Bits	Field	Access	Reset	Description
31:8	Reserved	-	-	Reserved
7	fatal_software	R	0x0	A FATAL_SOFTWARE error was observed.
6	lifecycle_escalation	R	0x0	A LIFECYCLE_ESCALATION error was observed.
5	illegal_bus_access	R	0x0	A ILLEGAL_BUS_ACCESS error was observed.

Bits	Field	Access	Reset	Description
4	<code>bad_internal_state</code>	R	0x0	A <code>BAD_INTERNAL_STATE</code> error was observed.
3	<code>bus_intg_violation</code>	R	0x0	A <code>BUS_INTG_VIOLATION</code> error was observed.
2	<code>reg_intg_violation</code>	R	0x0	A <code>REG_INTG_VIOLATION</code> error was observed.
1	<code>dmem_intg_violation</code>	R	0x0	A <code>DMEM_INTG_VIOLATION</code> error was observed.
0	<code>imem_intg_violation</code>	R	0x0	A <code>IMEM_INTG_VIOLATION</code> error was observed.

Table 171. *INSN_CNT Register*

Bits	Field	Access	Reset	Description
31:0	<code>insn_cnt</code>	RW	0x0	The number of executed instructions.

Table 172. *LOAD_CHECKSUM Register*

Bits	Field	Access	Reset	Description
31:0	<code>checksum</code>	RW	0x0	Checksum accumulator

4.6. CPU Subsystem

SEP uses the 32-bit RISC-V [VeeR EL2 CPU](#) as its host. The core has been silicon proven by Western Digital and others. The [EL2 Programmer's Reference Manual](#) provides the details required for programming the CPU.

4.6.1. CPU Feature Summary

- RV32IMC including bit manipulation extensions Zb(abcsefbr)
- 4-stage pipeline
- DCLS (Dual Core Lock Step) option required for automotive applications for FUSA compliance
 - Cores are replicated, but ICCM and DCCM are not
 - Configurable, constant delay between the two cores
- Closely coupled instruction and data memory
 - Parity and ECC (32+7)
- RISC-V PMP (Physical Memory Protection) - Privileged Architecture standard
 - 32-bit address space divided into 16 regions of 256 MB fixed size
 - Each region has execute, write and read permissions controlled by the Region Access Control Register (mrac)
 - Precisely trapped exception on usage violation
- PIC (Programmable Interrupt Controller) is integrated supporting up to 255 external interrupts
- Two independent or linked internal timers with standard RV *mip* interrupts
- Debug unit with JTAG interface and internal AXI4 Debug Master
- Hardware Secure Boot Unlock

4.6.2. Configuration Straps

There shall be a minimum of 8 static configuration straps (preferably 16) which are associated with the GPIO interface

pins. Note that the GPIO interface is shared with the SMC and resides in the SMC complex. GPIO pin states are sampled upon power up and state is stored in memory mapped registers associated with the GPIO interface. In a system context, the initial GPIO states can be driven either by static high-impedance pull-up or pull-down resistors or dynamically driven by the board controller hardware. The configuration values are read by the BL0 ROM code during the boot process to determine startup execution options. Examples would be configuring the device as a primary versus secondary chiplet and recovery mode versus normal mode.

4.6.3. Reset Vector

The SEP CPU reset vector is held in a test data register (TDR) in the EL2 debug TAP, selected by JTAG instruction opcode 0x18. The TDR resets to the Boot ROM base address (`SEP_BOOT_ROM_MEM_BASE_ADDR`) on JTAG reset (TRST assertion or the Test-Logic-Reset TAP state), so the CPU executes the BL0 Root-of-Trust code after any cold boot.

A debugger may redirect the first instruction fetch by shifting an alternate address into the TDR through JTAG while the SEP CPU is held in reset. The programmed value persists until the TAP is reset. This supports bring-up and failure analysis flows that execute from a memory other than the Boot ROM.

4.6.4. Boot ROM

The Boot ROM holds the BL0 immutable Root-of-Trust code which is the first code running on the SEP host upon power on. It has the following features:

- 64 kB capacity
- SECDED Error Correction - single error detect double error correct
 - 32-bit data + 7-bit ECC tag

There is no ROM patching facility in the first generation implementation. This feature is planned for the second generation design and will use OTP as the backup storage.

4.6.5. BL0 Secure Boot and BL1 Execution Unlock

The SEP CPU Boot ROM firmware AKA BL0 is responsible for bringing the SEP system into a runnable state after power on reset (PoR) and running the first stage of external firmware AKA BL1. Below is a summary of the steps involved.

- SEP initialization
 - Read configuration strap values from GPIO registers
 - Determine boot options, if any
- Configure CPU CSRs
 - PMP
- Configure peripherals
 - SPI host controller
 - OTP controller
 - DMA controller
 - HMAC-SHA256
- Start C runtime and execute the following boot tasks
- Load second stage bootloader firmware BL1

- Instruct SPI host controller to fetch BL1 firmware from an attached external SPI flash memory device. Stream image from SPI to TCM memory via DMA controller.
- Verify integrity of BL1 firmware image via RSA digital signature verification algorithm
 - Read manifest and extract firmware signature, RSA public key (n, e) and OTP personality mask
 - Run SHA256 firmware to generate the hash digest ``h" of \{BL1 firmware, OTP fuse profile}
 - Configure OTBN PK cryptographic accelerator image and parameters
 - Extract the RSA3k digital signature verification firmware from BL0 and load into the OTBN Imem, i.e. the instruction memory
 - Provision the BL1 manifest's public key (n, e) into Key Manager via software write port.
 - Exponent e is expected to be a short value such as $2^{16}+1$, so we don't need 3k bits even if RSA3k is being used, only 17 bits
 - Modulus n would still be the full 3k bits for RSA3k
 - Run RSA firmware to verify the BL1 signature stored in its manifest using its public key also stored in the manifest
 - Compare hash digest ``h" with the decrypted signature. This is to guard against fault attacks.
 - Read decrypted signature from OTBN Dmem.
 - Remove any padding from digest (e.g. PKCS#1) to produce h'
 - Perform comparison $PASS := (h=h')$
 - Repeat this operation at least twice with a random delay in between and only pass verification check if individual checks pass. This is a guard against fault injection attacks.
- Anti-rollback checks are completed at this point: security version of manifest is compared to security version in OTP
- Verification of the integrity of SEP / SMC BL1 payload image based on payload hash/length
- Execution unlock logic in the CPU PMP to enable BL1 code execution if hash comparisons have passed.
 - Failure to complete any of the above checks will result in processor entering fail loop requiring a reset event to exit

Future versions of the SEP will also have additional hardware redundancy by running the CPU in Dual Core Lock Step. This will further protect against fault attacks.

4.6.6. Dual Scratch Register Banks

There shall be two sets of 16 x 32-bit scratch register banks instantiated at the SEP top level. Both are designed as simple SystemRDL defined configuration registers with AXI4-lite slave interfaces. The configuration registers have the following properties:

- Software read and write access
- No hardware read access - they are only intended as a software register

The two sets are distinguished by their different reset properties.

Scratch register set 0 has the default **cold reset** behavior. This means that its state is only changed by either a software write operation or a power-on-reset event.

Scratch register set 1 has **additional warm reset** behavior. It can be reset by a signal from the Watchdog Timer (WDT).

4.6.7. Direct Memory Access Controller (DMA)

The module is based on the [OpenTitan DMA Controller](#) design. Its primary purpose is to offload the SEP CPU from the overhead of large or slow data copying operations. The DMA has a built-in SHA2 engine for inline hashing.

Transfers operate on a word by word basis. This is because the TL-UL bus, used in the OpenTitan implementation, does not support burst transfers. AXI4-lite also does not support bursts.

Features:

- Transfer modes
 - Memory to memory
 - e.g. Scratch SRAM to CPU ICCM SRAM
 - Memory to peripheral
 - e.g. Scratch SRAM to I3C
 - Peripheral to memory
 - e.g. SPI to Scratch SRAM
- Addressing modes
 - Autoincrementing source and/or destination address for memories
 - Fixed address for peripherals
 - Circular (wrapping) address range for buffers and queues in memories
- Inline hashing during data transfer
 - SHA2-256 or SHA2-384
 - Digest stored in DMA CSR
- Interrupt Outputs: full transaction done, chunk transaction done, error
- Hardware handshake via interrupt inputs
 - Interrupts from IP peripherals enable streaming data transfers: UART, SPI
 - Peripheral signals read FIFO data available and transmit FIFO space available to DMA for automatic flow control

OCH Modifications

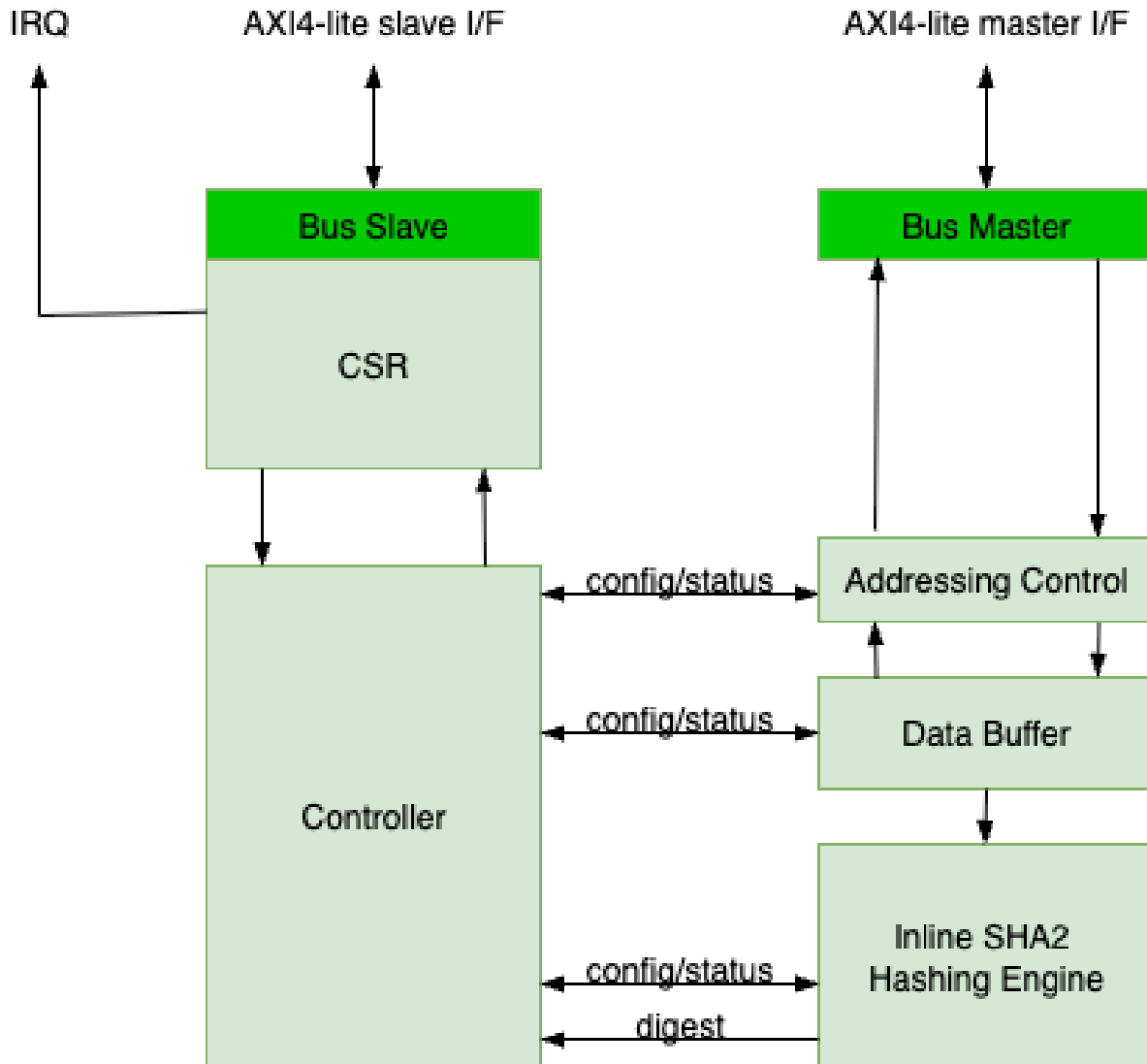
- 64-bit AXI4-lite slave interface (replaces or translates the 32-bit TL-UL bus protocol)

Future Enhancements

The following feature enhancements will be considered for the next generation implementation:

- Inline AES encryption or decryption (with/without inline hashing)
- Inline hashing with only digest written to target slave
- Optimize implementation by using cryptographic resources (AES, HMAC-SHA2) engines inside the cryptographic subsystem and adding streaming interfaces

Replace the FSM based controller with microsequencer or simple CPU (potentially the same one as used in the Key



4.6.8. Alarm Aggregation Processor (AGP)

Note that this module will not be implemented in the first generation SEP and will be deferred to the second generation. The AGP use case is primarily for edge device implementations such as automotive and robotics where fault attacks are in scope.

The AGP module is a hardware component responsible for collection and monitoring alarm event signals throughout the SEP. It is programmed to detect abnormalities and to initiate rapid countermeasures when an attack is deemed to be underway.

The AGP is configured by the SEP CPU to monitor input events, filter them and issue response actions. Response actions include:

- Force wipe of all sensitive state
- Initiate reset sequence on local components, subsystems or the entire SEP

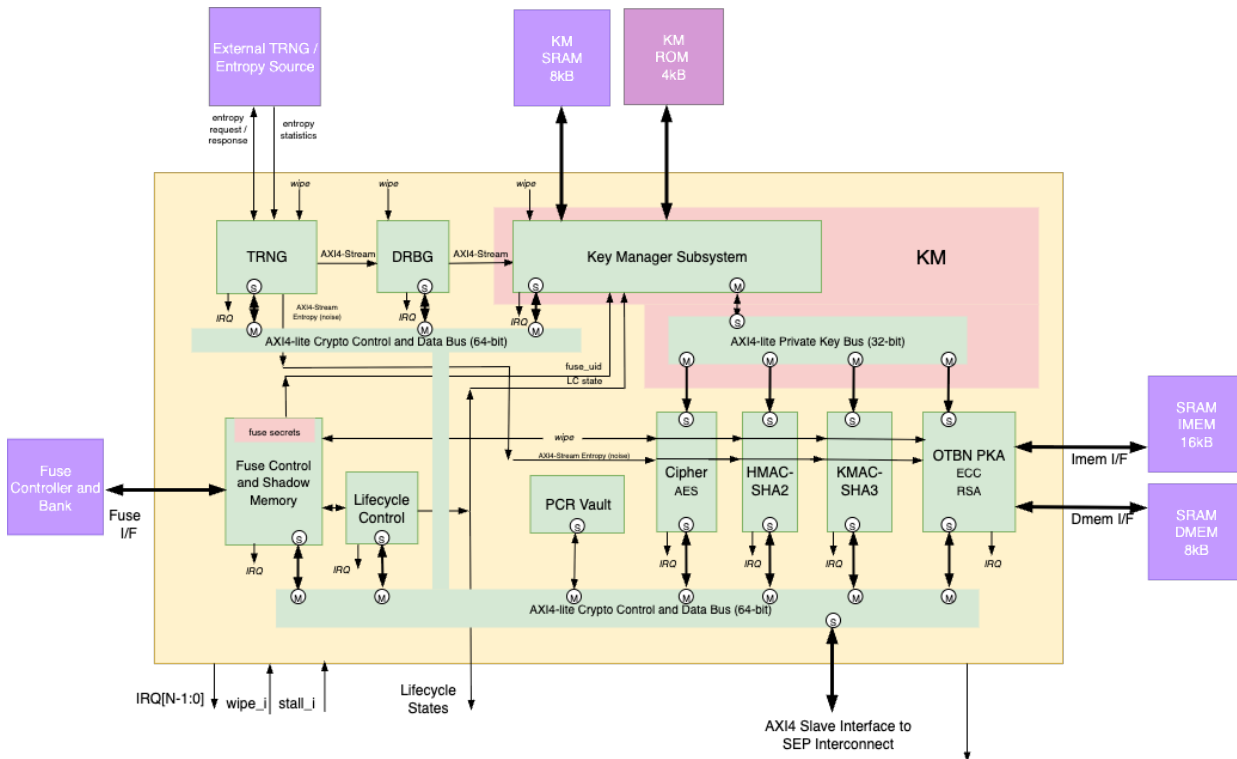
4.7. Cryptographic Subsystem

The cryptographic subsystem is composed of hardware and software. The hardware subsystem has a number of accelerators interconnected by two distinct buses:

- AXI4-lite data and CSR bus
 - Mastered by top host AXI4-lite bridge
- AXI4-lite secret key bus
 - Mastered by Key Manager

The subsystem's hardware components are modified versions of the OpenTitan components and the firmware is also based on the [OpenTitan cryptographic library](#).

The primary modifications to the hardware is the separation of the key and data bus interfaces and the switch from TL-UL interface protocol to AXI4-lite. In OpenTitan, keys are transferred from the Key Manager to the Cryptographic accelerators via dedicated "side loading" interfaces. While this does physically separate key and data transport it is not scalable and requires modifications to the Key Manager whenever a new accelerator is added to the subsystem.



The `sep_crypto` module implements the integration layer. A single 64-bit AXI slave port from the SEP system fabric feeds an `axi_demux` that decodes the address and routes each transaction to the appropriate accelerator or internal peripheral. Each OpenTitan wrapper converts the AXI bus to TL-UL through a standard three-stage pipeline: `axi_dw_converter` (64-bit to 32-bit), `axi_to_axi_lite`, and `axi_lite_to_tlul`. The Key Manager drives a private AXI4-Lite key bus to each accelerator wrapper, which maps key words into the sideload structures expected by the OpenTitan cores.

Alerts from all crypto components are collected by 11 `prim_alert_receiver` instances within `sep_crypto` and aggregated into a single `crypto_alert_o` output. Each accelerator has its own independent software-controllable reset, gated with the system reset through the SEP Reset Controller.

The crypto complex uses whole-instance Class 2 scan boundaries so all sequential key, round-key, digest-working, seed,

and random-distribution state is covered without per-register marking. The marked roots in `sep_crypto` are `hmac_wrapper_s3c_scan`, `sep_crypto_otbn_wrapper_s3c_scan`, `aes_wrapper_s3c_scan`, `kmac_wrapper_s3c_scan`, `u_sep_crypto_abr_wrapper_s3c_scan`, `u_entropy_source_s3c_scan`, `u_drbg_s3c_scan`, `u_axis_edn_crypto_s3c_scan`, and `u_key_manager_s3c_scan`. The integration module itself, public bus infrastructure, and combinational entropy muxes are not Class 2 roots.

4.7.1. HMAC-SHA2

The HMAC component is based on the [OpenTitan HMAC](#) design.

OCH Modifications

- AXI4-lite key bus slave interface replaces side loading interface
- AXI4-lite control/status bus slave interface

The `hmac_wrapper` receives a 64-bit AXI slave port from the crypto crossbar and narrows it to 32-bit AXI, then to AXI-Lite, and finally to TL-UL for the `tt_hmac` core. The address path subtracts the HMAC base address and masks to a 13-bit window (8 KB), accommodating both the register space and the message FIFO window at offset `0x1000`. The Key Manager provides an 8-word-per-share key through the `hmac_wrapper_key_reg` block, which maps directly to the `keymgr_key_i` sideload input. HMAC exposes a single fatal alert channel (index [0] in the `sep_crypto` aggregation), three interrupts (`done`, `fifo_empty`, `err`), and an `idle_o` status output. Unlike the other crypto wrappers, HMAC has no EDN connection.

For the full HMAC theory of operation and register map, refer to the [OpenTitan HMAC documentation](#).

4.7.2. OTBN PKA Public Key Accelerator

The module is based on the [OpenTitan OTBN](#) design.

Features:

- Fully programmable big integer processor
 - 256-bit data path with 32 entry register file
 - 32-bit control path with 32 entry register file
 - Complex flow control: conditional branching, loops, call/return with optimized hardware for performance
 - Dual status control bits to assist in big integer computation
 - Optimized modular arithmetic on big integers
- Access to entropy noise stream from the TRNG

The OTBN is programmed in assembly and its runtime driver library currently supports the following algorithms:

- ECDSA P256, P384
- ECDH P256, P384
- Ed25519
- X25519
- RSA 2k, 3k, 4k
 - Key generation, signature generation and signature verification

- SHA256, SHA-512

Since the OTBN is programmable, more algorithms can easily be developed at a later time. The microcode is loaded into SRAM based instruction memory allowing firmware updates to be made in the field.

OCH Modifications

- AXI4-lite key bus slave interface replaces side loading interface
- AXI4-lite control/status bus slave interface

The `sep_crypto_otbn_wrapper` is the most complex of the OpenTitan wrappers. The 64-bit AXI slave port requires a `CACHE_MODIFIABLE` bit fix before the `axi_dw_converter`, because the VeeR CPU may emit non-modifiable cache attributes for MMIO accesses that would be rejected by the AXI-to-AXI-Lite converter. After data-width conversion and protocol bridging, TL-UL drives the standard `otbn` core configured with `Stub = 0` and `RegFile = RegFileFF`.

OTBN's instruction and data memories are external SRAM macros; the IMEM and DMEM `prim_ram_1p_scr` ports are brought out of the wrapper to the `sep_crypto` top level. The key bus is wider than other accelerators: the `otbn_wrapper_key_reg` block maps 12 words per share to the `keymgr_pkg::otbn_key_req_t` structure. The two random-number request ports (`edn_rnd`, `edn_urnd`) are served over native-protocol EDN by the shared `drbg_axis_edn_adapter` instance `u_axis_edn_crypto_s3c_scan` in `sep_crypto`, which converts the post-CSRNG DRBG AXI-Stream feed assigned to the crypto sink domain into per-client EDN responses. The adapter forwards the per-beat FIPS provenance through a generic AXI-Stream `tuser` sideband so OTBN RND observes compliant entropy and does not assert `rnd_fips_chk_fail`. The OTP scramble key handshake is tied to fixed values. Life-cycle escalation and RMA request signals are tied to their inactive state. OTBN occupies alert indices [2:1] in the crypto aggregation and exports a single `intr_done_o` interrupt.

For the full OTBN theory of operation, instruction set, and register map, refer to the [OpenTitan OTBN documentation](#).

4.7.3. AES Cipher

The module is based on the [OpenTitan AES](#) design.

Key sizes: 128, 192 and 256 Modes: ECB, CBC, CFB, OFB, CTR, CTR-GCM

The accelerator consumes 128-bit data blocks written into its CSR memory space and produces 128-bit output blocks *round* number of cycles. The *round* value depends on the key size used.

- 128 bit key: 10 rounds
- 192 bit key: 12 rounds
- 256 bit key: 14 rounds

OCH Modifications

- AXI4-lite key bus slave interface replaces side loading interface
- AXI4-lite control/status bus slave interface

The `aes_wrapper` follows the standard three-stage bus conversion (64-bit AXI → 32-bit AXI → AXI-Lite → TL-UL) without address adjustment or cache-attribute patching. The `aes_wrapper_key_reg` block provides an 8-word-per-share key mapping to the `keymgr_pkg::hw_key_req_t` sideload input. EDN requests are served by the shared `drbg_axis_edn_adapter` instance `u_axis_edn_crypto_s3c_scan`, which converts the post-CSRNG DRBG AXI-Stream feed for the crypto sink domain into AES's native-protocol EDN response. The adapter forwards the per-beat FIPS bit from the stream's `tuser` sideband

into `edn_fips`; AES itself does not consume the flag. The AES core uses `rst_ni` for both the main and shadow reset domains. Life-cycle escalation is tied to its inactive state. AES occupies alert indices [4:3] in the crypto aggregation and exports an `idle_o` status output but no interrupt.

For the full AES theory of operation, modes, and register map, refer to the [OpenTitan AES documentation](#).

4.7.4. KMAC-SHA3

The module is based on the [OpenTitan KMAC](#) design.

OCH Modifications

- AXI4-lite key bus slave interface replaces side loading interface
- AXI4-lite control/status bus slave interface

The `kmac_wrapper` uses the same bus conversion pipeline as AES. The address path subtracts the KMAC base address and masks to a 12-bit window (4 KB) before the `axi_to_axi_lite` stage, aligning the internal decoder with the OpenTitan register block. The `kmac_wrapper_key_reg` block provides the same 8-word-per-share key mapping. EDN is handled identically to AES: KMAC is one of the four clients arbitrated by `u_axis_edn_crypto_s3c_scan` (the shared `drbg_axis_edn_adapter`), served from the same post-CSRNG DRBG AXI-Stream feed. The `app_i` application interface is tied off since the SEP does not use the KMAC hardware application path. Life-cycle escalation is inactive. KMAC occupies alert indices [6:5] in the crypto aggregation and exports three interrupts: `done`, `fifo_empty`, and `err`.

For the full KMAC theory of operation and register map, refer to the [OpenTitan KMAC documentation](#).

4.7.5. True Random Number Generator (TRNG)

Entropy bits are fundamental data for key generation as well as for attack countermeasure implementations. The entropy source must be high quality, meaning that it meets the [NIST SP 800-90B](#) statistical requirements for a cryptographic source.

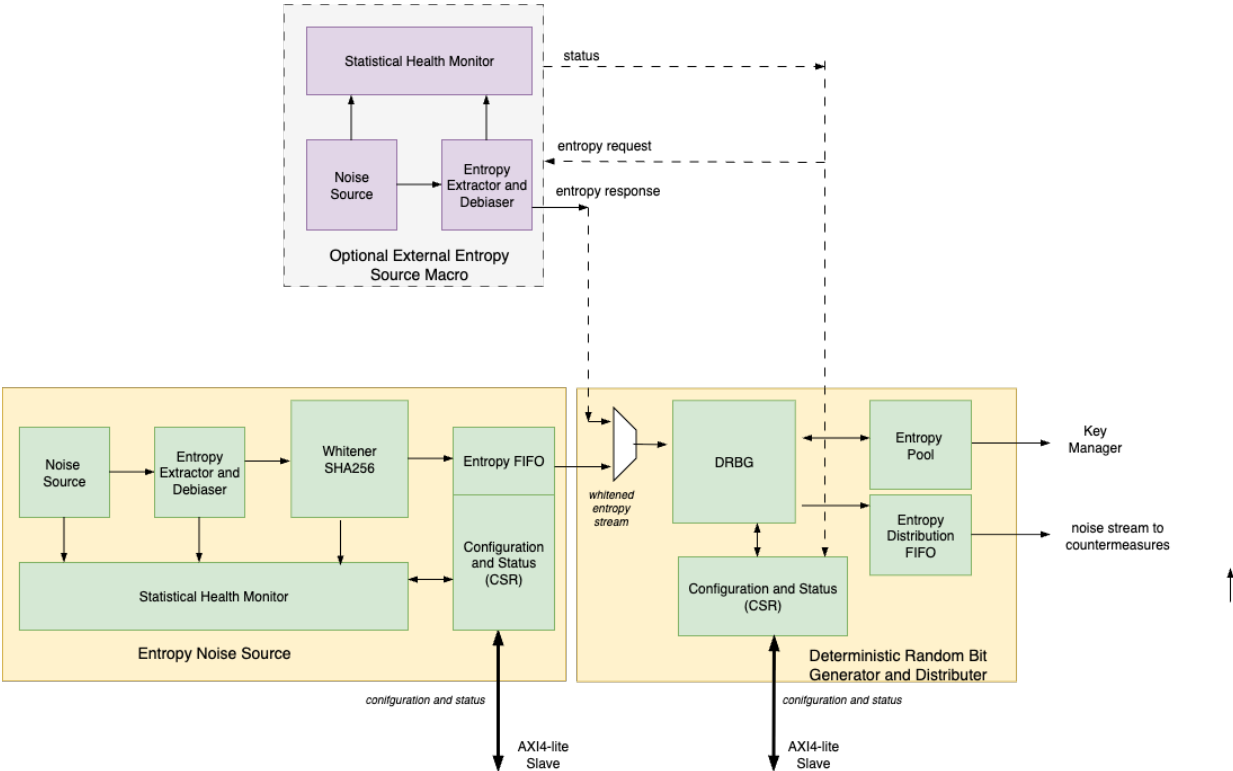
The True Random Number Generator subsystem consists of the following elements:

- Entropy source
- Entropy extractor, debiaser and whitener
- Statistical health online testing and reporting
- Entropy distributor

The entropy source is typically a digital circuit employing ring oscillators which produces the random bit stream. Random bits are generated based on metastable sampling effects resulting from process, voltage and temperature variations in the delay of the asynchronous oscillator sampling. Once the entropy source is enabled it generates a continuous stream of random bits. The raw entropy stream will usually exhibit statistical biases. Debiasing is done with an LFSR like circuit which combines bits at a distance and down samples the stream. The debiaser is followed by a whitener which is most often implemented using a cryptographic hash function such as SHA256. Acceptable choices are listed in the NIST specification. The debiaser and whitener will improve the statistical properties of the stream, increasing the entropy rate, while reducing the actual bit stream rate.

The online health tester monitors statistical metrics on both the raw entropy stream as well as the post-processed, whitened entropy stream on the TRNG output. Health monitoring tests include repetition count and markov tests, specified by NIST in addition to other tests such as bucket and Markov tests. The goal of the health monitor is to detect

any measurable degradation in entropy quality due to external effects such as environment or active attack and to alert the security system of the degradation via alarm interrupts. Metrics can be queried by software via the CSRs.



In OCAH the whitened entropy stream from the noise source is not routed directly to the cryptographic accelerators. Instead it seeds the DRBG, and the post-CSRNG output of the DRBG (NIST SP 800-90A; HMAC-SHA2 or AES-CTR construction) is what fans out to the rest of the crypto subsystem. The DRBG exposes two AXI-Stream EDN endpoints (`u_drbg_s3c_scan.edn_axis_o[1:0]`) that carry conditioned, DRBG-generated bits, one per sink domain:

- endpoint `[0]` feeds the Key Manager sink domain;
- endpoint `[1]` feeds the four native-EDN crypto clients (AES, KMAC, OTBN `edn_rnd`, OTBN `edn_urnd`) through `u_axis_edn_crypto_s3c_scan` — the `drbg_axis_edn_adapter` that converts the 32-bit AXI-Stream feed into a small round-robin native-protocol EDN fan-out.

FIPS provenance is carried per-beat on a generic AXI-Stream `tuser` sideband. The adapter forwards this bit into each client's `edn_fips` so that FIPS-aware consumers such as OTBN RND — which latches the bit and raises `rnd_fips_chk_fail` on a non-FIPS ack — observe the correct provenance of the bits they are receiving.

The Class 2 entropy boundary starts at `u_trng_wrapper_s3c_scan`, whose sequencer, 128-bit release stage, and stream FIFOs retain seed material. Its vendor core remains covered by its delivered DFT view. In `sep_crypto`, `u_entropy_source_s3c_scan` covers the entropy FIFO and SHA-256 whitener, `u_drbg_s3c_scan` covers CSRNG/EDN and endpoint state, and `u_axis_edn_crypto_s3c_scan` covers staging outside the DRBG root. The combinational source-selection muxes are not scan assets.

The noise is distributed via streaming interfaces. The stream interface is generic and compatible with AXI-Stream and TLUL-Stream. The following table describes the stream interface. Source is the generator of the signal. Sink is the consumer. The entropy streaming interface will not require address or mask signals.

Signal	Polarity	Width Range	Description
valid	source	1	Active high indicates that the streaming data beat is valid.

Signal	Polarity	Width Range	Description
ready	sink	1	Backpressure from the receiver. Active high indicates the sink can accept new data. Low informs the transmitter that it should hold off sending more data until the sink is ready again.
data	source	[1,N]	Streaming payload data can be any width 1 to N bits. If the mask signal is used its width must be a multiple of bytes.
address	source	[0,M]	Optional. Used when multiple sink endpoints are available. M can be any integer width
mask	source		Optional. Byte mask if only a subset of the data is consumed by the sink

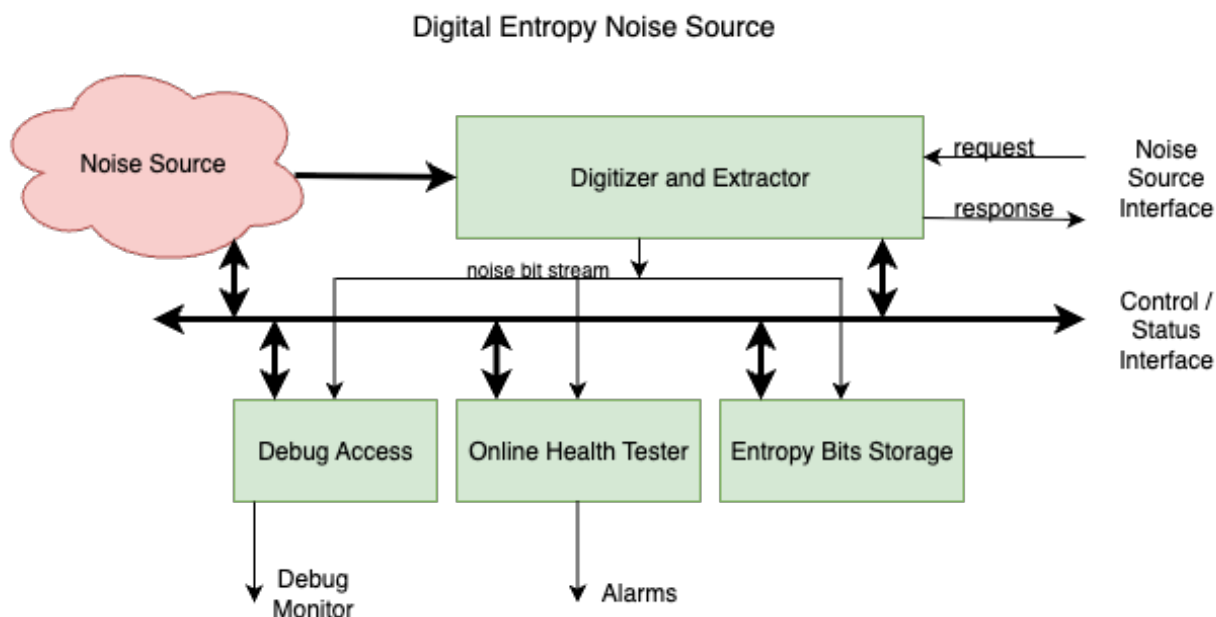
4.7.6. Entropy Source

The detailed specification for the [entropy source](#) is linked here. The summary is included here for convenience.

The entropy source component is a random number generator which is based on multiple asynchronous ring oscillators sampled in such a way to generate random bits from the jitter noise and metastable latching behavior. It forms the basis of a True Random Number Generator (TRNG).

The generated stream is ``raw" noise in the sense that there may still be some statistical biases present which need to be filtered out by an optional entropy conditioner post processing unit which would consume the output noise bits. NIST specifies a set of acceptable conditioners. Common conditioners are based on standard hash functions such as SHA256 and ASCON as well as standard keyed hash functions such as AES-CMAC and HMAC.

The goal of this test component is to assess the statistical properties of the raw entropy source, so we will not add a conditioner here.



The diagram below shows the entropy source component which implements the above concept. The client controls the component via the APB bus interface which provides access to a set of memory mapped configuration and status registers (CSRs) described later in this document. The raw entropy stream is connected to the (optional) downstream conditioning unit mentioned above as well as the health statistics tester, debug monitor and entropy bits storage.

The entropy source is designed according to guidelines set forth in the NIST SP800-90B specification and is planned to be

ready for FIPS 140-2 and 140-3 certification. It will also be compliant with AIS31 statistical requirements specified by the BSI agency.

Noise Generation

The entropy source uses 12 independent ring oscillators as its noise source. Each ring oscillator is constructed from manually instantiated generic logic cells (`gnand2`, `gbuff`, `gmux2`) to prevent synthesis optimization from disrupting the asynchronous feedback loop. Each oscillator has two configurable lengths: a `TOTAL_LENGTH` and a `TAPPED_LENGTH`, where a multiplexer selects between the short and long feedback paths. The `entropy_rosc_tune_fsm` toggles between these two lengths on a rising edge of any health test failure, providing a rudimentary frequency detuning mechanism.

The oscillator output is sampled by a metastable D flip-flop (`gdff`) clocked by a sampling clock. The sampled bit is then synchronized to the system clock domain through a two-stage synchronizer. Each lane can be clocked either by an external sampling clock (`rosc_sample_clk_i`) or by a shared internal ring oscillator (total length 109, tapped length 53). A per-lane ripple divider provides division ratios of 1, 2, 4, 8, 16, and 32, selected through the CSR.

Decorrelation and BIW Mixing

Each sampled noise bit enters a serial decorrelator (`entropy_decorrelator`), which shifts bits through a 29-stage shift register with XOR feedback. A downsample counter derived from the `sample_clk_div` CSR field determines how frequently the top 8 bits of the shift register are latched and emitted as an entropy byte. A per-byte mask allows individual bit lanes to be disabled. The decorrelator can be bypassed through a CSR control bit, in which case raw noise bits pass directly to the output.

The 12 decorrelated byte streams are combined by the Balanced Iterative Whirlpool (BIW) stage, which uses four `gf_muladd` instances performing $y = a*b + c$ in $GF(2^8)$ with the AES field polynomial (0x1B). The 12 input bytes are grouped into three sets of four and reduced to a single 32-bit word. This mixing step improves the statistical independence of the output without introducing significant latency.

Health Testing

Health tests operate at two levels. Each of the 12 `entropy_generator` instances runs an 8-bit `entropy_health_test` on its individual decorrelated output. At the top level, a second 32-bit `entropy_health_test` instance monitors the post-BIW (or post-bypass) stream.

The health test wrapper instantiates three OpenTitan-derived test modules. The repetition count test (`entropy_src_repnt_ht`) tracks consecutive identical values per bit lane and flags when a threshold is exceeded. The adaptive proportion test (`entropy_src_adaptp_ht`) accumulates per-bit ones counts within a configurable window and compares against high thresholds. The Markov test (`entropy_src_markov_ht`) counts bit-pair transitions and compares to high and low thresholds at window boundaries. Window synchronization is driven by a `prim_count` counter in the `entropy_generator_complex`.

Health test thresholds, enable bits, and window sizes are all programmable through the CSR interface. Test status is reported as an 8-bit vector where bit 0 indicates a repetition count failure, bit 3 an adaptive proportion failure, and bits 4-5 Markov high/low failures.

SHA-256 Whitening

When the SHA-256 whitener is enabled via the CSR, the 32-bit BIW output stream feeds into the `entropy_sha256_whitener` module. This module accumulates 16 consecutive 32-bit words (one SHA-256 block of 512 bits), computes the SHA-256 digest using the OpenTitan `prim_sha2_32` primitive, and emits the 8-word (256-bit) digest. The compression ratio is 2:1 (16 input words produce 8 output words). When the whitener is disabled, the BIW output passes through to the FIFO

without modification.

Output FIFO

The conditioned entropy stream is stored in a 64-entry FIFO (`entropy_fifo`). Each entry is 36 bits wide: 32 data bits plus 4 byte-wise odd parity bits. The FIFO includes pointer integrity checking (normal and inverted pointer pairs with mismatch detection) and parity verification on reads. An optional churn mode XORs the incoming data with a word read from the diametrically opposite FIFO position, improving entropy distribution. Overflow, underflow, parity error, and pointer error conditions are all reported to the CSR block and can trigger interrupts.

The FIFO output drives the `entropy_stream_data_o` and `entropy_stream_vld_o` top-level ports. These connect to the DRBG ingress path and to the distribution network for entropy consumers such as cryptographic accelerators.

4.7.7. DRBG Deterministic Random Bit Generator

This module is [NIST 800-90A](#) compliant. The module is based on the OpenTitan design.

The DRBG is seeded by the TRNG entropy stream and generates an arbitrary length bitstream which is used as foundational random data by the Key Generator.

This component uses the AES-CTR-DRBG version of the 800-90A.

The `drbg` top-level module wraps the OpenTitan CSRNG and EDN blocks with bus-protocol adapters. It accepts a raw 32-bit entropy stream from the entropy source, feeds it directly into the CSRNG seed path, and exposes AXI-Stream (and optionally native) EDN endpoints for deterministic random data delivery.

Parameters

Parameter	Type	Default	Description
<code>SEED_FIFO_DEPTH</code>	<code>int unsigned</code>	1	Depth of the seed FIFO in <code>drbg_csrng_seed_adapter</code> . Holds packed seeds awaiting CSRNG consumption.
<code>EDN_ENDPOINT_COUNT</code>	<code>int unsigned</code>	1	Number of EDN output endpoints. Each endpoint produces an independent AXI-Stream of random data.
<code>EDN_NATIVE_ENDPOINT_COUNT</code>	<code>int unsigned</code>	0	Number of native EDN req/rsp endpoints. Each exposes a raw <code>edn_pkg</code> bundle that bypasses AXI-Stream buffering.
<code>ENDPOINT_FIFO_DEPTH</code>	<code>int unsigned</code>	8	Depth of the per-endpoint FIFO in <code>drbg_edn_axis_adapter</code> .

External Interfaces

The DRBG presents several interface types to the rest of the SEP subsystem.

Interface	Description
Entropy ingress	Fire-and-forget: <code>entropy_stream_data_i[31:0]</code> and <code>entropy_stream_vld_i</code> from the entropy source. There is no backpressure; each valid word feeds the CSRNG seed-packing path directly on the cycle it arrives.
EDN AXI-Stream	<code>edn_axis_o</code> / <code>edn_axis_i</code> (one pair per endpoint). Carries deterministic random data generated by CSRNG through the EDN. This is the primary interface consumed by the Key Manager DRBG sampler.
Native EDN	<code>edn_native_req_i</code> / <code>edn_native_rsp_o</code> (one pair per native endpoint, present when <code>EDN_NATIVE_ENDPOINT_COUNT > 0</code>). Raw <code>edn_pkg</code> req/rsp handshake for consumers that bypass AXI-Stream buffering, such as the SEP entropy-pool FIFO fill path.
CSRNG AXI-Lite (64-bit)	<code>csrng_axil_req_i</code> / <code>csrng_axil_rsp_o</code> . Control-plane access to the CSRNG register set for instantiation, reseeding, and status monitoring.

Interface	Description
EDN AXI-Lite (64-bit)	<code>edn_axil_req_i</code> / <code>edn_axil_rsp_o</code> . Control-plane access to the EDN register set for mode configuration and status.
Life-cycle / OTP	<code>otp_en_csrng_sw_app_read_i</code> (MuBi8) and <code>lc_hw_debug_en_i</code> (lc_tx_t). These signals gate CSRNG software application read access based on device life-cycle state.
Alerts	Two alert pairs (CSRNG and EDN) following the OpenTitan alert protocol.
Interrupts	CSRNG: command done, entropy request, hardware instance exception, fatal error. EDN: command done, fatal error.

Seed Assembly

The `drbg_csrng_seed_adapter` receives one 32-bit entropy word per valid cycle directly from the entropy source (there is no upstream routing or distribution FIFO) and packs the words into full CSRNG-width seeds using an OpenTitan `prim_packer_fifo`. The packed seed is pushed into a shallow FIFO (default depth 1) along with a FIPS compliance flag set to a provisional value (`DRBG_CSRNG_SEED_FIPS_PROVISIONAL = 1`). The CSRNG pulls seeds through the standard `entropy_src_hw_if` handshake: `es_req` / `es_ack` / `es_bits` / `es_fips`. Because ingress is fire-and-forget with no backpressure, the producer must supply entropy no faster than the seed adapter can pack it.

OpenTitan CSRNG and EDN

The CSRNG and EDN instances are standard OpenTitan IP blocks. The CSRNG implements AES-CTR-DRBG per NIST SP 800-90A. It accepts entropy seeds from the hardware interface, processes firmware-issued commands (instantiate, reseed, generate, uninstantiate) through TL-UL, and serves random data to multiple hardware application ports. In this integration, EDN is connected to CSRNG hardware application port 0; any additional hardware application ports are tied off.

The EDN automates generate and reseed command sequences so that downstream consumers receive a continuous stream of random data without firmware intervention. The EDN endpoint interface uses a simple request/acknowledge handshake. The `drbg_edn_axis_adapter` converts this into the AXI-Stream protocol expected by the rest of the SEP fabric, buffering data in per-endpoint FIFOs.

For a detailed description of the CSRNG and EDN theory of operation, register maps, and command protocol, refer to the OpenTitan documentation at [CSRNG](#) and [EDN](#).

Bus Protocol Adaptation

The CSRNG and EDN register interfaces use TL-UL, following the OpenTitan convention. The DRBG wrapper provides two levels of adaptation. The `drbg_axil64_lane_adapter` accepts 64-bit AXI-Lite transactions from the SoC interconnect and narrows them to 32-bit AXI-Lite by selecting the appropriate data lane based on address bit [2]. Only aligned single-lane accesses with strobe `0x0F` or `0xF0` are forwarded; all others are rejected with SLVERR. The downstream `axi_lite_to_tlul` bridge then converts 32-bit AXI-Lite into TL-UL for the CSRNG and EDN register blocks.

4.7.8. Key Manager (KM)

The KM module aims to support a superset of the [OpenTitan Key Manager](#) use cases. It utilizes a small RISC-V CPU for control which provides flexibility to implement all currently required key management operations:

- Derivation
- Storage and destruction
- Policy Checking

- Transfer to cryptographic accelerator

The CPU implementation enables future key management feature enhancements to be delivered with firmware. One example of this is TPM 2.0 attestation.

The OpenTitan documentation details the theory of operation for key derivation. To summarize, KM uses a collection of device identity and root key data to generate a set of keys using a derivation function. This data comes from the following sources:

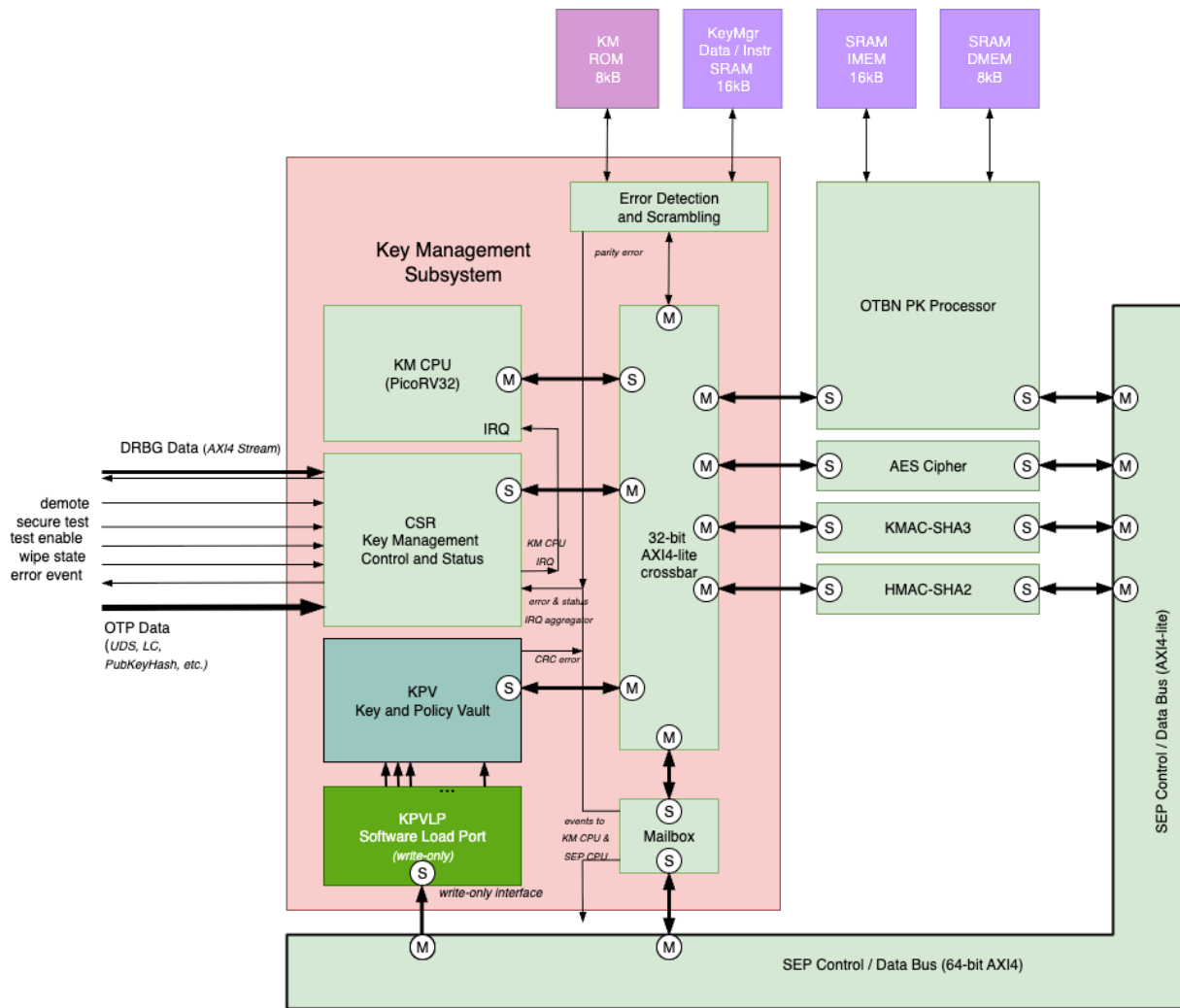
- Entropy stream from DRBG
- Life Cycle (LC) state stored in OTP and managed by the LC controller
- Demotion state (CSR)
- OTP fields: CHIPLET_UID, SIP_UID, SYS_UID, CLASS_KEY
 - Once these fields have been pushed into the Key Manager, the corresponding read lock bits will be set in order to prevent software from accessing these values
- Test control signals
- Configuration parameters e.g. version numbers

The SEP cryptographic subsystem physically separates key and data processing facilities. The KM straddles the two domains and enforces separation for enhanced security. The KM can receive SEP host command requests via the mailbox interface. The KM firmware reads those requests, determines whether the actions are permitted and performs those in an isolated environment. The host never has direct access to any of the generated keys.

Key material provisioned into the KPV by SEP is handled via the mailbox command protocol; the KM firmware writes into KPV slots on the SEP's behalf.

KM communicates with the cryptographic accelerators (OTBN, AES, KMAC, HMAC, and the Adams Bridge post-quantum engine) and the SEP OTP/eFuse controller via a single AXI4-Lite crossbar that routes transactions to internal slaves and external master ports based on address decoding. The crossbar enforces address-based separation between internal peripherals and external engine master ports. All secret data such as keys are transferred over internal bus paths mastered by the KM CPU and are not directly accessible by any other external hosts. SEP host firmware communicates with the KM via the mailbox component. SEP host can send commands and data argument messages into the KM and receive response data in return. KM can also send unsolicited messages to the SEP host via the mailbox response data.

Both the KM ROM and SRAM memories are protected by byte-wise odd parity bits to detect errors. No attempt is made to correct errors. A detected parity error interrupts the KM CPU, which treats it as an unrecoverable fault: the KM notifies the SEP host and halts (see [Unrecoverable Faults](#)).



Parameters

Parameter	Type	Default	Description
ROM_SIZE_BYTES	int unsigned	16384	Size of the KM boot ROM in bytes (16 KB).
SRAM_SIZE_BYTES	int unsigned	16384	Size of the KM data SRAM in bytes (16 KB).
MAILBOX_DEPTH	int unsigned	16	Minimum FIFO depth for each mailbox direction (must be a power of two).
LATCHED_MEM_RDATA	bit	10b0	When set, the PicoRV32 core treats memory read data as latched (hold until next request).

Memory Map

The KM CPU addresses its internal peripherals through a flat 32-bit address space. The AXI4-Lite crossbar decodes the address and routes transactions to the corresponding slave port.

Address Range	Component	Description
0x0000_0000 – 0x0000_3FFF	ROM	16 KB boot ROM (read-only, parity-protected).
0x0000_4000 – 0x0000_7FFF	SRAM	16 KB data SRAM (parity-protected, optional scrambling, region-based write-lock).
0x0000_D000 – 0x0000_DFFF	KPV	Key and Policy Vault register and key storage region.
0x0000_E000 – 0x0000_EFFF	KMCSR	Key Manager Control and Status Registers.

Address Range	Component	Description
0x0000_F000 – 0x0000_FFFF	DRBG Sampler	DRBG random data, configuration, and status registers.
0x0001_0000 – 0x0001_0FFF	Mailbox	KM-side mailbox read/write and control registers.
0x0001_1000 – 0x0001_1FFF	OTP/eFuse	External AXI4-Lite pass-through to SEP eFuse controller. Hardware remaps KM-local addresses to <code>OTP_EFUSE_REMAP_BASE</code> .
0x0001_8000 – 0x0001_CFFF	Crypto Masters	External AXI4-Lite master ports: OTBN, AES, KMAC, HMAC, ABR (4 KB each).

Reset Conditioning

The KM exposes two independent external reset inputs that map to two distinct reset domains. This split allows the SEP host to recover the KM CPU and volatile state without disturbing non-volatile configuration or forcing a full power-on style reset of the subsystem.

Input port	Polarity	Synchronicity	Integration tie-off (SEP)
<code>cold_rst_ni</code>	Active-low	Asynchronous	SEP system cold reset (<code>sep_reset_n</code>).
<code>warm_rst_ni</code>	Active-low	Synchronous to <code>clk_i</code>	SEP KM sub-component software-reset CSR bit (<code>km_sw_rst_ni</code>).

The `km_reset_conditioner` submodule conditions these inputs and the KMCSR-sourced soft reset into two clean, glitch-free, active-low output resets:

Cold path (`rst_cold_aasd_no`)

- Source: `cold_rst_ni` only.
- Conditioning: a single `prim_rst_sync` synchronizer providing asynchronous assert / synchronous deassert (AASD). The raw incoming cold reset is used directly — there is no pulse-stretching hold counter on the cold path.
- Scope: resets the **entire** KM (all components and all reset domains).

Warm path (`rst_warm_sync_no`)

- Synchronous sources: `warm_rst_ni` (external pulse from the integrator) or `soft_rst_ni` (the KMCSR software-triggered reset, see below). Both are assumed synchronous to `clk_i`.
- Cold coupling: `rst_cold_aasd_no` is wired as the asynchronous reset of the warm-path hold counter flop. A cold-reset event therefore immediately asserts `rst_warm_sync_no` as well, guaranteeing that all warm-reset state is also cleared on a cold reset. (Cold reset is deliberately kept out of the synchronous trigger term to avoid a CDC hazard inside the posedge-only counter.)
- Pulse extension: the warm reset is held asserted for a minimum of `MIN_RESET_CYCLES` clock cycles (default 10) after the trigger deasserts.
- Scope: resets the KM CPU and volatile state only — see the domain table below.

Scan-mode multiplexers (`scan_rst_ni` / `scanmode_i`) allow the DFT infrastructure to override both output paths consistently. An assertion guarantees that the warm reset is always asserted whenever the cold reset is asserted.

Reset domain assignment

The two output resets are distributed across the KM components as follows:

Reset domain	Components
Cold only (<code>rst_cold_aasd_no</code>)	ROM/SRAM memory interfaces and parity logic; non-volatile SRAM scrambler lock state (<code>SCRAMBLER_CTRL.lock</code> , <code>SCRAMBLER_KEY</code>); the SEP-facing mailbox FIFOs and state machines; shared mailbox separator/status bits; the <code>BOOT_STATUS.COLD_BOOT_DONE</code> flag; the <code>OTP_READ_LOCK_COLD</code> register (cold-reset-domain OTP read-lock bits).
Warm + cold (<code>rst_warm_sync_no</code>)	PicoRV32 CPU core; the KM-internal AXI4-Lite crossbar/fabric (reset to guarantee no in-flight transaction lingers across a warm reset — see below); the KMCSR warm subset (<code>IRQ_STATUS/IRQ_ENABLE/IRQ_SET</code> , <code>SOFT_RST_CODE</code> , <code>RECOVERABLE_ERR</code> , <code>SRAM_LOCK</code> write-lock bits, <code>SRAM_LOCK</code> violation bits, <code>SRAM_EXEC_MODE</code> , <code>IRQ_ENTRY_ADDR/IRQ_ENTRY_LOCK</code> , <code>OTP_READ_LOCK</code> , <code>OTP_CHANGE_STATUS</code> , and the firmware test-bench registers); all DRBG sampler CSRs and datapath-control state; the KPV per-slot lock bits (<code>lock_write</code> , <code>lock_use</code>); and the KM-side mailbox write/read state machines, <code>IRQ</code> , control, and status fields.

Fabric quiescence across a warm reset

Because the warm reset is applied to the entire KM-internal AXI4-Lite crossbar in addition to the CPU, no outstanding read or write transaction can linger on the KM system fabric across a warm reset. In the PeakRDL-generated register blocks this is reinforced by the `WARM_RST_N` reset signal carrying `cpuif_reset`, which resets each block's AXI CPU interface state machine so that a partially completed transaction does not survive the warm reset.

As discussed earlier, the KM acts as a server to the SEP host CPU and does not provide it with direct access to its private resources. The SEP CPU requests service by sending command messages to KM via the Mailbox component which queues them up and notifies the KM CPU.

Command messages correspond to function calls in the KM CPU firmware. The messages encapsulate a function call identifier, a sequence number along with a declared number of arguments. Commands can perform a number of different programmable operations such as

- Key generation
- Key destruction
- Key transfer
- Software key provisioning
- Key Manager Instruction and Data Memory loading

The command/response message format is intended to be very general to facilitate adoption of new features when required. The message format is illustrated below. It is flexible and can be redefined in future firmware versions. The message format, CRC verification, and command legality checks are entirely firmware-defined. The mailbox hardware does not parse message headers, compute or verify CRCs, or enforce command legality — it behaves as a raw 32-bit word FIFO transport.

header_crc8[31:24]	message_length[23:16]	function_id[15:8]	sequence_number[7:0]
Data_1[31:0]			
Data_2[31:0]			
...			
Data_N[31:0]			
CRC32[31:0] - checksum			

Command/response messages without a data payload (`message_length` = 0x00) drop the payload CRC32 and only contain the header:

header_crc8[31:24]	0x00	function_id[15:8]	sequence_number[7:0]
--------------------	------	-------------------	----------------------

The mailbox consists of two FIFO queues (inbound: SEP → KM, outbound: KM → SEP, minimum depth 16 words each)

which buffer message words. Each word stored in the FIFOs has an associated separator bit; the sender sets this bit when writing the last word of a message into the FIFO (write-only for sender). The receiver can compare the length field in the message header to the number of words read and check the separator bit to verify message completeness.

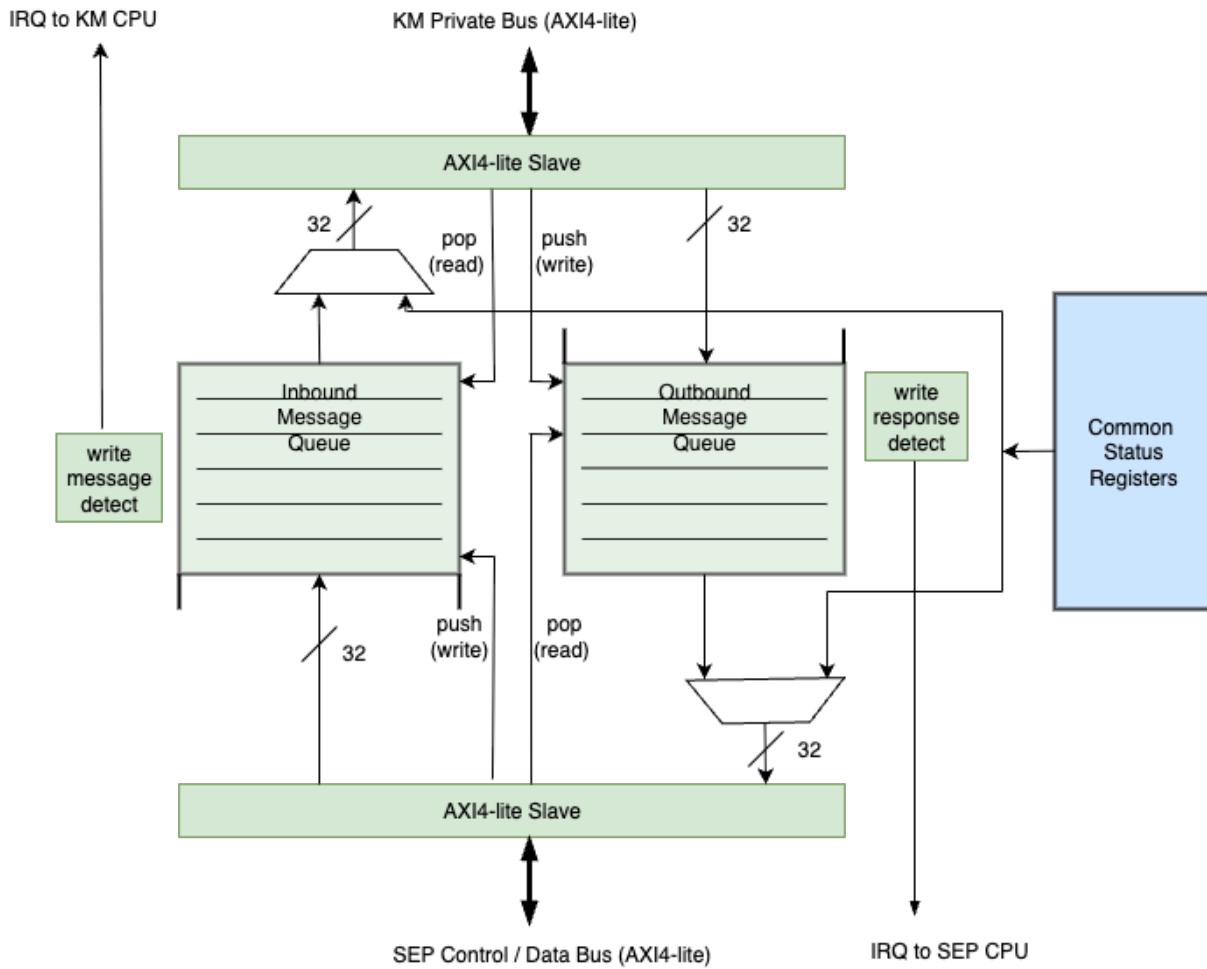
When a new word is written on the inbound channel, the mailbox asserts a level-sensitive interrupt to the KM CPU that remains asserted while the inbound queue is non-empty. The CPU firmware reads (pops) the message out of the mailbox, checks the header and message CRC in firmware, and decides whether or not to execute the corresponding function. When the function has completed, the CPU firmware pushes a response into the outbound channel; the mailbox asserts a level-sensitive interrupt to the SEP CPU that remains asserted while the outbound queue is non-empty.

The mailbox provides status registers indicating queue fill levels and empty/full conditions. When a write is attempted to a full FIFO, the mailbox returns an AXI error response (SLVERR by default, configurable to OKAY), drops the new data, and preserves all existing FIFO contents. Reads from an empty FIFO also return an AXI error response (SLVERR by default, configurable to OKAY). Overflow and underflow conditions set sticky interrupt status bits (one per FIFO direction per condition).

Both KM and SEP sides have a flush control bit (write-1-to-trigger). Writing 1 flushes all FIFOs (inbound and outbound). When SEP flushes, KM receives a sticky interrupt (if enabled). When KM flushes, SEP receives a sticky FLUSHED_BY_KM interrupt (if enabled).

All mailbox interrupt sources on each side (data available, overflow, underflow, flushed-by-other) are aggregated into a single IRQ output per side: one to KMCSR, one to the SEP interrupt controller.

The mailbox and command processing firmware act as a firewall and the KM will only execute legal commands. The number of framed message words received must be consistent with the header's message_length field, and the CRC checks for both the header and message payload must be valid. When illegal commands are detected, the receiver firmware will issue an interrupt as well as push an illegal/abort message into the status FIFO. When corrupt or improperly framed messages are received, the receiver will flush the entire mailbox and an interrupt will be transmitted to the sender. The SEP will perform the exact same message validations as the KM.



Cryptographic keys are stored in the Key and Policy Vault (KPV) block. The life cycle and demotion states are provided to the KM CSR register block and are consumed by KM firmware as key-derivation inputs. The test-mode signal (`secure_tm`) is a **hardware** input to the same block but is not a firmware-readable value: it gates the secret-bearing scan and scan-dump paths and inhibits key derivation in hardware during scan test (see [Scan and Scan-Dump Protection](#)). The KPV key-entry contents are secret-bearing: they may be placed on a scan chain for test coverage but must never be scan-dumpable (see [Scan and Scan-Dump Protection](#)).

The HMAC-SHA256 algorithm is used as the Key Derivation Function (KDF). It is implemented in firmware running on the KM CPU. This function is stored in the KM ROM. Other KDFs could also be implemented in firmware loaded into the instruction/data SRAM.

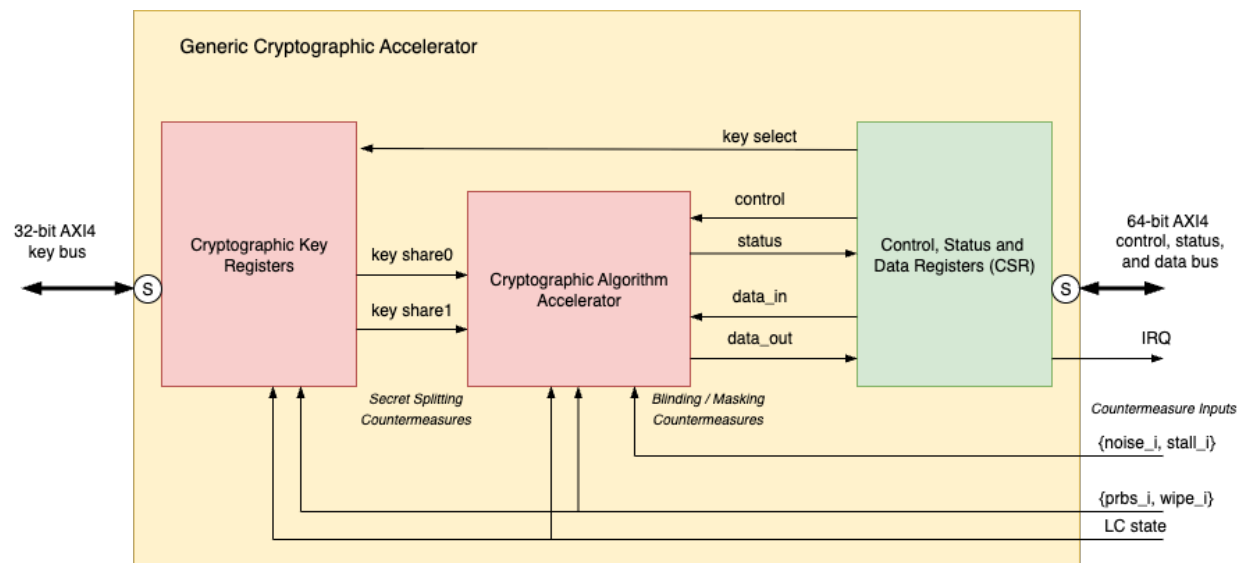
The 32-bit memory map of the KM subsystem is shown below. KM only consumes the bottom 4MB of address space leaving the remaining space unallocated.

Base Address	End Address	Size	Unit	Description
0x0000_0000	0x0000_3FFF	16 kB	KM ROM	Key Manager Program ROM
0x0000_4000	0x0000_7DFF	~15.5 kB	KM SRAM	Key Manager Prgm/Data SRAM
0x0000_7E00	0x0000_7FFF	512 B	KM SRAM (warm-persist)	ROM warm-persistent region (<code>rom_persist</code>)
0x0000_8000	0x0000_CFFF			<i>Reserved</i>
0x0000_D000	0x0000_DFFF	4kB	KPV	Key and Policy Vault

Base Address	End Address	Size	Unit	Description
0x0000_E000	0x0000_EFFF	4kB	KMCSR	Configuration, Status, Interrupt and Interrupt Mask Registers
0x0000_F000	0x0000_FFFF	4kB	DRBG	DRBG Sampling Interface
0x0001_0000	0x0001_0FFF	4kB	MB	Mailbox Message
0x0001_1000	0x0001_1FFF	4kB	OTP/eFuse	SEP eFuse pass-through. KM-local addresses remapped to SEP the value of OTP_EFUSE_REMAP_BASE .
0x0001_2000	0x0001_7FFF			<i>Reserved</i>
0x0001_8000	0x0001_8FFF	4kB	OTBN	Public Key Cryptography Accelerator key storage
0x0001_9000	0x0001_9FFF	4kB	AES	AES block cipher key storage
0x0001_A000	0x0001_AFFF	4kB	KMAC	KMAC key storage
0x0001_B000	0x0001_BFFF	4kB	HMAC	HMAC key storage
0x0001_C000	0x0001_CFFF	4kB	Adams Bridge	ABR key storage
0x0001_D000	0xFFFF_FFFF			<i>Reserved</i>

Separating the key and data domains explicitly as described above increases security as it makes it more difficult for an adversary to compromise the secret key information via a software based attack. In the figure below, the secrets which must be protected reside in the red blocks. The secret data is physically separated from the software accessible bus.

Physical key recovery attacks such as side channel analysis and differential fault injection are still possible, but these require direct access to the device and are more difficult to achieve. These types of attacks are, for the most part, only possible on physically accessible devices such as automobiles and not those in data centers. To detect faults, we recommend including a CRC32 checksum field to the key data structure as described below.



Adams Bridge Key Sideload Interfaces

The KM provides dedicated sideload interfaces to the Adams Bridge (ABR) post-quantum engine, exposed as a single AXI4-Lite register block mapped at **0x0001_C000-0x0001_CFFF**. The block has two directions: four seed inputs the KM drives into ABR, and one shared-key output ABR drives back into the KM.

Seed sideload inputs (KM → ABR). The four seed inputs carry key material the KM side-loads into ABR:

Interface	Width	Description
ML-DSA seed	256-bit	Seed for ML-DSA key generation.
ML-KEM seed D	256-bit	ML-KEM seed D.
ML-KEM seed Z	256-bit	ML-KEM seed Z.
ML-KEM message	256-bit	ML-KEM message input.

Each seed uses the two-share masked layout (two key-share register banks whose XOR is the real value, plus a `key_valid` bit) and is write-only to KM firmware. The seeds are valid `dest_valid` destinations (bits 4–7), so they are targets of `CMD_KEY_TRANSFER` and `CMD_ENGINE_SHRED`, and ROM shreds them during the mutable-firmware handover.

ML-KEM shared-key output (ABR → KM). ABR computes an ML-KEM shared key and presents it to the KM. The interface comprises a 256-bit (8-word) key register array, a `KEY_CTRL` register holding `key_valid`, and an `IRQ_STATUS` / `IRQ_ENABLE` interrupt pair.

- The key words and `KEY_CTRL.key_valid` are read-only to KM firmware: hardware sets `key_valid` and loads the key words when ABR delivers a new shared key. Firmware consumes the key by writing `KEY_CTRL = 0`, which clears `key_valid` and hardware-zeroizes (`hwclr`) the key words. Software can clear `key_valid` but cannot set it.
- `IRQ_STATUS.key_valid` is a sticky, write-one-to-clear bit (`hwset stickybit + onwrite=woclr`). Hardware sets it on the key-ready event; firmware clears it by writing 1. The masked interrupt to the KM CPU is `IRQ_STATUS & IRQ_ENABLE`, on a dedicated PicoRV32 interrupt (bit 5).

The KM firmware ISR acknowledges the IRQ by clearing `IRQ_STATUS` and flags the main loop; it does not consume the key. The shared key is later captured into a KPV slot in response to a SEP `CMD_ABR_SK_TRANSFER` command (see the firmware documentation).

Key Manager CPU Interrupt Processing

The KM receives interrupts from the KM Management CSR register block. The register block collects error and information status from the following sources and aggregates these into a single interrupt:

- ROM parity error
- SRAM parity error
- ROM write attempt
- SRAM write-lock violation
- AXI SLVERR (read and write response monitoring)
- AXI DECERR (read and write response monitoring)
- DRBG sampler error (aggregated from timeout or stream error)
- CPU software writeable via `IRQ\SET` register (for ISR testing)
- Wipe state (rising edge of `wipe\state` input)
- Mailbox inbound data available (from mailbox aggregated IRQ)
- Demote - debug in product LC mode

These events are captured in sticky interrupt status registers (WOC) and aggregated into an interrupt signal which is forwarded to the KM CPU. All event signals can also be triggered by a parallel software interrupt register to enable testing of interrupt service routines (ISR).

Key and Policy Vault (KPV)

The Key and Policy Vault (KPV) is a custom implementation with a single-port AXI4-Lite interface for full read/write access by KM firmware.

KPV has the following registers:

- 32 Key Entry registers, each of 512 bits (16 words)
- 32 corresponding 32-bit control registers

RSA3K keys will not naturally fit into a single key entry, so will be concatenated into several as indicated by the new control register field `xxx[2:0]`.

KM control registers for each slot have the following fields:

- Lock_write[0]
 - Prevents KM firmware from overwriting this key entry's data register and the control fields *extend* and *last_dword* until the next KM reset (cleared by either a cold or a warm reset — the KPV per-slot lock bits are in the warm reset domain). The control bits *lock_write* and *lock_use* (lower two bits) remain writable even when *lock_write* is set. Writes to the CTRL register when *lock_write* is set block all but the lower three bits (no update to blocked fields). Writes to key entry data when *lock_write* is set trigger AXI SLVERR and do not update storage. This field is read-any, write-one-only.
- Lock_use[1]
 - Prevents KM firmware from reading this key entry's data register until the next KM reset (cold or warm). Key entry reads when *lock_use* is set return AXI SLVERR with read data 0. This field is read-any, write-one-only.
- Erase[2]
 - Write-one to start a hardware erase of this slot. The hardware overwrites all 16 key words of the slot with pseudorandom data and clears the slot CTRL register — including *lock_write* and *lock_use* — so the slot becomes reusable. Erase is intentionally **not** blocked by *lock_write* or *lock_use*. The hardware self-clears this bit when the erase completes; firmware can poll it to detect completion. This field is read-any, write-one-only.
- Reserved[3]
- Extend[6:4]
 - Indicates the zero-indexed number of slots for wide key fields. For example, RSA3K modulus requires 3kbit, so occupies 6 consecutive key slot entries with *extend* set to 5. This is enforced by firmware.
- Reserved[8:7]
 - TBD
- Reserved[16:9]
- Last_dword[20:17]
 - Last valid key entry word index [1,15] e.g. *last_dword*=7 for a 256-bit AES key. All invalid key words return 0 if read.
- Reserved[31:21]
 - TBD

Reset values: All KPV control registers (CTRL) reset to 0. All KPV key entry data registers (32 slots × 512 bits) have no reset value: they power up with random (undefined) values for security. The implementation must not connect reset to key entry storage flops.

KPV Scrambling

The KPV supports address and data scrambling during normal operation for all key entry data reads and writes, using the same PRESENT-based approach as SRAM scrambling. When enabled, the stored representation of key data is scrambled; the KM port sees the logical (unscrambled) data on the bus, with scrambling/descrambling performed transparently by the KPV.

The KPV has a scrambler key register (32-bit) and a scrambler control register with an enable bit and a lock bit. These registers are accessible only via the KM port. The lock bit is write-one-only (0→1): software sets lock by writing 1; writing 0 has no effect; lock cannot be cleared until the next **cold** KM reset. The KPV scrambler enable and lock are non-volatile state in the cold reset domain — they are preserved across a warm reset and cleared only by a cold reset. Scrambling is disabled by default after a cold reset.

When scrambling is enabled, the KPV uses the same scrambler algorithm as SRAM but sized for 512×32 words (32 slots × 16 words). The scrambler address input is the linear register index (0..511): $slot_index \times 16$ *word_index*. Address scrambling maps the logical index to a scrambled physical storage index. Data is scrambled before storage and descrambled after retrieval. Only key entry data words are scrambled; CTRL registers are not scrambled.

The KPV scrambler key register has no reset value (powers up random/undefined for security). The scrambler control register (enable and lock) resets to 0. On an emergency wipe (rising edge of *wipe_state*), the entire KPV is reset to zero including key data, control registers, and the KPV scrambler key and control registers.

Wipe State

The KM has a *wipe_state* control input for emergency wipe. The *wipe_state* signal must be held high for at least one complete KM clock period.

On every rising edge of this signal, the KM sets the wipe state interrupt in the KMCSR.

On the next clock cycle after the rising edge, the KPV is synchronously reset to zero: all 512 key entry data registers, all 32 KPV control registers, and the KPV scrambler key and scrambler control registers (enable and lock) are cleared to zero.

In the SEP, *wipe_state* is driven by the software-controlled `sep_cpu_ctrl.KM_WIPE_CTRL.wipe_state` register bit. SEP firmware triggers a wipe by writing the bit to 0 and then to 1; the bit must be returned to 0 before a subsequent write can produce a new rising edge. Because the bit is a registered level, the one-clock-period hold requirement is satisfied by the register itself. The emergency wipe is software-initiated only: no hardware tamper or alarm source asserts *wipe_state*.

Scan and Scan-Dump Protection

For simplicity, the **entire** KM — every register and downstream logic block, including the KPV, KMCSR, mailbox, and DRBG sampler — is treated as **Class 2** in the SEP scan-chain protection model (see the SEP Lifecycle Controller chapter, *Asset and Scan-Chain Protection*):

- **Scannable for test.** KM registers may be placed on a scan chain so DFT achieves maximum coverage. During scan test the secure-test-mode signal `secure_tm = 1` disconnects the secret-source shadow registers (`*_UID`, `CLASS_KEY`) at the eFuse boundary, before they reach the Key Manager, so the Key Manager captures no real secret and key derivation is starved of key material while registers are being shifted.
- **Never scan-dumpable.** During debug (`secure_tm = 0`) the dedicated KM scan chain is tied off, so no KM register or key material can ever be extracted via scan dump — in any LC state and for any owner (chiplet or SiP). This protection is governed solely by `secure_tm`; it is **not** gated by, and cannot be overridden by, the `*_DBG` feature-control

bits.

The complete Key Manager forms one Class 2 scan boundary. This boundary includes the processor, key vault, control and status logic, mailboxes, entropy sampling, memory-scrambling logic, internal interconnect, reset conditioning, and register blocks. Classifying the full subsystem intentionally includes some non-secret control state and avoids fragile per-register policy.

Unique identifiers and the class root key supplied by OTP are Class 1 assets and remain outside every scan chain. In secure test mode, these sources are replaced with a valid zero value at the eFuse shadow boundary, so the Key Manager only ever observes zero and needs no isolation logic of its own. Lifecycle and demotion policy remain available.

The Key Manager copies of the unique identifiers and the class root key are volatile: they are cleared by cold reset and by warm reset, which also carries the firmware soft reset, and they reload from the shadow bus on the cycle after the reset releases. Lifecycle and demotion captures persist across a warm reset because firmware relies on them for policy decisions.

Firmware-enforced runtime access policy is independent of scan and scan-dump protection.

The writable Key Manager SRAM is a Class 2 companion memory because it may retain firmware, key, or message working state. Its memory-test policy must enforce the same never-dump requirement. Immutable ROM is excluded because it cannot retain runtime secrets.

General-logic debug enablement does not apply to the Key Manager's secret-bearing scan chain.

Remote Attestation Support in KPV and PCR

This section describes facilities to support TPM 2.0 attestation. Here are the basic constraints and procedures:

The Endorsement Key (EK) is a public-private key pair. We will support ECDSA P256 and P384, but not RSA. The public and private keys are injected into the device (or generated on the device) and stored in OTP. The public key is recorded in the HSM database. The EK is only used for proving identity. It only performs decryption operations and no signing.

The Attestation Key (AK) is also a ECDSA public-private key pair but is generated inside the device during device manufacturing. The private key is stored in the OTP while the public key is exported for recording in the HSM database. The private key is used as a limited signing key of internal device measurements such as PCR quotes.

The EK and AK are stored in the KPV storage structure described above.

Platform Configuration Registers (PCR) are a set of registers which store system integrity measurement data. These are stored in a dedicated component called the PCR Vault (PCR). PCR implementation is deferred to a future OCAH version.

Another use case is loading the EK and AK attestation keys into the KPV. The SEP host provisions these keys over the mailbox command protocol; KM firmware validates the request and writes the supplied key material into KPV slots on the host's behalf, then asserts the slot lock control field. The host has no direct write path into the KPV—all provisioning is mediated by KM firmware. Once the entries are in place, the KM can read them and derive further keys, populating other KPV slots with the KDF results.

PK Key Generation support

KM must be able to generate a public-private key pair during device test and provisioning manufacturing steps. RSA is a complicated process as two primes need to be generated and this requires running the Miller-Rabin test on large random numbers. It is also much larger than equivalent security ECC keys. The primary use case is TPM 2.0 attestation

signing keys and we primarily will be using ECDSA P256 and possibly P384. The keys are only 256 or 384 bits long and are relatively simple to generate. Once generated, the private key stays on the device and the public key is exported. Export is performed through the outbound mailbox path.

Key Manager Control and Status (KMCSR)

This block of 32-bit registers is separate from the Key and Policy register block. Its purpose is to manage the configuration and operation of the Key Manager. It receives a number of external inputs described below which alter the state or operation of the Key Manager. Behavior is changed for example, when the device finds itself in different Life Cycle states and in debug mode. Inputs from the OTP such as UDI and DRBG random data are used in the key derivation process.

Any differentially encoded register fields are checked for validity. An error will be triggered if the encoding is violated on any of those fields and will be forwarded to the interrupt status register.

The KMCSR is implemented in `km_csr` using a PeakRDL-generated register block (`km_csr_reg`). The register set contains the following functional groups.

The **VERSION** register (offset `0x000`) is a read-only register that reports the Key Manager hardware revision using semantic versioning. It uses the following canonical 32-bit layout, mandated so all software and testbenches decode the version identically:

Bits	Field	Description
[7:0]	PATCH	Patch version component.
[15:8]	MINOR	Minor version component.
[23:16]	MAJOR	Major version component.
[31:24]	RESERVED	Reserved; reads as 0.

The current hardware revision reads as version **1.0.0** (MAJOR=1, MINOR=0, PATCH=0) after reset with no software initialization. The register is read-only: a write completes with an AXI SLVERR response and does not change the value.

The interrupt aggregation logic collects eleven hardware event sources into a single CPU interrupt. Each source has a sticky status bit in `IRQ_STATUS` (write-one-to-clear), a mask bit in `IRQ_ENABLE`, and a software-trigger bit in `IRQ_SET` for ISR testing. The aggregated interrupt output is the logical OR of the masked status bits (`IRQ_STATUS & IRQ_ENABLE`).

Bit	Source	Description
0	ROM Parity Error	Parity mismatch detected on a ROM read.
1	SRAM Parity Error	Parity mismatch detected on an SRAM read.
2	ROM Write Error	A write was attempted to the read-only ROM region.
3	SRAM Write-Lock Violation	A write targeted a locked 512-byte SRAM region.
4	AXI SLVERR	An AXI4-Lite slave returned SLVERR.
5	AXI DECERR	An AXI4-Lite slave returned DECERR.
6	DRBG Error	The DRBG sampler reported a timeout or stream error.
7	Wipe State	Rising edge detected on the <code>wipe_state_i</code> input.
8	OTP Change	A monitored OTP readout field's decoded value changed (see <code>OTP_CHANGE_STATUS</code>).
9	OTP Integrity Violation	OTP dual-rail integrity violation (value and complement not complementary).
10	Execute Violation	Instruction fetch from a non-whitelisted (non-executable) memory region.

Bits [31:11] of `IRQ_STATUS` are reserved. Two interrupt sources are not part of the KMCSR `IRQ_STATUS` aggregation and are

served separately: the mailbox sources (FIFO data-available/space-available and overflow/underflow/flush conditions), and the Adams Bridge ML-KEM shared-key interrupt on its own PicoRV32 interrupt (bit 5, see [Adams Bridge Key Sideload Interfaces](#)).

The CPU IRQ entry vector is programmable through two registers. `IRQ_ENTRY_ADDR` holds the effective IRQ entry address and resets to the built-in default `0x0000_0010`; firmware may write any 32-bit value (the hardware performs no alignment or validity checking) and the register is always readable, returning the current effective entry address. `IRQ_ENTRY_LOCK` is a write-one-only lock bit that can only transition from 0 to 1: while it is 0 firmware may reprogram `IRQ_ENTRY_ADDR` freely; once it is 1, writes to `IRQ_ENTRY_ADDR` are ignored (the value remains readable and unchanged) and further writes to `IRQ_ENTRY_LOCK` have no effect. Both registers are in the warm reset domain: every reset that re-enters ROM (cold, external asynchronous, and KM soft reset) restores `IRQ_ENTRY_ADDR` to `0x0000_0010` and clears `IRQ_ENTRY_LOCK`.

The scrambler lock logic in the KMCSR is described in the SRAM Scrambling section of this document. The SRAM write-lock uses the `SRAM_LOCK` register, which is implemented as write-one-to-set (RDL `woset`). Each bit corresponds to a 512-byte SRAM region. Once a bit is set, it is cleared by the next warm or cold KM reset — `SRAM_LOCK` is in the warm reset domain. The SRAM scrambler key/enable/lock (`SCRAMBLER_KEY`, `SCRAMBLER_CTRL`) remain non-volatile state in the cold reset domain and are preserved across a warm reset. Because `SRAM_LOCK` clears on warm reset, the ROM is responsible for re-applying the write-lock policy on every boot (cold and warm) before handing control to mutable firmware.

The soft reset mechanism is guarded by a magic code. Firmware must write the value `0x53525354` (ASCII "SRST") to the `SOFT_RST_CODE` register; when the code matches, the KMCSR asserts an active-low soft reset (`soft_rst_n`) that feeds the **warm** path of the reset conditioner. This produces a synchronous warm reset of the KM CPU and volatile state (see Reset Conditioning) — it does not reset the cold-only domain. The warm reset remains asserted until the hold counter expires (`MIN_RESET_CYCLES`).

The `BOOT_STATUS` register (offset `0x02C`) carries a single `COLD_BOOT_DONE` flag. It is a write-one-only (RDL `woset`-style) status bit: ROM firmware writes 1 at the end of its cold-boot initialization sequence (in `rom_boot_init`), and the bit then remains set until the next cold reset. `COLD_BOOT_DONE` deliberately uses the default (cold) reset and is **not** tagged with the warm reset signal, so it survives warm resets. Firmware reads this bit to distinguish a cold boot (reads 0) from a warm reboot (reads 1) and can branch its initialization accordingly.

Execute-Permission Whitelist (NX)

The PicoRV32 enforces a whitelist of executable memory regions on every committed instruction fetch (`mem_valid && mem_instr`). Any fetch from outside the whitelist immediately pulses `KMCSR.IRQ_STATUS.EXEC_VIOLATION`, which (if unmasked) routes to the ISR and triggers the unrecoverable fault `ROM_KM_UFAULT_EXEC`.

The whitelist mode is controlled by the `KMCSR.SRAM_EXEC_MODE.enable` bit (write-1-only, warm-reset domain):

<code>enable</code>	Executable regions	Forbidden regions
0 (ROM mode)	ROM (<code>0x0000–0x1FFF</code>) and VROM*	All SRAM; Peripheral/AXI
1 (SRAM mode)	ROM, VROM*, and write-locked SRAM regions (<code>SRAM_LOCK.lock_bits</code>)	Unlocked SRAM; Peripheral/AXI

- VROM (`0x1000_0000`) is a testbench-only virtual ROM used in the block TB. It is treated as always executable in both modes so that firmware running from VROM in simulation is not falsely faulted. VROM does not exist in production silicon.

`SRAM_EXEC_MODE.enable` is set by the stack-less handover assembly (`rom_handover_jump.S`) immediately before the `jalr` to mutable firmware, after SRAM write-lock regions have already been committed. This minimises the window between enabling SRAM execution and the first fetch from SRAM. The bit is write-1-only and cleared by warm reset, so the ROM-only whitelist is re-applied on every boot cycle.

Error Events

Unrecoverable Faults

These events cause the CPU to halt when the corresponding interrupt is unmasked and the ISR determines recovery is not possible. They include, but are not limited to, the following conditions:

- CPU trap events
- ROM or SRAM parity errors
- Invalid writes to ROM or SRAM
- AXI bus faults (SLVERR or DECERR)
- Instruction fetch from a non-whitelisted (non-executable) memory region
- Any other unhandled exception or fault

The PicoRV32 includes a trap state that asserts a trap signal. When the CPU enters this state execution halts. The CPU will enter this state under the following conditions:

- An EBREAK or other illegal instruction is encountered *and* the associated interrupt is masked.
- The CPU encounters a misaligned access *and* the associated interrupt is masked.
- An EBREAK or other illegal instruction is encountered within an ISR.

If an ISR determines that an error event is not recoverable, it will call the EBREAK instruction to trap and halt the core. The CPU trap signal is passed out of KM to indicate an unrecoverable fault. This signal indicates the KM has encountered an unrecoverable fault and is no longer executing (it will not respond to further commands). The signal remains asserted for as long as the KM remains in the trap state. The trap is cleared by resetting the KM CPU — either a cold reset or a warm reset (the PicoRV32 core is in the warm reset domain) restarts the core and clears this signal.

Recoverable Faults

These events do not cause the CPU to halt. They include:

- Receiving corrupt or malformed command messages.
- Detection of possible tampering or attack conditions that are not immediately terminal.
- Any one of the previously listed error interrupts from which the corresponding ISR is able to gracefully recover.

The KM provides an output signal that indicates when a recoverable fault has been successfully handled. This signal is driven by a dedicated register bit in the KMCSR. When an ISR recovers gracefully from an error, it sets this register bit. The bit remains set until the SEP sends a firmware command to the KM to clear it (KM firmware clears the register bit in response to that command).

DRBG Sampler

There are four DRBG Sampler registers: a DRBG data register, a configuration register, a status register, and a prefetch data register. The data register is read by the KM firmware to obtain random words provided by the DRBG component. The data is transferred into the sampler via the AXI4-Stream interface. The Key Manager (sampler) is the slave device on that interface; the DRBG is the master device. The KM slave interface has the following signals to manage flow control and data transfer:

- TREADY (output)

- The sampler asserts TREADY when it is requesting data from the DRBG. We expect that TVALID from the DRBG should almost always be asserted (data available) so when TREADY is asserted, the sampler can immediately accept TDATA. Consider this signal equivalent to a request.
- TVALID (input)
 - DRBG asserts when valid data words are being transferred. It is illegal for the DRBG to assert TVALID and then deassert it before TREADY is asserted; if this occurs, the sampler treats it as an error.
- TDATA[31:0] (input)
 - DRBG sends the data words to the sampler.
- TSTRB[3:0] (input)
 - DRBG can indicate individual byte validity. This will generally be tied to 0xF with all bytes being valid. When TSTRB is not 0xF, the sampler collects only the valid bytes and issues further requests until a full 32-bit word is assembled before returning data to the CPU or updating the prefetch register.

The DRBG data register is read by the KM firmware to obtain a word of random data. A read triggers the sampler to assert TREADY and wait for the DRBG to assert TVALID; if the DRBG has not yet provided data, the sampler holds the KM system bus until a word is received or a configurable timeout expires (then the read completes with SLVERR and zero data). The sampler may prefetch up to one word; when prefetch is disabled, the prefetch data register is cleared. Every read returns a fresh random sample or an error; the same sample is never returned twice. Prefetching is not subject to timeout; only active KM CPU read transactions are subject to timeout.

TSTRB handling: When the DRBG indicates TSTRB not equal to 0xF, the sampler collects only the valid bytes and issues further requests to the DRBG until a full 32-bit word is assembled before responding to a CPU read or updating the prefetch register. Byte order of the random data is not significant.

TVALID protocol violation: In AXI-Stream it is illegal for the source (DRBG) to assert TVALID and then deassert it before TREADY is asserted. If this occurs, the sampler treats it as a protocol violation: it responds SLVERR for the active read (RDATA zero), sets STREAM_ERR (W1C) in the status register, increments COUNT_BAD, and sends the aggregated error interrupt to the KMCSR (IRQ_STATUS.DRBG_ERR).

Error interrupt aggregation: The DRBG Sampler sends an aggregated pulsed interrupt to KMCSR whenever a sampler error occurs (timeout or stream error). KMCSR sets IRQ_STATUS.DRBG_ERR (sticky, W1C) when this interrupt is received; KMCSR provides a corresponding IRQ_ENABLE bit and IRQ_SET bit for testing.

Write rejection: Write requests to the DRBG sampler receive an AXI SLVERR response.

The configuration register is written by the KM firmware; the status register is read-only and updated by the sampler hardware. The configuration register has the following fields:

- PREFETCH[0]
 - Enables prefetching. When disabled, the prefetch data register is cleared. Default: 0 (disabled).
- TIMEOUT[31:16]
 - The number of cycles to wait for the DRBG before timing out an active CPU read. A value of 0 disables timeout. Default: 256. On timeout, the response is SLVERR with RDATA zero.

The status register holds the following read-only fields:

- DRBG_READY[0]
 - TVALID is asserted on the AXI-Stream bus from the DRBG.

- PREFETCHED[1]
 - The sampler has prefetched data available.
- TIMEOUT_ERR[2]
 - Set whenever a DRBG read from the KM CPU times out. Write one to clear.
- STREAM_ERR[3]
 - Set when there is an error on the DRBG AXI-Stream interface (e.g. TVALID deasserted before TREADY). Write one to clear.
- COUNT_BAD[15:8]
 - The number of failed transactions (timeout or other error).
 - Saturates at 0xFF.
 - Clearing: writing any value other than 0 to this field resets the counter to 0; writing 0 has no effect.
- COUNT_GOOD[31:16]
 - The number of successful words transferred to the KM CPU (excludes prefetched words).
 - Saturates at 0xFFFF.
 - Clearing: writing any value other than 0 to this field resets the counter to 0; writing 0 has no effect.

The prefetch data register is exposed as a read-only register in the DRBG sampler memory map for debugging.

Reset behavior: On KM reset (cold or warm — all DRBG Sampler CSRs are in the warm reset domain), DRBG Sampler control and status state (counters COUNT_GOOD and COUNT_BAD, TIMEOUT_ERR and STREAM_ERR status bits, CFG register PREFETCH and TIMEOUT, and any in-flight request or byte-assembly FSM state) are cleared or set to defined reset/default values. DRBG Sampler datapath registers (prefetch data register, assembled word buffer, response data slot) have no reset value: they power up with random (undefined) values.

The sampler hardware is implemented as a five-state FSM in `km_drbg_sampler`. The states and their transitions are summarised in the table below.

State	Description
StIdle	No active request. If a CPU read arrives for the DATA register and a prefetched word is available, the FSM returns the prefetched value and transitions to StRespond. Otherwise it advances to StRequest.
StRequest	The sampler asserts <code>ready</code> toward the DRBG. If <code>tvalid</code> is asserted simultaneously, a data beat is captured and the FSM moves to StByteAssembly. A cycle counter tracks elapsed time against the timeout threshold.
StByteAssembly	Valid bytes indicated by <code>tstrb</code> are accumulated into a 32-bit word. When all four bytes have been collected (<code>tstrb = 4'b1111</code> on a single beat, or multiple partial beats), the FSM proceeds to StRespond. Additional <code>ready</code> pulses are issued for each partial beat.
StRespond	The assembled word is returned on the AXI read data channel with OKAY. The COUNT_GOOD counter increments. If prefetch is enabled and no prefetched word is stored, the FSM re-enters StRequest to prefetch the next word; otherwise it returns to StIdle.
StTimeout	Entered from StRequest or StByteAssembly when the cycle counter exceeds <code>CFG.TIMEOUT</code> (and timeout is enabled). The read completes with SLVERR, <code>RDATA = 0</code> , <code>TIMEOUT_ERR</code> is set, and COUNT_BAD increments. The FSM returns to StIdle.

OTP

The KM consumes two things from the OTP/eFuse, both delivered over private hardwired paths so their values cannot be spoofed by software:

- the fuse-provisioned UIDs — `CHIPLET_UID`, `SiP_UID`, `SYS_UID` — the root secrets from which the KM derives the root

keys; and

- the life-cycle state, one of the contexts a derived key is bound to (see [\[chip-configuration\]](#)).

UID delivery and protection. The UIDs are held in a KM-local copy. ROM reads that copy to derive the root keys, after which it is protected by the KM's own warm-domain read-lock (see [\[uid-provisioning-and-access\]](#)) so mutable firmware cannot read it. This KM-local lock is separate from, and additional to, the eFuse controller's own locks.

Life-cycle state. The life-cycle state is hardwired into the KM as authoritative, software-readable status. Being non-secret it is neither locked nor wiped; the KM uses it directly when binding a key, and ROM can compare it against the value a live key was bound to (see [\[key-revocation\]](#)).

eFuse access path. The KM also has a programmatic (AXI-Lite) path to the eFuse controller — for example to request a read or a program operation. That path is treated exactly like a system-host access: same access filtering, and it honors the eFuse controller's native per-field read/write locks, so the KM gains no read privilege over the fuse array beyond the SEP host's. Key binding always uses the hardwired life-cycle value above, not a value fetched over this path.

Demotion and PCR. The owner demotion state is hardwired from the Life Cycle Controller (LCC); PCR values come from the KM's internal PCR Vault (PCRV). Together with the life-cycle state, these are the life-cycle/runtime contexts a key can be bound to (see [\[chip-configuration\]](#)).

KMCSR readout registers. The OTP-derived fields above are exposed to KM firmware in the KMCSR region as hardware-driven, read-only registers. Each field is delivered to the KMCSR as dual-rail complementary values (`{~value, value}`) and decoded within the KMCSR, which continuously verifies the complement relationship.

Register	Width	Contents
<code>OTP_LIFE_CYCLE</code>	8-bit	Differential-encoded life-cycle state <code>{~lc[3:0], lc[3:0]}</code> .
<code>OTP_DEMOTION_STATE</code>	4-bit	Two 1-bit demotion flags, each differentially encoded into 2 bits: <code>{demote_2[1:0], demote_1[1:0]}</code> .
<code>OTP_CHIPLET_UID_VAL/CPL_0..7</code>	8 × 32-bit	256-bit CHIPLET_UID dual-rail words.
<code>OTP_SIP_UID_VAL/CPL_0..7</code>	8 × 32-bit	256-bit SIP_UID dual-rail words.
<code>OTP_SYS_UID_VAL/CPL_0..7</code>	8 × 32-bit	256-bit SYS_UID dual-rail words.
<code>OTP_CLASS_KEY_VAL/CPL_0..7</code>	8 × 32-bit	256-bit CLASS_KEY dual-rail words.
<code>OTP_READ_LOCK</code>	6-bit	Write-one-to-set read-lock bits (warm reset domain), one per field. Cleared by warm or cold reset.
<code>OTP_READ_LOCK_COLD</code>	6-bit	Write-one-to-set read-lock bits (cold reset domain), one per field. Survives warm reset; cleared only by cold reset. Effective lock = <code>OTP_READ_LOCK OTP_READ_LOCK_COLD</code> .
<code>OTP_CHANGE_STATUS</code>	6-bit	Sticky write-1-to-clear flags: set when a monitored field's decoded value changes.

Dual-rail integrity. The hardware continuously checks that each value word is the bitwise inverse of its complement word. Any violation raises the `OTP_SIGINT` interrupt, which ROM firmware treats as an unrecoverable fault.

Change detection. When a monitored field's decoded value changes, the hardware sets the corresponding `OTP_CHANGE_STATUS` bit and raises the `OTP_CHANGE` interrupt. ROM firmware reads the status, invokes an overridable change hook, and clears the flag. Spurious change events are suppressed until a valid OTP value has been captured after cold reset.

Unique Identifiers (UID) — provisioning and access

The KM consumes three fuse-provisioned Unique Identifiers — `CHIPLET_UID`, `SIP_UID`, and `SYS_UID` (256 bits each) —

provisioned respectively by the chiplet creator, the SiP owner, and the system owner. Each UID is the root secret from which the KM derives that owner's software and hardware root keys (see the **Root keys and the key hierarchy** section below).

The KM is a **ROM-firmware-based key server**: its CPU runs immutable ROM firmware as its steady-state firmware, at cold boot and throughout run time, and keeps servicing commands from ROM while the main SEP CPU is in its own run time. SRAM-loaded firmware is an **optional future extension** for adding KM functions; the baseline performs all required key-management functions, including run-time key derivation, from ROM alone. Consequently the UIDs and hardware root keys are only ever handled by trusted, immutable code.

UIDs and hardware root keys are **ROM-only secrets**: they must never be readable by **mutable** (SRAM-loaded) KM firmware, nor by the SEP host. In the ROM-only baseline this holds by construction — no mutable firmware runs — and the SEP host has no access path to them. For the future SRAM-firmware extension the KM provides a confidentiality safeguard: a read-lock (write-one-to-set, **warm** reset domain) on the UID and hardware-root-key registers. Before ROM ever transfers control to SRAM-loaded firmware it wipes the hardware secrets and sets these locks, so the mutable firmware cannot read them; once locked, reads return zero and the lock cannot be cleared by software — only a reset clears it.

The read-locks live in the **warm** (not cold) reset domain because the KM CPU core is itself warm-reset: any cold, warm, or soft reset restarts the CPU at its ROM reset vector and re-runs boot before any SRAM-loaded firmware can execute. ROM therefore re-establishes the protection on **every** boot — it re-reads the UIDs, re-derives key material, wipes the hardware key from any downstream crypto engine, and re-applies the read-locks before handing control to mutable firmware. Because the CPU is always forced back into ROM first, there is no window in which mutable firmware can run with the locks clear; the locks need not survive a warm reset, and no cold reset is required to restore keys in the ROM-only baseline.

Concretely, this safeguard is the **OTP_READ_LOCK** register: one write-one-to-set bit per OTP readout field (life-cycle, demotion, and the four 256-bit secrets). Setting a bit causes the hardware to return zero for that field's value and complement words. ROM applies the lock using a triple-write convention (three consecutive writes) so it commits even under a single-bit glitch. **OTP_CHANGE_STATUS** shares the same warm reset domain for the same reasons.

A parallel **cold-reset-domain** register, **OTP_READ_LOCK_COLD**, provides the same write-one-to-set, per-field locking semantics but survives warm reset (including soft reset) and is cleared only by cold reset. The effective read-lock for each field is the OR of the corresponding bits in **OTP_READ_LOCK** and **OTP_READ_LOCK_COLD**; either register alone is sufficient to suppress CSR readback. The SEP host can set cold-domain lock bits at any time by issuing **CMD_OTP_READ_LOCK_COLD** (0x28) over the KM mailbox. This allows the SEP host to establish read-locks that persist across warm resets for the entire duration of a power cycle, independently of the warm-reset lock applied by ROM firmware.

Root keys and the key hierarchy

From each UID the KM (ROM firmware) derives two domain-separated root keys. The current life-cycle state is an input to both derivations:

- a **software root key** — $SW_root = KDF(UID, "SW_ROOT", lc_state)$
- a **hardware root key** — $HW_root = KDF(UID, "HW_ROOT", lc_state)$

The distinct domain-separation labels make the two roots cryptographically independent even though they share a UID, and the life-cycle binding makes the roots — and every key derived from them — specific to the life-cycle state they were derived under. The demotion state is deliberately **not** a root input: it is a volatile control, cleared on reset and set by firmware during boot, so it is undefined at root-derivation time. Keys that must be bound to demotion bind it at leaf-key derivation instead (see [\[chip-configuration\]](#)). The roots are then used, with additional context, to derive the **key hierarchy**. A leaf key is $KDF(selected_root, context || user_input || label)$, where $||$ denotes concatenation and

context is the selected configuration (life-cycle state, demotion, PCR, ...).

A core KM principle applies throughout: **derived secret keys are never returned to the requester and are never readable by any agent other than the consuming crypto engine**. The KM side-loads each derived key into the engine's write-only key registers; only non-secret results (for example a generated public key) are returned over the mailbox. Side-loading — not key return — is the reason the KM exists.

Because ROM is the KM's steady-state firmware, derivation runs on demand at run time, and both branches can bind to **run-time** inputs: PCR values, read by the KM directly from the PCR Vault, and **user_input**, supplied in the mailbox command that requests the derivation. After a warm reset ROM re-reads the UID and re-derives keys as needed; no cold reset is required in the ROM-only baseline.

The two roots differ only in their exposure to **mutable** KM firmware (the optional SRAM-loaded extension):

- **Software root (SW_root)** — may be read by KM software, including future mutable firmware, so an extension can derive further keys from it.
- **Hardware root (HW_root)** — never readable by mutable firmware; it is used only by immutable ROM and side-loaded into crypto engines over a private hardware path into their write-only key registers (never returned to software). If the SRAM extension is ever used, before transferring control ROM wipes the hardware key from any downstream crypto engine still consuming it and read-locks the UID copy and the KPV slot(s) holding **HW_root** (ROM cannot clear a KPV slot, but a read-lock renders it inaccessible and unusable), preserving this property.

Chip configuration and key lifetime

The **chip configuration** is the named bundle of context that a derived key is bound to. The requester **selects the chip configuration** for a derivation; together with **user_input** it determines the key:

Field	Description
uid_select	Which owner's root is used: chiplet, SiP, or system.
hw_only	Hardware-only flag: selects the hardware root (a hardware-confined key) versus the software root, and sets the key's confidentiality policy.
lc_state	Life-cycle state the key is bound to. Always bound for UID-derived keys: the roots themselves bind the life-cycle state (see [root-keys-and-hierarchy]), so explicit selection is meaningful only for keys derived from injected user-input keys.
demote_1	Demotion 1 (local-debug / DBG_1) state the key is bound to.
demote_2	Demotion 2 (inter-chiplet debug-bypass / DBG_2) state the key is bound to.
pcr	Which PCR(s), if any, the key is bound to; the KM reads their authentic digests from the PCR Vault. Omitted for keys that are not PCR-bound.

user_input (a 512-bit value carried in the mailbox derivation command) is a further per-derivation input.

Selection. The requester chooses **which configuration fields to bind** — any subset of the fields above (for example a demotion state only, or a PCR plus both demotion states) — and there is no restriction on the values it may select (access control over **uid_select** is left open for a future revision). The KM derives the key bound to exactly that selected subset, and ROM records the binding so the key's lifetime can be tracked. A UID-derived key is life-cycle-bound regardless of this selection, through its root ancestry.

Lifetime. The lifetime of every UID-derived key is tied to the specific fields it is bound to. When a bound demotion state or PCR changes, that key becomes stale and is revoked; keys that do not bind the changed field are unaffected. The life-cycle state is the exception: because the roots bind it, a life-cycle transition invalidates the roots and, with them, the entire UID-derived hierarchy — regardless of each key's selected binding. A revoked key must be re-derived before use. The mechanism is described in **Key revocation** below.

Key revocation

ROM maintains a registry of every derived key, recording its configuration binding (the specific fields and values it was derived under), its KPV storage location, its crypto-engine binding, and its derivation parent.

Triggers. A key is revoked when either:

- any configuration field it is bound to changes value — a life-cycle transition, a demotion, or (for a PCR-bound key) an extend of the bound PCR; or
- its derivation parent is revoked — revocation cascades to every key derived from it. (This is also how an explicitly revoked user key takes its derived keys with it.)

A life-cycle transition additionally invalidates the roots themselves — the roots bind the life-cycle state — so it revokes the **entire** UID-derived hierarchy through the parent cascade, regardless of each key's selected binding.

Action. Revoking a key erases it from its KPV slot — using the per-slot hardware Erase control, which overwrites the key data through the KPV scrambler and clears the slot CTRL register (including its lock bits) so the slot is immediately reusable — and shreds its crypto-engine binding (zeroes the side-loaded key and drops its valid indication). A revoked key must be re-derived before it can be used again.

Mechanism. Revocation is performed entirely by ROM firmware; there is no hardware key-use interlock. On **any** change to a bound configuration field, ROM walks its key registry and revokes the affected keys — hardware-erasing them from the KPV and shredding their crypto-engine bindings — together with any keys derived from them; the keys are re-derived on next use. Hardware support is the per-slot KPV Erase controller plus the interrupts that notify the KM CPU:

- **Life-cycle state and demotion** — typically configured per boot, but ROM does not assume they are immutable at run time: a change at any time triggers the same registry walk and revocation. A demotion change revokes only the keys that bound demotion; a life-cycle change sweeps the entire UID-derived hierarchy, roots included.
- **PCR** — a run-time field. A PCR change raises an interrupt to the KM CPU, which then revokes the keys that bound that PCR.
- **Parent revocation** — an explicit revoke (for example of a user input key, requested over the mailbox) raises the KM inbound interrupt and triggers the same sweep, cascading to derived keys.

A cold reset wipes all key material unconditionally; a warm reset re-enters ROM, which re-establishes keys under the current configuration.

User input keys

Besides the UID-derived hierarchy, the SEP host may **inject** its own keys — **user input keys** — for the KM to manage and derive from. Unlike the UID keys and hardware root keys, user input keys are **software-visible**: they are supplied by the host and live in the software-accessible key domain, not in the ROM-only secret class.

Injection. The host injects a user input key over the mailbox command protocol; KM firmware writes it into a KPV slot and registers the key (integrity check and slot write-lock) so it can be tracked, derived from, and revoked.

Derivation. A user input key may serve as a derivation root: the KM offers the same derivation support for user input keys as for UID roots, deriving child keys with the same chip-configuration context and domain-separation labels. Keys derived from a user input key bind to the selected configuration exactly as UID-derived keys do, and therefore carry the same configuration-tied lifetime. As with all derived keys, the resulting secret keys are side-loaded into the crypto engines and are never returned.

Revocation. The SEP host revokes a user input key by a request over the mailbox, which raises the KM inbound interrupt; ROM then removes the key and **cascades** to every key derived from it — those derived keys are wiped from KM key storage and shredded in the crypto engines — using the same parent-revoke mechanism as configuration-driven revocation (see **Key revocation** above).

ECC key metadata in signatures

The **metadata** associated with a derived ECC private key is the **selected chip configuration** under which that key was generated (the same configuration fields recorded for the key’s lifetime; see **Chip configuration and key lifetime**).

When the key signs, this metadata may optionally be **included in the hash that is signed** — the KM hashes the message together with the metadata. This binds the configuration into the signature, so a relying party can determine from the signature how the signing key was generated (under which life-cycle state, demotion, PCR, and so on). Inclusion is selected per signing operation; only inclusion in the signed hash cryptographically attests the configuration.

CRC Accelerator

The KM CPU is fitted with a CRC accelerator attached through the PicoRV32 PCPI (Pico Co-Processor Interface). It provides hardware versions of the CRC functions the firmware uses on its high-value integrity paths — message header CRC-8/ROHC, message payload CRC-32C, and key-data integrity CRC-32C — so these computations consume far less CPU time than a software loop. The accelerator is always present in the KM build; firmware invokes it unconditionally and does not retain a software fallback path.

The accelerator exposes three R-type custom instructions on the PicoRV32 **custom0** opcode. Each instruction takes two operands — the current CRC chaining state (**rs1**) and the next input unit (**rs2**) — and writes the updated chaining state to **rd**.

Table 173. CRC Accelerator Instructions

Instruction	funct7	funct3	opcode	Input unit
CRC-32C word update	7b0101100	3b000	custom0	rs2[31:0], four bytes little-endian
CRC-32C byte update	7b0101100	3b001	custom0	rs2[7:0] (rs2[31:8] ignored)
CRC-8/ROHC byte update	7b0101100	3b010	custom0	rs2[7:0] (rs2[31:8] ignored)

Behavioral properties:

- The CRC-32C word-update instruction consumes its 32-bit operand as four bytes in little-endian order ([7:0], [15:8], [23:16], [31:24]).
- The CRC-32C instructions expose the internal running remainder **before** the final XOR. Firmware seeds the chaining state with **0xFFFF_FFFF** and applies the final XOR only when it needs the externally visible CRC value, so word updates, byte updates, and intermediate states chain freely.
- For non-word-sized buffers, firmware processes whole words with the word-update instruction and any remaining 1–3 tail bytes with the byte-update instruction; no software tail path is used.
- The CRC-8/ROHC instruction carries its 8-bit state in **rd[7:0]** and zero-extends **rd[31:8]**.
- Empty-input results are identical to the software definitions: with no input units consumed, the seeded state (after the firmware’s final XOR for CRC-32C) is the empty-input CRC.

The three instructions share a single byte-update datapath. CRC-32C word mode runs four byte-update cycles per operation; the two byte modes run one cycle each. The datapath uses reflected CRC update behavior with the reflected polynomial **0x82F6_3B78** for CRC-32C and **0x0000_00E0** for CRC-8/ROHC (the reflection of the **0x07** CRC-8/ROHC polynomial), and never applies the CRC-32C final XOR. The CRC parameter definitions (init, reflection, and final XOR) are given in the

firmware CRC function library documentation.

Any other encoding in the `custom0` CRC instruction family is not recognized by the accelerator: it does not assert the PCPI ready/wait/write handshakes, so PicoRV32's normal illegal-instruction handling is preserved. The feature introduces no new top-level KM ports or CSRs, and the PicoRV32 built-in multiply/divide support remains enabled and unchanged.

Key Manager ROM Code

The KM ROM will be as minimal as possible and support just a few functions described next. Any other functions will be loaded dynamically into the Instruction/Data SRAM by BL1 software or higher via the SEP CPU.

Memory Scrambler Provisioning

Instruction and data SRAM contents are protected by a simple, key based scrambler.

- Provision the key by reading the DRBG register and moving the data into the scrambler key slot
- Set the write-lock bit to prevent any updates on this key slot until the next cold reset (the scrambler key and its lock are non-volatile, cold-domain state preserved across warm resets)
- Enable data content *and* address scrambling on write and descrambling on read operations

Basic Interrupt Service Handler

Support inbound mailbox message handling.

Mailbox Message Handler

This utility is triggered by the ISR and will support handling all inbound mailbox message processing tasks. The most important message task type is supporting the ability for the SEP host CPU to load KM CPU program images into SRAM. This is the method by which we bootstrap more KM functionality. Below are *some* of the tasks the message handler must support.

- Respond to interrupt, and determine it comes from the inbound mailbox
- Retrieve inbound mailbox data from the mailbox by copying it to a segment of local SRAM
- Check validity of the data and command encapsulated in the message and unlock data or perform action if message is deemed to be valid and legal

Key derivation function (HMAC-SHA256)

HMAC-SHA256 is the underlying primitive for both deterministic key derivation and random key generation. The two uses are kept distinct:

Deterministic derivation (root keys and the key hierarchy). Root keys and all UID-derived hierarchy keys are derived deterministically: the inputs are the UID / root-key material plus a fixed context (domain-separation label, life-cycle state, demotion, PCR, user input, ...), and **no** fresh DRBG entropy is mixed in. Determinism is required so the same inputs always reproduce the same key — it lets ROM re-derive keys after a warm reset and binds each key to its selected configuration. When a derived key must incorporate randomness, the requester supplies it as `user_input`, which keeps derivation deterministic; the KM mixes DRBG entropy into a derivation only in rare cases. The derivation:

- Collects the input arguments from the management CSRs and root-key storage into dedicated KDF argument locations.
- Computes the HMAC-SHA256 digest and writes the result to its destination (a KPV slot, or a side-load to a crypto

engine).

- Optionally repeats the computation and compares the two digests to detect a fault attack, reporting pass/fail as an interrupt-status error event.

Random key generation. When a fresh, non-reproducible key is required (rather than a derived one), the KM instead draws DRBG entropy (via the DRBG sampler) as the key material. Such keys are random, not re-derivable, and are not tied to a configuration; they are used where a unique secret — not a deterministic derivation — is needed.

ROM and SRAM Error Detection

Data will be stored with byte-wise odd parity. This feature cannot be turned off, but the interrupt mask register can disable this event from propagating up the interrupt tree to the KM CPU as well as to the external error event signal output.

Parity Error Protection

Parity bits will be "out-of-band" from the data and hence not part of the scrambled word. The motivation for this is to minimize any data path extra latency on top of what is already added by scrambling and descrambling. Parity is on a byte-wise basis so will add an additional 4 bits next to the 32-bit data word.

SRAM Write Locking

The SRAM interface incorporates a write-locking mechanism to designate specific 512-byte memory regions as read-only. This feature is primarily designed to protect dynamically loaded KM firmware from unauthorized modification.

Control of this functionality resides within the KM Control and Status Registers (CSRs). Once a memory region is locked, it cannot be unlocked within the current boot session. It will remain in the read-only state until the next warm or cold reset of the KM. The write-lock state is in the warm reset domain and is cleared on every warm or cold reset.

Protection across warm resets is maintained by the ROM, which is responsible for re-applying the full write-lock policy on every boot (both cold and warm) before handing control to mutable firmware. Because the KM CPU always re-enters ROM after a warm reset, mutable firmware never executes against an unlocked SRAM region.

ROM Warm-Persistent SRAM Region

The highest 512 bytes of KM SRAM (region 31, `0x0000_7E00–0x0000_7FFF`) are reserved as a ROM-managed warm-persistent data structure (`rom_persist`). The mutable-firmware load area, stack, and BSS/data occupy the SRAM below it.

This region is owned exclusively by ROM firmware: it holds ROM state that must survive warm reset and is not general-purpose scratch for mutable (SRAM-loaded) firmware. Mutable firmware must not place its own data here, and once ROM write-locks the region it cannot write it at all (see **Write-lock policy** below).

SRAM storage is cold-domain hardware: its contents survive any warm or soft reset and are erased only by a cold (power-on) reset. ROM firmware exploits this property to carry state across warm resets without requiring non-volatile storage beyond the SRAM macro itself.

Cold initialization. On every cold boot, `rom_boot_init()` detects a cold boot via the `COLD_BOOT_DONE` flag (reads 0 before it is set at the end of boot) and calls `rom_persist_cold_init()`, which writes known-zero values to all fields. On a warm or soft reset, `COLD_BOOT_DONE` already reads 1 and `rom_persist_cold_init()` is skipped, preserving the contents of the region.

Contents. The `rom_persist_t` structure currently contains one field:

Field	Type	Description
<code>sram_fw_size</code>	<code>uint32_t</code>	Byte count of the mutable firmware image currently loaded in SRAM (<code>FW_WORDS × 4</code>). Zero means no mutable firmware is present. Written by <code>CMD_SRAM_LOAD_EXEC</code> on a successful load and verified by <code>CMD_SRAM_EXEC</code> before jumping.

Write-lock policy. Before handing control to mutable (SRAM-loaded) firmware, ROM must call `rom_persist_lock()`, which sets `SRAM_LOCK` bit 31 via a triple write for glitch resistance. After locking, any write into `0x7E00–0x7FFF` is silently dropped and a `SRAM_WRITE_LOCK_VIOLATION` IRQ is raised. The lock is cleared on the next warm or cold reset; ROM re-establishes it before transferring control to mutable code.

Mutable Firmware Model

The KM ROM supports loading and executing a mutable software image from SRAM, controlled by the `CMD_SRAM_LOAD_EXEC` and `CMD_SRAM_EXEC` mailbox commands.

Memory map. Mutable firmware is loaded into SRAM starting at `0x4000` (SRAM word-address 0) and ROM jumps to `0x4000` as the entry point. ROM does not program the CPU IRQ entry vector for the loaded image; the mutable firmware is responsible for setting `IRQ_ENTRY_ADDR` to its own handler before it re-enables interrupts.

SRAM region write-locking. After a successful load, ROM write-locks the 512-byte SRAM regions that contain the firmware image (regions 0 through $\text{ceil}(\text{fw_size_bytes} / 512) - 1$). These locks are applied immediately before the handover jump and remain active for the lifetime of that execution epoch. `SRAM_LOCK` is in the warm-reset domain and is cleared by a warm reset; ROM re-applies the write-lock on each subsequent warm boot when `CMD_SRAM_EXEC` is processed, before handing control to mutable firmware. The `rom_persist` region (region 31) is always locked separately.

Load bounds. The firmware image must fit entirely below a *static* load ceiling, `km_fw_load_limit`, rather than below the live stack pointer at load time. This matters because the ROM stack is in volatile SRAM and grows downward toward the image: after a warm reset that follows mutable-firmware execution, the ROM re-runs and a `CMD_SRAM_EXEC` restart may drive the stack deeper than it was at original load, so a live-SP bound could let the ROM stack later overwrite executable image words. The linker computes the ceiling as `STACK - rom_max_stack` (rounded down to a 512-byte write-lock region), where `__rom_max_stack` is the worst-case ROM stack depth validated against the production ELF by a post-link call-graph analyzer; the firmware build is gated on the analyzer. `CMD_SRAM_LOAD_EXEC` rejects oversized images, and a per-word guard during streaming provides defense-in-depth.

The region-disjointness invariant guarantees that no ROM runtime region (stack, `.data`, `.bss`) shares a 512-byte block with any firmware-image region. This ensures that write-locking the firmware regions does not accidentally lock ROM-owned data, which must remain erasable by the whole-SRAM scramble during handover. Aligning the load ceiling down to a write-lock region boundary is what enforces this invariant.

Sideload key shred. Before locking and the stack-less handoff, ROM shreds the HMAC, KMAC, AES, and OTBN sideload key-share registers, as well as the four Adams Bridge seed sideload interfaces (ML-DSA seed, ML-KEM seed D, ML-KEM seed Z, and ML-KEM message), overwriting them with PRNG data and clearing each `key_valid`. These crypto-engine register banks are outside the SRAM address space, so the whole-SRAM scramble cannot reach them; shredding them explicitly prevents mutable firmware from reading any key material the ROM previously sideloaded.

KPV lock/shred. KPV key-entry data has no warm reset and is descrambled on KM-port reads, and the load path only write-locks (not read-locks) provisioned keys, so live key material would otherwise be plaintext-readable by mutable firmware at the KPV KM-port window. During handover ROM walks every KPV slot, using the firmware key registry to distinguish live slots from free ones: each registered slot is read-locked (`lock_use`, which is independent of `lock_write`, so the hardware returns zero for KM-port key reads), and each free slot is overwritten with PRNG data (falling back to a read-lock if a free slot is already write-locked, such as a revoked key, and so cannot be shredded).

Mailbox flush. After the outbound `RESP_CMD` confirmation has been sent and consumed by the SEP (and, for

`CMD_SRAM_LOAD_EXEC`, after the image stream has been fully drained), ROM flushes both mailbox FIFOs. This erases any residual inbound payload — for example key material from an earlier `CMD_KEY_LOAD` — so mutable firmware cannot recover it from the shared mailbox.

IRQ vector. ROM does not program `IRQ_ENTRY_ADDR` during handoff. Before the stack-less assembly handoff ROM masks all CPU IRQ sources and clears all KMCSR/mailbox IRQ enables, then jumps with the vector left at the ROM default (`0x10`, re-asserted by ROM at boot). The loaded firmware is responsible for programming `IRQ_ENTRY_ADDR` to its own handler before it re-enables interrupts; because every CPU IRQ is masked across the jump, the vector value is immaterial until the firmware does so. `IRQ_ENTRY_LOCK` is left unset so the firmware can program the vector freely.

`IRQ_ENTRY_ADDR` and `IRQ_ENTRY_LOCK` are warm-reset-domain registers: every reset that re-enters ROM (cold, external async, and KM soft reset) restores the vector to the ROM default (`0x10`) and clears the lock, and ROM re-asserts that value at boot before unmasking interrupts. ROM therefore never executes with a mutable-firmware-chosen vector, and an attacker-controlled vector cannot coexist with ROM-privileged execution.

sram_fw_size semantics. ROM sets `sram_fw_size = 0` at the start of a load command (before image transfer) and updates it to `FW_WORDS × 4` only after a successful CRC-32C verification. `CMD_SRAM_EXEC` rejects execution if `sram_fw_size == 0`, preventing jumps when no valid image is present. The field persists across warm resets so mutable firmware can be restarted without reloading the image.

Post-handover IRQ state. ROM clears all KMCSR and mailbox IRQ enables and clears any pending sticky IRQ status bits before the assembly handoff. This ensures mutable firmware does not inherit spurious pending interrupts (in particular, the `sram_write_lock_err` that would otherwise be raised by the whole-SRAM scramble writing into locked firmware regions).

SRAM Scrambling

Both the address as well as data can optionally be scrambled. The goal of the scrambler is not secure data encryption but rather basic obfuscation to impede simple memory dumping by an attacker aiming to recover secrets. The algorithm would be straight forward to break by a trained cryptanalyst. The implementation is intentionally very simple so that the timing overhead on memory read and write transactions is minimal.

The scrambler is disabled by default, but a configuration register field can be set to turn this feature on after booting and the scrambling key is installed. When it is on, all write data transactions will have both the address and data scrambled before performing a write to memory. On read operations, the address will be scrambled in the same fashion as for writes so we have an equivalent address mapping. The read data however must be descrambled. Address data tweaks the key, which means it is mixed in to ensure that the same data at different address locations has a different scrambled value.

The scrambling key is provisioned by the ROM by copying the DRBG register contents into the local key register.

The scrambler IP consists of a generic top-level module (`scrambler`) and a set of pre-configured size-specific wrappers that fix the address width to match a particular SRAM depth. The generic module selects the correct address scrambling function at elaboration time through generate blocks keyed on `ADDR_WIDTH`. The `scrambler_addr_tweak` submodule expands and mixes the memory address into a 32-bit round key. All cryptographic primitives (S-boxes, permutations, and address scramble functions) are declared as `automatic` functions in `scrambler_pkg`.

Parameters

The scrambler module accepts three parameters. Only 32-bit data widths are supported in the current implementation.

Parameter	Type	Default	Description
ADDR_WIDTH	int unsigned	10	Address width of the target SRAM. Determines which address scramble function is selected (6 through 13). Values outside this range fall through to a pass-through assignment.
DATA_WIDTH	int unsigned	32	Data width. Only 32-bit operation is supported.
BYTE_WISE	int unsigned	0	Selects the permutation mode. When 0, a single 32-bit permutation (perm32) is applied to the full word. When 1, four independent 8-bit permutations (perm8) are applied to each byte lane.

Port Interface

The scrambler is purely combinational. It has no clock or reset ports.

Port	Dir	Width	Description
addr_i	in	ADDR_WIDTH	Memory address input.
byte_mask_i	in	DATA_WIDTH/8	Byte mask. Reserved for future per-byte-lane gating; currently unused.
scrambler_key_i	in	32	Scrambling key provided by the Key Manager CSR block.
scrambled_addr_o	out	ADDR_WIDTH	Scrambled address output sent to the SRAM macro.
write_data_i	in	DATA_WIDTH	Plaintext write data from the CPU.
scrambled_write_data_o	out	DATA_WIDTH	Scrambled write data to be stored in the SRAM macro.
scrambled_read_data_i	in	DATA_WIDTH	Scrambled data read back from the SRAM macro.
read_data_o	out	DATA_WIDTH	Descrambled read data returned to the CPU.

Data Scrambling Algorithm

The scrambling algorithm is an extremely rudimentary, round reduced, block cipher inspired by the PRESENT cipher. PRESENT works on 64 bit blocks, while this scrambler works on 32-bit blocks. PRESENT operates on 32 repeated rounds, while this scrambler performs only a single round. The player (permutation function) is also altered to accommodate the reduced block size i.e. word length.

The data path implements the PRESENT 4-bit S-box (**sbox4**) from **scrambler_pkg**. Eight parallel S-box instances process the 32-bit word one nibble at a time. The S-box mapping is shown in the table below.

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
S(x)	C	5	6	B	9	0	A	D	3	E	F	8	4	7	1	2

The inverse S-box (**ibox4**) reverses the mapping and is used on the descramble path.

The permutation layer follows the PRESENT structure with progressive offsets of 8 bits rather than 16. Index i indicates the input bit position and $P(i)$ indicates the output bit position.

i	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
P(i)	0	8	16	24	1	9	17	25	2	10	18	26	3	11	19	27

i	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
P(i)	4	12	20	28	5	13	21	29	6	14	22	30	7	15	23	31

Scrambling Procedure

First the tweaked key is xor'ed with the input write data. The output is split into 8 4-bit nibble lanes and which are fed into 16 sbox functions. The resulting values are concatenated and fed into the player permutation function to form the final scrambled data value which is written to memory.

Descrambling Procedure

The descrambling process is exactly the reverse as scrambling. The scrambled data value is read from memory and then we first perform the iplayer inverse permutation operation. This is followed by 8 inverse sbox functions and the outputs are concatenated. This word is then xor'ed with the tweaked key to recover the unscrambled read data.

Address Tweaking:

```
Rkey = scrambler_key[31:0] ^ {addr[N-1:0] repeated to fill 32-bits}
```

Scrambling:

```
Write_data_sbi = Write_data ^ Rkey
Write_data_sb = {
    sbox(write_data_ak[31:28]),
    sbox(write_data_ak[27:24]),
    sbox(write_data_ak[23:20]),
    sbox(write_data_ak[19:16]),
    sbox(write_data_ak[15:12]),
    sbox(write_data_ak[11:8]),
    sbox(write_data_ak[7:4]),
    sbox(write_data_ak[3:0])}
Scrambled_write_data = player(Write_data_sb)
```

Descrambling:

```
Read_data_sbi = iplayer(Scrambled_read_data)
Read_data_sbo = {
    ibox(Read_data_sbi[31:28]),
    ibox(Read_data_sbi[27:24]),
    ibox(Read_data_sbi[23:20]),
    ibox(Read_data_sbi[19:16]),
    ibox(Read_data_sbi[15:12]),
    ibox(Read_data_sbi[11:8]),
    ibox(Read_data_sbi[7:4]),
    ibox(Read_data_sbi[3:0])}
Read_data = Read_data_sbo ^ Rkey
```

Address Tweaking

As previously mentioned, address tweaking is necessary so that the same data at different addresses scrambles to unique values. Tweaking is performed by simply xor mixing the address with the random scrambler key to form the

round key. If the address is less than 32 bits wide, then we just repeatedly concatenate the address with itself until we have 32 bits, truncating any excess bits.

The `scrambler_addr_tweak` submodule implements this expansion. For power-of-two-aligned widths (8 and 16), the address is replicated exactly to fill 32 bits. For non-power-of-two widths, the address is concatenated three times and the upper portion of the third copy is truncated to produce exactly 32 bits. The table below lists the expansion formula used for each supported width.

ADDR_WIDTH	Expansion to 32 bits
8	{addr, addr, addr, addr}
9	{addr, addr, addr, addr[8:4]} (9+9+9+5 = 32)
10	{addr, addr, addr, addr[9:8]} (10+10+10+2 = 32)
11	{addr, addr, addr[10:1]} (11+11+10 = 32)
12	{addr, addr, addr[11:4]} (12+12+8 = 32)
13	{addr, addr, addr[12:7]} (13+13+6 = 32)
16	{addr, addr}
32	Identity (no expansion).

The expanded address is XORed with `scrambler_key_i` to produce the 32-bit `round_key_o`.

Address Scrambling

Address scrambling is done in a similar fashion as data scrambling with an arbitrary permutation function. The address bits are first xor'ed with the key bits according to the address width. The result is then split into 3 or 4 bit lanes, and passed through one sbox per lane. Finally the sbox results are concatenated and fed through a linear permutation network. There is *no descrambling operation on the address*, only scrambling.

Each supported address width has a dedicated `addr_scrambleN` function in `scrambler_pkg`. These functions partition the address-key-XOR result into lanes of 3 or 4 bits, apply `sbox3` (Pyjamask) or `sbox4` (PRESENT) to each lane, concatenate the S-box outputs, and pass the result through the corresponding `permN` fixed permutation. The lane partitioning varies by width to ensure the full address is covered without padding.

Width	S-box Lane Partition	S-box Types	Permutation
6	[5:3], [2:0]	sbox3, sbox3	perm6
7	[6:3], [2:0]	sbox4, sbox3	perm7
8	[7:4], [3:0]	sbox4, sbox4	perm8
9	[8:6], [5:3], [2:0]	sbox3 x3	perm9
10	[9:6], [5:3], [2:0]	sbox4, sbox3 x2	perm10
11	[10:8], [7:4], [3:0]	sbox3, sbox4 x2	perm11
12	[11:8], [7:4], [3:0]	sbox4 x3	perm12
13	[12:9], [8:6], [5:3], [2:0]	sbox4, sbox3 x3	perm13

Byte-Wise Scrambling Mode

When the `BYTE_WISE` parameter is set to 1, the permutation layer operates on each byte independently rather than across the full 32-bit word. On the write path, the eight S-box outputs are grouped into four byte lanes and each lane is permuted with `perm8`. On the read path, `iperm8` is applied per byte lane before the inverse S-box stage. The S-box and round-key XOR stages are identical regardless of the `BYTE_WISE` setting.

This mode is used in the Key Manager SRAM interface where byte-level write strobes require that scrambling of one byte lane does not depend on data in other lanes.

Instantiation Variants

Five pre-configured modules are provided, each fixing the `ADDR_WIDTH` parameter for a specific SRAM depth. All variants default to `DATA_WIDTH` = 32 and `BYTE_WISE` = 0, though the `BYTE_WISE` parameter can be overridden at instantiation.

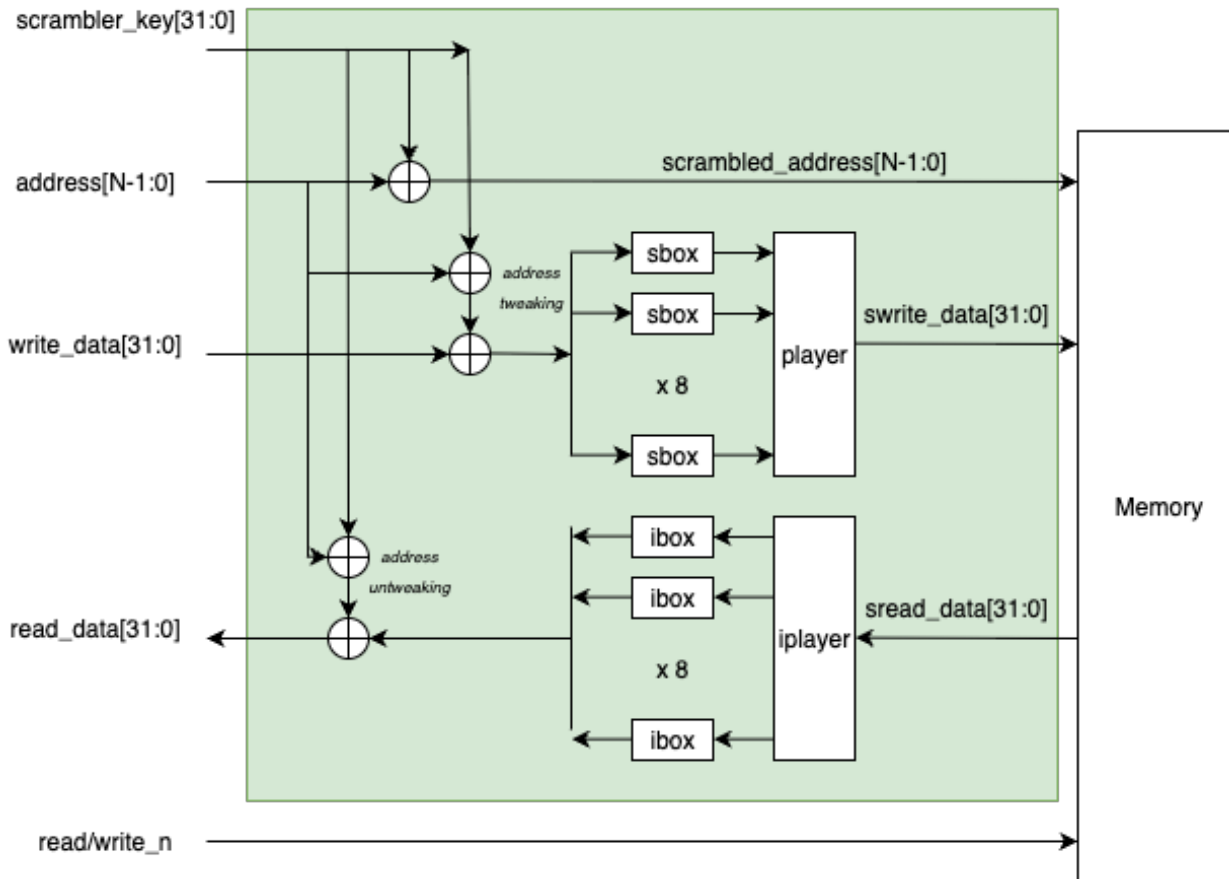
Module	ADDR_WIDTH	SRAM Depth	Address Scramble Function
<code>scrambler_512x32</code>	9	512 words	<code>addr_scramble9</code>
<code>scrambler_1024x32</code>	10	1024 words	<code>addr_scramble10</code>
<code>scrambler_2048x32</code>	11	2048 words	<code>addr_scramble11</code>
<code>scrambler_4096x32</code>	12	4096 words	<code>addr_scramble12</code>
<code>scrambler_8192x32</code>	13	8192 words	<code>addr_scramble13</code>

Integration in the Key Manager

The Key Manager SRAM interface (`km_sram_interface`) instantiates two `scrambler_4096x32` instances with `BYTE_WISE` = 1. The first instance (`u_req_scrambler`) scrambles the write data and address on the request path. The second instance (`u_rsp_descrambler`) descrambles read data on the response path using the address that was captured when the read was issued.

The scrambler is controlled through the `SCRAMBLER_KEY` and `SCRAMBLER_CTRL` registers in the Key Manager CSR block (`km_csr`). The `SCRAMBLER_CTRL` register has two fields: `enable` at bit 0 and `lock` at bit 1. When `lock` is written to 1, the transition is permanent until the next reset. Once locked, the key register reads as zero, writes to it are ignored, and the scrambler is forced into the enabled state. This prevents firmware from disabling scrambling or reading back the key after provisioning.

A bypass multiplexer gates the scrambled data path. When `scrambler_en_i` is deasserted, addresses and data pass through unmodified. Parity is always computed on the plaintext data before scrambling, ensuring that the parity check on the read path operates on the descrambled value.



Key Manager (KM) Firmware

This section is the developer reference for the KM ROM firmware that runs on the PicoRV32 and services SEP requests. It documents the SEP-facing command/response API in detail, along with the firmware-internal subsystems that implement it. Use this document when writing SEP firmware that drives the KM mailbox.

The KM firmware is responsible for bringing the subsystem to a secure operational state, servicing SEP mailbox commands, managing key material in the KPV, sideloading keys into approved crypto engines, and handling recoverable and unrecoverable fault conditions. In this revision, the ROM firmware includes the following major functional areas:

- Boot and secure initialization
- Mailbox command/response protocol
- Message receive/transmit buffering
- Command parsing and validation
- Interrupt and fault handling
- DRBG and firmware PRNG support
- KPV management
- Crypto-engine sideload drivers for HMAC, KMAC, AES, OTBN, and Adams Bridge
- CRC support functions
- SP 800-108r1 counter-mode key derivation
- Key registry and key lifecycle management

- Mutable firmware handover (load, lock, scramble, jump)
- Main event loop and runtime state management

The firmware supports the SEP-facing command set for version and status queries, random key generation, key loading, key transfer, key revocation, engine shredding, recoverable-fault acknowledgement, and mutable firmware handover. In this ROM revision, **CMD_SRAM_VER** returns failure unless mutable (SRAM-loaded) firmware is present and executing; because the ROM hands control to the mutable firmware before returning, **CMD_SRAM_VER** is only meaningful when sent to running mutable firmware.

Firmware Global Parameters

SHRED_ITER indicates the number of extra overwrite passes used during shred operations. Any shred sequence therefore performs **SHRED_ITER + 1** total passes.

Command Message Reference

The SEP communicates with the KM through a framed command/response protocol carried over the mailbox FIFOs. Every direct SEP command elicits a **RESP_CMD** response from the KM. The KM may also emit unsolicited responses, including **RESP_KM_READY**, **RESP_RECOVERABLE_FAULT**, and **RESP_UNRECOVERABLE_FAULT**.

All reserved fields read as 0 and must be written as 0. The maximum payload length for a single message is 255 32-bit words (1020 bytes). The message header CRC and payload CRC are validated independently before command execution. Header validation, sequence checking, command decoding, payload-length checking, and payload CRC checking all occur before a command handler is allowed to act on the payload.

All command messages use the following container format.

Table 174. Command Message

[31 1	[30 1	[29 1	[28 1	[27 1	[26 1	[25 1	[24 1	[23 1	[22 1	[21 1	[20 1	[19 1	[18 1	[17 1	[16 1	[15 1	[14 1	[13 1	[12 1	[11 1	[10 1	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
HEADER_CRC8[31:24]								PAYLOAD_LEN[23:16]								COMMAND_ID[15:8]								CMD_SEQ_NUM[7:0]							
PAYLOAD_WORD[0][31:0]																															
...																															
PAYLOAD_WORD[n-1][31:0]																															
PAYLOAD_CRC32[31:0]																															

Table 175. Message Field Description

Field	Description
CMD_SEQ_NUM	The command message sequence number. It is reset to zero during initialization or during any message buffer flush. This value must increment with each message. It must roll over to 0 after reaching the maximum value.
COMMAND_ID	The ID of the command for the KM to execute. This determines how the PAYLOAD_WORD s are interpreted.
PAYLOAD_LEN	The length of the message payload in number of 32-bit words, <i>not</i> including the PAYLOAD_CRC32 . A payload length of 0 indicates the message has no payload.
HEADER_CRC8	A CRC-8/ROHC computed over the 24-bit message header { PAYLOAD_LEN , COMMAND_ID , CMD_SEQ_NUM } in little endian order, starting with CMD_SEQ_NUM .
PAYLOAD_WORD[n]	A 32-bit data word for the command message payload, where <i>n</i> is always less than PAYLOAD_LEN . PAYLOAD_LEN equal to 0 implies no PAYLOAD_WORD s are present. The contents of the PAYLOAD_WORD s are command-specific.
PAYLOAD_CRC32	A CRC-32C (Castagnoli) computed from the command payload in little endian order from PAYLOAD_WORD[0] to PAYLOAD_WORD[n-1] . PAYLOAD_LEN equal to 0 implies no PAYLOAD_CRC32 is present.

The rest of this section describes all valid commands and their payloads.

Table 176. KM Command Summary

ID	Command	Purpose
0x00	CMD_HW_VER	Request KM hardware version.
0x01	CMD_ROM_VER	Request KM ROM firmware version.
0x02	CMD_SRAM_VER	Request KM SRAM firmware version (handled by mutable firmware once loaded).
0x03	CMD_STAT	Request KM status information.
0x04	CMD_RECOV_ACK	Acknowledge a recoverable fault.
0x10	CMD_EXEC_ROM	Commit KM to ROM execution; disable all subsequent handover commands.
0x11	CMD_SRAM_LOAD_EXEC	Stream a firmware image from the mailbox, load to SRAM, and execute.
0x12	CMD_SRAM_EXEC	Jump to previously-loaded mutable firmware in SRAM.
0x22	CMD_KEY_GENERATE	Generate a random key.
0x23	CMD_KEY_REVOKE	Revoke an existing key.
0x24	CMD_KEY_TRANSFER	Transfer a key to crypto engines.
0x25	CMD_ENGINE_SHRED	Purge crypto engine key registers.
0x26	CMD_KEY_LOAD	Load host-provided key material into the KPV.
0x27	CMD_ABR_SK_TRANSFER	Capture the Adams Bridge ML-KEM shared key into the KPV.
0x28	CMD_OTP_READ_LOCK_COLD	Set cold-reset-domain OTP field read-lock bits (survive warm reset).

0x00 - CMD_HW_VER

Description

Requests KM hardware version information.

Payload

This command has no payload.

Response

This command has a RESP_CMD response with a return code of success, and a return argument containing the version of the hardware.

Table 177. RESP_CMD Success Return Argument

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1										
RESERVED								HW_MAJOR_REV[23:16]								HW_MINOR_REV[15:8]								HW_PATCH_REV[7:0]							

0x01 - CMD_ROM_VER

Description

Requests KM ROM firmware version information.

Payload

This command has no payload.

Response

This command has a **RESP_CMD** response with a return code of *success*, and a return argument containing the version of the ROM firmware.

Table 178. RESP_CMD Success Return Argument

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1										
RESERVED								ROM_MAJOR_REV[23:16]								ROM_MINOR_REV[15:8]								ROM_PATCH_REV[7:0]							

0x02 - CMD_SRAM_VER

Description

Requests KM SRAM firmware version information (if any).

Mutable SRAM firmware may now be loaded via **CMD_SRAM_LOAD_EXEC**. If the KM has jumped to mutable firmware (via either **CMD_SRAM_LOAD_EXEC** or **CMD_SRAM_EXEC**), the mutable firmware is responsible for handling this command and returning its own version information. While still in ROM, the ROM itself handles this command and returns *failure* with no return argument, indicating no SRAM firmware is currently active.

Payload

This command has no payload.

Response

If no SRAM firmware is loaded, this command has a **RESP_CMD** response with a return code of *failure*, and no return argument.

If SRAM firmware is loaded and executing, this command has a **RESP_CMD** response with a return code of *success*, and a return argument containing the version of the SRAM firmware.

Table 179. RESP_CMD Success Return Argument

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1										
RESERVED								SRAM_MAJOR_REV[23:16]								SRAM_MINOR_REV[15:8]								SRAM_PATCH_REV[7:0]							

0x03 - CMD_STAT

Description

Requests KM status information.

Payload

This command has no payload.

Response

This command has a **RESP_CMD** response with a return code of *success*, and a return argument containing the KM status information.

Table 180. **RESP_CMD** Success Return Argument

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1										
KM_STATUS[31:0]																															

Table 181. **KM_STATUS** Payload

Bit	Name	Description
0	recov_fault	The value of the RECOVERABLE_ERR register bit. Set to 1 if the KM is waiting for the SEP to acknowledge a recoverable fault.
[31:1]	reserved	All other fields reserved.

0x04 - CMD_RECOV_ACK

Description

Acknowledges a recoverable error event in the KM. This command clears the **RECOVERABLE_ERR** bit in the KMCSR.

Payload

This command has no payload.

Response

This command has a **RESP_CMD** response with a return code of *success*, and no return argument.

0x10 - CMD_EXEC_ROM

Description

Commits the KM to continued ROM execution for the lifetime of the current warm-reset epoch. Once this command succeeds, all subsequent **CMD_EXEC_ROM**, **CMD_SRAM_LOAD_EXEC**, and **CMD_SRAM_EXEC** commands return *failure* without taking any action. The inhibit flag is cleared on every warm or cold reset, so the SEP may choose again after reboot.

Payload

This command has no payload.

Response

This command has a **RESP_CMD** response with a return code of *success* on first call, or *failure* if a handover command has already been issued in this epoch (including a prior **CMD_EXEC_ROM**).

0x11 - CMD_SRAM_LOAD_EXEC

Description

Instructs the KM ROM to accept a mutable firmware image via the mailbox FIFO, load it into SRAM starting at address **0x4000**, verify the image integrity with CRC-32C, and hand control to the loaded firmware.

After sending the command, the SEP waits for the KM to confirm that it is ready to receive the image. The KM sends a **RESP_CMD** success response ("confirmation"), then immediately disables inbound mailbox buffering and waits for the image frame. The SEP must then send exactly one mailbox frame containing the **FW_WORDS** firmware words followed by a CRC-32C of the entire image as the final word. The frame separator must be set on the CRC word.

If the CRC check fails, the KM triggers an unrecoverable fault (**fw_crc** fault code) and halts. Recovery requires a warm or cold reset.

If the CRC check passes, the KM:

1. Locks OTP secret fields and the **rom_persist** SRAM region, and seals the KPV (read-locks registered slots, shreds free slots).
2. Write-locks the SRAM regions containing the firmware image.
3. Disables all IRQ sources, masks all CPU IRQs, and flushes both mailbox FIFOs to erase residual inbound payload. ROM does not program **IRQ_ENTRY_ADDR**; the loaded firmware sets its own IRQ entry vector before re-enabling interrupts (**IRQ_ENTRY_LOCK** is left unset so it may do so).
4. Scrambles the entire SRAM with a pseudo-random pattern (locked regions are silently skipped by hardware).
5. Clears all CPU general-purpose registers.
6. Jumps to **0x4000** (SRAM base).

The ROM never returns to the main loop. The mutable firmware is responsible for sending its own ready message to the SEP.

Notes

- If the handover-disabled flag is set (from a prior **CMD_EXEC_ROM**), the command returns *failure* immediately and the image is not requested.
- **FW_WORDS** must be non-zero and the loaded image must fit entirely below the static load ceiling **_km_fw_load_limit** (the worst-case ROM stack bound, region-aligned to a 512-byte SRAM write-lock boundary), otherwise **_invalid_arg** is returned and no image is requested. The ceiling is a build-time constant, so a warm-reset **CMD_SRAM_EXEC** restart whose ROM stack runs deeper than at load time can never descend into the loaded image.
- The two execution paths (**CMD_SRAM_LOAD_EXEC** and **CMD_SRAM_EXEC**) both call **rom_handover_finish()**. The stack-less assembly handoff (**rom_handover_jump.S**) runs after all C-level operations complete. This ordering is required because the assembly wipes the C stack.

Payload

Table 182. **CMD_SRAM_LOAD_EXEC** Payload

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1										
RESERVED																FW_WORDS[15:0]															

Table 183. **CMD_SRAM_LOAD_EXEC** Payload Fields

Field	Name	Description
[31:16]	RESERVED	Must be written as zero.
[15:0]	FW_WORDS	Firmware image size in 32-bit words. Does <i>not</i> include the trailing CRC-32C word appended by the SEP. Must be non-zero.

Image Frame

After receiving the **RESP_CMD** confirmation, the SEP sends exactly one mailbox frame with the following structure:

Table 184. **CMD_SRAM_LOAD_EXEC** Image Frame (sent by SEP after confirmation)

Word	Content
0 .. FW_WORDS-1	Firmware image words, in order.
FW_WORDS	CRC-32C of the image (words 0 to FW_WORDS-1), in little-endian byte order. The mailbox separator bit must be set on this final word.

Response

If validation succeeds, the KM sends a **RESP_CMD** success response (the "confirmation") directly to the outbound FIFO before reading the image. This response has no return argument. The KM never returns to the dispatcher after sending this response; no second **RESP_CMD** is emitted.

If validation fails (handover disabled, zero FW_WORDS, reserved bits set, or image exceeds the stack guard region), a **RESP_CMD** error response is sent by the normal dispatcher path.

0x12 - CMD_SRAM_EXEC

Description

Instructs the KM ROM to jump to previously-loaded mutable firmware stored in SRAM. The firmware must have been loaded by a prior **CMD_SRAM_LOAD_EXEC** command in an earlier boot epoch (i.e., `sram_fw_size > 0` in the `rom_persist` region).

The KM sends a **RESP_CMD** success response directly, then executes the same handover finish sequence as **CMD_SRAM_LOAD_EXEC** (lock sensitive data, lock SRAM firmware regions, scramble, clear GPRs, jump). The mutable firmware is responsible for sending its own ready message to the SEP.

Payload

This command has no payload.

Response

If a firmware image is present (`sram_fw_size > 0`), the KM sends a **RESP_CMD** success response directly and then never returns.

If no firmware has been loaded (`sram_fw_size == 0`), the command returns *invalid_arg* via the normal dispatcher.

If the handover-disabled flag is set, the command returns *failure*.

0x22 - CMD_KEY_GENERATE

Description

Requests a freshly generated random key.

Payload

The size of the requested key in multiples of 32-bit words (equal to `REQ_SIZE+1`), as well as the valid destinations for the key.

Table 185. Command Payload

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1										
RESERVED																							REQ_SIZE[6:0]								
RESERVED																							DEST_VALID[7:0]								

Table 186. DEST_VALID Payload

Bit	Name	Description
0	hmac_sha2	Key transfer to the HMAC engine is permitted when set to 1.
1	kmac_sha3	Key transfer to the KMAC engine is permitted when set to 1.
2	aes	Key transfer to the AES engine is permitted when set to 1.
3	otbn	Key transfer to the OTBN engine is permitted when set to 1.
4	abr_mlds_a_seed	Key transfer to the Adams Bridge ML-DSA seed is permitted when set to 1.
5	abr_mlkem_seed_d	Key transfer to the Adams Bridge ML-KEM seed D is permitted when set to 1.
6	abr_mlkem_seed_z	Key transfer to the Adams Bridge ML-KEM seed Z is permitted when set to 1.
7	abr_mlkem_msg	Key transfer to the Adams Bridge ML-KEM message is permitted when set to 1.

Response

If any arguments are invalid, this command has a RESP_CMD response with a return code of *invalid_arg*, and a return argument pointing to the index of the invalid payload word.

The KM will respond with *failure* if the KPV does not have REQ_SIZE+1 number of consecutive free slots available to grant.

If the request is granted, this command has a RESP_CMD response with a return code of *success*, and a return argument containing the parameters of the generated key, including its handle, its size, and its valid destinations. The granted size must always be equivalent to the requested size.

Table 187. RESP_CMD Success Return Argument

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1										
RESERVED									DEST_VALID[23:16]							RESERVED	REQ_SIZE[14:8]							KEY_HANDLE[7:0]							

0x23 - CMD_KEY_REVOKE

Description

This command will revoke an existing key from the KPV, given a valid key handle.

Payload

The handle for the key to invalidate.

Table 188. Command Payload

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1										
RESERVED																							KEY_HANDLE[7:0]								

Response

If any arguments are invalid, this command has a **RESP_CMD** response with a return code of *invalid_arg*, and a return argument pointing to the index of the invalid payload word.

If the request cannot be satisfied for any other reason, this command has a **RESP_CMD** response with a return code of *failure*, and no return argument.

If the request is granted, this command has a **RESP_CMD** response with a return code of *success*, and a return argument containing the handle of the revoked key.

Table 189. *RESP_CMD* Success Return Argument

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]						
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1																
RESERVED																							KEY_HANDLE[7:0]														

0x24 - CMD_KEY_TRANSFER

Description

This command transfers a key from the KPV to one or more crypto engines, given a valid key handle and a bitfield of requested destinations. Requested destination crypto engines must be permitted by the destination-policy bitmask stored for the key handle in the key registry.

Payload

The handle of the key to transfer in the KPV and the bitmask of the destination crypto engines.

Table 190. *Command Payload*

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]				
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1														
RESERVED																							KEY_HANDLE[7:0]												
RESERVED																							DEST_ENGINE[7:0]												

Table 191. *DEST_ENGINE* Payload

Bit	Name	Description
0	hmac_sha2	Key transfer to the HMAC engine is requested when set to 1.
1	kmac_sha3	Key transfer to the KMAC engine is requested when set to 1.
2	aes	Key transfer to the AES engine is requested when set to 1.
3	otbn	Key transfer to the OTBN engine is requested when set to 1.
4	abr_mlds_a_seed	Key transfer to the Adams Bridge ML-DSA seed is requested when set to 1.
5	abr_mlkem_seed_d	Key transfer to the Adams Bridge ML-KEM seed D is requested when set to 1.
6	abr_mlkem_seed_z	Key transfer to the Adams Bridge ML-KEM seed Z is requested when set to 1.
7	abr_mlkem_msg	Key transfer to the Adams Bridge ML-KEM message is requested when set to 1.

Response

If any arguments are invalid, this command has a **RESP_CMD** response with a return code of *invalid_arg*, and a return argument pointing to the index of the invalid payload word.

If the request cannot be satisfied for any other reason, this command has a **RESP_CMD** response with a return code of *failure*, and no return argument. If *any* requested crypto engines are disallowed, the request will result in *failure* and no requested key transfers will take place.

If the request is granted, this command has a **RESP_CMD** response with a return code of *success*, and a return argument containing the key handle and a bitfield of the crypto engines to which the key was transferred.

Table 192. **RESP_CMD** Success Return Argument

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1										
RESERVED																DEST_ENGINE[15:8]								KEY_HANDLE[7:0]							

Table 193. **DEST_ENGINE** Payload

Bit	Name	Description
0	hmac_sha2	Key transfer to the HMAC engine was completed when set to 1.
1	kmac_sha3	Key transfer to the KMAC engine was completed when set to 1.
2	aes	Key transfer to the AES engine was completed when set to 1.
3	otbn	Key transfer to the OTBN engine was completed when set to 1.
4	abr_mldsa_seed	Key transfer to the Adams Bridge ML-DSA seed was completed when set to 1.
5	abr_mlkem_seed_d	Key transfer to the Adams Bridge ML-KEM seed D was completed when set to 1.
6	abr_mlkem_seed_z	Key transfer to the Adams Bridge ML-KEM seed Z was completed when set to 1.
7	abr_mlkem_msg	Key transfer to the Adams Bridge ML-KEM message was completed when set to 1.

0x25 - CMD_ENGINE_SHRED

Description

This command will purge the key data registers from one or multiple crypto engines. These registers will be scrambled with random data. Any number of engines may be selected.

Payload

A bitfield indicating the crypto engines to purge.

Table 194. **Command** Payload

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1										
RESERVED																							DEST_ENGINE[7:0]								

Table 195. **DEST_ENGINE** Payload

Bit	Name	Description
0	hmac_sha2	Key purging of the HMAC engine is requested when set to 1.
1	kmac_sha3	Key purging of the KMAC engine is requested when set to 1.
2	aes	Key purging of the AES engine is requested when set to 1.
3	otbn	Key purging of the OTBN engine is requested when set to 1.
4	abr_mldsa_seed	Key purging of the Adams Bridge ML-DSA seed is requested when set to 1.
5	abr_mlkem_seed_d	Key purging of the Adams Bridge ML-KEM seed D is requested when set to 1.

Bit	Name	Description
6	<code>abr_mlkem_seed_z</code>	Key purging of the Adams Bridge ML-KEM seed Z is requested when set to 1.
7	<code>abr_mlkem_msg</code>	Key purging of the Adams Bridge ML-KEM message is requested when set to 1.

Response

If any arguments are invalid, this command has a `RESP_CMD` response with a return code of *invalid_arg*, and a return argument pointing to the index of the invalid payload word.

If the request cannot be satisfied for any other reason, this command has a `RESP_CMD` response with a return code of *failure*, and no return argument.

If the request is granted, this command has a `RESP_CMD` response with a return code of *success*, and a return argument indicating the crypto engines that were purged.

Table 196. *RESP_CMD Success Return Argument*

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1										
RESERVED																							DEST_ENGINE[7:0]								

Table 197. *DEST_ENGINE Payload*

Bit	Name	Description
0	<code>hmac_sha2</code>	Key material of the HMAC engine was purged when set to 1.
1	<code>kmac_sha3</code>	Key material of the KMAC engine was purged when set to 1.
2	<code>aes</code>	Key material of the AES engine was purged when set to 1.
3	<code>otbn</code>	Key material of the OTBN engine was purged when set to 1.
4	<code>abr_mldsa_seed</code>	Key material of the Adams Bridge ML-DSA seed was purged when set to 1.
5	<code>abr_mlkem_seed_d</code>	Key material of the Adams Bridge ML-KEM seed D was purged when set to 1.
6	<code>abr_mlkem_seed_z</code>	Key material of the Adams Bridge ML-KEM seed Z was purged when set to 1.
7	<code>abr_mlkem_msg</code>	Key material of the Adams Bridge ML-KEM message was purged when set to 1.

0x26 - CMD_KEY_LOAD

Description

This command loads and write-locks host-provided key material into the KPV.

Payload

The SEP must provide the key size in multiples of 32-bit words (where the key size is `KEY_SIZE+1`) and the valid destinations for the key, followed by the key material itself.

Table 198. *Command Payload*

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1										
RESERVED																							KEY_SIZE[6:0]								
RESERVED																							DEST_VALID[7:0]								
KEY_DATA[0][31:0]																															
...																															

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1										
KEY_DATA[KEY_SIZE][31:0]																															

Table 199. DEST_VALID Payload

Bit	Name	Description
0	hmac_sha2	Key transfer to the HMAC engine is permitted when set to 1.
1	kmac_sha3	Key transfer to the KMAC engine is permitted when set to 1.
2	aes	Key transfer to the AES engine is permitted when set to 1.
3	otbn	Key transfer to the OTBN engine is permitted when set to 1.
4	abr_mlds_a_seed	Key transfer to the Adams Bridge ML-DSA seed is permitted when set to 1.
5	abr_mlkem_seed_d	Key transfer to the Adams Bridge ML-KEM seed D is permitted when set to 1.
6	abr_mlkem_seed_z	Key transfer to the Adams Bridge ML-KEM seed Z is permitted when set to 1.
7	abr_mlkem_msg	Key transfer to the Adams Bridge ML-KEM message is permitted when set to 1.

On-wire payload integrity is provided by the standard mailbox **PAYLOAD_CRC32** trailer; there is no command-specific key CRC word. After the key is accepted, the firmware computes its own CRC-32C over **KEY_DATA** and stores it in the key registry for later integrity checks (see [Key Registry](#)).

Argument validation

The KM validates the payload header words in a strict, fixed order and returns **RESP_CMD** with return code *invalid_arg* at the first failure. The return argument is the fixed header-word index of the offending field (0 for **KEY_SIZE** / length, 1 for **DEST_VALID**), not an arbitrary **KEY_DATA** position.

Table 200. CMD_KEY_LOAD Validation Order

Step	Check	Return Code	Return Arg
1	PAYLOAD_LEN < 3 (cannot parse KEY_SIZE/DEST_VALID)	<i>invalid_arg</i>	0
2	KEY_SIZE reserved bits set ([31:7] != 0)	<i>invalid_arg</i>	0
3	PAYLOAD_LEN != KEY_SIZE + 3 (length inconsistent)	<i>invalid_arg</i>	0
4	DEST_VALID reserved bits set ([31:8])	<i>invalid_arg</i>	1
5	DEST_VALID == 0 (no destination selected)	<i>invalid_arg</i>	1

Capacity failure

After argument validation, the KM checks that the KPV has **KEY_SIZE+1** eligible consecutive free slots and that at least one key handle is free before mutating any KPV state. If either resource is exhausted, the command returns **RESP_CMD failure** with no return argument. A *failure* response is also returned, before the handler runs, whenever **RECOVERABLE_ERR** is set in the KMCSR. In every *failure* case the KPV state and key registry are left exactly as they were before the command arrived; the load is atomic.

Success

If the request is granted, the KM shreds the chosen slots, writes the **KEY_DATA**, write-locks the slots, stores the key CRC-32C, and allocates a new non-zero handle. The command responds with **RESP_CMD success** and a return argument containing the parameters of the loaded key: its handle, its size, and its valid destinations. The granted size always equals the requested size, and the returned handle is usable by every other key-management command with no distinction from a **CMD_KEY_GENERATE** handle.

Table 201. RESP_CMD Success Return Argument

[31] 1	[30] 1	[29] 1	[28] 1	[27] 1	[26] 1	[25] 1	[24] 1	[23] 1	[22] 1	[21] 1	[20] 1	[19] 1	[18] 1	[17] 1	[16] 1	[15] 1	[14]	[13]	[12]	[11]	[10] 1	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]	
RESERVED								DEST_VALID[23:16]								RES ERV ED	REQ_SIZE[14:8]								KEY_HANDLE[7:0]							

0x27 - CMD_ABR_SK_TRANSFER**Description**

Captures the Adams Bridge ML-KEM shared key into the KPV. Adams Bridge delivers a freshly computed ML-KEM shared key into the KM over the shared-key sideload interface and raises the shared-key interrupt; the KM firmware notifies the SEP with an unsolicited **RESP_ABR_SHARED_KEY_READY** (see [0x56 - RESP_ABR_SHARED_KEY_READY](#)). The SEP then issues this command to direct the KM to consume the pending shared key and store it as a managed KPV key.

The KM reads the 256-bit (8-word) shared key, clears the interface **KEY_VALID** (which zeroizes the source key registers), writes the key into a freshly allocated KPV slot, records the SEP-supplied **DEST_VALID** in the key registry, write-locks the slot, and returns a key handle usable by every other key-management command.

Payload

The valid crypto engine destinations for the captured key.

Table 202. Command Payload

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1										
RESERVED																							DEST_VALID[7:0]								

Table 203. DEST_VALID Payload

Bit	Name	Description
0	hmac_sha2	Key transfer to the HMAC engine is permitted when set to 1.
1	kmac_sha3	Key transfer to the KMAC engine is permitted when set to 1.
2	aes	Key transfer to the AES engine is permitted when set to 1.
3	otbn	Key transfer to the OTBN engine is permitted when set to 1.
4	abr_mldsa_seed	Key transfer to the Adams Bridge ML-DSA seed is permitted when set to 1.
5	abr_mlkem_seed_d	Key transfer to the Adams Bridge ML-KEM seed D is permitted when set to 1.
6	abr_mlkem_seed_z	Key transfer to the Adams Bridge ML-KEM seed Z is permitted when set to 1.
7	abr_mlkem_msg	Key transfer to the Adams Bridge ML-KEM message is permitted when set to 1.

Response

The KM validates the payload in a fixed order and returns **RESP_CMD** with return code *invalid_arg* (return argument 1, the **DEST_VALID** word index) if **DEST_VALID** has any reserved bit set ([31:8]) or is zero.

If no shared key is currently available (the interface **KEY_VALID** is clear), the command returns **RESP_CMD failure** with no return argument. A *failure* response is also returned if the KPV cannot allocate a slot for the key.

If the request is granted, the command returns **RESP_CMD success** with a return argument containing the handle of the newly stored key.

Table 204. RESP_CMD Success Return Argument

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1										
RESERVED																							KEY_HANDLE[7:0]								

0x28 - CMD_OTP_READ_LOCK_COLD**Description**

Sets one or more cold-reset-domain OTP field read-lock bits in the **OTP_READ_LOCK_COLD** register. The effective read-lock for each OTP field is the OR of the corresponding bit in **OTP_READ_LOCK** (warm reset domain) and **OTP_READ_LOCK_COLD** (cold reset domain). When either lock bit is set, hardware returns zero for that field's value and complement words.

Unlike **OTP_READ_LOCK**, bits set by this command survive warm reset (including soft reset) and are cleared only by cold reset. This allows the SEP host to apply permanent read-locks that persist for the lifetime of a power cycle, regardless of subsequent warm resets.

Payload

A bitmask selecting which OTP fields to lock. Bits [31:6] are reserved and must be zero; a non-zero reserved field returns **invalid_arg**. Writing 0 is a no-op; already-set bits are unaffected (write-one-to-set, idempotent).

Table 205. Command Payload

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]	
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1											
RESERVED																							CK					SU	SiU	cU	D	LC

Table 206. LOCK_BITS Field Encoding

Bit	Field	Description
0	LC	Lock OTP_LIFE_CYCLE reads.
1	D	Lock OTP_DEMOTION_STATE reads.
2	cU	Lock OTP_CHIPLET_UID_VAL/CPL reads.
3	SiU	Lock OTP_SIP_UID_VAL/CPL reads.
4	SU	Lock OTP_SYS_UID_VAL/CPL reads.
5	CK	Lock OTP_CLASS_KEY_VAL/CPL reads.

Response

If any reserved bit [31:6] is set, the command returns **RESP_CMD invalid_arg** with **return_arg** = the raw **lock_bits** word received.

On success the command applies the lock bits (triple-write convention) and returns **RESP_CMD success** with **return_arg** = the resulting **OTP_READ_LOCK_COLD** register value after the write.

Table 207. RESP_CMD Success Return Argument

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]	
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1											
RESERVED																											CK	SU	SiU	cU	D	LC

Response Message Reference

Table 208. Response Message

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
HEADER_CRC8[31:24]								PAYLOAD_LEN[23:16]								RESPONSE_ID[15:8]								RESP_SEQ_NUM[7:0]							
PAYLOAD_WORD[0][31:0]																															
...																															
PAYLOAD_WORD[n-1][31:0]																															
PAYLOAD_CRC32[31:0]																															

The KM sends response messages to the SEP using the exact same basic message container format as command messages.

The sequence numbers of response messages do not correspond to the sequence numbers of command messages. There is also no direct relationship between command IDs and response IDs.

Table 209. KM Response Summary

ID	Response	Trigger
0x00	RESP_CMD	Reply to every direct SEP command.
0x55	RESP_KM_READY	Unsolicited; KM finished init and entered main loop.
0x56	RESP_ABR_SHARED_KEY_READY	Unsolicited; Adams Bridge ML-KEM shared key is available to capture.
0xFE	RESP_RECOVERABLE_FAULT	Unsolicited; KM hit a recoverable fault.
0xFF	RESP_UNRECOVERABLE_FAULT	Unsolicited; KM hit an unrecoverable fault.

0x00 - RESP_CMD

Description

A general command response message. The key manager responds to all direct commands from the SEP with this response.

Payload

The sequence number from the source command, the source command ID, a signed 8-bit return code, and an optional argument.

Table 210. Response Payload

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1										
RESERVED																								CMD_SEQ_NUM[7:0]							
RESERVED																								COMMAND_ID[7:0]							
RESERVED																								RETURN_CODE[7:0]							
RETURN_ARG[31:0] (Optional)																															

Table 211. Return Codes

Code	Name	Description	Argument Value
0	success	General success code.	Specific to the type of source COMMAND_ID.

Code	Name	Description	Argument Value
-1	failure	General failure code.	Specific to the type of source COMMAND_ID .
-2	header_crc	The header CRC-8/ROHC of the source command was invalid. No action was taken.	The actual CRC-8/ROHC of the header that was received.
-3	cmd_noseq	The CMD_SEQ_NUM of the source command was not the expected sequence number. No action was taken.	The next expected command sequence number.
-4	invalid_cmd	COMMAND_ID did not map to a valid command. No action was taken.	None.
-5	invalid_len	The PAYLOAD_LEN of the source command did not correspond to the size of the mailbox message frame. No action was taken.	The actual length of the message payload that was received.
-6	payload_crc	The payload CRC-32C of the source command was invalid. No action was taken.	The actual CRC-32C of the payload that was received.
-7	invalid_arg	One or more payload arguments of the source command were invalid. No action was taken.	The index for the payload word of the received command with the invalid argument. If the index points to a location outside of the received payload arguments, it indicates an argument was missing.

0x55 - RESP_KM_READY

Description

An unsolicited response message that indicates the KM has completed initialization and has entered its main event loop. Serves as a proof of life indicator for the SEP.

Payload

This message has no payload.

0x56 - RESP_ABR_SHARED_KEY_READY

Description

An unsolicited response message that notifies the SEP that Adams Bridge has delivered an ML-KEM shared key into the KM and it is available to capture. The KM emits this notification from its main loop after the shared-key interrupt fires and its ISR flags the pending event. The SEP responds by issuing **CMD_ABR_SK_TRANSFER** (see [0x27 - CMD_ABR_SK_TRANSFER](#)) to store the key in the KPV and obtain a handle.

Payload

This message has no payload.

0xFE - RESP_RECOVERABLE_FAULT

Description

An unsolicited response message that indicates the KM has encountered a recoverable fault.

Payload

A signed 8-bit code indicating the cause of the fault.

Table 212. Response Payload

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1										
RESERVED																							FAULT_CODE[7:0]								

Table 213. Fault Codes

Code	Name	Description
-1	key_slot_crc	The CRC-32C check for a key slot has failed. Whatever operation triggered the fault was aborted.
-2	rx_buff_overflow	The message receive buffer has encountered a message overflow. This is caused by a singular incomplete message filling up the message receive buffer. All message buffers and mailbox FIFOs have been flushed by the KM.
-3	mbox_overflow	The KM outgoing mailbox FIFO has encountered an overflow condition. All message buffers and mailbox FIFOs have been flushed by the KM.
-4	mbox_underflow	The KM incoming mailbox FIFO has encountered an underflow condition. All message buffers and mailbox FIFOs have been flushed by the KM.
-5	flushed_by_sep	The KM mailbox was flushed by the SEP. All message buffers have been flushed by the KM. The mailbox FIFOs are not flushed.

0xFF - RESP_UNRECOVERABLE_FAULT

Description

An unsolicited response message that indicates the KM has encountered an unrecoverable fault. Core execution has halted.

Payload

A signed 8-bit code indicating the cause of the fault.

Table 214. Response Payload

[31]	[30]	[29]	[28]	[27]	[26]	[25]	[24]	[23]	[22]	[21]	[20]	[19]	[18]	[17]	[16]	[15]	[14]	[13]	[12]	[11]	[10]	[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1										
RESERVED																							FAULT_CODE[7:0]								

Table 215. Fault Codes

Code	Name	Description
-1	wipe_state	The wipe state signal was asserted and the KM has purged all sensitive data.
-2	rom_parity	The ROM encountered a parity error.
-3	sram_parity	The SRAM encountered a parity error.
-4	rom_write	The ROM encountered a write error.
-5	sram_write_lock	The SRAM encountered a write error to a write-locked region.
-6	axi_decerr	The KM system fabric encountered an AXI DECERR.
-7	axi_slverr	The KM system fabric encountered an unhandled AXI SLVERR.
-8	drbg_err	There was an error reading from the DRBG.
-9	illegal_insn	An illegal instruction was encountered.

Code	Name	Description
-10	bus_error	The CPU core encountered an internal bus error or misaligned access.
-11	ebreak	An EBREAK instruction was executed.
-12	spurious_irq	An unrecognised IRQ source was asserted.
-13	otp_sigint	The OTP controller reported a dual-rail integrity violation.
-14	fw_crc	The loaded mutable firmware image failed its CRC-32C integrity check. The KM halts; recovery requires a warm or cold reset.
-15	fw_stack_ovf	The firmware load destination reached the stack guard region boundary, indicating the SEP sent more words than declared in <code>FW_WORDS</code> . The KM halts; recovery requires a warm or cold reset.
-16	shred_range	A shred operation was requested with a word count that exceeded the shred-order buffer. The KM halts; recovery requires a warm or cold reset.
-17	exec	An instruction fetch was attempted from a non-whitelisted (non-executable) memory region (execute-permission whitelist violation). The KM halts; recovery requires a warm or cold reset.

Common Sequences

Initial Boot

As soon as the KM is taken out of reset, it begins executing out of the ROM. The ROM performs basic setup routines such as initialization of peripherals and their drivers, as well as scrambling sensitive memory locations like the KPV and crypto engine sideload key registers. Once the boot process is complete, the KM sends a liveness message to the SEP via `RESP_KM_READY` and waits for further instruction via mailbox commands.

At this point the SEP may want to query hardware and/or firmware version and status via the `CMD_HW_VER`, `CMD_ROM_VER`, and `CMD_STAT`.

SEP Key Loading

The KM supports loading host-provided key material into the KPV directly over the mailbox using the `CMD_KEY_LOAD` command. The SEP never writes KPV slots directly; the KM firmware performs all slot allocation and key writes on the SEP's behalf. The typical flow is as follows:

1. The SEP issues a `CMD_KEY_LOAD` command containing the key size, the valid crypto engine destinations, and the key material itself, then waits for a response from the KM.
2. The KM allocates a set of free KPV slots, writes the key material into them, write-locks the slots, and registers a key handle.
3. The KM responds with *success* and a return argument containing the key handle, size, and valid destinations.
4. The SEP may now use the provided key handle for any further key management operations.

Key Transfer and Sideload Purge

The SEP requests transfer of a key from the KPV to any crypto engine sideload key register via the `CMD_KEY_TRANSFER` command. The SEP provides a handle for the key and a bitmask indicating the destination sideload key registers. The KM will respond with *success* when the operation is complete. If the key handle is invalid or any requested destination sideload key registers are disallowed, the KM will respond with *failure*.

When the SEP is done with the crypto engine, it issues the `CMD_ENGINE_SHRED` command to purge key material in the key sideload registers. The KM will respond with *success* when the operation is complete.

Adams Bridge Shared-Key Capture

The Adams Bridge ML-KEM shared-key interface flows in the opposite direction from the seed and engine sideloads: Adams Bridge computes a shared key and delivers it *into* the KM. The capture flow is as follows:

1. Adams Bridge loads the 256-bit shared key into the KM shared-key interface and the hardware sets the sticky `IRQ_STATUS.key_vali_d` bit, which (when `IRQ_ENABLE` is set) raises the shared-key interrupt on PicoRV32 bit 5.
2. The KM ISR acknowledges the interrupt by clearing the sticky `IRQ_STATUS` bit (write-1-to-clear) and flags the main loop. The ISR does not consume the key; `KEY_VALID` remains set.
3. From the main loop, the KM sends the SEP an unsolicited `RESP_ABR_SHARED_KEY_READY` notification.
4. The SEP issues `CMD_ABR_SK_TRANSFER`, supplying the `DEST_VALID` for the captured key.
5. The KM reads the shared key, clears the interface `KEY_VALID` (zeroizing the source registers), writes the key into a freshly allocated KPV slot, write-locks the slot, and responds with *success* and the new key handle.
6. The SEP may then use the handle with any other key-management command, including transferring the captured key to a permitted engine.

Key Revocation

The SEP requests revocation of a key in the KPV via the `CMD_KEY_REVOKE` command. The KM will read-lock the key and remove the key from the key registry. It is now no longer accessible or transferable by either the SEP or KM.

Error Recovery

The KM can encounter a variety of recoverable and unrecoverable errors.

A recoverable error signals a condition that halts critical KM services until acknowledged. It has the following flow:

1. The KM asserts the recoverable error signal.
2. The KM flushes the mailbox *only* for any recoverable error involving the mailbox or message buffers.
3. The KM sends the `RESP_RECOVERABLE_FAULT` response message to the SEP with a code indicating the cause of the fault.
4. The KM will *not* process any command **except**:
 - a. `CMD_HW_VER`
 - b. `CMD_ROM_VER`
 - c. `CMD_SRAM_VER`
 - d. `CMD_STAT`
 - e. `CMD_RECOV_ACK`
5. The SEP takes any action it deems necessary before it clears the fault.
6. The SEP sends a `CMD_RECOV_ACK` command message to clear the fault.
7. The KM deasserts the recoverable error signal.
8. The KM resumes normal operation, handling all valid commands.

When a recoverable fault is triggered by a command handler (for example, a `key_slot_crc` fault raised while processing a command), the KM sends **both** the `RESP_RECOVERABLE_FAULT` and that command's own `RESP_CMD` error response (*failure*). The paired `RESP_CMD` follows immediately after the fault response. The SEP must drain both messages before issuing `CMD_RECOV_ACK`; otherwise a subsequent response read consumes the stale `RESP_CMD` instead of the acknowledgement's

reply. Faults raised from within the mailbox interrupt handler (`mbox_overflow`, `mbox_underflow`, `flushed_by_sep`) are not paired with a `RESP_CMD`, because the transmit buffer has been flushed and the fault message is written directly to the outgoing mailbox FIFO.

An unrecoverable error signals a condition that puts the KM into a permanent halt state. It has the following flow:

1. The KM shreds all crypto engine key sideload registers and zeroizes the Adams Bridge ML-KEM shared key.
2. The KM flushes the mailbox.
3. The KM sends the `RESP_UNRECOVERABLE_FAULT` response message to the SEP with a code indicating the cause of the fault.
4. The KM shreds the entire SRAM using one pass of a Weyl sequence followed by two passes of a xoshiro128++ PRNG.
5. The KM CPU permanently halts, which sets the unrecoverable error signal to interrupt the SEP.
6. The SEP must hard-reset the KM to bring it back through the boot sequence.

SRAM Firmware Handover

The KM ROM supports handing control from immutable ROM to mutable SRAM firmware via the `CMD_SRAM_LOAD_EXEC` (load + run) or `CMD_SRAM_EXEC` (re-run previously loaded firmware) commands.

The high-level steps for `CMD_SRAM_LOAD_EXEC` are:

1. **Validate.** The ROM checks `FW_WORDS > 0` and that the firmware image fits below the static load ceiling `km_fw_load_limit` – a build-time constant computed by the linker as `STACK - rom_max_stack`, rounded down to a 512-byte SRAM write-lock region boundary, where `_rom_max_stack` is the worst-case ROM stack depth validated against the production ELF (see [ROM Stack Budget and the Static Load Ceiling](#)). If either check fails, the command returns `_invalid_arg` and no image is requested.
2. **Confirm.** The ROM clears `sram_fw_size`, waits for the TX buffer to drain, disables inbound mailbox FIFO IRQs, and sends a `RESP_CMD` success ("confirmation") directly to the outbound FIFO. The SEP may then begin streaming the image.
3. **Stream.** The ROM polls the inbound FIFO directly (bypassing the normal message buffer), storing each word into SRAM starting at `0x4000` and accumulating a running CRC-32C. A per-word guard (defense-in-depth on top of the static `__km_fw_load_limit` validation) ensures the destination address never reaches the load ceiling.
4. **Verify.** The trailing CRC-32C word (sent by the SEP as the last word of the frame) is compared against the computed value. A mismatch triggers the `fw_crc` unrecoverable fault.
5. **Persist.** On success, `sram_fw_size` is updated to `FW_WORDS × 4`.
6. **Shred sideload keys.** The HMAC, KMAC, AES, and OTBN sideload key-share registers, together with the four Adams Bridge seed sideload interfaces (ML-DSA seed, ML-KEM seed D, ML-KEM seed Z, ML-KEM message), are overwritten with PRNG data and their `key_valid` bits cleared, and the Adams Bridge ML-KEM shared key is zeroized (`KEY_CTRL` cleared, hardware wipes `KEY[*]`). These register banks lie outside the SRAM address space, so the whole-SRAM scramble below cannot reach them; shredding here denies mutable firmware any key material the ROM previously sideloaded or any shared key delivered but not yet consumed.
7. **Lock.** OTP secret fields and the `rom_persist` SRAM region are locked, and the KPV is sealed against mutable firmware: using the key registry, every registered slot is read-locked (`lock_use`) and every free slot is overwritten with PRNG data (a free-but-write-locked slot is read-locked instead). Revoked keys are hardware-erased at revocation time, so their slots are wiped and fully unlocked (ordinary free slots) by this point. The SRAM write-lock mask is set to cover all 512-byte regions that contain the loaded image.
8. **Disable IRQs and flush the mailbox.** All KMCSR/mailbox IRQ enables are cleared, pending sticky IRQ status bits

are cleared so mutable firmware does not inherit spurious interrupts, and all CPU IRQ sources are masked. Both mailbox FIFOs are flushed to erase any residual inbound payload (e.g. key material from an earlier `CMD_KEY_LOAD`); this is safe because the outbound `RESP_CMD` has already been consumed by the SEP. ROM does not program `IRQ_ENTRY_ADDR`; the vector is left at the ROM default and the loaded firmware sets its own before re-enabling interrupts.

9. **Scramble.** A stack-less assembly routine (`rom_handover_jump.S`) seeds a `xoshiro128++` PRNG from the value passed from C, iterates over the full SRAM range `0x40000000-0x80000000`, and overwrites each address with a pseudo-random word. Hardware silently discards writes to write-locked regions (firmware image and `rom_persist`), so only ROM runtime data (stack, `.data`, `.bss`) is actually overwritten. No stack pointer or memory-resident variable is accessed during or after this step.
10. **Clear GPRs.** All general-purpose registers except the jump target (`x5`) are cleared to zero. `x5` is then used to jump, after which it too is zeroed by `x0` convention.
11. **Enable SRAM execution mode.** Immediately before the jump, the assembly performs a triple write to `KMCSR.SRAM_EXEC_MODE.enable` (write-1-only, warm-reset domain). This switches the execute-permission whitelist from ROM-only to ROM + write-locked SRAM. The triple write hardens against voltage-glitch attacks that might cause a single store to be skipped. Setting this bit as the very last operation before the jump minimises the window during which SRAM execution is enabled before the first SRAM fetch.
12. **Jump.** `jalr zero, 0(x5)` transfers control to `0x40000000`. The ROM never returns. The mutable firmware is responsible for sending its own ready message to the SEP.

For `CMD_SRAM_EXEC`, the handover begins at step 6 (no image load), provided `sram_fw_size > 0`.

ROM Stack Budget and the Static Load Ceiling

Mutable firmware is loaded from the SRAM base (`0x40000000`) and grows upward, while the ROM stack grows downward from `STACK` (the top of the ROM's `.data/.bss`). After a warm reset that follows mutable-firmware execution, the ROM re-runs and a subsequent `CMD_SRAM_EXEC` restart may drive the stack deeper than it was when the image was first loaded. The maximum image size must therefore be bounded by the **worst-case** ROM stack depth, not by the live stack pointer at load time.

The bound is enforced statically:

- The ROM firmware is built with a fully **static** stack — no variable-length arrays or `alloca`. All scratch buffers are fixed-size locals or file-scope `static`.
- `tools/km_stack_analyze.py` walks the disassembled call graph of the linked production ELF (rooted at `main`, and separately at the dedicated IRQ entry `rom_irq`) and sums per-function stack frames along the deepest path. It fails the build if it finds a dynamic frame, an unresolved indirect call on a reachable path, or a call-graph cycle.
- The firmware `Makefile` passes the budget to the linker via `--defsym=__rom_max_stack=$(ROM_KM_MAX_STACK_BYTES)` (default `0x600`) and gates `build_production` on the analyzer reporting a worst case within that budget. `make stack_report` prints the worst-case depth and deepest path.
- The linker (`km_sram_layout.ld`) derives the load ceiling `km_fw_load_limit = (STACK - rom_max_stack)`, rounded down to a 512-byte SRAM write-lock region so the image and ROM stack never share a lock region, and asserts that a non-empty load area remains. `rom_cmd_sram_load_exec()` rejects any image whose top exceeds this symbol.

Run `make stack_report` to see the analyzer's current worst-case main and IRQ stack depths and deepest paths for the production ELF. Raise `ROM_KM_MAX_STACK_BYTES` only if the analyzer reports a worst case that exceeds the configured budget.

Bootloader

Boot Sequence

The ROM bootloader will perform the following sequence of steps:

1. Enable all fault and error related interrupts.
2. Initialize the DRBG sampler.
3. Seed the firmware PRNG from the DRBG.
4. If the SRAM scrambler is not enabled:
 - a. Write the SRAM scrambler key register `SHRED_ITER+1` number of times with random words sourced from the DRBG.
 - b. Using inline assembly, enable & lock the SRAM scrambler then restart CPU execution (jump to address 0).
5. Initialize the KPV.
 - a. Initialize the KPV scrambler.
 - b. Enable the KPV scrambler.
 - c. Lock the KPV scrambler.
 - d. Shred all KPV slots.
6. Initialize the HMAC engine key interface.
 - a. Shred the HMAC key registers.
7. Initialize the KMAC engine key interface.
 - a. Shred the KMAC key registers.
8. Initialize the AES engine key interface.
 - a. Shred the AES key registers.
9. Initialize the OTBN engine key interface.
 - a. Shred the OTBN key registers.
10. Initialize the message buffers.
11. Clear and disable the outgoing mailbox FIFO interrupt.
12. Clear and enable the incoming mailbox FIFO interrupt.
13. Send a `RESP_KM_READY` response message to the SEP.
14. Finally, enter the main event loop.

Message Buffers

The message receive and transmit buffers are fixed size SRAM-based circular buffers that store message frames that have been read from or are ready to be written to the mailbox FIFOs. The buffers must also keep track of pointers to the start and end of each message frame stored in the buffer. Since the KM firmware does not have a heap, the KM must allocate global or dynamic memory for message receive and transmit buffers. Since the maximum message payload length is 255 words, each message buffer is 257 words in size (1 header word + 255 payload words + 1 CRC-32C word) and a single message buffer can store multiple messages.

All access to the message buffers must go through a well defined interface. The interface must define functions for

initializing, reading, writing, and getting status from a message buffer. The interface must support reading and writing partial message frames, and it must prevent buffer overflow and underflow. In order to avoid synchronization issues, the mailbox IRQs must be disabled during buffer read/write operations and then returned to their previous state once complete.

Pseudorandom Generators

Firmware PRNG

The KM firmware shall use the xoshiro128++ algorithm as its general-purpose pseudorandom number generator. The PRNG is used for generating shred data and seeding pseudorandom-order operations. It must not be used for generating cryptographic key material; all key material must be derived from a master key or sourced directly from the DRBG.

State

The PRNG maintains a 128-bit internal state consisting of four 32-bit words, denoted `s[0]`, `s[1]`, `s[2]`, and `s[3]`.

Seed

The PRNG is seeded by reading four consecutive words from the DRBG and assigning them to `s[0]` through `s[3]`. An all-zero state is an absorbing fixed point of xoshiro128++ and must be avoided. If all four seed words are zero, `s[0]` must be set to 1 before use. The PRNG is reseeded on-demand from the DRBG via a function call.

Implementation Constraints

Since the PRNG is used for SRAM shredding, the PRNG routines and the PRNG state must reside entirely in ROM code and CPU registers, respectively. The SRAM shred routine and all PRNG routines must be implemented using inline functions and assembly. The PRNG state, write pointer, loop bound, and temporaries must be held in CPU registers for the duration of the SRAM shred loop. The PRNG state must not be stored in SRAM at any point during SRAM shredding.

Register Access Shuffling

All operations requiring pseudorandom-order register access must use the Fisher-Yates shuffle to generate the access order. This includes all shredding and key write operations on the KPV, crypto engine key registers, SRAM scrambler key, and KPV scrambler key. The shuffle operates on an index array initialized to `[0, 1, ..., n-1]` and produces a uniformly random permutation.

Index Generation

The random index `j` in the range `[0, bound)` is generated using bit-masked rejection sampling from a PRNG-fed bit pool. This avoids division and eliminates modulo bias.

The bit pool is a 32-bit register refilled from the PRNG whenever insufficient bits remain. Bits are consumed from LSB to MSB.

Interrupt Handlers

The PicoRV32 has a very basic interrupt controller that does not support hardware interrupt priority or nested interrupts. Due to these limitations, interrupt priority must be managed by firmware and all interrupt handlers must be kept as short as possible to minimize interrupt response latency. The main ISR must process all currently active interrupts within a single call.

Interrupt Priority

Interrupt priority must be handled by firmware within the PicoRV32's main interrupt service routine (ISR). Any new interrupts that arrive while the PicoRV32 ISR is executing will trigger a pending interrupt signal that will then re-execute the main ISR after it returns. Since higher priority interrupts cannot preempt lower priority interrupts, short and deterministic ISR execution time is important.

The ISR should prioritize interrupts in the following order, from highest priority to lowest priority:

1. EBREAK / Illegal instruction
2. Bus error or misaligned access
3. Wipe state
4. ROM parity error
5. SRAM parity error
6. ROM write error
7. SRAM write lock error
8. AXI DECERR
9. AXI SLVERR
10. DRBG error
11. OTP integrity violation
12. OTP change status
13. Mailbox
14. Adams Bridge ML-KEM shared key
15. Spurious IRQ (catch-all for unrecognised sources)

EBREAK / Illegal Instruction

The EBREAK / illegal instruction interrupt handler distinguishes the faulting instruction by reading the opcode at the saved PC. An EBREAK instruction triggers an unrecoverable fault with the `ebreak` fault code. Any other illegal instruction triggers an unrecoverable fault with the `illegal_insn` fault code. The interrupt status bit is not cleared.

Bus Error or Misaligned Access

The bus error or misaligned access interrupt handler will trigger an unrecoverable fault. The interrupt status bit is not cleared. This behavior may change in future firmware revisions.

Wipe State

The wipe state interrupt handler will purge keys from all crypto engines, trigger an unrecoverable fault, and finally shred the entire SRAM. Hardware will automatically clear the KPV, so firmware can skip it. The interrupt status bit is not cleared.

SRAM shredding is done last, and must be implemented entirely within assembly code that does not perform any SRAM read operations. Any operations after SRAM shredding must also be implemented in assembly since all program state (stack, etc.) has been destroyed.

NOTE

To ensure the wipe state process does not hang, the PRNG is *never* reseeded from the DRBG during any shred operation in the wipe state flow. This diverges from typical shred operations that normally reseed the PRNG from the DRBG.

ROM Parity Error

The ROM parity error interrupt handler will trigger an unrecoverable fault. The interrupt status bit is not cleared. This behavior may change in future firmware revisions.

SRAM Parity Error

The SRAM parity error interrupt handler will trigger an unrecoverable fault. The interrupt status bit is not cleared. This behavior may change in future firmware revisions.

ROM Write Error

The ROM write error interrupt handler will trigger an unrecoverable fault. The interrupt status bit is not cleared. This behavior may change in future firmware revisions.

SRAM Write Lock Error

The SRAM write lock error interrupt handler will trigger an unrecoverable fault. The interrupt status bit is not cleared. This behavior may change in future firmware revisions.

AXI DECERR

The AXI DECERR interrupt handler will trigger an unrecoverable fault. The interrupt status bit is not cleared. This behavior may change in future firmware revisions.

AXI SLVERR

The AXI SLVERR interrupt handler will trigger an unrecoverable fault. The interrupt status bit is not cleared. This behavior may change in future firmware revisions.

DRBG Error

The DRBG error interrupt handler will trigger an unrecoverable fault. The interrupt status bit is not cleared. This behavior may change in future firmware revisions.

OTP Integrity Violation

The OTP integrity violation interrupt handler will trigger an unrecoverable fault with the `otp_sigint` fault code. This interrupt is asserted when the OTP controller detects a dual-rail integrity violation on a redundantly-encoded OTP field. The interrupt status bit is not cleared. This behavior may change in future firmware revisions.

OTP Change Status

The OTP change status interrupt is an informational (non-fault) source that indicates one or more monitored OTP fields have changed. Unlike the fault handlers, this handler does not halt the KM. It has the following execution flow:

1. Read the OTP change status to determine which fields changed.

2. Invoke the OTP change hook to take any firmware-defined action. In the ROM this is a weak no-op hook intended to be overridden by later firmware.
3. Clear (write-1-to-clear) the change status bits that were handled.

Mailbox

The mailbox interrupt handler transfers message frames between the mailbox FIFOs and the message buffers. It also handles any other interrupts raised by the mailbox due to error or status conditions. It does not check the properties of incoming messages or construct the headers of outgoing messages, as this is the job of the message handlers.

The mailbox interrupt handler has the following execution flow:

1. Check for and handle any mailbox error interrupts.
 - a. If the **OUTBOUND_OVERFLOW** interrupt is enabled and asserted:
 - i. Set the **RECOVERABLE_ERR** register of the KMCSR.
 - ii. Flush *all* message buffers and mailbox FIFOs.
 - iii. Message sequence numbers are reset to zero.
 - iv. A **RESP_RECOVERABLE_FAULT** response is sent to the SEP with a *mbx_overflow* fault code through the mailbox FIFOs directly. The message transmit buffer is bypassed.
 - v. Clear the **OUTBOUND_OVERFLOW** interrupt status bit.
 - vi. Enable the incoming mailbox FIFO interrupt if necessary.
 - vii. Return immediately.
 - b. If the **INBOUND_UNDERFLOW** interrupt status is enabled and asserted:
 - i. Set the **RECOVERABLE_ERR** register of the KMCSR.
 - ii. Flush *all* message buffers and mailbox FIFOs.
 - iii. Message sequence numbers are reset to zero.
 - iv. A **RESP_RECOVERABLE_FAULT** response is sent to the SEP with a *mbx_underflow* fault code through the mailbox FIFOs directly. The message transmit buffer is bypassed.
 - v. Clear the **INBOUND_UNDERFLOW** interrupt status bit.
 - vi. Enable the incoming mailbox FIFO interrupt if necessary.
 - vii. Return immediately.
 - c. If the **FLUSHED_BY_SEP** interrupt status is enabled and asserted:
 - i. Set the **RECOVERABLE_ERR** register of the KMCSR.
 - ii. Flush *all* message buffers, but do not flush the mailbox FIFOs.
 - iii. Message sequence numbers are reset to zero.
 - iv. A **RESP_RECOVERABLE_FAULT** response is sent to the SEP with a *flushed_by_sep* fault code through the mailbox FIFOs directly. The message transmit buffer is bypassed.
 - v. Clear the **FLUSHED_BY_SEP** interrupt status bit.
 - vi. Enable the incoming mailbox FIFO interrupt if necessary.
 - vii. Return immediately.
2. Check for any pending words in the message transmit buffer.

- a. Calculate the minimum between the number of words currently in the message transmit buffer, and the available capacity of the outgoing mailbox FIFO.
 - b. Transfer this determined count of words from the message transmit buffer into the outgoing mailbox FIFO.
 - c. Set the separator bit in the FIFO before writing the final message frame word.
 - d. After words are transferred, if the message transmit buffer is empty, disable the outgoing mailbox FIFO interrupt.
3. Check for any pending words in the incoming mailbox FIFO.
 - a. Calculate the minimum between the number of words currently in the incoming mailbox FIFO, and the available capacity of the message receive buffer.
 - b. Transfer this determined count of words from the incoming mailbox FIFO into the message receive buffer.
 - c. Check the separator bit after each word is moved from the FIFO to the buffer. The separator bit is aligned with the last word of each message frame.
 - d. Record pointers for the start/end words of each message frame.
 - e. After the words are transferred, if the message receive buffer is full, disable the incoming mailbox FIFO interrupt.

Adams Bridge ML-KEM Shared Key

The Adams Bridge shared-key interrupt is an informational (non-fault) source delivered on its own PicoRV32 interrupt (bit 5), separate from the KMCSR-aggregated and mailbox interrupts. It signals that Adams Bridge has delivered a new ML-KEM shared key into the KM. The handler is intentionally minimal:

1. Acknowledge the interrupt by clearing the sticky `IRQ_STATUS.key_valid` bit (write-1-to-clear) in the shared-key sideload register block.
2. Set a pending-notify flag for the main loop.

The handler does not read or consume the shared key — `KEY_VALID` stays set so the key remains available for the subsequent SEP-issued `CMD_ABR_SK_TRANSFER`. Because the interrupt status is sticky and cleared via W1C (rather than by disabling `IRQ_ENABLE`), a later shared-key delivery re-arms the interrupt automatically. The main loop sends `RESP_ABR_SHARED_KEY_READY` to the SEP when it observes the pending-notify flag.

Spurious IRQ

After all known IRQ sources have been serviced, the ISR checks for any unrecognised bits in the IRQ mask. If any are set, the ISR triggers an unrecoverable fault with the `spurious_irq` fault code. This ensures security-critical firmware never silently ignores unexpected interrupt sources.

Incoming Message Handler

The incoming message handler parses incoming message frames from the message receive buffer in SRAM, checks their properties, and calls the associated command handler. The incoming message handler is called as part of the main task loop of the KM system firmware, and it processes a single message frame per-call to the function. It must ensure that invalid or malformed messages are not processed by the KM. It must use functions provided by the message buffer interface and may not manipulate the message buffer directly. Partial message frames, i.e. message frames that have not been fully received with a message separator indicator, are skipped.

The incoming message handler has no arguments. It has the following execution flow:

1. If the message receive buffer is full AND it only contains a single partial message frame:
 - a. Set the `RECOVERABLE_ERR` register of the KMCSR.
 - b. Flush *all* message buffers and mailbox FIFOs.
 - c. A `RESP_RECOVERABLE_FAULT` response is sent to the SEP with an `rx_buff_overflow` fault code.
 - d. Enable the incoming mailbox FIFO interrupt if necessary.
 - e. Return immediately.
2. If the message receive buffer is empty OR it only contains a single partial message frame:
 - a. Enable the incoming mailbox FIFO interrupt if necessary.
 - b. Return immediately.
3. Acquire the next fully valid message frame from the message receive buffer.
4. If the message header CRC-8/ROHC is invalid:
 - a. A `RESP_CMD` response is sent back to the SEP with a `header_crc` return code and the handler returns.
5. If the sequence number is not the next expected value:
 - a. A `RESP_CMD` response is sent back to the SEP with a `cmd_noseq` return code and the handler returns.
6. If the command ID is not a valid command:
 - a. A `RESP_CMD` response is sent back to the SEP with an `invalid_cmd` return code and the handler returns.
7. If the payload length is not consistent with the message frame length counted during transfer from the incoming mailbox FIFO to the message receive buffer:
 - a. A `RESP_CMD` response is sent back to the SEP with an `invalid_len` return code and the handler returns.
8. If the message has a payload and the payload CRC-32C is invalid:
 - a. A `RESP_CMD` response is sent back to the SEP with a `payload_crc` return code and the handler returns.
9. If the `RECOVERABLE_ERR` bit is set in the KMCSR:
 - a. Only allow the following commands:
 - i. `CMD_HW_VER`
 - ii. `CMD_ROM_VER`
 - iii. `CMD_SRAM_VER`
 - iv. `CMD_STAT`
 - v. `CMD_RECOV_ACK`
 - b. A `RESP_CMD` response is sent back to the SEP with the return code and return argument from the command handler.
 - c. Respond to all other commands with a `RESP_CMD` response of *failure* and do not execute the command.
10. If the `RECOVERABLE_ERR` bit is *not* set in the KMCSR:
 - a. Call the command handler corresponding to the command ID and include the following arguments:
 - i. The length of the message payload.
 - ii. A structure containing the message payload.
 - b. A `RESP_CMD` response is sent back to the SEP with the return code and return argument from the command handler.

11. Enable the incoming mailbox FIFO interrupt if necessary.
12. Return.

Outgoing Message Handler

The outgoing message handler constructs a message from a given response payload and writes it into the message transmit buffer in SRAM. The outgoing message handler is called from command handlers and other system functions to send responses to the SEP. It must use functions provided by the message buffer interface and may not manipulate the message buffer directly.

The outgoing message handler takes a response ID and a payload as arguments. It has the following execution flow:

1. The next message sequence number is inserted into the message header.
2. The response ID is inserted into the message header.
3. The payload length is calculated and inserted into the message header.
4. The message header CRC-8/ROHC is calculated and inserted into the header.
5. If the payload length is nonzero, the payload CRC-32C is calculated and appended to the message.
6. The entire message frame is written into the message transmit buffer. If the buffer is full, enable the outgoing mailbox FIFO interrupt and block (spin-wait) until enough free space is available to write the entire frame. Further execution depends on the SEP draining the outgoing mailbox FIFO.
7. Enable the outgoing mailbox FIFO interrupt if necessary.
8. Return.

Command Handlers

Each supported KM command has an associated command handler function. They are called by the incoming message handler, validate the arguments provided in the command message, and then call the appropriate system or key management function. If a command handler encounters an error it will return with an error code and an optional return argument. This includes errors encountered during payload parsing. Otherwise, if there are no errors it will return a *success* code and provide an optional return argument.

DRBG Driver

Initialize DRBG

Configures the DRBG and waits for it to be ready. It has the following execution flow:

1. Wait for **DRBG_READY** in the STATUS register.

Get Status

Returns the value of the STATUS register.

Get Word

Returns a single random word from the DRBG.

Get Block

Gets a block of random words from the DRBG, given a number of words to read. It has the following execution flow:

1. Enable DRBG prefetch.
2. Read the given number of random words.
3. Disable DRBG prefetch.
4. Return the random words.

KPV Driver

These functions provide firmware abstractions for the KPV.

Initialize Scrambler

If unlocked, writes the KPV scrambler key register **SHRED_ITER+1** number of times with random words sourced from the DRBG.

Scrambler Enable

Enables the KPV scrambler.

Scrambler Lock

Locks the KPV scrambler.

Erase Slot

Hardware-erases the key at a given base slot index. The function uses the **EXTEND** field of the base slot control register to span all associated key slots, and for each spanned slot:

1. Asserts the slot's **CTRL.erase** trigger. The write-1 trigger is issued three times per slot so a single skipped store (e.g. from a fault-injection glitch) cannot prevent the erase from starting.
2. Polls the slot's **erase** bit until the hardware self-clears it.

The hardware overwrites every key word of each slot (through the KPV scrambler) and clears the slot control register, including the lock bits. Erase is not blocked by **lock_write** or **lock_use**, so the slots are wiped and reusable when the function returns.

Shred Slot

Shreds a single key slot by invoking **Erase Slot** on it **SHRED_ITER+1** times. Because it uses the hardware erase path it also wipes write-locked slots and clears their control registers.

Shred KPV

Shreds every KPV slot by invoking **Shred Slot** on each slot, so each slot is hardware-erased **SHRED_ITER+1** times. Write-locked slots are wiped too.

Write Key

Writes key data and control information to a range of slot indexes, given the following:

- A base slot index for storing the key

- The key data
- The key length (in 32-bit words) It has the following execution flow:
 1. If any required key slots are write locked:
 - a. The function returns an error. No key data is written to the KPV.
 2. The key is written to the KPV, starting with the given base slot index.
 - a. The total number of required key slots (**EXTEND+1**) is equal to $(\text{KEY_LENGTH} - 1) / 16 + 1$.
 - b. The 4-bit **LAST_DWORD** field of the final key slot is equal to $(\text{KEY_LENGTH} \% 16 == 0) ? 15 : ((\text{KEY_LENGTH} \% 16) - 1)$.
 - c. The **LAST_DWORD** field of all other required key slots must be set to 15.

Read Key

Reads key data and control information, given a base slot index. The function returns the following:

- The key data
- The key length (in 32-bit words) It has the following execution flow:
 1. If any required key slots are read locked:
 - a. The function returns an error. No key data is returned from the KPV.
 2. The key is read from the KPV, starting with the given base key slot.
 - a. The total number of required key slots is equal to **EXTEND + 1**, where **EXTEND** comes from the base key slot.
 - b. The number of words in the final key slot is equal to **LAST_DWORD + 1**, where **LAST_DWORD** comes from the final key slot.
 - c. **LAST_DWORD** for all other required key slots *must* be equal to 15.
 - d. The total key length (in 32-bit words) is thus $16 * \text{EXTEND} + (\text{LAST_DWORD} + 1)$, where **EXTEND** comes from the base key slot and **LAST_DWORD** comes from the final key slot.

Write-Lock Key

Write-locks the key at a given base slot index. The function uses the **EXTEND** field of the base slot control register to lock all associated key slots.

Read-Lock Key

Read-locks the key at a given base slot index. The function uses the **EXTEND** field of the base slot control register to lock all associated key slots.

Crypto Engine Drivers (HMAC / KMAC / AES / OTBN / Adams Bridge)

Each crypto engine (HMAC, KMAC, AES, OTBN) and each of the four Adams Bridge seed inputs (ML-DSA seed, ML-KEM seed D, ML-KEM seed Z, ML-KEM message) has a driver that manages key material for that destination. All of these drivers expose the same interface and share the same execution flow; they differ only in the engine they target.

Shred Key

Purges the sideload key share registers with random data from the PRNG. It has the following execution flow:

1. Clears the key valid bit.
2. Writes random data sourced from the PRNG to both key share registers.
 - a. Reseeds the PRNG from the DRBG.
 - b. Writes the registers with random data from the PRNG across both key share registers in a pseudorandom order that is seeded from the PRNG. Write order should be randomly interleaved across both key share registers to enhance obfuscation.
 - c. Performs this entire sequence a total of `SHRED_ITER+1` times.

Write Key

Writes key data to the sideload key share registers in a pseudorandom order that is seeded from the PRNG. The two key share registers XOR together within the accelerator to produce the real key. It has the following execution flow:

1. Pads the given key, `KEY`, with random data from the DRBG up to the full length of one of the sideload key share registers.
2. Collect a random array of data, `RAND`, from the DRBG the same size as the padded key.
3. For each word, `i`: `KEY_RAND[i] = XOR(KEY[i], RAND[i])`.
4. Writes `RAND` to `KEY_SHARE0` and `KEY_RAND` to `KEY_SHARE1` in a pseudorandom order that is seeded from the PRNG. Write order should be randomly interleaved across both key share registers to enhance obfuscation.
5. Sets the key valid bit.

Adams Bridge ML-KEM shared-key driver

The Adams Bridge ML-KEM shared-key interface is the one inbound path (Adams Bridge → KM); it is read-only to firmware and consumed rather than written:

- **Read (consume) shared key.** Reads the eight 256-bit key words into a caller buffer, then writes `KEY_CTRL = 0` to clear `KEY_VALID`, which triggers a hardware zeroize of the source key registers. Invoked by the `CMD_ABR_SK_TRANSFER` handler.
- **Acknowledge interrupt.** Clears the sticky `IRQ_STATUS.key_valid` bit by writing 1 (W1C). Called by the shared-key ISR to acknowledge the interrupt without consuming the key.
- **Interrupt enable / disable.** Set or clear the `IRQ_ENABLE.key_valid_en` bit. The masked CPU interrupt is `IRQ_STATUS & IRQ_ENABLE`; firmware enables it during boot and leaves it enabled, relying on the sticky-status W1C acknowledgement rather than toggling the enable per event.

Key Derivation Function Library

The `rom_kdf` library implements the NIST SP 800-108r1 counter-mode key derivation function (KDF) with HMAC-SHA-256 as its pseudorandom function. The KDF is defined by the following construction, where `i` and `L` are unsigned 16-bit big-endian values and `L` is the requested output length in bits:

```
K(i) = HMAC-SHA-256(key, BE16(i) || input[192] || BE16(L))
out  = leftmost L bits of K(1) || K(2) || ...
```

The counter starts at 1. The library computes `ceil(L / 256)` HMAC iterations and copies `ceil(L / 8)` bytes to the caller's output buffer. `L` is in the range 1 through 8192 bits. If `L` is not byte-aligned, the final output byte is copied in full and the

caller ignores the bits beyond `L`. Because `L` is included in every HMAC message, outputs requested at different lengths are cryptographically separated; a shorter output is not a prefix of a longer output.

The input is a fixed 192-byte `km_kdf_input_t`. The library copies this structure into the HMAC message verbatim. Structure fields use the PicoRV32 in-memory byte representation; only the surrounding counter and output-length fields are big-endian.

Table 216. KDF Fixed Input Layout

Byte Offset	Field	Description
0-1	<code>version</code>	KM ABI version mixed into the derivation.
2	<code>out_class</code>	Class of the derived key.
3	<code>out_type</code>	Type of the derived key.
4	<code>out_owner</code>	Owner of the derived key.
5	<code>out_domain</code>	Parent-inherited domain, forged by the KM.
6-7	<code>flags</code>	Provenance and operational flags, forged by the KM.
8-9	<code>purpose</code>	Public purpose metadata.
10-11	<code>out_bits</code>	Requested derived-key size recorded in the input.
12-15	<code>caps</code>	Capabilities requested for the derived key.
16-19	<code>device_state</code>	Live life-cycle and demotion state sampled by the KM.
20-31	<code>rsvd</code>	Reserved; all bytes are zero.
32-63	<code>label</code>	Fixed operation label providing domain separation.
64-127	<code>context</code>	Caller- or ROM-provided derivation context.
128-191	<code>entropy</code>	DRBG input for ephemeral operations; otherwise all zero.

`rom_kdf_800_108()` uses `rom_hmac_sha256()` for each counter iteration. An HMAC key of at most 64 bytes is passed directly to HMAC. A longer key is first reduced to a 32-byte SHA-256 digest as specified by RFC 2104. This normalization permits parent keys up to the KM symmetric-key limit of 1024 bytes while retaining the HMAC library's 64-byte key limit.

The KDF rejects null required pointers, a null key with non-zero length, and output lengths outside the supported range. Invalid arguments leave the output buffer unchanged. If SHA-256 or HMAC fails after validation, the KDF zeroes the complete requested output region before returning an error. The implementation also securely clears its normalized key, HMAC message, and digest scratch buffers before returning.

`rom_kdf_selftest()` provides the KDF power-on known-answer test. It derives a 512-bit output from an embedded 32-byte key and 192-byte input, thereby exercising two counter iterations, and compares the result with the embedded expected value. A boot flow that derives real key material must run this test first and fail closed if it does not pass.

CRC Function Library

The CRC function library contains routines for CRC computations. The high-level functions (`rom_crc8_rohc`, `rom_crc32c`) preserve their public signatures and golden-vector results, but are implemented on top of the hardware CRC accelerator rather than software lookup loops: each consumes one accelerator instruction per input unit (one byte per CRC-8/ROHC call, one 32-bit word per CRC-32C word call, and one byte per CRC-32C tail byte). The accelerator returns the CRC-32C running remainder before the final XOR, so `rom_crc32c` seeds `0xFFFF_FFFF`, chains word and tail-byte updates, and applies the final XOR once at the end. See the CRC Accelerator section of the architecture documentation for the instruction encodings and engine behavior.

CRC-8/ROHC

Compute the CRC-8/ROHC for a given block of data.

Table 217. CRC-8/ROHC Parameters

Parameter	Value
Width	8
Poly	0x07
Init	0xFF
RefIn	true
RefOut	true
XorOut	0x00
Check ("123456789")	0xD0

The CRC-8/ROHC is computed over the input bytes in little endian order, from least significant byte to most significant byte.

CRC-32C

Compute the CRC-32C (Castagnoli) for a given block of data.

Table 218. CRC-32C Parameters

Parameter	Value
Width	32
Poly (normal)	0x1EDC6F41
Poly (reflected)	0x82F63B78
Init	0xFFFFFFFF
RefIn	true
RefOut	true
XorOut	0xFFFFFFFF
Check ("123456789")	0xE3069283

The CRC-32C is computed over the input bytes in little endian order, from least significant byte to most significant byte.

Key Registry

KM firmware maintains a mapping between key handles and key slots in the KPV, called the key registry. Key handles are unique 8-bit numbers that act as aliases to key slots in the KPV. Handle 0x00 is reserved for "null", i.e. no handle is assigned. Key handles are simply incremented as they are allocated in the registry. Since there are only 32 key slots, key handle assignment can technically never accidentally roll over to 0. However, to be safe, the key registry should still check for key exhaustion before allocating new keys. Key handles cannot be reused, and are permanently invalidated when they are destroyed.

There are two data structures in the key registry:

- A mapping from key handles to a base slot index, a CRC-32C value for the associated key, and an 8-bit **DEST_VALID** destination-policy bitmask.
- A mapping from slot indexes to a key handle. A key handle value of NULL (0x00) indicates the slot does not have an

associated key handle.

All access to the key registry must go through a well defined interface. The following functions manipulate the key registry.

Get Key Handle

Returns the registered key handle, given a key slot index. The given key slot index does not need to be the base slot for the key. Returns an error if the key slot does not have an associated handle, i.e. the handle is NULL (0x00).

Get Key Slot

Returns the registered base key slot index, given a key handle. Returns an error if the key handle does not exist or has been invalidated.

Get Key CRC

Returns the registered CRC-32C for a key, given a key handle. Returns an error if the key handle does not exist or has been invalidated.

Get Key Destinations

Returns the registered **DEST_VALID** destination-policy bitmask for a key, given a key handle. Returns an error if the key handle does not exist or has been invalidated.

Generate Handle

Returns and registers a unique 8-bit key handle, given the following:

- The base slot index for a key
- A CRC-32C of the key data
- The valid crypto engine destinations for the key

It has the following execution flow:

1. Returns with an error if any of the key's slots are already registered.
2. Selects a unique 8-bit key handle not invalidated or currently assigned to any existing key slot.
 - a. Walk through the registry and find the next unassigned key handle.
 - b. Return with an error if no key handles are available.
3. Updates the registry with the generated key handle, CRC-32C, and destination-policy bitmask.

Destroy Handle

Invalidates a given key handle in the registry. The key handle may not be used again. Returns with an error if the given key handle does not exist.

Key Management

These functions provide high-level key management operations. In general, they are called by command handler functions.

Check Key

Checks the integrity of a stored key, given a key handle. Returns an error if the key handle does not exist or the CRC-32C is invalid. Triggers a recoverable fault if the CRC-32C is invalid. It has the following execution flow:

1. Get the key slot and CRC-32C for the given key handle, returning with an error if invalid.
2. Calculate the CRC-32C for the key data located in the key slot.
3. If the calculated CRC-32C does not match the CRC-32C stored in the key registry:
 - a. Set the **RECOVERABLE_ERR** register of the KMCSR.
 - b. A **RESP_RECOVERABLE_FAULT** response is sent to the SEP with a *key_slot_crc* fault code.
 - c. Return with an error.

Generate Key

Generates a random key, given the following:

- The size of the key in multiples of 32-bit words
- Valid crypto engine destinations for the key

It has the following execution flow:

1. Finds a set of unoccupied consecutive key slots, returning with an error if none are available.
 - a. The base key slot must be selected randomly using data from the DRBG.
 - b. Slots allocated to the SEP must be avoided.
 - c. Selected slots must not be write-locked or read-locked.
 - d. Selected slots must not be associated with an existing key handle.
2. Shreds all slots associated with the key.
3. Collects random data from the DRBG for the key data.
4. Writes the key to the KPV.
5. Write-locks all slots associated with the key.
6. Generates and returns a handle for the key.

Load Key

Loads host-provided key material into the KPV, given the following:

- The size of the key in multiples of 32-bit words
- Valid crypto engine destinations for the key
- The key data

It has the following execution flow:

1. Finds a set of unoccupied consecutive key slots, returning with an error if none are available.
 - a. The base key slot must be selected randomly using data from the DRBG.
 - b. Selected slots must not be write-locked or read-locked.

- c. Selected slots must not be associated with an existing key handle.
2. Shreds all slots associated with the key.
3. Writes the provided key data to the KPV.
4. Write-locks all slots associated with the key.
5. Generates and returns a handle for the key.

Revoke Key

Revokes a key, given a key handle. It has the following execution flow:

1. Gets the base slot index associated with the key handle, returning with an error if invalid.
2. Hardware-erases all slots associated with the key: each slot's `CTRL.erase` trigger is asserted, and firmware waits until the hardware self-clears it. The erase overwrites the key data through the KPV scrambler and clears each slot's CTRL register (including the lock bits), so the slots are wiped and reusable.
3. Destroys the given key handle.
4. Returns success.

Transfer Key

Transfers a key from the KPV to one or more crypto engines, given the following:

- The handle for the key
- A bitfield of crypto engine destinations for the key

It has the following execution flow:

1. Gets the base slot index associated with the key handle, returning with an error if invalid.
2. Checks the integrity of the key associated with the given handle, returning with an error if invalid.
3. Gets the destination-policy bitmask from the key registry and checks that **all** requested crypto engine destinations are allowed, returning with an error if any are invalid.
4. Transfers the key to all requested crypto engines.
5. Returns success.

Main Event Loop

After the bootloader initializes the KM, the main loop of the KM continuously monitors the incoming message buffer for commands. When the incoming message buffer is empty, the main loop should wait for an interrupt. All other tasks are handled by the main ISR.

The KM processes commands strictly in the order that they are received.

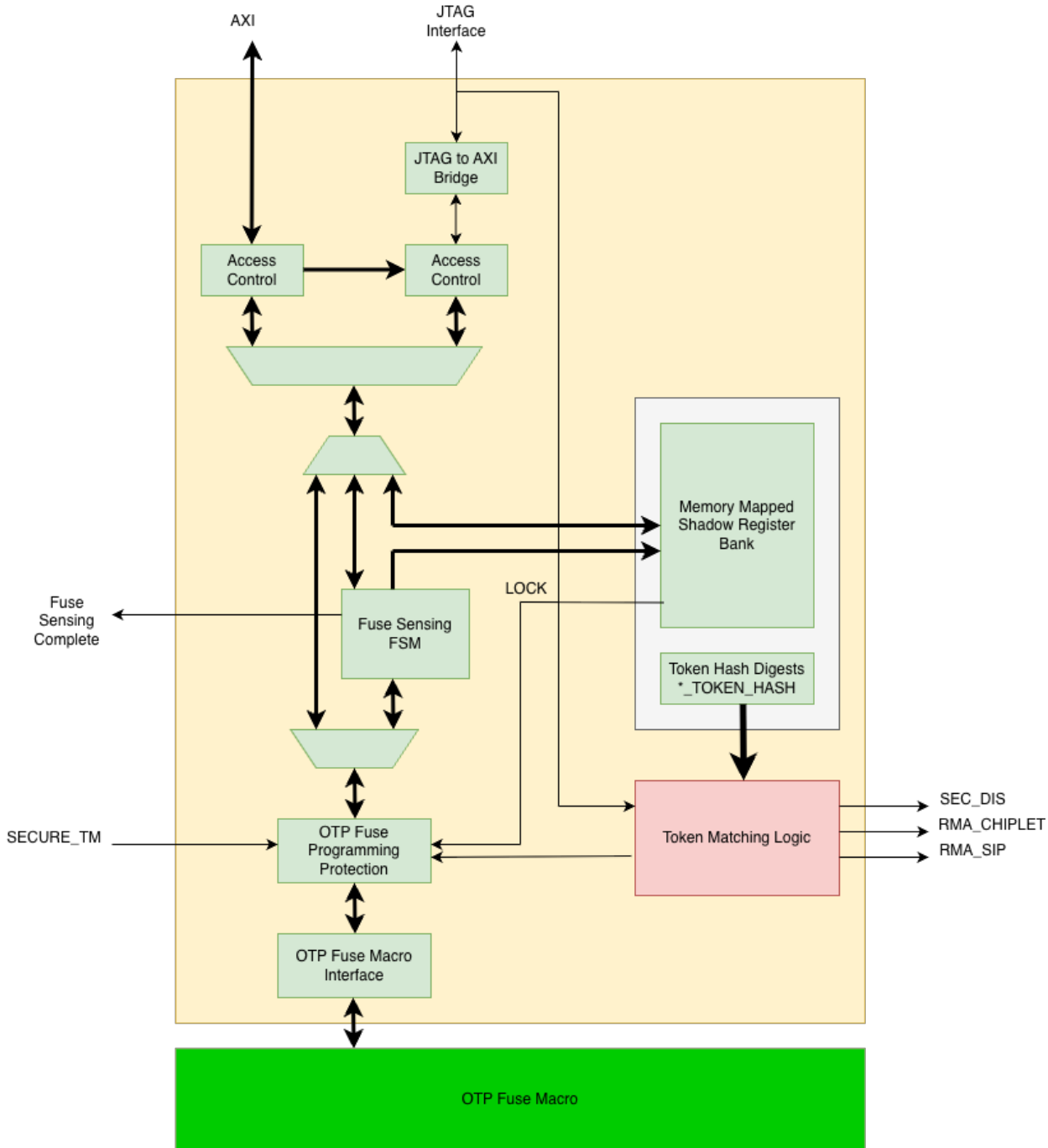
The main loop also dispatches the Adams Bridge shared-key notification. The shared-key ISR sets a pending-notify flag; each iteration the main loop samples and clears that flag and, if set, sends the SEP an unsolicited `RESP_ABR_SHARED_KEY_READY`. The mailbox and shared-key interrupts are masked around the flag sample/clear and the wait-for-interrupt decision so that a notification set by the ISR can neither be dropped (lost update) nor leave the CPU sleeping with work pending (lost wakeup).

Fault Handling

When the KM encounters an unrecoverable fault, it will flush the mailbox FIFOs and then write a `RESP_UNRECOVERABLE_FAULT` message directly to the outgoing mailbox FIFO. The message transmit buffer is entirely bypassed for this operation. Finally, the PicoRV32 will trap so that it halts execution. At this point, the SEP must hard-reset the KM.

When the KM encounters a recoverable fault, it should set the `RECOVERABLE_ERR` register bit in the KMCSR. The intent is that the KM should halt critical services until the SEP has had a chance to acknowledge the fault and take any desired action. If this bit is set, the KM should not process any key management commands until the SEP formally acknowledges the fault with a `CMD_RECOV_ACK` command. The `RESP_RECOVERABLE_FAULT` message is always sent back to the SEP in addition to any error messages returned from any command that may have triggered the fault.

4.7.9. Fuse Controller (OTP)



When SEP is released from reset (chiplet_cold_reset), the sensing FSM automatically copies each fuse field into its dedicated memory-mapped shadow register.

The *output* of the LC_STATE shadow register shall remain 4'b1111 until fuse sensing completes; only then is the LC_STATE value sensed from the fuses driven to downstream logic such as the lifecycle controller. The fuse-field table below defines whether software can read and write the shadow register of each fuse field. Shadow registers that are not software-visible drive their fuse values directly to downstream hardware. All fuse fields other than LOCKS and LC_STATE are made available to downstream logic as soon as they are ready.

After the fuses have been read into their shadow registers, a sense-completion indicator is generated. This triggers the DFT-based memory-repair machine for the SEP SRAMs. Once memory repair completes or is bypassed, the SEP CPU is enabled and fetches its first instruction from the SEP ROM.

Every fuse field has two lock bits in the LOCKS field: one write-lock bit and one read-lock bit. When the write-lock bit is set to 1, the corresponding fuse field can no longer be programmed, and its shadow register is no longer writable. When the read-lock bit is set to 1, the shadow register of the corresponding fuse field is readable neither by software nor by JTAG. The read lock is effective only for fuse fields that are software-visible.

The fuse block's internal space, including the fuse-programming configuration space, is accessible through both a JTAG interface and an AXI interface, as shown in the figure, and each interface has its own access control. In addition to the shadow-register access control defined by the fuse-field table below, the JTAG port shall be further restricted: in the PROD and RMA_SIP states it can only read and write RMA_SIP_TOKEN_I and RMA_CHIPLET_TOKEN_I.

The fuse-programming protection shall enable the fuse-programming function only after the LOCKS field has been sensed, and shall honor the write-lock value of each field. Programming LC_STATE is subject to additional restrictions: LC_STATE[2:1] indicate whether the chiplet is in an RMA state (see the LC State Machine section of the Life Cycle Controller). The SEP fuse controller implements two 256-bit registers, RMA_SIP_TOKEN_I and RMA_CHIPLET_TOKEN_I.

These two registers can be populated by both SEP software and JTAG. Each input is hashed (SHA-256) and compared with the stored RMA_SIP_TOKEN_DIGEST and RMA_CHIPLET_TOKEN_DIGEST, respectively. Only on a match can LC_STATE[1] and LC_STATE[2] be programmed. This is described in more detail in the next section.

The fuse bank shall support keeping the programming-voltage rail active for an unlimited lifetime. This is achieved by using only the internal charge-pump-generated programming voltage rather than the external programming voltage.

The charge pump is calibrated early in wafer test, and its calibration values are burned into the corresponding fuse bits; this is the only time the external fuse-programming voltage is used. Consequently, software-initiated fuse programming never needs to configure the programming voltage.

- Programming a fuse does not update its shadow register until the next fuse sensing occurs.
- The shadow registers of LOCKS, LC_STATE, SIP_DIS, and SYS_DIS are non-scannable. Any register on the path from these registers to the TAP test-enable or fuse write-enable signals is likewise non-scannable.
- When SECURE_TM is 1 (see the Test Mode section), the shadow registers of LOCKS, LC_STATE, SIP_DIS, and SYS_DIS are blocked from any update. Other fuse fields may optionally follow the same rule.

To prevent test patterns from accidentally or maliciously changing or reading out the fuse values:

- When SECURE_TM is 1, the fuse write-enable inputs to the fuse IP block are tied inactive, and the fuse-bank outputs are disconnected from downstream logic. If needed, dedicated test registers can be added to drive the downstream logic during test.

Fuse Fields

Field	Field index	Size (bits)	SW writable	SW readable	Description
LOCK	0	64	set-only	true	Lock control for each fuse field. There are two lock bits for each of the 32 fields. When write-locked, the corresponding shadow register is not writable. When read-locked, neither software nor JTAG can read the corresponding shadow register or fuse field. The fuse-sensing FSM is <i>always</i> able to read the fuses in order to populate the shadow registers.
LC_STATE	1	32 (use only 4 bits)	set-only	true	Life-cycle state of the chiplet. <i>LC_STATE shall never be write-locked or read-locked by the corresponding LOCKS bits.</i> LC_STATE[3] and LC_STATE[0] are set-only. LC_STATE[2:1] shall be set-only when the corresponding *_TOKEN has a match. Otherwise, LC_STATE[2:1] cannot be updated.

Field	Field index	Size (bits)	SW writable	SW readable	Description
SBOOT_DIS	2	32 (use only 1 bit)	true	true	Disables secure boot when set to 1, in any lifecycle state (global override, including the otherwise-mandatory enforcement in PROD and PROD_END); see Secure-Boot Enforcement in the Life Cycle Controller section.
TRANS_RMA_EN	3	32 (use only 1 bit)	true	true	When TRANS_RMA_EN == 1, token-matching operations are permitted. On a token match, the chip is automatically placed into the corresponding RMA state by updating the LC_STATE shadow-register bits, with no chip reset required. A match on RMA_CHIPLET_TOKEN takes priority. The user decides whether to reset the chip before entering the RMA state. When TRANS_RMA_EN == 0, a match on the *_TOKENs does not automatically change the LC_STATE shadow register.
SIP_DIS	4	64	set-only	true	SiP owner's feature-control disable vector, provisioned by the chiplet owner. A bit set to 1 disables the corresponding feature.
SYS_DIS	5	64	set-only	true	System owner's feature-control disable vector, provisioned by the SiP owner. A bit set to 1 disables the corresponding feature.
RMA_SIP_TOKEN_DIGEST	6	256	true	true	Hash digest of the secret RMA_SIP_TOKEN, that enables entering RMA_SIP Life Cycle state
RMA_CHIPLET_TOKEN_DIGEST	7	256	true	true	Hash digest of the secret RMA_CHIPLET_TOKEN that enables entering RMA_CHIPLET Life Cycle state
CLASS_KEY	8	256	true	true	Can be used for firmware encryption and decryption.
CHIPLET_PUBK_REV_OKE	9	32	set-only	true	Global public-key revocation vector covering every boot-verification key of every owner (classical and PQC). SEP ROM embeds a set of public-key hash digests and accesses additional fused key hashes; each key entry maps to one bit here. Once set, SEP ROM rejects the corresponding key. Bit map: [7:0] ROM chiplet-creator classical keys (bits 0,1 are development keys); [15:8] ROM chiplet-creator PQC keys; [16]/[17] CHIPLET_PUBK_HASH0/1; [18]/[19] CHIPLET_PUBK_PQC_HASH0/1; [20]/[21] SIP_PUBK_HASH0 / SIP_PUBK_PQC_HASH0; [22]/[23] SYS_PUBK_HASH / SYS_PUBK_PQC_HASH; [24]/[25] SIP_PUBK_HASH1 / SIP_PUBK_PQC_HASH1; [31:26] reserved. Before the chiplet owner transfers a part to the customer, the development-key bits shall be set (revoked). (Name retained for compatibility; scope is now all owners.)
BL1_VERSION	10	256	set-only	true	Anti-Rollback Counter (ARBC) for BL1: the minimum BL1 version that SEP ROM will load. The version is encoded as the population count of 1 bits in this field. The maximum version is 240 (TBD), which leaves redundancy for any fuses that fail to blow.
BL2_VERSION	11	256	set-only	true	Anti-Rollback Counter (ARBC) for BL2: the minimum BL2 version that can be loaded. The version is encoded as the population count of 1 bits in this field. The maximum version is 240 (TBD), which leaves redundancy for any fuses that fail to blow.
CHIPLET_UID	12	256	true	true	Silicon creator's <i>secret</i> device unique ID (also referred to as the UDS, Unique Device Secret). The chiplet creator is responsible for CHIPLET_UID provisioning and public identity enrollment. <i>FUTURE IMPROVEMENT: HW enforced key provisioning measurement recording. When locked, both writability and software visibility are disabled.</i>
SIP_PUBK_HASH0	13	256	true	true	Hash digest of SiP owner's root public key for secure boot (slot 0). Note: The chiplet creator's actual root public key is embedded in the SEEPROM. The digest is used to validate that public key.
SIP_UID	14	256	true	true	SiP owner's <i>secret</i> device unique ID. The SiP owner is responsible for SIP_UID provisioning and public identity enrollment. When locked, both writability and software visibility are disabled. <i>FUTURE IMPROVEMENT: HW enforced key provisioning measurement recording; keep previous owners' PUBK and UID pair(s) still valid but reflected in the key derivation hierarchy.</i>

Field	Field index	Size (bits)	SW writable	SW readable	Description
SYS_PUBK_HASH	15	256	true	true	Hash digest of system owner's root public key for secure boot.
SYS_UID	16	256	true	true	System owner's <i>secret</i> device unique ID. The system owner is responsible for SYS_UID provisioning and public identity enrollment. When locked, both writability and software visibility are disabled.
STATUS_RPT	17	32 (2 used)	true	true	When STATUS_RPT[1:0] == 2'b00, the boot ROM samples an SMC GPIO (STATUS_RPT) to determine whether to report status. When STATUS_RPT[1:0] == 2'b01, the boot ROM reports status; otherwise, it does not.
ROM_CTL	18	32	true	true	ROM_CTL[0]: 0 = little-endian, 1 = big-endian. ROM_CTL[5:1]: row-swap control; swaps ROM slices when set. ROM_CTL[31:6]: reserved.
SPI_CONFIG	19	288	true	true	SPI discovery has 8 configuration values as well as 1 enable bitfield. Each word is 32-bits in length: [0] SPI_CONFIG_EN Enable for each of the following 8 SPI_CONFIG field words. SPI_CONFIG_EN[7:0] corresponds to registers 1-8 below. When bit N is set in this field, then configuration register N+1 is valid in fuses and that value will be read from fuses and written into the specified SPI configuration register during the SPI controller initialization. SPI_CONFIG_EN[31:8] are reserved. [1] SPI_CTRL_DISCOVERY_CTRL_REG [2] SPI_CTRL_INIT_PHY_DQ_TIMING_REG_ADDR phy_dq_timing_reg [3] SPI_CTRL_INIT_PHY_DQS_TIMING_REG_ADDR phy_dqs_timing_reg [4] SPI_CTRL_INIT_PHY_GATE_LPBK_CTRL_REG_ADDR phy_gate_lpbk_ctrl_reg [5] SPI_CTRL_INIT_PHY_DLL_SLAVE_CTRL_REG_ADDR phy_dll_slave_ctrl_reg [6] SPI_CTRL_INIT_PHY_DLL_MASTER_CTRL_REG_ADDR phy_dll_master_ctrl_reg [7] SEP_SPI_PHY_MISC [8] INIT_RB_VALID_TIME
CHIPLET_PUBK_HASH0	20	256	true	true	Hash of chiplet-owner authentication key 0, fused. These fused chiplet-owner key hashes can be used in addition to the hashes embedded in the ROM to authenticate manifests.
CHIPLET_PUBK_HASH1	21	256	true	true	Hash of chiplet-owner authentication key 1, fused.
REQUIRED_SIGNERS	22	32 (use only 2 bits)	true	true	Owners whose signature a manifest must carry to be authenticated by SEP ROM. Per owner, 1 = required, 0 = not-required; the default all-zero value requires no additional owner. [0] SiP owner, [1] system owner, [31:2] reserved. The chiplet creator is the root of trust — its classical and PQC root digests are embedded in SEP ROM and always authenticate BL1 — so it has no bit here. Burning a fuse bit only ever adds a required signer, so the fuse is monotonic: the standard LOCK bits exist but locking this field is unnecessary, since a later burn can never weaken security. The value is applied via the shadow register (loaded from the fuse at cold reset, SW-writable): firmware may update the shadow to set the required signers for each manifest it verifies (BL1, BL2, runtime, ...). See Secure-Boot Signing Model in the Life Cycle Controller section.
REQUIRED_ALGS	23	32 (use only 12 bits)	set-only	true	Per-owner required signature algorithms, applied only when that owner is required. Each owner has a 4-bit nibble; bit 0 = classical (ECDSA P-384 or RSA), bit 1 = post-quantum (ML-DSA), bits 3:2 reserved. The specific classical scheme and ML-DSA parameter set (e.g., ML-DSA-44/65/87) are set by the owner's provisioned keys, not mandated here. [3:0] chiplet, [7:4] SiP owner, [11:8] system owner, [31:12] reserved. For each required (owner, algorithm) the manifest must carry a signature verified against that owner's burned, non-revoked key of that type; all required pairs must verify, and any signature from a non-required owner is ignored.
CHIPLET_PUBK_PQC_HASH0	24	256	true	true	Chiplet owner ML-DSA (PQC) authentication key 0 hash, fused. PQC counterpart of CHIPLET_PUBK_HASH0.

Field	Field index	Size (bits)	SW writable	SW readable	Description
CHIPLET_PUBK_PQC_HASH1	25	256	true	true	Chiplet owner ML-DSA (PQC) authentication key 1 hash, fused. PQC counterpart of CHIPLET_PUBK_HASH1.
SIP_PUBK_PQC_HASH0	26	256	true	true	Hash digest of the SiP owner's ML-DSA (PQC) root public key for secure boot (slot 0). PQC counterpart of SIP_PUBK_HASH0.
SYS_PUBK_PQC_HASH	27	256	true	true	Hash digest of the system owner's ML-DSA (PQC) root public key for secure boot. PQC counterpart of SYS_PUBK_HASH.
SIP_PUBK_HASH1	28	256	true	true	Hash digest of the SiP owner's root public key for secure boot, slot 1. Second slot so the SiP owner retains a usable root key after revoking slot 0 via CHIPLET_PUBK_REVOKE[24].
SIP_PUBK_PQC_HASH1	29	256	true	true	Hash digest of the SiP owner's ML-DSA (PQC) root public key for secure boot, slot 1. PQC counterpart of SIP_PUBK_HASH1; revocable via CHIPLET_PUBK_REVOKE[25].
reserved4	30	2048	true	true	2048 bit reserved field with lock bit
reserved5	31	800	true	true	800 bit reserved field with lock bit

The JTAG_ID field has been removed from the fuse map and will be implemented as a top-level port.

Anti-Rollback Counter (ARBC)

The counter is implemented as a "population count" in fuses instead of a binary counter. Each 256-bit counter field allocates 240 fuse bits for counting, with 16 spares as redundancy for fuses that fail to blow. Every counter increment blows one additional fuse bit, and the counter state equals the fuse population count — the number of fuses (out of 256) that have been blown.

A new device starts with an ARBC state of zero fuse bits blown. Each time the required production firmware version is incremented, one more fuse bit is blown as described above. Firmware signature verification includes the ARBC state along with the firmware version, ensuring that older firmware versions fail signature verification and cannot be loaded.

RTL Integration

The eFuse subsystem in the SEP is implemented by `sep_efuse_wrapper`, which is instantiated inside `sep_crypto`. The wrapper converts the SEP-side AXI bus (64-bit) to AXI-Lite (32-bit) through an `axi_dw_converter` and `axi_to_axi_lite` bridge, and then connects to the shared `efuse_interface_controller` IP from `hw/periph/efuse/`. A separate JTAG AXI-Lite port provides debug and manufacturing access.

The `efuse_interface_controller` is instantiated with the `HAS_LC_STATE` parameter set to `1'b1`. When this parameter is asserted, the controller implements special handling for the `LC_STATE` fuse field: the shadow register output for `LC_STATE` is driven to `4'b1111` (invalid) until fuse sensing completes, preventing downstream logic from acting on an incorrect life-cycle state during the boot sequence. The life-cycle state width is 4 bits (`sep_pkg::LC_STATE_BIT_WIDTH`), starting at bit position 64 in the shadow register map (`sep_pkg::LC_STATE_BIT_POSITION`).

After sensing, the shadow register map (`sep_efuse_pkg::efuse_map_t`) is exposed to the `sep_lifecycle_ctrl` module. The lifecycle controller decodes the differentially-encoded `LC_STATE` field using `prim_diff_decode_multi` and derives feature control vectors from the `SIP_DIS` and `SYS_DIS` fields. It also generates two demotion state outputs (`lcc_demote_state_1_o`, `lcc_demote_state_2_o`) through differential encoders.

The Key Manager receives a subset of the OTP data through the `km_otp_data_t` structure, which is assembled in `sep_crypto.sv`. The `life_cycle` field comes from the differentially-encoded `LC_STATE` shadow register, the `demotion_state_*` fields come from the lifecycle controller outputs (not directly from fuses), and the `chiplet_uid` field comes from the `CHIPLET_UID` shadow register. The ARBC fields (`BL1_VERSION`, `BL2_VERSION`), RMA tokens, and public key

hashes are not wired into the Key Manager; they remain accessible only through the eFuse shadow register interface and are consumed by the SEP boot firmware.

For simulation, adopters may provide an external eFuse macro model through an overlay-specific shim. The open tree keeps only the generic controller and shim interfaces.

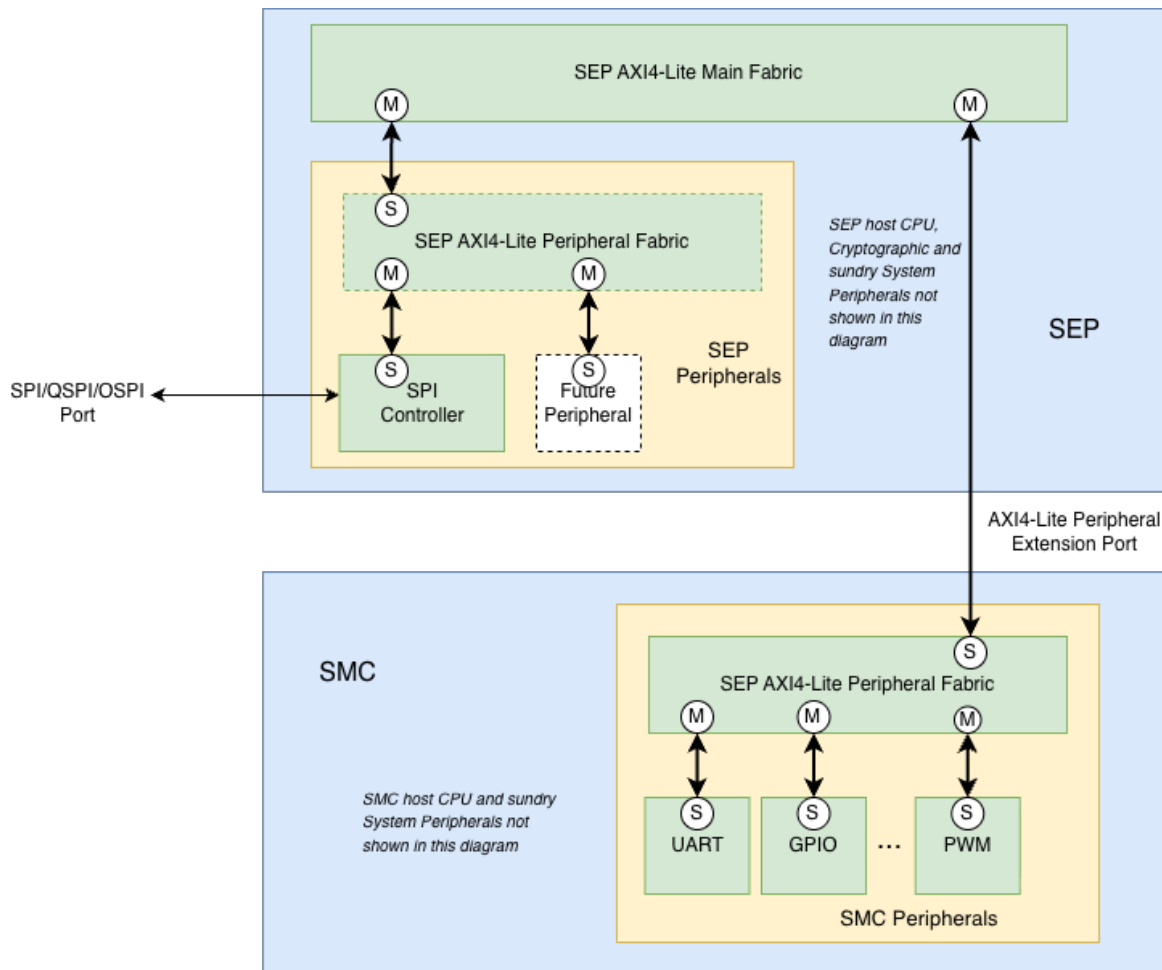
For the canonical eFuse IP documentation covering the `efuse_interface_controller` internals, sensing FSM, and programming protocol, see [hw/ip/efuse/doc/index.adoc](#) (included in the SMC documentation via [hw/sys/smc/doc/periphs.adoc](#)).

4.8. Peripheral Subsystem

SEP will primarily rely on external serial IO peripherals resident in the SMC complex. SEP has an AXI4 extension port which has unfettered access to the SMC. Any peripherals in SMC will thus also be available for use by SEP.

The exception to this will be the SPI host controller which is required for SEP booting. It will for now be the only peripheral unit which is locally instantiated.

The requirement for SEP accessible GPIO straps was stated earlier in the Boot BL0 section of this document. We must ensure we have the minimum set of straps available on the SMC GPIO port which are dedicated to the SEP BL0 functionality.



4.8.1. Serial Peripheral Interface (SPI)

SEP will have one SPI host to support access to external NOR flash devices which are the primary BL1 image source. Modern devices are MT35 type and have octal x8 DDR interfaces. We will also support the older MT25 category devices using x4 and x1 channels SPI as these are less expensive.

Use Cases

- BL1 boot image access from external flash device
- Sensor interfaces (automotive primarily)

Features

The component will be based on the [OpenTitan SPI Host](#) design and support the following features:

- All channel types are supported:
 - Single channel (x1) mode, clock frequency up to 80 MHz
 - QSPI (x4), quad channel mode, clock frequency up to 166 MHz
 - Octal (x8) in DDR mode, clock frequency up to 200 MHz
- DQS read data strobe input
- Separate FIFOs for transmit and receive. Each can be parameterized for distinct depth.
- AXI4-lite slave port for system interconnect

The SPI pass-through mode supported by OpenTitan is used on discrete secure element designs as a security bridge to filter, modify or otherwise modify traffic coming from a separate SPI slave device. We do *not support* this feature and will leave this interface on the component unconnected or terminated. Furthermore, there is no SPI Slave device present in the SEP peripheral subsystem.

The SEP provides two SPI integration paths. The `sep_ot_spi_wrap` module wraps the modified OpenTitan-derived `spi_controller` with a native AXI4-Lite slave interface (no TL-UL conversion required). It exposes the standard SPI pad signals (SCK, CS, 4-bit bidirectional I/O), an interrupt output, a busy indicator, and a DMA trigger (`lsio_trigger_o`). The `NUM_CS` parameter controls the number of chip-select outputs. A second path, `sep_cdns_spi_wrap` (provided by the nonfree overlay), integrates a vendor xSPI controller and PHY for high-performance octal DDR flash access. This module presents both an AXI-Lite register interface and a full AXI4 flash read window, along with dedicated PHY pad connections and initialization status outputs. The selection between these two SPI paths is determined by the system configuration.

For the OpenTitan SPI host theory of operation and register map, refer to the [OpenTitan SPI Host documentation](#).

4.8.2. Watchdog Timer (WDT)

The Watchdog timer is a component used to enhance reliability and ensure that the system is notified if the SEP subsystem is ``hanging". The SEP CPU must periodically visit the WDT and reset its counter, proving that the CPU is still alive and well i.e. not hanging. If the WDT timer reaches zero in its countdown, it fires an external interrupt indicating that its timeout has expired and there is a problem within the SEP.

The `sep_wdt_wrap` module integrates the OpenTitan `aon_timer` IP as the watchdog timer. The 64-bit AXI slave port from the system crossbar is narrowed to 32-bit AXI through an `axi_dw_converter`. The address path subtracts the WDT base address so that the internal TL-UL decoder sees zero-based offsets. After `axi_to_axi_lite` and `axi_lite_to_tlul`

conversion, the TL-UL bus drives the standard `aon_timer` register block.

The wrapper provides a dedicated watchdog clock input (`clk_wdt_i`) that is synchronized through a `prim_sync_reset` to produce the `rst_aon_ni` reset for the always-on timer domain. The `wdt_debug_sleep_mode_i` input controls whether the timer continues to count during debug halt. Life-cycle escalation is tied to its inactive state. Alert receivers are initialized to the idle encoding to avoid spurious integrity errors; alert outputs are left unconnected since the SEP aggregates watchdog events through the interrupt path.

The wrapper exports two signals: `intr_wdog_timer_bark_o`, the watchdog bark interrupt that fires when the first timeout stage expires, and `wdt_timer_rst_req_o`, the bite reset request that fires when the second timeout stage expires and the system failed to respond to the bark.

For the full `aon_timer` theory of operation, watchdog configuration, and register map, refer to the [OpenTitan Always-On Timer documentation](#).

4.8.3. SEP Reset Controller

The SEP Reset Controller provides software-controllable resets for the Key Manager and crypto accelerators. It is accessible via AXI4-Lite at base address `0x10A5_0000` within the System Peripherals crossbar.

The controller contains a single 64-bit register `SW_RESET_N` at offset `0x0` with the following fields:

Bit	Field	Access	Default	Description
0	<code>km_sw_rst_n</code>	RW	0	KM reset (0=assert, 1=release)
1	<code>otbn_sw_rst_n</code>	RW	1	OTBN reset (0=assert, 1=release)
2	<code>aes_sw_rst_n</code>	RW	1	AES reset (0=assert, 1=release)
3	<code>hmac_sw_rst_n</code>	RW	1	HMAC reset (0=assert, 1=release)
4	<code>kmac_sw_rst_n</code>	RW	1	KMAC reset (0=assert, 1=release)

On power-up, `SW_RESET_N` resets to `0x1E`: KM is held in reset, while OTBN, AES, HMAC, and KMAC are released. The SEP host firmware writes `1` to release an IP from reset and writes `0` to assert its software reset.

Inside `sep_reset_ctrl`, each bit is ANDed with `sep_reset_n` and then passed through a per-IP JTAG override mux (`jtag_sep_reset_ctrl_i`). The `sep_reset_n` term holds every IP in reset until eFuse sensing completes during boot.

The Key Manager reset (`km_sw_rst_n`) is applied directly as the KM warm reset. The four crypto accelerators (`otbn`, `aes`, `hmac`, `kmac`) are AXI subordinates on the `sep_crypto` demux, so resetting one mid-transaction would strand an in-flight request and hang its master. Their reset bits are therefore treated as requests: `sep_crypto_axi_isolate` first isolates the accelerator's AXI port, draining outstanding transactions and terminating new ones with DECERR, and only then asserts the reset. The isolation is held for the duration of the reset and released when the software bit deasserts.

4.8.4. Token Processing

Maintaining a known, measurable state of the SEP system is a core requirement to ensure trust and security. Consequently, changing the security state of SEP is a very high security operation that must be protected. Tokens are high security password strings which are used to alter the security state of SEP. They are used to initiate RMA LC state transitions as well as disable security features. Tokens are:

- Security Disable (SEC_DIS) bypasses fuse sensing
- Package RMA (RMA_SIP) used in LC state machine

- Chiplet RMA (RMA_CHIPLET) used in LC state machine

The tokens directly affect the life cycle state machine processing and fuse sensing, so token processing is naturally done in the OTP Controller. Tokens are entered via the JTAG port and captured in a serial register for match processing described in the next section. The serial register has the following important properties:

- Non-scannable
- Non-resettable

Token Generation, Storage and Provisioning

All token processing besides final provisioning takes place within the confines of an HSM, a very high security environment. RTL constant tokens are embedded into the design database very late in the development process to ensure minimal exposure while OTP fuse tokens are provisioned at manufacturing test time. All tokens are under tight security control.

Token values are initially generated within the HSM using a secure TRNG, KDF and other facilities. Values to keep track of are the token itself and any encoded value such as the SHA256 hash value of it for example.

All token values are securely stored in the HSM and securely transmitted to the remote tester which programs any OTP fuse values. In the case of SEP tokens, we provision the 256-bit hash digest values into the OTP fuse bank. Values could be global or unique per device. Examples of unique values are the UID, private ECDSA signing keys, and special purpose tokens, granting special TBD privileges on a per device basis. Processing at the device level can handle each scenario.

Token Matching Mechanism and Attack Countermeasures

The tokens are 256-bit values which are processed in an analogous manner as passwords. We have tokens:

- RMA_SIP_TOKEN
 - LC state
 - Digest RMA_SIP_TOKEN_DIGEST in OTP
- RMA_CHIPLET_TOKEN
 - LC state
 - Digest RMA_CHIPLET_TOKEN_DIGEST in OTP
- SEC_DIS_TOKEN
 - Security disable
 - Digest SEC_DIS_TOKEN_DIGEST in hardware gates (global)

The separate security disable token SEC_DIS_TOKEN overrides the entire LC control mechanism; the resulting feature control follows the SEC_DIS / SECURE_TM rules in the Lifecycle Controller chapter. This feature is an *emergency fallback* intended to enable A0 silicon debugging and shall be permanently disabled in subsequent A1/B0 revisions via metal mask only ECO.

The tokens are not directly compared with an internal value stored in OTP but instead first hashed using the SHA2-256 cryptographic hash algorithm. OTP holds the hash value, also known as the TOKEN_DIGEST. The OTP controller shall have a local, dedicated SHA256 hardware accelerator unit to perform the hashing of the input token, entered via JTAG.

```
token_match = (SHA256(token[255:0]) == <NAME>_TOKEN_DIGEST[255:0])
```

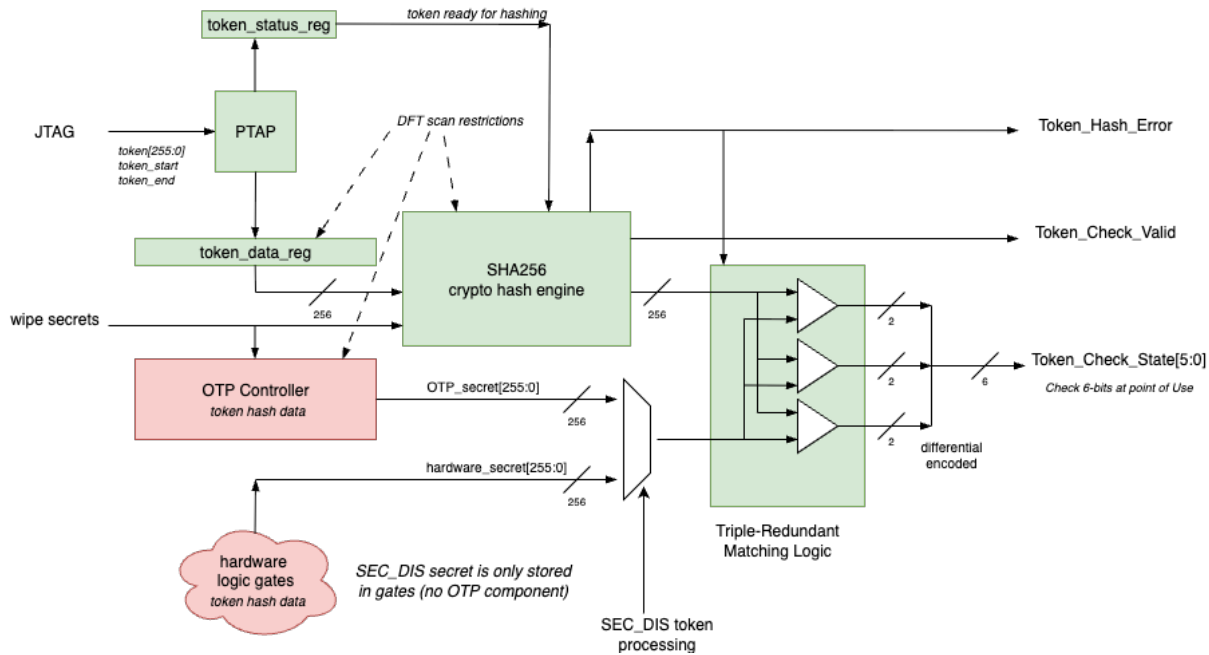

The above token matching logic is purely combinational and input values are never stored in any downstream, potentially scannable registers.

The stored secrets are very security sensitive so special care must be taken when provisioning them in the chiplet. OTP_secret[255:0] is a SHA256[token] which is stored in the OTP. The SEC_DIS token is handled differently with its SHA256[token] value stored in hardware gates. (aka RTL secret) rather than in OTP. It is global to all devices by its nature. In both cases, the hash preimage attack to deduce the input token value is considered practically infeasible so even if the secret digest is extracted from OTP or gates, the token will remain unknown to the attacker.

As just stated, the *SEC_DIS secret is stored only in gates, not in OTP*. The reason for this is that SEC_DIS is an emergency feature used to unlock the chip when the OTP fuse sensing logic is non-functional but we still need to unlock the chip. If we were to rely on the non-functioning OTP, we would be unable to unlock the chip.

The actual matching logic as a vulnerable attack point via FIB or fault injection. We protect this by using triple-redundant matching logic on fully differential signals resulting in a 6-bit output as illustrated below. The SHA256 hashing engine is also protected with redundancy coding of sensitive internal state. Any internally detected errors are signalled on the output error port.

Note that this is not a majority vote scheme i.e. all three comparators must return the same result. If a mismatch is detected, this may indicate a tampering attack and an alarm interrupt must be raised. Furthermore, if an error is detected in the SHA256 hashing engine or the OTP shadow register memory bank, then matching will always return a failure code on all three redundant outputs.



The 6-bit matching signal is distributed as is to each individual point of use i.e. logic where individual features are enabled or disabled. Distributing the final enable logic to each feature diversifies the gate keeping and helps prevent global unlocking.

Match = token_match[5:0] = 6'b010101 Mismatch = 6'b101010 Error = 6'b111111

Post Token Matching Handling

After a token matching operation has completed and a LC state transition operation has been performed, the following takes place:

disable all {Debug, DFT, Functional} features making system inert if token match successful: initiate OTP LC state update operation

Note that the above procedure takes place regardless whether the token match and LC state transition is successful or not.

4.8.5. Security Disable (SEC_DIS)

Security Disable (SEC_DIS) is an emergency bring-up bypass for early (A0) silicon. On deassertion of power-on reset, the fuse sense logic runs before anything else, and a malfunction there—the fuse state machine hanging, or invalid values being written into the shadow register bank—would leave the life-cycle control closed and turn the chiplet into a disabled “brick”. SEC_DIS provides a secure unlock mechanism to recover such a part for debug: it overrides the life-cycle control, forcing all feature-control bits on regardless of the LC state and the SIP_DIS / SYS_DIS disables. Test-group (scan/DFT) features additionally require SECURE_TM = 1, per the SEC_DIS / SECURE_TM rules in the Life Cycle Controller chapter.

SEC_DIS is activated by presenting a secret token: SEC_DIS_TOKEN_I is passed into the SEP via the JTAG port, hashed, and redundantly compared against an embedded hash constant. On a match, i.e. `hash(SEC_DIS_TOKEN_I) == SEP_SEC_DISABLE_TOKEN_DIGEST`, the life-cycle control is overridden to open debug and test access, and the feature-control output follows the SEC_DIS / SECURE_TM rules in the Life Cycle Controller chapter. Once SEC_DIS is active, the SEP can boot even if the fuse sense logic never completes.

SEC_DIS is intended for A0 silicon only and is removed in later revisions (A1/B0) by a simple metal-mask ECO: the token comparator’s enable is implemented in a metal-programmable revision cell, strapped to 1 (comparator enabled) in A0 silicon and re-strapped to 0 at top metal in later revisions, so the feature is killed without a re-spin.

Implementation requirements:

- The token secret itself is never stored in RTL; the RTL stores only the token hash value, and the hardware forces every presented token through the hash engine before the digest comparison—there is no path that presents a digest to the comparator directly. The 256-bit SEP_SEC_DISABLE_TOKEN_DIGEST constant is therefore not itself a secret (recovering the token from it requires a SHA-256 preimage): it may be publicly known and may be carried in plain RTL. It resides in the fuse controller component hardware gates (*not the OTP fuse bank*).
- The comparison logic must be purely combinational, and the SEP_SEC_DISABLE_TOKEN_DIGEST constant must never be latched into any potentially scannable register. This is an integrity requirement, not confidentiality: the expected digest must not be perturbable—for example scan-shifted—into a value whose preimage an attacker knows.
- The JTAG-accessible register holding the token input values (SEC_DIS_TOKEN_I, RMA_SIP_TOKEN_I, and RMA_CHIPLET_TOKEN_I) **shall not have a reset**, so that the SEC_DIS state is retained across reset for early-event debug. That register must also not be scannable: DFT scan tests may be running while SEC_DIS is asserted, and a scan pattern must not inadvertently overwrite the register—dropping the security unlock and stopping the test.
- The secrecy of the scheme rests entirely on the SEC_DIS_TOKEN value, never on the hash. The token shall be a full-entropy 256-bit random value, and knowledge of it shall be strictly controlled: it shall be generated and stored in a secure database (HSM). Because only its integrity matters, the SEP_SEC_DISABLE_TOKEN_DIGEST may be bound at any convenient point in the implementation flow—for example as an RTL parameter passed down the instance hierarchy (chiplet top to SMU to SEP to fuse controller). One member of the DV team shall perform final verification of the SEC_DIS_TOKEN functionality on the post-layout extracted netlist with *gate-level simulation*.

4.8.6. Life Cycle Controller (LCC)

The Life Cycle Controller (LCC) supports multiple security domains in chiplet-based systems. It manages the device life cycle: it decodes the life-cycle state and generates the debug/test/function feature control vector that corresponds to that state.

The LCC performs the following actions to support life cycle management:

- Receive inputs from the SEP OTP (fuse) shadow registers (populated by the OTP sense FSM at power-on)
 - LC_STATE[3:0]
 - Disable vectors: SIP_DIS and SYS_DIS
- Generate the per-LC-state debug/test/function feature control vector (`feat_ctrl_o`) and send it to its downstream destinations (TAP, DAP, DFT, peripherals, downstream chiplets, Key Manager).
- Maintain volatile in-field debug demotion state via the DEMOTE_1, DEMOTE_2, DEMOTE_LOCK_1, and DEMOTE_LOCK_2 CSRs, and apply the resulting demotion feature-control profile to `feat_ctrl_o`. See [Demotion 1 and Demotion 2 — In-Field Debug Demotion](#).
- Detect signal-integrity errors on the differentially encoded LC_STATE input arriving from the OTP shadow register; fail closed (force `feat_ctrl_o = 0`) on detection.

Demotion 1 and Demotion 2 — In-Field Debug Demotion

In addition to the LC state stored in OTP, SEP implements two volatile demotion CSRs (DEMOTE_1, DEMOTE_2) and two companion lock CSRs (DEMOTE_LOCK_1, DEMOTE_LOCK_2). When asserted, the demotion CSRs grant additional in-field debug access without changing the OTP-stored LC state. SEP cold reset clears all four registers, restoring the device to its OTP-stored LC state.

Demotion 1 and Demotion 2 are two independent controls that differ only in which feature-control bits they gate. Demotion 1 (DEMOTE_1) gates the **local** debug group (DBG_1)—debug of the chiplet’s own logic. Demotion 2 (DEMOTE_2) gates the **inter-chiplet debug-bypass** group (DBG_2)—the SiP-owner debug-access path at this chiplet, which admits external (PTAP) debug traffic into the chiplet and forwards it onward to the next chiplet in the chain (see [Multiple Security Domains](#)). There is no ordering or dependency between the two demotion bits: either may be asserted without the other. The controls are independent even though their effects compose: opening the local debug group (DBG_1) alone does not make local debug externally reachable until the SIP_DBG access path (DBG_2) is also open. Each control selects a per-LC-state feature-control profile (see [Per-LC-state feature control profile](#)), and its authorization is governed by the [Secure-Boot Signing Model](#).

Secure-Boot Enforcement

Whether SEP enforces secure boot — that is, whether a manifest must be verified before its image is allowed to run — is decided by the lifecycle state and two fuse controls. This decision is separate from the signing model described below, which governs who must sign once enforcement applies.

- **PROD and PROD_END:** secure boot is enforced, unless it has been explicitly disabled by the SBOOT_DIS fuse (below).
- **TEST_DEV:** secure boot is not enforced by default, so unsigned development images can be loaded during bring-up and firmware development. It is enforced, however, when chiplet-scope debug is disabled (CHIPLET_DBG set in SIP_DIS or SYS_DIS), so a locked-down debug posture cannot be bypassed by loading unsigned code.
- **SBOOT_DIS:** a global secure-boot disable. Burning SBOOT_DIS to 1 disables secure boot in every lifecycle state, overriding the otherwise-mandatory enforcement in PROD and PROD_END. An owner that relies on secure boot

must therefore leave SBOOT_DIS unburned and write-lock it via the LOCK field.

Secure-Boot Signing Model

SEP secure boot decides, for each manifest it verifies, which owners must sign it and with which algorithms. Enforcement—including the demotion authorization that rides on it—is delegated to the SEP boot firmware; the LCC hardware does not check signing authority or inspect manifest contents. The one hardware-rooted anchor is SEP ROM, which reads the fuse-provided requirement to verify the first manifest (BL1). Up to three owners participate: the chiplet creator (root of trust; its classical and PQC root digests are embedded in SEP ROM), the SiP owner, and the system owner.

Two eFuse policy fields govern the model (see the *Fuse Controller (OTP)* fuse-field table for field details):

- **REQUIRED_SIGNERS**—owners whose signature a manifest must carry, one bit each for the SiP owner and the system owner. Per owner, 1 = required, 0 = not-required; the default all-zero value requires no additional owner. The chiplet creator is the root of trust and always authenticates BL1, so it has no bit—it is implicitly always required. Burning a bit only ever adds a required signer (the fuse is monotonic), so the standard LOCK bits exist but are not needed here.
- **REQUIRED_ALGS**—per owner, which algorithms (classical: ECDSA P-384 or RSA; post-quantum: ML-DSA) that owner must sign with, applied only when the owner is required.

Key provisioning is implicit: there is no separate provisioning fuse. If an owner's key is not provisioned, its **REQUIRED_SIGNERS** bit is simply left unburned so that owner is not required.

Requirement rule. For the manifest being verified, an owner must sign if it is marked required in the shadow—the chiplet creator always, and the SiP or system owner when its **REQUIRED_SIGNERS** bit is set. If so, the manifest must carry a valid signature for each **REQUIRED_ALGS**-selected algorithm, verified against that owner's burned, non-revoked key of that type. Signatures from non-required owners are ignored.

Propagation. SEP ROM verifies the BL1 manifest against the shadow value loaded from the fuse at cold reset. Thereafter, each successfully verified—and therefore trusted—boot stage sets the required signers for the next manifest by writing the shadow: because a stage runs only once it has been authenticated, it is by definition authorized to make the change, and may freely add or drop owners. Because verification is performed by firmware, this propagation is enforced by the authenticated firmware itself—a chain of trust rooted in SEP ROM and the fuse-provided requirement that ROM uses to admit BL1; no additional hardware gate is required, since a compromised stage would bypass its own verification regardless.

Like the SiP owner, the system owner is required exactly when its **REQUIRED_SIGNERS** bit is set: if the bit is burned, the shadow register reloads that requirement at every cold reset and BL1—and each later stage—must carry the system-owner signature; if that is not wanted, the bit is left unburned.

Post-quantum signing. Motivated by the NIST standardization of post-quantum signatures (FIPS 204, ML-DSA), each owner may be provisioned with both a classical and a post-quantum root key, and **REQUIRED_ALGS** selects, per owner, whether one or both must sign. Requiring both yields a combined classical-plus-post-quantum signature for that owner, neither part of which alone is sufficient.

Demotion authorization. Demotion is carried in a boot manifest and is authorized by this same signing model rather than by a hardware-ordered transition. Demotion 1 and Demotion 2 are authorized identically; Demotion 2 also drives the dedicated inter-chiplet debug-bypass field, but this adds no special policy. By default—with no **REQUIRED_SIGNERS** bit burned—any boot stage whose manifest authenticates under the current required-signer set may assert a demotion, so a later, already-authenticated stage that asserts one is implicitly authorized through the chain of trust. Because the inter-chiplet bypass is a SiP-owner concern, the SiP owner is given the option to gate Demotion 2: by burning its own

REQUIRED_SIGNERS bit it forces every demotion-asserting manifest to carry the SiP-owner signature, and it can then lock Demotion 2 (see [Lock Behavior](#)) to prevent demotion outright. This is distinct from RMA, whose state transitions are hardware-ordered. The LCC hardware does not check any of this; it simply applies the feature control profile that corresponds to the DEMOTE_1 / DEMOTE_2 register bits. The demotion assertion is not tied to a specific boot stage: any bootloader stage may carry it, because what authorizes it is the set of signing authorities on the manifest, not which stage performs the write.

NOTE

Public keys themselves are never stored in eFuse—only their hash digests are. The chiplet creator’s root public-key digests (classical and PQC) are **embedded in SEP ROM**; **SIP_PUBK_HASH0/1** / **SIP_PUBK_PQC_HASH0/1** (two slots, so the SiP owner retains a usable root after revoking one) and **SYS_PUBK_HASH** / **SYS_PUBK_PQC_HASH** hold the SiP- and system-owner digests in eFuse; and **CHIPLET_PUBK_HASH0/1** / **CHIPLET_PUBK_PQC_HASH0/1** hold additional **chiplet-owner** authentication-key hashes that supplement the chiplet-creator digests in SEP ROM. Every such key entry can be revoked individually through **CHIPLET_PUBK_REVOKE** (see the *Fuse Controller (OTP)* fuse-field table in this chapter).

Signing-Model Examples

These examples show which signers are required **when secure boot is enforced for a manifest**. Whether it is enforced at all is a separate decision, governed by the rules in [Secure-Boot Enforcement](#); the signing model governs only the required-signer set once enforcement applies.

No REQUIRED_SIGNERS bits burned. BL1 requires the chiplet-creator signature only (the chiplet creator is always required; no additional owner is).

SiP owner required. **REQUIRED_SIGNERS[SiP]** is burned, typically at ownership transfer. BL1 requires the chiplet creator and the SiP owner. BL1, having been signed by both, may drop the chiplet requirement for BL2 (handoff), so BL2 requires only the SiP owner.

System owner added downstream. With the SiP owner required, a downstream stage adds the system owner in the shadow for its successor, so that manifest requires the system owner (or the SiP owner plus the system owner, if the SiP owner co-signs it).

In every case the shadow register reloads from the **REQUIRED_SIGNERS** fuse at the next cold reset, so the chain restarts from the fuse-provided requirement.

Lock Behavior

- When **DEMOTE_LOCK_1/2** = 0, firmware can assert **DEMOTE_1/2** = 1.
- Once **DEMOTE_LOCK_1/2** = 1, **DEMOTE_1/2** can no longer be changed until SEP cold reset clears both registers.
- A signed boot image whose manifest **takes ownership** of the demotion decision for this boot programs **DEMOTE_LOCK_1/2** = 1 as part of committing the decision. The locked value may be **DEMOTE_1/2** = 0 (commit non-demoted state) or **DEMOTE_1/2** = 1 (commit demoted state)—in either case, locking commits the decision and prevents any later boot stage from modifying it until cold reset.
- If the manifest instead **defers** the demotion decision to a later boot stage, it leaves **DEMOTE_LOCK_1/2** = 0 so the next stage can complete the programming. The demotion-decision protocol a manifest uses across boot stages (selector / value / deferral hint) is described in the OCAH SEP boot specification.

Reset Behavior

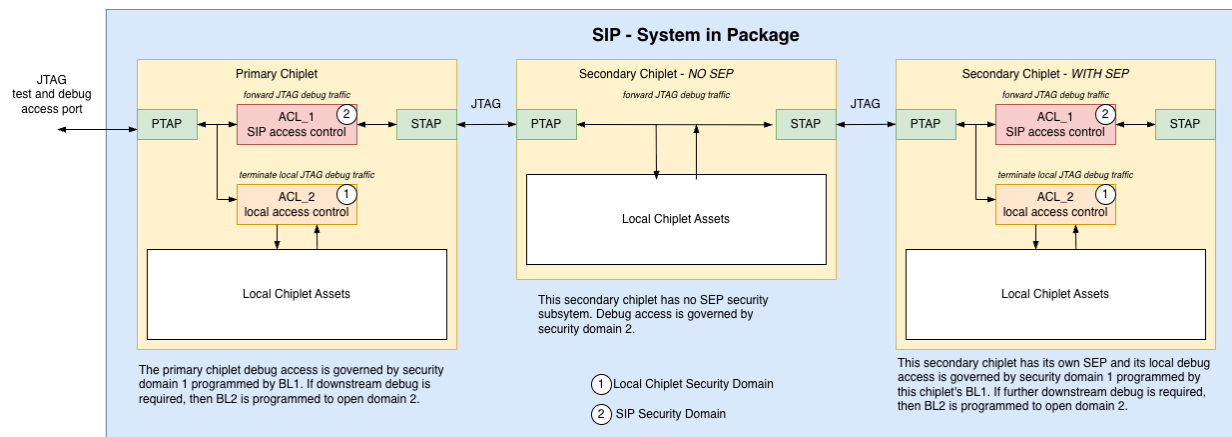
SEP cold reset clears all four demote registers (**DEMOTE_1**, **DEMOTE_2**, **DEMOTE_LOCK_1**, **DEMOTE_LOCK_2**), restoring the device to its OTP-stored LC state.

Multiple Security Domains

OCAH defines two **types** of security domain, distinguished by which owner controls debug/test access to the assets in that domain:

- **SiP-owner domain** — controlled by the **SiP owner**. Governs debug access to SiP-owned assets and the debug-access path into every chiplet (the PTAP chain), including onward access to downstream (secondary) chiplets.
- **Chiplet-owner domain** — controlled by a **chiplet owner**. Governs debug access to that chiplet's own internal assets.

A system instantiates **one** SiP-owner domain (shared) plus **one chiplet-owner domain per chiplet** that has its own protected assets. The diagram below shows three chiplets — one primary and two secondaries — giving three security domains: the SiP-owner domain, chiplet domain 1 (primary), and chiplet domain 2 (secondary). A chiplet may or may not have a local SEP and life-cycle state.



Access is structured as follows:

- **The SiP-owner domain is the debug-access network.** SIP_DBG gates the SiP-owner domain at each chiplet that implements its own SEP/LCC: it admits external (PTAP) debug traffic into that chiplet and passes it onward toward downstream chiplets. Debugging a chiplet's assets requires SIP_DBG to be enabled on each LCC-bearing chiplet along the debug path, including the target chiplet itself when it has its own LCC. A chiplet without a local SEP/LCC has no disable vectors of its own; its debug gates are instead driven by the feature-control bits forwarded from the primary chiplet's LCC, so it is governed by the SiP-owner domain (see [Non-Primary Chiplets](#)).
- **Chiplet-owner domains gate each chiplet's local assets.** A chiplet's own protected assets additionally require that chiplet's local gate (CHIPLET_DBG); the SEP's own internal state has a further local gate, SEP_DBG (see [Disable Vector Format \(SIP_DIS / SYS_DIS\)](#)). Chiplet-owner domains are mutually independent — each is opened separately under its own chiplet owner's control, and opening one chiplet's domain does not require any other chiplet's domain to be open.

For the three-chiplet system in the diagram (chiplet 1 primary, chiplet 2 intermediate secondary without a SEP, chiplet 3 secondary):

- Debugging chiplet 1's local assets requires SIP_DBG(1) and CHIPLET_DBG(1).
- Debugging chiplet 2 requires only SIP_DBG(1): with no local SEP/LCC, chiplet 2 has no disable vectors of its own; its gates are driven by the enables forwarded from the primary chiplet's LCC (see [Non-Primary Chiplets](#)).
- Debugging chiplet 3's local assets requires SIP_DBG(1), SIP_DBG(3), and CHIPLET_DBG(3): chiplet 2 contributes no gate on the path, and the primary chiplet's CHIPLET_DBG(1) is not required.

Local access control is therefore exercised by **both** owners: the SiP owner controls the debug path to every chiplet (the SiP-owner domain), and each chiplet owner controls its own chiplet-owner domain.

LC State Machine

The LC controller is a stateless decoder of the **LC_STATE** shadow register provided by the OTP/Fuse controller. The OTP controller initializes the shadow-register **output** to the INVALID encoding (4'b1111) and only drives it to the OTP-stored LC state value once fuse sensing is complete (see *Fuse Controller (OTP)* in this chapter). The LCC's output therefore tracks fuse sensing automatically: it produces the INVALID feature-control profile before fuse sensing completes and the per-LC-state profile (TEST_DEV / PROD / PROD_END / RMA_SIP / RMA_CHIPLET) afterward.

Field debugging of production packages is supported by the two volatile demotion CSRs (DEMOTE_1, DEMOTE_2) rather than by a separate LC encoding. Demotion acts in TEST_DEV and PROD, opening the corresponding debug group (DBG_1 for Demotion 1, DBG_2 for Demotion 2) as given in [Per-LC-state feature control profile](#); it does not affect DFT or Function, and does not change the RMA_SIP or RMA_CHIPLET profiles, which are fixed by the LC state. A chiplet cold reset resets the DEMOTE_1/2 and DEMOTE_LOCK_1/2 bits.

Demotion 1 and Demotion 2 are independent (see [Demotion 1 and Demotion 2 — In-Field Debug Demotion](#)): DEMOTE_1 opens the local debug group (DBG_1) and DEMOTE_2 opens the inter-chiplet debug-bypass group (DBG_2). Any combination of the two is legal; there is no ordering between them, and neither is tied to an owner level or to an RMA state. Their authorization is governed by the firmware signing model (see [Secure-Boot Signing Model](#)).

Entering RMA_CHIPLET requires two successive token matches: the device must first enter RMA_SIP via the RMA_SIP_TOKEN, then transition to RMA_CHIPLET via the RMA_CHIPLET_TOKEN. Unlike demotion, this RMA ordering is enforced by hardware.

The Key Manager's key derivation function (KDF) uses the DEMOTE_1/2 bits in addition to CHIPLET_UID, SIP_UID, SYS_UID, and LC_STATE as inputs. Mixing the DEMOTE_1/2 bits into the KDF ensures that the derived keys in a demoted (debug) state are distinct from the live keys in the non-demoted PROD state.



During normal chiplet operation, the current LC state is retrieved from OTP and stored differentially in shadow registers, then propagated to the rest of the chiplet—and, from a primary chiplet, potentially onward to secondary chiplets.

The encodings shown in this table are the raw 4-bit LC_STATE values as programmed in OTP. When the fuses are sensed, each value is differentially (complementary) encoded into the shadow registers noted above; the LCC decodes that shadow value with a signal-integrity check that fails closed (forces `feat_ctrl_o = 0`) on any non-complementary codeword, guarding the LC_STATE path against fault injection (e.g. laser or FIB modification).

LC State Name	Encoding	Description
TEST_DEV	4'b0000	Default silicon state at manufacturing time. The chip is open for ATE/SLT testing, fuse provisioning, and mission-mode software development, with certain chiplet features disabled as defined by the chiplet creator, the SiP owner, and the system owner. In this state, the hash digests of the secret tokens (RMA_CHIPLET_TOKEN and RMA_SIP_TOKEN) are programmed into OTP by the chiplet creator and the SiP owner, respectively. The disable vectors (SIP_DIS and SYS_DIS) are likewise programmed into OTP by the chiplet creator and the SiP owner.
PROD	4'b0001	Product functioning state. In-field debug is enabled by the volatile DEMOTE_1 / DEMOTE_2 registers (not a separate LC encoding): DEMOTE_1 opens the local debug group (DBG_1) and DEMOTE_2 opens the inter-chiplet debug-bypass group (DBG_2). See Demotion 1 and Demotion 2 — In-Field Debug Demotion.
PROD_END	4'b1000	Terminal product state; the only permitted transition is to INVALID. This state is entered from TEST_DEV for special use cases in which RMA is not allowed.
RMA_SIP	4'b001X	Enables the SiP owner to debug and test the part. The secret token (RMA_SIP_TOKEN) must be presented to enable the transition from PROD or TEST_DEV.
RMA_CHIPLET	4'b011X	Enables the chiplet owner to debug and test the part. All features are enabled (neither SIP_DIS nor SYS_DIS applies), unlike TEST_DEV, which still honors SIP_DIS/SYS_DIS. The secret token (RMA_CHIPLET_TOKEN) must be presented to enable the transition from RMA_SIP.
INVALID	All other values	Silicon with an invalid OTP state. This is the end-of-life state: the chip is permanently inoperable.

Debug, Test, and Functional Feature Control

Debug, test, and functional features are gated at runtime by the feature control vector `feat_ctrl_o`. This vector is not stored directly: it is **derived** from two 64-bit **disable** vectors held in OTP **shadow registers** — **SIP_DIS** and **SYS_DIS** — as the inverted OR of the two, gated per LC state, then overridden by SEC_DIS and finally gated by SECURE_TM:

```
`feat_ctrl_o = ~(SIP_DIS | SYS_DIS)` (per-LC-state gating; then SEC_DIS / SECURE_TM,
see below)
```

In other words, each feature's enable is the inverse of its disable bits.

Disable Vector Format (SIP_DIS / SYS_DIS)

The table below defines the **disable** vector format—the bit layout shared by both SIP_DIS and SYS_DIS (each is 64 bits). Each defined bit gates a specific feature or its associated scan chain; the remaining bits are reserved for adopter extension (see [Asset and Scan-Chain Protection](#) and [Adopter Extension](#)).

Throughout this chapter **T** (True) means a feature is **enabled** and **F** (False) means it is **disabled**. The **SIP_DIS / SYS_DIS** bits are disable masks: a bit value of **10b1** disables its feature and **10b0** enables it (so a feature is in the **T/enabled** state exactly when its DIS bit is **10b0**).

For demotion, the 32-bit debug group is split into two sub-groups: **DBG_1**—bits **[15:0]** (local debug)—gated by Demotion 1, and **DBG_2**—bits **[31:16]** (the inter-chiplet debug-bypass path)—gated by Demotion 2. The demotion profile in [Per-LC-state feature control profile](#) refers to these sub-groups as SIP_DIS.DBG_1 / DBG_2 and SYS_DIS.DBG_1 / DBG_2.

DIS bit index	Feature Category	Feature Name	Feature Description (10b1 disables the feature; 10b0 enables it)
0	debug (DBG_1)	SEP_DBG	Coarse-grained debug enable for the SEP scope: the local gate for SEP internal state. 10b1 : SEP-scope debug / scan-dump access disabled and inbound (ext) traffic is filtered before it can reach SEP internal state. 10b0 : enabled — inbound traffic bypasses the filter and scan dump is allowed. SEP_DBG is a local gate, not the chiplet ingress: SIP_DBG (bit 16) admits external debug traffic into the chiplet, and SEP_DBG then governs whether that traffic can reach SEP internal state — debugging SEP assets from an external debugger requires both.
1	debug (DBG_1)	CHIPLET_DBG	Coarse-grained debug enable for the chiplet scope. 10b1 : chiplet-scope debug / scan-dump access disabled. 10b0 : enabled. Does not gate the crypto / KM secret-bearing chain, which is secure_tm -gated and never dumpable. In addition, when chiplet-scope debug is disabled (CHIPLET_DBG set in SIP_DIS or SYS_DIS), secure boot is enforced in the TEST_DEV lifecycle state, so a locked-down debug posture cannot be bypassed by loading unsigned code in TEST_DEV.
2-15	debug (DBG_1)	<i>Reserved</i>	Available for adopter-defined local (DBG_1) debug scan chains. See Adopter Extension .
16	debug (DBG_2)	SIP_DBG	Coarse-grained debug enable for the SiP scope: the inter-chiplet debug-bypass gate (DBG_2, gated by Demotion 2) that admits external (PTAP) debug traffic into this chiplet and forwards it onward to the next chiplet via the outbound JTAG PTAP. Debugging a chiplet's assets requires SIP_DBG enabled on each LCC-bearing chiplet along the debug path, including the target itself when it has its own LCC; chiplets without a local SEP/LCC contribute no gate (see Multiple Security Domains). 10b1 : disabled. 10b0 : enabled.
17-31	debug (DBG_2)	<i>Reserved</i>	Available for adopter-defined inter-chiplet-bypass (DBG_2) debug scan chains. See Adopter Extension .
32	test	SEP_FUSE_TEST	10b1 : SEP fuse controller and fuse-block interface scan is disabled. 10b0 : enabled.
33-36	test	<i>Reserved</i>	Available for adopter-defined test scan chains. See Adopter Extension .
37	test	SMC_FUSE_TEST	10b1 : SMC fuse controller and fuse-block interface scan is disabled. 10b0 : enabled. (The bit is produced by SEP-LCC's feat_ctrl_o , exported on the SEP boundary, and consumed by SMC to gate its own fuse-test path.)
38	test	FUSE_VENDOR_TEST	10b1 : OTP IP-vendor test and calibration interface is disabled. 10b0 : enabled. (The gated path is the OTP vendor's, used during ATE for vendor-specific OTP test functions and internal-voltage calibration; the gate itself lives in SEP-LCC's feat_ctrl_o .)
39-47	test	<i>Reserved</i>	Available for adopter-defined test scan chains. See Adopter Extension .
48-63	function	<i>Reserved</i>	Available for adopter-defined functional feature gating. See Adopter Extension .

DIS Vector Ownership: Predecessor-Programs-Disables

OCAH uses a **predecessor-programs-disables** convention. Each X_DIS vector specifies restrictions imposed by an earlier owner that should apply for the rest of the part's life (until the chiplet creator itself regains full access via

RMA_CHIPLET):

- **SIP_DIS** — programmed by the **chiplet creator** before the part is transferred to the SiP integrator. A bit value of 1 disables the corresponding feature for everyone below the chiplet owner — it remains in force through SiP-owner access and is cleared only at chiplet-owner access (RMA_CHIPLET).
- **SYS_DIS** — programmed by the **SiP owner** before the part is transferred to the system integrator. A bit value of 1 disables the corresponding feature for the system owner only — it does not bind the SiP owner or chiplet owner, and is cleared once SiP-owner access (RMA_SIP) is reached.

The system owner has no successor in the OCAH ownership chain, so there is no third DIS vector restricting it. The system owner's own fuse fields (**SYS_PUBK_HASH**, **SYS_UID**) are provisioned by the system owner directly before transitioning the part from TEST_DEV to PROD. See the *Fuse Controller (OTP)* section in this chapter for the full multi-owner provisioning sequence.

Combining Rule: OR then Invert

The per-LC-state formulas combine SIP_DIS and SYS_DIS with bitwise OR, then invert ($\sim(\text{SIP_DIS} \mid \text{SYS_DIS})$) to produce the positive-logic **feat_ctrl_o** output ("1 = feature enabled"). The OR is the natural fuse-level combiner:

- DIS bits live in OTP fuses, which are write-once: a fuse bit can only transition from 0 to 1, never from 1 to 0.
- The DIS convention is "1 = disabled," so each owner's restriction is expressed as a 1 fused into their vector.
- Combining multiple owners' restrictions therefore uses **OR**: a feature is disabled if **any** owner has fused a 1 in that bit position. There is no way for a later owner to "undo" an earlier owner's restriction once fused.
- The output **feat_ctrl_o** uses positive logic ("1 = feature enabled"), so the combined disable is inverted: $\sim(\text{SIP_DIS} \mid \text{SYS_DIS})$.

When Each DIS Vector Applies

Both SIP_DIS and SYS_DIS apply at TEST_DEV, PROD, and PROD_END — including under demotion in PROD, where a demoted debug group still honors the DIS vectors. At **SiP-owner** access (RMA_SIP) only SIP_DIS applies; at **chiplet-owner** access (RMA_CHIPLET) neither applies, because the chiplet creator has full access and no predecessor has authority to restrict them. Two further controls — the SEC_DIS override and the SECURE_TM final gate — sit on top of this; see [Overrides and Final Gates \(SEC_DIS, SECURE_TM\)](#).

LC state	Applicable DIS vectors
TEST_DEV	Both SIP_DIS and SYS_DIS apply. TEST_DEV is the shared phase the part passes through under the chiplet creator, SiP owner, and system owner in turn.
PROD / PROD_END	Both SIP_DIS and SYS_DIS apply.
RMA_SIP	Only SIP_DIS applies. At the SiP-owner level, the SiP owner sits above the system owner, so its own SYS_DIS — which restricts the system owner — does not bind it; only the chiplet creator's SIP_DIS restriction remains in force.
RMA_CHIPLET	Neither SIP_DIS nor SYS_DIS apply (all features enabled).
INVALID	Neither applies (all features disabled by fail-closed default).
(any) with SEC_DIS = 1	Neither DIS vector applies (all features enabled) — but the SECURE_TM gate still forces the test group off when SECURE_TM = 0 (see Overrides and Final Gates (SEC_DIS, SECURE_TM)).

Overrides and Final Gates (SEC_DIS, SECURE_TM)

Two controls sit **on top of** the per-LC-state / DIS-vector computation, applied in this order (a later step overrides an earlier one):

1. **Signal-integrity fail-closed (base):** if the differentially encoded LC_STATE fails its integrity check (`lc_sigint_err`), the base `feat_ctrl` is forced to all-0s (all features disabled).
2. **SEC_DIS override:** when `SEC_DIS = 1`, `feat_ctrl` is forced to all-1s—every feature enabled, regardless of the LC state, the DIS vectors, or the sigint result (an emergency bring-up bypass on A0 silicon; see the Security Disable section).
3. **SECURE_TM final gate (applied last):** when `SECURE_TM = 0` (normal operation—not in secure test mode), the **entire test group** `feat_ctrl_o[47:32]` (`SEP_FUSE_TEST`, `SMC_FUSE_TEST`, `FUSE_VENDOR_TEST`, and the reserved test bits) is forced to 0. Because this gate is applied **after** the SEC_DIS override, the test features are disabled even when `SEC_DIS = 1`—test features are available only when `SECURE_TM = 1`. The debug group `[31:0]` and function group `[63:48]` are not affected by `SECURE_TM`.

Net effect on the test group: a test feature is enabled only if the LC-state/DIS computation (or SEC_DIS) enables it **and** `SECURE_TM = 1`. See also [secure_tm](#) and [Scan Dump](#) for how `secure_tm` relates to the per-LC-state scandump gating.

Per-LC-state Feature Control Profile

Each LC state has a feature control profile that determines which features are enabled on the chiplet. The table below gives, for each combination of LC state, SEC_DIS, and demotion (DEMOTE_1 / DEMOTE_2), the resulting feature control vector as a tuple of four fields: DBG_1 / DBG_2 / DFT / Function. DBG_1 (local debug) is gated by Demotion 1 and DBG_2 (inter-chiplet debug bypass) by Demotion 2; DFT and Function are not affected by demotion. The DFT (test) field is additionally qualified by the SECURE_TM gate, shown inline in the table: the 16-bit test group is forced to 0 unless SECURE_TM = 1 (see [Overrides and Final Gates \(SEC_DIS, SECURE_TM\)](#)). True (T) indicates that a feature is enabled and False (F) indicates that it is disabled. The disable vectors (SIP_DIS / SYS_DIS), defined above, act as masks: a bit value of 1'b0 enables the corresponding feature, and 1'b1 disables it.

Table 219. Per-LC-state feature control profile

LC_STATE	SEC_DIS	DEMOTE (1 / 2)	Feature Control: {DBG_1 / DBG_2 / DFT / Function}	Description
TEST_DEV (4'b0000)	0	0 / 0	$\sim(\text{SIP_DIS.DBG_1} \mid \text{SYS_DIS.DBG_1}) / \sim(\text{SIP_DIS.DBG_2} \mid \text{SYS_DIS.DBG_2}) / (\sim(\text{SIP_DIS.DFT} \mid \text{SYS_DIS.DFT})) \& \backslash\{16\{\text{SECURE_TM}\}\} / \sim(\text{SIP_DIS.FUNC} \mid \text{SYS_DIS.FUNC})$	ATE/SLT test, fuse provisioning (SIP_DIS / SYS_DIS and the RMA tokens), and product firmware development. Both SIP_DIS and SYS_DIS apply; a demotion forces its debug group open, overriding the DIS vectors. DFT and Function are not affected by demotion.
	0	1 / 0	$T / \sim(\text{SIP_DIS.DBG_2} \mid \text{SYS_DIS.DBG_2}) / (\sim(\text{SIP_DIS.DFT} \mid \text{SYS_DIS.DFT})) \& \backslash\{16\{\text{SECURE_TM}\}\} / \sim(\text{SIP_DIS.FUNC} \mid \text{SYS_DIS.FUNC})$	
	0	0 / 1	$\sim(\text{SIP_DIS.DBG_1} \mid \text{SYS_DIS.DBG_1}) / T / (\sim(\text{SIP_DIS.DFT} \mid \text{SYS_DIS.DFT})) \& \backslash\{16\{\text{SECURE_TM}\}\} / \sim(\text{SIP_DIS.FUNC} \mid \text{SYS_DIS.FUNC})$	
	0	1 / 1	$T / T / (\sim(\text{SIP_DIS.DFT} \mid \text{SYS_DIS.DFT})) \& \backslash\{16\{\text{SECURE_TM}\}\} / \sim(\text{SIP_DIS.FUNC} \mid \text{SYS_DIS.FUNC})$	
	1	X	$T / T / \backslash\{16\{\text{SECURE_TM}\}\} / T$	
PROD (4'b0001)	0	0 / 0	$F / F / F / \sim(\text{SIP_DIS.FUNC} \mid \text{SYS_DIS.FUNC})$	Product state. A demotion relaxes its debug group to honor the DIS vectors (one level down from disabled). Both SIP_DIS and SYS_DIS still apply; DFT and Function are not affected by demotion.
	0	1 / 0	$\sim(\text{SIP_DIS.DBG_1} \mid \text{SYS_DIS.DBG_1}) / F / F / \sim(\text{SIP_DIS.FUNC} \mid \text{SYS_DIS.FUNC})$	
	0	0 / 1	$F / \sim(\text{SIP_DIS.DBG_2} \mid \text{SYS_DIS.DBG_2}) / F / \sim(\text{SIP_DIS.FUNC} \mid \text{SYS_DIS.FUNC})$	
	0	1 / 1	$\sim(\text{SIP_DIS.DBG_1} \mid \text{SYS_DIS.DBG_1}) / \sim(\text{SIP_DIS.DBG_2} \mid \text{SYS_DIS.DBG_2}) / F / \sim(\text{SIP_DIS.FUNC} \mid \text{SYS_DIS.FUNC})$	
	1	X	$T / T / \backslash\{16\{\text{SECURE_TM}\}\} / T$	
PROD_END (4'b1000)	0	—	$F / F / F / \sim(\text{SIP_DIS.FUNC} \mid \text{SYS_DIS.FUNC})$	Terminal product state—normal function only; the only permitted transition is to INVALID. Demotion has no effect.
	1	—	$T / T / \backslash\{16\{\text{SECURE_TM}\}\} / T$	

LC_STATE	SEC_DIS	DEMOTED (1 / 2)	Feature Control: {DBG_1 / DBG_2 / DFT / Function}	Description
RMA_SIP (4'b001X)	0	—	~SIP_DIS.DBG_1 / ~SIP_DIS.DBG_2 / (~SIP_DIS.DFT) & {16{SECURE_TM}} / ~SIP_DIS.FUNC	SiP-owner debug/test. Honors SIP_DIS only (SYS_DIS does not bind the SiP owner). Entered via RMA_SIP_TOKEN. Demotion has no effect.
	1	—	T / T / {16{SECURE_TM}} / T	
RMA_CHIPLET (4'b011X)	X	—	T / T / {16{SECURE_TM}} / T	Chiplet-owner debug/test — all features enabled. Entered via RMA_CHIPLET_TOKEN.
INVALID (others)	0	—	F / F / F / F	The chip is not live: end of life, immediately after reset, or an invalid LC_STATE.
	1	—	T / T / {16{SECURE_TM}} / T	

In the DEMOTE column, a dash means demotion has no effect in that state, and X means it is irrelevant because SEC_DIS forces all features on. Demotion acts in TEST_DEV and PROD: in TEST_DEV a demotion forces its debug group fully open (overriding SIP_DIS / SYS_DIS), while in PROD it relaxes its debug group to honor the DIS vectors (one level down). The two demotions are independent — DEMOTE_1 gates DBG_1 and DEMOTE_2 gates DBG_2 — and neither affects DFT or Function. On the DFT field, the SECURE_TM term is the inline form of the SECURE_TM final gate — the 16-bit test group is forced to 0 unless SECURE_TM = 1 (see [Overrides and Final Gates \(SEC_DIS, SECURE_TM\)](#)).

Asset and Scan-Chain Protection

Two distinct DFT operations use the scan chains, and the protection model treats them differently:

- **Scan test** — shift in a test pattern, run the functional clock for one or two cycles, then shift the result out. Used for manufacturing / DFT coverage.
- **Scan dump** — shift the live internal state out directly. Used for debug.

The design goal is to **maximize** DFT coverage (keep as many flops on the scan chain as possible) while ensuring secrets can never be **dumped**. This section maps SEP asset classes to scan chains and to the gates that control each.

Asset Classes

SEP assets are partitioned into three classes. Class 1 (non-scannable) is split by protection rationale — 1a (confidentiality) and 1b (integrity). Class 2 (crypto / Key Manager secret-bearing) is scannable for test coverage but is **never** scan-dumpable. Class 3 is everything else.

Class	Asset Examples	Scannable (test)	Scan-dumpable (debug)	Gate & Scan-Chain Treatment
1a (non-scannable; confidentiality)	UIDs (CHIPLET_UID, SIP_UID, SYS_UID); *_TOKEN_I JTAG input registers (SEC_DIS_TOKEN_I, RMA_SIP_TOKEN_I, RMA_CHIPLET_TOKEN_I); CLASS_KEY	No	No	Excluded from all scan chains. Confidentiality rationale: these registers hold the actual secrets; the data lives in them, so the only safe option is to keep them off every scan chain.
1b (non-scannable; integrity)	LOCKS, LC_STATE, SIP_DIS, SYS_DIS shadow registers; *_TOKEN_DIGEST shadow registers; token-hash digest/valid and RMA match state	No	No	Excluded from all scan chains. Integrity rationale: the values are not secret (digests are SHA-256 preimage-resistant), but scan shifting could change them and illegitimately enable a disabled feature or make an attacker-presented token match. Stability against scan-induced perturbation is required.

Class	Asset Examples	Scannable (test)	Scan-dumpable (debug)	Gate & Scan-Chain Treatment
2 (crypto / KM secret-bearing)	The complete Key Manager; AES, HMAC-SHA2, KMAC-SHA3, OTBN, entropy, DRBG/EDN, Adams Bridge, and TRNG sequential state; KM/OTBN companion SRAMs (see Class 2 Asset Manifest and Naming)	Yes	Never	Dedicated secret-bearing scan chain. Scannable for DFT coverage when <code>secure_tm = 1</code> ; the secret-bearing shadow registers (UIDs / <code>CLASS_KEY</code>) remain disconnected from downstream, so these consumers hold no real secret during scan test. Scan dump is never permitted—for any owner or LC state: under <code>secure_tm = 0</code> the chain is tied off (see <code>secure_tm</code> and Scan Dump). This is independent of the <code>*_DBG</code> feature control bits.
3 (general logic)	The rest of the logic not classified above	Yes	Yes (gated)	General scan chains. Scannable and scan-dumpable when permitted by <code>secure_tm</code> , the <code>*_DBG</code> feature control bits (the scope's local gate— <code>SEP_DBG</code> or <code>CHIPLET_DBG</code> —plus the <code>SIP_DBG</code> debug-access path; see Multiple Security Domains), and the LC state.

secure_tm and Scan Dump

`secure_tm` is the secure-test-mode indicator: the `TEST_EN` strap latched into the internal `SECURE_TM` signal. It is latched when SEP fuse sensing completes if `SEC_DIS` is not asserted, or on chiplet cold-reset release if `SEC_DIS` is asserted (see the Test Mode section). Tests can be enabled only when `SECURE_TM = 1`.

`secure_tm` protects secrets at **two** points, which together let the crypto and Key Manager logic stay on the scan chain (maximizing DFT coverage) while never exposing secrets:

- **During scan test (`secure_tm = 1`):** the secret-bearing shadow-register outputs (`CHIPLET_UID` / `SIP_UID` / `SYS_UID`, `CLASS_KEY`) are **disconnected** from downstream logic, OTP fuse-write enables are tied inactive, and `LOCKS` / `LC_STATE` / `SIP_DIS` / `SYS_DIS` are locked against updates (see *Fuse Controller (OTP)*). So Class 2 consumers can be shifted and tested without any real secret being present in them.
- **During scan dump (`secure_tm = 0`):** the dedicated Class 2 secret-bearing scan chain is tied off, so its contents can never be extracted via debug—**independent** of the `*_DBG` feature control bits and of the LC state.

The Class 1 vs Class 2 split follows from **where each asset sits**:

- **Class 2 can be scannable** because its registers are **consumers** of secrets reached over functional paths; `secure_tm = 1` cuts those paths during test.
- **Class 1 cannot be scannable** because its registers **are** the secrets (Class 1a) or the integrity-critical control fields (Class 1b)—the data lives in the registers themselves, so they are excluded from every scan chain.

The `_DBG` feature control bits are a separate, coarser control over `*general` (Class 3) debug / scan-dump access. `SEP_DBG` and `CHIPLET_DBG` are local gates for their scopes' logic; `SIP_DBG` gates the debug-access path into the chiplet, so an external debugger reaches a scope's logic only when `SIP_DBG` and that scope's local gate are both open (see [Multiple Security Domains](#)). None of these bits affect the Class 2 secret-bearing chain, which is governed solely by `secure_tm`.

Feature-Bit to Scan-Chain Mapping

Each **general** (Class 3) scan chain is gated by a per-scope `_DBG` feature control bit in `feat_ctrl_o`; the bit's owner—which predecessor DIS vector sets its default (see [DIS Vector Ownership: Predecessor-Programs-Disables](#))—sets the multi-owner default policy. External access to any general chain additionally requires the `SIP_DBG` debug-access path into the chiplet to be open (see [Multiple Security Domains](#)). The Class 2 secret-bearing chain is **not* gated by a feature control bit; it is governed by `secure_tm` and is never dumpable.

Asset Class	Scan Chain	Dump Gate	Notes
1a / 1b (non-scannable)	—	—	Excluded from all scan chains.
2 (crypto / KM secret-bearing)	Dedicated secret-bearing chain	<code>secure_tm</code> (chain tied off when <code>secure_tm = 0</code>)	Never scan-dumpable, any owner / LC state. Scannable under <code>secure_tm = 1</code> , with shadow secrets disconnected.
3 (general logic)	Per-scope general chains	Local scope <code>*_DBG</code> bit (<code>SEP_DBG</code> , <code>CHIPLET_DBG</code>) + <code>SIP_DBG</code> access path + LC state	Coarse-grained: a scope's general logic is scan-dumpable only when its local <code>*_DBG</code> bit enables it, the <code>SIP_DBG</code> debug-access path is open, and the LC state permits.

Reserved-bit ranges available for adopter-defined scan chains (see [Adopter Extension](#)):

- Debug group: bits 2-15 (DBG_1, 14 reserved bits) and bits 17-31 (DBG_2, 15 reserved bits)
- Test group: bits 39-47 (9 reserved bits)
- Function group: bits 48-63 (16 reserved bits)

Non-Primary Chiplets

Primary chiplets always include a local SEP and LCC; the asset partitioning described in [Asset and Scan-Chain Protection](#) applies directly. **Non-primary** chiplets may not include a local SEP, in which case lifecycle and debug enablement is governed by the **SiP-owner domain**, forwarded from the primary chiplet's LCC (see [Multiple Security Domains](#)).

For non-primary chiplets, the adopter is responsible for:

- Identifying which non-primary chiplet assets require protection.
- Partitioning those assets across scan chains analogous to the class scheme in [Asset Classes](#) (1a/1b, 2, 3).
- Wiring the non-primary chiplet's scan / DFT enables to the appropriate feature control bits forwarded from the primary chiplet (typically driven by the primary chiplet's `SIP_DBG` bit and other downstream-debug signals in `feat_ctrl_o`).
- Applying the same scan-protection treatment to any non-primary chiplet logic that handles secrets: Class 1 excluded from scan chains, Class 2 scandump-gated by the forwarded feature control bits.

The cross-chiplet propagation of feature control bits (via the `SIP_DBG` mechanism documented in [Disable Vector Format \(SIP_DIS / SYS_DIS\)](#)) is the contract by which the primary chiplet's LCC reaches the non-primary chiplet.

Adopter Extension

If an adopter integrates SEP into a chiplet with assets that require their own scan-chain protection — assets not covered by the chiplet-scope SEP scan chain or the general-logic scan chain in [Asset and Scan-Chain Protection](#) — the LCC architecture accommodates this via the **reserved bits** in `feat_ctrl_o` (and the matching positions in `SIP_DIS` / `SYS_DIS`):

- Debug group: bits 2-15 (DBG_1, 14 reserved bits) and bits 17-31 (DBG_2, 15 reserved bits)
- Test group: bits 39-47 (9 reserved bits)
- Function group: bits 48-63 (16 reserved bits)

Each new scan chain is gated by exactly one reserved bit. The corresponding bit position in `SIP_DIS` or `SYS_DIS` (chosen by which predecessor owner sets the default policy — see [DIS Vector Ownership: Predecessor-Programs-Disables](#)) determines the default OTP-programmed restriction.

Constraints any adopter extension must satisfy:

- **One feature control bit per scan chain.**
- **Asset-class compliance.** Secret-bearing fields used by the new chain must follow the asset-class scheme (see [Asset and Scan-Chain Protection](#)): actual secrets and integrity-critical state are excluded from scan chains (Class 1), and any scannable secret-consuming logic (Class 2) has its scandump gated by the chain's feature control bit — the same treatment that makes Class 2 safely scannable.
- **Per-LC-state policy.** The new bit follows the standard per-LC-state formulas (see [Per-LC-state feature control profile](#)): disabled in PROD and PROD_END, bypassed by SEC_DIS, enabled at chiplet-owner access (RMA_CHIPLET) or under demotion, and subject to the predecessor-programs-disables convention for TEST_DEV and SiP-owner access.

NOTE

The specific bit-allocation convention — which reserved bits are reserved for which adopters, and how bit assignments are advertised in the RDL / package outputs — is currently **open** and will be defined in a future revision of this specification.

4.9. Test Mode Entry

Chiplets shall implement a strap TEST_EN for test mode entry. TEST_EN is latched when SEP fuse sensing is done if SEC_DIS is not asserted. TEST_EN is latched on chiplet cold reset release if SEC_DIS is asserted.

The latched value becomes an internal signal SECURE_TM. Only when SECURE_TM is asserted to 1 can any tests be enabled. While Table tbd defines the static enablement of different tests, SECURE_TM provides dynamic control on top of that.

The test feature control signals sent from SEP to TAP are qualified by SECURE_TM == 1 in order to enable the tests. The SECURE_TM is also used in SEP to protect secrets that cannot be erased before entering RMA modes. The latched SECURE_TM value can only be cleared by chip reset.

Chapter 5. System Management Controller (SMC)

5.1. Overview

The System Management Controller (SMC) is a subsystem in a chiplet that takes on the system management tasks, including managing clock/voltage/reset, configuring the hardware platform, bringing up the system, and reporting system status.

Given its essential role, a SiP with multiple chiplets integrates one SMC per chiplet. Each SMC instance in a chiplet is responsible for system management of the local chiplet. On top of that, a management hierarchy is expected to be built among different SMC instances, according to the OCA hierarchical system architecture, , Sec. 3.1 and 3.2. This means an SMC instance in the primary chiplet is expected to play additional roles to manage SiP level tasks for the secondary chiplets.

The SMC consists of a micro-controller and different facilitative components for various purposes. At high level, SMC's main control logic is implemented in the firmware running on the micro-controller. Hardware facilitative components are added only when certain tasks cannot be implemented by firmware.

The SMC architecture is organized around a RISC-V CPU cluster that executes firmware-based control logic. The CPU operates within the primary SMC clock domain and includes associated cache hierarchies for efficient instruction and data access. The system employs a multi-domain clocking architecture with a high-speed SMC clock domain for computational components, a stable reference clock domain for timing-sensitive operations, and a peripheral clock domain for external interfaces.

The interconnect fabric provides AXI4 and AXI4-Lite bus infrastructure connecting the CPU to various system components. This fabric includes address remapping engines for security and virtualization support, traffic filtering mechanisms for access control, and crossbar switches for efficient data routing.

Memory resources include read-only memory (ROM) for firmware storage and scratchpad memory for runtime data. The memory system is complemented by a DMA controller for efficient bulk data transfers and a memory zeroer for secure data clearing operations.

Peripheral interfaces enable communication and control of external systems. These include GPIO controllers for general-purpose I/O, I2C and UART interfaces for serial communication, AVSBus controller for adaptive voltage scaling, and PVT (Process, Voltage, Temperature) sensors for environmental monitoring. A telemetry receiver collects system performance data, while the system timer OCTS provides multi-chiplet time synchronization.

System management functions include clock and reset controllers for power and reset sequencing, an interrupt controller (PLIC) for handling system events, and core-local interruptors (CLINT) for timer and software interrupt management. Debug infrastructure provides JTAG access and RISC-V debug module support for development and troubleshooting.

The figure below shows the SMC microarchitectural diagram.

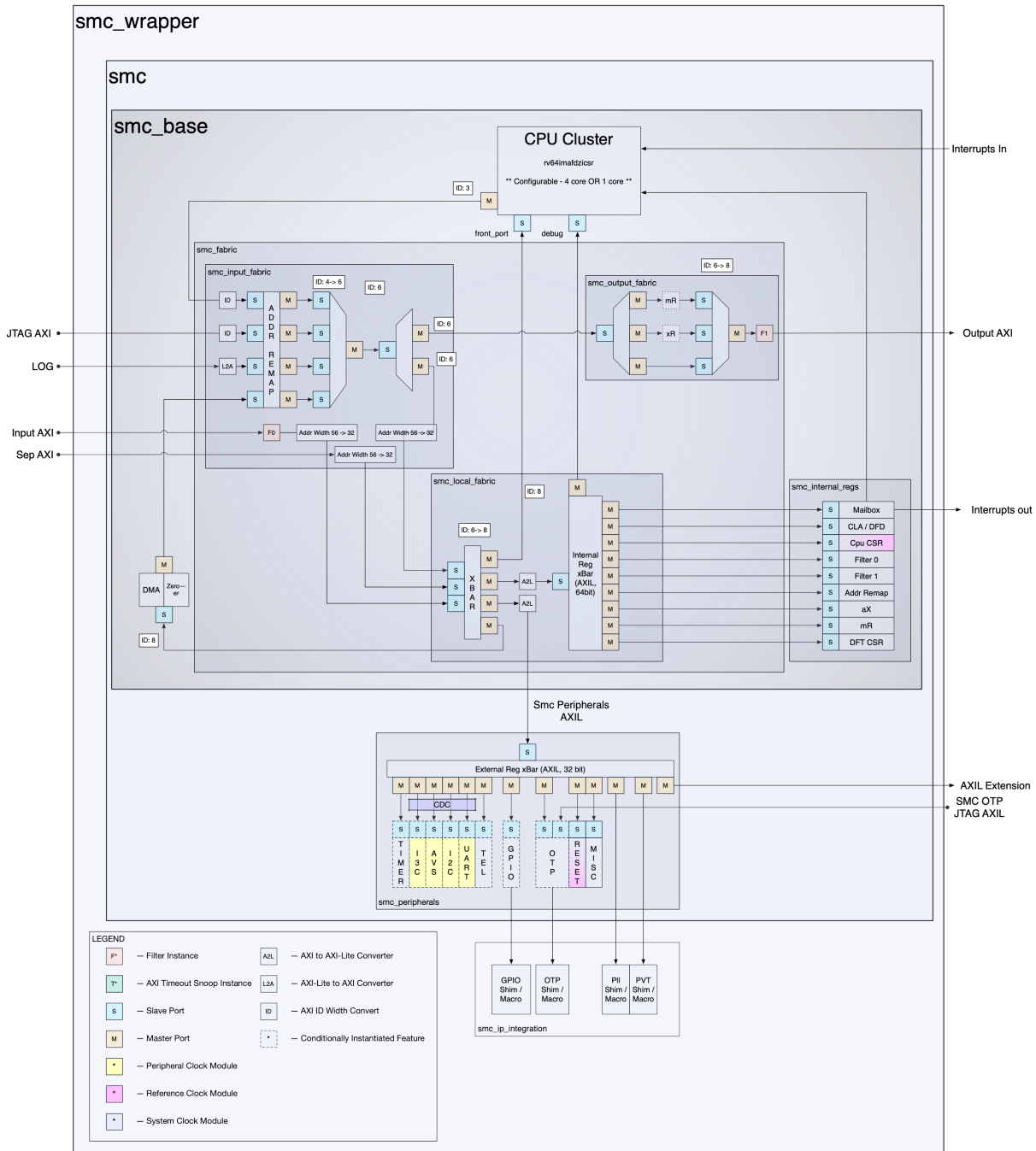


Figure 2. SMC Top-Level Block Diagram

The rest of this documentation describes architectural requirements and microarchitectural implementation of SMC's components in OCAH. A consolidated description of OCAH use cases is available in the top-level [Use Cases](#) section.

5.2. Memory Map

This section provides a comprehensive overview of the SMC memory map. The SMC address space is organized to provide access to various functional units including CPU control, peripheral interfaces, system management functions, and fabric control mechanisms.

All addresses below are expressed as **BASE + offset**, where **BASE** can be either the local alias (**LOCAL_BASE**, fixed read-only at **0xC000_0000**) or the global system address (**GLOBAL_BASE**, programmable by firmware). The fabric routes transactions to SMC resources identically regardless of which base is used, as long as the access falls inside the relevant local or global

aperture. The current SMC **REGION_SIZE** CSR reset is 16 MiB for the chiplet-global aperture; the input fabric’s internal local-alias window is controlled by the RTL **LOCAL_ALIAS_REGION_SIZE** parameter, currently fixed at 16 MiB. Addresses are 32-bit aligned and follow standard AXI4-Lite addressing conventions. For details on how **LOCAL_BASE** and **GLOBAL_BASE** relate and how an adopter maps this aperture into the system address space, refer to the SMC Address Map Integration section of the Integration Guide.

5.2.1. Address Space Organization

The SMC address space is divided into several major functional regions:

Table 220. SMC Address Space Layout

Address Range	Size	Function	Description
BASE + 0x000_0000 - BASE + 0x000_0FFF	4KB	Watchdog Timers	Per-core watchdog timer interfaces (4 cores, 1KB each)
BASE + 0x000_1000 - BASE + 0x000_1FFF	4KB	Debug Unit	RISC-V Debug Module interface
BASE + 0x000_2000 - BASE + 0x000_2FFF	4KB	System Control	Reset unit and miscellaneous system control registers
BASE + 0x000_3000 - BASE + 0x000_3FFF	4KB	GPIO Interface	General Purpose I/O control and configuration (65 GPIO instances)
BASE + 0x000_4000 - BASE + 0x000_5FFF	8KB	I3C Controllers	OCA I3C controller instances (6 instances)
BASE + 0x000_6000 - BASE + 0x000_8FFF	12KB	Peripheral Controllers	AVSBus, I2C, and UART interfaces
BASE + 0x000_9000 - BASE + 0x000_DFFF	20KB	Security, Timing & DFT	eFuse, telemetry, system timer, DTP control, and DFT status
BASE + 0x001_0000 - BASE + 0x001_7FFF	32KB	Fabric Control	CPU control, address remapping, and filtering
BASE + 0x001_8000 - BASE + 0x003_7FFF	128KB	Mailbox	Inter-processor communication mailbox (32 channels: 16 inbound + 16 outbound)
BASE + 0x003_8000 - BASE + 0x003_83FF	1KB	Data Processing	DMA controller and memory zeroer
BASE + 0x004_0000 - BASE + 0x007_FFFF	256KB	Memory Regions	ROM and scratchpad memory spaces
BASE + 0x016_0000 - BASE + 0x016_8FFF	36KB	CLA	Debug and trace (CLA) module
BASE + 0x040_0000 - BASE + 0x040_57FF	22KB	AXI-Lite External	Adopter extension window: mandatory technology-specific blocks (GPIO pads, PLL, PVT, eFuse SHIM) plus optional supplementary devices
BASE + 0x080_0000 - BASE + 0x1FF_FFFF	24MB	Address Remapping	M-mode and Xvisor remapped address regions
BASE + 0x400_0000 - BASE + 0x43F_FFFF	4MB	Interrupt Controller	RISC-V PLIC (Platform-Level Interrupt Controller)
BASE + 0x800_0000 - BASE + 0x801_3FFF	80KB	Timer & Bus Error	CLINT and per-core Bus Error Units

5.2.2. Detailed Address Map

The following table provides specific address assignments for all SMC components:

Table 221. SMC Component Address Map

Component	Address Range	Instance	Description
WDT Core 0-3	BASE + 0x000_0000 + (N×0x400)	4 instances	Watchdog timers for each CPU core
Debug Module	BASE + 0x000_1000 - BASE + 0x000_1FFF	1 instance	RISC-V debug interface
Reset Unit	BASE + 0x000_2000 - BASE + 0x000_27FF	1 instance	System reset control
Miscellaneous Wrapper	BASE + 0x000_2800 - BASE + 0x000_2FFF	1 instance	System configuration registers
GPIO Interface	BASE + 0x000_3000 + (N×0x10)	65 instances	GPIO interface control
GPIO Control	BASE + 0x000_3440 + (N×0x20)	65 instances	GPIO pad configuration
OCA I3C Controllers	BASE + 0x000_4000 + (N×0x500)	6 instances	OCA I3C controller instances
AVSBus Controller	BASE + 0x000_6000 - BASE + 0x000_6FFF	1 instance	Adaptive voltage scaling control
I2C Wrapper	BASE + 0x000_7000 - BASE + 0x000_7FFF	1 instance	I2C/SMBus communication interface
UART Wrapper	BASE + 0x000_8000 - BASE + 0x000_8FFF	1 instance	UART communication interface
eFuse Map	BASE + 0x000_9000 - BASE + 0x000_9FFF	1 instance	eFuse memory mapping interface
eFuse Interface	BASE + 0x000_A000 - BASE + 0x000_AFFF	1 instance	eFuse interface controller
Telemetry Receiver	BASE + 0x000_B000 - BASE + 0x000_BFFF	1 instance	Telemetry data processing
System Timer OCTS	BASE + 0x000_C000 - BASE + 0x000_CFFF	1 instance	Multi-chiplet time synchronization
DTP Control Registers	BASE + 0x000_D000 - BASE + 0x000_D7FF	512 entries	DTP control register block
DFT Control/Status	BASE + 0x000_D800 - BASE + 0x000_DFFF	1 instance	DFT control and status for ROM
CPU Control	BASE + 0x001_0000 - BASE + 0x001_1FFF	1 instance	CPU cluster management
Alias Remap	BASE + 0x001_2000 + (N×0x20)	8 instances	Address alias remapping
M-mode Remap	BASE + 0x001_3000 + (N×0x8)	8 instances	Machine mode address remapping
Xvisor Remap	BASE + 0x001_4000 + (N×0x8)	8 instances	Hypervisor address remapping
Inbound Filters	BASE + 0x001_5000 + (N×0x20)	16 instances	Inbound traffic filtering
Outbound Filters	BASE + 0x001_6000 + (N×0x20)	16 instances	Outbound traffic filtering
Mailbox	BASE + 0x001_8000 - BASE + 0x003_7FFF	32 channels	Inter-processor communication (16 inbound + 16 outbound)
DMA Controller	BASE + 0x003_8000 - BASE + 0x003_81FF	1 instance	Direct memory access controller
Memory Zeroer	BASE + 0x003_8200 - BASE + 0x003_83FF	1 instance	Memory initialization unit
SPM ROM Memory	BASE + 0x004_0000 - BASE + 0x005_FFFF	128KB	Scratchpad ROM memory space
SPM Memory	BASE + 0x006_0000 - BASE + 0x007_FFFF	128KB	Scratchpad memory space
SMC CLA	BASE + 0x016_0000 - BASE + 0x016_8FFF	1 instance	Debug and trace (CLA) module

Component	Address Range	Instance	Description
AXI-Lite External	BASE + 0x040_0000 - BASE + 0x040_57FF	1 window	Adopter extension window, split into a mandatory and a supplementary region (see the AXI-Lite External Window section below)
M-mode Region	BASE + 0x100_0000 - BASE + 0x17F_FFFF	8MB	Machine mode remapped space
Xvisor Region	BASE + 0x180_0000 - BASE + 0x1FF_FFFF	8MB	Hypervisor remapped space
PLIC	BASE + 0x400_0000 - BASE + 0x43F_FFFF	4MB	Platform-Level Interrupt Controller
CLINT	BASE + 0x800_0000 - BASE + 0x800_FFFF	64KB	Core Local Interruptor
Bus Error Units	BASE + 0x801_0000 + (N×0x1000)	4 instances	Per-core bus error handling

5.2.3. AXI-Lite External Window

The `smc_external` window at `BASE + 0x040_0000` is the adopter extension window. OCAH exposes this AXI-Lite bus so an adopter can attach their own technology- and vendor-specific implementations of memory-mapped blocks. The window is split into two regions (see `smc_external.rdl`); hardware treats both identically — the split is purely organizational:

- **Mandatory** (`BASE + 0x040_0000`): blocks every integration must implement. Their function is dictated by the OCAH specifications, but their implementation is inherently technology- and vendor-specific — the GPIO pad controls, the PLL macro, the PVT sensors, and the eFuse SHIM.
- **Supplementary** (`BASE + 0x040_4000`): optional region for any additional memory-mapped devices the adopter wants to expose. The reference integration places its vendor I3C controllers (nonfree overlay) here.

Addresses below are expressed as `BASE + offset` (`BASE = 0xC000_0000`).

Table 222. SMC AXI-Lite External Window — Mandatory Region

Component	Address Range	Instance	Description
PLL Clock-Obs GPIO	BASE + 0x040_0000 (intf), BASE + 0x040_000C (ctrl)	1 instance	PLL clock observation GPIO
PVT Clock-Obs GPIO	BASE + 0x040_0010 (intf), BASE + 0x040_001C (ctrl)	1 instance	PVT clock observation GPIO
GPIO PoC/PBias Control	BASE + 0x040_0020	1 instance	Power-on and pad bias control
GPIO Refclk Control	BASE + 0x040_002C	1 instance	Reference clock GPIO control
Per-Pad GPIO Control	BASE + 0x040_0100 + (N×0x20)	65 instances	Per-pad control registers for the pad ring
PLL Wrapper	BASE + 0x040_1000	1 instance	Clock generation and management
PVT Wrapper	BASE + 0x040_2000	1 instance	Process, Voltage, Temperature monitoring
eFuse SHIM	BASE + 0x040_3000	1 instance	Adopter eFuse hardware abstraction layer

Table 223. SMC AXI-Lite External Window — Supplementary Region (optional)

Component	Address Range	Instance	Description
Vendor I3C Controllers	BASE + 0x040_4000 + (N×0x400)	6 instances	Vendor I3C controller wrappers (nonfree overlay)

The unallocated remainder of the window is passed through to the chip-level adopter external port.

5.2.4. Register Interface Standards

All SMC components use standard AMBA interfaces. We summarize the interfaces in the following table:

Table 224. Register Interface Standards

Interface	Address Width	Data Width	Description
AXI4-Lite	32-bit	64-bit	High-performance components
APB4	32-bit	32-bit	Peripheral components
Address Alignment: 32-bit registers on 4-byte boundaries, 64-bit on 8-byte boundaries			
Endianness: Little-endian byte ordering throughout			
Protection: AXI protection bits used for secure/non-secure access control			

5.2.5. Access Control and Security

The SMC implements the following access control mechanisms:

Table 225. Access Control Mechanisms

Mechanism	Description
Address Filtering	Configurable inbound and outbound address filters
Protection Levels	AXI protection attribute enforcement
Secure Regions	eFuse and security-critical registers require elevated privileges
Error Handling	Bus error units provide comprehensive error logging and reporting

5.2.6. Component-Specific Register Documentation

For detailed register-level documentation of each SMC component, refer to the individual component documentation:

Table 226. SMC Component Documentation

Category	Description	Reference
CPU and System Control	CPU Control, PLIC, CLINT, Watchdog Timers, and Bus Error Units	cpu
Interrupt Controller	Platform-Level Interrupt Controller (PLIC) details	interrupts
DMA Controller	DMA Controller detailed register interface	dma
Memory Zeroer	Memory Zeroer control registers	zeroer
Fabric Control	Address remapping, inbound/outbound filters, and fabric control registers	fabric
Peripherals and I/O	GPIO, UART, I2C, PVT, eFuse, Telemetry, and System Timer registers	periphs
Reset Management	Reset unit and system control registers	clk_rst
Clock Generation	PLL wrapper and clock generation registers	pll

5.3. Clock and Reset Management

The SMC clock and reset management architecture supports multiple clock domains, hierarchical reset structures, and selective power management through clock gating.

5.3.1. Clock Architecture

The SMC employs a multi-domain clocking architecture.

The SMC Clock Domain

The primary SMC clock domain (`clk_smc_i`) serves as the operational clock for the majority of SMC functionality. This domain encompasses the computationally intensive elements of the system management controller.

Within this clock domain, the RISC-V CPU cluster operates with its associated cache hierarchies.

The local fabric infrastructure, including the AXI crossbar and interconnect components, shares this clock domain to maintain coherent, high-performance data movement between system management components.

Control logic components including address remapping engines and filtering mechanisms also belong to this clock domain.

The Reference Clock Domain

The reference clock domain (`clk_ref_i`) provides a low-speed, stable timing foundation for operations that require consistent, predictable timing behavior.

System timers, including the Core Local Interruptor (CLINT) and specialized system timer OCTS (On-Chip Time Synchronization), operate within this reference domain.

This clock is used as reference clock for PLL control and frequency synthesis.

The reference clock domain also serves as the synchronization anchor for reset domain crossings, providing the stable timing reference necessary for implementing robust reset synchronization mechanisms.

Debug infrastructure, including JTAG interfaces and debug module operations, benefit from this stable timing reference to ensure reliable debug access independent of system operational state.

The Peripheral Clock Domain

The peripheral clock domain (`clk_periph_i`) provides a dedicated timing source for all peripheral controllers instantiated within the SMC peripheral wrapper (`smc_external_peripheral_wrapper`). This clock is supplied by the platform's clock generation infrastructure. A minimum frequency of 100 MHz applies; the upper limit is not fixed and is set by the integrator according to timing closure, peripheral IP limits, and system power and performance goals.

The following peripherals operate within this clock domain:

- AVSBus Controller
- I2C Controller
- UART 16550
- I3C Controller

The peripheral register crossbar also operates in this clock domain. A CDC bridge at the crossbar's upstream slave port synchronizes register accesses arriving from the CPU fabric.

The register path to the GPIO controller and the pad interface includes a dedicated AXI-Lite CDC from the peripheral

clock domain into the reference clock domain. The pad multiplexing logic and the physical pad I/O interface operate in the always-on reference clock domain.

Decoupling peripheral timing from the high-frequency CPU and fabric clock enables independent frequency scaling of the peripheral subsystem, reducing dynamic power consumption while still meeting the timing requirements of external interfaces.

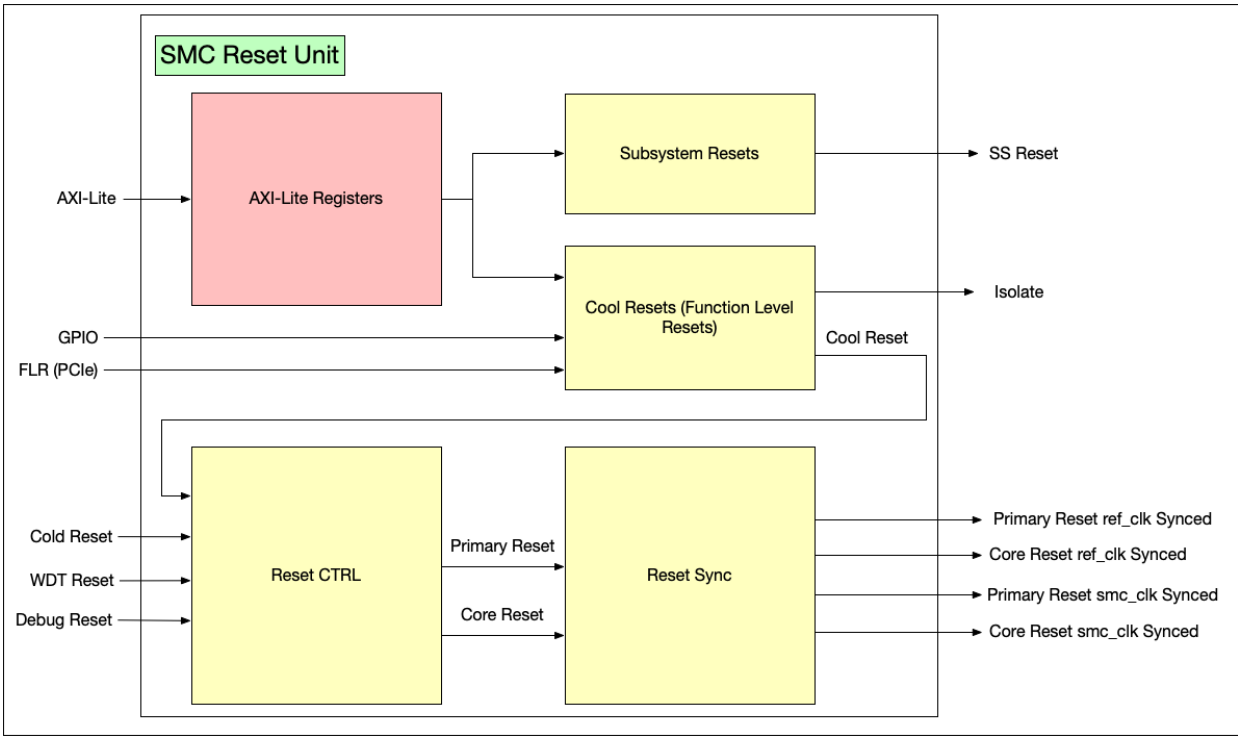
The Telemetry Clock Domain

The telemetry clock domain (clk_telemetry_i) provides timing for the telemetry receiver module. This clock is sourced from the telemetry data interface itself and does not require a dedicated PLL within the SMC. The integrator supplies this clock along with the incoming telemetry data stream, ensuring proper synchronization between the clock and data.

The telemetry receiver uses this clock for capturing and processing incoming ATB (Advanced Trace Bus) telemetry data from multi-chiplet systems.

5.3.2. Reset Architecture

The SMC employs a hierarchical reset strategy with multiple reset sources and synchronization mechanisms. The following diagram shows the reset architecture.



The following table lists the reset sources and their characteristics. BP_POWERGOOD is the board power-good indicator used as the power-on reset (POR) root: it is stretched into powergood_stable, which qualifies the SMC functional cold reset path, and it is also passed into the JTAG/DTP stack as pwr_on_rst_ni. In jtag_ptap, pwr_on_rst_ni is ANDed with the pad TRST signal to form the effective TAP reset, so loss of power-good forces TAP/TDR logic into reset even when TRST is released. BP_RESETN drives the SMC functional cold reset path after power-good is stable.

Table 227. SMC Reset Sources and Characteristics

Reset Source	Scope	Activation Method	Primary Use Case
Power-On Reset	Full reset	BP_POWERGOOD	Full initialization, including JTAG/TDR reset state
Functional Cold Reset	Primary functional reset	BP_RESETN	POR-class functional reset excluding JTAG/TDR reset state
Cool Reset	Primary Reset with isolation	External controller/BMC	Selective system reset
Watchdog Reset	Core Reset	Timer expiration	Automated fault recovery
Debug Reset	Core Reset	Debug interface	Development and debugging
Function Level Reset	Configurable subsystem	PCIe interface	Hot-plug and system integration

The two primary reset levels are **Primary Reset** and **Core Reset**:

Primary Reset (rst_primary_no)

The top-level functional reset for the SMC subsystem, including CPU cores and cache hierarchies, fabric infrastructure, peripheral controllers and interfaces, and SMC control and configuration registers. JTAG/TDR reset state is reset by POR, not by `rst_primary_no` alone.

Primary reset is activated by reset sources in the following table:

Table 228. Primary Reset Activation Sources

Reset Source	Activation Method
Power-On Reset	Holds SMC functional reset gated until power-good is stable; resets JTAG/TDR state via TAP reset (<code>TRSTN</code> & <code>pwr_on_rst_ni</code>)
Cool Reset	System-level reset allowing selective isolation
Functional Cold Reset	Resets SMC functional logic through <code>rst_primary_no</code> ; excludes JTAG/TDR reset state

Warm Reset (rst_warm_no)

A more targeted reset affecting CPU cores and associated peripherals. Specifically, CPU cores and their private cache hierarchies, PLIC (Platform-Level Interrupt Controller), CLINT (Core Local Interruptor), per-core watchdog timers and bus error units

Warm reset is activated by reset sources in the following table:

Table 229. Warm Reset Activation Sources

Reset Source	Activation Method
Primary Reset	Cascaded from higher-level reset
Watchdog Timeout	Internal or external watchdog expiration
Debug Reset	Debug interface initiated reset

Function Level Reset (FLR)

Function Level Reset (FLR) addresses the requirements of systems where PCIe-based reset operations must be able to selectively reset system management functionality without affecting the entire SoC. The FLR mechanism responds to PCIe FLR requests (`cfg_flr_pf_active`) and orchestrates a controlled reset sequence that allows selective subsystem isolation.

Isolation Control Architecture

The FLR implementation provides isolation control for up to 32 individual subsystems, each with configurable isolation capabilities. The isolation request output (isolate_req_o[31:0]) aggregates three independent control sources:

- **Software Control:** Direct firmware control via the ISOLATE_REQ_REG register, enabling programmatic isolation for operational scenarios
- **Pin-Based Control:** External pin-driven isolation via isolate_req_pin_i and ISOLATE_REQ_PINEN_REG, supporting board-level control requirements
- **FLR-Triggered Control:** Automatic isolation assertion upon PCIe FLR detection, configured through ISOLATE_REQ_SMCEN_REG to specify which subsystems participate in FLR sequences

The final isolation state for each subsystem is the logical OR of all three sources, providing flexible control while maintaining clear precedence for safety-critical operations.

Programmable Reset Timing

The FLR implementation provides two programmable timing phases that control the reset sequence with cycle-accurate precision:

1. **Pre-Reset Delay** (ISOLATE_REQ_FLR_COUNTER_VALUE): Configurable delay from FLR trigger to cool reset assertion, allowing sufficient time for subsystem isolation to complete before reset propagation
2. **Reset Assertion Duration** (ISOLATE_REQ_FLR_RESET_COUNTER_VALUE): Defines the cool reset hold time, ensuring adequate reset propagation through isolated subsystems

The timing control operates in the reference clock domain to ensure consistent, predictable behavior independent of the SMC operational clock frequency. The implementation employs a two-stage state machine with down-counters that coordinate isolation establishment and reset assertion.

Memory Test Bypass

Memory repair and Built-In Self-Test (MBIST) operations can be selectively bypassed during FLR operations (skip_mem_repair_o), enabling rapid reset sequences when comprehensive memory testing is not required. This bypass activates automatically when either FLR-triggered or pin-based isolation is asserted.

Clock Domain Crossing and Synchronization

The FLR trigger signal crosses from the PCIe clock domain into both the SMC and reference clock domains through multi-stage synchronizers. Rising edge detection on the synchronized FLR active signal initiates the isolation and reset sequence. This architecture ensures reliable FLR operation regardless of clock frequency relationships between the PCIe interface and SMC subsystems.

Reset Synchronization and Timing Integrity

The SMC reset architecture implements synchronization mechanisms that ensure reset operations maintain timing integrity across multiple clock domains.

The reset synchronization implementation employs proven prim_rst_sync as synchronization primitives with the target clock domain. Reset assertion is asynchronous and deassertion is synchronous.

The synchronization primitives include multiple flip-flop stages that provide metastability protection.

Table 230. Reset Synchronization Domains

Synchronized Reset Signal	Target Clock Domain	Functional Purpose
rst_primary_smc_clk_no	SMC Clock Domain	Primary reset coordination for computational elements
rst_warm_smc_clk_no	SMC Clock Domain	Core-specific reset coordination within SMC domain
rst_primary_ref_clk_no	Reference Clock Domain	Primary reset coordination for timing-critical functions
rst_primary_periph_clk_no	Peripheral Clock Domain	Primary reset coordination for peripheral controllers

Advanced Subsystem Reset Capabilities

Beyond basic reset control, SMC supports several features for demanding system management needs:

Table 231. Advanced Reset Features

Feature	Description
Reference Clock Forcing	Forces use of a stable clock during resets for timing reliability.
Configuration Hold	Retains selected configuration during resets for fast recovery.
SRAM Preservation	Keeps memory contents intact across certain reset events.
Debug State Preservation	Maintains debug and signal states through resets for advanced troubleshooting.

These features enable flexible reset management, data retention, and robust operation during complex reset scenarios.

Register Interface

Register interface for subsystem reset control:

Table 232. Register Map

Address	Name	Access	Description
0x20	SS_CONFIG	RW	-
0x24	SS_CONFIG_LOCK	RW	-
0x40	SS_COLD_RESET_N	RW	-
0x44	SS_WARM_RESET_N	RW	-
0x48	SS_CONFIG_HOLD	RW	-
0x4C	SS_SRAM_HOLD	RW	-
0x50	SS_CRITICAL_HOLD	RW	-
0x54	SS_DEBUG_HOLD	RW	-
0x60	SS_RESET_COMPLETE	R	-
0x70	SS_COLD_RESET_LOCK	RW	-
0x80	SS_FORCE_TO_REF_CLK	RW	-
0x90	STRAPS_LO	R	-
0x94	STRAPS_HI	R	-
0xA8	SYNC_REG	RW	-
0xB0	ISOLATE_REQ_REG	RW	-
0xB4	ISOLATE_REQ_PINEN_R EG	RW	-
0xB8	ISOLATE_REQ_SMC_RE G	RW	-

Address	Name	Access	Description
0xBC	ISOLATE_REQ_SMCEN_REG	RW	-
0xC0	ISOLATE_REQ_VIS	R	-
0xC4	ISOLATE_REQ_FLR_COUNTER_VALUE	RW	-
0xC8	ISOLATE_REQ_FLR_RESET_COUNTER_VALUE	RW	-

Table 233. SS_CONFIG Register

Bits	Field	Access	Reset	Description
31:0	ss_config	RW	0x0	Configuration registers to setup system configuration. Usage is integration-specific; see the adopter chiplet specification.

Table 234. SS_CONFIG_LOCK Register

Bits	Field	Access	Reset	Description
31:0	config_lock	RW	0x0	Written to lock down SS config. If bit 0 is written, then bit 0 of other SS config cannot be written to again

Table 235. SS_COLD_RESET_N Register

Bits	Field	Access	Reset	Description
31:0	reset_n_n0_scan	RW	0x0	Subsystem active low cold resets

Table 236. SS_WARM_RESET_N Register

Bits	Field	Access	Reset	Description
31:0	reset_n_n0_scan	RW	0xFFFFFFFF	Subsystem active low warm resets

Table 237. SS_CONFIG_HOLD Register

Bits	Field	Access	Reset	Description
31:0	configuration_state_hold	RW	0x0	Used with warm_reset_n. Asserting ensures all registers configured during boot, including fuse values for export compliance, SRAM repair, other fuse values, etc. are preserved

Table 238. SS_SRAM_HOLD Register

Bits	Field	Access	Reset	Description
31:0	sram_hold	RW	0x0	Used with warm_reset_n. Asserting ensures SRAMs, latch arrays and anything else that MBIST is used to test and initialize during cold reset are preserved

Table 239. SS_CRITICAL_HOLD Register

Bits	Field	Access	Reset	Description
31:0	critical_signal_hold	RW	0x0	Used with warm_reset_n. Asserting ensures security, RAS, NMIs, perf counters, debug/trace and other subsystem specific registers are preserved

Table 240. SS_DEBUG_HOLD Register

Bits	Field	Access	Reset	Description
31:0	<code>debug_hold</code>	RW	0x0	Used with warm_reset_n. Debug hold signal to ensure any debug registers are preserved

Table 241. SS_RESET_COMPLETE Register

Bits	Field	Access	Reset	Description
31:0	<code>reset_complete</code>	R	0x0	Response signal from subsystem to indicate reset (or some other critical step) has completed. Bitwise write 1 to clear

Table 242. SS_COLD_RESET_LOCK Register

Bits	Field	Access	Reset	Description
31:0	<code>cold_reset_lock</code>	RW	0x0	Written to lock down SS cold reset. If bit 0 is written, then bit 0 of other SS cold reset cannot be written to again

Table 243. SS_FORCE_TO_REF_CLK Register

Bits	Field	Access	Reset	Description
31:0	<code>force_ss_to_ref_clk_n</code>	RW	0x0	Forces clock to be ref clk. The ensures reset reaches all FFs within a subsystem in same clock cycle

Table 244. STRAPS_LO Register

Bits	Field	Access	Reset	Description
31:0	<code>straps</code>	R	0x0	Strap value from GPIOs 0 - 31

Table 245. STRAPS_HI Register

Bits	Field	Access	Reset	Description
31:0	<code>straps</code>	R	0x0	Strap value from GPIOs 32 - 63

Table 246. SYNC_REG Register

Bits	Field	Access	Reset	Description
31:1	<code>Reserved</code>	-	-	Reserved
0	<code>sync</code>	RW	0x0	Set to assert global sync bit

Table 247. ISOLATE_REQ_REG Register

Bits	Field	Access	Reset	Description
31:0	<code>isolate_req_reg</code>	RW	0x0	Isolate request. This prevents reset from being propagated to SS, keeping them alive in case of a cold reset

Table 248. ISOLATE_REQ_PINEN_REG Register

Bits	Field	Access	Reset	Description
31:0	<code>isolate_req_pinen_reg</code>	RW	0x0	Isolate request pin enable. This enables the external isolate request pin to have an effect on ss isolate requests

Table 249. ISOLATE_REQ_SMC_REG Register

Bits	Field	Access	Reset	Description
31:1	<code>Reserved</code>	-	-	Reserved

Bits	Field	Access	Reset	Description
0	isolate_req_smc_reg	RW	0x0	Isolate request smc. Asserted once cfg_flr_pf_active is asserted, de-asserted by sw

Table 250. ISOLATE_REQ_SMCEN_REG Register

Bits	Field	Access	Reset	Description
31:0	isolate_req_smcen_reg	RW	0x0	Isolate request smc enable. This enables the FLR lock to have an effect on ss isolate requests

Table 251. ISOLATE_REQ_VIS Register

Bits	Field	Access	Reset	Description
31:9	Reserved	-	-	Reserved
8	cool_reset_n_o	R	0x1	Visibility bit for cool reset output
7:5	Reserved	-	-	Reserved
4	cool_reset_n_i	R	0x1	Visibility bit for cool reset input
3:1	Reserved	-	-	Reserved
0	isolate_req_pin	R	0x1	Visibility bit for isolate request pin

Table 252. ISOLATE_REQ_FLR_COUNTER_VALUE Register

Bits	Field	Access	Reset	Description
31:0	isolate_req_flr_counter_value	RW	0x0	Number of cycles post cfg_flr_pf_active assertion is waited until cool_reset_n is asserted

Table 253. ISOLATE_REQ_FLR_RESET_COUNTER_VALUE Register

Bits	Field	Access	Reset	Description
31:0	isolate_req_flr_reset_counter_value	RW	0x0	Number of cycles cool_reset_n is asserted for. NOTE: The cool reset flow requires this value to be greater than 0; a value of 0 will prevent initiation, regardless of ISOLATE_REQ_FLR_COUNTER_VALUE's value

5.3.3. Clock and Reset-Based Power Management Integration

The following table lists the clock gating control parameters.

Table 254. Clock Gating Control Parameters

Parameter	Configuration Range	Operational Impact
Hysteresis Control	6-bit programmable	Prevents clock gating oscillation under varying load
Activity Detection	Per-module basis	Enables fine-grained power management per component
Enable Threshold	Configurable delay	Balances responsiveness vs. power savings
Module Gating	Individual enable/disable	Allows selective power optimization per functional block

The reset architecture supports power management beyond clock gating by allowing individual subsystems to be reset and isolated, enabling power islands where unused subsystems can be fully powered down.

5.4. CPU & Memory System

This section describes the SMC’s CPU subsystem, which is based on the Rocket RV64GC RISC-V processor core (see: [Rocket Chip](#)). The cluster is generated using the Chipyard Rocket-chip generator and supports 1 to 4 processor cores, each with private L1 cache and access to a shared local SRAM.

Key features include:

- Up to 4 Rocket CPU cores (RV64GC ISA, 56-bit physical addressing)
- Core-local L1 instruction and data caches, each configurable in size and associativity
- Shared local SRAM/Scratchpad and Boot ROM
- Multi-core system management, deterministic performance for embedded management workloads
- Full support for RISC-V privilege levels (Machine, Supervisor, User)

5.4.1. Processor Core Overview

The table below summarizes the key features of the Rocket CPU core, and its cache hierarchy

Attribute	Description
Cores	1–4, Rocket RV64GC
ISA	RV64GC (I, M, A, F, D, C extensions)
Microarchitecture	In-order, 5-stage pipeline
Privilege Levels	M/S/U
Phys. Address	56 bits
Reset Vector	Per-core, independently configurable (56b)

5.4.2. Cache and Memory Hierarchy

Component	Default Size	Notes	Configurability
L1 I-Cache per core	4 KiB (32 sets × 2 ways)	Write-through, Parity	icache_tag/data_cfg
L1 D-Cache per core	4 KiB (32 sets × 2 ways)	Write-back, SECCDED ECC	dcache_tag/data_cfg
Local SRAM/Scratchpad	1 MiB (32 banks)	Deterministic access, SECCDED ECC, bank power gating	scratch_ram_cfg
Boot ROM	128 KiB	Read-only, stores boot firmware	rom_cfg

All memory components (SRAM, caches, ROM) are configurable via top-level parameters. The scratchpad SRAM allows deterministic accesses, independent of cache state, and supports per-bank sizing and power management.

Internally, the CPU cluster uses a TileLink bus to connect cores to the scratchpad, Boot ROM, PLIC, CLINT, and Bus Error Units. At the cluster boundary, TileLink is protocol-bridged to AXI4 and AXI4-Lite for integration with the SMC fabric (see [Fabric](#)).

5.4.3. Reset, Boot, and Initialization

Each CPU core supports an independent reset mechanism and a programmable reset vector, configured via

rst_core_N_ni and reset_vector_N_i signals for cores N=0 to 3. The design provides separate reset controls for the uncore logic and debug subsystems, enabling flexible platform management and debug flows.

SRAM initialization is handled automatically by hardware on reset, ensuring memory consistency and error-free operation at startup; this feature can be disabled using disable_sram_auto_init_i to support specialized or custom boot procedures. The completion of SRAM initialization is reported through the init_mem_done_o status signal, allowing software to reliably detect when memory is ready for use.

5.4.4. Memory Repair

The SMC CPU subsystem includes Built-In Redundancy Analysis (BIRA) memory repair support to address manufacturing defects in SRAM arrays. Repair uses redundant memory elements configured with data stored in eFuse memory, applied during the boot sequence.

Memory repair executes automatically after eFuse sensing completes and before MBIST operations. The process reads repair data from the eFuse BIRA register (16-kilobit `repair_data` field) and configures redundant memory elements, ensuring SRAM arrays are functional before the CPU cluster begins execution.

Control Mechanisms:

- `fuse_sense_done_o`: eFuse controller output indicating sensing completion. External DFT logic monitors this signal to trigger repair operations.
- `skip_mem_repair_o`: Reset unit output that bypasses repair when asserted, activating during FLR operations or when the `BYPASS_SRAM_REPAIR` strap is set.
- `BYPASS_SRAM_REPAIR` strap (GPIO pin 13): Hardware-level bypass control that bypasses repair logic when asserted, assuming SRAM arrays are defect-free.

Status Reporting:

Repair completion and success are reported through DFT Control Status registers (`DFX_CTRL_STATUS_SOC` at `0xC000D800` and `DFX_CTRL_STATUS_SEP_SMC` at `0xC000D80C`). The `mem_repair_done` bit indicates completion, while `mem_repair_success` indicates successful repair. The SMC ROM firmware monitors these bits during initialization to verify repair completion before proceeding with boot operations.

Boot Sequence Integration:

Memory repair integrates into the boot sequence as follows: after power-on reset deasserts and eFuse sensing begins, `fuse_sense_done_o` assertion triggers repair operations. Repair data is read from eFuse shadow registers and applied to SRAM, with completion reported through DFT Control Status registers. MBIST operations begin after repair completes (or is bypassed), and the CPU cluster reset releases after both repair and MBIST complete. The repair process must complete successfully before the CPU cluster can safely access SRAM arrays.

During FLR operations, repair is automatically bypassed via `skip_mem_repair_o` to enable rapid reset sequences. Normal cold reset sequences always execute repair unless explicitly bypassed via strap configuration.

5.4.5. System Interfaces

Interface	Protocol	Description
MMIO	AXI4-Lite	56b addressing, low-latency
L2 Frontend	AXI4	For shared memory/coherency

Interface	Protocol	Description
Debug	APB, JTAG	Non-intrusive, per-core

5.4.6. Cluster Boundary Isolation

The cluster is reset independently of the surrounding SMC fabric. If the cluster were reset while a transaction on one of its AXI ports was still in flight, the fabric on the other side would be left waiting on a response that never returns, hanging the bus, and could sample boundary signals while they glitch through reset. To avoid this, each AXI port is wrapped in an `axi_isolate` block that drains the outstanding transactions and holds the boundary at safe values before the reset is allowed to take effect.

Two instances wrap the AXI ports listed above, each sized for up to four outstanding transactions:

- the **L2 frontend** slave port, which blocks new transactions rather than terminating them, and
- the **MMIO** master port, which terminates outstanding transactions with an error response to keep reset-time X values from propagating into the fabric.

The non-AXI status crossings (`wb_pc_valid`, `wb_reg_pc`, and the per-core `wdt_reset`) are clamped to constants over the same window so they cannot leak glitches either. The boundary also self-isolates on cold boot, releasing only once SRAM initialization completes and the cores and uncore are out of reset.

Drain Handshake

Isolation is coordinated with the control logic (`smc_cpu_ctrl_wrap`) through two signals:

- `isolate_req_i` — asserted by the control logic whenever a software reset of the cluster is pending. The control logic withholds the reset itself until the cluster reports that it is safe to apply.
- `drained_o` — asserted once both `axi_isolate` instances have drained and isolated their ports, telling the control logic the withheld reset may now be released.

Reset-Domain Crossing

The flops driving the isolate-control logic are held by the cluster's uncore/core reset, while the flops inside `axi_isolate` are held by the Primary Reset (see [Clock and Reset Management](#)). The uncore/core reset is driven from two sources:

- a Warm Reset from a watchdog timeout, which is **asynchronous** to the isolate logic, and
- a CPU CSR write (the reset-control register in `smc_cpu_ctrl_wrap`), which is **synchronous**.

Because the two sets of flops sit in different reset domains, the asynchronous warm-reset path is a reset-domain crossing into `axi_isolate`. This is acceptable: only the watchdog asserts the warm reset asynchronously, and a watchdog timeout is followed by a cold reset, so the brief crossing window has no functional consequence. The CSR path is synchronous and crosses no domains.

5.4.7. Interrupt and Control Integration

All interrupt wiring—MMIO bases for PLIC, CLINT, watchdog, and bus-error units; the authoritative `cpu_interrupts_o` map; PLIC/CLINT register documentation; and BEU NMI delivery—is documented in [Interrupts](#).

5.4.8. Watchdog Timer System

Each CPU core includes a dedicated watchdog timer (at 0xC000_0000 + N * 0x400), enabling per-core fault detection and recovery without impacting other cores. Base addresses and interrupt-related context for the WDT are summarized in [Interrupts](#).

Timeouts are configurable to suit different workloads, and staged warnings (followed by reset) allow software intervention before hardware reset is triggered.

Watchdog monitoring can be disabled during debugging to prevent false triggers, supporting uninterrupted development and troubleshooting.

5.4.9. Advanced Bus Infrastructure Monitoring and Error Management

Per-core bus error monitors (at 0xC801_0000 + N * 0x1000) track AXI transactions, allowing the system to quickly detect and respond to infrastructure faults that could impact system management.

AXI error capture detects and logs decode, slave, and timeout errors so management software has clear insight into infrastructure health and can tailor its response.

Classifying errors lets software address each type effectively: decode errors may need reconfiguration; timeout errors may require isolating faults and recovery procedures.

Error address and transaction capture aid debugging, preserving context like addresses and data to support thorough fault analysis and recovery coordination.

5.4.10. Debug

Debugging is facilitated by a dedicated APB interface and JTAG, enabling per-core access to registers, memory, and both hardware and software breakpoints. These debug capabilities function independently of functional resets and allow override of reset vectors to support firmware development and diagnostics.

5.4.11. Detailed Register Map

NOTE	Register Arrays: This register map contains the following register arrays:
	• abstractdata[12]: 12 registers ◦ Base Address: 0x10 ◦ Address Increment: 0x4 per register
	• confstrptr[3]: 3 registers ◦ Base Address: 0x64 ◦ Address Increment: 0x4 per register
	• progbuf[16]: 16 registers ◦ Base Address: 0x80 ◦ Address Increment: 0x4 per register

Table 255. Register Map

Address	Name	Access	Description
0x10 - 0x3C	abstractdata[12]	RW	Abstract Data Register. Used for arguments/return values of abstract commands.
0x40	dmcontrol	RW	Controls the overall Debug Module and selected harts. Writes to action bits (haltreq, resumereq, etc.) apply to the new value of hartsel and hasel.
0x44	dmstatus	R	Reports status for the Debug Module and currently selected harts. This entire register is read-only.
0x48	hartinfo	R	Information about the hart currently selected by hartsel. Optional; reads all-zero if not present. Entire register is read-only.
0x4C	haltsum1	R	Read-only. Each bit indicates if any hart in a group of 32 is halted. May not be present in systems with fewer than 33 harts. Unavailable/nonexistent harts are not considered halted.
0x50	hawindowssel	RW	Selects which 32-bit portion of the hart array mask register is accessible in hawindow.
0x54	hawindow	RW	R/W access to a 32-bit portion of the hart array mask register, selected by hawindowssel.
0x58	abstractcs	RW	Controls and status for abstract commands. Writing this register while an abstract command is executing (busy=1) sets cmderr to 1 (busy) if cmderr was 0.
0x5C	command	RW	Writes cause corresponding abstract command to execute. Writing while busy sets cmderr=1 (if cmderr=0). Ignored if cmderr is non-zero.
0x60	abstractauto	RW	Optional. Allows more efficient burst accesses. Writing while busy sets cmderr=1 (if cmderr=0). Detect support by R/W test.
0x64 - 0x6C	confstrptr[3]	R	Part of configuration string pointer address (if dmstatus.confstrptrvalid=1) or unspecified identifier information. Read-only.
0x74	nextdm	R	If multiple DMs on DMI, this holds base address of the next DM. 0 if this is the last. Read-only.
0x80 - 0xBC	progbuf[16]	RW	Program Buffer word. For debugger to write small programs.
0xC0	authdata	RW	32-bit serial port to/from authentication module. Use when dmstatus.authbusy is clear.
0xD0	haltsum2	R	Read-only. Each bit indicates if any hart in a group of 1024 is halted. May not be present if < 1025 harts.
0xD4	haltsum3	R	Read-only. Each bit indicates if any hart in a group of 32768 is halted. May not be present if < 32769 harts.
0xDC	sbaddress3	RW	Bits 127:96 of the physical system bus address for System Bus Access. Not present if sbcs.sbasize < 97. Writes while sbcs.sbbusy=1 set sbcs.sbbusyyerror.
0xE0	sbcs	RW	Controls and status for the System Bus Access master. Writes while sbbusy=1 result in undefined behavior.
0xE4	sbaddress0	RW	Bits 31:0 of the physical system bus address. Not present if sbcs.sbasize=0. Writes while sbcs.sbbusy=1 set sbcs.sbbusyyerror. If sbcs.sbberror=0, sbcs.sbbusyyerror=0, and sbcs.sbreadonaddr=1, write triggers read sequence.
0xE8	sbaddress1	RW	Bits 63:32 of the physical system bus address. Not present if sbcs.sbasize < 33. Writes while sbcs.sbbusy=1 set sbcs.sbbusyyerror.
0xEC	sbaddress2	RW	Bits 95:64 of the physical system bus address. Not present if sbcs.sbasize < 65. Writes while sbcs.sbbusy=1 set sbcs.sbbusyyerror.
0xF0	sbdata0	RW	Bits 31:0 of system bus data. Not present if all sbcs.sbaccess* flags are 0. Accesses do nothing if sbcs.sbberror or sbcs.sbbusyyerror are non-zero. Accesses while sbcs.sbbusy=1 set sbcs.sbbusyyerror. Write triggers bus write sequence. Read triggers read sequence including conditional next read if sbcs.sbreadondata=1.

Address	Name	Access	Description
0xF4	sbdata1	RW	Bits 63:32 of system bus data. Not present if sbcs.sbaccess64 and sbcs.sbaccess128 are 0. Accesses while sbcs.sbbusy=1 set sbcs.sbbusyerror.
0xF8	sbdata2	RW	Bits 95:64 of system bus data. Only exists if sbcs.sbaccess128=1. Accesses while sbcs.sbbusy=1 set sbcs.sbbusyerror.
0xFC	sbdata3	RW	Bits 127:96 of system bus data. Only exists if sbcs.sbaccess128=1. Accesses while sbcs.sbbusy=1 set sbcs.sbbusyerror.
0x100	haltsum0	R	Read-only. Each bit indicates whether one specific hart is halted. Unavailable/nonexistent harts are not considered halted. LSB for hart {hartsel[19:5],5'h0}, MSB for hart {hartsel[19:5],5'h1f}.

Table 256. *abstractdata[12] Register*

Bits	Field	Access	Reset	Description
31:0	data	RW	0x0	Data value for abstract commands.

Table 257. *dmcontrol Register*

Bits	Field	Access	Reset	Description
31	haltreq	RW	0x0	Halt request for currently selected harts. Write 0 to clear, 1 to set.
30	resumereq	RW	0x0	Resume request for currently selected harts. Writing 1 causes halted harts to resume and clears their resume ack bit. Ignored if haltreq is set.
29	hartreset	RW	0x0	Optional. Hart reset for currently selected harts. Write 1, then 0 to deassert. Debugger must not change selection while this is 1. If not implemented, reads as 0.
28	ackhavereset	RW	0x0	Writing 1 clears havereset for any selected harts.
27	_reserved_27	R	0x0	Reserved.
26	hasel	RW	0x0	Selects definition of currently selected harts. 0: single hart by hartsel. 1: hart by hartsel plus those by hart array mask. Tied to 0 if hart array mask not implemented.
25:16	hartsello	RW	0x0	Low 10 bits of hartsel: DM-specific index of the hart to select.
15:6	hartselhi	RW	0x0	High 10 bits of hartsel: DM-specific index of the hart to select.
5:4	_reserved_5_4	R	0x0	Reserved.
3	setresethaltreq	RW	0x0	Optional (if dmstatus.hasresethaltreq=1). Writing 1 sets halt-on-reset request bit for selected harts (unless clrresethaltreq is also 1).
2	clrresethaltreq	RW	0x0	Optional (if dmstatus.hasresethaltreq=1). Writing 1 clears halt-on-reset request bit for selected harts.
1	ndmreset	RW	0x0	Controls reset signal from DM to system (ndmreset). Write 1, then 0 to deassert.
0	dmactive	RW	0x0	Debug Module active and reset. 0: DM takes reset values (except this bit). 1: DM functions normally.

Table 258. *dmstatus Register*

Bits	Field	Access	Reset	Description
31:23	_reserved_31_23	R	0x0	Reserved.
22	impebreak	R	0x0	1: Implicit ebreak after Program Buffer. Must be 1 if abstractcs.progbufsize is 1.
21:20	_reserved_21_20	R	0x0	Reserved.

Bits	Field	Access	Reset	Description
19	<code>allhavereset</code>	R	0x0	1: All currently selected harts have been reset and not acknowledged.
18	<code>anyhavereset</code>	R	0x0	1: At least one currently selected hart has been reset and not acknowledged.
17	<code>allresumeack</code>	R	0x0	1: All currently selected harts have acknowledged their last resume request.
16	<code>anyresumeack</code>	R	0x0	1: Any currently selected hart has acknowledged its last resume request.
15	<code>allnonexistent</code>	R	0x0	1: All currently selected harts do not exist in this platform.
14	<code>anynonexistent</code>	R	0x0	1: Any currently selected hart does not exist in this platform.
13	<code>allunavail</code>	R	0x0	1: All currently selected harts are unavailable.
12	<code>anyunavail</code>	R	0x0	1: Any currently selected hart is unavailable.
11	<code>allrunning</code>	R	0x0	1: All currently selected harts are running.
10	<code>anyrunning</code>	R	0x0	1: Any currently selected hart is running.
9	<code>allhalted</code>	R	0x0	1: All currently selected harts are halted.
8	<code>anyhalted</code>	R	0x0	1: Any currently selected hart is halted.
7	<code>authenticated</code>	R	0x0	0: Authentication required. 1: Authentication passed. Preset to 1 if no authentication implemented.
6	<code>authbusy</code>	R	0x0	0: Authentication module ready. 1: Authentication module busy.
5	<code>hasresethaltreq</code>	R	0x0	1: DM supports halt-on-reset functionality (setresethaltreq/clrresethaltreq bits in dmcontrol).
4	<code>confstrptrvalid</code>	R	0x0	0: confstrptr0-3 are not relevant to config string. 1: confstrptr0-3 hold config string address.
3:0	<code>version</code>	R	0x2	Debug Module version. 0=No DM, 1=v0.11, 2=v0.13, 15=Non-conformant.

Table 259. hartinfo Register

Bits	Field	Access	Reset	Description
31:24	<code>_reserved_31_24</code>	R	0x0	Reserved.
23:20	<code>nscratch</code>	R	0x0	Number of dscratch registers available for debugger use during program buffer execution (dscratch0 upwards).
19:17	<code>_reserved_19_17</code>	R	0x0	Reserved.
16	<code>dataaccess</code>	R	0x0	0: Data registers (dataN) are shadowed by CSRs. 1: Data registers are shadowed in hart's memory map.
15:12	<code>datasize</code>	R	0x0	If dataaccess=0: Number of CSRs for dataN. If dataaccess=1: Number of 32-bit words in memory map for dataN. Max value 12.
11:0	<code>dataaddr</code>	R	0x0	If dataaccess=0: Number of first CSR for dataN. If dataaccess=1: Signed address of RAM where dataN are shadowed (relative to zero).

Table 260. haltsum1 Register

Bits	Field	Access	Reset	Description
31:0	<code>haltsum1</code>	R	0x0	Halt status summary. LSB for harts {hartsel[19:10],10'h0}-{hartsel[19:10],10'h1f}, MSB for harts {hartsel[19:10],10'h3e0}-{hartsel[19:10],10'h3ff}.

Table 261. hawindowssel Register

Bits	Field	Access	Reset	Description
31:15	<code>_reserved_31_15</code>	R	0x0	Reserved. High bits may be tied to 0 depending on hart array mask size.
14:0	<code>hawindowssel</code>	RW	0x0	Window select for hart array mask register. E.g., on a system with 48 harts, only bit 0 may be writable.

Table 262. hawindow Register

Bits	Field	Access	Reset	Description
31:0	<code>maskdata</code>	RW	0x0	32-bit window into the hart array mask register. Some bits may be constant 0, depending on hawindowssel and implemented harts.

Table 263. abstractcs Register

Bits	Field	Access	Reset	Description
31:29	<code>_reserved_31_29</code>	R	0x0	Reserved.
28:24	<code>progbufsize</code>	R	0x0	Size of the Program Buffer, in 32-bit words. Valid sizes are 0-16.
23:13	<code>_reserved_23_13</code>	R	0x0	Reserved.
12	<code>busy</code>	R	0x0	1: An abstract command is currently being executed. Set when 'command' is written, cleared on completion.
11	<code>_reserved_11</code>	R	0x0	Reserved.
10:8	<code>cmderr</code>	RW	0x0	Abstract command error status. Remains set until cleared. Valid if <code>busy</code> =0. 0=none, 1=busy, 2=not_supported, 3=exception, 4=halt/resume, 5=bus, 7=other.
7:4	<code>_reserved_7_4</code>	R	0x0	Reserved.
3:0	<code>datacount</code>	R	0x0	Number of data registers (dataN) implemented. Valid sizes are 1-12.

Table 264. command Register

Bits	Field	Access	Reset	Description
31:24	<code>cmdtype</code>	RW	0x0	Type of abstract command (e.g., 0 for Access Register, 1 for Quick Access, 2 for Access Memory).
23:0	<code>control</code>	RW	0x0	Command-specific control field.

Table 265. abstractauto Register

Bits	Field	Access	Reset	Description
31:16	<code>autoexecprogbuf</code>	RW	0x0	Bitmask (16 bits). If bit N=1, R/W to progbufN re-executes command in 'command' register.
15:12	<code>_reserved_15_12</code>	R	0x0	Reserved.
11:0	<code>autoexecdata</code>	RW	0x0	Bitmask (12 bits). If bit N=1, R/W to dataN re-executes command in 'command' register.

Table 266. confstrptr[3] Register

Bits	Field	Access	Reset	Description
31:0	<code>addr</code>	R	0x0	Bits of configuration string pointer or identifier information.

Table 267. nextdm Register

Bits	Field	Access	Reset	Description
31:0	addr	R	0x0	Base address of the next Debug Module or 0.

Table 268. progbuff[16] Register

Bits	Field	Access	Reset	Description
31:0	data	RW	0x0	Program instruction word.

Table 269. authdata Register

Bits	Field	Access	Reset	Description
31:0	data	RW	0x0	Data for authentication mechanism.

Table 270. haltsum2 Register

Bits	Field	Access	Reset	Description
31:0	haltsum2	R	0x0	Halt status summary. LSB for harts {hartsel[19:15],15'h0}-{hartsel[19:15],15'h3ff}, etc.

Table 271. haltsum3 Register

Bits	Field	Access	Reset	Description
31:0	haltsum3	R	0x0	Halt status summary. LSB for harts 20'h0-20'h7ff, etc.

Table 272. sbaddress3 Register

Bits	Field	Access	Reset	Description
31:0	address	RW	0x0	Address bits 127:96 for system bus access.

Table 273. sbcs Register

Bits	Field	Access	Reset	Description
31:29	sbversion	R	0x1	System Bus interface version. 0=pre-2018 drafts, 1=this version (0.13).
28:23	_reserved_28_23	R	0x0	Reserved.
22	sbbusyerror	RW	0x0	System bus busy error. Set if access attempted while busy or read while read in progress. No new access if set.
21	sbbusy	R	0x0	1: System bus master is busy. Goes high on access request, low on completion.
20	sbreadonaddr	RW	0x0	1: Every write to sbaddress0 automatically triggers a system bus read at the new address.
19:17	sbaccess	RW	0x2	Access size for system bus accesses: 0=8-bit, 1=16-bit, 2=32-bit, 3=64-bit, 4=128-bit. If unsupported, access fails, sberror=4.
16	sbautoincrement	RW	0x0	1: sbaddress is incremented by access size (from sbaccess) after every system bus access.
15	sbreadondata	RW	0x0	1: Every read from sbdata0 automatically triggers a system bus read at (possibly auto-incremented) address.
14:12	_reserved_14_12	R	0x0	Reserved.
11:9	sberror	RW	0x0	System bus master error flags. No new access if non-zero. 0=none, 1=timeout, 2=bad_address, 3=alignment, 4=unsupported_size, 7=other.

Bits	Field	Access	Reset	Description
8:5	<i>sbasize</i>	R	0x0	Width of system bus addresses in bits (e.g., 32 for 32-bit addresses). 0 indicates no bus access support. NOTE: This field is 4 bits wide ([8:5]), which is insufficient to directly store typical bus widths like 32 or 64. This implies the value is an encoding or there is a spec issue. The text implies it stores values like 32, 64, etc (e.g. 'if sbasize < 33').
4	<i>sbaccess128</i>	R	0x0	1: 128-bit system bus accesses are supported.
3	<i>sbaccess64</i>	R	0x0	1: 64-bit system bus accesses are supported.
2	<i>sbaccess32</i>	R	0x0	1: 32-bit system bus accesses are supported.
1	<i>sbaccess16</i>	R	0x0	1: 16-bit system bus accesses are supported.
0	<i>sbaccess8</i>	R	0x0	1: 8-bit system bus accesses are supported.

Table 274. sbaddress0 Register

Bits	Field	Access	Reset	Description
31:0	<i>address</i>	RW	0x0	Address bits 31:0.

Table 275. sbaddress1 Register

Bits	Field	Access	Reset	Description
31:0	<i>address</i>	RW	0x0	Address bits 63:32.

Table 276. sbaddress2 Register

Bits	Field	Access	Reset	Description
31:0	<i>address</i>	RW	0x0	Address bits 95:64.

Table 277. sbdata0 Register

Bits	Field	Access	Reset	Description
31:0	<i>data</i>	RW	0x0	Data bits 31:0.

Table 278. sbdata1 Register

Bits	Field	Access	Reset	Description
31:0	<i>data</i>	RW	0x0	Data bits 63:32.

Table 279. sbdata2 Register

Bits	Field	Access	Reset	Description
31:0	<i>data</i>	RW	0x0	Data bits 95:64.

Table 280. sbdata3 Register

Bits	Field	Access	Reset	Description
31:0	<i>data</i>	RW	0x0	Data bits 127:96.

Table 281. haltsum0 Register

Bits	Field	Access	Reset	Description
31:0	<i>haltsum0</i>	R	0x0	Halt status for 32 harts based on hartsel[19:5].

Table 282. Register Map

Address	Name	Access	Description
0x0	CAUSE	RW	-
0x8	PHYS_ADDR	R	-
0x10	ENABLE	RW	-
0x18	PLIC_ENABLE	RW	-
0x20	ACCRUED_ENABLE	RW	-
0x28	LOCAL_ENABLE	RW	-

Table 283. CAUSE Register

Bits	Field	Access	Reset	Description
63:3	Reserved	-	-	Reserved
2:0	data	RW	0x0	Cause register. Enumerates different kinds of errors

Table 284. PHYS_ADDR Register

Bits	Field	Access	Reset	Description
63:56	Reserved	-	-	Reserved
55:0	value	R	-	Physical address of error event. No reset value

Table 285. ENABLE Register

Bits	Field	Access	Reset	Description
63:8	Reserved	-	-	Reserved
7	dcache_uncorrectable	RW	0x1	Enable recording error information for DCache ECC uncorrectable bus error. Enabled on reset.
6	dcache_correctable	RW	0x1	Enable recording error information for DCache ECC correctable bus error. Enabled on reset.
5	dcache_tlbus	RW	0x1	Enable recording error information for DCache Tilelink bus error. Enabled on reset.
4	rsvd4	RW	0x0	Reserved
3	rsvd3	RW	0x0	Reserved
2	icache_correctable	RW	0x1	Enable recording error information for ICache ECC correctable bus error. Enabled on reset.
1	icache_tlbus	RW	0x1	Enable recording error information for ICache Tilelink bus error. Enabled on reset.
0	rsvd0	RW	0x0	Reserved

Table 286. PLIC_ENABLE Register

Bits	Field	Access	Reset	Description
63:8	Reserved	-	-	Reserved
7	dcache_uncorrectable	RW	0x0	PLIC interrupt enable mask for DCache ECC uncorrectable bus error.
6	dcache_correctable	RW	0x0	PLIC interrupt enable mask for DCache ECC correctable bus error.
5	dcache_tlbus	RW	0x0	PLIC interrupt enable mask for DCache Tilelink bus error.
4	rsvd4	RW	0x0	Reserved
3	rsvd3	RW	0x0	Reserved

Bits	Field	Access	Reset	Description
2	<i>icache_correctable</i>	RW	0x0	PLIC interrupt enable mask for ICache ECC correctable bus error.
1	<i>icache_tlbus</i>	RW	0x0	PLIC interrupt enable mask for ICache Tilelink bus error.
0	<i>rsvd0</i>	RW	0x0	Reserved

Table 287. ACCRUED_ENABLE Register

Bits	Field	Access	Reset	Description
63:8	<i>Reserved</i>	-	-	Reserved
7	<i>dcache_uncorrectable</i>	RW	0x0	Coarse grain interrupt enable mask for DCache ECC uncorrectable bus error.
6	<i>dcache_correctable</i>	RW	0x0	Coarse grain interrupt enable mask for DCache ECC correctable bus error.
5	<i>dcache_tlbus</i>	RW	0x0	Coarse grain interrupt enable mask for DCache Tilelink bus error.
4	<i>rsvd4</i>	RW	0x0	Reserved
3	<i>rsvd3</i>	RW	0x0	Reserved
2	<i>icache_correctable</i>	RW	0x0	Coarse grain interrupt enable mask for ICache ECC correctable bus error.
1	<i>icache_tlbus</i>	RW	0x0	Coarse grain interrupt enable mask for ICache Tilelink bus error.
0	<i>rsvd0</i>	RW	0x0	Reserved

Table 288. LOCAL_ENABLE Register

Bits	Field	Access	Reset	Description
63:8	<i>Reserved</i>	-	-	Reserved
7	<i>dcache_uncorrectable</i>	RW	0x0	Core local interrupt enable mask for DCache ECC uncorrectable bus error.
6	<i>dcache_correctable</i>	RW	0x0	Core local interrupt enable mask for DCache ECC correctable bus error.
5	<i>dcache_tlbus</i>	RW	0x0	Core local interrupt enable mask for DCache Tilelink bus error.
4	<i>rsvd4</i>	RW	0x0	Reserved
3	<i>rsvd3</i>	RW	0x0	Reserved
2	<i>icache_correctable</i>	RW	0x0	Core local interrupt enable mask for ICache ECC correctable bus error.
1	<i>icache_tlbus</i>	RW	0x0	Core local interrupt enable mask for ICache Tilelink bus error.
0	<i>rsvd0</i>	RW	0x0	Reserved

Table 289. Register Map

Address	Name	Access	Description
0x0	CTRL	RW	KEY reg must be written to before any other register in this addrmap can be written to, unlocking the WDT. After writing to another register, the WDT will be locked again, and KEY must be written to again to unlock it.
0x8	COUNT	RW	-
0x10	SCALED_COUNT	RW	-
0x18	FEED	RW	-
0x1C	KEY	RW	-

Address	Name	Access	Description
0x20	CMP	RW	-

Table 290. CTRL Register

Bits	Field	Access	Reset	Description
31:29	rsvd3	R	0x0	Reserved
28	wdogip0	RW	0x0	Pending interrupt
27:14	rsvd2	R	0x0	Reserved
13	wdogcoreawake	RW	0x0	Enable WDT to only count when the core is not in reset
12	wdogenalways	RW	0x0	Enable WDT to always count. This has priority over wdogcoreawake
11:10	rsvd1	R	0x0	Reserved
9	wdogzerocmp	RW	0x0	Enable cycle counter to be reset to 0 when compare value has been reached
8	wdogrsten	RW	0x0	Enables a sticky reset signal to outputted to top level, can be used for a second stage wdt
7:4	rsvd0	R	0x0	Reserved
3:0	wdogscale	RW	0x0	WDT counter scale value. Scales the current cycle count (time) by count / 2 ^ wdogscale value

Table 291. COUNT Register

Bits	Field	Access	Reset	Description
31	Reserved	-	-	Reserved
30:0	wdogcount	RW	0x0	31 bit wdt counter registers. On write, the counter will be set to the written value, then continue counting up. KEY reg must be written to before any other register in this addrmap can be written to, unlocking the WDT. After writing to another register, the WDT will be locked again, and KEY must be written to again to unlock it.

Table 292. SCALED_COUNT Register

Bits	Field	Access	Reset	Description
31:16	Reserved	-	-	Reserved
15:0	wdogs	RW	0x0	Scaled value of wdt counter. (i.e. count >> wdogscale). Writing to this reg has no effect, but will lock the WDT (meaning KEY must be written to KEY reg to unlock it again). KEY reg must be written to before any other register in this addrmap can be written to, unlocking the WDT. After writing to another register, the WDT will be locked again, and KEY must be written to again to unlock it.

Table 293. FEED Register

Bits	Field	Access	Reset	Description
31:0	wdogfeed	RW	0x0	Feed register. Writing 0xD09F00D will reset the counter value back to 0. KEY reg must be written to before any other register in this addrmap can be written to, unlocking the WDT. After writing to another register, the WDT will be locked again, and KEY must be written to again to unlock it.

Table 294. KEY Register

Bits	Field	Access	Reset	Description
31:0	<code>wdogkey</code>	RW	0x0	Key register. Magic key (0x51F15E) must be written to this register before a write to any other register in this addrmap. Must be written every time. Reading this reg returns 1 if key has been written to it and it is unlocked, 0 otherwise.

Table 295. CMP Register

Bits	Field	Access	Reset	Description
31:16	<code>Reserved</code>	-	-	Reserved
15:0	<code>wdogcmp0</code>	RW	0x1000	Compare register. Value that gets compared to wdogs. KEY reg must be written to before any other register in this addrmap can be written to, unlocking the WDT. After writing to another register, the WDT will be locked again, and KEY must be written to again to unlock it.

5.5. Boot ROM

The SMC Boot ROM provides the initial firmware that executes immediately after power-on reset, performing essential hardware initialization and system bring-up functions. The ROM contains the SMC Production ROM firmware, a minimal bootloader designed to establish a known good hardware state before transitioning control to more sophisticated firmware layers.

5.5.1. ROM Architecture

The Boot ROM is implemented as a read-only memory region mapped at base address `0xC0040000` with a configurable size of up to 64KB. The ROM is accessed by the CPU cluster through an AXI-Lite interface, with read requests issued via the `rom_intf_req_o` signal and responses received through `rom_intf_rsp_i`. This interface provides the CPU with direct access to the ROM contents for instruction fetch and read-only data access during the boot sequence.

The ROM memory layout is organized into standard executable sections, with code residing in the `.text` section, read-only data in `.rodata`, and initialization routines in `.init`. Initialized data variables are stored in the `.data` section within ROM but are automatically copied to SRAM during the boot process, allowing the ROM to remain read-only while providing writable data space for the firmware. The detailed memory layout and section placement are defined in the linker script located at `hw/sys/smc/bootrom/prod/linker/smc/smc_rom.ld`, which specifies the exact address ranges and alignment requirements for each memory region.

Within the SMC's memory hierarchy, the ROM serves as the primary boot source, providing the initial reset vector target for all CPU cores. The ROM operates independently of the cache hierarchy, ensuring deterministic boot behavior, and complements the scratchpad SRAM by providing persistent code storage while the SRAM provides runtime data and stack space for the boot firmware.

5.5.2. ROM Hardware Configuration

The ROM wrapper module (`OCAHSMCCluster_rom_wrapper`) provides hardware support for **memory bank configuration** and **endianness swapping**, enabling flexibility in ROM implementation and compatibility with different system requirements.

Memory bank configuration is controlled through the `mem_cfg_i` input signal, which provides an 11-bit configuration value passed to the underlying ROM primitive. This configuration mechanism allows the ROM memory banks to be configured for different memory technologies, repair strategies, or foundry-specific requirements. The configuration follows the same pattern used by other memory components in the SMC, including cache tag and data arrays and

scratchpad RAM banks, providing a consistent interface for memory configuration across the system.

The ROM bank configuration can be controlled through shadow register values read from efuse data, similar to the endianness control, allowing bank swapping and memory configuration to be set during manufacturing or through efuse programming.

Endianness swapping support enables the ROM to provide data in either little-endian or big-endian byte order, regardless of the native byte order stored in the ROM memory. The `rom_flip_endianness_i` control signal, when asserted, causes the ROM wrapper to perform byte-level swapping on 64-bit words read from the ROM before returning them to the CPU. The endianness flip function swaps byte pairs within each 64-bit word, transforming little-endian data to big-endian format and vice versa. This hardware-level byte swapping occurs transparently to the CPU, allowing the same ROM image to be used in systems with different endianness requirements without modifying the firmware image itself.

The endianness control signal is derived from shadow register values read from efuse data during system initialization, allowing the endianness configuration to be set during manufacturing or through efuse programming. This configuration remains fixed throughout system operation, ensuring consistent ROM access behavior across all boot phases and runtime operation.

5.5.3. ROM Boot Code

The SMC Production ROM firmware implements a comprehensive boot sequence designed to initialize hardware components, read configuration data, and establish communication interfaces for multi-chiplet coordination. The boot code is organized into a structured multi-phase initialization process that progressively establishes system functionality.

The boot sequence begins with Phase 0, which initializes POST (Power-On Self-Test) codes and the scratchpad memory system. POST codes provide external visibility into the boot progress, enabling debug and monitoring equipment to track initialization stages. The scratchpad initialization establishes the memory environment required for subsequent boot phases.

Phase 1: focuses on reading hardware configuration through strap pins and efuse registers. Hardware straps, sampled from GPIO pins during power-on reset, determine fundamental system behavior including chiplet role (primary or secondary), boot mode selection, and feature enables. Efuse data provides device-specific configuration including I2C/I3C addresses, security settings, and chiplet identification. This configuration data drives all subsequent initialization decisions.

For a secondary chiplet, Phase 1.5 performs PLL (Phase-Locked Loop) configuration to establish the primary system clock. The ROM firmware configures the PLL through AXI-Lite transactions, switching from the reference clock to the synthesized PLL output clock. This phase is conditional based on chiplet type detection and the `BL0_PLLCLK` strap configuration.

Phase 2: handles SRAM setup and memory protection. The ROM establishes memory protection boundaries to prevent corruption of its own code and data regions, implementing defense-in-depth security measures. SRAM initialization may be performed by hardware auto-zero functionality or through software-based zeroing, depending on the `SRAM_AUTO_ZERO_DISABLE` strap setting.

Phase 3: configures communication interfaces based on the chiplet role and boot mode. The ROM initializes I2C and I3C controllers, configuring them according to addresses read from efuse data. Interface selection logic determines which communication channels are enabled based on whether the chiplet is primary or secondary and the security state of the device.

Phase 4: initializes the OCCP (Open Chiplet Communication Protocol) interface, which enables coordination between chiplets in a multi-chiplet system. The OCCP protocol provides a standardized command interface for boot image distribution, status reporting, and chiplet coordination. Successful OCCP initialization indicates that the ROM is ready to

participate in system-level boot coordination.

Phase 5: enters the OCCP command processing loop, where the ROM continuously processes commands received over the configured communication interfaces. This loop handles firmware image loading, manifest processing for secure boot scenarios, and coordination commands from primary chiplets. The ROM remains in this processing loop until it receives commands to transition control to subsequent bootloader stages or application firmware.

The boot code is organized into several functional components. The boot sequence components, located in `fw/smc/prod_rom/boot/`, provide assembly-level startup code, trap handlers for exception processing, and SRAM initialization routines. Hardware drivers in `fw/smc/prod_rom/drivers/` implement interfaces to strap pin reading, efuse access, PLL configuration, and I2C/I3C peripheral control. Library components in `fw/smc/prod_rom/lib/` provide higher-level functionality including status reporting through ring buffers, security state detection, OCCP protocol implementation, and interface mapping logic.

The ROM firmware is optimized for size, with the production release build occupying approximately 20KB of ROM space. Debug builds, which include console output and additional diagnostic capabilities, expand to approximately 960KB. The detailed boot architecture, protocol specifications, and implementation details are documented in the SMC ROM Boot Architecture Specification located at `fw/smc/prod_rom/specification/smc_rom.adoc`.

5.5.4. Integration and Operation

The ROM must be populated with a valid firmware image before the system initiates the boot sequence. The firmware image is typically programmed into the ROM memory during manufacturing or through a secure programming interface before the first power-on reset. The ROM contents remain fixed throughout system operation, ensuring a reliable and tamper-resistant boot source.

The ROM firmware implements chiplet coordination functionality, distinguishing between primary and secondary chiplet roles based on hardware strap configuration. Primary chiplets act as boot masters, coordinating the initialization of secondary chiplets and distributing boot images across the multi-chiplet system. Secondary chiplets wait for coordination commands and boot images from the primary chiplet, enabling synchronized system bring-up.

The OCCP protocol implementation enables secure and reliable communication between chiplets during the boot process. The protocol supports both secure and non-secure boot modes, with secure modes requiring manifest validation through coordination with the SEP (Security and Exception Processor) subsystem. The ROM protects its own memory regions from corruption through explicit access control checks, preventing unauthorized modification of boot code even in non-secure operational modes.

5.6. Interrupts

The SMC can handle interrupt sources across diverse system management functions. The architecture centers around the RISC-V Platform-Level Interrupt Controller (PLIC) and Core Local Interruptor (CLINT), with interrupt routing and priority handling.

This chapter is the single place for interrupt **address maps**, the `cpu_interrupts_o` vector, **PLIC/CLINT register** includes, and **bus error (NMI)** behavior.

5.6.1. Interrupt controllers (MMIO)

The CPU cluster integrates with system interrupt controllers via standardized interfaces. PLIC (AXI4-Lite) and CLINT (AXI4-Lite) are used for external and timer/software interrupts, respectively. Base addresses and interfaces (SMC-local view):

Block	Base Addr	Size	Interface
CPU Control Regs	0xC001_0000	8 KB	AXI4-Lite
PLIC	0xC400_0000	4 MB	AXI4-Lite
CLINT	0xC800_0000	64 KB	AXI4-Lite
Watchdog Timers (N)	0xC000_0000 + (N * 0x400)	1 KB x4	APB4
Bus Error Units (N)	0xC801_0000 + (N * 0x1000)	4 KB x4	AXI4-Lite

5.6.2. SMC interrupt vector (`cpu_interrupts_o`)

The PLIC exposes 332 global interrupt sources (indices 0–331) in the 4-core configuration. Most indices are driven from the SMC `cpu_interrupts_o` assembly documented below; additional globals (for example per-core Chipyard watchdog timers at indices 328–331) are bound inside the CPU cluster and do not appear on the raw `cpu_interrupts_o` bus.

Bus Error Unit interrupts

The Bus Error Units (BEU) provide per-core bus error monitoring. Unlike other interrupt sources, BEU interrupts bypass the PLIC and connect directly to each CPU core’s interrupt input as a non-maskable interrupt (NMI-like behavior). This ensures immediate notification of bus errors regardless of PLIC configuration or interrupt masking.

For the raw `cpu_interrupts_o` bit map traced back to the SMC RTL, including separate 4-core and 1-core layouts and the distinction between raw vector bits and PLIC source IDs, see the following section.

5.6.3. SMC CPU Interrupt Vector Map

This section documents the raw `cpu_interrupts_o` vector assembled in `hw/smc/smc_base.sv`. The vector is built from four sources:

- `ext_interrupts_i`
- `peripheral_interrupts_i[31:0]`
- `mailbox_interrupts[31:0]`
- four internal interrupt signals

Unless otherwise noted, the bit numbers below are raw `cpu_interrupts_o` bit indices, not PLIC source IDs. When these signals feed the PLIC, the PLIC source ID is `cpu_interrupts_o` bit index + 1.

The vector layout is:

```

cpu_interrupts_o[NUM_EXT_INTERRUPTS-1:0]      = ext_interrupts_i;
cpu_interrupts_o[NUM_EXT_INTERRUPTS+:32]      = peripheral_interrupts_i;
cpu_interrupts_o[(NUM_EXT_INTERRUPTS+32)+:32] = mailbox_interrupts;
cpu_interrupts_o[(NUM_EXT_INTERRUPTS+64)]      = tdr_dbg_ctrl_clocks_stopped_by_cla_o;
cpu_interrupts_o[(NUM_EXT_INTERRUPTS+64)+1]    = cla_interrupt;
cpu_interrupts_o[(NUM_EXT_INTERRUPTS+64)+2]    = dma_intp;
cpu_interrupts_o[(NUM_EXT_INTERRUPTS+64)+3]    = zeroer_intp;

```

For the 4-core configuration, `NUM_EXT_INTERRUPTS` = 256 and `NUM_CPU_INTERRUPTS` = 328. For the 1-core configuration, `NUM_EXT_INTERRUPTS` = 32 and `NUM_CPU_INTERRUPTS` = 104.

Exact Indexed Map

The table below shows the exact raw `cpu_interrupts_o` indexing for both supported CPU configurations. The **Source** column is intentionally human-readable; implementation signal names are called out in the **RTL trace** column instead.

4-core bit(s)	1-core bit(s)	Source	Description	RTL trace
[255:0]	[31:0]	External interrupts	Integration-defined external interrupt inputs	<code>hw/smc/smc_base.sv</code> drives <code>cpu_interrupts_o[NUM_EXT_INTERRUPTS-1:0] = ext_interrupts_i</code> .
256	32	SEP mailbox interrupt 0	SEP-to-SMC mailbox channel 0 interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[0] = sep_mailbox_interrupts_i[0]</code> .
257	33	SEP mailbox interrupt 1	SEP-to-SMC mailbox channel 1 interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[1] = sep_mailbox_interrupts_i[1]</code> .
258	34	SEP mailbox interrupt 2	SEP-to-SMC mailbox channel 2 interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[2] = sep_mailbox_interrupts_i[2]</code> .
259	35	SEP mailbox interrupt 3	SEP-to-SMC mailbox channel 3 interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[3] = sep_mailbox_interrupts_i[3]</code> .
260	36	SEP mailbox interrupt 4	SEP-to-SMC mailbox channel 4 interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[4] = sep_mailbox_interrupts_i[4]</code> .
261	37	SEP mailbox interrupt 5	SEP-to-SMC mailbox channel 5 interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[5] = sep_mailbox_interrupts_i[5]</code> .
262	38	SEP mailbox interrupt 6	SEP-to-SMC mailbox channel 6 interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[6] = sep_mailbox_interrupts_i[6]</code> .
263	39	SEP mailbox interrupt 7	SEP-to-SMC mailbox channel 7 interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[7] = sep_mailbox_interrupts_i[7]</code> .
264	40	Telemetry interrupt 0	Telemetry receiver instance 0 interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[8] = telemetry_irq[0]</code> .
265	41	Telemetry interrupt 1	Telemetry receiver instance 1 interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[9] = telemetry_irq[1]</code> .
266	42	Telemetry interrupt 2	Telemetry receiver instance 2 interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[10] = telemetry_irq[2]</code> .
267	43	NDM Reset Request	OR-reduced synchronized NDM reset request interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[11] = ndmreset_request_smc_clk</code> .
268	44	I3C interrupt 0	I3C controller instance 0 interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[12] = i3c_irqs_smc_clk[0]</code> ; <code>i3c_irqs_periph_clk</code> is currently tied to '0'.
269	45	I3C interrupt 1	I3C controller instance 1 interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[13] = i3c_irqs_smc_clk[1]</code> ; <code>i3c_irqs_periph_clk</code> is currently tied to '0'.
270	46	I3C interrupt 2	I3C controller instance 2 interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[14] = i3c_irqs_smc_clk[2]</code> ; <code>i3c_irqs_periph_clk</code> is currently tied to '0'.
271	47	I3C interrupt 3	I3C controller instance 3 interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[15] = i3c_irqs_smc_clk[3]</code> ; <code>i3c_irqs_periph_clk</code> is currently tied to '0'.
272	48	I3C interrupt 4	I3C controller instance 4 interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[16] = i3c_irqs_smc_clk[4]</code> ; <code>i3c_irqs_periph_clk</code> is currently tied to '0'.

4-core bit(s)	1-core bit(s)	Source	Description	RTL trace
273	49	I3C interrupt 5	I3C controller instance 5 interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[17] = i3c_irqs_smc_clk[5];</code> <code>i3c_irqs_periph_clk</code> is currently tied to '0'.
274	50	UART/Log Engine interrupt 0	Combined UART IRQ, UART error, and log-engine interrupt for instance 0	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[18] = uart_irq_combined_smc_clk[0];</code> the combined signal is formed in <code>smc_peripherals_cdc.sv</code> .
275	51	UART/Log Engine interrupt 1	Combined UART IRQ, UART error, and log-engine interrupt for instance 1	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[19] = uart_irq_combined_smc_clk[1];</code> the combined signal is formed in <code>smc_peripherals_cdc.sv</code> .
276	52	UART/Log Engine interrupt 2	Combined UART IRQ, UART error, and log-engine interrupt for instance 2	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[20] = uart_irq_combined_smc_clk[2];</code> the combined signal is formed in <code>smc_peripherals_cdc.sv</code> .
277	53	UART/Log Engine interrupt 3	Combined UART IRQ, UART error, and log-engine interrupt for instance 3	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[21] = uart_irq_combined_smc_clk[3];</code> the combined signal is formed in <code>smc_peripherals_cdc.sv</code> .
278	54	AVSBus interrupt	Adaptive voltage scaling interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[22] = avsbus_irq_flopped.</code>
279	55	I2C interrupt 0	I2C controller instance 0 interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[23] = i2c_irqs_smc_clk[0].</code>
280	56	I2C interrupt 1	I2C controller instance 1 interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[24] = i2c_irqs_smc_clk[1].</code>
281	57	I2C interrupt 2	I2C controller instance 2 interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[25] = i2c_irqs_smc_clk[2].</code>
282	58	SEP watchdog reset	External watchdog reset indication from SEP path	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[26] = ~rst_ext_wdt_ni.</code>
283	59	Temperature sensor interrupt	PVT temperature interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[27] = temp_interrupt_i.</code>
284	60	Locked field access	eFuse locked-field access violation interrupt	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[28] = locked_field_access_interrupt,</code> sourced from the eFuse controller path.
285	61	GPIO interrupt OR, lower half	OR-reduction across the lower half of bonded GPIO interrupt sources	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[29] = gpio_interrupt[smc_pkg::NUM_BONDED_GPIO/2-1:0].</code>
286	62	GPIO interrupt OR, upper half	OR-reduction across the upper half of bonded GPIO interrupt sources	<code>smc_peripherals.sv</code> drives <code>peripheral_interrupts_o[30] = gpio_interrupt[smc_pkg::NUM_BONDED_GPIO-1:smc_pkg::NUM_BONDED_GPIO/2].</code>
287	63	Reserved	Unused peripheral interrupt slot; tied to zero	<code>peripheral_interrupts_o</code> is initialized to '0', and bit 31 is never overwritten in <code>smc_peripherals.sv</code> .
[319:288]	[95:64]	SMC mailbox interrupts [31:0]	Inbound SMC mailbox interrupt channels 31:0	<code>smc_base.sv</code> drives <code>cpu_interrupts_o[(NUM_EXT_INTERRUPTS+32)+:32] = mailbox_interrupts;</code> <code>mailbox.sv</code> drives <code>inbound_interrupt_o[m]</code> per mailbox channel.
320	96	CLA Clock Stop	DFD CLA clock-stop status interrupt	<code>smc_base.sv</code> drives this bit from <code>tdr_dbg_ctrl_clocks_stopped_by_cla_o,</code> sourced through <code>u_smc_dfd_wrap</code> in <code>smc_internal_regs.sv</code> .
321	97	CLA Interrupt	DFD CLA interrupt	<code>smc_base.sv</code> drives this bit from <code>cla_interrupt,</code> sourced from <code>external_action_debug_interrupt_o</code> on <code>u_smc_dfd_wrap</code> in <code>smc_internal_regs.sv</code> .

4-core bit(s)	1-core bit(s)	Source	Description	RTL trace
322	98	DMA transfer complete	Falling-edge pulse when DMA busy deasserts	<code>smc_base.sv</code> drives this bit from <code>dma_intp</code> ; <code>idma_wrapper.sv</code> generates <code>dma_intp_o</code> as a falling-edge pulse of <code>dma_busy_o</code> .
323	99	Zeroer transfer complete	Falling-edge pulse when Zeroer busy deasserts	<code>smc_base.sv</code> drives this bit from <code>zeroer_intp</code> ; <code>zeroer.sv</code> generates <code>zeroer_intp_o</code> on the falling edge of <code>zeroer_busy_o</code> .
[327:324]	[103:100]	Reserved	Unused tail bits; tied to zero	<code>cpu_interrupts_o</code> is initialized to '0', and no RTL overwrites these tail bits in <code>smc_base.sv</code> .

Trace Notes

- Peripheral interrupt construction is defined in `hw/smc/smc_peripherals/rtl/smc_peripherals.sv`.
- Mailbox interrupt construction is defined in `hw/ip/mailbox/rtl/mailbox.sv` and is routed into `cpu_interrupts_o` by `hw/smc/smc_base.sv`.
- Internal interrupt construction is defined by `hw/smc/smc_base.sv`, with the DFD CLA signals sourced through `hw/smc/smc_internal_regs.sv`, the DMA completion pulse sourced by `hw/comp/idma_wrapper/rtl/idma_wrapper.sv`, and the Zeroer completion pulse sourced by `hw/comp/zeroer/rtl/zeroer.sv`.
- The raw vector documents SMC-generated sources only. The meaning of each external interrupt bit is defined by the enclosing SoC integration.

5.6.4. RISC-V PLIC

The SMC uses a standards-compliant RISC-V Platform-Level Interrupt Controller (PLIC) at 0xC400_0000–0xC43F_FFFF (4MB) to manage 332 interrupt sources (326 active, 6 reserved) with multi-core delivery in the 4-core configuration. The PLIC provides hardware prioritization, per-core routing, and configurable thresholds for efficient, parallel interrupt handling. The SMC-driven contribution to global interrupt indices is given in [SMC CPU Interrupt Vector Map](#) above.

Software must initialize PLIC priority and enable registers to known values (typically priority 0 and disabled) before enabling external interrupts; these registers have no hardware reset, and undefined values can block setting `mstatus.MIE`.

The `mie` CSR also has no reset value. Clear and initialize it before setting individual enable bits and `mstatus.MIE`.

5.6.5. System Management Interrupt Source Integration

The SMC interrupt system brings together a broad set of interrupt sources that support reliable and responsive system monitoring and control, summarized in the following table.

Table 296. SMC interrupt sources

Interrupt Component	Address Range & Configuration	System Management Function	Response Characteristics
Platform-Level Interrupt Controller (PLIC)	0xC400_0000 - 0xC43F_FFFF (4MB), 332 sources (326 active), multi-core delivery	Standards-compliant interrupt management with system management optimization	Prioritized, multi-level response with adaptive thresholds
Core Local Interruptor (CLINT)	0xC800_0000 - 0xC800_FFFF, per-core timer/software interrupts	Core-local timing, inter-core coordination, periodic operations	Deterministic timing, low-latency inter-core communication
Watchdog Timer System (WDT)	0xC000_0000 + (N × 0x400), 4 per-core instances with escalation	System reliability monitoring, fault detection, automated recovery	Escalating response: warning → reset → external intervention

Interrupt Component	Address Range & Configuration	System Management Function	Response Characteristics
Bus Error Detection Units	0xC801_0000 + (N × 0x1000), 4 per-core AXI monitoring instances, direct to cores (not PLIC)	Bus infrastructure integrity monitoring and fault isolation	Immediate fault reporting as NMI-like interrupt

For CPU cluster context (caches, TileLink attachment), see [CPU & Memory System](#).

5.6.6. Multi-Tier Interrupt Classification and Response Strategy

The SMC interrupt architecture classifies interrupt sources by system management function and response needs. System-wide events are delivered via the SMC_CPU_INTERRUPT_WIDTH interface, which supports flexible routing for multi-core systems.

CLINT-generated local interrupts provide core-specific timing and coordination, while software interrupts allow efficient inter-core communication. Dedicated interrupts for watchdog timeouts and bus errors ensure system management can respond quickly to faults or infrastructure issues with appropriate, targeted actions.

5.6.7. Multi-Core Interrupt Distribution and Load Management

The SMC supports independent interrupt delivery to each CPU core, enabling distribution of interrupt handling and specialized core roles. Critical and real-time events can be routed to low-latency cores, while high-volume interrupts like telemetry are handled by throughput-oriented cores.

Hardware-managed priorities allow efficient resolution of competing interrupts without software overhead. Per-core interrupt masking gives software precise control, so interrupt delivery can be dynamically tuned to operational needs and system management policies.

5.6.8. RISC-V CLINT and Precise System Management Timing

The Core Local Interruptor (CLINT) in the SMC provides per-core timer and software interrupt functionality for precise, deterministic timing and inter-core coordination.

The CLINT occupies 64KiB of address space (0xC800_0000–0xC800_FFFF). It enables each core to schedule time-critical operations and issue low-latency software interrupts directly, bypassing the PLIC. This supports both accurate RTOS scheduling and fast inter-processor communication essential to reliable multi-core system management.

5.6.9. Fault Detection and Recovery Architecture

The SMC provides fault detection and recovery through per-core watchdog timers and bus error monitoring, enabling both local fault recovery and coordination with broader system reliability mechanisms. They are described in the following sections.

5.6.10. Advanced PLIC Management and Adaptive Priority Control

In the PLIC, each interrupt source has a configurable priority, letting software focus on critical events without overloading the system.

Per-core and per-context enables let cores handle only select subsets of interrupts, so real-time events go to low-latency cores and bulk tasks go to high-throughput cores.

Threshold control allows filtering interrupts based on system conditions or load. The software can adjust thresholds

dynamically, keeping the right balance of responsiveness and system overhead.

The atomic claim and completion system ensures reliable, efficient interrupt handling with low software cost. It prevents lost or duplicated interrupts and smoothly handles high interrupt rates.

5.6.11. Multi-Context Architecture and Privilege Level Integration

The multi-context architecture supports security boundaries and privilege separation in interrupt handling. Machine-mode and supervisor-mode contexts allow software components to use different system access levels for interrupts.

Per-core independence means each core's interrupt configuration does not affect others, allowing specialized functionality while keeping system behavior consistent.

Context isolation with separate enable and threshold settings per privilege level enforces security so that privilege escalation cannot compromise interrupt handling, while still allowing flexible processing at different privilege levels.

5.6.12. Precision Timer Management and Inter-Core Coordination

The CLINT timer system delivers precise timing for managing both individual cores and coordinated multi-core operations, using a 64-bit timer for a wide range of real-time or long-duration tasks.

Per-core timer compare allows each core to run its own timing routines, enabling high-frequency interrupts for critical tasks and lower-frequency ones for routine maintenance with minimal overhead.

A shared timer counter provides a unified time reference for multi-core coordination, allowing easy synchronization across cores without complex software logic.

Software interrupts enable fast communication between cores, supporting efficient and responsive multi-core management.

5.6.13. Register Interfaces

Register Arrays: This register map contains the following register arrays:

- **PRIORITY[337]:** 337 registers
 - Base Address: 0x0
 - Address Increment: 0x4 per register
- **PENDING[11]:** 11 registers
 - Base Address: 0x1000
 - Address Increment: 0x4 per register
- **CORE0_MEIP_ENABLE[11]:** 11 registers
 - Base Address: 0x2000
 - Address Increment: 0x4 per register
- **CORE1_MEIP_ENABLE[11]:** 11 registers
 - Base Address: 0x2080
 - Address Increment: 0x4 per register

NOTE

- **CORE2_MEIP_ENABLE[11]:** 11 registers
 - Base Address: 0x2100
 - Address Increment: 0x4 per register
- **CORE3_MEIP_ENABLE[11]:** 11 registers
 - Base Address: 0x2180
 - Address Increment: 0x4 per register
- **CORE0_SEIP_ENABLE[11]:** 11 registers
 - Base Address: 0x2200
 - Address Increment: 0x4 per register
- **CORE1_SEIP_ENABLE[11]:** 11 registers
 - Base Address: 0x2280
 - Address Increment: 0x4 per register
- **CORE2_SEIP_ENABLE[11]:** 11 registers
 - Base Address: 0x2300
 - Address Increment: 0x4 per register
- **CORE3_SEIP_ENABLE[11]:** 11 registers
 - Base Address: 0x2380
 - Address Increment: 0x4 per register

Table 297. Register Map

Address	Name	Access	Description
0x0 - 0x540	PRIORITY[337]	RW	-
0x1000 - 0x1028	PENDING[11]	R	-
0x2000 - 0x2028	CORE0_MEIP_ENABLE[11]	RW	-
0x2080 - 0x20A8	CORE1_MEIP_ENABLE[11]	RW	-
0x2100 - 0x2128	CORE2_MEIP_ENABLE[11]	RW	-
0x2180 - 0x21A8	CORE3_MEIP_ENABLE[11]	RW	-
0x2200 - 0x2228	CORE0_SEIP_ENABLE[11]	RW	-
0x2280 - 0x22A8	CORE1_SEIP_ENABLE[11]	RW	-
0x2300 - 0x2328	CORE2_SEIP_ENABLE[11]	RW	-
0x2380 - 0x23A8	CORE3_SEIP_ENABLE[11]	RW	-
0x200000	CORE0_MEIP_THRESHOLD	RW	-
0x200004	CORE0_MEIP_CLAIM_COMPLETE	RW	-

Address	Name	Access	Description
0x201000	CORE1_MEIP_THRESHOLD	RW	-
0x201004	CORE1_MEIP_CLAIM_COMPLETE	RW	-
0x202000	CORE2_MEIP_THRESHOLD	RW	-
0x202004	CORE2_MEIP_CLAIM_COMPLETE	RW	-
0x203000	CORE3_MEIP_THRESHOLD	RW	-
0x203004	CORE3_MEIP_CLAIM_COMPLETE	RW	-
0x204000	CORE0_SEIP_THRESHOLD	RW	-
0x204004	CORE0_SEIP_CLAIM_COMPLETE	RW	-
0x205000	CORE1_SEIP_THRESHOLD	RW	-
0x205004	CORE1_SEIP_CLAIM_COMPLETE	RW	-
0x206000	CORE2_SEIP_THRESHOLD	RW	-
0x206004	CORE2_SEIP_CLAIM_COMPLETE	RW	-
0x207000	CORE3_SEIP_THRESHOLD	RW	-
0x207004	CORE3_SEIP_CLAIM_COMPLETE	RW	-

Table 298. PRIORITY[337] Register

Bits	Field	Access	Reset	Description
31:3	Reserved	-	-	Reserved
2:0	value	RW	-	Priority of interrupt. The higher the set value, the higher the priority. For interrupts with the same priority, the tie is broken by interrupt ID; interrupts with the lowest ID has highest effective priority. Interrupts start counting at 1.

Table 299. PENDING[11] Register

Bits	Field	Access	Reset	Description
31:0	value	R	0x0	Set to 1 if interrupt source 1 is pending, regardless of its enable or priority setting. Interrupts start counting at 1.

Table 300. CORE0_MEIP_ENABLE[11] Register

Bits	Field	Access	Reset	Description
31:0	value	RW	-	Set bits to 1 if interrupt should be enabled. Interrupts start counting at 1.

Table 301. CORE1_MEIP_ENABLE[11] Register

Bits	Field	Access	Reset	Description
31:0	value	RW	-	Set bits to 1 if interrupt should be enabled. Interrupts start counting at 1.

Table 302. CORE2_MEIP_ENABLE[11] Register

Bits	Field	Access	Reset	Description
31:0	value	RW	-	Set bits to 1 if interrupt should be enabled. Interrupts start counting at 1.

Table 303. CORE3_MEIP_ENABLE[11] Register

Bits	Field	Access	Reset	Description
31:0	value	RW	-	Set bits to 1 if interrupt should be enabled. Interrupts start counting at 1.

Table 304. CORE0_SEIP_ENABLE[11] Register

Bits	Field	Access	Reset	Description
31:0	value	RW	-	Set bits to 1 if interrupt should be enabled. Interrupts start counting at 1.

Table 305. CORE1_SEIP_ENABLE[11] Register

Bits	Field	Access	Reset	Description
31:0	value	RW	-	Set bits to 1 if interrupt should be enabled. Interrupts start counting at 1.

Table 306. CORE2_SEIP_ENABLE[11] Register

Bits	Field	Access	Reset	Description
31:0	value	RW	-	Set bits to 1 if interrupt should be enabled. Interrupts start counting at 1.

Table 307. CORE3_SEIP_ENABLE[11] Register

Bits	Field	Access	Reset	Description
31:0	value	RW	-	Set bits to 1 if interrupt should be enabled. Interrupts start counting at 1.

Table 308. CORE0_MEIP_THRESHOLD Register

Bits	Field	Access	Reset	Description
31:3	Reserved	-	-	Reserved
2:0	value	RW	-	Interrupt & claim threshold. Maximum value is 7. Any interrupt with a priority less than the threshold cannot interrupt the cpu

Table 309. CORE0_MEIP_CLAIM_COMPLETE Register

Bits	Field	Access	Reset	Description
31:0	value	RW	-	Claim/Complete register. Reading this register returns the claimed interrupt number and makes it no longer pending. Writing the interrupt number back completes the interrupt.

Table 310. CORE1_MEIP_THRESHOLD Register

Bits	Field	Access	Reset	Description
31:3	Reserved	-	-	Reserved
2:0	value	RW	-	Interrupt & claim threshold. Maximum value is 7. Any interrupt with a priority less than the threshold cannot interrupt the cpu

Table 311. CORE1_MEIP_CLAIM_COMPLETE Register

Bits	Field	Access	Reset	Description
31:0	value	RW	-	Claim/Complete register. Reading this register returns the claimed interrupt number and makes it no longer pending. Writing the interrupt number back completes the interrupt.

Table 312. CORE2_MEIP_THRESHOLD Register

Bits	Field	Access	Reset	Description
31:3	Reserved	-	-	Reserved
2:0	value	RW	-	Interrupt & claim threshold. Maximum value is 7. Any interrupt with a priority less than the threshold cannot interrupt the cpu

Table 313. CORE2_MEIP_CLAIM_COMPLETE Register

Bits	Field	Access	Reset	Description
31:0	value	RW	-	Claim/Complete register. Reading this register returns the claimed interrupt number and makes it no longer pending. Writing the interrupt number back completes the interrupt.

Table 314. CORE3_MEIP_THRESHOLD Register

Bits	Field	Access	Reset	Description
31:3	Reserved	-	-	Reserved
2:0	value	RW	-	Interrupt & claim threshold. Maximum value is 7. Any interrupt with a priority less than the threshold cannot interrupt the cpu

Table 315. CORE3_MEIP_CLAIM_COMPLETE Register

Bits	Field	Access	Reset	Description
31:0	value	RW	-	Claim/Complete register. Reading this register returns the claimed interrupt number and makes it no longer pending. Writing the interrupt number back completes the interrupt.

Table 316. CORE0_SEIP_THRESHOLD Register

Bits	Field	Access	Reset	Description
31:3	Reserved	-	-	Reserved
2:0	value	RW	-	Interrupt & claim threshold. Maximum value is 7. Any interrupt with a priority less than the threshold cannot interrupt the cpu

Table 317. CORE0_SEIP_CLAIM_COMPLETE Register

Bits	Field	Access	Reset	Description
31:0	value	RW	-	Claim/Complete register. Reading this register returns the claimed interrupt number and makes it no longer pending. Writing the interrupt number back completes the interrupt.

Table 318. CORE1_SEIP_THRESHOLD Register

Bits	Field	Access	Reset	Description
31:3	Reserved	-	-	Reserved
2:0	value	RW	-	Interrupt & claim threshold. Maximum value is 7. Any interrupt with a priority less than the threshold cannot interrupt the cpu

Table 319. CORE1_SEIP_CLAIM_COMPLETE Register

Bits	Field	Access	Reset	Description
31:0	value	RW	-	Claim/Complete register. Reading this register returns the claimed interrupt number and makes it no longer pending. Writing the interrupt number back completes the interrupt.

Table 320. CORE2_SEIP_THRESHOLD Register

Bits	Field	Access	Reset	Description
31:3	Reserved	-	-	Reserved
2:0	value	RW	-	Interrupt & claim threshold. Maximum value is 7. Any interrupt with a priority less than the threshold cannot interrupt the cpu

Table 321. CORE2_SEIP_CLAIM_COMPLETE Register

Bits	Field	Access	Reset	Description
31:0	value	RW	-	Claim/Complete register. Reading this register returns the claimed interrupt number and makes it no longer pending. Writing the interrupt number back completes the interrupt.

Table 322. CORE3_SEIP_THRESHOLD Register

Bits	Field	Access	Reset	Description
31:3	Reserved	-	-	Reserved
2:0	value	RW	-	Interrupt & claim threshold. Maximum value is 7. Any interrupt with a priority less than the threshold cannot interrupt the cpu

Table 323. CORE3_SEIP_CLAIM_COMPLETE Register

Bits	Field	Access	Reset	Description
31:0	value	RW	-	Claim/Complete register. Reading this register returns the claimed interrupt number and makes it no longer pending. Writing the interrupt number back completes the interrupt.

5.7. Fabric

The SMC fabric provides interconnect infrastructure with address management, traffic filtering, and security enforcement mechanisms.

5.7.1. Configuration Parameters

The fabric supports the following configuration parameters to tune performance and area:

Table 324. Fabric Configuration Parameters

Parameter	Default	Description
AXI4 Network Parameters		

Parameter	Default	Description
MaxTrans	32	Maximum outstanding transactions per AXI ID in output fabric
MAX_INFLIGHT_IDS	4	Maximum number of in-flight transaction IDs for ID remapping
FABRIC_MAX_TRANS	32	Maximum outstanding transactions per AXI ID in fabric crossbars
ERR_SLV_MAX_TRANS	32	Maximum outstanding transactions for error slave (across all IDs)
NumFilters	16	Number of inbound and outbound traffic filter instances
FilterReqPipelineEnable	0	Enable request path pipelining in filters (timing vs. latency trade-off)
FilterRspPipelineEnable	0	Enable response path pipelining in filters (timing vs. latency trade-off)
AXI4-Lite Network Parameters		
AXI_MAX_WRITE_TXNS	4	Maximum outstanding write transactions in local fabric crossbar
AXI_MAX_READ_TXNS	4	Maximum outstanding read transactions in local fabric crossbar
Address Remap Parameters		
MmodeBaseAddr	0x100_0000	Base address for M-mode remapped region (relative to SMC base)
XvisorBaseAddr	0x180_0000	Base address for Xvisor remapped region (relative to SMC base)
NoRemap	0	Disable address remapping when set (reduces area)

These parameters control fabric throughput, buffering depth, and feature enablement. The AXI4 network parameters (MaxTrans, FABRIC_MAX_TRANS) define the pipeline depth for high-performance traffic, while AXI4-Lite parameters (AXI_MAX_WRITE_TXNS, AXI_MAX_READ_TXNS) control outstanding transactions to peripherals.

5.7.2. Dual-Network Architecture

The SMC fabric implements two separate networks optimized for different traffic types:

Table 325. Network Characteristics

Network	Connected Components	Protocol	Optimization Target
High-Performance	CPU cluster, SRAM, DMA, filtering, external AXI ports	AXI4	Bandwidth, low latency
Low-Performance	Peripherals, sensors, configuration registers	AXI4-Lite	Reliability, deterministic access

Both networks use 64-bit data bus width for consistent data handling. The CPU cluster connects to the fabric through AXI4 (high-bandwidth data path) and AXI4-Lite (MMIO) interfaces; internally, the cluster uses a TileLink bus that is protocol-bridged at the cluster boundary.

5.7.3. AXI Interface Widths

The table below lists the common AXI signal widths shared across all AXI4 fabric interfaces.

Table 326. AXI Common Signal Widths

Signal	Width (bits)	Description
DATA_WIDTH	64	Data bus width for all AXI4 interfaces
STRB_WIDTH	8	Write strobe width (DATA_WIDTH / 8)
ADDR_WIDTH (system)	56	Address width on system-facing external ports
ADDR_WIDTH (local)	32	Address width on SMC-internal interfaces
USER_WIDTH	12	User sideband width for all AXI4 channels

The AXI transaction ID width varies at each fabric stage because crossbars and multiplexers prepend additional bits to distinguish the originating port. The table below lists the ID width at each point in the fabric.

Table 327. AXI ID Widths by Fabric Stage

Fabric Stage / Port	ID Width (bits)	Direction
System AXI input (sys_axi_in)	6	Inbound
JTAG AXI input (jtag_axi_in)	2	Inbound
SEP AXI input (sep_axi_in)	6	Inbound
Input fabric (slave side)	4	Internal
Local / Output fabric (slave side)	6	Internal
Local fabric crossbar (master side)	8	Internal
Output fabric (master side)	8	Outbound
System AXI output (output_axi)	8	Outbound

Some internal paths also use a 32-bit data width for AXI4-Lite peripheral access, with corresponding STRB_WIDTH of 4 bits.

5.7.4. Traffic Managers and Subordinates

The SMC fabric supports multiple traffic sources (managers) and destinations (subordinates), providing flexible routing with security enforcement.

Traffic Managers

Traffic managers initiate transactions on the fabric, accessing both local and remote resources through the dual-network architecture.

Table 328. Fabric Traffic Managers

Manager	Count	Primary Paths	Key Features
CPU Cluster	1	Local (SRAM, PLIC, CLINT, WDT), External (filtered/remapped)	Dual routing paths, privilege-based control, L2 coherent access
DMA Controller	1	Alias remap → filtering → destination	High-bandwidth data movement, security enforcement
JTAG2AXI Bridge	1	Alias remap → filtering → destination	Debug and test access, privilege control
Log Engine	1	Alias remap → filtering → destination	System logging, telemetry collection

The CPU cluster provides multiple interface types: the local path optimized for minimal latency to local resources, the external path with filtering and remapping for system-wide access, the MMIO interface for register access with privilege-based control, and the L2 coherent path for cache-coherent high-bandwidth operations.

Traffic from DMA, Log Engine, JTAG2AXI, and CPU external path passes through alias remap logic before reaching their destination, enabling flexible address management between local and global address spaces.

Traffic Subordinates

Traffic subordinates respond to transactions initiated by the managers, organized by network type and function.

Table 329. Fabric Traffic Subordinates

Subordinate	Count	Network	Function
SRAM	1	AXI4 (High-Performance)	Local instruction and data memory
PLIC	1	AXI4 (High-Performance)	Platform-level interrupt controller
CLINT	1	AXI4 (High-Performance)	Core-local interrupt controller, timer
Watchdog Timer	1	AXI4 (High-Performance)	System integrity monitoring
Local Peripherals	Multiple	AXI4-Lite (Low-Performance)	SMC peripheral blocks (UART, I2C, GPIO, etc.)
Configuration Registers	Multiple	AXI4-Lite (Low-Performance)	Fabric control, filtering, remapping
External AXI Input Ports	3	AXI4 (High-Performance)	System NoC (sys_axi_in), JTAG debug (jtag_axi_in), SEP (sep_axi_in)
External AXI Output Port	1	AXI4 (High-Performance)	System NoC interface (output_axi)
Error Slave	1	Both networks	Invalid address response generation

The subordinates are distributed across both networks based on their performance requirements and access patterns. High-performance subordinates (SRAM, PLIC, CLINT, WDT, external ports) connect to the AXI4 network for maximum bandwidth and minimal latency, while low-performance subordinates (peripherals, configuration registers) use the AXI4-Lite network for reliable, deterministic access.

5.7.5. Address Remapping

The SMC fabric implements two complementary address remapping mechanisms in ``smc_fabric/smc_fabric_misc/``: alias remapping and privilege-controlled remapping.

5.7.6. Alias Remapping

Alias remapping translates local SMC addresses to global system addresses, decoupling software-visible addresses from physical assignments. This simplifies integration and software portability, letting local agents access resources consistently across different SoC mappings.

The alias remap logic organizes eight independently configurable regions, each specifying an input address range, a remap offset for translation, and a cacheable flag. This enables:

- Translation between local alias addresses (0xC000_0000 at reset) and global addresses, ensuring compatibility with both internal and external software
- Support for multiple software images by allowing address regions to be reassigned for different execution instances
- Setting cacheable attributes on locally initiated transactions for optimal performance

At reset, remap logic defaults to transparent mapping (no remap), using the local alias base address (0xC000_0000) for local resources until programmed by firmware.

5.7.7. Privilege-Controlled Remapping

Privilege-controlled address remapping enables programmable, privilege-aware translation of outbound SMC transactions. It enforces access boundaries based on the originating master's privilege level, supporting secure memory compartmentalization and reducing privilege escalation risks.

The fabric provides separate remap regions for Machine-mode (highest privilege) and Xvisor/Hypervisor (intermediate privilege) operations. When accessing shared resources like shared memory, SMC traffic passes through the appropriate privilege-controlled remap stage to ensure proper address translation and privilege enforcement.

a. rubric:: Traffic Filtering

The filtering system (filter_ctrl.rdl`) provides security control for both inbound and outbound traffic, examining transaction attributes to enforce access policies.

5.7.8. Inbound Filtering

Inbound filtering protects SMC resources from unauthorized external access by evaluating three dimensions of each transaction:

- **Address-based control:** Defines precise access zones, protecting memory-mapped registers and internal memory from unauthorized external requests
- **Source ID authorization:** Permits or blocks accesses based on the originating system component's identity, ensuring only trusted sources access sensitive resources
- **Protection attribute filtering:** Examines AXI protection signals (security and privilege attributes) to differentiate between secure vs. non-secure or privileged vs. user transactions

The default filter configuration varies by chiplet type. Most SMC instances default to blocking all external transactions, requiring firmware to explicitly configure access policies during cold boot.

The inbound filter interface provides 16 independent filter instances, each configurable with its own address ranges, source ID matching, and protection attribute policies.

5.7.9. Outbound Filtering

Outbound filtering manages which external resources the SMC can access, restricting SMC-initiated requests to defined system peripherals or memory regions. The filtering system:

- **Controls destinations:** Specifies allowable external resource access, reinforcing separation and limiting SMC influence on the rest of the SoC
- **Adapts to privilege levels:** Dynamically adjusts policies based on software privilege level—higher privilege modes receive broader access while lower privilege software is constrained
- **Enforces security:** Serves as a gatekeeper preventing unauthorized access attempts by blocking traffic that doesn't meet configured privilege or destination criteria

All outbound traffic, after passing through any privilege remap stages, is subject to outbound filtering enforcement based on address, security attributes (NS), and source ID. The outbound filters use the same register structure as inbound filters.

5.7.10. Protection Requirements

The SMC fabric implements two types of protection-based filtering mechanisms to enforce security policies based on AXI protection attributes:

AXI Protection Bit Encoding

AXI protection bits (**aw.prot** and **ar.prot**) are 3-bit fields that encode transaction security and privilege attributes:

Table 330. Protection Bit Encoding

Bit [2]	Bit [1]	Bit [0]	Value	Description
0	0	0	0x0	Data, Secure, User
0	0	1	0x1	Data, Secure, Privileged
0	1	0	0x2	Data, Non-Secure, User
0	1	1	0x3	Data, Non-Secure, Privileged
1	0	0	0x4	Instruction, Secure, User
1	0	1	0x5	Instruction, Secure, Privileged
1	1	0	0x6	Instruction, Non-Secure, User
1	1	1	0x7	Instruction, Non-Secure, Privileged

Non-Secure Bit Filtering (AXI4 Filters)

The primary protection filtering mechanism for AXI4 full interfaces examines the Non-Secure (NS) bit (`prot[1]`) to control secure vs. non-secure access. This filtering is implemented in both inbound and outbound traffic filters.

Each filter entry in the 16 inbound and 16 outbound filter instances includes an `allow_ns` configuration field that determines whether non-secure accesses are permitted:

- When `allow_ns = 0`: Only secure accesses (`prot[1] = 0`) are allowed through the filter
- When `allow_ns = 1`: Both secure and non-secure accesses are allowed

This filtering occurs in `axi_filter_wrap`, where each transaction's `aw.prot[1]` (write) or `ar.prot[1]` (read) is checked against the filter's `allow_ns` configuration. Transactions that fail the protection check are routed to an error slave, which returns a decode error response.

Full 3-Bit Protection Matching (AXI-Lite)

For AXI4-Lite interfaces, the fabric supports exact matching on all three protection bits. This is implemented in the `prim_axil_prot_filter` module, which is used by GPIO POC/PBIAS control and other AXI-Lite peripherals.

The full protection matching requires an exact match between the transaction's protection bits and a configured requirement:

- `awprot_requirement[2:0]`: Required protection value for write transactions
- `arprot_requirement[2:0]`: Required protection value for read transactions
- `write_filter_enable`: Enable write protection filtering
- `read_filter_enable`: Enable read protection filtering

When filtering is enabled, only transactions with matching protection values are allowed through. All other transactions receive an error response.

Protection Bit Pass-Through

All protection bits are passed through the fabric without modification. The filters only block or allow transactions based on their protection attributes; they do not alter the protection bits themselves. This ensures that downstream slaves receive the original protection attributes from the originating master.

Configuration Examples

Example 1: Secure-only access to a peripheral

Configure filter with:

- allow_ns = 0
- Address range matching peripheral region

Result: Only secure transactions (prot[1] = 0) are allowed

Example 2: Privileged-only register access

Configure AXI-Lite filter with:

- awprot_requirement = 0x1 (Data, Secure, Privileged)
- arprot_requirement = 0x1
- write_filter_enable = 1
- read_filter_enable = 1

Result: Only privileged transactions (prot[0] = 1) are allowed

5.7.11. Traffic Routing and Flow

Local and Remote Resource Access

The SMC fabric routing logic determines how transactions reach their intended destinations, supporting both local SMC resources and remote system components. Local resources can be addressed using either their global address or a local alias address.

The fabric distinguishes the chiplet-global SMC aperture from the internal local-alias window. The `GLOBAL_BASE` and `REGION_SIZE` CSRs describe the chiplet-global aperture used by SMU-level routing and SEP-to-SMC decode. The current `REGION_SIZE` reset value is `0x0100_0000` (16 MiB), and firmware may reprogram it during early boot to match the chiplet address assignment.

The SMC input fabric's local/global split uses the RTL `LOCAL_ALIAS_REGION_SIZE` parameter rather than the CSR `REGION_SIZE`. In the current RTL this parameter is 16 MiB. This keeps the SMC-internal local address window fixed while allowing the chiplet-facing global aperture to be programmed independently.

After global-aperture reconfiguration, software may use global addresses for accesses that leave the local SMC view. Global address derivation requires knowledge of the chiplet ID or chiplet BAR, both provisioned in fuses and available to firmware.

Outbound Traffic Flow

SMC outbound traffic serves dual purposes: accessing shared system services and extending the SMC's private domain across the system.

For shared resources (e.g., shared memory), traffic passes through privilege-controlled remap logic—Xvisor-controlled for hypervisor-managed resources or M-mode-controlled for machine-mode resources. For private memory or memory-mapped I/O resources, traffic bypasses remap stages and goes directly to the system NoC, reducing latency.

Each outbound transaction carries a Source ID identifying its privilege level and routing path, plus a cacheable flag from the alias remap configuration:

Table 331. Source ID by Traffic Path

Traffic Path	Source ID
Direct to NoC	SMC_ID
Through Xvisor remap	OTHER_ID
Through M-mode remap	MMODE_ID

5.7.12. Detailed Register Maps

Alias Remap Registers

The alias remap registers control local address aliasing and attribute mapping for SMC-initiated transactions. These registers configure how local resources are accessed and which address translation is applied.

Table 332. Register Map

Address	Name	Access	Description
0x0	region_start	RW	-
0x8	region_end	RW	-
0x10	region_attrs	RW	-

Table 333. region_start Register

Bits	Field	Access	Reset	Description
63:56	Reserved	-	-	Reserved
55:12	start_addr	RW	0x0	Start of remap region
11:0	Reserved	-	-	Reserved

Table 334. region_end Register

Bits	Field	Access	Reset	Description
63:56	Reserved	-	-	Reserved
55:12	end_addr	RW	0x0	End of remap region (non-inclusive).
11:0	Reserved	-	-	Reserved

Table 335. region_attrs Register

Bits	Field	Access	Reset	Description
63	valid	RW	0x0	If set, this remap will be active
62	cacheable	RW	0x0	If set, top two bits of a*cache will be tied high
61:56	Reserved	-	-	Reserved
55:12	offset	RW	0x0	The value of this field is added to bits [55:12] of the input address when it falls within the remap region. The lower 12 bits of the address are preserved unchanged.
11:0	Reserved	-	-	Reserved

Output Remap Registers

The output remap registers control privilege-controlled address remapping for outbound transactions. The fabric provides separate remap regions for Machine-mode (highest privilege) and Xvisor/Hypervisor (intermediate privilege) operations.

Table 336. Register Map

Address	Name	Access	Description
0x0	region_attrs	RW	-

Table 337. region_attrs Register

Bits	Field	Access	Reset	Description
63:56	Reserved	-	-	Reserved
55:0	offset	RW	0x0	Address translation for remap region. RTL will use the number of bits appropriate for region granularity. e.g. SMC: 8 * 1MB regions -> 20 bits of address space per region, 36 MSBs from this register used SEP: 16 * 512KB regions -> 19 bits of address space per region, 37 MSBs from this register used

Inbound Traffic Filter Control Registers

The inbound filter control registers protect SMC resources from unauthorized external access. Each of the 16 independent filter instances can be configured with address ranges, source ID matching, and protection attribute policies to control inbound traffic.

Table 338. Register Map

Address	Name	Access	Description
0x0	FILTER_CONFIG	RW	-
0x8	START_ADDR	RW	-
0x10	END_ADDR	RW	-

Table 339. FILTER_CONFIG Register

Bits	Field	Access	Reset	Description
63	locked	RW	0x0	Lock filter configurations. Write once register
62:25	Reserved	-	-	Reserved
24	allow_burst	RW	0x0	Allow burst transactions through this filter
23:20	group_id	RW	0x0	Group Id field to match to pass filter
19:16	src_id	RW	0x0	Source Id field to match to pass filter
15	Reserved	-	-	Reserved
14:12	data_bus_width	R	0x3	Data bus width in bytes, encoded as 2^value
11:9	Reserved	-	-	Reserved
8	allow_ns	RW	0x0	Only transactions with axprot[1] equal to this field will be allowed to pass
7:5	Reserved	-	-	Reserved
4	entry_enabled	RW	0x0	Filter entry enabled. A = 0 means this entry is disabled, A = 1 means this entry is enabled and will be used to filter
3:2	Reserved	-	-	Reserved

Bits	Field	Access	Reset	Description
1	<code>write_allowed</code>	RW	0x0	Filter range has write permissions
0	<code>read_allowed</code>	RW	0x0	Filter range has read permissions

Table 340. START_ADDR Register

Bits	Field	Access	Reset	Description
63:56	<code>Reserved</code>	-	-	Reserved
55:0	<code>start_addr</code>	RW	0x0	Start of allowable address range

Table 341. END_ADDR Register

Bits	Field	Access	Reset	Description
63:56	<code>Reserved</code>	-	-	Reserved
55:0	<code>end_addr</code>	RW	0x7	End of allowable address range. Defaults to 7 on reset to match the minimum granularity when burst is disabled

Outbound Traffic Filter Control Registers

The outbound filter control registers determine which external resources the SMC can access, restricting SMC-initiated requests to defined system peripherals or memory regions. These registers are implemented as 16 independent outbound filter instances, each structured as a 32-byte control block and accessible via an AXI4-Lite interface. The outbound filters use the same register structure as the inbound filters (documented above).

5.8. Direct Memory Access (DMA) Engine

The SMC DMA controller provides data movement services for telemetry collection, security data processing, and inter-component communication without CPU intervention. The controller uses a frontend-backend architecture based on the PULP `idma` engine, supporting multi-channel operation, scatter-gather transfers, and integration with SMC fabric filtering.

Table 342. DMA Controller Integration

Parameter	Description
Address Range	0xC003_8000 - 0xC003_81FF (512 bytes)
Control Interface	AXI4-Lite (9-bit address space, 64-bit data width)
Data Interface	AXI4 Master (56-bit address, 64-bit data, 4-bit ID)
Channel Support	Multiple independent concurrent transfer channels
Security	Integration with SMC fabric filtering mechanisms

5.8.1. Architecture Overview

The DMA controller implements a frontend-backend architecture with the following modules:

- **Frontend** (`idma_frontend_wrapper`): Register interface, command processing, transfer descriptor management, and configuration control
- **Backend** (`idma_backend_wrapper`): High-performance data movement engine with AXI master interface based on `idma`
- **Request Manager** (`idma_request_manager_wrapper`): Round-robin arbitration to distribute requests across

multiple master interfaces

This separation enables independent optimization and power management of control plane and data plane operations.

5.8.2. Configuration Parameters

The DMA controller supports the following configuration parameters:

Table 343. DMA Configuration Parameters

Parameter	Default	Description
NUM_CTRL_INTERFACES	1	Number of independent control/register interfaces (must be ≥ 1)
NUM_CTRL_STREAMS	1	Number of control streams per interface (must be ≥ 1)
NUM_MST_INTERFACES	1	Number of independent AXI master interfaces (must be ≥ 1)
DMA_MST_MAX_TXNS	16	Maximum outstanding AXI transactions per master interface
F2M_FIFO_DEPTH	4	Frontend-to-midend FIFO depth (minimum 1 to avoid stalls)
M2B_FIFO_DEPTH	0	Midend-to-backend FIFO depth (0 = passthrough, no FIFO)
EN_R_AW_COUPLING	1	Enable read/write address coupling (recommended for performance)
BYPASS_DMA_CTRL_FLOPS	0	Bypass register cut stages on control interface (timing optimization)
BYPASS_DMA_MST_FLOPS	0	Bypass register cut stages on master interface (timing optimization)
CG_HYSTERESIS_W	6	Clock gating hysteresis width in bits (6-bit = 0-63 cycle delay)

5.8.3. DMA Frontend

The frontend (idma_frontend_wrapper) manages register interface, command processing, and transfer descriptor handling.

Control Interface

The table below describes the frontend control interface.

Table 344. Frontend Control Interface

Feature	Description
Register Interface	AXI4-Lite (9-bit address, 64-bit data)
Control Interfaces	Multiple configurable control interfaces for independent DMA access
Atomic Operations	Atomic command submission and status reading
Error Reporting	Immediate visibility into operation status and error conditions
Configuration Lock	Protection against modification during active transfers

Transfer Descriptor Support

The table below describes the transfer descriptor support.

Table 345. Transfer Capabilities

Transfer Type	Description
Linear Transfers	Straightforward source-to-destination data movement
2D Transfers	Rectangular data blocks with configurable source/destination stride

Transfer Type	Description
Repeat Patterns	Configurable repeat count for repetitive operations
Scatter-Gather	Non-contiguous memory regions with hardware descriptor parsing

Command Processing

The DMA frontend streamlines command processing by parsing hardware descriptors directly, freeing software from interpretation overhead. It automatically calculates addresses for multi-dimensional transfers and optimizes burst lengths for efficient memory usage.

FIFO buffering between the frontend and midend ensures pipelined operation, while back-pressure handling absorbs system overloads without losing commands.

DMA Request Manager

The request manager (`idma_request_manager_wrapper`) sits between the frontend and backend, providing flexible request routing when multiple control interfaces or master interfaces are configured. In the simplest case (single control interface and single master interface), the module operates as a passthrough with no arbitration overhead.

Request Arbitration

When multiple control or master interfaces are present, the request manager employs round-robin arbitration to fairly distribute requests. Each master interface has its own round-robin arbiter that selects among pending control interface requests. To prevent duplicate execution, once a master accepts a request, that request is masked from other masters until completion.

Request Tracking

The manager maintains a tracking FIFO (depth 4) for each master interface to record which control interface originated each request. When a request is accepted by a master, the control interface ID is pushed into the tracking FIFO. When the master completes the transfer and returns a response, the ID is popped to route the response back to the correct control interface.

Response Routing

Responses flow from masters back through tracking FIFOs to identify their originating control interfaces. A single-entry response buffer decouples timing dependencies between master and control interfaces. When multiple masters attempt to respond to the same control interface simultaneously, a second level of round-robin arbitration ensures fair and deadlock-free response delivery.

5.8.4. DMA Backend

The backend (`idma_backend_wrapper`) implements the data movement engine based on iDMA.

The table below describes the AXI master interface.

Table 346. AXI4 Master Characteristics

Parameter	Specification
Address Width	56-bit addressing for large system integration
Data Width	64-bit data path

Parameter	Specification
ID Width	4-bit external (3-bit internal to iDMA backend)
Outstanding Transactions	Configurable maximum (default 16 via DMA_MST_MAX_TXNS)
Buffer Depth	Internal re-order buffer depth of 3
R/AW Coupling	Optional read/write address coupling (default enabled)
Burst Support	Full AXI burst support with automatic length optimization
Address Alignment	Intelligent alignment handling for optimal memory access

5.8.5. DMA Configuration and Control

Transfer Configuration

The table below describes the transfer configuration parameters of the DMA backend.

Table 347. Transfer Parameters

Parameter	Description
Source Address	64-bit source address with byte-level granularity
Destination Address	64-bit destination address with full range support
Transfer Size	64-bit transfer size supporting large transfers
Alignment Handling	Automatic handling of unaligned transfers
Source Stride	64-bit source stride for 2D transfer patterns
Destination Stride	64-bit destination stride configuration
Repeat Count	Configurable repeat count for pattern transfers

Status and Control

The table below describes the status and control features of the DMA backend.

Table 348. Status Monitoring and Control

Feature	Description
Busy Status	Real-time busy indication per channel
Completion Tracking	Transfer completion ID tracking
Error Status	Error reporting and classification
Progress Monitoring	Real-time transfer progress indication
Transfer Initiation	Atomic transfer start with ID allocation
Abort Capability	Safe transfer abort with state preservation
Configuration Lock	Protection against inadvertent configuration changes
Interrupt Control	Configurable interrupt generation and masking

Performance Optimization and Power Management

The two tables below describe the iDMA backend’s performance optimization and power management features.

Table 349. Performance Features

Feature	Description
Automatic Burst Sizing	iDMA backend calculates optimal burst lengths
Page Boundary Handling	Automatic burst fragmentation at page boundaries
Outstanding Transactions	Up to 16 concurrent AXI transactions (configurable via DMA_MST_MAX_TXNS)
Pipeline Efficiency	Overlapped command processing and data movement
Request Manager	Round-robin arbitration distributes requests across multiple master interfaces
FIFO Buffering	Frontend-to-midend FIFO (F2M_FIFO_DEPTH=4), optional midend-to-backend FIFO (M2B_FIFO_DEPTH)
R/AW Coupling	Optional read/write address coupling for memory controller optimization

Table 350. Clock Gating Configuration

Feature	Description
Clock Gating Implementation	Single prim_clk_gater_hysteresis gates clock to frontend, request manager, and backend
Hysteresis Width	6-bit configurable hysteresis (CG_HYSTERESIS_W=6)
Activity Detection	Clock enabled when frontend wakeup or backend busy
Gating Control	cg_enable_i input enables/disables clock gating
Test Mode	Clock gating bypassed during test mode

5.8.6. System Integration

The DMA wrapper (idma_wrapper) integrates the frontend, backend, request manager, and clock gating into a complete DMA subsystem. The architecture orchestrates a clear data flow from software configuration through hardware execution.

Request Path

Transfer requests enter through AXI4-Lite control interfaces, where the frontend receives configuration and commands from software. The frontend processes these N-dimensional transfer descriptors, converting complex 2D patterns into linear iDMA requests. The request manager sits between the frontend and backend, using round-robin arbitration to distribute requests from M control interfaces across N master interfaces. Finally, independent iDMA backends execute the transfers on separate AXI4 master interfaces, enabling parallel data movement.

Response Path

As backends complete transfers, responses flow back through the request manager to the appropriate frontend, which updates status registers and generates interrupts. This completes the transfer loop and notifies software of completion.

Type Definitions

The wrapper configures iDMA for 2D transfer support with 56-bit transfer length and stride widths (matching the address width) and 32-bit repetition counts. This enables efficient handling of both linear and 2D data movement patterns.

5.8.7. Detailed Register Map

Table 351. Register Map

Address	Name	Access	Description
0x0	CONFIG	RW	-

Address	Name	Access	Description
0x4	STATUS_0	R	-
0x8	STATUS_1	R	-
0xC	STATUS_2	R	-
0x10	STATUS_3	R	-
0x14	STATUS_4	R	-
0x18	STATUS_5	R	-
0x1C	STATUS_6	R	-
0x20	STATUS_7	R	-
0x24	STATUS_8	R	-
0x28	STATUS_9	R	-
0x2C	STATUS_10	R	-
0x30	STATUS_11	R	-
0x34	STATUS_12	R	-
0x38	STATUS_13	R	-
0x3C	STATUS_14	R	-
0x40	STATUS_15	R	-
0x48	NEXT_ID_0	R	-
0x50	NEXT_ID_1	R	-
0x58	NEXT_ID_2	R	-
0x60	NEXT_ID_3	R	-
0x68	NEXT_ID_4	R	-
0x70	NEXT_ID_5	R	-
0x78	NEXT_ID_6	R	-
0x80	NEXT_ID_7	R	-
0x88	NEXT_ID_8	R	-
0x90	NEXT_ID_9	R	-
0x98	NEXT_ID_10	R	-
0xA0	NEXT_ID_11	R	-
0xA8	NEXT_ID_12	R	-
0xB0	NEXT_ID_13	R	-
0xB8	NEXT_ID_14	R	-
0xC0	NEXT_ID_15	R	-
0xC8	DONE_0	R	-
0xCC	DONE_1	R	-
0xD0	DONE_2	R	-
0xD4	DONE_3	R	-
0xD8	DONE_4	R	-
0xDC	DONE_5	R	-
0xE0	DONE_6	R	-
0xE4	DONE_7	R	-
0xE8	DONE_8	R	-

Address	Name	Access	Description
0xEC	DONE_9	R	-
0xF0	DONE_10	R	-
0xF4	DONE_11	R	-
0xF8	DONE_12	R	-
0xFC	DONE_13	R	-
0x100	DONE_14	R	-
0x104	DONE_15	R	-
0x108	DST_ADDRESS_LO	RW	-
0x10C	DST_ADDRESS_HI	RW	-
0x110	SRC_ADDRESS_LO	RW	-
0x114	SRC_ADDRESS_HI	RW	-
0x118	LENGTH_LO	RW	-
0x11C	LENGTH_HI	RW	-
0x120	DST_STRIDE_LO	RW	-
0x124	DST_STRIDE_HI	RW	-
0x128	SRC_STRIDE_LO	RW	-
0x12C	SRC_STRIDE_HI	RW	-
0x130	NUM_REPETITIONS_LO	RW	-
0x134	NUM_REPETITIONS_HI	RW	-

Table 352. CONFIG Register

Bits	Field	Access	Reset	Description
31:11	Reserved	-	-	Reserved
10	ENABLED_ND	RW	0x0	Not used
9:7	DST_MAX_LLEN	RW	0x0	Not used
6:4	SRC_MAX_LLEN	RW	0x0	Not used
3	DST_REDUCE_LEN	RW	0x0	Not used
2	SRC_REDUCE_LEN	RW	0x0	Not used
1	DECOUPLE_RW	RW	0x0	Not used
0	DECOUPLE_AW	RW	0x0	Not used

Table 353. STATUS_0 Register

Bits	Field	Access	Reset	Description
31:10	Reserved	-	-	Reserved
9:0	BUSY	R	0x0	DMA busy bits. Reading 0 means the DMA has written the last data beat but not necessarily seen the last response

Table 354. STATUS_1 Register

Bits	Field	Access	Reset	Description
31:10	Reserved	-	-	Reserved
9:0	BUSY	R	0x0	DMA busy bits. Reading 0 means the DMA has written the last data beat but not necessarily seen the last response

Table 355. STATUS_2 Register

Bits	Field	Access	Reset	Description
31:10	Reserved	-	-	Reserved
9:0	BUSY	R	0x0	DMA busy bits. Reading 0 means the DMA has written the last data beat but not necessarily seen the last response

Table 356. STATUS_3 Register

Bits	Field	Access	Reset	Description
31:10	Reserved	-	-	Reserved
9:0	BUSY	R	0x0	DMA busy bits. Reading 0 means the DMA has written the last data beat but not necessarily seen the last response

Table 357. STATUS_4 Register

Bits	Field	Access	Reset	Description
31:10	Reserved	-	-	Reserved
9:0	BUSY	R	0x0	DMA busy bits. Reading 0 means the DMA has written the last data beat but not necessarily seen the last response

Table 358. STATUS_5 Register

Bits	Field	Access	Reset	Description
31:10	Reserved	-	-	Reserved
9:0	BUSY	R	0x0	DMA busy bits. Reading 0 means the DMA has written the last data beat but not necessarily seen the last response

Table 359. STATUS_6 Register

Bits	Field	Access	Reset	Description
31:10	Reserved	-	-	Reserved
9:0	BUSY	R	0x0	DMA busy bits. Reading 0 means the DMA has written the last data beat but not necessarily seen the last response

Table 360. STATUS_7 Register

Bits	Field	Access	Reset	Description
31:10	Reserved	-	-	Reserved
9:0	BUSY	R	0x0	DMA busy bits. Reading 0 means the DMA has written the last data beat but not necessarily seen the last response

Table 361. STATUS_8 Register

Bits	Field	Access	Reset	Description
31:10	Reserved	-	-	Reserved
9:0	BUSY	R	0x0	DMA busy bits. Reading 0 means the DMA has written the last data beat but not necessarily seen the last response

Table 362. STATUS_9 Register

Bits	Field	Access	Reset	Description
31:10	Reserved	-	-	Reserved

Bits	Field	Access	Reset	Description
9:0	BUSY	R	0x0	DMA busy bits. Reading 0 means the DMA has written the last data beat but not necessarily seen the last response

Table 363. STATUS_10 Register

Bits	Field	Access	Reset	Description
31:10	Reserved	-	-	Reserved
9:0	BUSY	R	0x0	DMA busy bits. Reading 0 means the DMA has written the last data beat but not necessarily seen the last response

Table 364. STATUS_11 Register

Bits	Field	Access	Reset	Description
31:10	Reserved	-	-	Reserved
9:0	BUSY	R	0x0	DMA busy bits. Reading 0 means the DMA has written the last data beat but not necessarily seen the last response

Table 365. STATUS_12 Register

Bits	Field	Access	Reset	Description
31:10	Reserved	-	-	Reserved
9:0	BUSY	R	0x0	DMA busy bits. Reading 0 means the DMA has written the last data beat but not necessarily seen the last response

Table 366. STATUS_13 Register

Bits	Field	Access	Reset	Description
31:10	Reserved	-	-	Reserved
9:0	BUSY	R	0x0	DMA busy bits. Reading 0 means the DMA has written the last data beat but not necessarily seen the last response

Table 367. STATUS_14 Register

Bits	Field	Access	Reset	Description
31:10	Reserved	-	-	Reserved
9:0	BUSY	R	0x0	DMA busy bits. Reading 0 means the DMA has written the last data beat but not necessarily seen the last response

Table 368. STATUS_15 Register

Bits	Field	Access	Reset	Description
31:10	Reserved	-	-	Reserved
9:0	BUSY	R	0x0	DMA busy bits. Reading 0 means the DMA has written the last data beat but not necessarily seen the last response

Table 369. NEXT_ID_0 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Reading this register starts the DMA transfer. Returns an ID value for the cumulative number of transfers. Returns 0 if command was not set up correctly

Table 370. NEXT_ID_1 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Reading this register starts the DMA transfer. Returns an ID value for the cumulative number of transfers. Returns 0 if command was not set up correctly

Table 371. NEXT_ID_2 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Reading this register starts the DMA transfer. Returns an ID value for the cumulative number of transfers. Returns 0 if command was not set up correctly

Table 372. NEXT_ID_3 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Reading this register starts the DMA transfer. Returns an ID value for the cumulative number of transfers. Returns 0 if command was not set up correctly

Table 373. NEXT_ID_4 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Reading this register starts the DMA transfer. Returns an ID value for the cumulative number of transfers. Returns 0 if command was not set up correctly

Table 374. NEXT_ID_5 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Reading this register starts the DMA transfer. Returns an ID value for the cumulative number of transfers. Returns 0 if command was not set up correctly

Table 375. NEXT_ID_6 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Reading this register starts the DMA transfer. Returns an ID value for the cumulative number of transfers. Returns 0 if command was not set up correctly

Table 376. NEXT_ID_7 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Reading this register starts the DMA transfer. Returns an ID value for the cumulative number of transfers. Returns 0 if command was not set up correctly

Table 377. NEXT_ID_8 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Reading this register starts the DMA transfer. Returns an ID value for the cumulative number of transfers. Returns 0 if command was not set up correctly

Table 378. NEXT_ID_9 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Reading this register starts the DMA transfer. Returns an ID value for the cumulative number of transfers. Returns 0 if command was not set up correctly

Table 379. NEXT_ID_10 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Reading this register starts the DMA transfer. Returns an ID value for the cumulative number of transfers. Returns 0 if command was not set up correctly

Table 380. NEXT_ID_11 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Reading this register starts the DMA transfer. Returns an ID value for the cumulative number of transfers. Returns 0 if command was not set up correctly

Table 381. NEXT_ID_12 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Reading this register starts the DMA transfer. Returns an ID value for the cumulative number of transfers. Returns 0 if command was not set up correctly

Table 382. NEXT_ID_13 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Reading this register starts the DMA transfer. Returns an ID value for the cumulative number of transfers. Returns 0 if command was not set up correctly

Table 383. NEXT_ID_14 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Reading this register starts the DMA transfer. Returns an ID value for the cumulative number of transfers. Returns 0 if command was not set up correctly

Table 384. NEXT_ID_15 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Reading this register starts the DMA transfer. Returns an ID value for the cumulative number of transfers. Returns 0 if command was not set up correctly

Table 385. DONE_0 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Holds the cumulative number of completed transfers. Only accumulates when response is seen

Table 386. DONE_1 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Holds the cumulative number of completed transfers. Only accumulates when response is seen

Table 387. DONE_2 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Holds the cumulative number of completed transfers. Only accumulates when response is seen

Table 388. DONE_3 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Holds the cumulative number of completed transfers. Only accumulates when response is seen

Table 389. DONE_4 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Holds the cumulative number of completed transfers. Only accumulates when response is seen

Table 390. DONE_5 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Holds the cumulative number of completed transfers. Only accumulates when response is seen

Table 391. DONE_6 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Holds the cumulative number of completed transfers. Only accumulates when response is seen

Table 392. DONE_7 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Holds the cumulative number of completed transfers. Only accumulates when response is seen

Table 393. DONE_8 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Holds the cumulative number of completed transfers. Only accumulates when response is seen

Table 394. DONE_9 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Holds the cumulative number of completed transfers. Only accumulates when response is seen

Table 395. DONE_10 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Holds the cumulative number of completed transfers. Only accumulates when response is seen

Table 396. DONE_11 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Holds the cumulative number of completed transfers. Only accumulates when response is seen

Table 397. DONE_12 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Holds the cumulative number of completed transfers. Only accumulates when response is seen

Table 398. DONE_13 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Holds the cumulative number of completed transfers. Only accumulates when response is seen

Table 399. DONE_14 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Holds the cumulative number of completed transfers. Only accumulates when response is seen

Table 400. DONE_15 Register

Bits	Field	Access	Reset	Description
31:0	DATA	R	0x0	Holds the cumulative number of completed transfers. Only accumulates when response is seen

Table 401. DST_ADDRESS_LO Register

Bits	Field	Access	Reset	Description
31:0	DATA	RW	0x0	Destination address for DMA to write to, lower 32 bits

Table 402. DST_ADDRESS_HI Register

Bits	Field	Access	Reset	Description
31:0	DATA	RW	0x0	Destination address for DMA to write to, upper 32 bits

Table 403. SRC_ADDRESS_LO Register

Bits	Field	Access	Reset	Description
31:0	DATA	RW	0x0	Source address for DMA to read from, lower 32 bits.

Table 404. SRC_ADDRESS_HI Register

Bits	Field	Access	Reset	Description
31:0	DATA	RW	0x0	Source address for DMA to read from, upper 32 bits.

Table 405. LENGTH_LO Register

Bits	Field	Access	Reset	Description
31:0	DATA	RW	0x0	Number of bytes to move, lower 32 bits.

Table 406. LENGTH_HI Register

Bits	Field	Access	Reset	Description
31:0	DATA	RW	0x0	Number of bytes to move, upper 32 bits.

Table 407. DST_STRIDE_LO Register

Bits	Field	Access	Reset	Description
31:0	DATA	RW	0x0	Stride amount on destination address side, lower 32 bits

Table 408. DST_STRIDE_HI Register

Bits	Field	Access	Reset	Description
31:0	DATA	RW	0x0	Stride amount on destination address side, upper 32 bits

Table 409. SRC_STRIDE_LO Register

Bits	Field	Access	Reset	Description
31:0	DATA	RW	0x0	Stride amount on source address side, lower 32 bits

Table 410. SRC_STRIDE_HI Register

Bits	Field	Access	Reset	Description
31:0	DATA	RW	0x0	Stride amount on source address side, upper 32 bits

Table 411. NUM_REPETITIONS_LO Register

Bits	Field	Access	Reset	Description
31:0	DATA	RW	0x0	Number of times transfer should be repeated, lower 32 bits. If stride is set, stride will be added to address after every repetition

Table 412. NUM_REPETITIONS_HI Register

Bits	Field	Access	Reset	Description
31:0	DATA	RW	0x0	Number of times transfer should be repeated, upper 32 bits. If stride is set, stride will be added to address after every repetition

5.9. Zeroer

The SMC Memory Zeroer efficiently initializes large memory regions to zero without CPU intervention. The unit uses a state machine-based architecture with dual clock domains.

Table 413. Memory Zeroer Integration

Parameter	Description
Address Range	0xC003_8200 - 0xC003_83FF (512 bytes)
Control Interface	AXI4-Lite
Data Interface	AXI4 Master (56-bit address, 64-bit data, 4-bit ID)

Parameter	Description
Maximum Burst Length	255 beats (AXI maximum)
Outstanding Transactions	Up to 32 concurrent AXI transactions

5.9.1. Use Cases

- **System Initialization:** Clear memory regions during boot or reset sequences
- **Security Operations:** Sanitize sensitive data areas
- **Memory Management:** Initialize allocated memory blocks
- **Error Recovery:** Clear corrupted memory regions

5.9.2. Architecture

State Machine

The zeroer implements a three-state operation model:

Table 414. State Machine States

State	Description
ST_IDLE (3'b001)	Idle state waiting for configuration and trigger
ST_ISSUE_ADDR (3'b010)	Address phase - issuing AXI write addresses
ST_ISSUE_DATA (3'b100)	Data phase - streaming zero data for all beats

Operation Flow

1. **Configuration:** Software writes destination address and size to registers
2. **Trigger:** Write to control/status register initiates operation
3. **Address Phase:** Hardware calculates optimal burst patterns and issues write addresses
4. **Data Phase:** Hardware streams zero data matching issued address beats
5. **Completion:** State machine returns to idle with status update and optional interrupt

AXI Master Interface

Table 415. AXI4 Master Characteristics

Parameter	Specification
Address Width	56-bit addressing for large system integration
Data Width	64-bit data path
ID Width	4-bit transaction ID (internal 3-bit usage)
User Signals	12-bit user field for system-specific attributes
Maximum Burst Length	255 beats for optimal memory efficiency
Burst Calculation	Automatic optimal burst sizing
Address Alignment	Intelligent handling of unaligned start addresses
Boundary Handling	Automatic burst fragmentation at page boundaries

Outstanding Transaction Management

The zeroer efficiently manages outstanding AXI transactions using a 32-bit counter to track in-flight writes. When the number of pending transactions reaches the allowed maximum, built-in flow control applies back-pressure until resources become available. Write responses are tracked to ensure accurate completion status, while AXI error responses are processed to handle faults robustly.

Multiple write addresses can be pipelined to maximize throughput, and the zero data stream operates independently of response timing, allowing continuous operation. By supporting multiple bursts in flight, the zeroer achieves high parallelism and optimal memory bandwidth utilization.

Dual Clock Domain Architecture

The zeroer implements dual clock domain operation for power management and interface isolation.

Clock Domains

Table 416. Clock Domain Characteristics

Clock Domain	Description
AXI Clock (axi_clk)	Dedicated clock for AXI master operations, enabled only during active zeroing
Register Clock (reg_clk)	Separate clock for register interface, independent of AXI activity

Clock Gating

Table 417. Clock Gating Control

Domain	Gating Logic
AXI Clock	axi_clk_enable = disable_cg zeroer_busy_o ~rst_ni
Register Clock	reg_clk_enable = disable_cg register_activity ~rst_ni
Implementation	prim_clkgater for glitch-free operation
Test Support	Test enable override for manufacturing test
Reset Override	Ensures proper reset domain operation

Control and Status

Operation Control

Table 418. Configuration and Control

Register	Description
DEST_ADDR	64-bit byte address for destination memory region
SIZE	64-bit byte count for zeroing operation
CTRL_STATUS	Control and status register (trigger, busy status)
Interrupt Enable	Optional completion interrupt enable
Validation	Hardware validates configuration parameters

Status Features

- **Busy Status:** Real-time indication of zeroing operation in progress

- **Completion Status:** Updates when all write responses received
- **Error Status:** AXI error response detection and reporting
- **Interrupt Generation:** Optional interrupt on operation completion

5.9.3. Operation Details

Table 419. Data Generation and Control

Feature	Description
Zero Data Generation	Hardware generates zero data for all beats
Strobe Calculation	Correct byte strobes for partial transfers and unaligned addresses
Last Beat Handling	Proper w_last assertion for burst completion
Size Tracking	Accurate remaining size calculation throughout operation
Overflow Protection	Size overflow detection and handling
Safe Abort	Ability to safely abort operation if needed

5.9.4. Detailed Register Map

Table 420. Register Map

Address	Name	Access	Description
0x0	DEST_ADDR	RW	-
0x8	SIZE	RW	-
0x10	CTRL_STATUS	RW	Writing this register will trigger start of zeroer

Table 421. DEST_ADDR Register

Bits	Field	Access	Reset	Description
63:0	DEST_ADDR	RW	0x0	Byte address to write zeros to

Table 422. SIZE Register

Bits	Field	Access	Reset	Description
63:0	SIZE	RW	0x0	Size in bytes of zeros to write

Table 423. CTRL_STATUS Register

Bits	Field	Access	Reset	Description
63:33	Reserved	-	-	Reserved
32	STATUS	R	0x0	Returns status of whether zeroer has completed.
31:1	Reserved	-	-	Reserved
0	INT_EN	RW	0x0	If enabled, an interrupt will be outputted once zeroer completes.

5.10. Peripherals

This section documents the main peripherals integrated within the System Management Controller (SMC). These peripherals provide essential I/O and communication capabilities for the OCH system, supporting various protocols and interfaces required for system operation and external connectivity.

The following table summarizes the key peripherals, their communication protocols, and default instance counts:

Table 424. SMC Peripheral Summary

Peripheral	Description	Protocol	Instances
Mailbox	FIFO-based bidirectional communication channels for inter-processor or inter-chiplet communication.	AXI4-Lite	32
System Timer OCTS	64-bit timer for multi-chiplet time synchronization with PRIMARY/SECONDARY modes.	APB4/AXI4-Lite	1
eFuse	Secure access to One-Time Programmable (OTP) memory with layered security architecture.	APB4/AXI4-Lite	1
UART 16550	Industry-standard UART with enhanced FIFO capabilities and comprehensive interrupt support.	AXI4-Lite	4
Log Engine	DMA-based logging system for efficient transfer of log messages to UART interface.	AXI4-Lite	4
I2C	Full I2C/SMBus/PMBus communication controller supporting both controller and target modes.	AXI4-Lite	3
AVSBus Controller	Adaptive voltage scaling communication controller for managing voltage rails and power conditions.	APB4/AXI4-Lite	1
GPIO	General Purpose Input/Output with dual-mode operation and comprehensive interrupt system (64 bonded + 4 unbonded).	APB4/AXI4-Lite	68
Telemetry Receiver	Hardware component for decoding and processing telemetry data from multi-chiplet systems.	APB4/AXI4-Lite	3
I3C	I3C controller for enhanced serial communication (RESERVED - NOT YET IMPLEMENTED).	AXI4-Lite	6
Reset Unit	Reset control and synchronization, deals with SMC resets (core, primary), subsystem (chiplet) resets, and the cool reset sequence.	AXI4-Lite	1
SMC Misc	Wrapper for miscellaneous registers: scratch registers, chip config and NDM reset.	AXI4-Lite	1

5.10.1. Mailbox Unit

Overview

This document specifies the Mailbox Unit hardware IP functionality. The Mailbox Unit provides FIFO-based bidirectional communication channels for inter-processor or inter-chiplet communication using AXI4-Lite interfaces. It enables efficient data exchange between processing elements through dedicated hardware channels with minimal software overhead.

Description and Features

The Mailbox Unit enables secure, FIFO-based, bidirectional communication between processing elements or chiplets in multicore systems. Each configurable mailbox instance consists of paired inbound and outbound channels, allowing simultaneous data transfer with a 64-bit data path. Multiple independent mailbox pairs are supported, each equipped with dedicated control registers, status monitoring, and programmable interrupt thresholds.

The architecture uses standard AXI4-Lite interfaces for integration and provides robust error detection, reporting, and software-controllable FIFO flushing. Channel depth is adjustable per instance to suit system needs, while hardware minimizes software overhead for efficient, low-latency data exchange.

Architecture

The SMC Mailbox IP provides inter-processor and inter-chiplet communication through a register-based interface. The design separates protocol handling from storage implementation, allowing system integrators to choose appropriate FIFO implementations while maintaining consistent software interfaces.

Configuration Parameters

Parameter	Default	Description
NUM_MAILBOXES	2 (max: 32)	Scales from point-to-point to multi-processor coordination
MAILBOX_DEPTH	8 entries	External FIFO depth for buffering
MAX_TRANS	32	Maximum outstanding AXI4-Lite transactions
MAILBOX_SIZE	0x800 (2KB)	Register space per mailbox for control and status

Address Space

The architecture supports up to 64 mailbox instances (32 bidirectional pairs):

- **Address Width:** 18 bits (0x20008 bytes total)
- **Data Width:** 64 bits
- **Per-Mailbox Space:** 0x800 bytes (2KB)
- **Decoding:** Efficient address decode logic for up to 32 pairs

The 64-bit data width optimizes handling of addresses, data payloads, and status information without multi-word transfers. The 2KB allocation per mailbox enables threshold configuration, priority handling, and diagnostic features.

Hardware Interface

The SystemRDL-generated register block provides hardware interfaces to external FIFO implementations through two signal structures:

Hardware Input Interface (mailbox_wrap_in_t)

Signal	Description
rd_ack	Read acknowledge for handshake protocol
wr_ack	Write acknowledge for handshake protocol
rd_data[63:0]	64-bit read data from external FIFO

The acknowledge signals implement handshake protocol for reliable data transfer regardless of external FIFO timing. The 64-bit read data interface eliminates width conversion overhead.

Hardware Output Interface (mailbox_wrap_out_t)

Signal	Description
req	Transaction request to external FIFO
addr[6:0]	Local address within mailbox space
req_is_wr	Transaction type (1=write, 0=read)
wr_data[63:0]	64-bit write data to external FIFO

Signal	Description
<code>wr_biten[63:0]</code>	Byte enable for write data

The 7-bit local address enables multiple registers per mailbox. Byte enables provide granular write control for external FIFO optimization.

Transaction Processing

AXI4-Lite transactions follow this flow:

1. **Address Decode:** Register block decodes mailbox instance and operation type
2. **Hardware Interface:** Generates `req`, `addr`, `req_is_wr` signals to external FIFO
3. **FIFO Processing:** External logic processes request according to timing characteristics
4. **Acknowledge:** External FIFO asserts `rd_ack/wr_ack` when complete
5. **AXI Response:** Register block completes AXI4-Lite protocol

This coordination mechanism maintains consistent AXI4-Lite timing while allowing external FIFOs to optimize internal operations for specific performance or resource requirements.

Bidirectional Communication

Each mailbox pair provides separate inbound and outbound register spaces:

- **Inbound Space:** Optimized for low-latency command processing
- **Outbound Space:** Optimized for high-throughput status/telemetry transfer
- **Independence:** No mutual interference or resource contention

Separate register spaces enable different optimization strategies per direction. Interrupt generation integrates with external FIFO status reporting for event notification.

Clock and Reset

The mailbox uses a single clock domain for integration reliability:

Signal	Type	Description
<code>clk_i</code>	Single clock	Unifies functional logic and AXI4-Lite interface
<code>rst_ni</code>	Asynchronous, active-low	Primary reset, initializes all mailbox state
<code>test_en_i</code>	Test enable	DFT support

No internal clock domain crossings eliminates CDC complexity. Single clock domain simplifies timing analysis and integration verification.

Interrupt Architecture

The mailbox provides independent interrupt outputs for each mailbox instance:

- `inbound_interrupt_o[NUM_MAILBOXES-1:0]`: Inbound communication events
- `outbound_interrupt_o[NUM_MAILBOXES-1:0]`: Outbound communication events

Separate inbound/outbound interrupts reflect different processing requirements. Inbound interrupts indicate incoming commands requiring immediate processing; outbound interrupts indicate successful transmission or buffer availability.

The following table reports the interrupt sources.

Source	Description
Write Threshold	FIFO fill level exceeds configured threshold (proactive buffer management)
Read Threshold	FIFO data available for efficient retrieval scheduling
Overflow Error	Write to full FIFO (fault detection)
Underflow Error	Read from empty FIFO (fault detection)

Each interrupt output aggregates these sources. Threshold-based interrupts reduce polling overhead. Error interrupts enable fault recovery and diagnostic analysis.

Interface

The Mailbox Unit provides both struct-based and flattened AXI4-Lite interfaces (selectable via generation template).

AXI4-Lite Interface

```
input mailbox_axil_req_t mailbox_axi_req_i,  
output mailbox_axil_resp_t mailbox_axi_resp_o
```

Flattened Interface

```
// Write Address Channel  
input logic          awvalid_i,  
output logic         awready_o,  
input  addr_t        awaddr_i,  
input  axi_pkg::prot_t awprot_i,  
  
// Write Data Channel  
input logic          wvalid_i,  
output logic         wready_o,  
input  data_t        wdata_i,  
input  strb_t         wstrb_i,  
  
// Write Response Channel  
output logic         bvalid_o,  
input  logic         bready_i,  
output axi_pkg::resp_t bresp_o,  
  
// Read Address Channel  
input logic          arvalid_i,  
output logic         arready_o,  
input  addr_t        araddr_i,  
input  axi_pkg::prot_t arprot_i,
```

```
// Read Data Channel
output logic          rvalid_o,
input  logic          rready_i,
output data_t         rdata_o,
output axi_pkg::resp_t rresp_o
```

Table 425. Interface Characteristics

Parameter	Value
Address Width	18 bits (s_axil_araddr[17:0])
Data Width	64 bits
Strobe Width	8 bits
Total Address Space	0x20008 bytes (up to 32 mailbox pairs)
Protocol	Standard AXI4-Lite with separate read/write channels

Interrupt Outputs

```
output logic [NUM_MAILBOXES-1:0] inbound_interrupt_o,
output logic [NUM_MAILBOXES-1:0] outbound_interrupt_o
```

Table 426. Interrupt Characteristics

Parameter	Value
Type	Configurable (default: level-triggered)
Polarity	Active high
Sources	Write threshold, read threshold, overflow, underflow

Control Signals

```
input logic clk_i,      // System clock
input logic rst_ni,     // Active-low asynchronous reset
input logic test_en_i   // Test enable for DFT
```

Table 427. Control Signal Descriptions

Signal	Description
clk_i	Primary system clock for all mailbox logic and AXI interface
rst_ni	Asynchronous reset (active low), resets all mailbox state
test_en_i	Test enable for design-for-test (DFT) functionality

Timing Characteristics

Parameter	Value
Latency	Single-cycle FIFO access (when not full/empty)

Parameter	Value
Throughput	One 64-bit transfer per clock cycle
Pipeline Depth	Configurable via MAX_TRANS (default: 32)

Error Handling

Error Condition	Behavior
Read from empty FIFO	Sets <code>read_error</code> flag, returns previous data
Write to full FIFO	Sets <code>write_error</code> flag, discards new data
Interrupt Generation	Triggered if enabled via IRQEN register

Integration Requirements

Requirement	Description
Address Space	4KB per mailbox pair; Total = 4KB × NUM_MAILBOXES × 2
Software	Poll STATUS or use interrupts for data availability; configure WIRQT/RIRQT thresholds; check ERROR register
Hardware	Connect interrupts to system controller; proper reset sequencing; single clock domain (no CDC required)

Memory Map

Address Space Organization

Each mailbox instance occupies a dedicated 2KB (0x800) address space. The total address space is 0x20008 bytes supporting up to 64 mailbox instances (32 pairs):

```
Base + 0x00000 - 0x007FF: Outbound Mailbox 0
Base + 0x00800 - 0x00FFF: Inbound Mailbox 0
Base + 0x01000 - 0x017FF: Outbound Mailbox 1
Base + 0x01800 - 0x01FFF: Inbound Mailbox 1
Base + 0x02000 - 0x027FF: Outbound Mailbox 2
Base + 0x02800 - 0x02FFF: Inbound Mailbox 2
...
Base + 0x1F000 - 0x1F7FF: Outbound Mailbox 31
Base + 0x1F800 - 0x1FFFF: Inbound Mailbox 31
Base + 0x20000 - 0x20007: MAILBOX_RSVD (Reserved space)
```

Address Space Characteristics:

- **Per-Mailbox Size:** 2KB (0x800)
- **Total Address Space:** 0x20008 bytes
- **Address Width:** 18 bits
- **Maximum Mailboxes:** 32 pairs (64 instances)

Per-Mailbox Register Map

Within each 4KB mailbox address space, registers are organized as follows:

Table 428. Register Address Map

Offset	Register Name	Width	Access	Description
0x00	WRITE_DATA	64-bit	WO	Write data to mailbox FIFO
0x08	READ_DATA	64-bit	RO	Read data from mailbox FIFO
0x10	STATUS	64-bit	RO	FIFO status (empty, full, threshold)
0x18	ERROR_FLAGS	64-bit	RO	Error flags (write/read errors)
0x20	WIRQT	64-bit	RW	Write interrupt threshold
0x28	RIRQT	64-bit	RW	Read interrupt threshold
0x30	IRQS	64-bit	RW	Interrupt status (write-1-to-clear)
0x38	IRQEN	64-bit	RW	Interrupt enable control
0x40	IRQP	64-bit	RO	Interrupt pending status
0x48	CTRL	64-bit	WO	Control register (FIFO flush)

Address Examples

For a system with 2 mailbox pairs (NUM_MAILBOXES = 2) at base address 0x10000000:

Table 429. Address Map Example (2 Mailbox Pairs)

Mailbox Pair	Address Range	Description
0	0x10000000 - 0x100007FF	Outbound Mailbox 0
0	0x10000800 - 0x10000FFF	Inbound Mailbox 0
1	0x10001000 - 0x100017FF	Outbound Mailbox 1
1	0x10001800 - 0x10001FFF	Inbound Mailbox 1

Specific Register Examples:

Table 430. Register Address Example (Mailbox Pair 0)

Address	Register Name	Mailbox Direction
0x10000000	WRITE_DATA	Outbound
0x10000008	READ_DATA	Outbound
0x10000010	STATUS	Outbound
0x10000018	ERROR_FLAGS	Outbound
...	...	Outbound
0x10000800	WRITE_DATA	Inbound
0x10000808	READ_DATA	Inbound
0x10000810	STATUS	Inbound
0x10000818	ERROR_FLAGS	Inbound
...	...	Inbound

Detailed Register Map

Table 431. Register Map

Address	Name	Access	Description
0x0	WRITE_DATA	W	-
0x8	READ_DATA	R	-
0x10	STATUS	R	-
0x18	ERROR_FLAGS	R	-
0x20	WIRQT	RW	-
0x28	RIRQT	RW	-
0x30	IRQS	RW	This register is used to read and clear interrupt requests, regardless of IRQEN register and whether the interrupt is enabled or disabled.
0x38	IRQEN	RW	This register is used to enable and disable which interrupts are sent to the CPU.
0x40	IRQP	R	This register is used to read the current status of the interrupts sent to the CPU.
0x48	CTRL	W	-

Table 432. WRITE_DATA Register

Bits	Field	Access	Reset	Description
63:0	write_data	W	0x0	Write data to mailbox fifo

Table 433. READ_DATA Register

Bits	Field	Access	Reset	Description
63:0	read_data	R	0x0	Read data from mailbox fifo

Table 434. STATUS Register

Bits	Field	Access	Reset	Description
63:4	Reserved	-	-	Reserved
3	read_level_above_threshold	R	0x0	If set, read fifo level is higher than threshold set in RIRQT
2	write_level_above_threshold	R	0x0	If set, write fifo level is higher than threshold set in WIRQT
1	full	R	0x0	0: Space is available to write, 1: Space is not available to write
0	empty	R	0x0	0: Data is available to read, 1: Data is not available to read

Table 435. ERROR_FLAGS Register

Bits	Field	Access	Reset	Description
63:2	Reserved	-	-	Reserved
1	write_error	R	0x0	Attempted write to a full mailbox
0	read_error	R	0x0	Attempted read from an empty mailbox

Table 436. WIRQT Register

Bits	Field	Access	Reset	Description
63:8	Reserved	-	-	Reserved

Bits	Field	Access	Reset	Description
7:0	wirqt	RW	0x0	When the usage pointer of the FIFO connected to the W channel exceeds this value, a write threshold IRQ is triggered and the corresponding STATUS register bit is set. When a value larger than or equal to the MailboxDepth parameter is written to this register, it gets reduced to MailboxDepth - 1 to ensure an IRQ is triggered when the write FIFO is full.

Table 437. RIRQT Register

Bits	Field	Access	Reset	Description
63:8	Reserved	-	-	Reserved
7:0	rirqt	RW	0x0	When the fill pointer of the FIFO connected to the R channel exceeds this value, a read threshold IRQ is triggered and the corresponding STATUS register bit is set. When a value larger than or equal to the MailboxDepth parameter is written to this register, it gets reduced to MailboxDepth - 1 to ensure an IRQ is triggered when the read FIFO is full.

Table 438. IRQS Register

Bits	Field	Access	Reset	Description
63:3	Reserved	-	-	Reserved
2	eirq	RW	0x0	On read: [0]: No interrupt request [1]: Error on mailbox access On write: [0]: No acknowledge [1]: Acknowledge and clear interrupt request
1	rtirq	RW	0x0	On read: [0]: No interrupt request [1]: Usage level threshold in read mailbox exceeded On write: [0]: No acknowledge [1]: Acknowledge and clear interrupt request
0	wtirq	RW	0x0	On read: [0]: No interrupt request [1]: Usage level threshold in write mailbox exceeded On write: [0]: No acknowledge [1]: Acknowledge and clear interrupt request

Table 439. IRQEN Register

Bits	Field	Access	Reset	Description
63:3	Reserved	-	-	Reserved
2	eirq	RW	0x0	[0]: Error IRQ disabled [1]: Error IRQ enabled
1	rtirq	RW	0x0	[0]: Read threshold IRQ disabled [1]: Read threshold IRQ enabled
0	wtirq	RW	0x0	[0]: Write threshold IRQ disabled [1]: Write threshold IRQ enabled

Table 440. IRQP Register

Bits	Field	Access	Reset	Description
63:3	Reserved	-	-	Reserved
2	eirq	R	0x0	[0]: No error IRQ pending [1]: Error IRQ pending
1	rtirq	R	0x0	[0]: No read threshold IRQ pending [1]: Read threshold IRQ pending
0	wtirq	R	0x0	[0]: No write threshold IRQ pending [1]: Write threshold IRQ pending

Table 441. CTRL Register

Bits	Field	Access	Reset	Description
63:2	Reserved	-	-	Reserved
1	rflush	W	0x0	Flush the read FIFO for this port
0	wflush	W	0x0	Flush the write FIFO for this port

5.10.2. System Timer OCTS

Overview

The System Timer OCTS (Open Chiplet Time Synchronization) IP is a sophisticated 64-bit timer designed specifically for multi-chiplet systems requiring precise time synchronization across different clock domains. It implements OCA's OCTS protocol as defined in the OCA specification, version .

This hardware IP provides a robust solution for coordinated timing operations between multiple timer instances, enabling accurate timestamping and synchronized operations in distributed chiplet architectures.

Description and Features

The System Timer OCTS offers both PRIMARY and SECONDARY modes to serve as either a timing master or follower within a synchronized network of chiplets. It features a 64-bit high-precision counter for nanosecond-level accuracy, supports both APB4 and AXI4-Lite interfaces for flexible integration, and incorporates reliable clock domain crossing with a 2FF synchronizer.

Periodic resynchronization is achieved through a programmable credit-based mechanism, while the width of synchronization pulses can be easily adjusted. Real-time status and timer values are always available through readable registers.

Architecture

The System Timer OCTS (Open Chiplet Time Synchronization) IP provides precise time coordination across distributed multi-chiplet systems. The design implements a hierarchical PRIMARY/SECONDARY architecture for nanosecond-level timing precision across chiplets that may operate with different clock domains, processing loads, and environmental conditions.

The PRIMARY/SECONDARY architecture separates centralized time authority from distributed time management. PRIMARY timers generate master timelines while SECONDARY timers maintain local temporal alignment using credit-based counting. This approach maintains system-wide synchronization under varying operational conditions, network latencies, and component performance variations.

The implementation follows Open Chiplet Architecture (OCA) specifications, version , using credit-based synchronization for continuous maintenance that adapts to varying system loads and communication delays.

Timer Core Processing

The timer core implements 64-bit counter management supporting both PRIMARY and SECONDARY operational modes.

In **PRIMARY** mode, the timer acts as the central timekeeper for the system. It maintains a continuously incrementing 64-bit counter, advancing with each clock cycle to provide an accurate global timeline.

As the system authority, the PRIMARY timer generates synchronization pulses, including both sync_load and credit signals, which are distributed to SECONDARY timers to ensure coordinated operation across chiplets. During

initialization, the timer supports preset loading, enabling simultaneous start and consistent alignment throughout the system.

By combining precise counting and periodic synchronization outputs, the PRIMARY timer anchors time coordination for multi-chiplet environments.

SECONDARY timers sustain local time alignment across chiplets using a credit-based scheme. The timer maintains an internal credit accumulator (`cur_credits`) that increments by the step input value (`timer_cnt_step_i`) each clock cycle when credits are available. When `cur_credits` reaches or exceeds the programmed credit value (`CREDIT_VAL`), the timer halts counting until new credit pulses arrive from the PRIMARY timer.

The step input (`timer_cnt_step_i`) is an 8-bit external signal that controls the increment rate per clock cycle, typically set to 1 for standard per-cycle increment but can be tuned for different clock ratios or calibration needs. The system continuously tracks an expected counter value that gets updated on `sync_load` or credit pulse events, validating alignment with the overall timeline. Credit expiration is monitored via a 32-bit counter that increments when `cur_credits` exceeds `CREDIT_VAL`, providing diagnostics for synchronization health.

Clock Domain Crossing

The CDC architecture maintains signal integrity and timing precision across clock domains with different frequencies, phase relationships, and jitter characteristics.

Edge Detection

To ensure robust metastability protection during clock domain crossing, the design employs `prim_edge_detector` modules. These modules reliably convert level-based synchronization signals into single-cycle pulses, minimizing hold time constraints and simplifying timing requirements. As a result, they enable precise temporal coordination across clock domains and maintain stable operation despite potential variations in signal arrival times or delays.

Synchronization Protocol

The synchronization protocol relies on a two-level mechanism to establish and maintain timer alignment across chiplets. During initialization, a global **`sync_load`** pulse resets all timers simultaneously, ensuring they start in a coordinated state.

For ongoing operation, periodic **credit pulses** are issued to secondary timers, enabling them to resynchronize and compensate for any accumulated clock drift or environmental variations such as temperature changes. Edge-triggered signaling is used throughout, removing tight timing dependencies and promoting robust synchronization regardless of clock differences.

Credit-Based Synchronization

Credit-based synchronization balances autonomous operation with centralized coordination, allowing **SECONDARY** timers to maintain temporal progression during communication interruptions while ensuring long-term accuracy through periodic resynchronization.

Credit Management

The credit management system operates differently in PRIMARY and SECONDARY modes:

Feature	Description
PRIMARY Credit Counter	8-bit counter (0 to CREDIT_VAL-1) that increments each cycle when enabled. When it reaches CREDIT_VAL-1, it resets to 0 and generates a credit pulse output.
SECONDARY Credit Accumulator	8-bit accumulator (cur_credits) that increments by step input value each cycle when cur_credits < CREDIT_VAL. Resets to 0 on sync_load or credit pulse events.
Credit Value	Software-programmable via CTRL.CREDIT_VAL register (8-bit, range 1-255)
Step Input	8-bit external signal (timer_cnt_step_i) controlling increment rate in SECONDARY mode
Expiration Tracking	32-bit counter that increments when cur_credits >= CREDIT_VAL in SECONDARY mode, providing diagnostics

Synchronization Logic

In SECONDARY mode, the timer advances when credits are available. The credit accumulator (cur_credits) increments by the step input value (`timer_cnt_step_i`) each clock cycle as long as `cur_credits < CREDIT_VAL`. When `cur_credits` reaches or exceeds `CREDIT_VAL`, the timer halts counting and the `credit_expired` counter increments, indicating that synchronization credits have been exhausted.

Credits are replenished when a credit pulse arrives from the PRIMARY timer or when a `sync_load` pulse is received, both of which reset `cur_credits` to 0 and reset the `credit_expired` counter. The expected counter value is updated on these synchronization events: on `sync_load`, it's set to the preset value; on credit pulse, it's incremented by `CREDIT_VAL` using a carry-select adder. This expected value provides a basis for synchronization validation and helps detect drift between the actual timer count and the expected timeline.

Pulse Generation

PRIMARY timers generate synchronization signals for SECONDARY timer coordination.

Pulse State Machine

State	Description
PULSE_IDLE	Normal operation, monitors for synchronization events
PULSE_SYNC_LOAD	Generates <code>sync_load</code> pulses for coordinated timer initialization
PULSE_CREDIT	Generates credit pulses for ongoing synchronization maintenance

Pulse Width Control

The pulse width for synchronization can be configured between 1 and 255 cycles using the `CTRL.PULSE_WIDTH` register. If `PULSE_WIDTH` is set to 0, it defaults to 1 cycle. A pulse counter tracks the active pulse duration, incrementing from 0 to (`PULSE_WIDTH - 1`) before returning to idle state. This flexibility ensures that synchronization pulses are long enough for reliable clock domain crossing (CDC), while minimizing any overhead to system performance. By allowing adjustment of the pulse duration, the design supports robust operation across different clock ratios and propagation delays.

The credit value (`CREDIT_VAL`) must be greater than the pulse width (`PULSE_WIDTH`) to ensure credit pulses are not missed during pulse generation.

Data Flow and Control Path

In PRIMARY mode, the timer core begins with software issuing a preset value and asserting the `START` register. This action loads the counter with the specified preset (from `{reg_preset_hi, reg_preset_lo}`), establishing a consistent starting point across the system.

Once enabled, the timer increments the counter by one each clock cycle using a 64-bit carry-select adder, maintaining precise progression. Throughout operation, the timer autonomously generates synchronization pulses: periodic credit pulses are issued when the credit counter reaches CREDIT_VAL-1, and sync_load pulses are produced whenever START is asserted. The width of these pulses is tunable via the PULSE_WIDTH setting, allowing adaptation to varying system requirements and ensuring reliable communication between chiplets.

In SECONDARY mode, the timer remains idle until it detects a sync_load pulse from the PRIMARY timer. Upon receiving this pulse, it loads its counter with the preset value to achieve initial alignment. Thereafter, the timer advances based on a credit-based mechanism: it increments the counter by the step input value (`timer_cnt_step_i`) each cycle when `cur_credits < CREDIT_VAL`, using a 64-bit carry-select adder for the addition. The credit accumulator (`cur_credits`) increments by the step value each cycle as long as credits are available.

Credits are replenished automatically when a credit pulse arrives from the PRIMARY or when a sync_load pulse is received, both of which reset `cur_credits` to 0. Throughout operation, the timer continuously tracks its expected count using a separate 64-bit counter that gets updated on synchronization events, providing a basis for synchronization validation. The expected count is incremented by CREDIT_VAL on credit pulse events using a carry-select adder.

Control Path Overview

The control path of the System Timer OCTS integrates subsystems for enabling, mode selection, and credit tracking. The global enable signal is managed through software (via the START register), by detecting sync_load pulses in SECONDARY mode, or by retaining its previous enabled state for sticky operation.

The timer's behavior is determined at runtime by the `is_primary_i` input signal, which defines whether the timer operates as PRIMARY or SECONDARY. In PRIMARY mode, the timer increments its counter by 1 each cycle and generates synchronization pulses. In SECONDARY mode, the timer increments by the step input value and receives synchronization pulses. This runtime mode selection allows flexible system configuration and dynamic mode switching if needed.

Credit management operates at the heart of ongoing time alignment. In PRIMARY mode, an 8-bit credit counter increments from 0 to CREDIT_VAL-1 each cycle when enabled, generating a credit pulse when it reaches CREDIT_VAL-1 and then resetting to 0. In SECONDARY mode, an 8-bit credit accumulator (`cur_credits`) increments by the step input value each cycle when `cur_credits < CREDIT_VAL`, and resets to 0 on sync_load or credit pulse events. The system ensures reliable, edge-triggered credit reception using `prim_edge_detector` modules, updates expected timer values with each credit using carry-select adders, and provides a real-time 32-bit counter (`credit_expired_o`) that increments when `cur_credits >= CREDIT_VAL`—essential for synchronization health diagnostics.

Pulse Generation in PRIMARY Mode

In PRIMARY mode, the timer autonomously produces two kinds of synchronization pulses: sync_load, which resets all timers system-wide (e.g., after START assertion), and periodic credit pulses distributed to SECONDARY timers. Both pulses feature a programmable width (1–255 cycles via the PULSE_WIDTH register, defaulting to 1 if set to 0), ensuring timing integrity across clock domains.

Pulse emission is governed by a dedicated state machine with three states: PULSE_IDLE, PULSE_SYNC_LOAD, and PULSE_CREDIT. A pulse counter tracks the active pulse duration, incrementing from 0 to (PULSE_WIDTH - 1) before the state machine returns to PULSE_IDLE. Sync_load pulses are triggered when START is asserted and the state machine is idle. Credit pulses are triggered when the credit counter reaches CREDIT_VAL-1, the timer is enabled, and the state machine is idle. This mechanism guarantees reliable and flexible pulse delivery, supporting seamless multi-chiplet synchronization even in complex environments.

Interface

The System Timer OCTS IP provides interfaces supporting both PRIMARY and SECONDARY timer modes, with flexible support for either AXI4-Lite or APB4 bus protocols. All interface signals operate synchronously to the system clock.

Configuration Parameters

Parameter	Default	Description
BUS_PROTOCOL	AXI4-Lite	Register interface protocol (AXI4-Lite or APB4)
ADDR_WIDTH	32	Address bus width (8-bit effective)
DATA_WIDTH	32	Data bus width

NOTE

Timer mode selection (PRIMARY vs SECONDARY) is controlled at runtime via the `is_primary_i` input signal, not through a compile-time parameter.

Global Interface

Table 442. Global Interface Signals

Signal Name	Width	Direction	Active	Description
<code>clk_i</code>	1	Input	Rising	System clock. All logic operates synchronously to this clock. Frequency determines timer resolution and synchronization timing.
<code>rst_ni</code>	1	Input	Low	System reset. Asynchronous assertion and synchronous deassertion. Resets all counters, state machines, and registers to initial values.
<code>is_primary_i</code>	1	Input	High	Runtime mode selection: 1=PRIMARY (master timer that generates synchronization pulses), 0=SECONDARY (slave timer that receives synchronization pulses). This signal determines timer behavior, register visibility, and synchronization signal direction.

Test Interface

Table 443. Test Interface Signals

Signal Name	Width	Direction	Active	Description
<code>test_en_i</code>	1	Input	High	Test enable signal for manufacturing test mode and debug access.
<code>scan_rst_ni</code>	1	Input	Low	Scan reset for manufacturing test. Provides independent reset control for scan chain operation and ATPG support.

OCTS Synchronization Interface

Table 444. OCTS Synchronization Interface Signals

Signal Name	Width	Direction	Active	Description
<code>timer_sync_load_i</code>	1	Input	High	Synchronization load pulse input. Triggers coordinated timer initialization from the preset value.
<code>timer_cnt_credit_i</code>	1	Input	High	Credit pulse input. Provides periodic credit replenishment for SECONDARY timer synchronization.
<code>timer_sync_load_o</code>	1	Output	High	Synchronization load pulse output. Generated by the PRIMARY timer for system-wide initialization.
<code>timer_cnt_credit_o</code>	1	Output	High	Credit pulse output. Generated by the PRIMARY timer for maintaining synchronization of SECONDARY timers.

When operating in PRIMARY mode, the timer generates synchronization pulses: `timer_sync_load_o` is asserted when the START register is written, and `timer_cnt_credit_o` is asserted periodically according to internal credit generation logic. Any synchronization input signals are ignored in PRIMARY mode. In contrast, SECONDARY mode relies on input from the PRIMARY: `timer_sync_load_i` directs the timer to initialize using a preset value, and `timer_cnt_credit_i` delivers credit pulses that allow continuous synchronization. In this mode, the outputs remain inactive.

Timer Counter Interface

Table 445. Timer Counter Interface Signals

Signal Name	Width	Direction	Description
<code>timer_cnt_step_i</code>	8	Input	Timer count step value (used in SECONDARY mode) determines the increment per clock cycle.
<code>timer_count_o</code>	64	Output	Real-time 64-bit timer value available to the system.

For SECONDARY timers, the step input (`timer_cnt_step_i`) enables granular control over the increment rate. Typically set to 1 for a standard per-cycle increment, this parameter can be tuned to adjust for differing clock ratios or calibration needs, but is only effective when credits are available (i.e., when the credit counter permits counting).

The timer's count output (`timer_count_o`) provides direct and continuous access to the current counter value, updating every clock cycle while the timer is running. This value is useful for timestamping, monitoring, or debugging, and can also be read from the COUNT_LO and COUNT_HI registers.

Timing Specifications

All input signals are required to meet setup and hold times with respect to the rising edge of `clk_i`.

Table 446. Timing Requirements

Signal Group	Setup Time	Hold Time	Notes
Register Interface	2.0 ns	0.5 ns	AXI4-Lite/APB4 protocol signals
Synchronization Inputs	N/A	N/A	Edge detection provides internal synchronization
Control Inputs	2.0 ns	0.5 ns	<code>test_en_i</code> , <code>scan_rst_ni</code> , <code>timer_cnt_step_i</code>
Timer Count Output	N/A	N/A	Output signal with 3.0 ns clock-to-output delay

The interface ensures that register outputs are valid within 3.0 ns of a rising clock edge, while the timer count output also updates within 3.0 ns of the clock. Synchronization outputs are generated as pulses with programmable widths.

Regarding clock domain crossing, the synchronization protocol relies on pulses wide enough to be reliably detected: each pulse should last for at least two destination clock cycles, and the maximum supported frequency ratio between domains is 10:1.

Pulse events must be separated by at least 10 destination clock cycles. Edge detection logic ensures synchronization latency of 2–3 destination clock cycles and provides robust protection against metastability (using a 2FF synchronizer with closure on all relevant timing), with internal logic maintaining the width of each transmitted pulse.

Reset Behavior and Initialization

When reset is asserted asynchronously (`rst_ni`), all outputs immediately transition to safe, inactive states. The timer count is forced to zero, synchronization outputs are driven low, and register interface outputs indicate the reset condition. Deasserting reset synchronously causes all internal state machines to return to their initial states, resets all registers to their defaults, disables the timer until the START register is accessed, and prepares the synchronization

inputs for edge detection.

A recommended reset sequence is to assert `rst_ni` for at least 5 clock cycles, ensure a glitch-free deassertion, allow 3 cycles for the reset to propagate internally, and only then begin register configuration.

Initialization occurs in a defined startup sequence: First, during the reset phase, all timers are held in reset. Next, software configures timer parameters. Synchronization follows, as the PRIMARY timer asserts START and generates a `sync_load` pulse, which launches synchronized operation across all timers.

It's important to ensure the `sync_load` pulse is delivered to all SECONDARY timers within the synchronization window, and credit pulses must continue to arrive at intervals to guarantee ongoing synchronization. Clock domain crossing will add 2–3 cycles of latency to these signals.

Integration Guidelines

For multi-chiplet integration, the PRIMARY timer should be placed centrally to achieve the most efficient signal distribution and minimize routing distances to all SECONDARY timers, with inter-chiplet signal integrity always taken into account. The synchronization inputs of SECONDARY timers should be connected directly to the PRIMARY outputs, with proper termination provided for long routes and buffering added as needed for greater fan-out.

The synchronization protocol is supported across clock domains operating at a variety of frequencies; edge detection circuits must be properly timed, and the effects of jitter and phase noise should be factored in when synchronization accuracy is ensured.

At the system level, the timer's register interface is connected to the system control bus with appropriate address mapping for each instance. Interrupt connections can be implemented as required. Reset integration should guarantee that all timer instances receive reset signals simultaneously and reliably, with careful attention given to reset sequencing. For power management, clock gating is used for unused timers, power domain isolation is implemented where appropriate, and low-power operating modes are planned such that timer state is preserved across power events.

Memory Map

The System Timer OCTS IP provides a concise and efficient register interface through a 32-byte address space (0x00-0x1F) accessible via AXI4-Lite or APB4 protocols. All registers are 32-bit aligned and optimized for both PRIMARY and SECONDARY timer modes.

Programming Sequences

Complete system initialization for multi-timer synchronization:

1. Reset and Configuration:

```
// Ensure all timers are in reset
// Release reset to all timers simultaneously

// Configure PRIMARY timer
write32(primary_base + 0x04, 0x0A05); // CTRL: CREDIT_VAL=10, PULSE_WIDTH=5
write32(primary_base + 0x0C, preset_lo); // TIMER_PRESET_LO
write32(primary_base + 0x10, preset_hi); // TIMER_PRESET_HI
```

2. Configure SECONDARY Timers:

```

for (int i = 0; i < num_secondary; i++) {
    write32(secondary_base[i] + 0x04, 0x0A05); // Same CTRL settings
    write32(secondary_base[i] + 0x0C, preset_lo); // Same preset
    write32(secondary_base[i] + 0x10, preset_hi); // Same preset
}

```

3. Start Synchronized Operation:

```

// Start PRIMARY timer (generates sync_load to all SECONDARY)
write32(primary_base + 0x00, 0x01); // TIMER_START.START = 1

```

4. Verify Synchronization:

```

// Check all timers are running
for (int i = 0; i < total_timers; i++) {
    uint32_t status = read32(timer_base[i] + 0x08);
    assert(status & 0x10); // RUNNING bit should be set
}

```

Primary Timer Operation

PRIMARY timer management and monitoring:

1. Credit Generation Monitoring:

```

// Monitor credit pulse generation (implementation dependent)
// Check synchronization signal integrity at system level

```

2. Runtime Configuration Changes:

```

// Pause timer if needed (implementation dependent)
// Modify CTRL register for different credit parameters
write32(primary_base + 0x04, new_ctrl_value);
// Resume operation

```

3. Performance Monitoring:

```

// Read current count for performance measurements
uint32_t count_hi = read32(primary_base + 0x18);
uint32_t count_lo = read32(primary_base + 0x14);
uint64_t current_time = ((uint64_t)count_hi << 32) | count_lo;

```

Secondary Timer Operation

SECONDARY timer synchronization monitoring and maintenance:

1. Synchronization Health Check:

```
uint32_t credit_expired = read32(secondary_base + 0x1C);

if (credit_expired > SYNC_THRESHOLD) {
    printf("Warning: Synchronization degraded, expired=%u\n", credit_expired);

    // Reset credit expired counter
    write32(secondary_base + 0x1C, 0);

    // Check signal connectivity and PRIMARY timer operation
}
```

2. Periodic Synchronization Validation:

```
void validate_synchronization() {
    // Read all timer values simultaneously
    uint64_t primary_count = read_timer_64(primary_base);
    uint64_t secondary_count = read_timer_64(secondary_base);

    int64_t delta = (int64_t)(primary_count - secondary_count);

    if (abs(delta) > MAX_SYNC_ERROR) {
        printf("Synchronization error: delta=%ld cycles\n", delta);
        // Implement recovery procedure
    }
}
```

Error Handling and Recovery

The OCTS timer provides comprehensive error detection and recovery procedures:

1. Synchronization Failure Recovery:

```
void recover_synchronization() {
    // Stop all timers (implementation dependent)

    // Reset synchronization
    for (int i = 0; i < num_secondary; i++) {
        write32(secondary_base[i] + 0x1C, 0); // Reset CREDIT_EXPIRED
    }
}
```

```

// Restart synchronized operation
write32(primary_base + 0x00, 0x01); // Restart PRIMARY

// Verify recovery
sleep_ms(100);
validate_synchronization();
}

```

2. Clock Domain Issues:

```

// Adjust pulse width for clock domain crossing issues
uint32_t ctrl = read32(primary_base + 0x04);
ctrl = (ctrl & 0x00FF) | (10 << 8); // Increase PULSE_WIDTH to 10
write32(primary_base + 0x04, ctrl);

```

3. System Health Monitoring:

```

void monitor_timer_health() {
    // Check all timers are running
    for (int i = 0; i < total_timers; i++) {
        uint32_t status = read32(timer_base[i] + 0x08);
        if (!(status & 0x10)) {
            printf("Timer %d not running\n", i);
            // Implement recovery
        }
    }

    // Check synchronization health
    for (int i = 0; i < num_secondary; i++) {
        uint32_t expired = read32(secondary_base[i] + 0x1C);
        if (expired > WARNING_THRESHOLD) {
            printf("Secondary %d sync warning: %u cycles\n", i, expired);
        }
    }
}

```

Address Alignment and Access Requirements

The System Timer OCTS uses a 36-byte (0x00–0x23) register space, with each register 32-bit aligned for easy mapping on AXI4-Lite or APB4 buses.

Only 32-bit accesses are supported; byte or halfword accesses are not allowed and may cause errors. All writes must use full-word strobes (4'b1111).

64-bit values (e.g., timer preset/count) are split into adjacent 32-bit low and high registers. To read atomically, check high, read low, then confirm high is unchanged. Always write the low word first, then the high word, to ensure correct

latching.

An example of this access pattern in C is shown below:

```
// Reading a 64-bit value atomically
uint32_t hi1, lo, hi2;
do {
    hi1 = read32(base + high_reg_addr);
    lo  = read32(base + low_reg_addr);
    hi2 = read32(base + high_reg_addr);
} while (hi1 != hi2);
uint64_t value = ((uint64_t)hi1 << 32) | lo;

// Writing a 64-bit value
write32(base + low_reg_addr, (uint32_t)(value & 0xFFFFFFFF));
write32(base + high_reg_addr, (uint32_t)(value >> 32));
```

Following these guidelines ensures robust, predictable access to all timer registers—crucial for correct operation in high-reliability multi-chiplet systems.

Reset Values and Power-On Behavior

Table 447. Register Reset Values and Power-On Behavior

Register	Reset Value	Notes
TIMER_START	0x00000000	No automatic start, requires software initiation
CTRL	0x0001020A	Default: CREDIT_VAL=10, PULSE_WIDTH=2, STEP=1
STATUS	0x0000000X	RUNNING=0 on reset; MODE reflects the runtime is_primary_i input signal (0=PRIMARY, 1=SECONDARY)
TIMER_PRESET_LO/HI	0x00000000	Zero preset; must be configured by software before use
TIMER_COUNT_LO/HI	0x00000000	Timer starts at zero count on reset
CREDIT_EXPIRED	0x00000000	No credit expiration on startup

Detailed Register Map

Table 448. Register Map

Address	Name	Access	Description
0x0	TIMER_START	RW	System Timer Start Register
0x4	CTRL	RW	System Timer Control Register
0x8	STATUS	R	System Timer Status Register
0xC	TIMER_PRESET_LO	RW	System Timer Preset Value [31:0]
0x10	TIMER_PRESET_HI	RW	System Timer Preset Value [63:32]
0x14	TIMER_COUNT_LO	R	Current System Timer Count [31:0]
0x18	TIMER_COUNT_HI	R	Current System Timer Count [63:32]
0x1C	CREDIT_EXPIRED	RW	Credit Expired Register
0x20	TIMER_GPIO_ENABLE	RW	System Timer GPIO Enable Register

Table 449. TIMER_START Register

Bits	Field	Access	Reset	Description
31:1	Reserved	-	-	Reserved
0	START	RW	0x0	Start the system timer. Asserts the sync_load signal to start synchronization (only does anything if primary mode). Ensure both primary and secondary are out of reset.

Table 450. CTRL Register

Bits	Field	Access	Reset	Description
31:24	Reserved	-	-	Reserved
23:16	STEP	RW	0x1	Step amount for secondary timer
15:8	PULSE_WIDTH	RW	0x2	Pulse width for sync load and credit signals. A value of 0 will be rounded up to 1. WARNING: This value must be less than CREDIT_VAL
7:0	CREDIT_VAL	RW	0xA	Credit value factor. WARNING: This value must be greater than PULSE_WIDTH

Table 451. STATUS Register

Bits	Field	Access	Reset	Description
31:5	Reserved	-	-	Reserved
4	RUNNING	R	0x0	Timer is currently enabled and running
3:1	Reserved	-	-	Reserved
0	MODE	R	0x0	Timer mode: 0=PRIMARY, 1=SECONDARY (set by module parameter)

Table 452. TIMER_PRESET_LO Register

Bits	Field	Access	Reset	Description
31:0	PRESET_LO	RW	0x0	Lower 32 bits of preset value

Table 453. TIMER_PRESET_HI Register

Bits	Field	Access	Reset	Description
31:0	PRESET_HI	RW	0x0	Upper 32 bits of preset value

Table 454. TIMER_COUNT_LO Register

Bits	Field	Access	Reset	Description
31:0	COUNT_LO	R	0x0	Lower 32 bits of current timer count

Table 455. TIMER_COUNT_HI Register

Bits	Field	Access	Reset	Description
31:0	COUNT_HI	R	0x0	Upper 32 bits of current timer count

Table 456. CREDIT_EXPIRED Register

Bits	Field	Access	Reset	Description
31:0	MAX_CYCLES_EXPIRED	RW	0x0	Maximum number of clock cycles since a count credit expired (SECONDARY only). To reset the value of this register, write anything to it. If this number is very high (exact value depends on clock speed and credit value), this indicates that the secondary has not recieved a count credit pulse in a while, and is likely not syncing properly with the primary. This register can also be used to determine the clock skew between the primary and secondary. If PRIMARY, this field is always 0.

Table 457. TIMER_GPIO_ENABLE Register

Bits	Field	Access	Reset	Description
31:1	Reserved	-	-	Reserved
0	GPIO_ENABLE	RW	0x0	Enable the GPIO pad lsio interface for the system timer. This is done to prevent X-prop on reset into the secondary timer, which can cause the secondary timer to start counting early.

Programming

Multi-Chiplet Deployment Example

```
// System with 1 PRIMARY + 3 SECONDARY timers
#define PRIMARY_TIMER_BASE    0x40000000
#define SECONDARY_0_BASE     0x50000000
#define SECONDARY_1_BASE     0x60000000
#define SECONDARY_2_BASE     0x70000000

void initialize_multi_chiplet_timing() {
    uint32_t timer_bases[] = {
        PRIMARY_TIMER_BASE,
        SECONDARY_0_BASE,
        SECONDARY_1_BASE,
        SECONDARY_2_BASE
    };

    // Configure all timers with same parameters
    for (int i = 0; i < 4; i++) {
        write32(timer_bases[i] + 0x04, 0x0805); // Credit=8, Pulse=5
        write32(timer_bases[i] + 0x0C, 0x00000000); // Preset_L0=0
        write32(timer_bases[i] + 0x10, 0x00000000); // Preset_HI=0
    }

    // Start PRIMARY (automatically synchronizes SECONDARY timers)
    write32(PRIMARY_TIMER_BASE + 0x00, 0x01);

    // Verify all timers started
    for (int i = 0; i < 4; i++) {
        uint32_t status = read32(timer_bases[i] + 0x08);
        assert(status & 0x10); // RUNNING bit
    }
}
```



```
}
}
```

High-Precision Timestamping

```
// Get high-precision timestamp from any timer
uint64_t get_timestamp(uint32_t timer_base) {
    uint32_t hi1, lo, hi2;

    // Double-read method for consistency
    do {
        hi1 = read32(timer_base + 0x18); // COUNT_HI
        lo = read32(timer_base + 0x14);  // COUNT_LO
        hi2 = read32(timer_base + 0x18); // COUNT_HI again
    } while (hi1 != hi2);

    return ((uint64_t)hi1 << 32) | lo;
}

// Performance measurement example
void performance_measurement() {
    uint64_t start_time = get_timestamp(PRIMARY_TIMER_BASE);

    // Execute operation being measured
    critical_operation();

    uint64_t end_time = get_timestamp(PRIMARY_TIMER_BASE);
    uint64_t duration = end_time - start_time;

    printf("Operation took %llu cycles\n", duration);
}
```

Synchronization Health Monitoring

```
void monitor_synchronization_health() {
    uint32_t secondary_bases[] = {
        SECONDARY_0_BASE,
        SECONDARY_1_BASE,
        SECONDARY_2_BASE
    };

    for (int i = 0; i < 3; i++) {
        uint32_t credit_expired = read32(secondary_bases[i] + 0x1C);

        if (credit_expired == 0) {
```

```

        printf("Secondary %d: Excellent sync\n", i);
    } else if (credit_expired < 100) {
        printf("Secondary %d: Good sync (expired=%u)\n", i, credit_expired);
    } else if (credit_expired < 1000) {
        printf("Secondary %d: WARNING - sync degraded (expired=%u)\n",
            i, credit_expired);
        // Reset counter
        write32(secondary_bases[i] + 0x1C, 0);
    } else {
        printf("Secondary %d: CRITICAL - sync failed (expired=%u)\n",
            i, credit_expired);
        // Implement recovery procedure
        recover_synchronization();
    }
}
}

```

Dynamic Reconfiguration

```

void adjust_synchronization_parameters() {
    // Read current configuration
    uint32_t ctrl = read32(PRIMARY_TIMER_BASE + 0x04);
    uint8_t current_credit = ctrl & 0xFF;
    uint8_t current_pulse = (ctrl >> 8) & 0xFF;

    printf("Current: Credit=%u, Pulse=%u\n", current_credit, current_pulse);

    // Adjust for tighter synchronization
    uint8_t new_credit = 5; // Reduce credit value
    uint8_t new_pulse = 10; // Increase pulse width

    uint32_t new_ctrl = (new_pulse << 8) | new_credit;
    write32(PRIMARY_TIMER_BASE + 0x04, new_ctrl);

    printf("Updated: Credit=%u, Pulse=%u\n", new_credit, new_pulse);

    // Monitor effect on synchronization quality
    sleep_ms(1000);
    monitor_synchronization_health();
}

```

5.10.3. eFuse

TODO

Description and Features

Architecture

The eFuse IP provides secure One-Time Programmable (OTP) memory management with foundry-agnostic design. The layered architecture separates security control logic from foundry-specific implementation details, enabling deployment across different OTP technologies while maintaining consistent security characteristics and software interfaces.

Interface Controller

The eFuse Interface Controller (efuse_interface_controller.sv) provides dual-path access control with security policy enforcement.

Access Paths

- **SMC AXI4-Lite Interface:** Functional access for normal system operations
- **JTAG AXI4-Lite Interface:** Debug access for manufacturing test and development

Destination Endpoints

Endpoint	Description
Shadow Registers	High-performance boot-time caching for rapid access without OTP timing constraints
eFuse CSR	Control and status registers for read, program, and verification operations
MMR	Memory-Mapped Registers for SEP security tokens with specialized access control
Interface SHIM CSR	Configuration registers for foundry-specific timing adaptation. Implemented by the adopter, but always accessed via the interface controller.

Shadow Register Subsystem

The shadow register subsystem accelerates OTP access by caching eFuse contents at boot time. On power-up, a dedicated FSM automatically issues a bulk read command (with `access_length_words` set to cover the full eFuse bank), copying all OTP data into the shadow registers. This ensures the shadow registers are fully populated and always synchronized with the OTP state at each boot.

Features

The feature list for the shadow register subsystem is as follows:

Feature	Description
Capacity	Configurable (default: 24576 bits / 3KB, 768 × 32-bit words), set via the <code>SHADOW_REG_BITS</code> parameter
Initialization	Automatic boot-time loading via Fuse Sense FSM
Access Control	Multi-tier hardware and software lock mechanisms
Data Consistency	Maintains synchronization with underlying OTP storage upon each power-on reset

State Machine Architecture

The read interface FSM (efuse_read_interface.sv) operates in a simple two-state sequence, transitioning from idle to waiting for a response as it handles on-demand eFuse reads that fall outside the shadow register cache. This mechanism works alongside the shadow subsystem, providing direct access only when necessary, and actively reports status to support robust error handling and retry logic.

The program interface FSM (efuse_program_interface.sv) enforces the write-once nature of OTP hardware, ensuring that programmed data cannot be altered once set. It supports write-and-verify procedures for reliability, integrating verification steps that check programmed values against intended data, and fully blocks any unauthorized overwrites to maintain data integrity.

Parameterized Command Typedefs

The fuse command request/response structs exchanged between the controller and the SHIM are not declared with fixed types. Instead, they are generated from parameterized preprocessor macros defined in `efuse_typedef.svh`. This keeps the command interface technology-agnostic: each integration supplies its own address, data, access-length, and command types, while the macros guarantee a consistent struct layout and field ordering across all OTP variants.

Macro	Generated Type	Purpose
<code>EFUSE_COMMAND_REQ_T</code>	<code>fuse_command_req_t</code>	Controller-to-SHIM command request
<code>EFUSE_COMMAND_RESP_T</code>	<code>fuse_command_resp_t</code>	SHIM-to-controller command response

The request macro is parameterized over the struct name plus four payload types (`addr_t`, `data_t`, `access_length_words_t`, `command_t`), allowing the controller to size the address space, transfer length, and command encoding to match the target OTP technology:

```
`define EFUSE_COMMAND_REQ_T(fuse_command_req_t, addr_t, data_t, access_length_words_t,
command_t) \
    typedef struct packed {
        addr_t          address;          // Target address          \
        logic           program_data;      // Data to program (1'b1) \
        access_length_words_t access_length_words; // Transfer size          \
        command_t       command;          // Command type           \
        logic           valid;            // Request valid          \
    } fuse_command_req_t;
```

The response macro is parameterized over the struct name and the `data_t` read-data type:

```
`define EFUSE_COMMAND_RESP_T(fuse_command_resp_t, data_t) \
    typedef struct packed {
        data_t          data;             // Read data              \
        logic           status;          // Operation status       \
        logic           valid;          // Response valid         \
    } fuse_command_resp_t;
```

A typical instantiation expands both macros with the integration's local types:

```
`EFUSE_COMMAND_REQ_T(fuse_command_req_t, addr_t, data_t, access_length_words_t,
command_t)
`EFUSE_COMMAND_RESP_T(fuse_command_resp_t, data_t)
```

Foundry-Specific Signals

Table 458. Example Hardware Interfaces by OTP Variant

OTP Variant	Hardware Interface Signals
Adopter OTP macro	Adopter-defined macro-side signals
Current Example Model	APB interface with register-based control

Interface SHIM Layer

The Interface SHIM (`efuse_interface_shim.sv`) is an example, foundry-specific implementation of the custom fuse command interface. Its role is to translate the controller's generic commands into the signaling expected by a particular OTP macro; in this reference version the macro-side interface is APB, but a real integration would replace this with the foundry's native signals.

The SHIM decodes three command types from the controller:

- **Read** — fetch one or more words from the bank.
- **Program** — burn new bits into the bank.
- **Program/read-back** — program, then read back to verify success.

It implements these with dedicated read and program FSMs that sequence the macro-side transactions and return a `fuse_command_resp_t` to the controller. Foundry-specific timing and configuration (for example, program pulse widths) are exposed through the SHIM Control CSRs, accessed over the `fuse_bank_ctrl` AXI4-Lite interface.

eFuse Bank Model

The eFuse bank model (`efuse_bank_model.sv`) is a reference, simulation-only stand-in for the real foundry OTP macro. It is a 3KB (768 x 32-bit word) store fronted by a standard APB interface, and it emulates the set-once behavior of real OTP so the controller and SHIM can be exercised end-to-end without silicon. In a production integration it is replaced by the actual foundry macro driven by the SHIM.

Access Permissions and Security

Access control is layered for defense in depth: per-field permissions are described by `efuse_field_map_i`, lock state is enforced by the eFuse guard (`efuse_guard.sv`), security-disable tokens are validated, and access can additionally be gated by the chip lifecycle state.

Each eFuse field is described by a `rule_t` entry that pairs an address range with a 4-bit `lock` permission field:

```
typedef struct packed {
    logic [EFUSE_FIELD_MAP_IDX_WIDTH-1:0] idx;      // Field index
    logic [3:0] lock;                                // Permission bits
    logic [31:0] start_addr;                          // Field start address
    logic [31:0] end_addr;                            // Field end address
} rule_t;
```

The `lock` field encodes read, write, and secure-test-mode permissions for the field:

Bits	Function	Encoding
<code>lock[0]</code>	Read	<code>0</code> = readable, <code>1</code> = read-locked
<code>lock[2:1]</code>	Write	<code>00</code> = unlocked, <code>10</code> = set-only (1's may be added, never cleared), <code>11</code> = write-locked
<code>lock[3]</code>	Secure test mode	<code>0</code> = unrestricted, <code>1</code> = access requires secure test mode

Hardware vs. Software Locks

The same permission semantics are reached through two independent mechanisms, and the guard enforces whichever is more restrictive:

Property	Hardware Lock	Software Lock
Source	Permission bits fused into the eFuse array (the field map reflects the programmed values)	Runtime writes to the <code>LOCKS</code> CSR (write-1-to-set)
Persistence	Permanent — backed by one-time-programmable bits and cannot be cleared once set	Volatile — retained only until the next system reset
Reversibility	Irreversible by any means	Cleared by reset, allowing the field to be re-opened
Typical use	Final production lockdown of provisioned secrets	Temporary protection during boot, debug, or staged provisioning

Because software locks are write-1-to-set, firmware can tighten access at runtime but can never loosen a restriction without a reset; hardware locks cannot be loosened at all. On top of these per-field locks, sensitive fields can be further gated by the chip lifecycle state and by secure test mode (`lock[3]`), so a field may be accessible in one operational mode and fully blocked in another even when its read/write bits are clear.

For sensitive functions, such as those governed by the MMR registers, the system enforces token-based security. Only by providing the correct 256-bit tokens, such as the RMA SOP Token for state transitions, the RMA Chiplet Token for chiplet-specific operations, or the Security Disable Token to override protections, can access to these resources be granted.

Security Disable Bypass Mechanism

The eFuse interface controller supports a security disable bypass mechanism via the `security_disable_i` input signal. This mechanism is primarily used for test and debug scenarios where the automatic fuse sensing sequence needs to be bypassed.

Behavior when `security_disable_i` is asserted:

Behavior	Description
FSM Bypass	The automatic fuse sensing state machine is bypassed and does not issue read commands to the physical eFuse bank.
Shadow Register Output	Shadow register values are output directly to downstream logic via <code>shadow_regs_o</code> , regardless of <code>fuse_sense_done</code> status.
Shadow Register Loading	Shadow registers are not loaded from fuse command responses; they retain their reset values (all zeros for SMC).
APB Access	APB access to shadow registers is permitted without waiting for <code>fuse_sense_done</code> completion.
Reset Values	When the system is in reset state with <code>security_disable_i</code> asserted, shadow registers are initialized to zero (default reset value for SMC).

For SMC, the internal security disable signal is directly driven by the `security_disable_i` input port.

Lifecycle State (LC_STATE) Effects

The eFuse interface controller enforces access restrictions based on the chip's lifecycle state.

Lifecycle State Encoding:

State	Encoding	Description
TEST_DEV	0x0	Test/Development state (default)
PROD	0x1	Production state
PROD_DBG	0x1 + DEMOTE=1	Production Debug (volatile, clears on reset)
RMA_SOP	0x2, 0x3	RMA System-in-Package
RMA_CHIPLET	0x6-0x7	RMA Chiplet level
PROD_END	0x8	Production End (terminal state)
INVALID	All others	Invalid state (error fallback)

JTAG Access Control Based on LC_STATE:

When lifecycle state is PROD (0x1) or RMA_SOP (0x2, 0x3), JTAG access to eFuse registers is restricted:

Access Type	PROD/RMA_SOP State	Other States
JTAG Write	Blocked (error response)	Allowed
JTAG Read	Blocked except Chiplet ID	Allowed
JTAG Read Chiplet ID	Allowed	Allowed

The access control mechanism uses an AXI-Lite demux that routes blocked requests to an error slave. When a request is blocked, the error slave returns an error response with data value **0xbadcab1e**.

Chiplet ID Exception:

Even in PROD or RMA_SOP states, JTAG read access to the chiplet ID and package ID registers is permitted. This allows identification of the chip for manufacturing and field returns without exposing sensitive eFuse contents.

Secure Test Mode (secure_tm_i)

The eFuse interface controller supports a Secure Test Mode via the **secure_tm_i** input signal. This mode provides additional protection for sensitive security assets during secure manufacturing test operations.

Behavior when **secure_tm_i** is asserted:

Behavior	Description
Fuse Command Blocking	All fuse commands to the physical eFuse bank are blocked. The guard logic filters incoming requests and returns default responses.
RMA Token Protection	RMA SOP and RMA Chiplet tokens are disconnected from shadow register outputs (set to zero), preventing token exposure during test.
Write Lock Contribution	Secure test mode contributes to write lock status, preventing shadow register modifications.
Response Behavior	When fuse commands are blocked, default response values (all zeros) are returned to requesters.

SMC Configuration:

For SMC, **secure_tm_i** is tied to logic **0**, meaning Secure Test Mode is not active during normal SMC operation. This signal

is primarily relevant for SEP where sensitive RMA tokens must be protected during manufacturing test flows.

Interface

Configuration Parameters

Parameter	Default	Description
OTP_SIZE	3KB	Total OTP storage capacity
WORD_WIDTH	32 bits	OTP word width
NUM_WORDS	768	Total number of OTP words

Signal Interfaces

Clock and Reset

Signal Name	Direction	Width	Description
clk_i	Input	1	System clock input
rst_ni	Input	1	Active-low asynchronous reset

Main AXI4-Lite Interfaces

The eFuse controller provides two AXI4-Lite interfaces for register access:

SMC Interface (Functional Access)

Signal Name	Direction	Type	Description
axil_req_i	Input	efuse_axil_req_t	AXI4-Lite request from SMC
axil_resp_o	Output	efuse_axil_resp_t	AXI4-Lite response to SMC

JTAG Interface (Debug Access)

Signal Name	Direction	Type	Description
axil_jtag_req_i	Input	efuse_axil_req_t	AXI4-Lite request from JTAG
axil_jtag_resp_o	Output	efuse_axil_resp_t	AXI4-Lite response to JTAG

Fuse Bank Control Interface

APB interface for communicating with the eFuse bank model:

Signal Name	Direction	Type	Description
fuse_bank_ctrl_req_o	Output	efuse_apb_req_t	APB request to eFuse bank
fuse_bank_ctrl_resp_i	Input	efuse_apb_resp_t	APB response from eFuse bank

Fuse Command Interface

Custom interface for SHIM state machine communication:

Signal Name	Direction	Type	Description
<code>fuse_command_req_o</code>	Output	<code>fuse_command_req_t</code>	Command request to SHIM
<code>fuse_command_resp_i</code>	Input	<code>fuse_command_resp_t</code>	Command response from SHIM

Security and Control Signals

Signal Name	Direction	Width	Description
<code>lc_state_i</code>	Input	4	Life cycle state from SEP
<code>secure_tm_i</code>	Input	1	Secure test mode enable
<code>test_en_i</code>	Input	1	Test enable signal
<code>security_disable_i</code>	Input	1	Security disable override
<code>efuse_field_map_i</code>	Input	<code>rule_t</code> <code>[EFUSE_FIELDS-1:0]</code>	Field access control rules
<code>fuse_sense_done_o</code>	Output	1	Shadow register loading complete
<code>security_disable_o</code>	Output	1	Security disable status
<code>shadow_regs_o</code>	Output	<code>efuse_map_t</code>	Shadow register contents

Debug and Status Signals

Signal Name	Direction	Width	Description
<code>is_write_locked_shadow_regs_o</code>	Output	1	Shadow register CSRs are write locked
<code>is_read_locked_shadow_regs_o</code>	Output	1	Shadow register CSRs are read locked
<code>is_program_locked_o</code>	Output	1	eFuse program operations are locked
<code>is_read_locked_o</code>	Output	1	eFuse read operations are locked
<code>is_write_setup_only_o</code>	Output	1	Field is in write set-only mode (bits may be set, not cleared)
<code>is_lc_state_access_o</code>	Output	1	Access is gated by the current lifecycle state
<code>is_read_timeout_debug_o</code>	Output	1	A read command timed out waiting on the SHIM/bank
<code>is_program_timeout_debug_o</code>	Output	1	A program command timed out waiting on the SHIM/bank
<code>is_efuse_req_err_o</code>	Output	1	A request was blocked/errored by the eFuse guard
<code>is_secure_tm_blocked_o</code>	Output	1	Access blocked because secure test mode is not active
<code>is_rma_sip_token_match_debug</code>	Output	6	RMA SiP token match status code
<code>is_rma_chiplet_token_match_debug</code>	Output	6	RMA chiplet token match status code
<code>sec_disable_token_o</code>	Output	256	Security disable token value (8 x 32-bit words)

Fuse Command Protocol

Custom command interface between controller and SHIM:

Request Structure (`fuse_command_req_t`):

```
typedef struct packed {
    addr_t          address;           // Target address in bits
    logic           program_data;      // Data to program (1'b1)
    access_length_words_t access_length_words; // Transfer size
    command_t       command;           // Command type
    logic           valid;             // Request valid
}
```

```
} fuse_command_req_t;
```

Response Structure (`fuse_command_resp_t`):

```
typedef struct packed {
    data_t      data;      // Read data
    logic       status;    // Operation status
    logic       valid;     // Response valid
} fuse_command_resp_t;
```

Command Types:

- `FUSE_COMMAND_READ (2'b00)`: Read operation
- `FUSE_COMMAND_PROGRAM (2'b01)`: Program operation
- `FUSE_COMMAND_PROGRAM_READ_BACK (2'b10)`: Program with verification

Memory Map

Overview

The eFuse IP provides multiple register maps organized into four main address spaces:

Table 459. Address Space Overview

Address Range	Register Block	Description
0x0000-0x0FFF	Shadow Registers	Cached eFuse contents (read-only)
0x1000-0x1FFF	MMR Registers	Security tokens for authentication
0x2000-0x2FFF	eFuse Control CSR	Command and status registers
0x3000-0x3FFF	SHIM Control CSR	Foundry-specific configuration

Shadow Registers

The shadow registers provide cached access to eFuse contents, including chiplet identification, package information, I2C/I3C address configuration (9 instances for I2C_I3C_ID[0:8]), and other system configuration data. These registers are automatically loaded from the physical eFuse array during boot.

NOTE

Register Arrays: This register map contains the following register arrays:

- **I2C_I3C_ID[9]**: 9 registers
 - Base Address: 0x8AC
 - Address Increment: 0x8 per register
- **RESERVED[65]**: 65 registers
 - Base Address: 0xAFC
 - Address Increment: 0x4 per register

Table 460. Register Map

Address	Name	Access	Description
0x0	LOCKS	RW	Lock controls for each fuse field. When write locked, the corresponding field is not programmable. When read locked, the shadow register is not readable.
0x8	CHIPLET_ID	RW	Chiplet ID register
0x28	PACKAGE_ID	RW	Package ID register
0x48	BIRA	RW	Built-In Repair Analysis data
0x848	CLUSTER	RW	CPU cluster configuration
0x888	FABRIC	RW	Fabric configuration
0x8A8	SOP_TOPOLOGY	RW	System on Package topology
0x8AC - 0x8EC	I2C_I3C_ID[9]	RW	I2C/I3C interface ID
0x8F4	I2C_CLOCK_GATING	RW	I2C clock gating control
0x8F8	I3C_DISABLE	RW	I3C interface disable control
0x8FC	PLL_AND_SENSOR	RW	PLL and sensor configuration
0xAFC - 0xBFC	RESERVED[65]	RW	Reserved register

Table 461. LOCKS Register

Bits	Field	Access	Reset	Description
63:26	unused_lock_bits	RW	0x0	Unused lock bits for future expansion.
25	reserved_read_lock	RW	0x0	Reserved read lock. When set, reserved field is not readable.
24	reserved_write_lock	RW	0x0	Reserved write lock. When set, reserved field is not writable.
23	spi_config_read_lock	RW	0x0	SPI config read lock. When set, SPI config field is not readable.
22	spi_config_write_lock	RW	0x0	SPI config write lock. When set, SPI config field is not writable.
21	spi_ctrl_field_enable_read_lock	RW	0x0	SPI control field enable read lock. When set, SPI control field is not readable.
20	spi_ctrl_field_enable_write_lock	RW	0x0	SPI control field enable write lock. When set, SPI control field is not writable.
19	pll_and_sensor_read_lock	RW	0x0	PLL and sensor read lock. When set, PLL and sensor field is not readable.
18	pll_and_sensor_write_lock	RW	0x0	PLL and sensor write lock. When set, PLL and sensor field is not writable.
17	i3c_disable_read_lock	RW	0x0	I3C disable read lock. When set, I3C disable field is not readable.
16	i3c_disable_write_lock	RW	0x0	I3C disable write lock. When set, I3C disable field is not writable.
15	i2c_clock_gating_read_lock	RW	0x0	I2C clock gating read lock. When set, I2C clock gating field is not readable.
14	i2c_clock_gating_write_lock	RW	0x0	I2C clock gating write lock. When set, I2C clock gating field is not writable.
13	i2c_i3c_id_read_lock	RW	0x0	I2C/I3C ID read lock. When set, I2C/I3C ID field is not readable.
12	i2c_i3c_id_write_lock	RW	0x0	I2C/I3C ID write lock. When set, I2C/I3C ID field is not writable.
11	sop_topology_read_lock	RW	0x0	SOP topology read lock. When set, SOP topology field is not readable.
10	sop_topology_write_lock	RW	0x0	SOP topology write lock. When set, SOP topology field is not writable.

Bits	Field	Access	Reset	Description
9	<code>fabric_read_lock</code>	RW	0x0	Fabric read lock. When set, fabric field is not readable.
8	<code>fabric_write_lock</code>	RW	0x0	Fabric write lock. When set, fabric field is not writable.
7	<code>cluster_read_lock</code>	RW	0x0	Cluster read lock. When set, cluster field is not readable.
6	<code>cluster_write_lock</code>	RW	0x0	Cluster write lock. When set, cluster field is not writable.
5	<code>bira_dis_read_lock</code>	RW	0x0	BIRA disable read lock. When set, BIRA field is not readable.
4	<code>bira_dis_write_lock</code>	RW	0x0	BIRA disable write lock. When set, BIRA field is not writable.
3	<code>package_id_read_lock</code>	RW	0x0	Package ID read lock. When set, package ID field is not readable.
2	<code>package_id_write_lock</code>	RW	0x0	Package ID write lock. When set, package ID field is not writable.
1	<code>chiplet_id_read_lock</code>	RW	0x0	Chiplet ID read lock. When set, chiplet ID field is not readable.
0	<code>chiplet_id_write_lock</code>	RW	0x0	Chiplet ID write lock. When set, chiplet ID field is not writable.

Table 462. CHIPLET_ID Register

Bits	Field	Access	Reset	Description
255:0	<code>chiplet_id_value</code>	RW	0x0	Chiplet ID stored in fuse, divided into 8 words and defined as follows: [1:0]: Serial number of the chiplet. The format is flexible and could include time info of manufacture, (x, y) coordinates on the wafer, and a counter for example; [3]: Chiplet Design Version. The format is flexible and the word bits could be defined as reserved[31:16], major[15:8], minor[7:0] for example; [127:96]: Chiplet Design ID; [4]: Chiplet User ID. When TT assembles the package, we assign TT's ID (32'b0); [5]: Chiplet User Group ID; [7:6]: Reserved for extension.

Table 463. PACKAGE_ID Register

Bits	Field	Access	Reset	Description
255:0	<code>package_id_value</code>	RW	0x0	Package ID stored in fuse, divided into 8 words. The same value gets provisioned into SMCs in all chiplets of the same package. Words are defined as follows: [1:0]: Serial number of the package; [2]: Package Design Version, e.g. A1Q1/...; [3]: Package Design ID; [4]: Package User ID, e.g. if it's TT who assembles the system, use TT's GROUP_ID (32'b0); [5]: Package User Group ID; [7:6]: Reserved for extension

Table 464. BIRA Register

Bits	Field	Access	Reset	Description
16383:0	<code>repair_data</code>	RW	0x0	16KB memory repair data for SMC and SEP SRAMs

Table 465. CLUSTER Register

Bits	Field	Access	Reset	Description
511:0	<code>cluster_config</code>	RW	0x0	512-bit fuses reserved for CPU cluster configuration

Table 466. FABRIC Register

Bits	Field	Access	Reset	Description
255:0	<code>fabric_config</code>	RW	0x0	256-bit fuses reserved for fabric configuration

Table 467. SOP_TOPOLOGY Register

Bits	Field	Access	Reset	Description
31:0	topology_serial	RW	0x0	Serial number of topology configuration

Table 468. I2C_I3C_ID[9] Register

Bits	Field	Access	Reset	Description
63:0	interface_id	RW	0x0	I2C or I3C ID configuration: * Static ID mode: [6:0] contains ID, [63:7] reserved * Provisional ID mode: [63:0] full ID

Table 469. I2C_CLOCK_GATING Register

Bits	Field	Access	Reset	Description
31:0	clock_gating_config	RW	0x0	I2C clock gating configuration

Table 470. I3C_DISABLE Register

Bits	Field	Access	Reset	Description
31:0	disable_config	RW	0x0	I3C interface disable configuration

Table 471. PLL_AND_SENSOR Register

Bits	Field	Access	Reset	Description
4095:0	pll_sensor_config	RW	0x0	4KB fuses reserved for PLL and sensor configuration

Table 472. RESERVED[65] Register

Bits	Field	Access	Reset	Description
31:0	reserved_data	RW	0x0	Reserved for future use

MMR Registers

The MMR (Memory-Mapped Registers) provide access to security tokens for authentication, including RMA tokens and security disable tokens.

NOTE

Register Arrays: This register map contains the following register arrays:

- **RMA_SIP_TOKEN_I[8]:** 8 registers
 - Base Address: 0x0
 - Address Increment: 0x4 per register
- **RMA_CHIPLLET_TOKEN_I[8]:** 8 registers
 - Base Address: 0x20
 - Address Increment: 0x4 per register
- **SEC_DISABLE_TOKEN_I[8]:** 8 registers
 - Base Address: 0x40
 - Address Increment: 0x4 per register

Table 473. Register Map

Address	Name	Access	Description
0x0 - 0x1C	RMA_SIP_TOKEN_I[8]	RW	Software-writeable RMA token word. The assembled token is hashed and compared against the fused value to authorize lifecycle (LC) state transitions.
0x20 - 0x3C	RMA_CHIPLET_TOKEN_I[8]	RW	Software-writeable RMA token word. The assembled token is hashed and compared against the fused value to authorize lifecycle (LC) state transitions.
0x40 - 0x5C	SEC_DISABLE_TOKEN_I[8]	RW	Software-writeable security-disable token word. The assembled token is hashed and compared against the post-silicon injected value to authorize security_disable.
0x60	TOKEN_EOP	W	Token end-of-packet control. Writing the per-token go bit signals that all token words have been written and triggers a pulse to begin token hashing.
0x64	RMA_SIP_TOKEN_MATCH	R	Token match status. Reports the result of the token hash comparison (match, mismatch, or error).
0x68	RMA_CHIPLET_TOKEN_MATCH	R	Token match status. Reports the result of the token hash comparison (match, mismatch, or error).
0x6C	SEC_DISABLE_TOKEN_MATCH	R	Token match status. Reports the result of the token hash comparison (match, mismatch, or error).

Table 474. RMA_SIP_TOKEN_I[8] Register

Bits	Field	Access	Reset	Description
31:0	token	RW	-	Portion of writeable version of RMA_*_TOKEN, hashed, and compared with fused value to allow LC to change states.

Table 475. RMA_CHIPLET_TOKEN_I[8] Register

Bits	Field	Access	Reset	Description
31:0	token	RW	-	Portion of writeable version of RMA_*_TOKEN, hashed, and compared with fused value to allow LC to change states.

Table 476. SEC_DISABLE_TOKEN_I[8] Register

Bits	Field	Access	Reset	Description
31:0	token	RW	-	Portion of writeable version of SEC_DISABLE_TOKEN, hashed, and compared with the post silicon injected values to allow security_disable.

Table 477. TOKEN_EOP Register

Bits	Field	Access	Reset	Description
31:17	Reserved	-	-	Reserved
16	secure_disable_token_go	W	0x0	Write 1 to indicate that all token data has been written; this generates a pulse to begin token hashing.
15:9	Reserved	-	-	Reserved
8	rma_chiplet_token_go	W	0x0	Write 1 to indicate that all token data has been written; this generates a pulse to begin token hashing.
7:1	Reserved	-	-	Reserved
0	rma_sip_token_go	W	0x0	Write 1 to indicate that all token data has been written; this generates a pulse to begin token hashing.

Table 478. RMA_SIP_TOKEN_MATCH Register

Bits	Field	Access	Reset	Description
31:6	Reserved	-	-	Reserved
5:0	token_match_status	R	0x0	Store the status of the token match. Match = 6'b010101, Mismatch = 6'b101010, Error = 6'b111111

Table 479. RMA_CHIPLET_TOKEN_MATCH Register

Bits	Field	Access	Reset	Description
31:6	Reserved	-	-	Reserved
5:0	token_match_status	R	0x0	Store the status of the token match. Match = 6'b010101, Mismatch = 6'b101010, Error = 6'b111111

Table 480. SEC_DISABLE_TOKEN_MATCH Register

Bits	Field	Access	Reset	Description
31:6	Reserved	-	-	Reserved
5:0	token_match_status	R	0x0	Store the status of the token match. Match = 6'b010101, Mismatch = 6'b101010, Error = 6'b111111

eFuse Control Registers

The eFuse Control CSR provides command and status registers for read, write, and verification operations on the eFuse array.

Table 481. Register Map

Address	Name	Access	Description
0x0	EFUSE_INTERFACE_CTRL_STATUS	RW	eFuse interface status and error-clear register. Reports the sense-done flag and request/address error conditions, and provides write-1 strobes to clear those errors.
0x4	EFUSE_PROGRAM_CTRL	RW	eFuse program control and status. Configures the program address and data, triggers a program (optionally with read-back), and reports program progress and completion status.
0x8	EFUSE_READ_CTRL	RW	eFuse read control and status. Configures the read address, triggers a read, and reports read progress and completion status.
0xC	EFUSE_PROGRAM_INTERFACE_READ_DATA	R	Read data captured through the program interface. Hardware debug only; valid only while the program interface state machine is in its capture-data state.
0x10	EFUSE_READ_INTERFACE_READ_DATA	R	Read data returned through the eFuse read interface.
0x14	EFUSE_READ_REQ_TIMEOUT	RW	Read interface timeout configuration. Sets the number of cycles the read interface waits for a SHIM response and enables the read timeout.
0x18	EFUSE_PROGRAM_REQ_TIMEOUT	RW	Program interface timeout configuration. Sets the number of cycles the program interface waits for a SHIM response and enables the program timeout.

Table 482. EFUSE_INTERFACE_CTRL_STATUS Register

Bits	Field	Access	Reset	Description
31:11	Reserved	-	-	Reserved
10	efuse_read_addr_error_clear	W	0x0	Clears the read out of bounds error bit (efuse_read_addr_error) in efuse_ctrl_status
9	efuse_program_addr_error_clear	W	0x0	Clears the program out of bounds error bit (efuse_program_addr_error) in efuse_ctrl_status

Bits	Field	Access	Reset	Description
8	efuse_req_error_clear	W	0x0	Clears the blocked request error bit in efuse_ctrl_status
7	Reserved	-	-	Reserved
6	efuse_read_addr_error	R	0x0	Indicates that an invalid eFuse read address was used (address exceeds the implemented eFuse size). The error must be cleared by writing 1 to efuse_read_addr_error_clear.
5	efuse_program_addr_error	R	0x0	Indicates that an invalid eFuse program address was used (address exceeds the implemented eFuse size). The error must be cleared by writing 1 to efuse_program_addr_error_clear.
4	efuse_req_error	R	0x0	Indicates that the eFuse was blocked because of either read or a program lock. The error must be cleared by writing 1 to efuse_req_error_clear to this register
3:1	Reserved	-	-	Reserved
0	efuse_sense_done	R	-	Indicates if the eFuse state machine has completed

Table 483. EFUSE_PROGRAM_CTRL Register

Bits	Field	Access	Reset	Description
31:28	Reserved	-	-	Reserved
27	program_enable	RW	0x0	When set, enables the program request
26	program_status	R	0x0	0: No Error. 1: Error
25	program_done	R	0x0	When set, the program has completed, the status of which is in program_status
24	program_busy	R	0x0	When set, the program is in progress
23:19	Reserved	-	-	Reserved
18	efuse_program_read_back	RW	0x0	When set, trigger a read back after a program
17	efuse_program_go	RW	0x0	When set, triggers a program to the eFuse. This will be ignored if the lock bit is set
16	efuse_data	RW	0x0	Data to program to eFuse. If this bit is 0, the program request will be ignored
15:0	efuse_addr	RW	0x0	Bit address to program

Table 484. EFUSE_READ_CTRL Register

Bits	Field	Access	Reset	Description
31:29	Reserved	-	-	Reserved
28	read_enable	RW	0x0	When set, enables the read request
27	Reserved	-	-	Reserved
26	read_status	R	0x0	0: No Error. 1: Logic error, assert read_go when read is not enabled
25	read_done	R	0x0	When set, the read has completed, the status of which is in read_status
24	read_busy	R	0x0	When set, the read is in progress
23:17	Reserved	-	-	Reserved
16	efuse_read_go	RW	0x0	When set, triggers a read to efuse.
15:0	efuse_addr	RW	0x0	Bit address to read

Table 485. EFUSE_PROGRAM_INTERFACE_READ_DATA Register

Bits	Field	Access	Reset	Description
31:0	dout	R	0x0	Read Data from eFuse through program interface. This field is only valid when the efuse programming interface is utilized to burn the efuse IP and only when the state machine is during ST_CAPTUEER_DATA state. Hardware debug only

Table 486. EFUSE_READ_INTERFACE_READ_DATA Register

Bits	Field	Access	Reset	Description
31:0	dout	R	0x0	Read Data from eFuse through READ interface.

Table 487. EFUSE_READ_REQ_TIMEOUT Register

Bits	Field	Access	Reset	Description
31:29	Reserved	-	-	Reserved
28	read_req_timeout_enable	RW	0x0	Enable timeout on read interface
27:0	read_req_timeout_cycles	RW	0x800000	Number of cycles the read interface will wait for a response from the shim. At 100MHz this can count ~= 2.5 seconds.

Table 488. EFUSE_PROGRAM_REQ_TIMEOUT Register

Bits	Field	Access	Reset	Description
31:29	Reserved	-	-	Reserved
28	program_req_timeout_enable	RW	0x0	Enable timeout on program interface.
27:0	program_req_timeout_cycles	RW	0x800000	Number of cycles the program interface will wait for a response from the shim. At 100MHz this can count ~= 2.5 seconds.

SHIM Control Registers

The SHIM Control CSR provides foundry-specific configuration registers for timing adaptation and eFuse bank initialization.

Table 489. Register Map

Address	Name	Access	Description
0x0	EFUSE_BANK_INIT_TIME	RW	eFuse bank initialization time. Number of cycles to wait for the OTP macro to initialize before sensing can begin.

Table 490. EFUSE_BANK_INIT_TIME Register

Bits	Field	Access	Reset	Description
31:0	init_time	RW	0x20	Most OTP have a initalization time before sensing can be done

Programming

Usage Examples

Example 1: Programming a Single Bit

```
// Enable programming
EFUSE_PROGRAM_CTRL = (1 << 28) | (1 << 16) | (bit_addr & 0xFFFF);
```

```
// Trigger program
EFUSE_PROGRAM_CTRL |= (1 << 17);

// Wait for completion
while (EFUSE_PROGRAM_CTRL & (1 << 24)); // Wait for busy to clear

// Check status
if ((EFUSE_PROGRAM_CTRL >> 26) & 0x3) {
    // Handle error
}
```

Example 2: Reading Multiple Words

```
// Setup read operation
EFUSE_READ_CTRL = (1 << 28) | (num_words << 16) | (bit_addr & 0xFFFF);

// Trigger read
EFUSE_READ_CTRL |= (1 << 17);

// Wait for completion
while (EFUSE_READ_CTRL & (1 << 24)); // Wait for busy to clear

// Read data
uint32_t data = EFUSE_READ_INTERFACE_READ_DATA;
```

Program Operation Sequence

1. Setup Phase:

- Set `program_enable` = 1 in `EFUSE_PROGRAM_CTRL`
- Configure `efuse_addr` with target bit address
- Set `efuse_data` = 1 (only 1s can be programmed)
- Optionally set `efuse_program_read_back` = 1 for verification

2. Execute Phase:

- Set `efuse_program_go` = 1 to trigger operation
- Monitor `program_busy` until it goes low
- Check `program_done` = 1 for completion

3. Status Check:

- Read `program_status` field:
 - 0: Success
 - 1: Bad data (data was not 1)
 - 2: Bad enables (program_enable not set or lock active)

- 3: Programming failure (verification failed)

Read Operation Sequence

1. Setup Phase:

- Set `read_enable` = 1 in `EFUSE_READ_CTRL`
- Configure `efuse_addr` with target bit address
- Set `efuse_read_length_words` for multi-word reads

2. Execute Phase:

- Set `efuse_read_go` = 1 to trigger operation
- Monitor `read_busy` until it goes low
- Check `read_done` = 1 for completion

3. Data Retrieval:

- Read data from `EFUSE_READ_INTERFACE_READ_DATA`
- Check `read_status` for any errors

5.10.4. UART 16550

Overview

This document specifies the UART 16550 hardware Component functionality. The UART 16550 implements the National Semiconductor UART 16550 standard with additional enhancements including independently-configurable Transmitter and Receiver FIFO depths and FIFO data and pointer parity protection. It provides a full-featured serial communication interface compatible with industry-standard UART protocols.

Description and Features

- Full UART 16550 compatibility and functionality
- 32-bit AXI4-Lite register interface
- Independently-configurable Transmitter and Receiver FIFO depths (4 to 4096 entries)
- FIFO parity protection for data and pointers with error reporting
- Comprehensive interrupt support with 6 priority levels
- Direct Memory Access (DMA) signaling with immediate and hysteresis modes
- Modem control interface (CTS, DSR, RI, DCD, RTS, DTR, OUT1, OUT2)

Architecture

Overview

The UART 16550 IP uses a hierarchical architecture with three main layers: a top-level wrapper handling the register interface, the core UART logic with FIFO management, and the transmit/receive engines. This separation allows for clear division between interface, data flow, and serial processing.

Hierarchical Structure

Top Level (uart_16550)

The `uart_16550` top-level module acts as an interface wrapper. It manages AXI4-Lite transactions and register map routing, ensuring protocol compliance and reliable integration.

UART Core (uart_core)

The `uart_core` module handles the main UART functions. It manages transmit and receive FIFOs with configurable depths from 4 to 4096 entries, supporting diverse use cases from low-latency to high-throughput. Baud rate generation uses 16x oversampling with a programmable divisor for timing precision.

The interrupt controller handles six prioritized interrupt sources, and modem control supports all standard modem signals. DMA is supported in both immediate and hysteresis modes for efficient data transfers. Both internal and external loopback modes are supported for testing and diagnostics.

The core works together with the transmit engine (serialization), receive engine (deserialization), FIFO controllers for buffering, and timeout detection logic, providing a balance of performance and reliability.

Transmit Engine (uart_tx)

The `uart_tx` module serializes parallel data for transmission, maintaining timing accuracy and signal integrity. It generates frames with selectable start, parity, and stop bits to match protocol requirements and legacy compatibility. Baud rate alignment uses 16x oversampling for precise timing. Flow control honors CTS signals to avoid overrunning the receiver. It also supports break signal generation for protocol control or device reset.

Receive Engine (uart_rx)

The `uart_rx` module deserializes incoming data, maintaining integrity and providing error detection. It detects start bits, validates stop bits, and checks parity where enabled, reporting errors for diagnostic use. Error detection covers framing errors, parity errors, and break conditions. Sampling uses 16x oversampling with majority voting for noise immunity. Timeout detection flags when reception exceeds 4 character periods.

Data Flow Architecture

Transmit Data Flow

1. **CPU Write:** Software writes data to THR or FIFO
2. **FIFO Buffering:** Data goes into transmit FIFO
3. **Serialization:** Transmit engine converts to serial
4. **Output:** Data is sent via `tx_o`

Receive Data Flow

1. **Input Sampling:** Receive engine samples `rx_i` at 16x baud rate
2. **Frame Processing:** Detects start bit, extracts data, checks parity and stop bit
3. **FIFO Storage:** Stores valid data in receive FIFO with error flags
4. **CPU Read:** Software reads data from RBR or FIFO

When DLAB (Divisor Latch Access Bit) = 0: - Address 0x0: RBR (read), THR (write) - Address 0x4: IER - Standard UART

16550 register layout is active

Divisor Latch Access Mode

When DLAB = 1: - Address 0x0: DLL (Divisor Latch Low) - Address 0x4: DLM (Divisor Latch High) - Used for baud rate divisor programming

Clock Domain Considerations

UART 16550 operates in a single system clock domain (`clk_i`), with all external asynchronous inputs synchronized to prevent metastability. This simplifies integration and maintains reliable timing.

Modem control inputs (CTS, DSR, RI, DCD) are synchronized to avoid errors. Serial input sampling uses 16x oversampling and majority voting, which also helps with synchronization. Reset is asserted asynchronously for immediate response but deasserts synchronously for safe recovery and predictable operation.

Error Handling Architecture

The design includes thorough error detection and reporting:

FIFO Parity Protection

FIFO parity protection covers stored data and pointers, detecting single-bit errors and maintaining integrity in FIFO control structures. Errors generate interrupts and error outputs for quick response by system software.

Serial Communication Errors

Table 491. Serial Communication Errors

Error Type	Description
Framing Errors	Detection of invalid stop bits
Parity Errors	Validation of received parity bits (when enabled)
Overflow Errors	Detection of FIFO overflow conditions
Break Detection	Recognition of extended low periods indicating break signals

Interface

Configuration Parameters

The UART 16550 module provides configurable parameters to customize FIFO sizes according to system requirements.

Table 492. Module Parameters

Parameter	Type	Default	Description
<code>TX_FIFO_DEPTH</code>	<code>int unsigned</code>	16	Transmitter FIFO depth. Must be a power of 2 between 4 and 4096 (inclusive). Determines the number of characters that can be buffered for transmission.
<code>RX_FIFO_DEPTH</code>	<code>int unsigned</code>	16	Receiver FIFO depth. Must be a power of 2 between 4 and 4096 (inclusive). Determines the number of characters that can be buffered for reception.

Signal Interface

Global Interface

Table 493. Global Signals

Signal Name	Width	Direction	Active Level	Description
clk_i	1	Input	Rising Edge	System clock. All synchronous operations are performed on the rising edge of this clock.
rst_ni	1	Input	Low	System reset. Asynchronous assertion and synchronous deassertion. When asserted, all registers are reset to their default values and FIFOs are cleared.

UART Serial Interface

Table 494. UART Serial Signals

Signal Name	Width	Direction	Active Level	Description
rx_i	1	Input	High	UART receive input. Asynchronous serial data input. Idle state is high (logic 1).
tx_o	1	Output	High	UART transmit output. Serial data output. Idle state is high (logic 1).

Modem Control Interface

The modem control interface provides full RS-232 modem signaling compatibility with hardware flow control support.

Table 495. Modem Control Input Signals

Signal Name	Width	Direction	Active Level	Description
cts_ni	1	Input	Low	Clear To Send. Asynchronous modem input. When asserted (low), indicates remote device is ready to receive data. Used for hardware flow control.
dsr_ni	1	Input	Low	Data Set Ready. Asynchronous modem input. Indicates remote modem is connected and ready.
ri_ni	1	Input	Low	Ring Indicator. Asynchronous modem input. Indicates incoming call on modem connection.
dcd_ni	1	Input	Low	Data Carrier Detect. Asynchronous modem input. Indicates carrier signal detected on modem connection.

Table 496. Modem Control Output Signals

Signal Name	Width	Direction	Active Level	Description
rts_no	1	Output	Low	Request To Send. Modem control output. When asserted (low), indicates this device is ready to receive data. Used for hardware flow control.
dtr_no	1	Output	Low	Data Terminal Ready. Modem control output. Indicates this device is connected and ready for communication.
out1_no	1	Output	Low	General-purpose output 1. User-defined modem control output controlled by MCR register.
out2_no	1	Output	Low	General-purpose output 2. User-defined modem control output controlled by MCR register. Often used for interrupt enable in some systems.

DMA Interface

The DMA interface provides signals for Direct Memory Access controllers to manage data transfers automatically.

Table 497. DMA Interface Signals

Signal Name	Width	Direction	Active Level	Description
<code>rxrdy_o</code>	1	Output	High	Receiver ready. Indicates received data is available for reading. Behavior depends on DMA mode (immediate or hysteresis).
<code>txrdy_o</code>	1	Output	High	Transmitter ready. Indicates transmitter is ready to accept new data. Behavior depends on DMA mode (immediate or hysteresis).

Error Interface

Table 498. Error Interface Signals

Signal Name	Width	Direction	Active Level	Description
<code>err_o</code>	1	Output	High	FIFO parity error. Indicates parity error detected in FIFO data or pointer structures. This is separate from received character parity errors.

Interrupt Interface

Table 499. Interrupt Interface Signals

Signal Name	Width	Direction	Active Level	Description
<code>irq_o</code>	1	Output	High	Interrupt request. Asserted when any enabled interrupt condition occurs. Six priority levels supported with automatic prioritization.

Interface Timing Considerations

Clock Domain

All interface signals operate in a single clock domain referenced to `clk_i`. The following timing considerations apply:

- **AXI4-Lite Protocol:** All AXI4-Lite signals must meet setup and hold time requirements relative to `clk_i`
- **Asynchronous Inputs:** `rx_i` and modem control inputs (`cts_ni`, `dscr_ni`, `ri_ni`, `dcd_ni`) are asynchronous and are synchronized internally
- **Reset Requirements:** `rst_ni` should be asserted asynchronously and deasserted synchronously relative to `clk_i`

Serial Interface Timing

- **Baud Rate:** Serial data rate is determined by the divisor latch registers (DLL/DLM) and uses 16x oversampling
- **Maximum Baud Rate:** Limited to $\text{system_clock_frequency} / 16$
- **Data Frame:** Configurable start bit (always 1), 5-8 data bits, optional parity bit, 1-2 stop bits

Protocol Support

The interface supports standard UART communication protocols:

- **RS-232:** Full modem control and serial data signaling
- **RS-485:** Can be used with external transceivers (using RTS for direction control)
- **Industrial Protocols:** Compatible with Modbus RTU and other industrial serial protocols
- **Debug Interfaces:** Suitable for system debug and console applications

The comprehensive interface design ensures compatibility with a wide range of serial communication requirements while maintaining the flexibility needed for diverse embedded system applications.

Memory Map

Overview

The UART 16550 register map follows the industry-standard 16550 UART layout with additional extended registers. The register space spans 64 bytes (6-bit address) with 32-bit aligned registers. Register access behavior depends on the DLAB (Divisor Latch Access Bit) setting in the Line Control Register (LCR).

The register map uses a dynamic addressing scheme where certain register addresses map to different registers based on the DLAB bit setting, maintaining compatibility with the original 16550 UART while providing access to baud rate configuration registers.

Address Map Summary

Table 500. Register Address Map

Offset	Register Name	Access	Description
0x00	RBR/THR	R/W	Receiver Buffer Register (Read) / Transmitter Holding Register (Write) when DLAB=0
0x00	DLL	RW	Divisor Latch Low Register when DLAB=1
0x04	IER	RW	Interrupt Enable Register when DLAB=0
0x04	DLM	RW	Divisor Latch High Register when DLAB=1
0x08	IIR/FCR	R/W	Interrupt Identification Register (Read) / FIFO Control Register (Write)
0x0C	LCR	RW	Line Control Register
0x10	MCR	RW	Modem Control Register
0x14	LSR	R	Line Status Register
0x18	MSR	R	Modem Status Register
0x1C	SCR	RW	Scratch Register
0x20	ECR	RW	Extended Control Register
0x24	ITR	RW	Interrupt Test Register

Detailed Register Map

Table 501. Register Map

Address	Name	Access	Description
0x0	DLL	RW	Divisor Latch Least Significant Byte Register
0x4	DLM	RW	Divisor Latch Most Significant Byte Register

Table 502. DLL Register

Bits	Field	Access	Reset	Description
31:8	Reserved	-	-	Reserved
7:0	DLL	RW	0x0	Divisor Latch Least Significant Byte. Baud rate is calculated as $\text{baud_rate} = \text{system_clock_frequency} / (16 * (\text{divisor} + 1))$. When the divisor is set to 0, the transmitter and receiver logic are disabled.

Table 503. DLM Register

Bits	Field	Access	Reset	Description
31:8	Reserved	-	-	Reserved
7:0	DLM	RW	0x0	Divisor Latch Most Significant Byte. Baud rate is calculated as $\text{baud_rate} = \text{system_clock_frequency} / (16 * (\text{divisor} + 1))$. When the divisor is set to 0, the transmitter and receiver logic are disabled.

Programming

Baud Rate Configuration

To configure the baud rate:

1. Set DLAB=1 in LCR register (0x0C)
2. Calculate divisor: $\text{divisor} = \text{clock_frequency} / (16 * \text{desired_baud_rate})$
3. Write low byte of divisor to DLL (0x00)
4. Write high byte of divisor to DLM (0x04)
5. Clear DLAB=0 in LCR register to resume normal operation

Example for 115200 baud with 50MHz clock: $\text{divisor} = 50000000 / (16 * 115200) = 27$ (0x001B) - Write 0x1B to DLL - Write 0x00 to DLM

FIFO Configuration

To enable and configure FIFOs:

1. Write to FCR register (0x08): Set FIFOEN=1
2. Configure receive trigger level in RXTRIG field
3. Reset FIFOs if needed using RFIFORST and TFIFORST bits
4. Select DMA mode if using DMA transfers

Interrupt Configuration

To configure interrupts:

1. Enable desired interrupt sources in IER register (0x04)
2. Configure OUT2 in MCR register if needed for interrupt enable
3. Read IIR register (0x08) to identify pending interrupts
4. Service interrupts by reading appropriate status/data registers

The memory map provides comprehensive control over all UART 16550 functions while maintaining backward compatibility with the original 16550 standard. The dynamic register mapping through DLAB provides efficient access to both normal operation and baud rate configuration registers.

5.10.5. Log Engine

Overview

The OCA Log Engine is a DMA-based logging system that efficiently transfers log messages from a configured memory region to a UART interface. It provides a scalable solution for system-level logging with hardware acceleration, reducing CPU overhead for log transmission. The engine supports up to 16 independent log entries with fair round-robin arbitration, making it suitable for multi-threaded or multi-core logging scenarios.

Description and Features

The Log Engine features a multi-interface architecture, combining a 32-bit AXI4-Lite register interface with both 64-bit log fetch and 32-bit log write AXI4-Lite interfaces for efficient data movement. Its design scales to manage up to 16 independent log entries, each with flexible, configurable memory regions and 64-bit addressing.

Round-robin arbitration ensures fair scheduling among logs, and hardware integration with the UART transmitter—including flow control—streamlines log data output. Error conditions in log fetch or write operations are tracked via a comprehensive interrupt system for robust monitoring.

Architecture

Overview

The OCA Log Engine is a DMA-powered logging system that transfers log messages from memory to a UART interface. It supports up to 16 log entries with fair round-robin arbitration for efficient multi-source logging.

Functional Blocks

Control and Status Registers (CSRs)

CSRs, accessed via AXI4-Lite, configure the engine and report status. Key registers control global enable/disable (CTRL), region size/alignment (LOG_REGION_SIZE), 64-bit base address (LOG_REGION_ADDR), UART target (LOG_WRITE_ADDR), and per-log-entry control and length (LOG_CTRL[0:15]). Interrupt registers (INTR_STATUS/ENABLE/TEST) enable robust error detection and error recovery.

Round Robin Arbiter

A `prim_arbiter_tree`-based round robin arbiter ensures fair, starvation-free access to DMA for all 16 log entries. It systematically selects among entries with non-zero lengths, preventing monopolization and balancing throughput regardless of data rates across logs. Tight integration with the fetch FSM ensures smooth flow.

Log Fetch FSM

The fetch FSM coordinates DMA reads based on arbitration results and FIFO space. It cycles through idle, request, and wait states: checking for ready entries and FIFO space, issuing AXI4-Lite read requests, and handling responses with word counters. Addresses are computed as `log_region_addr + (log_region_size/16) * log_index + word_offset`, supporting efficient, aligned memory access. Progress tracking and FIFO handoff ensure reliable completion and seamless operation.

RDATA FIFO

A configurable-depth (default 4), synchronous 64-bit FIFO decouples memory reads from UART writes, smoothing timing mismatches. All-zero outputs when empty help security. Ready/valid handshakes on both sides ensure safe, lossless transfer through variable system conditions.

Log Write FSM

This FSM manages conversion from 64-bit log words to 8-bit UART writes. It only advances when UART is ready and FIFO data is available, sequencing through idle, request, and wait states. Byte pointers extract one byte at a time; FIFO is accessed every eight bytes. A completion signal updates registers, enabling accurate monitoring and coordination.

Data Flow and Control Path

Data Flow

1. Software configures memory region, UART address, and enables engine.
2. Non-zero LOG_CTRL[i] length triggers log transfer.
3. Arbiter selects a log entry.
4. Fetch FSM performs memory reads.
5. Data is buffered in RDATA FIFO.
6. Write FSM sends bytes to UART.
7. Transfer completion clears the length register.

Control Path

- Global enable gates operation.
- Fetch/write FSMs are coordinated via FIFO ready/valid signals.
- AXI4-Lite errors trigger interrupts.
- Completion updates registers and advances arbitration.

Clocking and Reset Methodology

The engine runs in one clock domain (**clk_i**) shared by all logic and AXI4-Lite interfaces.

- Reset (**rst_ni**) is active-low async, sync deassertion.
- Global reset clears FSMs, counters, and logic.
- FIFO can be cleared independently when the engine is disabled.
- On reset: FSMs idle, counters zeroed, FIFO flushed, registers preserved (except control fields).

Memory Interface Protocol

Log Fetch: 64-bit AXI4-Lite read, single-beat transfers, errors signaled via LOG_FETCH_ERR.

Log Write: 32-bit AXI4-Lite write, single byte accesses using strobes, errors signaled via LOG_WRITE_ERR.

Performance Characteristics

- **Fetch Bandwidth:** Limited by AXI4-Lite single transfers.
- **Write Throughput:** Determined by UART ready signal.
- **Latency:** Minimal, due to light buffering.
- **Fairness:** Maintained by round-robin selection.

Design Trade-offs

- *Memory Simplicity vs. Efficiency*: Fixed equal partitioning eases logic at expense of flexibility.
- *Throughput vs. Area*: Single-beat transfers are simple but limit bandwidth; FIFO depth is tunable.
- *Robustness vs. Complexity*: Comprehensive error checking improves reliability at increased logic cost.

Interface

The OCA Log Engine provides multiple AXI4-Lite interfaces for register access, memory fetching, and UART communication.

Configuration Parameters

Parameter	Default	Description
NUM_LOG_ENTRIES	16	Number of independent log entries
MAX_LOG_SIZE	Configurable	Maximum log region size (must be multiple of 16)
ADDR_WIDTH	64	Address bus width for log region
DATA_WIDTH	32	Data bus width

Signal Interfaces

Global Interface

Table 504. Global Interface Signals

Signal Name	Width	Direction	Active	Description
clk_i	1	Input	Rising	System clock. All interfaces operate synchronously to this clock.
rst_ni	1	Input	Low	System reset. Asynchronous assertion and synchronous deassertion. Resets all state machines, counters, and FIFO to initial state.

DMA Interface

The DMA interface provides flow control coordination with the UART transmitter.

Table 505. DMA Interface Signals

Signal Name	Width	Direction	Active	Description
uart_tx_ready_i	1	Input	High	UART ready to accept character(s) for transmission. Used as flow control to prevent UART FIFO overflow.

Flow Control Operation:

- The log write FSM waits for `uart_tx_ready_i` assertion before initiating write transfers
- This prevents overwhelming the UART transmitter and maintains data integrity
- Ready signal typically indicates UART FIFO has space for additional characters

Interrupt Interface

The interrupt interface provides error signaling to the system.

Table 506. Interrupt Interface Signals

Signal Name	Width	Direction	Active	Description
irq_0	1	Output	High	Interrupt request. Asserted when any enabled interrupt condition occurs.

Interrupt Sources:

- **LOG_FETCH_ERR:** AXI4-Lite log fetch bus returned error response
- **LOG_WRITE_ERR:** AXI4-Lite log write bus returned error response

Interrupt Control: * Individual enable bits in INTR_ENABLE register * Status bits in INTR_STATUS register (write-1-to-clear) * Test capability via INTR_TEST register

Memory Map

Overview

The Log Engine provides a comprehensive register interface for configuration, control, and status monitoring. All registers are accessible via a 32-bit AXI4-Lite interface and follow standard read/write access patterns. The register map is organized into logical groups for ease of use and programming.

Register Address Map

Table 507. Register Address Map

Address	Name	Access	Description
0x00	CTRL	RW	Control Register - Global enable/disable
0x04	LOG_REGION_SIZE	RW	Log Region Size Register - Memory region size in bytes
0x08	LOG_REGION_ADDR (Low)	RW	Log Region Address Register (64-bit, low word)
0x0C	LOG_REGION_ADDR (High)	RW	Log Region Address Register (64-bit, high word)
0x10	LOG_WRITE_ADDR	RW	Log Write Address Register - UART target address
0x14	INTR_STATUS	RW	Interrupt Status Register - Error status (write-1-to-clear)
0x18	INTR_ENABLE	RW	Interrupt Enable Register - Error interrupt enables
0x1C	INTR_TEST	W	Interrupt Test Register - Force interrupts for testing
0x40	LOG_CTRL[0]	RW	Log Control Register 0 - Entry 0 length control
0x44	LOG_CTRL[1]	RW	Log Control Register 1 - Entry 1 length control
...	...	...	...
0x7C	LOG_CTRL[15]	RW	Log Control Register 15 - Entry 15 length control

Detailed Register Map

NOTE

Register Arrays: This register map contains the following register arrays:

- **LOG_CTRL[16]:** 16 registers
 - Base Address: 0x40
 - Address Increment: 0x4 per register

Table 508. Register Map

Address	Name	Access	Description
0x0	CTRL	RW	Control Register
0x4	LOG_REGION_SIZE	RW	Log Region Size Register
0x8	LOG_REGION_ADDR	RW	Log Region Address Register
0x10	LOG_WRITE_ADDR	RW	Log Write Address Register
0x14	INTR_STATUS	RW	Interrupt Status Register
0x18	INTR_ENABLE	RW	Interrupt Enable Register
0x1C	INTR_TEST	W	Interrupt Test Register
0x40 - 0x7C	LOG_CTRL[16]	RW	Log Control Register

Table 509. CTRL Register

Bits	Field	Access	Reset	Description
31:1	Reserved	-	-	Reserved
0	EN	RW	0x0	Log Engine Enable. When set, enables the Log Engine. When unset, disables the Log Engine and resets all FSMs, flops, and FIFOs.

Table 510. LOG_REGION_SIZE Register

Bits	Field	Access	Reset	Description
31:20	Reserved	-	-	Reserved
19:0	LOG_REGION_SIZE	RW	0x0	Log Region Size. Specified in bytes. Needs to be a multiple of 16.

Table 511. LOG_REGION_ADDR Register

Bits	Field	Access	Reset	Description
63:32	LOG_REGION_ADDR_HI	RW	0x0	Log Region Start Address High.
31:0	LOG_REGION_ADDR_LO	RW	0x0	Log Region Start Address Low.

Table 512. LOG_WRITE_ADDR Register

Bits	Field	Access	Reset	Description
31:0	LOG_WRITE_ADDR	RW	0x0	Log Write Address. This is the address where the log data is written to.

Table 513. INTR_STATUS Register

Bits	Field	Access	Reset	Description
31:5	Reserved	-	-	Reserved
4	LOG_WRITE_ERR	RW	0x0	Log Write Error Interrupt. Asserted when the bus for writing log data returns an error.
3:1	Reserved	-	-	Reserved
0	LOG_FETCH_ERR	RW	0x0	Log Fetch Error Interrupt. Asserted when the bus for fetching log data returns an error.

Table 514. INTR_ENABLE Register

Bits	Field	Access	Reset	Description
31:5	Reserved	-	-	Reserved
4	LOG_WRITE_ERR	RW	0x0	Log Write Error Interrupt Enable.

Bits	Field	Access	Reset	Description
3:1	Reserved	-	-	Reserved
0	LOG_FETCH_ERR	RW	0x0	Log Fetch Error Interrupt Enable.

Table 515. INTR_TEST Register

Bits	Field	Access	Reset	Description
31:5	Reserved	-	-	Reserved
4	LOG_WRITE_ERR	W	0x0	Log Write Error Interrupt Test. Writing 1 forces the interrupt.
3:1	Reserved	-	-	Reserved
0	LOG_FETCH_ERR	W	0x0	Log Fetch Error Interrupt Test. Writing 1 forces the interrupt.

Table 516. LOG_CTRL[16] Register

Bits	Field	Access	Reset	Description
31:16	Reserved	-	-	Reserved
15:0	LOG_LEN	RW	0x0	Log Length. Writing a nonzero value representing the log length in bytes to this field starts the log transfer process. There are 16 copies of this register, each representing a log entry. The log entries go through round robin arbitration.

Programming

Simple Log Transfer

```
// Initialize log engine for 4KB region at 0x10000000
// UART at 0x40001000

#define LOG_ENGINE_BASE 0x50000000
#define LOG_REGION_ADDR 0x10000000
#define LOG_REGION_SIZE 0x1000
#define UART_TX_ADDR    0x40001000

// Disable and configure
write32(LOG_ENGINE_BASE + 0x00, 0);          // Disable
write32(LOG_ENGINE_BASE + 0x04, LOG_REGION_SIZE);
write32(LOG_ENGINE_BASE + 0x08, LOG_REGION_ADDR);
write32(LOG_ENGINE_BASE + 0x0C, 0);          // High address
write32(LOG_ENGINE_BASE + 0x10, UART_TX_ADDR);
write32(LOG_ENGINE_BASE + 0x18, 0x11);       // Enable error interrupts
write32(LOG_ENGINE_BASE + 0x00, 1);          // Enable

// Send 100-byte log via entry 0
write32(LOG_ENGINE_BASE + 0x40, 100);

// Wait for completion
while (read32(LOG_ENGINE_BASE + 0x40) != 0);
```

Multiple Entry Management

```
// Send logs via multiple entries simultaneously
write32(LOG_ENGINE_BASE + 0x40, 50); // Entry 0: 50 bytes
write32(LOG_ENGINE_BASE + 0x44, 75); // Entry 1: 75 bytes
write32(LOG_ENGINE_BASE + 0x48, 120); // Entry 2: 120 bytes

// Check completion status
for (int i = 0; i < 3; i++) {
    uint32_t len = read32(LOG_ENGINE_BASE + 0x40 + (i * 4));
    if (len == 0) {
        printf("Entry %d completed\n", i);
    } else {
        printf("Entry %d: %d bytes remaining\n", i, len);
    }
}
```

5.10.6. I2C Controller

Overview

The OCA I2C Interface Component is a comprehensive I2C/SMBus/PMBus communication controller that provides full-featured support for both controller (master) and target (slave) operation modes. Based on industry standards and featuring robust error handling, configurable FIFOs, and extensive interrupt support, this component enables reliable inter-chip communication in complex system-on-chip designs.

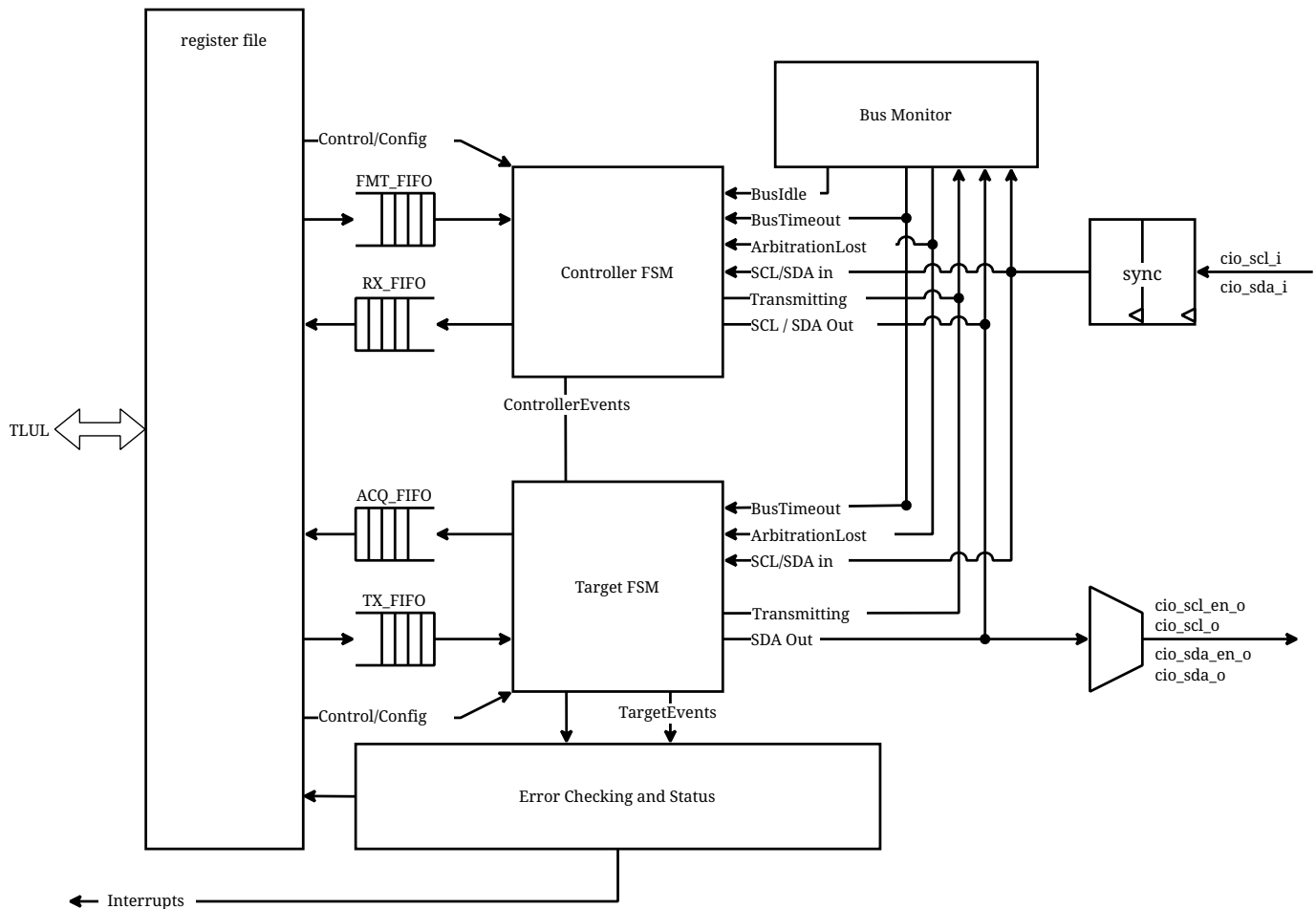
Description and Features

- **Multi-Protocol Support:** Full compliance with I2C Rev. 7.0, SMBus Rev. 3.2, and PMBus Rev. 1.4 specifications
- **Dual Operating Modes:** Configurable as controller-only, target-only, controller-target, or bus monitor
- **High-Speed Operation:** Support for Standard-mode (100 kbaud), Fast-mode (400 kbaud), and Fast-mode Plus (1 Mbaud)
- **Advanced Controller Features:** Clock stretching support, automatic ACK control, multi-controller arbitration
- **Flexible Target Capabilities:** Automatic clock stretching, programmable ACK control, broadcast transfer support
- **Configurable FIFO Architecture:** Independent TX/RX FIFO depths for both controller and target modes

Architecture

The I2C Interface Component is a dual-mode I2C/SMBus/PMBus communication controller supporting both controller (master) and target (slave) operations. It is based on the [OpenTitan I2C controller](#). The architecture provides high reliability with configurable timing, robust error handling, and AXI4-Lite integration.

Block Diagram



Functional Description

I2C IP is a controller-target combo that can function as an I2C controller and/or an I2C target. These functional modules are enabled at runtime by setting the register fields **CTRL.ENABLEHOST** and **CTRL.ENABLETARGET**. If the Controller Module is to be used in a multi-controller environment, **CTRL.MULTI_CONTROLLER_MONITOR_EN** would also need to be set. Note the ordering requirements in the register description for multi-controller configurations.

The SCL and SDA outputs from the block combine the SCL and SDA outputs from the Controller Module and Target Module. The modules operate as though they were independent devices on the same I2C bus, with any logic low value taking priority.

On the input path, the IP's clock samples SCL and SDA after a 2-flop synchronizer. However, the IP's input pins do not reflect the start point for the full input path delay, since pads and nets from I/O buffers also contribute. The RTL parameter **InputDelayCycles** must be set to the maximum number of cycles of delay from the IP's output pin driving SCL or SDA to the IP's corresponding input pin sensing the transition. **InputDelayCycles** can exclude the rise time that would be captured by **TIMING1.T_R**. The reference point for **InputDelayCycles** is the clock edge after the launch clock edge. In other words, a value of 0 means that the IP's 2-flop synchronizer would sample the IP's own outputs within one clock cycle (if the external pad driver could switch instantaneously).

Functional Blocks

Control and Status Registers (CSRs)

32-bit AXI4-Lite interface with 8-bit address space:

- **Control Registers:** Mode configuration, timing parameters, FIFO thresholds, feature enables
- **Status Registers:** Transaction status, FIFO levels, error flags, protocol state
- **Data Registers:** FIFO access ports for TX/RX data in controller and target modes
- **Interrupt Registers:** Status, enable, and test capabilities
- **Timing Registers:** Configurable parameters for I2C speed modes (Standard, Fast, Fast-Plus)

I2C Core Processing Engine

The core engine manages all I2C protocol operations through integrated state machines:

Controller FSM: Manages controller-initiated transactions including address transmission, data transfer, and bus arbitration. Handles START/STOP generation, ACK/NACK processing, and automatic retry mechanisms.

The Controller Module implements all mandatory controller features of the I2C protocol for multi-controller configurations, including clock synchronization and bus arbitration. It also implements support for handling clock stretching.

The state-machine-controlled Controller Module allows for higher-speed operation with less frequent software interaction. In this module, the I2C pins are controlled by the state machine, which in turn is controlled by a sequence of formatting indicators. These indicators determine:

- The sequence of bytes which should be transmitted on the SDA and SCL pins.
- The periods between transmitted bytes when the state-machine should stop transmission and instead read back a fixed number of bytes.
- Which bytes should be preceded by a START symbol.
- Which bytes should be followed by a STOP symbol.

The format indicator consists of 13-bits. That is of one single format byte (entered into the format FIFO through `FDATA.FBYTE`), and five 1-bit flags (entered into the format FIFO through registers `FDATA.READB`, `FDATA.RCONT`, `FDATA.START`, `FDATA.STOP` and `FDATA.NAKOK`)

The I2C Control Module reads each format indicator from the head of FMT_FIFO and processes them in turn. If none of the flags are set for the format indicator, the I2C FSM simply transmits the format byte onto the SCL and SDA pins according to the specification, waits for acknowledgement, and then proceeds to the next format indicator. The format flags modulate the behavior as follows:

- **READB** (corresponds to `FDATA.READB`): Signifies the format byte (`FDATA.FBYTE`) should be treated as an unsigned number, R, and prompts the state machine to read R bytes from the target device. Bytes read from the bus are inserted into the RX FIFO where they can be accessed by software. A value of 0 is treated as a read of 256 bytes. To read a larger byte stream, multiple 256-byte reads can be chained together using the **RCONT** flag.
- **RCONT** (corresponds to FIFO inputs `FDATA.RCONT`, only used with **READB**):
 - If **RCONT** is set, the format byte represents part of a longer sequence of reads, allowing for reads to be chained indefinitely.
 - The **RCONT** flag indicates the final byte returned with the current read should be responded to with an ACK, allowing the target to continue sending data. (Note that the first R-1 bytes read will still be acknowledged regardless of whether **RCONT** is asserted or not.)
- **START** (corresponds to `FDATA.START`, Ignored when used with **READB**): Issue a START condition before transmitting the format byte on the bus. This flag may also be used to issue a repeated start condition.

- STOP (corresponds to `FDATA.STOP`): Issue a STOP signal after processing this current entry in the FMT FIFO. Note that this flag is not compatible with (READB & RCONT), and will cause bus conflicts.
- NAKOK (corresponds to `FDATA.NAKOK`, Not compatible with READB): Typically every byte transmitted must also receive an ACK signal, and the IP will raise an exception if no ACK is received. However, there are some I2C commands which do not require an ACK. In those cases this flag should be asserted with FBYTE indicating no ACK is expected and no interrupt should be raised if the ACK is not received.

The Controller Module may proceed through all commands in the FMT FIFO as long as no exceptional event occurs. If the Controller Module receives an unexpected NACK, a bus timeout occurs, or arbitration is lost while the Controller Module is transmitting, the module halts. The module will not continue processing commands without software intervention.

Note that a full RX FIFO does not stop command processing. Ensure the RX FIFO has enough space, else overflows can occur.

Controller NACK Handling

If the Controller Module transmits a byte and the 9th bit is a NACK from the Target/Bus, the `controller_halt` interrupt is usually asserted (modulo the effect of `FDATA.NAKOK`). If the `controller_halt` interrupt is asserted, the Controller FSM will halt until the interrupt condition has been cleared in the `CONTROLLER_EVENTS.NACK` register. Software will likely want to do one of two things in handling this irq: 1. End the current transaction by setting `CTRL.ENABLEHOST` to `10b0`, and clear the FIFOs to be ready to begin a new transaction. 2. Clear the FIFOs, then begin a new transfer without ending the transaction by adding new FDATA indicators to the FMTFIFO. In this case, the first entry written to the FMTFIFO should have `FDATA.START` set to `10b1` to create a repeated start condition.

Software should then clear the `CONTROLLER_EVENTS.NACK` to proceed with the next operation. The `controller_halt` interrupt is a CIP status type, so it doesn't clear by writing to the `INTR_STATE.CONTROLLER_HALT` bit. Instead, the `CONTROLLER_EVENTS` CSR tracks the contributing events, and all must be cleared for the interrupt to de-assert.

The Halt-on-NACK behaviour may be problematic if SW is not responsive, so there is a timeout mechanism that can automatically end the transaction by creating a STOP condition and returning to halt in an idle state. This is configured using the `HOST_NACK_HANDLER_TIMEOUT` CSR, which allows software to configure the delay before hardware will terminate the transaction. When the timeout occurs, the `CONTROLLER_EVENTS.UNHANDLED_NACK_TIMEOUT` bit will set, and this condition will also cause the `controller_halt` interrupt to assert (or remain asserted). Like the original NACK event, the timeout bit will cause the Controller Module to remain halted until it is cleared. To continue with a new transaction, software should ensure that the `controller_halt` interrupt has been handled, with all contributing event bits cleared.

When the FSM first halts due to an unexpected NACK, there is still an ongoing transaction. The Controller Module stops immediately following the (N)ACK bit's hold time after the SCL falling edge, and SCL remains asserted low. If a NACK handling timeout occurs or `CTRL.ENABLEHOST` is cleared, then the FSM will halt again after the STOP condition is sent. The STOP condition terminates the transaction, leaving the FSM halted in the "bus idle" state, with SDA and SCL released.

Bus Arbitration

In a multi-controller configuration, the Controller Module may begin transmitting at the same time as another controller. The Bus Monitor will detect when the Controller Module loses arbitration, such as when it attempts to transmit a logic high, but another device is pulling SDA low. The monitor will inform the Controller Module, and the Controller Module will abort its command and halt. The `CONTROLLER_EVENTS.ARBITRATION_LOST` bit will be set, and the `controller_halt` interrupt state will reflect that the module is halted.

From here, software must decide what to do. The FMT FIFO will have the entries that remained, but typically, this event will result in software retrying the full transaction. In that case, software would clear the FMT FIFO (and possibly the RX

FIFO), then load the FMT FIFO with the original commands, starting from the beginning of the transaction. All bits in `CONTROLLER_EVENTS` must be cleared for the Controller Module to release from the halted state.

Target FSM: Provides complete target device functionality with address recognition, automatic clock stretching, and configurable ACK responses. Supports broadcast address handling and multi-controller environments.

The Target Module implements all mandatory features for targets in multi-controller configurations, as enumerated in the I2C specification. It also supports clock stretching from that same specification.

In addition, the Target Module has the necessary mechanisms to support mandatory SMBus 3.0 features, as well as dynamically-assigned addresses using the SMBus Address Resolution Protocol. Address bytes may be configured to be always ACK'd, and the Target ACK Control feature may be used to stretch the clock until software can decide whether to ACK the incoming byte. The Target Module can also be configured to observe SMBus's maximum cumulative target clock extension time. There is no hardware support for calculating or checking the Packet Error Code, however; these values would need to be handled in software.

Target Address Registers

Every transfer involving the I2C Target Module begins with an address. The I2C Target Module supports up to two 7-bit address and 7-bit mask pairs. The Target Module accepts a transaction if the result of the bitwise AND operation performed on the transaction address sent by the controller and a mask matches the assigned address corresponding to the mask. In other words, address matching is performed only for bits where the mask is "1". Thus, with the masks set to all ones (0x7F), the target device will respond to either of the two assigned unique addresses and no other. If the mask and the assigned address both have zeros in a particular bit position, that bit will be a match regardless of the value of that bit received from the controller. Note that if, in any bit position, the mask has zero and the assigned address has one, no transaction can match and such mask/address pair is effectively disabled.

Finally, in the special case where the mask is all zeroes (0x00), then that corresponding entry is ignored and will not match. If the user wishes to have the target respond to **all** addresses, then this can be achieved by setting mask0 and mask1 to 0x01 and setting address0 to 0x00 and address1 to 0x01.

The assigned address and mask pairs are set in registers `TARGET_ID.ADDRESS0`, `TARGET_ID.MASK0`, `TARGET_ID.ADDRESS1`, and `TARGET_ID.MASK1`.

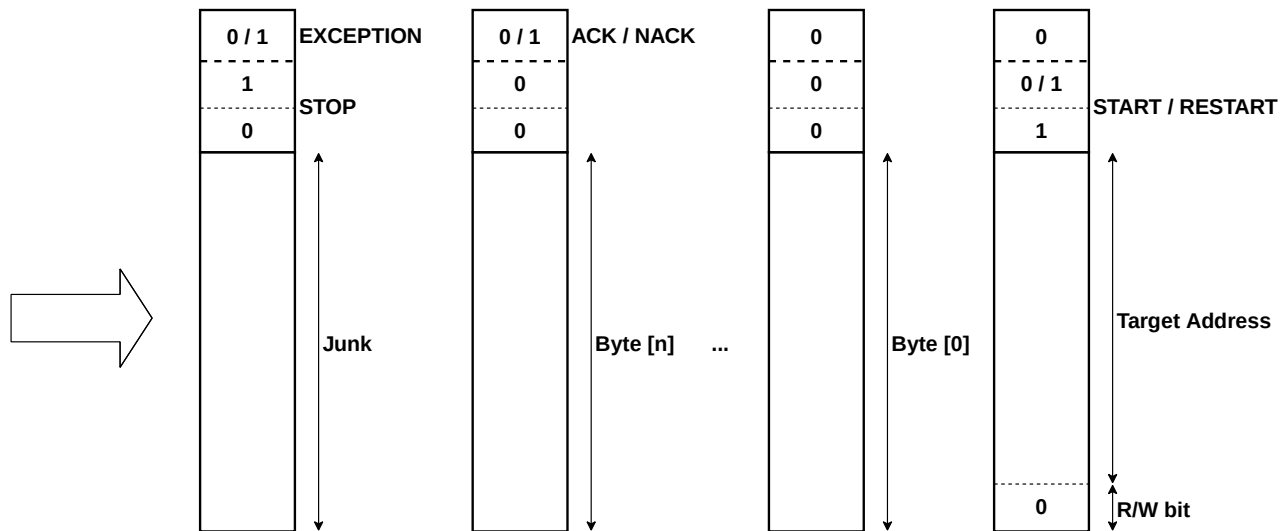
Target Transfers

When the Target Module accepts a transfer, it acquires the transfer address, read/write bit, and START signal sent by the controller and inserts them into ACQ FIFO, where they can be accessed by software. `ACQDATA` represents the ACQ FIFO output. See the register description for more information about the representation.

If the transfer is a write operation (R/W bit = 0), the target proceeds to read bytes from the bus and insert them into ACQ FIFO until the controller terminates the transfer by sending a STOP or a repeated START signal. A STOP or repeated START indicator is inserted into ACQ FIFO as the next entry following the last byte received, in which case other bits may be junk or the target address, respectively.

Note that the repeated START indicator only appears if it matches the address. Typically, this is the case, and a transaction that changes targets between transfers is strongly discouraged. If no additional transfers targeting this target's address occur, an entry representing the end of the transfer will wait for the STOP symbol.

The following diagram shows consecutive entries inserted into ACQ FIFO during a write operation, assuming no NACK conditions:

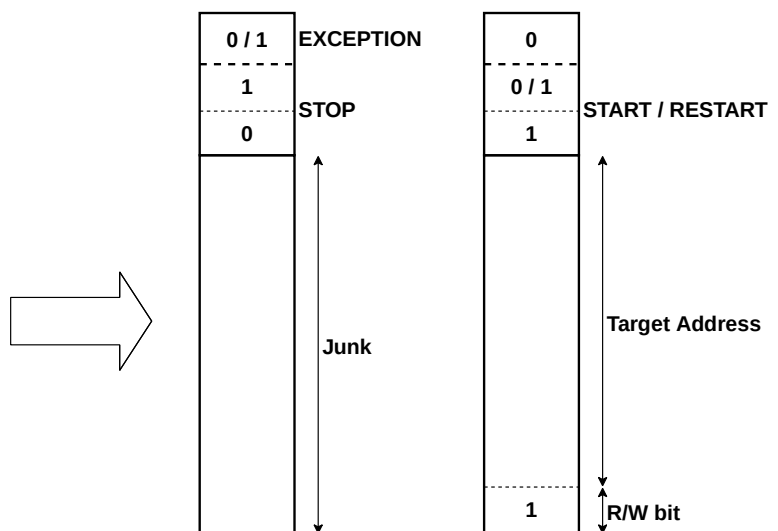


See [Target NACKs](#) for more information about NACK conditions and the effect on the operation.

If the transaction is a read operation (R/W bit = 1), the target pulls bytes out of TX FIFO and transmits them to the bus until the controller signals the end of the transfer by sending a NACK signal. Under certain conditions, the target will hold SCL low to stretch the clock and give software time to write data bytes into TX FIFO or handle the available command. The conditions include the following: * TX FIFO holding no data * the ACQ FIFO containing more than the 1 entry representing the address/command byte * **TARGET_EVENTS** holding at least one unhandled event

See [\[stretching-during-read\]](#) for more details. Note that data for TX FIFO is pushed through the register **TXDATA**.

Typically, a NACK signal is followed by a STOP or repeated START signal; the IP will raise an exception if the controller sends a STOP signal after an ACK. The ACK/NACK signal is represented in the MSB of **ACQDATA.SIGNAL**, in the same entry as the STOP or repeated START signal. For ACK signals, the value of the MSB is 0; for NACK signals, the value is 1. The following diagram shows consecutive entries inserted into ACQ FIFO during a read operation:



Target Clock Stretching

As described in the I2C specification, a target device can pause a transaction by holding SCL low. There are 3 cases in which this design stretches the clock: * After the target receives the address but before pushing it to the ACQ FIFO * Before the (N)ACK bit completes transmission during a write transfer * After the ACK bit is sent by the controller-transmitter during a read transfer

Stretching after address read

When a target device receives a start, the address and R/W bit are written into the ACQ FIFO. If there is no space in the ACQ FIFO to receive such a write, the target stretches the clock after the ACK bit and waits for software to make space. The `acq_stretch` interrupt is generated to alert software to such a situation.

Stretching during write

If the controller tries to write a data byte into the ACQ FIFO when there is no available space, the clock is stretched before the rising edge of SCL on the ACK bit. In addition, the Target Module may stretch if Target ACK Control Mode is enabled and the Auto ACK Counter has reached 0. See the [Target ACK Control Mode](#) section for more about it.

The `acq_stretch` interrupt is generated to alert software to these situations.

Stretching during read

When a target device receives a start and read command, it may stretch the clock for any of the following reasons: * If there is no data available to be sent back (TX FIFO empty case), the target stretches the clock until data is made available by software. * If there is more than 1 entry in the ACQ FIFO. Having more than 1 entry in the ACQ FIFO suggests there is potentially an unhandled condition (STOP / RESTART) or an unhandled command (START) that requires software intervention before the read can proceed. * If there is an unhandled event in `TARGET_EVENTS`.

The `tx_stretch` interrupt is generated to alert software to such a situation.

Target NACKs

When the Target Module replies with a NACK during the (N)ACK phase, it indicates that it is unable to accept the byte or command. The NACK response is sticky until the end of the transaction. In other words, once the Target Module issues its first NACK, it will NACK all subsequent bytes in the transaction, with one caveat.

Assuming `CTRL.NACK_ADDR_AFTER_TIMEOUT` is 0, a matching address will always elicit an ACK response, which is required by SMBus. However, if the Target Module had already issued a NACK in this transaction, that byte will not be pushed to the ACQ FIFO. After committing to NACKing the transaction, the Target Module will leave SDA released high for all bits except the ACK of a matching address byte.

The decision to NACK will show up in the ACQ FIFO by the end of the first transaction to be NACK'd. The final space in the ACQ FIFO is reserved for the STOP condition of a transaction, so in all cases where a `START` signal reached the ACQ FIFO, that first NACK'd transaction will always end with a `NACK_STOP` signal. For write transactions, the particular NACK'd data byte will also be in the ACQ FIFO if there was space.

If software does not handle any persisting issues that caused NACKing and the next transaction leads to the Target Module NACKing again, all associated data will be dropped while the ACQ FIFO is full.

The decision to NACK the transaction can occur for any of the following reasons: * A timeout after stretching the clock If enabled by `TARGET_TIMEOUT_CTRL` * Bus arbitration for the target was lost, such as during SMBus ARP broadcast

commands * The controller's SCL low period is detected to be too short for a response * Software directed the Target Module to NACK Software has some control if ACK Control Mode is enabled

Target ACK Control Mode

ACK Control Mode is a feature that allows software to take control of ACK/NACK decisions during a Write transfer. When ACK Control Mode is disabled in `CTRL.ACK_CTRL_EN`, the Target module will automatically ACK any incoming data byte, so long as there is sufficient space in the ACQ FIFO. If there isn't enough space for the data byte and a STOP, the Target module will stretch the clock until space is made available (or a stretch timeout occurs). However, if ACK Control Mode is enabled, the `TARGET_ACK_CTRL` CSR can also cause the Target module to stretch the clock at the (N)ACK phase.

In ACK Control Mode, an Auto ACK Counter influences whether the Target module automatically ACKs a data byte when there is space in the ACQ FIFO. The counter begins at 0 for every transfer, and it decrements for every data byte that is ACK'd. In addition, this counter only applies to the bytes in a Write transfer that follow the address byte. When a transfer reaches the (N)ACK phase and the `TARGET_ACK_CTRL.NBYTES` CSR is 0, the Target module stretches the clock and awaits instructions from software, up to the stretch timeout. Note that because the Auto ACK Counter begins every transfer at 0, the first data byte will always stretch the clock.

To release SCL before the timeout, software can choose whether to ACK or NACK the byte. To ACK the byte, software would program the Auto ACK Counter to a greater number, and if the value programmed is greater than 1, it would automatically ACK subsequent bytes as well. Even with a nonzero count, though, the Target module could still stop and stretch the clock if the ACQ FIFO doesn't have sufficient space. Otherwise, software can explicitly NACK that byte and the rest of the transaction by writing a 1 to `TARGET_ACK_CTRL.NACK`. If the Target module is not stretching the clock, writes to `TARGET_ACK_CTRL` are ignored.

With the data byte pending in the (N)ACK phase, it hasn't been written to the ACQ FIFO yet. Instead, software can read the `ACQ_FIFO_NEXT_DATA` CSR to decide what to do. Note that after software indicates its decision, this pending byte will still enter the ACQ FIFO if there is space, accompanied by the metadata reflecting whether it was ACK'd or NACK'd.

This feature is intended to support protocols that require mid-transfer NACK decisions based on the current data transferred (e.g. as in SMBus).

FIFO Management System

FIFO	Controller Mode	Target Mode
TX FIFO	64 entries (addresses, write data)	64 entries (read response data)
RX FIFO	64 entries (received data)	268 entries (addresses, write data)
Features	Configurable depth, threshold interrupts, DMA ready signals, overflow/underflow protection	Configurable depth, threshold interrupts, DMA ready signals, overflow/underflow protection

Data Flow and Control Path

Data Flow Architecture

Controller Mode Data Flow:

1. **Command Phase:** Software writes address and control information to FMT FIFO
2. **Transmission Phase:** Controller FSM reads from FMT FIFO and drives I2C bus
3. **Reception Phase:** Received data is stored in RX FIFO for software retrieval
4. **Completion Phase:** Transaction completion triggers interrupt and status updates

Target Mode Data Flow:

- 1. **Address Recognition:** Target FSM monitors bus for matching addresses
- 2. **Data Reception:** Incoming data is stored in ACQ FIFO with address information
- 3. **Data Transmission:** During read operations, data is sourced from TX FIFO
- 4. **Flow Control:** Automatic clock stretching manages timing constraints

Control Path Architecture

The control path coordinates operation between multiple state machines:

- **Register Interface:** Translates AXI4-Lite transactions to internal control signals
- **Mode Control:** Switches between controller, target, and monitor modes
- **FIFO Control:** Manages data flow and threshold-based signaling
- **Interrupt Control:** Aggregates status from multiple sources
- **Error Handling:** Detects and reports protocol violations and timeout conditions

State Machine Architecture

Controller State Machine

The controller FSM implements the complete I2C controller protocol:

Table 517. Controller FSM States

State	Description
IDLE	Waiting for FMT FIFO data or software commands
START	Generating START condition and preparing for address transmission
ADDRESS	Transmitting 7-bit address and R/W bit
ACK_ADDRESS	Waiting for and processing address acknowledgment
WRITE_DATA	Transmitting data bytes for write operations
ACK_WRITE	Processing acknowledgments for transmitted data
READ_DATA	Receiving data bytes for read operations
ACK_READ	Generating ACK/NACK for received data
STOP	Generating STOP condition and completing transaction
ARBITRATION	Handling multi-controller arbitration scenarios

Signal Conditioning and Synchronization

Input Signal Processing

To ensure reliable and noise-immune operation, all external I2C signals are conditioned through a multi-stage process. Incoming signals first undergo a configurable input delay (via the INPUT_DELAY_CYCLES parameter) to account for timing differences, followed by dual-stage synchronization to safely cross clock domains and guard against metastability.

A glitch filter rejects short noise pulses, and edge detection logic identifies precise rising and falling transitions needed

for protocol handling. Together, these mechanisms provide robust debouncing, level detection with hysteresis, and stable sampling in noisy environments, with the ability to configure delay compensation from 0 to 255 cycles.

Timing Control and Configuration

Timing Parameter Management

For Standard-mode, Fast-mode and Fast-mode Plus, the timing requirements for each transaction are detailed in Table 10 of the I2C specification (rev. 6). To claim complete compatibility at each mode, the state machine timings need to be adapted to whether there are Standard-mode, Fast-mode and Fast-mode Plus targets on the bus. Furthermore, depending on the actual capacitance of the bus, even a bus with all Fast-mode Plus capable targets may have to operate at slower speeds than 1 Mbaud. For example, the controller may need to run at lower frequencies, as discussed in Section 5.2 of the specification, but the computation of the nominal frequency will depend on timing specifications in Table 10, in this case particularly, the limits on t_{LOW} , t_{HIGH} , t_r , and t_f . Assuming no clock stretching, for a given set of these four parameters the baud rate is then given to be:

$$1/f_{\{SCL\}} = t_{\{LOW\}} + t_{\{HIGH\}} + t_{\{r\}} + t_{\{f\}}$$

Thus in order to ensure compliance with the spec in any particular configuration, software will program the I2C controller IP with explicit values for each of the following timing parameters, as defined in Figure 38 of the specification.

- t_{LOW} : set in register **TIMING0.TLOW**.
- t_{HIGH} : set in register **TIMING0.THIGH**.
- t_r : set in register **TIMING1.T_R**. (Note: The rise time cannot be explicitly controlled by internal hardware, and will be a function of the capacitance of the bus. Thus this parameter is largely budgetary, meaning that it tells the state machine how much time to wait for an RC rise.)
- t_f : set in register **TIMING1.T_F**. (Note: The fall time cannot be explicitly controlled by internal hardware, and is a function of the pin driver. Thus this parameter is also budgetary. Given that the actual fall time cannot be controlled to stay above the minimum values set in Table 10 of the specification, and so this in this regard this module currently is not strictly compliant to the I2C spec. The system design is responsible for meeting the spec's minimum fall time.)
- $t_{SU,STA}$: set in register **TIMING2.TSU_STA**
- $t_{HD,STA}$: set in register **TIMING2.THD_STA**
- $t_{SU,DAT}$: set in register **TIMING3.TSU_DAT**. Taken to be synonymous with $T_{SU,ACK}$
- $t_{HD,DAT}$: set in register **TIMING3.THD_DAT**. Taken to be synonymous with $T_{HD,ACK}$. Moreover, since the pin driver fall time is likely to be less than one clock cycle, this parameter is also taken to be synonymous with the parameters $T_{VD,DAT}$ and $T_{VD,ACK}$. This parameter controls the number of cycles after the falling edge of SCL that SDA is driven. In addition, when the IP operates as a target, the parameter specifies the required SCL high time after SDA changes to satisfy the Start and Stop symbol decoders.
- $t_{SU,STO}$: set in register **TIMING4.TSU_STO**.
- t_{BUF} : set in register **TIMING4.T_BUF**

The values programmed into the registers **TIMING0** through **TIMING4** are to be expressed in units of the input clock period. It is important that the internal clock is at least 50x the bus clock so that the proportions of the timings can be accurately captured. Note in order to ensure compliance with the I2C spec, firmware must program these registers with values within the ranges laid out in Table 10 of the specification. These values can be directly computed using DIFs given the

desired speed standard, the desired operating frequency, and the actual line capacitance. These timing parameters are then fed directly to the I2C state machine to control the bus timing.

A detailed description of the algorithm for determining these parameters—as well as a couple of concrete examples—are given in the [Timing Parameter Tuning Algorithm](#).

Note that the IP also has minimum values for `TIMING0.TLOW`, `TIMING0.THIGH`, `TIMING2.THLD_STA`, and `TIMING3.THLD_DAT`. `THLD_DAT` must be at least 1, and to disambiguate START conditions from early SDA arrival, `THLD_STA` must be at least 2 cycles longer than `THLD_DAT`.

The low count must be long enough to cover the round-trip time of the Controller Module's SCL drive to the target through the sampling of the target's data driven on SDA. As there is an output flop, a 2-flop synchronizer, a rise time, and a pad input delay all on the path, `TLOW` must be at least 3 cycles + `T_R` + `InputDelayCycles`.

Similarly, the high count must be long enough to cover the a round-trip time to sense outputs for various conditions (e.g. lost arbitration, target clock stretching, etc.). `THIGH` must be at least 4 cycles + `InputDelayCycles`.

The component supports complete I2C timing parameter configuration:

Table 518. Standard Timing Parameters

Parameter	Description
tHIGH	SCL high time (supports 100kHz, 400kHz, 1MHz modes)
tLOW	SCL low time (symmetric or asymmetric clock duty cycle)
tR/tF	Rise and fall time limits for signal transitions
tHD_STA	Hold time for START condition
tSU_STA	Setup time for repeated START condition
tSU_STO	Setup time for STOP condition
tSU_DAT/tHD_DAT	Data setup and hold times

Table 519. Timeout Parameters

Parameter	Description
Bus Active Timeout	Detection of stuck bus conditions
Stretch Timeout	Clock stretching duration limits
Host Timeout	Controller operation timeout
NACK Timeout	Target response timeout

Timeout Controls

Bus Timeout or Stretch Timeout

A malfunctioning (or otherwise very slow) target device can hold SCL low indefinitely, stalling the bus. In addition, in an SMBus context, a controller can purposefully hold SCL low for an extended period of time (`tTIMEOUT:MIN`) to force a bus timeout. For these reasons, `TIMEOUT_CTRL` provides a timeout mechanism for determining one of two variants of an SCL low timeout.

Enabling a timeout at all requires setting `TIMEOUT_CTRL.EN` and configuring `TIMEOUT_CTRL.VAL` with the desired number of cycles to count before timing out. With the configuration of `TIMEOUT_CTRL.MODE`, one of two types of timeouts can be selected.

If a `STRETCH_TIMEOUT` is selected, the Controller Module detects when some other device holds SCL low for more cycles than `TIMEOUT_CTRL.VAL` clock ticks. If the counter expires, the `stretch_timeout` interrupt will be asserted.

If a `BUS_TIMEOUT` is selected, the SCL low time accumulates across all sources, including this IP's Controller Module. This counter mode can be used to detect SMBus's definition of a bus timeout. If a bus timeout occurs while the Controller Module is active, the corresponding `CONTROLLER_EVENTS.BUS_TIMEOUT` bit is set, halting the Controller Module and requiring handling from software before the next command will process. Similarly, if a bus timeout occurs while the Target Module was addressed in the current transaction, a `NACK_STOP` signal is added to the ACQ FIFO. In addition, if the last command byte addressing the module issued a read operation, `TARGET_EVENTS.BUS_TIMEOUT` will be set, causing the Target Module to stretch the clock on read operations until software handles the event.

For both cases, the counter resets when SCL goes high, so this count only accumulates during a single bit transfer.

SCL Idle Timeout

The I2C IP provisions a counter for detecting when SCL has remained high for too long during an active transaction. `HOST_TIMEOUT_CTRL` is optionally enabled, determined by whether the register value is nonzero.

If this timeout occurs in multi-controller monitor mode with SDA released high, the Bus Monitor uses this as a "bus idle" condition, which allows the Controller Module to begin transmitting. This timeout can be used to implement SMBus's $t_{\text{HIGH,MAX}}$ bus idle condition. In addition, the SCL Idle Timeout is used by the Bus Monitor to set up waiting for a bus idle condition at init, if multi-controller monitor mode was enabled no later than the Controller Module.

If an active transfer is targeting this IP's Target Module, this timeout will trigger a `host_timeout` interrupt. Unlike the bus timeout, the idle timeout does not trigger an end to the command, so the controller could potentially recover. Instead, the interrupt is merely informative. However, a START condition from any device will terminate the transfer, and that includes the Controller Module. Note that this behavior could cause transaction boundaries to be lost; the Target Module will report that this next START condition is a repeated START.

Target Module Stretch Timeout

If enabled with `TARGET_TIMEOUT_CTRL.EN`, the Target Module will automatically NACK transactions if it stretches the clock for more cycles than `TARGET_TIMEOUT_CTRL.VAL`. This counter accumulates across an entire transaction, and with the correct choice of timeout value, it complies with SMBus's specified maximum cumulative clock extension time for the target.

Memory and FIFO Architecture

The I2C interface uses flexible, configurable memory blocks to implement its FIFOs, allowing the choice of memory type (SRAM, register, or hybrid) and standard RAM interface options for memory timing. When available, error correction (ECC) and power-saving features are supported.

FIFO depth is set at compile time, with each FIFO providing interrupt generation based on programmable thresholds, almost-full and almost-empty indicators, robust protection against overflow and underflow, as well as reset and flush mechanisms for reliable and efficient data handling.

Error Handling and Diagnostics

The I2C component provides comprehensive automatic detection and protection against protocol and system-level errors, including:

- Bus errors: Arbitration loss, address/data NACKs, clock stretching violations, and stuck bus detection.

- FIFO faults: Overflow, underflow, and threshold violations.
- Register errors: Invalid register access and parameter conflicts.

For recovery, the hardware supports automatic controller state resets, FIFO clearing, bus recovery sequences, and interrupt-based fault signals. Status and error registers capture detailed fault codes.

Diagnostic features include real-time monitoring of I2C activity, bus event logging with timestamps, and visibility into internal state machine status. Hardware counters track transaction completions and error events to aid debugging and performance tuning. SCL and SDA line status can be monitored to quickly identify electrical or protocol-level problems.

Interrupts

The I2C module has a few interrupts including general data flow interrupts and unexpected event interrupts.

Controller Module

Whenever the RX FIFO exceeds the designated number of entries, the interrupt `rx_threshold` is asserted to inform firmware. Firmware can configure the threshold value via the register `HOST_FIFO_CONFIG.RX_THRESH`.

Whilst the FMT FIFO level lies below a designated number of entries, the `fmt_threshold` interrupt is asserted. Firmware can configure the threshold value via the register `HOST_FIFO_CONFIG.FMT_THRESH`.

If the RX FIFO receives an additional write request when its FIFO is full, the interrupt `rx_overflow` is asserted and the character is dropped.

If the module transmits a byte and the 9th bit is a NACK from the Target/Bus, the `controller_halt` interrupt is usually asserted (modulo the effect of `FDATA.NAKOK`). If the `controller_halt` interrupt is asserted, the Controller Module FSM will halt until the interrupt has been acknowledged. See [Controller NACK Handling](#) above for more details on this behaviour. In addition, the interrupt will be asserted if any of the other halt-causing events occur, such as a bus timeout or a loss of arbitration. This interrupt is a status type that requires clearing all triggering event latches before it will deassert.

When the Controller Module is actively transmitting, the `scl_interference` or `sda_interference` interrupts will be asserted if the IP identifies that some other device (controller or target) on the bus is forcing either signal low and interfering with the transmission. It should be noted that the `scl_interference` interrupt is not raised in the case when the target device is stretching the clock. However, it may be raised if the target allows SCL to go high and then pulls SCL down before the end of the current clock cycle. The `scl_interference` interrupt should be masked in multi-controller environments, since clock synchronization between controllers can trigger its assertion.

The `sda_interference` interrupt is raised whenever the Controller Module detects that another device is pulling SDA low while the Controller Module is trying to transmit logic high.

On the other hand, it is legal for a target device to assert SCL low for clock stretching purposes. With clock stretching, the target can delay the start of the following SCL pulse by holding SCL low between clock pulses. However the target device must assert SCL low before the start of the SCL pulse. If SCL is pulled low during an SCL pulse which has already started, this interruption of the SCL pulse will be registered as an exception by the I2C core, which will then assert the `scl_interference` interrupt.

Though normal clock stretching does not count as SCL interference, if the module detects that a target device has held SCL low and stretched any given SCL cycle for more than `TIMEOUT_CTRL.VAL` clock ticks this will cause the stretch timeout interrupt to be asserted. This interrupt is suppressed, however, if `TIMEOUT_CTRL.EN` is deasserted low or if the bus timeout is selected for `TIMEOUT_CTRL.MODE`.

Except for START and STOP symbols, the I2C specification requires that the SDA signal remains constant whenever SCL is high. The `sda_unstable` interrupt is asserted if, when receiving data or acknowledgement pulse, the value of the SDA signal does not remain constant over the duration of the SCL pulse, causing an unexpected START or STOP symbol.

Transactions are terminated by a STOP signal, but individual transfers within a transaction can be completed by a STOP signal **or** a repeated START signal. When a transfer completes, the `cmd_complete` interrupt is asserted, in the beginning of a repeated START or at the end of a STOP. This interrupt is overloaded and represents the full IP's participation in any bus transactions. The Target Module's transfers also signal `cmd_complete`.

Target Module

As mentioned above, the interrupt `cmd_complete` is asserted whenever a RESTART or a STOP bit is observed by the target, for a transfer that addressed the Target Module.

The interrupt `tx_stretch` is asserted whenever the Target Module intends to transmit data but cannot. If software sets the bit `CTRL.TX_STRETCH_CTRL_EN`, the Target Module will require software to confirm release of the TX FIFO at the beginning of every read transfer addressed to it. This behaviour may be useful to software, as any remaining data in the TX FIFO after a Sr/P condition may no longer apply to the next transfer, so it may have to be cleared out via `FIFO_CTRL.TXRST`. For more on stretching on read transfers, see [\[stretching-during-read\]](#).

When a controller receives enough data from a target, it usually signals the end of the transaction by sending a NACK followed by a STOP or a repeated START. In a case when the Target Module receives a STOP without the prerequisite NACK, the interrupt `unexp_stop` is asserted. This interrupt just means that a STOP was unexpectedly observed during a controller read. It is not necessarily harmful, but software can be made aware just in case.

If the ACQ FIFO exceeds the designated number of entries, the interrupt `acq_threshold` is raised to inform firmware. Firmware can configure the threshold value via the register `TARGET_FIFO_CONFIG.ACQ_THRESH`.

Whilst the TX FIFO level is below a designated number of entries the `tx_threshold` interrupt is asserted. Firmware can configure the threshold value via the register `TARGET_FIFO_CONFIG.TX_THRESH`.

If the Target Module stretches the clock as a target-receiver, the interrupt `acq_stretch` is asserted. This can be due to a full ACQ FIFO, or it can be due to an ACK Control Mode stretch request. If ACK Control Mode is enabled, check the relevant bits in `STATUS` to determine the reason(s). The `acq_stretch` interrupt is a Status type, so it will only de-assert once the stretch conditions are cleared.

If a controller ceases to send SCL pulses at any point during an ongoing transaction, the target waits for a specified time period and then asserts the interrupt `host_timeout`. A controller sending an address and R/W bit to all target devices, writing to the selected target, or reading from the target are examples of ongoing transactions. The time period is counted from the last low-to-high SCL transition and continues counting until SCL goes low. Firmware can configure the timeout value via the register `HOST_TIMEOUT_CTRL`.

SMBus and PMBus Protocol Support

The I2C component provides comprehensive hardware support for SMBus (System Management Bus) and PMBus (Power Management Bus) protocols, which build upon the I2C physical layer with additional timing, electrical, and protocol requirements.

SMBus Physical Interface

The component includes dedicated SMBus signal pins beyond standard I2C:

- **SMBSUS# (SMBus Suspend):** Bidirectional signal for power management and bus suspension

- Controller Mode: Controlled via `SMBUS_CTRL.SMBSUS` register to drive the line low
- Target Mode: Status monitored via `SMBUS_STATUS.SMBSUS` register
- Used for system power state transitions and low-power modes
- **SMBALERT# (SMBus Alert):** Bidirectional interrupt signal for target-initiated communication
 - Target Mode: Controlled via `SMBUS_CTRL.SMBALERT` register to signal attention request
 - Controller Mode: Status monitored via `SMBUS_STATUS.SMBALERT` register, interrupt via `INTR_STATE.SMBALERT`
 - Supports SMBus Alert Response Address (ARA) protocol for multi-target systems

SMBus Timeout Management

SMBus protocols enforce strict timing requirements with cumulative timeout tracking:

Target Timeout Control (`TARGET_TIMEOUT_CTRL` register):

- Implements SMBus cumulative clock extension time requirements
- Tracks total clock stretching duration across an entire transaction
- Configurable timeout value in system clock cycles
- Automatic NACK generation when timeout expires
- Separate control for address byte handling via `CTRL.NACK_ADDR_AFTER_TIMEOUT`:
 - **0**: ACK address even on timeout (SMBus compliant behavior)
 - **1**: NACK address when timeout occurs

Controller Timeout Detection (`HOST_TIMEOUT_CTRL` register):

- Detects when a controller stops generating clock signals
- Target mode: Generates `HOST_TIMEOUT` interrupt
- Monitor mode: Required for proper bus state tracking
- Ensures detection of hung or failed controllers

SMBus ACK/NACK Control

SMBus protocols require sophisticated mid-transaction NACK capability for flow control and error handling:

ACK Control Mode (`CTRL.ACK_CTRL_EN = 1`):

- Software-controlled acknowledgment on per-byte basis
- `TARGET_ACK_CTRL.NBYTES` counter specifies number of bytes to accept
- When counter reaches zero, target stretches clock and sets `STATUS.ACK_CTRL_STRETCH`
- Software options:
 1. Accept additional bytes by reloading `NBYTES` counter
 2. Reject transaction by setting `TARGET_ACK_CTRL.NACK`
- `ACQ_FIFO_NEXT_DATA` register provides preview of pending byte for decision-making

Automatic ACK Mode (`CTRL.ACK_CTRL_EN = 0`):

- Traditional I2C behavior with FIFO-based flow control
- Target ACKs while ACQ FIFO has available space
- Automatic NACK when FIFO full

SMBus Packet Error Checking (PEC)

While the I2C hardware does not implement CRC-8 calculation in hardware, it provides the necessary software control points:

- Byte-by-byte data access through FIFOs enables software CRC calculation
- ACK Control Mode allows validation before accepting the PEC byte
- Mid-transaction NACK capability supports rejecting invalid packets
- `ACQ_FIFO_NEXT_DATA` preview enables PEC verification before acceptance

PMBus Protocol Extensions

PMBus builds on SMBus with power management specific features, all supported by the underlying I2C/SMBus implementation:

- **Group Command Protocol:** Multi-device command support through standard I2C repeated START
- **Extended Commands:** 16-bit command codes via sequential byte transmission
- **Block Read/Write:** Variable-length transfers using FIFO depth and threshold management
- **Alert Response Protocol:** Hardware `SMBALERT#` signal with ARA address support
- **Zone Read/Write:** Sequential register access via FIFO streaming

Hybrid Mode Support

The I2C component supports SMBus hybrid mode operation:

- Simultaneous controller and target enable (`ENABLEHOST = 1`, `ENABLETARGET = 1`)
- Bus monitor tracks multi-controller environment
- Automatic arbitration and collision detection
- Independent timeout monitoring for each mode

Implementation Details: Format Flag Parsing

To illustrate the behavior induced by various flags added to the formatting queue, the following figure shows a simplified version of the `I2C_Controller` state machine. In this simplified view, many sequential states have been collapsed into four sub-sequences of states (shown in square brackets) or have their names abbreviated: * Issue start * Issue stop * Transmit Byte * Read Bytes

Within each of these sub-sequences, state transitions depend only on the SDA/SCL inputs or internal timers. Each sub-sequence has a terminal event—generically labeled "[completed]" which prompts the transition to another sequence or state.

However, all transitions which are dependent on formatting flags are shown explicitly in this figure.

The Controller FSM operates as follows: * From IDLE state, when FMT FIFO is not empty, it transitions to ACTIVE state * From ACTIVE state: **If READB flag is set, transitions to Read Bytes sequence** If START flag is set (and not READB), transitions to Issue Start sequence, then to Transmit Byte **Otherwise, transitions directly to Transmit Byte sequence** * **After Read Bytes completes:** If STOP flag is set, transitions to Issue Stop sequence, then to POP_FMT_FIFO **Otherwise, transitions directly to POP_FMT_FIFO** * **After Transmit Byte completes:** If STOP flag is set, transitions to Issue Stop sequence, then to POP_FMT_FIFO **Otherwise, transitions directly to POP_FMT_FIFO** * **After Issue Stop completes, transitions to POP_FMT_FIFO** * **From POP_FMT_FIFO:** If FMT FIFO is empty, returns to IDLE ** Otherwise, transitions to ACTIVE for next format indicator

Similarly, the Target FSM operates as follows: * From IdleStart state, transitions to AddrRead * From AddrRead: **If address doesn't match, transitions to WaitForStop** If address matches, transitions to AddrAck * From AddrAck, if stretch_addr condition, transitions to StretchAddr, otherwise forks based on R/W bit: **If R/W = 0 (write), enters Tx state machine:** * If tx_fifo_rvalid and !tx_halt, transitions to TxByte, then TxAck * **If !tx_fifo_rvalid and !tx_halt, transitions to StretchTx, then TxByte when data available** * If ack received, returns to fork; if !ack, exits Tx state **If R/W = 1 (read), enters Rx state machine:** * If acq_fifo_wready and can_auto_ack, transitions to AcquireByte, then AcquireAck * **If !acq_fifo_wready or !can_auto_ack, transitions to StretchAcq, then AcquireByte when ready** * If ack sent, returns to fork; if !ack, exits Rx state * From Tx or Rx state, transitions to WaitForStop * From WaitForStop, returns to IdleStart

Application Notes

Virtual Open Drain

In devices which lack a true open drain buffer functionality, this IP implements a "virtual Open Drain" functionality. The SDA and SCL outputs are assumed to be connected to a tri-state buffer, with independent enable outputs for both signals.

Rather than toggling the buffer inputs, the buffer inputs are **continuously asserted low**, and instead the buffer **enable** signals are toggled. The SDA or SCL buffers are enabled for a logical "Low" output on the respective signal, and are disabled for logical "High" outputs. This arrangement allows the output pins to float high if there is no conflict from external devices, or to be pulled low if there is a conflict (as is required for clock-stretching or multi-controller functionality).

This arrangement is necessary for FPGA builds.

Glitch Filter

The IP does not currently implement the specification's required 50 ns glitch filter. For full spec compliance, the glitch filter must be added to the system design.

Override Mode for Direct Pin Access

The I2C hardware interface consists of two external pins, SCL and SDA, whose behavior is described in the I2C specification (rev. 6). These pins are typically controlled by an internal state machine. However, there is a simpler "override" mode, by which these pins can be directly manipulated by software. This override mode is useful for both troubleshooting and error recovery.

To enter override mode, the register field **OVRD.TXOVRDEN** is asserted by software. In this state the output drivers **scl_tx_o** and **sda_tx_o** are controlled directly by the register fields **OVRD.SCLVAL** and **OVRD.SDAVAL**. When **OVRD.SCLVAL** and **OVRD.SDAVAL** are set high, the virtual open drain configuration will leave the output resistively pulled high, and controllable by remote targets. In this state, with SCL or SDA asserted high, the register fields **VAL.SCL_RX** and **VAL.SDA_RX**

can be used to receive inputs (including remote acknowledgments) from target devices.

FSM control of SCL and SDA

SCL and SDA are generated through the internal state machine of the controller and target modules. Since SCL is directly decoded from the states, it can have short glitches during transition which the external target may be sensitive to if it is not using an over-sampling scheme. To counter this, the SCL and SDA outputs from the internal state machine are flopped before they are emitted.

This adds a one cycle module clock delay to both signals. If the module clock is sufficiently faster than I2C line speeds (for example 24MHz), this is not an issue. However if the line speeds and the module clock speeds become very close (2x), the 1 cycle delay may have an impact, as the internal state machine may mistakenly think it has sampled an SDA that has not yet been updated.

Therefore, it is recommended that the internal module clock frequency is much higher than the line speeds. Another reason to have this higher internal clock frequency is that the timing parameters can be more accurately defined, which helps attain the desired I2C clock rate. Since there are currently also a few cycles discrepancy between the specified timings and the actual ones (as described in the [Timing Parameter Tuning Algorithm](#)), it is recommended that the internal module clock frequency is at least 24x higher than the I2C line speeds.

Interface

The I2C Interface Component provides signal interfaces supporting I2C, SMBus, and PMBus protocols with AXI4-Lite register access and DMA integration.

Configuration Parameters

Parameter	Default	Description
CTRL_TX_FIFO_DEPTH	64	Controller mode TX FIFO depth (entries)
CTRL_RX_FIFO_DEPTH	64	Controller mode RX FIFO depth (entries)
TGT_TX_FIFO_DEPTH	64	Target mode TX FIFO depth (entries)
TGT_RX_FIFO_DEPTH	268	Target mode RX FIFO depth (entries)
ADDR_WIDTH	8	Register address width (bits)
DATA_WIDTH	32	Data bus width (bits)

Signal Interfaces

Global Interface

Table 520. Global Interface Signals

Signal Name	Width	Direction	Active	Description
clk_i	1	Input	Rising	System clock. All logic operates synchronously to this clock. Minimum frequency depends on desired I2C speed and timeout requirements.
rst_ni	1	Input	Low	System reset. Asynchronous assertion and synchronous deassertion. Resets all state machines, FIFOs, and control logic to known states.

Memory Configuration Interface

Table 521. Memory Configuration Interface Signals

Signal Name	Width	Direction	Description
ram_cfg_i	struct	Input	RAM configuration input. Controls memory timing, power settings, and test modes for internal FIFO memories.
ram_cfg_rsp_o	struct	Output	RAM configuration response. Provides status and acknowledgment for memory configuration requests.

Configuration Parameters: - **Timing Control:** Setup/hold times, access delays, and clock-to-output delays - **Power Management:** Sleep modes, retention settings, and power-on sequences - **Test Features:** BIST control, scan chain integration, and debug access - **ECC Support:** Error correction enable, threshold settings (when available)

I2C Physical Interface

The I2C physical interface implements true open-drain signaling with proper electrical characteristics.

Table 522. I2C Physical Interface Signals

Signal Name	Width	Direction	Active	Description
scl_i	1	Input	High	Serial Clock Line input. Connected to SCL bus through appropriate I/O cell. Must have pull-up resistor.
scl_o	1	Output	Low	Serial Clock Line output. Open-drain output - drives low only. External pull-up required.
sda_i	1	Input	High	Serial Data Line input. Connected to SDA bus through appropriate I/O cell. Must have pull-up resistor.
sda_o	1	Output	Low	Serial Data Line output. Open-drain output - drives low only. External pull-up required.

Electrical Characteristics:

- **Logic Levels:** Compatible with I2C specification voltage levels (3.3V, 1.8V, etc.)
- **Drive Strength:** Sufficient to drive bus capacitance at target frequencies
- **Input Impedance:** High impedance inputs suitable for wired-AND bus operation
- **Output Drive:** Open-drain outputs with configurable drive strength
- **Rise/Fall Times:** Configurable to meet I2C specification requirements

Input Delay Configuration

The IP's input pins do not reflect the start point for the full input path delay, since pads and nets from I/O buffers also contribute. The RTL parameter `InputDelayCycles` must be set to the maximum number of cycles of delay from the IP's output pin driving SCL or SDA to the IP's corresponding input pin sensing the transition. `InputDelayCycles` can exclude the rise time that would be captured by `TIMING1.T_R`. The reference point for `InputDelayCycles` is the clock edge after the launch clock edge. In other words, a value of 0 means that the IP's 2-flop synchronizer would sample the IP's own outputs within one clock cycle (if the external pad driver could switch instantaneously).

This parameter affects timing constraints: * `TLOW` must be at least 3 cycles + `T_R` + `InputDelayCycles` * `THIGH` must be at least 4 cycles + `InputDelayCycles`

Virtual Open Drain Implementation

In devices which lack a true open drain buffer functionality, this IP implements a "virtual Open Drain" functionality. The SDA and SCL outputs are assumed to be connected to a tri-state buffer, with independent enable outputs for both signals.

Rather than toggling the buffer inputs, the buffer inputs are **continuously asserted low**, and instead the buffer **enable** signals are toggled. The SDA or SCL buffers are enabled for a logical "Low" output on the respective signal, and are disabled for logical "High" outputs. This arrangement allows the output pins to float high if there is no conflict from external devices, or to be pulled low if there is a conflict (as is required for clock-stretching or multi-controller functionality).

This arrangement is necessary for FPGA builds and provides the same electrical behavior as true open-drain outputs.

Bus Configuration:

- **Pull-up Resistors:** External pull-up resistors required (typically 1kΩ-10kΩ)
- **Bus Capacitance:** Supports up to 400pF for standard/fast modes, 120pF for fast-mode plus
- **Cable Length:** Depends on speed - shorter for higher frequencies
- **Termination:** No special termination required beyond pull-up resistors

SMBus Extension Interface

SMBus-specific signals support System Management Bus protocol extensions.

Table 523. SMBus Extension Interface Signals

Signal Name	Width	Direction	Active	Description
smbsus_ni	1	Input	Low	SMBus Suspend input (SMBSUS#). System suspend/resume signaling. Wired-AND signal.
smbsus_no	1	Output	Low	SMBus Suspend output (SMBSUS#). Open-drain output for suspend signaling.
smbalert_ni	1	Input	Low	SMBus Alert input (SMBALERT#). Interrupt-style signaling from target devices. Wired-AND signal.
smbalert_no	1	Output	Low	SMBus Alert output (SMBALERT#). Open-drain output for alert generation.

SMBus Protocol Support:

- **SMBSUS# Handling:** Automatic suspend detection and response generation
- **SMBALERT# Processing:** Alert response protocol with address resolution
- **Timeout Extensions:** SMBus-specific timeout handling
- **PEC Support:** Packet Error Checking capability (software implemented)

Signal Characteristics:

- **Open-Drain Outputs:** Both outputs are open-drain with external pull-up requirements
- **Wired-AND Operation:** Multiple devices can share the same signal lines
- **Voltage Levels:** Compatible with SMBus voltage specifications
- **Timing:** Meets SMBus timing requirements for signal assertion and deassertion

DMA Interface

DMA interface signals provide efficient data transfer support for high-throughput applications.

Table 524. DMA Interface Signals

Signal Name	Width	Direction	Active	Description
controller_tx_ready_o	1	Output	High	Controller TX FIFO ready for writing. Asserted when FIFO level is below threshold.
controller_rx_ready_o	1	Output	High	Controller RX FIFO ready for reading. Asserted when FIFO level is above threshold.
target_tx_ready_o	1	Output	High	Target TX FIFO ready for writing. Asserted when FIFO level is below threshold.
target_rx_ready_o	1	Output	High	Target RX FIFO ready for reading. Asserted when FIFO level is above threshold.

DMA Operation Modes:

- **Single Transfer:** One data word per DMA request
- **Burst Transfer:** Multiple words per request (up to FIFO capacity)
- **Threshold Control:** Configurable thresholds determine when signals assert
- **Flow Control:** Ready signals provide natural flow control for DMA engines

Threshold Configuration:

- **TX Ready Threshold:** Signal asserts when FIFO has space for N or more entries
- **RX Ready Threshold:** Signal asserts when FIFO contains N or more entries
- **Hysteresis:** Configurable threshold hysteresis prevents oscillation
- **Disable Capability:** DMA ready signals can be disabled for polling-only operation

Interrupt Interface

Comprehensive interrupt interface aggregates all interrupt sources.

Table 525. Interrupt Interface Signals

Signal Name	Width	Direction	Active	Description
irq_o	1	Output	High	Interrupt request. Asserted when any enabled interrupt condition occurs. Level-sensitive.

Interrupt Sources:

- **FIFO Interrupts:** Threshold, overflow, underflow conditions
- **Transaction Interrupts:** Completion, NACK, arbitration loss
- **Error Interrupts:** Timeout, protocol violations, bus errors
- **Status Interrupts:** Bus idle, address match, mode changes

Interrupt Control:

- **Individual Enables:** Each interrupt source has dedicated enable bit
- **Status Registers:** Separate status bits for each interrupt condition
- **Test Capability:** Software can force interrupts for testing
- **Clear Methods:** Write-1-to-clear for most interrupt types

Memory Map

Overview

The I2C Interface Component provides a comprehensive register interface through an 8-bit address space (0x00-0xFF) accessible via AXI4-Lite protocol. The register map is organized into logical groups covering interrupt management, control configuration, data access, timing parameters, and mode-specific functionality. All registers are 32-bit aligned and support both controller and target operating modes.

Register Address Map

Table 526. Register Address Map

Address	Name	Access	Description
0x00	INTR_STATE	RW	Interrupt State Register - Status of all interrupt sources
0x04	INTR_ENABLE	RW	Interrupt Enable Register - Individual interrupt enables
0x08	INTR_TEST	RW	Interrupt Test Register - Force interrupts for testing
0x0C	SMBUS_CTRL	RW	SMBus Control Register - SMBus-specific features
0x10	CTRL	RW	Control Register - Global configuration and mode control
0x14	STATUS	R	Status Register - Current operational status
0x18	RDATA	R	Read Data Register - Controller mode received data
0x1C	FDATA	W	Format Data Register - Controller mode command/address/data
0x20	FIFO_CTRL	W	FIFO Control Register - FIFO management commands
0x24	HOST_FIFO_CONFIG	RW	Controller FIFO Configuration - Thresholds and settings
0x28	TARGET_FIFO_CONFIG	RW	Target FIFO Configuration - Thresholds and settings
0x2C	HOST_FIFO_STATUS	R	Controller FIFO Status - Current FIFO levels and flags
0x30	TARGET_FIFO_STATUS	R	Target FIFO Status - Current FIFO levels and flags
0x34	OVRD	RW	Override Control Register - Direct signal control for debug
0x38	VAL	R	Oversampled Values Register - Current signal states
0x3C	TIMING0	RW	Timing Register 0 - SCL low and high periods
0x40	TIMING1	RW	Timing Register 1 - Rise and fall times
0x44	TIMING2	RW	Timing Register 2 - START condition timing
0x48	TIMING3	RW	Timing Register 3 - Data setup and hold timing
0x4C	TIMING4	RW	Timing Register 4 - STOP condition and bus free timing
0x50	TIMEOUT_CTRL	RW	Timeout Control Register - Bus timeout configuration
0x54	TARGET_ID	RW	Target ID Register - Target mode address configuration
0x58	ACQDATA	R	Acquired Data Register - Target mode received data
0x5C	TXDATA	W	Transmit Data Register - Target mode transmit data
0x60	HOST_TIMEOUT_CTRL	RW	Controller Timeout Control - Controller-specific timeouts
0x64	TARGET_TIMEOUT_CTRL	RW	Target Timeout Control - Target-specific timeouts
0x68	TARGET_NACK_COUNT	RW	Target NACK Count Register - NACK counting configuration
0x6C	TARGET_ACK_CTRL	RW	Target ACK Control Register - ACK/NACK control
0x70	ACQ_FIFO_NEXT_DATA	R	ACQ FIFO Next Data Register - Preview next ACQ FIFO entry
0x74	HOST_NACK_HANDLER_TIMEOUT	RW	Controller NACK Handler Timeout - NACK response timing
0x78	CONTROLLER_EVENTS	RW	Controller Events Register - Controller-specific event flags

Address	Name	Access	Description
0x7C	TARGET_EVENTS	RW	Target Events Register - Target-specific event flags
0x80	SMBUS_STATUS	R	SMBus Status Register - SMBus-specific status information

Address Alignment and Access Width

Address Alignment: - All registers are aligned to 32-bit boundaries (addresses are multiples of 4) - 8-bit effective address space: only bits [7:0] of the AXI4-Lite address are used - Upper address bits are ignored

Access Width Requirements: - All registers must be accessed using 32-bit operations - Byte and halfword accesses are not supported - AXI4-Lite write strobe signals should be set to 4'b1111 for all writes

Endianness: - All multi-byte fields use little-endian byte ordering - Bit numbering follows standard convention (bit 0 is LSB)

Reset Values and Power-On State

Table 527. Register Reset Values

Register Group	Reset Value	Description
Interrupt Registers	0x00000000	All interrupts cleared and disabled
Control Registers	0x00000000	All modes disabled, default configuration
Status Registers	Varies	Hardware-determined based on initial state
Data Registers	N/A	FIFO access registers, no fixed reset value
Timing Registers	0x00000000	Must be configured before operation
Target ID Registers	0x00000000	No target addresses configured

Detailed Register Map

Table 528. Register Map

Address	Name	Access	Description
0x0	INTR_STATE	RW	Interrupt Status Register
0x4	INTR_ENABLE	RW	Interrupt Enable Register
0x8	INTR_TEST	RW	Interrupt Test Register
0xC	SMBUS_CTRL	RW	SMBus Control Register
0x10	CTRL	RW	Control Register
0x14	STATUS	R	Status Register
0x18	RDATA	R	Controller RX FIFO Access Register (Controller Mode only)
0x1C	FDATA	W	Controller TX FIFO Access Register (Controller Mode only)
0x20	FIFO_CTRL	W	FIFO Control Register
0x24	HOST_FIFO_CONFIG	RW	Controller FIFO Configuration Register (Controller Mode only)
0x28	TARGET_FIFO_CONFIG	RW	Target FIFOs Configuration Register (Target Mode only)
0x2C	HOST_FIFO_STATUS	R	Controller FIFOs Status Register (Controller Mode only)
0x30	TARGET_FIFO_STATUS	R	Target FIFOs Status Register (Target Mode only)
0x34	OVRD	RW	Override Control Register
0x38	VAL	R	Bus Oversampled Values Register

Address	Name	Access	Description
0x3C	TIMING0	RW	SCL LOW and HIGH Periods Register
0x40	TIMING1	RW	Bus Rise and Fall Times Register
0x44	TIMING2	RW	START Condition Setup and Hold Times Register
0x48	TIMING3	RW	Data Setup and Hold Times Register
0x4C	TIMING4	RW	STOP Condition Setup Time and Bus Free Time Register
0x50	TIMEOUT_CTRL	RW	Timeout Control Register
0x54	TARGET_ID	RW	Target ID Register (Target Mode only)
0x58	ACQDATA	R	Target RX FIFO Access Register (Target Mode only)
0x5C	TXDATA	W	Target TX FIFO Access Register (Target Mode only).
0x60	HOST_TIMEOUT_CTRL	RW	Controller Timeout Control Register
0x64	TARGET_TIMEOUT_CTRL	RW	Target Timeout Control Register (Target Mode only)
0x68	TARGET_NACK_COUNT	RW	Target NACK Count Register (Target Mode only)
0x6C	TARGET_ACK_CTRL	RW	Target ACK Control Register (Target Mode only)
0x70	ACQ_FIFO_NEXT_DATA	R	Target RX FIFO Next Byte Register (Target Mode only)
0x74	HOST_NACK_HANDLER_TIMEOUT	RW	Controller NACK Timeout Register (Controller Mode only)
0x78	CONTROLLER_EVENTS	RW	Controller Events Register (Controller Mode only)
0x7C	TARGET_EVENTS	RW	Target Events Register (Target Mode)
0x80	SMBUS_STATUS	R	SMBus Status Register

Table 529. INTR_STATE Register

Bits	Field	Access	Reset	Description
31:20	Reserved	-	-	Reserved
19	TARGET_RX_FIFO_ERROR	RW	0x0	Target Mode interrupt: asserted when the Target RX FIFO has a parity error. Write 1 to clear.
18	TARGET_TX_FIFO_ERROR	RW	0x0	Target Mode interrupt: asserted when the Target TX FIFO has a parity error. Write 1 to clear.
17	CONTROLLER_RX_FIFO_ERROR	RW	0x0	Controller Mode interrupt: asserted when the Controller RX FIFO has a parity error. Write 1 to clear.
16	CONTROLLER_TX_FIFO_ERROR	RW	0x0	Controller Mode interrupt: asserted when the Controller TX FIFO has a parity error. Write 1 to clear.
15	SMBALERT	RW	0x0	Controller Mode interrupt: asserted when <code>smbalert_ni</code> is asserted. Write 1 to clear.
14	HOST_TIMEOUT	RW	0x0	Target Mode interrupt: asserted when the controller stops generating the clock longer than <code>HOST_TIMEOUT_CTRL.VAL</code> . Write 1 to clear.
13	UNEXP_STOP	RW	0x0	Target Mode interrupt: asserted when the controller sends this target a STOP before this target NACKs. Write 1 to clear.
12	ACQ_STRETCH	R	0x0	Target Mode interrupt: remains asserted while the target is stretching the clock because 1) the Target RX FIFO is full or 2) the <code>TARGET_ACK_CTRL.NBYTES</code> count has reached 0 (only if <code>CTRL.ACK_CTRL_EN = 1</code>).
11	TX_THRESHOLD	R	0x0	Target Mode interrupt: remains asserted while the Target TX FIFO level is less than <code>TARGET_FIFO_CONFIG.TX_THRESH</code> .

Bits	Field	Access	Reset	Description
10	TX_STRETCH	R	0x0	Target Mode interrupt: remains asserted while this target is stretching the clock or has halted. The flags in the TARGET_EVENTS register explain the reason(s) for clock stretching/halting. Clearing the TARGET_EVENTS flags clears this interrupt.
9	CMD_COMPLETE	RW	0x0	Controller/Target Mode interrupt: asserted when this/the controller finishes generating a STOP or repeated START. Write 1 to clear.
8	SDA_UNSTABLE	RW	0x0	Controller Mode interrupt: asserted when the target fails to keep SDA stable during a transmission. Write 1 to clear.
7	STRETCH_TIMEOUT	RW	0x0	Controller Mode interrupt: asserted when the target stretches the clock longer than TIMEOUT_CTRL.VAL (valid only when TIMEOUT_CTRL.MODE = 0). Write 1 to clear.
6	SDA_INTERFERENCE	RW	0x0	Controller Mode interrupt: asserted when SDA is unexpectedly pulled LOW by another controller. Write 1 to clear.
5	SCL_INTERFERENCE	RW	0x0	Controller Mode interrupt: asserted when SCL is unexpectedly pulled LOW by another controller. Write 1 to clear.
4	CONTROLLER_HALT	R	0x0	Controller Mode interrupt: remains asserted while this controller halts. The flags in the CONTROLLER_EVENTS register explain the reason(s) for the halting. Clearing the CONTROLLER_EVENTS flags clears this interrupt.
3	RX_OVERFLOW	RW	0x0	Controller Mode interrupt: asserted when the Controller RX FIFO overflows. Write 1 to clear.
2	ACQ_THRESHOLD	R	0x0	Target Mode interrupt: remains asserted while the Target RX FIFO level is greater than TARGET_FIFO_CONFIG.ACQ_THRESH .
1	RX_THRESHOLD	R	0x0	Controller Mode interrupt: remains asserted while the Controller RX FIFO level is greater than HOST_FIFO_CONFIG.RX_THRESH .
0	FMT_THRESHOLD	R	0x0	Controller Mode interrupt: remains asserted while the Controller TX FIFO level is less than HOST_FIFO_CONFIG.FMT_THRESH .

Table 530. INTR_ENABLE Register

Bits	Field	Access	Reset	Description
31:20	Reserved	-	-	Reserved
19	TARGET_RX_FIFO_ERROR	RW	0x0	Enables the TARGET_RX_FIFO_ERROR interrupt.
18	TARGET_TX_FIFO_ERROR	RW	0x0	Enables the TARGET_TX_FIFO_ERROR interrupt.
17	CONTROLLER_RX_FIFO_ERROR	RW	0x0	Enables the CONTROLLER_RX_FIFO_ERROR interrupt.
16	CONTROLLER_TX_FIFO_ERROR	RW	0x0	Enables the CONTROLLER_TX_FIFO_ERROR interrupt.
15	SMBALERT	RW	0x0	Enables the SMBALERT interrupt.
14	HOST_TIMEOUT	RW	0x0	Enables the HOST_TIMEOUT interrupt.
13	UNEXP_STOP	RW	0x0	Enables the UNEXP_STOP interrupt.
12	ACQ_STRETCH	RW	0x0	Enables the ACQ_STRETCH interrupt.
11	TX_THRESHOLD	RW	0x0	Enables the TX_THRESHOLD interrupt.
10	TX_STRETCH	RW	0x0	Enables the TX_STRETCH interrupt.
9	CMD_COMPLETE	RW	0x0	Enables the CMD_COMPLETE interrupt.
8	SDA_UNSTABLE	RW	0x0	Enables the SDA_UNSTABLE interrupt.
7	STRETCH_TIMEOUT	RW	0x0	Enables the STRETCH_TIMEOUT interrupt.

Bits	Field	Access	Reset	Description
6	SDA_INTERFERENCE	RW	0x0	Enables the SDA_INTERFERENCE interrupt.
5	SCL_INTERFERENCE	RW	0x0	Enables the SCL_INTERFERENCE interrupt.
4	CONTROLLER_HALT	RW	0x0	Enables the CONTROLLER_HALT interrupt.
3	RX_OVERFLOW	RW	0x0	Enables the RX_OVERFLOW interrupt.
2	ACQ_THRESHOLD	RW	0x0	Enables the ACQ_THRESHOLD interrupt.
1	RX_THRESHOLD	RW	0x0	Enables the RX_THRESHOLD interrupt.
0	FMT_THRESHOLD	RW	0x0	Enables the FMT_THRESHOLD interrupt.

Table 531. INTR_TEST Register

Bits	Field	Access	Reset	Description
31:20	Reserved	-	-	Reserved
19	TARGET_RX_FIFO_ERROR	W	0x0	Writing 1 forces the TARGET_RX_FIFO_ERROR interrupt.
18	TARGET_TX_FIFO_ERROR	W	0x0	Writing 1 forces the TARGET_TX_FIFO_ERROR interrupt.
17	CONTROLLER_RX_FIFO_ERROR	W	0x0	Writing 1 forces the CONTROLLER_RX_FIFO_ERROR interrupt.
16	CONTROLLER_TX_FIFO_ERROR	W	0x0	Writing 1 forces the CONTROLLER_TX_FIFO_ERROR interrupt.
15	SMBALERT	W	0x0	Writing 1 forces the SMBALERT interrupt.
14	HOST_TIMEOUT	W	0x0	Writing 1 forces the HOST_TIMEOUT interrupt.
13	UNEXP_STOP	W	0x0	Writing 1 forces the UNEXP_STOP interrupt.
12	ACQ_STRETCH	RW	0x0	Writing 1 forces the ACQ_STRETCH interrupt. Writing 0 releases it.
11	TX_THRESHOLD	RW	0x0	Writing 1 forces the TX_THRESHOLD interrupt. Writing 0 releases it.
10	TX_STRETCH	RW	0x0	Writing 1 forces the TX_STRETCH interrupt. Writing a 0 releases it.
9	CMD_COMPLETE	W	0x0	Writing 1 forces the CMD_COMPLETE interrupt.
8	SDA_UNSTABLE	W	0x0	Writing 1 forces the SDA_UNSTABLE interrupt.
7	STRETCH_TIMEOUT	W	0x0	Writing 1 forces the STRETCH_TIMEOUT interrupt.
6	SDA_INTERFERENCE	W	0x0	Writing 1 forces the SDA_INTERFERENCE interrupt.
5	SCL_INTERFERENCE	W	0x0	Writing 1 forces the SCL_INTERFERENCE interrupt.
4	CONTROLLER_HALT	RW	0x0	Writing 1 forces the CONTROLLER_HALT interrupt. Writing 0 releases it.
3	RX_OVERFLOW	W	0x0	Writing 1 forces the RX_OVERFLOW interrupt.
2	ACQ_THRESHOLD	RW	0x0	Writing 1 forces the ACQ_THRESHOLD interrupt. Writing 0 releases it.
1	RX_THRESHOLD	RW	0x0	Writing 1 forces the RX_THRESHOLD interrupt. Writing 0 releases it.
0	FMT_THRESHOLD	RW	0x0	Writing 1 forces the FMT_THRESHOLD interrupt. Writing 0 releases it.

Table 532. SMBUS_CTRL Register

Bits	Field	Access	Reset	Description
31:5	Reserved	-	-	Reserved
4	SMBALERT	RW	0x0	Target Mode control: asserts the <code>smbAlertNo</code> output. This bit clears itself when the controller addresses this target.
3:1	Reserved	-	-	Reserved

Bits	Field	Access	Reset	Description
0	SMBSUS	RW	0x0	Controller Mode control: asserts the <code>smbus_no</code> output.

Table 533. CTRL Register

Bits	Field	Access	Reset	Description
31:8	Reserved	-	-	Reserved
7	ACQ_START_STOP_EN	RW	0x0	ACQ FIFO Start/Stop Enable (Target Mode only): * 0 - Start/Stop symbols are NOT written into the ACQ FIFO. The firmware should rely on <code>TARGET_EVENTS.START_DETECT</code> and <code>TARGET_EVENTS.STOP_DETECT</code> flags to detect Start/Stop events. * 1 - Start/Stop symbols ARE written into the ACQ FIFO (legacy behavior). When a START (or repeated START) is detected, an <code>AcqStart</code> or <code>AcqRestart</code> entry is written. When a STOP is detected, an <code>AcqStop</code> or <code>AcqNackStop</code> entry is written.
6	TX_STRETCH_CTRL_EN	RW	0x0	Target mode control: enables the Software TX Stretch Control Mechanism. If this bit is: * 0 - Automatic TX Stretch: When this target receives a READ address byte, it only stretches the clock if the TX FIFO is empty; otherwise, it pops the FIFO and transmits the data byte. The target never sets the <code>TARGET_EVENTS</code> register flags in this mode. * 1 - Software TX Stretch Mode: When this target receives a READ address, it always stretches the clock and sets the <code>TARGET_EVENTS.TX_PENDING</code> flag. The software can: 1. confirm the release and transmission of the TX FIFO data by writing a 1 to clear the <code>TARGET_EVENTS.TX_PENDING</code> flag; or 2. reset the TX FIFO by writing 1 to <code>FIFO_CTRL.TXRST</code> and load in new data via the <code>TXDATA</code> register—useful when the READ address is targeting a different function of the target. In this mode, the target always sets the <code>TARGET_EVENTS</code> register flags.
5	MULTI_CONTROLLER_MONITOR_EN	RW	0x0	Global control: enables the Bus Monitor. Set this bit to 1 only in a multi-controller environment. If a 0 → 1 transition happens while <code>ENABLEHOST</code> and <code>ENABLETARGET</code> are both 0, the Bus Monitor will enable and begin in the 'Bus Busy' state. To transition to a 'Bus Free' state, <code>HOST_TIMEOUT_CTRL</code> must be nonzero so the Bus Monitor may count out idle cycles to confirm the freedom to transmit. In addition, the Bus Monitor will track whether the bus is free based on the enabled timeouts and detected STOP symbols. For Multi-Controller Mode, ensure <code>MULTI_CONTROLLER_MONITOR_EN</code> becomes 1 no later than <code>ENABLEHOST</code> or <code>ENABLETARGET</code>. This bit can be set at the same time as either or both of the other two, though. Note that if <code>MULTI_CONTROLLER_MONITOR_EN</code> is set after <code>ENABLEHOST</code> or <code>ENABLETARGET</code>, the Bus Monitor will begin in the 'Bus Free' state instead. This would violate the proper protocol for a controller to join a multi-controller environment. However, if this controller is known to be the first to join, this ordering will enable skipping the idle wait. When 0, the bus monitor will report that the bus is always free, so the Controller FSM is never blocked from transmitting.
4	ACK_CTRL_EN	RW	0x0	Target Mode control: enables the Software ACK Control Mechanism. If this bit is: * 0 - This target ACKs a data byte whenever the ACQ FIFO has space. * 1 - This target ACKs the first <code>TARGET_ACK_CTRL.NBYTES</code> bytes. If another byte arrives, this target stretches the clock and awaits software intervention (and asserts <code>STATUS.ACK_CTRL_STRETCH</code>). The software can 1. accept the new byte(s) by reloading the <code>TARGET_ACK_CTRL.NBYTES</code> counter; or 2. reject the new byte(s) by writing 1 to <code>TARGET_ACK_CTRL.NACK</code> (useful for SMBus).

Bits	Field	Access	Reset	Description
3	NACK_ADDR_AFTER_TIMEOUT	RW	0x0	Target Mode control: NACK address after timeout. If this bit is: * 0 - This target ACKs the address byte even if a Stretch Timeout occurs (useful for SMBus). * 1 - This target NACKs the address byte when a Stretch Timeout occurs.
2	LLPBK	RW	0x0	Global control: enables line loop-back. In Controller Mode, the internal logic sees the <code>smbalert_ni</code> input as deasserted. In Target Mode, this target sends all received SDA data back out, and the internal logic sees received data as all 1's.
1	ENABLETARGET	RW	0x0	Global control: configures this device's operating mode. * ENABLEHOST = 1 and ENABLETARGET = 0 — Controller Mode * ENABLEHOST = 0 and ENABLETARGET = 1 — Target Mode * ENABLEHOST = 1 and ENABLETARGET = 1 — Hybrid Mode * ENABLEHOST = 0 and ENABLETARGET = 0 — Monitor Mode
0	ENABLEHOST	RW	0x0	Global control: configures this device's operating mode. * ENABLEHOST = 1 and ENABLETARGET = 0 — Controller Mode * ENABLEHOST = 0 and ENABLETARGET = 1 — Target Mode * ENABLEHOST = 1 and ENABLETARGET = 1 — Hybrid Mode * ENABLEHOST = 0 and ENABLETARGET = 0 — Monitor Mode

Table 534. STATUS Register

Bits	Field	Access	Reset	Description
31:11	Reserved	-	-	Reserved
10	ACK_CTRL_STRETCH	R	0x0	Target Mode status: indicates that this target is stretching the clock due to the Software ACK Control Mechanism. See <code>CTRL.ACK_CTRL_EN</code> for details.
9	ACQEMPTY	R	0x1	Target Mode status: Target RX FIFO empty.
8	TXEMPTY	R	0x1	Target Mode status: Target TX FIFO empty.
7	ACQFULL	R	0x0	Target Mode status: Target RX FIFO full.
6	TXFULL	R	0x0	Target Mode status: Target TX FIFO full.
5	RXEMPTY	R	0x1	Controller Mode status: Controller RX FIFO empty.
4	TARGETIDLE	R	0x1	Target Mode status: Target FSM idle.
3	HOSTIDLE	R	0x1	Controller Mode status: Controller FSM idle.
2	FMTEEMPTY	R	0x1	Controller Mode status: Controller TX FIFO Empty.
1	RXFULL	R	0x0	Controller Mode status: Controller RX FIFO Full.
0	FMTFULL	R	0x0	Controller Mode status: Controller TX FIFO Full.

Table 535. RDATA Register

Bits	Field	Access	Reset	Description
31:8	Reserved	-	-	Reserved
7:0	DATA	R	0x0	Reading this register pops the Controller RX FIFO.

Table 536. FDATA Register

Bits	Field	Access	Reset	Description
31:13	Reserved	-	-	Reserved
12	NAKOK	W	0x0	NACK OK. When set, this controller will not care if this byte is NACKed. It will not halt, set the <code>CONTROLLER_EVENTS.NACK</code> flag, or assert the <code>CONTROLLER_HALT</code> Interrupt. This behavior is useful for protocols like Serial Camera Control Bus (SCCB).

Bits	Field	Access	Reset	Description
11	RCONT	W	0x0	Read continue/stop: * 0 — Read Stop: The controller NACKs the last data byte. Use this if this is the last READ in a sequence. * 1 — Read Continue: The controller ACKs the last data byte. Use this if this is an intermediate READ in a sequence.
10	READB	W	0x0	Read/write: * 0 - Issue WRITE transaction * 1 - Issue READ transaction
9	STOP	W	0x0	Generate a STOP condition on the bus after sending the byte.
8	START	W	0x0	Generate a START condition on the bus before sending the byte.
7:0	FBYTE	W	0x0	Writing to this register pushes an entry into the Controller TX FIFO. The meaning of this field depends on the value of the READB field: * READB = 0 - This field is the WRITE data byte. * READB = 1 - This field specifies the number of bytes to read from the target. Setting this field to 0 reads 256 bytes.

Table 537. FIFO_CTRL Register

Bits	Field	Access	Reset	Description
31:9	Reserved	-	-	Reserved
8	TXRST	W	0x0	Target Mode control: writing 1 resets the Target TX FIFO.
7	ACQRST	W	0x0	Target Mode control: writing 1 resets the Target RX FIFO.
6:2	Reserved	-	-	Reserved
1	FMTRST	W	0x0	Controller Mode control: writing 1 resets the Controller TX FIFO.
0	RXRST	W	0x0	Controller Mode control: writing 1 resets the Controller RX FIFO.

Table 538. HOST_FIFO_CONFIG Register

Bits	Field	Access	Reset	Description
31:28	Reserved	-	-	Reserved
27:16	FMT_THRESH	RW	0x0	The FMT_THRESH interrupt remains asserted while the Controller TX FIFO level is less than this setting.
15:12	Reserved	-	-	Reserved
11:0	RX_THRESH	RW	0x0	The RX_THRESH interrupt remains asserted while the Controller RX FIFO level is greater than this setting.

Table 539. TARGET_FIFO_CONFIG Register

Bits	Field	Access	Reset	Description
31:28	Reserved	-	-	Reserved
27:16	ACQ_THRESH	RW	0x0	The ACQ_THRESH interrupt remains asserted while the Target RX FIFO level is greater than this setting.
15:12	Reserved	-	-	Reserved
11:0	TX_THRESH	RW	0x0	The TX_THRESH interrupt remains asserted while the Target TX FIFO level is less than this setting.

Table 540. HOST_FIFO_STATUS Register

Bits	Field	Access	Reset	Description
31:28	Reserved	-	-	Reserved
27:16	RXLVL	R	-	Controller RX FIFO fill level.
15:12	Reserved	-	-	Reserved

Bits	Field	Access	Reset	Description
11:0	FMTLVL	R	-	Controller TX FIFO fill level.

Table 541. TARGET_FIFO_STATUS Register

Bits	Field	Access	Reset	Description
31:28	Reserved	-	-	Reserved
27:16	ACQLVL	R	0x0	Target RX FIFO fill level.
15:12	Reserved	-	-	Reserved
11:0	TXLVL	R	0x0	Target TX FIFO fill level.

Table 542. OVRD Register

Bits	Field	Access	Reset	Description
31:3	Reserved	-	-	Reserved
2	SDAVAL	RW	0x0	Global control: SDA override value: * 0 - Pull the SDA line low * 1 - Release the SDA line
1	SCLVAL	RW	0x0	Global control: SCL override value: * 0 - Pull the SCL line low * 1 - Release the SCL line
0	TXOVRDEN	RW	0x0	Global control: enables control of the SDA and SCL lines through the SDA_VAL and SCL_VAL fields, respectively.

Table 543. VAL Register

Bits	Field	Access	Reset	Description
31:16	SDA_RX	R	0x0	Global status: contains the last 16 SDA over-sampled values. LSB is most recent.
15:0	SCL_RX	R	0x0	Global status: contains the last 16 SCL over-sampled values. LSB is most recent.

Table 544. TIMING0 Register

Bits	Field	Access	Reset	Description
31:29	Reserved	-	-	Reserved
28:16	TLOW	RW	0x0	Global control: specifies the LOW period of SCL (t _{LOW}) in system clock cycles. Must be ≥ 2. See Table 11 in the I2C Specification for more details. This field is sized to meet the I2C Standard-mode (100 kHz) minimum t _{LOW} = 4.7 μs requirement, assuming a 1 GHz system clock.
15:13	Reserved	-	-	Reserved
12:0	THIGH	RW	0x0	Global control: specifies the HIGH period of SCL (t _{HIGH}) in system clock cycles. Must be ≥ 2. See Table 11 in the I2C Specification for details. This field is sized to meet the I2C Standard-mode (100 kHz)'s minimum t _{HIGH} = 4.0 μs requirement, assuming a 1 GHz system clock.

Table 545. TIMING1 Register

Bits	Field	Access	Reset	Description
31:25	Reserved	-	-	Reserved

Bits	Field	Access	Reset	Description
24:16	T_F	RW	0x0	Global control: specifies the fall time of SDA and SCL (t_f) in system clock cycles. The fall time is measured from 70% to 30% of the signal swing. See Table 11 in the I2C Specification for more details. This field is sized to meet I2C Standard-mode (100 kHz)'s maximum $t_f = 300\text{ ns}$ requirement, assuming a 1 GHz system clock.
15:10	Reserved	-	-	Reserved
9:0	T_R	RW	0x0	Global control: specifies the rise time of SDA and SCL (t_r) in system clock cycles. The rise time is measured from 30% to 70% of the signal swing. See Table 11 in the I2C Specification for more details. This field is sized to meet I2C Standard-mode (100 kHz)'s maximum $t_r = 1000\text{ ns}$ requirement, assuming a 1 GHz system clock.

Table 546. TIMING2 Register

Bits	Field	Access	Reset	Description
31:29	Reserved	-	-	Reserved
28:16	THD_STA	RW	0x0	Global control: specifies the setup time for a (repeated) START condition ($t_{HD;STA}$) in system clock cycles. See Table 11 in the I2C Specification for details. This field is sized to meet I2C Standard-mode (100 kHz)'s minimum $t_{HD;STA} = 4.0\text{ }\mu\text{s}$ requirement, assuming a 1 GHz system clock.
15:13	Reserved	-	-	Reserved
12:0	TSU_STA	RW	0x0	Global control: specifies the setup time for a repeated START condition ($t_{SU;STA}$) in system clock cycles. See Table 11 in the I2C Specification for details. This field is sized to meet I2C Standard-mode (100 kHz)'s minimum $t_{SU;STA} = 4.7\text{ }\mu\text{s}$ requirement, assuming a 1 GHz system clock.

Table 547. TIMING3 Register

Bits	Field	Access	Reset	Description
31:29	Reserved	-	-	Reserved
28:16	THD_DAT	RW	0x0	Global control: specifies the data and (N)ACK bits hold time ($t_{HD;DAT}$) in system clock cycles. See Table 11 in the I2C Specification for details. This field is sized to meet the I2C Standard-mode (100 kHz)'s $t_{HD;DAT} = 5.0\text{ }\mu\text{s}$ minimum requirement, assuming a 1 GHz system clock.
15:9	Reserved	-	-	Reserved
8:0	TSU_DAT	RW	0x0	Global control: specifies the data setup time ($t_{SU;DAT}$) in system clock cycles. See Table 11 in the I2C Specification for details. This field is sized to meet the I2C Standard-mode (100 kHz)'s $t_{SU;DAT} = 250\text{ ns}$ minimum requirement, assuming a 1 GHz system clock.

Table 548. TIMING4 Register

Bits	Field	Access	Reset	Description
31:29	Reserved	-	-	Reserved
28:16	T_BUF	RW	0x0	Global control: specifies the time between a STOP and START condition (t_{BUF}) in system clock cycles. See Table 11 in the I2C Specification for details. This field is sized to meet I2C Standard-mode (100kHz)'s $t_{BUF} = 4.7\text{ }\mu\text{s}$ minimum requirement, assuming a 1 GHz system clock.
15:13	Reserved	-	-	Reserved

Bits	Field	Access	Reset	Description
12:0	TSU_STO	RW	0x0	Global control: specifies the setup time for a STOP condition ($t_{SU;STO}$) in system clock cycles. See Table 11 in the I ² C Specification for details. This field is sized to meet I2C Standard-mode (100 kHz)'s $t_{SU;STO} = 4.0 \mu s$ minimum requirement, assuming a 1 GHz system clock.

Table 549. TIMEOUT_CTRL Register

Bits	Field	Access	Reset	Description
31	EN	RW	0x0	Timeout Enable.
30	MODE	RW	0x0	Global control: timeout mode. * 0 - Stretch Timeout (Controller Mode only). If the target stretches the clock for more time than TIMEOUT_CTRL.VAL, the STRETCH_TIMEOUT interrupt will be asserted. * 1 - Bus Timeout. If SCL is LOW for more time than TIMEOUT_CTRL.VAL: * In Controller Mode, the CONTROLLER_EVENTS.BUS_TIMEOUT flag will be asserted, triggering the CONTROLLER_HALT interrupt. * In Target Mode, the TARGET_EVENTS.BUS_TIMEOUT flag will be set, triggering the TX_STRETCH interrupt.
29:0	VAL	RW	0x0	Global control: Specifies the timeout value in system clock cycles. The meaning of this field depends on MODE.

Table 550. TARGET_ID Register

Bits	Field	Access	Reset	Description
31:28	Reserved	-	-	Reserved
27:21	MASK1	RW	0x0	ADDRESS1 mask.
20:14	ADDRESS1	RW	0x0	Target Address 1. This target responds if the 7-bit address matches ADDRESS1 & MASK1. MASK1 = 0x0 disables this address.
13:7	MASK0	RW	0x0	ADDRESS0 mask.
6:0	ADDRESS0	RW	0x0	Target Address 0. This target responds if the 7-bit address matches ADDRESS0 & MASK0. MASK0 = 0x0 disables this address.

Table 551. ACQDATA Register

Bits	Field	Access	Reset	Description
31:11	Reserved	-	-	Reserved
10:8	SIGNAL	R	0x0	This field indicates if this FIFO entry represents/is associated with control signal(s): * 0x0 - The entry is an ordinary data byte that has been ACKed. * 0x1 - The entry is an address byte preceded by a START. * 0x2 - The entry is a STOP condition following ACKed data bytes. * 0x3 - The entry is an address byte preceded by a repeated START. * 0x4 - The entry is a NACKed data byte. * 0x5 - The entry is an address byte preceded by a (repeated) START. However, the data bytes that followed this address byte were NACKed. * 0x6 - Error. A transaction preceding ended abnormally, for example, due to an unexpected STOP condition, a Bus or Stretch Timeout, a software NACK, or a lost arbitration. If the FIFO does not have enough space to record a complete transaction, an Error (0x6) entry may appear alone.
7:0	ABYTE	R	0x0	Reading this register pops the ACQ FIFO. The meaning of this field depends on SIGNAL: * SIGNAL = 0x0, 0x1, 0x3, 0x4, or 0x5 - This field contains an address or data byte sent by the controller. * SIGNAL = 0x2, 0x6 - This field is meaningless.

Table 552. TXDATA Register

Bits	Field	Access	Reset	Description
31:8	Reserved	-	-	Reserved
7:0	DATA	W	0x0	Writing to this register pushes data into the Target TX FIFO. The controller reads data from this FIFO during a READ transaction.

Table 553. HOST_TIMEOUT_CTRL Register

Bits	Field	Access	Reset	Description
31	Reserved	-	-	Reserved
30:0	VAL	RW	0x0	Target/Monitor Mode control: controller clock generation timeout value specified in system clock cycles. * Target Mode - If the controller stops generating the clock for more time than this setting, this target asserts the Controller Timeout Interrupt. * Monitor Mode - this field is required to be nonzero for the Bus Monitor to transition out of the initial Busy state. Set this field to 0x0 to disable this behavior.

Table 554. TARGET_TIMEOUT_CTRL Register

Bits	Field	Access	Reset	Description
31	EN	RW	0x0	Enable Target Timeout.
30:0	VAL	RW	0x0	When this target has stretched the clock for more time than this setting, this target will NACK incoming data bytes or release the SDA line for outgoing data bytes. The count is cumulative over an entire transaction. In other words, this is SMBus's cumulative target clock extension time. The behavior for the address byte is configurable via CTRL.NACK_ADDR_AFTER_TIMEOUT.

Table 555. TARGET_NACK_COUNT Register

Bits	Field	Access	Reset	Description
31:8	Reserved	-	-	Reserved
7:0	TARGET_NACK_COUNT	RW	0x0	Indicates the number of transactions NACKed by this target since the last read of this register, saturating at 255. This field can be used to track how many transactions were missed when the Target RX FIFO is full.

Table 556. TARGET_ACK_CTRL Register

Bits	Field	Access	Reset	Description
31	NACK	W	0x0	When STATUS.ACK_CTRL_STRETCH = 1, writing 1 to this field causes this target to NACK all bytes of the transaction. Effective only when CTRL.ACK_CTRL_EN = 1.
30:9	Reserved	-	-	Reserved
8:0	NBYTES	RW	0x0	When STATUS.ACK_CTRL_STRETCH = 1, writing to this register specifies the number of bytes this target should ACK. The count decrements per byte ACKed. Effective only when CTRL.ACK_CTRL_EN = 1. See CTRL.ACK_CTRL_EN for details. This control field is used to implement SMBus's mid-transfer (N)ACK responses.

Table 557. ACQ_FIFO_NEXT_DATA Register

Bits	Field	Access	Reset	Description
31:8	Reserved	-	-	Reserved
7:0	DATA	R	0x0	This field contains the next byte to be pushed into the Target RX FIFO, allowing software to decide whether to accept or reject it. Valid only when <code>STATUS.ACK_CTRL_STRETCH = 1</code> . See <code>CTRL.ACK_CTRL_EN</code> for details.

Table 558. HOST_NACK_HANDLER_TIMEOUT Register

Bits	Field	Access	Reset	Description
31	EN	RW	0x0	Enables Controller NACK Timeout.
30:0	VAL	RW	0x0	Timeout value (specified in system clock cycles) for this controller Mode to automatically STOP a transaction when <code>CONTROLLER_EVENTS.NACK</code> is set.

Table 559. CONTROLLER_EVENTS Register

Bits	Field	Access	Reset	Description
31:4	Reserved	-	-	Reserved
3	ARBITRATION_LOST	RW	0x0	Indicates that the controller has halted due to it losing an arbitration against another controller. This bit triggers the <code>CONTROLLER_HALT</code> interrupt. Writing 1 clears this bit.
2	BUS_TIMEOUT	RW	0x0	Indicates that this controller has halted due to a Bus Timeout. See <code>TIMEOUT_CTRL</code> for details. This bit triggers the <code>CONTROLLER_HALT</code> interrupt. Writing 1 clears this bit.
1	UNHANDLED_NACK_TIMEOUT	RW	0x0	Indicates that this controller has halted due to a Controller NACK Timeout. See <code>HOST_NACK_HANDLER_TIMEOUT</code> for details. This bit triggers the <code>CONTROLLER_HALT</code> interrupt. Writing 1 clears this bit.
0	NACK	RW	0x0	Indicates that this controller has halted due to an unexpected NACK sent by the target. This behavior can be disabled by writing 1 to <code>FDATA.NAKOK</code> . This bit triggers the <code>CONTROLLER_HALT</code> interrupt. Writing 1 clears this bit.

Table 560. TARGET_EVENTS Register

Bits	Field	Access	Reset	Description
31:5	Reserved	-	-	Reserved
4	STOP_DETECT	RW	0x0	Stop Detect Flag (Target Mode only). Set to 1 by hardware when a STOP is detected while <code>CTRL.ENABLETARGET = 1</code> . Cleared when software writes 1.
3	START_DETECT	RW	0x0	Start Detect Flag (Target Mode only). Set to 1 by hardware when a START (or repeated START) is detected while <code>CTRL.ENABLETARGET = 1</code> . Cleared when software writes 1.
2	ARBITRATION_LOST	RW	0x0	Indicates that a controller has lost arbitration, causing a READ transaction to end. This bit triggers the <code>TX_STRETCH</code> interrupt. Writing 1 clears this bit.
1	BUS_TIMEOUT	RW	0x0	Indicates that this target has halted due a Bus Timeout terminating a READ transaction. See <code>TIMEOUT_CTRL</code> for details. This bit triggers the <code>TX_STRETCH</code> interrupt. Writing 1 clears this bit.

Bits	Field	Access	Reset	Description
0	TX_PENDING	RW	0x0	Indicates that the target is stretching the clock due to receiving a READ address byte and waiting for software to confirm the release of the Target TX FIFO data. Valid only if CTRL.TX_STRETCH_CTRL_EN = 1. See CTRL.TX_STRETCH_CTRL_EN for details. This bit triggers the TX_STRETCH interrupt. Writing 1 clears this bit.

Table 561. SMBUS_STATUS Register

Bits	Field	Access	Reset	Description
31:5	Reserved	-	-	Reserved
4	SMBALERT	R	0x0	Controller Mode status: indicates that the smbalert_ni input is asserted.
3:1	Reserved	-	-	Reserved
0	SMBSUS	R	0x0	Target Mode status: indicates that the smbsus_ni input is asserted.

Programming

Initialization Sequence

After reset, the initialization of the I2C HWIP primarily consists of four steps:

1. Timing parameter initialization
2. FIFO reset and configuration
3. Interrupt configuration
4. Enable I2C Controller or Target functionality

Timing Parameter Initialization

Of the four initialization steps, the timing parameter initialization is the most involved. With so many timing parameters, it is essential to have dedicated device interface functions (DIFs) to determine appropriate values for the 10 timing parameters.

See [Timing Parameter Tuning Algorithm](#) for a detailed algorithm for determining these parameters.

FIFO Reset and Configuration

Reset all FIFOs and configure thresholds:

```
// Reset all FIFOs
write32(base + 0x20, 0x1E); // FIFO_CTRL: Reset all FIFOs

// Controller FIFO configuration
write32(base + 0x24, 0x00080010); // HOST_FIFO_CONFIG: RX_THRESH=8, FMT_THRESH=16

// Target FIFO configuration
write32(base + 0x28, 0x00080010); // TARGET_FIFO_CONFIG: ACQ_THRESH=8, TX_THRESH=16
```

Interrupt Configuration

Enable desired interrupts:

```
// Clear all interrupts first
write32(base + 0x00, 0xFFFF); // INTR_STATE: Clear all interrupts

// Enable desired interrupts
write32(base + 0x04, 0x0203); // INTR_ENABLE: Enable common interrupts
```

Enable Operation Mode

Enable controller or target functionality:

```
// Enable controller mode
write32(base + 0x10, 0x01); // CTRL: ENABLEHOST=1

// Or enable target mode
// write32(base + 0x10, 0x02); // CTRL: ENABLETARGET=1
```

Timing Parameter Tuning Algorithm

The values of these parameters will depend primarily on three bus details:

- The speed mode of the slowest device on the bus: Standard-mode (100 kbaud), Fast-mode (400 kbaud) or Fast-mode Plus (1 Mbaud).
- The input clock period, t_{clk} in ns.
- The expected signal rise time, t_r , in ns.
 - This is not a firmware-controlled parameter. Rather, it is a function of the capacitance and physical design of the bus. The specification provides detailed guidelines on how to manage capacitance in an I2C system:
 - Section 5.2 of the I2C specification indicates that Fast-mode Plus devices may operate at reduced clock speeds if the bus capacitance drives signal rise times (t_r) outside the nominal 120ns limit. Excess capacitance can also be compensated for by reducing the size of the bus pullup resistor, so long as the total open-drain current does not exceed 20mA for Fast-mode Plus devices (as described in section 7.1 of the I2C specification). However the specification places a hard limit on rise times capping them at 1000ns.
 - If there are Standard- or Fast-mode target devices on the bus, the specified open-drain current limit is reduced to 3mA (section 7.1), thus further restricting the minimum value of the pull-up resistor.
 - In Fast-mode bus designs, where the total line capacitance exceeds 200pF, the specification recommends replacing the pull-up resistor with an active current source, supplying 3mA or less (section 5.1). Regardless of the physical construction of the bus, the rise time (t_r) is a system dependent, parameter that needs to be made known to firmware for I2C initialization.
- The expected fall time, t_f , in ns.
 - Like t_r , this parameter is not firmware controlled rather it is a function of the SCL driver, which in a strictly compliant device is expected to manage the slew-rate for the falling edge of the SDA and SCL signals, through proper design of the SCL output buffer.

- See table 10 of the I2C specification for more details.
- (optional) The desired SCL cycle period, $t_{SCL,user}$ in ns.
 - By default the device should operate at the maximum frequency for that mode. However, If the system developer wishes to operate at slower than the mode-specific maximum, a larger than minimum period could be allowed as an additional functional parameter when calculating the timing parameters.

Additional Constraints:

- To guarantee clock stretching works correctly with the Controller Module, there is a requirement of $THIGH \geq 4 + InputDelayCycles$. This constraint derives from the fact that there is a latency between the Controller FSM driving the bus and observing the effect of driving the bus. The implementation requires $THIGH$ to be at least this large to guarantee that if the Target stretches the clock, we can observe it in time, and react accordingly. Note that $InputDelayCycles$ is a hardware instance parameter provided by the top-level integration.

Based on the inputs, the timing parameters may be chosen using the following algorithm:

1. Step 1: Look up minimum timing values

The physical timing parameters $t_{HD,STA}$, $t_{SU,STA}$, $t_{HD,DAT}$, $t_{SU,DAT}$, t_{BUF} , and t_{STO} , t_{HIGH} , and t_{LOW} all have minimum allowed values which depend on the choice of speed mode (Standard-mode, Fast-mode or Fast-mode Plus). Using the speed mode input, look up the appropriate minimum value (in ns) for each parameter (i.e. $t_{HD,STA,min}$, $t_{SU,STA,min}$, etc)

2. Step 2: Convert to integer minimums

For each of these eight parameters, obtain an integer minimum by dividing the physical minimum parameter by the clock period and rounding up to the next highest integer:

$$\text{\texttt{THIGH_MIN}} = \max(\lceil t_{HIGH,min} / t_{clk} \rceil, 4)$$

$$\text{\texttt{TLOW_MIN}} = \lceil t_{LOW,min} / t_{clk} \rceil$$

$$\text{\texttt{THD_STA_MIN}} = \lceil t_{HD,STA,min} / t_{clk} \rceil$$

$$\text{\texttt{TSU_STA_MIN}} = \lceil t_{SU,STA,min} / t_{clk} \rceil$$

$$\text{\texttt{THD_DAT_MIN}} = \lceil t_{HD,DAT,min} / t_{clk} \rceil$$

$$\text{\texttt{TSU_DAT_MIN}} = \lceil t_{SU,DAT,min} / t_{clk} \rceil$$

$$\text{\texttt{T_BUF_MIN}} = \lceil t_{BUF,min} / t_{clk} \rceil$$

$$\text{\textrm{T_STO_MIN}} = \lceil t_{\text{\{STO,min\}}}/t_{\text{\{clk\}}} \rceil$$

+ Note that **T_HD_DAT_MIN** must be at least 1, and **T_HD_STA_MIN** and **T_BUF_MIN** must be greater than **T_HD_DAT_MIN**.

1. Step 3: Program timing registers

Input the integer timing parameters, THD_STA_MIN, TSU_STA_MIN, THD_DAT_MIN, TSU_DAT_MIN, T_BUF_MIN and T_STO_MIN into their corresponding registers (**TIMING2.TH_STA**, **TIMING2.TSU_STA**, **TIMING3.TH_DAT**, **TIMING3.TSU_DAT**, **TIMING4.T_BUF**, **TIMING4.T_STO**) **This step allows the firmware to manage SDA signal delays to ensure that the SDA outputs are compliant with the specification.** The registers **TIMING0.THIGH** and **TIMING0.TLOW** will be taken care of in a later step.

2. Step 4: Convert rise and fall times

Take the given values for t_r and t_f and convert them to integer counts as well:

$$\text{\textrm{T_R}} = \lceil t_{\text{\{r\}}}/t_{\text{\{clk\}}} \rceil$$

$$\text{\textrm{T_F}} = \lceil t_{\text{\{f\}}}/t_{\text{\{clk\}}} \rceil$$

1. Step 5: Store T_R and T_F

Store T_R and T_F in their corresponding registers: **TIMING1.T_R** and **TIMING1.T_F**.

2. Step 6: Calculate minimum SCL period

Based on the input speed mode, look up the maximum permissible SCL frequency ($f_{\text{SCL,max}}$) and calculate the minimum permissible SCL period:

$$t_{\text{\{SCL,min\}}} = 1/f_{\text{\{SCL,max\}}}$$

1. Step 7: Convert to integer periods

As with each of the other physical parameters convert $t_{\text{SCL,min}}$ and, if provided, the $t_{\text{SCL,user}}$ to integers, MINPERIOD and USERPERIOD.

$$\text{MINPERIOD} = \lceil t_{\text{\{SCL,min\}}}/t_{\text{\{clk\}}} \rceil$$

$$\text{USERPERIOD} = \lceil t_{\text{\{SCL,user\}}}/t_{\text{\{clk\}}} \rceil$$

1. Step 8: Determine PERIOD

Let **PERIOD** = **max(MINPERIOD, USERPERIOD)**.

2. Step 9: Calculate THIGH and TLOW

Each SCL cycle will now be at least **PERIOD** clock cycles in duration, divided between four segments: **T_R**, **THIGH**, **T_F**, and **TLOW**. **In other words: $PERIOD = T_R + THIGH + T_F + TLOW$** . With **T_R** and **T_F** already established, the remaining integer parameters **THIGH** and **TLOW** are to be divided among the remaining clock cycles in **PERIOD**:

$$\text{THIGH} + \text{TLOW} \geq \text{PERIOD} - \text{T}_F - \text{T}_R$$

- Since t_{HIGH} and t_{LOW} both have minimum allowable values, which depends on the mode, high values of t_r or t_f may force an increase in the total SCL period, slowing down the data transit rate.
- The balance between t_{HIGH} and t_{LOW} can be manipulated in a variety of different ways (depending on the desired SCL duty cycle).
- It is, for instance, perfectly acceptable to simply set TLOW to the minimum possible value:

$$\text{TIMING0.TLOW} = \text{TLOW}_{MIN}$$

+ THIGH is then set to satisfy both constraints in the desired SCL period and in the minimum permissible values for t_{HIGH} :

$$\text{TIMING0.THIGH} = \max(\text{PERIOD} - \text{T}_R - \text{TIMING0.TLOW} - \text{T}_F, \text{THIGH}_{MIN})$$

Note that the actual frequency will generally be lower, due to the need to round up counts after dividing. In addition, clock stretching can delay transitions. To achieve nominal rates within 5% or so, we recommend that the internal clock is driven at least 24x higher than the desired line speed.

Timing Parameter Examples

The following tables show a couple of examples for calculating timing register parameters for Fast-mode Plus devices. Both examples assume a desired data rate of 1 Mbaud (the bus maximum) for an SCL period of 1 μ s, and an internal device clock period of 3 ns.

Table 562. Fast-mode Plus Example (Normal Capacitance)

Parameter	Spec. Min. (ns)	Reg. Val.	Phys. Val (ns)	Comment
TIMING0.THIGH	260	120	360	Chosen to satisfy SCL Period Minimum
TIMING0.TLOW	500	167	501	Spec. t_{LOW} Minimum
TIMING1.T _F	20ns * (VDD/5.5V)	7	21	Signal slew-rate should be controlled
TIMING1.T _R	0	40	120	Based on pull-up resistance, line capacitance
SCL Period	1000	N/A	1002	Constraint on THIGH+TLOW+T _R +T _F
TIMING2.THD_STA	260	87	261	Spec. Minimum
TIMING2.TSU_STA	260	87	261	Spec. Minimum
TIMING3.THD_DAT	0	0	0	Spec. Minimum
TIMING3.TSU_DAT	260	87	261	Spec. Minimum
TIMING4.T_BUF	500	167	501	Spec. Minimum

Parameter	Spec. Min. (ns)	Reg. Val.	Phys. Val (ns)	Comment
TIMING4.T_STO	260	87	161	Spec. Minimum

This next example shows how the first SCL timing registers: **TIMING0** and **TIMING1** are altered in a high-capacitance Fast-mode Plus bus, where the physical value of t_r driven to an atypical value of 400ns. As in the previous example the integer register values are determined based on a system clock period, t_{clk} , of 3ns. All other parameters in registers **TIMING2**, **TIMING3**, **TIMING4** are unchanged from the previous example.

Table 563. High-Capacitance Fast-mode Plus Example

Parameter	Spec. Min. (ns)	Reg. Val.	Phys. Val (ns)	Comment
TIMING0.THIGH	260	87	261	Spec. t_{HIGH} Minimum
TIMING0.TLOW	500	167	501	Spec. t_{LOW} Minimum
TIMING1.T_F	20ns * (VDD/5.5V)	7	21	Signal slew-rate should be controlled
TIMING1.T_R	0	134	402	Atypically high line capacitance
SCL Period	1000	N/A	1185	Forced longer than minimum by long T_R

Controller Module Operations

Writing n bytes to a device

- Address the device for writing by writing to:
 - FDATA.START** = 1;
 - FDATA.FBYTE** = <7-bit address + write bit>.
- Fill the FMT FIFO by writing to **FDATA.FBYTE** n-1 times.
- Send last byte with the stop bit by writing to:
 - FDATA.STOP** = 1;
 - FDATA.FBYTE** = <last byte>.
- Wait for and check the result of the transaction.
 - If **INTR_STATE.CONTROLLER_HALT** is 1, the transaction failed.
 - Check **CONTROLLER_EVENTS** for the reason, reset the FMT FIFO with **FIFO_CTRL.FMTRST**, and clear the latched events.
 - Otherwise, if **INTR_STATE.CMD_COMPLETE** is 1 and **STATUS.FMEMPTY** is 1, the transaction succeeded.

Example code:

```
// Write address with START and write bit
write32(base + 0x1C, 0x0400A0); // FDATA: START=1, FBYTE=7-bit addr (0x50) + W

// Write data bytes (n-1 times)
write32(base + 0x1C, 0x00AA); // FDATA: Data byte 0xAA
write32(base + 0x1C, 0x00BB); // FDATA: Data byte 0xBB

// Send last byte with STOP
write32(base + 0x1C, 0x0200AA); // FDATA: STOP=1, FBYTE=last byte 0xAA
```

```

// Wait for completion
while (!(read32(base + 0x00) & 0x200)); // Wait for CMD_COMPLETE

// Check for errors
if (read32(base + 0x00) & 0x40) { // CONTROLLER_HALT
    // Check CONTROLLER_EVENTS for reason
    uint32_t events = read32(base + 0x78);
    // Handle error...
    // Reset FMT FIFO
    write32(base + 0x20, 0x08); // FIFO_CTRL: FMTRST=1
    // Clear events
    write32(base + 0x78, events);
}

// Clear interrupt
write32(base + 0x00, 0x200); // Clear CMD_COMPLETE

```

Reading n bytes from a device

1. Address the device for reading by writing to the FMT FIFO:
 - **FDATA.START** = 1;
 - **FDATA.FBYTE** = <7-bit address + read bit>.
2. Issue a read transfer by writing to the FMT FIFO.
 - **FDATA.READ** = 1;
 - **FDATA.STOP** = 1;
 - **FDATA.FBYTE** = <`n`>.
3. Wait for the transfer to finish or interrupt by checking:
 - If **STATUS.FMTEMPT** bit is 1.
 - Or if **INTR_STATE.FMT_THRESHOLD** bit is 1 (as long as **FIFO_CTRL.FMTILVL** is set to 1).
 - If **INTR_STATE.CONTROLLER_HALT** is 1.
4. If the **INTR_STATE.CONTROLLER_HALT** bit is 1, then set **FIFO_CTRL.FMTRST** to reset the FMT FIFO, clear latched events in **CONTROLLER_EVENTS**, and go back to step 1. Otherwise, proceed.
5. If **STATUS.FMTEMPT** is 1, the read transfer has completed.
 - Retrieve the data from the FIFO by reading **RDATA** n times.

Example code:

```

// Address device for reading
write32(base + 0x1C, 0x0400A1); // FDATA: START=1, FBYTE=7-bit addr (0x50) + R

// Issue read transfer
write32(base + 0x1C, 0x030004); // FDATA: READ=1, STOP=1, FBYTE=n=4

```



```

// Wait for completion or error
while (!(read32(base + 0x00) & (0x200 | 0x40))); // Wait for CMD_COMPLETE or
CONTROLLER_HALT

if (read32(base + 0x00) & 0x40) { // CONTROLLER_HALT
    // Reset FMT FIFO
    write32(base + 0x20, 0x08); // FIFO_CTRL: FMTRST=1
    // Clear events
    uint32_t events = read32(base + 0x78);
    write32(base + 0x78, events);
    // Retry from step 1...
} else if (read32(base + 0x14) & 0x01) { // STATUS.FMEMPTY
    // Read data from RX FIFO
    uint8_t data[4];
    for (int i = 0; i < 4; i++) {
        data[i] = read32(base + 0x18) & 0xFF; // RDATA
    }
}

// Clear interrupt
write32(base + 0x00, 0x200); // Clear CMD_COMPLETE

```

Target Mode Configuration Sequence

1. Configure Target Address:

```

// Set target address to 0x48
write32(base + 0x54, 0x48); // TARGET_ID: ADDRESS0=0x48

```

2. Enable Target Mode:

```

// Enable target mode
write32(base + 0x10, 0x02); // CTRL: ENABLETARGET=1

```

3. Service Target Transactions:

```

// Check for received data
if (read32(base + 0x00) & 0x04) { // ACQ_THRESHOLD interrupt
    uint32_t acq_data = read32(base + 0x58); // Read ACQDATA

    // Process received data based on address/data format
    // For read transactions, load response data:
    write32(base + 0x5C, response_data); // TXDATA
}

```

```

// Clear interrupt
write32(base + 0x00, 0x04); // Clear ACQ_THRESHOLD
}

```

Error Handling Sequences

1. Bus Error Recovery:

```

uint32_t intr_state = read32(base + 0x00);

if (intr_state & 0x60) { // SCL or SDA interference
    // Reset controller state
    uint32_t ctrl = read32(base + 0x10);
    write32(base + 0x10, ctrl & ~0x01); // Disable controller
    write32(base + 0x10, ctrl); // Re-enable controller

    // Clear error interrupts
    write32(base + 0x00, 0x60);
}

```

2. FIFO Overflow Handling:

```

if (read32(base + 0x00) & 0x08) { // RX_OVERFLOW
    // Reset RX FIFO
    write32(base + 0x20, 0x02); // FIFO_CTRL: RXRST=1

    // Clear overflow interrupt
    write32(base + 0x00, 0x08);

    // Restart transaction as needed
}

```

Usage Examples and Best Practices

High-Performance Controller Operation

```

// High-throughput controller configuration
#define I2C_BASE 0x40000000

// Configure for maximum performance
write32(I2C_BASE + 0x24, 0x00040030); // Aggressive FIFO thresholds
write32(I2C_BASE + 0x04, 0x0003); // Enable FMT and RX threshold interrupts

// Use interrupt-driven transfers for efficiency
void i2c_controller_write_burst(uint8_t addr, uint8_t* data, int len) {

```

```

// Load entire transaction into FIFO
write32(I2C_BASE + 0x1C, 0x0400 | (addr << 1)); // START + address + W
for (int i = 0; i < len; i++) {
    write32(I2C_BASE + 0x1C, data[i]);
}
write32(I2C_BASE + 0x1C, 0x0200); // STOP

// Wait for completion interrupt
while (!(read32(I2C_BASE + 0x00) & 0x200));
write32(I2C_BASE + 0x00, 0x200); // Clear interrupt
}

```

Multi-Address Target Configuration

```

// Configure target with multiple address ranges
void configure_multi_address_target() {
    // Primary address: 0x50
    write32(I2C_BASE + 0x54, 0x50);

    // Enable target mode with manual ACK control for address filtering
    write32(I2C_BASE + 0x10, 0x0A); // ENABLETARGET=1, ACK_CTRL_EN=1

    // Configure ACK control for selective response
    write32(I2C_BASE + 0x6C, 0x01); // TARGET_ACK_CTRL: Enable manual control
}

```

Error Recovery and Diagnostics

```

// Comprehensive error handling
void handle_i2c_errors() {
    uint32_t intr_state = read32(I2C_BASE + 0x00);

    if (intr_state & 0x60) { // Bus interference
        printf("Bus contention detected: 0x%x\n", intr_state);

        // Attempt bus recovery
        write32(I2C_BASE + 0x34, 0x07); // Override mode: drive both lines high
        usleep(100); // Brief delay
        write32(I2C_BASE + 0x34, 0x00); // Release override

        // Reset controller
        uint32_t ctrl = read32(I2C_BASE + 0x10);
        write32(I2C_BASE + 0x10, 0);
        write32(I2C_BASE + 0x10, ctrl);
    }
}

```

```

    if (intr_state & 0x08) { // RX overflow
        printf("RX FIFO overflow - possible data loss\n");
        write32(I2C_BASE + 0x20, 0x02); // Reset RX FIFO
    }

    // Clear all error interrupts
    write32(I2C_BASE + 0x00, intr_state);
}

```

Performance Monitoring

```

// Monitor FIFO utilization for optimization
void monitor_fifo_performance() {
    uint32_t host_status = read32(I2C_BASE + 0x2C);
    uint32_t target_status = read32(I2C_BASE + 0x30);

    printf("Controller FIFOs - FMT: %d, RX: %d\n",
           (host_status >> 16) & 0x7F, host_status & 0x7F);
    printf("Target FIFOs - ACQ: %d, TX: %d\n",
           (target_status >> 16) & 0x1FF, target_status & 0x7F);

    // Adjust thresholds based on utilization patterns
    if ((host_status & 0x7F) > 32) { // RX FIFO getting full
        write32(I2C_BASE + 0x24, 0x00100010); // Increase RX threshold
    }
}

```

Device Interface Functions (DIFs)

For convenience, OpenTitan provides Device Interface Functions (DIFs) that abstract the low-level register programming details. These DIFs handle timing parameter calculation, FIFO management, and transaction sequencing automatically.

For more information, refer to the OpenTitan DIF documentation: [Device Interface Functions](#)

5.10.7. AVSBus Controller

Overview

The AVSBus Controller is a hardware IP that implements the AVSBus 1.3.1 specification for adaptive voltage scaling communication. It acts as a controller (master) that communicates with a single AVS target device (slave) to manage voltage rails, monitor power conditions, and perform real-time voltage adjustments for optimal performance and energy efficiency. This version supports 32-bit frame-based communication without multi-target addressing.

Description and Features

- **Single-Target Support:** Communicates with a single AVS target device through a two-wire protocol interface
- **Dual Bus Interface:** Supports both APB4 and AXI4-Lite register interfaces for system integration flexibility
- **Command FIFO:** 8-deep command queue for efficient transaction management and burst operations
- **Readback FIFO:** 8-deep response buffer with automatic retry mechanism for reliable data collection
- **Flexible Clock Configuration:** Programmable AVS clock generation with configurable divider and duty cycle control
- **Comprehensive Interrupt Support:** Status monitoring with configurable interrupt sources for system-level event handling
- **Slave Resync Operation:** Automatic periodic resynchronization with configurable intervals

Architecture

The AVSBus Controller implements an Adaptive Voltage Scaling Bus (AVSBus) 1.3.1 controller for dynamic voltage and frequency scaling (DVFS) in single-target systems. The controller enables communication with a single AVS target device through a standardized two-wire protocol interface.

The modular design includes separate command processing, response handling, clock generation, and CRC verification subsystems. FIFO-based command queuing enables autonomous operation with monitoring and error handling capabilities.

AVS Protocol Engine

The protocol engine serves as the heart of AVSBus communication, orchestrating all state machine transitions and transaction management. Incoming transactions are first queued through an 8-entry command FIFO, which supports all AVSBus command types—including commit write, hold write, commit, and read—while automatically constructing proper frames with built-in CRC calculation. Configurable timeouts and retry logic ensure that even in the face of transient failures, reliable communication is maintained.

On the response side, an 8-deep readback FIFO buffers data returned from the target, allowing for efficient response handling and minimizing latency. The protocol engine provides real-time acknowledgment tracking for the target device, and each response is immediately subjected to hardware CRC verification for error detection. For operations demanding the lowest latency, the engine includes a bypass path enabling instantaneous access to target responses.

Transaction control is further enhanced by automatic slave resync operations, allowing periodic resynchronization with configurable intervals. An abort function immediately cancels pending operations as needed, while automatic idle state detection enables power-efficient clock gating. Throughout, the protocol engine offers detailed status monitoring and debug access to facilitate robust system integration and troubleshooting.

Clock Generation and Management

Multi-Source Clock Selection

Clock Source	Description
clk_reg_i	Register interface clock, synchronous with system register access
clk_ref_i	Reference clock, independent AVS bus timing
Selection	Software-configurable via AVS_CFG_1 register

The clock generation logic features a programmable divider with 8-bit resolution, enabling AVS clock frequencies to be finely adjusted anywhere between divide-by-2 and divide-by-255. Both the division ratio and the duty cycle (also 8-bit configurable) can be optimized by software for signal integrity, with the controller shipping by default at a divide-by-4, 50% duty cycle setting. These clock parameters may be reprogrammed on the fly without interrupting protocol activity.

Power efficiency is built in at multiple levels: the AVS clock is automatically gated during bus idle conditions, and all unused input clocks are selectively disabled to further reduce dynamic power consumption.

CRC Verification Engine

The controller features a hardware-accelerated CRC-3 engine, implemented as the dedicated avsbus_crc3 module. This logic works in real time, processing each 32-bit message and generating or checking the required 3-bit CRC with zero-cycle latency, ensuring continuous protocol operation.

CRC calculation runs in parallel for both outgoing and incoming frames, automatically verifying all received responses and flagging the result—good or bad—on a per-transaction basis. Any CRC errors are tracked in dedicated statistics registers, and the CRC system is tightly integrated with the retry mechanism to ensure robust error recovery.

Command and Response FIFO System

FIFO	Command FIFO	Response FIFO
Depth	8 entries	8 entries
Purpose	Command queue for batch operations	Response buffer with automatic collection
Features	Full/empty status, automatic frame formatting, CRC insertion, overflow protection	Acknowledgment tracking, CRC verification, underflow protection

Reset and Initialization

The AVSBus Controller employs a dual reset scheme to ensure robust, phased hardware bring-up. The primary reset signal (rst_ni) governs all core clocking and register functions; it must be asserted first to stabilize the system. Only after this clock and register domain is reliably up can configuration begin.

The clock divider reset (rst_clk_div_ni) must be deasserted before the main reset to ensure proper clock generation initialization.

Initialization Sequence:

- 1. **Clock Configuration Phase:**
 - Configure clock source selection (AVS_CFG_1.AVS_CLOCK_SELECT)
 - Set clock divider and duty cycle parameters (AVS_CFG_1)
 - Enable appropriate clock gating controls
- 2. **Clock Divider Reset Release:**
 - Deassert rst_clk_div_ni to enable clock divider
 - Verify clock divider stability
- 3. **Primary Reset Release:**
 - Deassert rst_ni to enable clocking subsystems
 - Verify clock stability and register access functionality
 - Configure basic controller parameters

4. Protocol Initialization:

- Configure retry parameters (AVS_CFG_0.MAX_RETRIES)
- Configure resync interval (AVS_CFG_0.RESYNC_INTERVAL)
- Set up interrupt masks and status monitoring
- Clear any residual FIFO contents

5. Begin Operations:

- Enable protocol engine and FIFO operations
- Begin normal command processing operations

Data Flow and Control Path

Command Processing Data Flow

1. **Command Initiation:** Software writes transaction parameters to AVS_CMD register
2. **FIFO Queuing:** Command parameters queued in 8-deep command FIFO
3. **Frame Construction:** Protocol engine constructs 32-bit AVSBus frame with:
 - Command type and data type encoding
 - 16-bit data payload
 - 3-bit CRC calculated by avsbus_crc3 module
4. **Serial Transmission:** Frame transmitted bit-by-bit on avs_mdata_o
5. **Response Collection:** Target response collected from avs_sdata_i
6. **CRC Verification:** Received response verified using CRC-3 engine
7. **FIFO Storage:** Valid response queued in readback FIFO for software access

Command Flow Control:

```
// Typical command processing sequence
while (!cmd_fifo_full) {
    // Queue multiple commands
    write_reg(AVS_CMD, {cmd_type, data_type, rail_sel, data});
}

// Monitor command execution
while (cmd_fifo_not_empty) {
    status = read_reg(AVS_NORMAL_STATUS);
    if (status.bus_idle) break;
}
```

Response Processing Data Flow

1. **Target Response Reception:** Serial collection from target via avs_sdata_i
2. **Target Acknowledgment Processing:** ACK/NACK detection and status update

3. **CRC Validation:** Hardware CRC-3 verification for response frame
4. **Retry Logic:** Automatic retry for CRC errors or missing acknowledgments
5. **Status Update:** Real-time update of AVS_SLAVE_STATUS register
6. **FIFO Queuing:** Valid response stored in 8-deep readback FIFO
7. **Interrupt Generation:** Configurable interrupts for response availability

Response Bypass Mode: - AVS_LATEST_SLAVE_SUBFRAME provides immediate access to target responses - Bypasses FIFO for real-time monitoring applications - Useful for debugging and system commissioning

Clock Domain Architecture

The AVSBus Controller is designed around a multi-clock domain architecture to balance performance, flexibility, and safe communication across hardware boundaries. At its heart is the Register Interface Domain, clocked by `clk_reg_i`, which orchestrates all register accesses, manages FIFO write operations, updates status registers, and handles the generation of interrupts. This domain is responsible for interfacing with software and ensuring that the system configuration and state reporting remain responsive and reliable.

Parallel to this, the AVS Protocol Domain operates using its own clock (`clk_avs`), which is flexibly derived from either the register interface clock (`clk_reg_i`) or a separate reference clock (`clk_ref_i`). The choice of source, along with programmable division ratios and tunable duty cycles, enables precise control over protocol timing and ensures the protocol engine state machine can meet demanding bit-level timing requirements. This separation allows the protocol core to run independently of software activity, optimizing throughput and timing closure.

To ensure robust operation between these asynchronous domains, the design employs classic cross-domain synchronization strategies. Gray-coded counters facilitate the safe passage of FIFO pointer information, minimizing hazards from metastability. Control signals crossing domain boundaries are handled through two-stage synchronizers, guarding against timing violations. With these measures in place, all domain crossings benefit from strong metastability protection, maintaining data integrity and system stability even as the domains operate at different rates.

Clock Generation:

The AVSBus Controller's clock generation subsystem is both flexible and power-efficient, adapting seamlessly to system requirements. Clock source selection is based on a straightforward logic: when the `AVS_CLOCK_SELECT` signal is set to 00 or 01, the register interface clock (`clk_reg_i`) is chosen as the protocol clock; when set to 10 or 11, the controller selects the reference clock (`clk_ref_i`). This mechanism enables precise control over protocol timing, allowing the designer to align the AVS domain with either the register clock or an external reference source as system needs dictate.

To further refine clock characteristics, the controller incorporates a programmable divider. This divider accepts an 8-bit value, enabling clock ratios from 2 up to 255, and supports fine-grained duty cycle adjustment via another 8-bit parameter that maps directly to a 0–100% range. These features together allow for highly accurate protocol clock generation and make it possible to match stringent timing requirements or optimize interface throughput.

The design includes automatic clock gating that disables the protocol clock during idle periods, complemented by software controls that can explicitly turn off the protocol clock when needed to conserve energy. This strategy minimizes unnecessary clock activity, reducing overall power consumption and ensuring that the clock tree is only as active as required by ongoing operations.

Protocol State Machine

The protocol engine implements a state machine for AVSBus communication:

IDLE State: - Waiting for commands from software - AVS clock optionally gated for power savings - Monitoring for FIFO empty/full conditions

FRAME_SETUP State: - Frame construction and CRC calculation - Clock enable assertion and timing setup - Command preparation

TRANSMIT State: - Bit-by-bit frame transmission on avs_mdata_o - Parallel bit counter and frame position tracking - Clock generation and data synchronization

RESPONSE_WAIT State: - Target response collection from avs_sdata_i - Acknowledgment detection and timeout monitoring - CRC verification preparation

RESPONSE_PROCESS State: - CRC validation and error detection - Response data extraction and formatting - FIFO write operations and status updates

RETRY_DECISION State: - Error analysis and retry determination - Retry counter management - Abort condition evaluation

RESYNC State: - Slave resynchronization operation (34 clock cycles) - Automatic periodic resync with configurable intervals

Performance Characteristics

Characteristic	Value
Protocol Timing	
Max AVS clock frequency	system clock / 2
Frame transmission time	32 AVS clock cycles minimum
Response processing	2-5 AVS clock cycles
Command-to-response latency	40-80 AVS clock cycles (including retries)
Sustainable throughput	~1 command per 50 AVS clock cycles
Resource Utilization (estimates)	
Logic resources	~1200 LUTs total (protocol: ~400, FIFO: ~300, clock: ~200, CRC: ~100, register: ~200)
Memory	576 bits (command FIFO: 320, response FIFO: 256), register file: 128-256 bytes
Clock resources	2-4 global clocks, 8-16 local enables, 4-8 gating cells
Error Handling	
Configurable retry count	1-255 attempts (via AVS_CFG_0.MAX_RETRIES)
Error recovery time	3-10 AVS clock cycles per retry
Max transaction latency	(retry_count + 1) × base_latency

Error Handling and Diagnostics

Robust error handling in the AVSBus Controller is achieved through a combination of proactive detection, on-the-fly diagnostics, and versatile recovery strategies. The controller continuously monitors for common protocol errors such as CRC mismatches, missed acknowledgments, and protocol violations. CRC errors trigger automatic retry, while target response timeouts are detected using programmable thresholds. Additional safeguards guard against FIFO overflow or underflow, errors in clock domain crossings, and excessive retry attempts, ensuring resilient bus operation even under adverse conditions.

To aid diagnostics, the controller offers real-time visibility into internal protocol states and maintains detailed target

response status. A transaction history and error logging subsystem assist with deeper debugging, supplemented by a comprehensive register interface designed for both system monitoring and low-level inspection. Real-time metrics on bus utilization further augment the visibility available to software and hardware integrators.

When recovery is necessary, the controller combines automatic mechanisms—including configurable retry, slave resync operations, transaction aborts, and FIFO flushing—with software-intervenable controls such as manual abort triggers, direct diagnostic access, and customization of error thresholds. This multi-tiered approach provides both rapid error response and ample opportunity for system-level management and adaptation.

Interface

The AVSBus Controller provides a configurable system bus interface (APB4 or AXI4-Lite), a two-wire AVSBus protocol interface, and clock/reset interfaces for system integration. Dual clock domain support enables flexible clock source selection for optimal timing and power management.

Configuration Parameters

Parameter	Default	Description
BUS_PROTOCOL	APB4	System bus interface (APB4 or AXI4-Lite)
CMD_FIFO_DEPTH	8	Command FIFO depth (entries)
RSP_FIFO_DEPTH	8	Response FIFO depth (entries)
CLK_DIV_WIDTH	8	Clock divider bit width (2-255 ratios)
DUTY_CYCLE_WIDTH	8	Duty cycle control bit width

AVSBus Protocol Interface

AVSBus Signal Definition

Table 564. AVSBus Protocol Signals

Signal Name	Width	Direction	Description
avs_clock_o	1	Output	AVSBus protocol clock - Generated by controller
avs_mdata_o	1	Output	Master data - Serial command transmission
avs_sdata_i	1	Input	Slave data - Serial response from target device

AVSBus Clock Characteristics: - Frequency: Programmable division of system clock (divide by 2-255) - Duty Cycle: Configurable 8-bit control (0-255 maps to 0-100%) - Clock Gating: Automatic gating during idle periods if enabled - Clock Domain: Independent from register interface clock

Serial Data Protocol: - avs_mdata_o: LSB-first serial transmission at avs_clock_o rate - Frame Length: 32 bits total (29 data + 3 CRC bits) - Bit Rate: 1 bit per avs_clock_o rising edge - Idle State: avs_mdata_o held high between transactions

Target Response Collection: - avs_sdata_i: Serial collection from target device - Response sampled on avs_clock_o edges - Target acknowledgment extracted from response frame

AVSBus Frame Format

Command Frame Structure (32 bits total):

Table 565. AVSBus Command Frame Format

Field Name	Bit Range	Width	Description
Command Type	[31:30]	2	Operation type (0=commit write, 1=hold write, 2=read)
Command Group	[29]	1	Command group (0=standard, 1=manufacturer-specific)
Command Code	[28:25]	4	Data category (0=voltage, 1=rate, 2=current, 3=temp, etc.)
Rail Select	[24:21]	4	Rail selection (0-15, 0xF=broadcast)
Data Payload	[20:5]	16	Command-specific data (voltage values, configuration, etc.)
Reserved	[4:3]	2	Reserved
CRC-3	[2:0]	3	Cyclic redundancy check for error detection

Command Type Encoding:

Table 566. AVSBus Command Types

Code	Command Name	Description
0x0	Commit Write	Write data and immediately commit voltage changes
0x1	Hold Write	Write data but hold changes until commit command
0x2	Read	Read current values from target device

Data Type Encoding:

Table 567. AVSBus Data Types

Code	Data Type	Resolution	Description
0x0	Target Rail Voltage	1mV	16-bit voltage values for power rails
0x1	Vout Transition Rate	1mV/μs	Rise/fall rate configuration
0x2	Rail Current	10mA	Current measurement readback
0x3	Temperature	0.1°C	Temperature sensor readings
0x4	Force Voltage Reset	N/A	Emergency voltage reset command
0x5	Power Mode	N/A	Efficiency vs performance mode
0xE	AVSBus Status	N/A	Target status and error reporting
0xF	AVSBus Version	N/A	Target version readback

Response Frame Structure (32 bits):

Table 568. AVSBus Response Frame Format

Field Name	Bit Range	Width	Description
Slave Acknowledgment	[31:30]	2	ACK/NACK status (0=action performed, 1=resource unavailable, 2=bad CRC, 3=bad data)
Status Response	[29:25]	5	5-bit status field indicating general condition of slave
Response Data	[24:5]	20	Target-specific response data
Reserved	[4:3]	2	Reserved
CRC-3	[2:0]	3	Cyclic redundancy check for error detection

Clock and Reset Interface

Multi-Clock Architecture

The controller supports flexible clocking for optimal system integration and timing closure.

Clock Input Signals:

Table 569. Clock Input Signals

Signal Name	Width	Direction	Description
clk_reg_i	1	Input	Register interface clock - Synchronous with bus operations
clk_ref_i	1	Input	Reference clock - Independent AVS timing source
test_en_i	1	Input	Test enable - DFT override for clock selection
scan_rst_ni	1	Input	Scan reset - Test infrastructure reset control

Clock Output Signals:

Table 570. Clock Output Signals

Signal Name	Width	Direction	Description
avs_clock_o	1	Output	AVSBus protocol clock - Drives target device

Clock Configuration and Control: - Clock source selection via AVS_CFG_1.AVS_CLOCK_SELECT[1:0] - Programmable divider: AVS_CFG_1.CLK_DIVIDER_VALUE[7:0] (range 2-255) - Duty cycle control: AVS_CFG_1.CLK_DIVIDER_DUTY_CYCLE_NUMERATOR[7:0] (0-255) - Clock gating controls for power management (AVS_CFG_1.STOP_AVS_CLOCK_ON_IDLE)

Reset Architecture

Dual-domain reset system providing staged initialization control.

Reset Input Signals:

Table 571. Reset Input Signals

Signal Name	Width	Direction	Description
rst_ni	1	Input	Primary system reset - Active low asynchronous
rst_clk_div_ni	1	Input	Clock divider reset - Active low asynchronous
scan_rst_ni	1	Input	Scan chain reset - Test infrastructure control

Internal Reset Domains:

Table 572. Internal Reset Domains

Reset Domain	Control	Description
Clock Divider Domain	rst_clk_div_ni	Controls clock divider (must be deasserted first)
Clock/Register Domain	rst_ni	Controls basic clocking and register access

Reset Timing Requirements: 1. Assert rst_clk_div_ni and rst_ni to initialize all domains 2. Configure clock sources and parameters 3. Deassert rst_clk_div_ni to enable clock divider 4. Deassert rst_ni to enable register access 5. Configure protocol parameters and FIFO settings 6. Begin normal AVSBus operations

Interrupt Interface

Comprehensive interrupt generation for system-level event handling and status monitoring.

Interrupt Output Signal:

Table 573. Interrupt Output

Signal Name	Width	Direction	Description
irq_o	1	Output	Interrupt request - Active high level assertion

Interrupt Sources and Control: - Command FIFO status events (full, empty, overflow) - Response FIFO status events (data available, full, underflow) - Transaction completion and error conditions - Target acknowledgment and timeout events - Protocol state changes and error recovery - Clock domain issues and reset events - Slave resync operations

Interrupt Configuration: - Individual interrupt source enable/disable control via AVS_INTERRUPT_MASK - Interrupt status register (AVS_INTERRUPT) with clear-on-write functionality (AVS_INTERRUPT_CLEAR) - Level triggering for all interrupt sources

GPIO and Test Interface

Additional interface signals for system integration and testing support.

General Purpose I/O:

Table 574. GPIO Signals

Signal Name	Width	Direction	Description
AVS_GPIO_ENABLE	1	Register	GPIO mode enable - Overrides normal AVS operation (AVS_CONFIG register)
GPIO pins	3	I/O	External AVS clock, master data, and slave data (GPIO mode)

Test and Debug Interface:

Table 575. Test Interface Signals

Signal Name	Width	Direction	Description
test_en_i	1	Input	Test mode enable - Affects clock selection logic
scan_rst_ni	1	Input	Scan chain reset for manufacturing test
scan_mode_i	1	Input	Scan mode control - Enables scan chain operation

Protocol Timing Specifications

AVSBus Timing Parameters

Clock Timing: - Minimum avs_clock_o period: $2 * \text{clk_reg_i period}$ - Maximum avs_clock_o frequency: $\text{clk_reg_i frequency} / 2$ - Duty cycle range: 10% to 90% (configurable) - Clock jitter tolerance: $\pm 5\%$ of clock period

Data Setup and Hold Times: - avs_mdata_o setup time: 1ns before avs_clock_o rising edge - avs_sdata_i setup time: 2ns before avs_clock_o rising edge - avs_sdata_i hold time: 1ns after avs_clock_o rising edge - Propagation delay: avs_clock_o to avs_mdata_o < 10ns

Transaction Timing: - Frame transmission time: 32 avs_clock_o cycles - Slave resync operation: 34 avs_clock_o cycles - Inter-frame gap: Automatic based on protocol state - Target response timeout: Configurable per application - Maximum

transaction latency: Frame time + response time + processing overhead

System Integration Guidelines

Bus Interface Integration

APB4 Integration:

```
avsbus_controller_apb4 u_avs_ctrl (  
    // APB4 interface  
    .pclk_i      (apb_clk),  
    .presetn_i   (apb_rst_n),  
    .paddr_i     (apb_paddr[7:0]), // 8-bit address for 256-byte space  
    .psel_i      (apb_psel_avs),  
    .penable_i   (apb_penable),  
    .pwrite_i    (apb_pwrite),  
    .pwrite_i    (apb_pwrite),  
    .pwrite_i    (apb_pwrite),  
    .pstrb_i     (apb_pstrb),  
    .pready_o    (apb_pready_avs),  
    .prdata_o    (apb_prdata_avs),  
    .pslverr_o   (apb_pslverr_avs),  
  
    // Clock and reset  
    .clk_reg_i   (apb_clk),  
    .clk_ref_i   (ref_clk_100m),  
    .rst_ni      (system_rst_n),  
    .rst_clk_div_ni (system_rst_n),  
  
    // AVSBus interface  
    .avs_clock_o (avs_clk_out),  
    .avs_mdata_o (avs_cmd_out),  
    .avs_sdata_i (avs_response_in),  
  
    // System interface  
    .irq_o       (avs_irq)  
);
```

AXI4-Lite Integration:

```
avsbus_controller_axi4lite u_avs_ctrl (  
    // AXI4-Lite interface  
    .aclk_i      (axi_clk),  
    .aresetn_i   (axi_rst_n),  
  
    // AXI4-Lite Write Channels  
    .awvalid_i   (axi_awvalid),  
    .awready_o   (axi_awready),
```

```

        .awaddr_i    (axi_awaddr[7:0]),
        .wvalid_i    (axi_wvalid),
        .wready_o    (axi_wready),
        .wdata_i     (axi_wdata),
        .wstrb_i     (axi_wstrb),
        .bvalid_o    (axi_bvalid),
        .bready_i    (axi_bready),
        .bresp_o     (axi_bresp),

// AXI4-Lite Read Channels
        .arvalid_i   (axi_arvalid),
        .arready_o   (axi_arready),
        .araddr_i    (axi_araddr[7:0]),
        .rvalid_o    (axi_rvalid),
        .rready_i    (axi_rready),
        .rdata_o     (axi_rdata),
        .rresp_o     (axi_rresp),

// Clock and reset
        .clk_reg_i   (axi_clk),
        .clk_ref_i   (ref_clk_100m),
        .rst_ni      (system_rst_n),
        .rst_clk_div_ni (system_rst_n),

// AVSBus interface
        .avs_clock_o (avs_clk_out),
        .avs_mdata_o (avs_cmd_out),
        .avs_sdata_i (avs_response_in),

// System interface
        .irq_o       (avs_irq)
);

```

Memory Map

Overview

The AVSBus Controller provides a comprehensive register interface through a 256-byte address space (0x00-0xFF) accessible via AXI4-Lite or APB4 protocols. The register map is organized into functional groups including control registers for transaction management, status registers for monitoring operations, configuration registers for system parameters, and FIFO interfaces for command and response data handling.

All registers are 32-bit aligned and designed for efficient software access patterns. The interface supports both polling and interrupt-driven operation modes, with extensive status reporting and diagnostic capabilities for robust system integration and debugging.

Register Address Map

Table 576. Register Address Map

Address	Name	Access	Description
0x00	AVS_CMD	W	Command and Data Register - Write-only FIFO for AVS transactions
0x04	AVS_READBACK	R	Response Data Register - Read-only FIFO for target responses
0x08	AVS_DEBUG_READBACK	R	Debug Readback Register - Non-destructive read of FIFO contents
0x0C	AVS_LATEST_SLAVE_SUBFRAME	R	Latest Slave Subframe - Bypass FIFO for immediate access
0x20	AVS_NORMAL_STATUS	R	FIFO and Bus Status - Standard operation indicators
0x24	AVS_SLAVE_STATUS	R	Slave Status Information - Target status and acknowledgment
0x28	AVS_FIFOS_STATUS	R	FIFO Occupancy Counters - Detailed FIFO utilization
0x30	AVS_INTERRUPT	R	Interrupt Status Register - Interrupt source identification
0x34	AVS_INTERRUPT_MASK	RW	Interrupt Mask Register - Individual interrupt source enables
0x38	AVS_INTERRUPT_CLEAR	W	Interrupt Clear Register - Clear interrupt bits
0x50	AVS_CFG_0	RW	Configuration Register 0 - Retry and resync configuration
0x54	AVS_CFG_1	RW	Configuration Register 1 - Clock configuration and control
0x58	AVS_CONFIG	RW	General Configuration - GPIO enable and system controls
0x5C-0xFF	Reserved		Reserved address space for future extensions

Address Alignment and Access Requirements

Address Alignment: - All registers aligned to 32-bit boundaries (addresses multiples of 4) - 256-byte total address space (0x00-0xFF) - Compatible with standard AXI4-Lite and APB4 addressing schemes

Access Width Requirements: - All registers must be accessed using 32-bit operations - Byte and halfword accesses not supported - may cause unpredictable behavior - AXI4-Lite: Write strobes should be 4'b1111 for all register writes - APB4: All pstrb_i bits should be asserted for register writes

Atomic Access Considerations: - FIFO registers (AVS_CMD, AVS_READBACK) are inherently atomic - Status registers provide consistent snapshots of hardware state - No special atomic access patterns required for normal operation

Reset Values and Power-On Behavior

Table 577. Register Reset Values

Register	Reset Value	Description
AVS_CMD	N/A	Write-only register, no reset state
AVS_READBACK	0x00000000	Empty FIFO, no response data
AVS_DEBUG_READBACK	0xFFFFFFFF	No data available
AVS_LATEST_SLAVE_SUBFRAME	0x0000FFFF	No response data available
AVS_NORMAL_STATUS	0x00000000	Bus idle, FIFOs empty
AVS_SLAVE_STATUS	0x00000000	No slave status available
AVS_FIFOS_STATUS	0x88000000	FIFOs empty (8 vacant slots each)
AVS_INTERRUPT	0x00000000	No pending interrupts
AVS_INTERRUPT_MASK	0x000001FF	All interrupts masked by default
AVS_INTERRUPT_CLEAR	0x00000000	No effect on read
AVS_CFG_0	0x00051000	Max retries=5, resync interval=0x1000

Register	Reset Value	Description
AVS_CFG_1	0x00000003	Div=4 (default), duty=128 (50%), clk_ref_i divided
AVS_CONFIG	0x00000001	GPIO enabled by default

Detailed Register Map

Table 578. Register Map

Address	Name	Access	Description
0x0	AVS_CMD	W	The command+data to be transferred to the AVS bus
0x4	AVS_READBACK	R	Response data from AVS slave device (updated in response to both AVS read and write commands). Reading this register causes the AVS response readback fifo pointer to advance.
0x8	AVS_DEBUG_READBACK	R	This register mirrors the AVS_READBACK register, but reading it does NOT affect the AVS readback fifo pointer. Provided for debug purposes.
0xC	AVS_LATEST_SLAVE_SUBFRAME	R	This register contains the most recently received subframe from the AVS slave, by-passing the AVS readback fifo. It is provided so that APB can see the current slave StatusResponse and SlaveAck settings in a timely fashion, since the status data from the next entry in the readback fifo might have considerable latency relative to when it was initially received from the AVS Slave.
0x20	AVS_NORMAL_STATUS	R	Various Status bits indicating normal operating conditions for APB2AVS bridge block. These all have corresponding maskable bits in the AVS_INTERRUPT register
0x24	AVS_SLAVE_STATUS	R	Contains the most recent status feedback from the AVS slave device
0x28	AVS_FIFOS_STATUS	R	Contains the number of occupied and vacant slots in the AVS master's command and readback FIFOs
0x30	AVS_INTERRUPT	R	Interrupt register - A '1' indicates an interrupt has occurred for that bit. To clear the interrupt bit, write to the corresponding field in the AVS_INTERRUPT_CLEAR register.
0x34	AVS_INTERRUPT_MASK	RW	Interrupt mask register - Determines which AVS_INTERRUPT bits can trigger an interrupt
0x38	AVS_INTERRUPT_CLEAR	W	Write 1 to clear corresponding interrupt bit
0x50	AVS_CFG_0	RW	Configuration register for APB2AVS bridge block
0x54	AVS_CFG_1	RW	Configuration register for APB2AVS bridge block
0x58	AVS_CONFIG	RW	AVS Configuration Register

Table 579. AVS_CMD Register

Bits	Field	Access	Reset	Description
31:30	Reserved	-	-	Reserved
29:28	R_OR_W	W	0x0	Read or write type: [list] []0x0 - Commit Write []0x1 - Hold Write [*]0x2 - Read [/list]
27	CMD_GRP	W	0x0	Command group: Set to 1 for manufacturer-specific Command Code (cmd_code), otherwise 0
26:23	CMD_CODE	W	0x0	Command Code (For cmd_group=0. For cmd_group=1, this is manufacturer-specific): [list] []0x0 - Target Rail Voltage Read/Write []0x1 - Vout Transition Rate Read/Write []0x2 - Rail Current Read []0x3 - Temperature Read []0x4 - Force Voltage Reset (Requires wr_cmd_data=0x0) []0x5 - Power Mode Read/Write []0xe - AVSBus Status Read/Write []0xf - AVSBus Version Read [/list]

Bits	Field	Access	Reset	Description
22:19	RAIL_SEL	W	0x0	Rail Select: 0x0 - Rail 0 0x1 - Rail 1 0xf - Broadcast
18:3	CMD_DATA	W	0x0	Write Cmd Data. Formats: [list] [Voltage (cmd_code 0x0): 16-bit unsigned int, 1LSB=1mV] [Transition Rate (cmd_code 0x1): [list] [MSByte: Rise Rate, 8-bit unsigned int, 1LSB=1mV/us] [LSByte: Fall Rate, 8-bit unsigned int, 1LSB=1mV/us] [Reset Voltage (cmd_code 0x4): 16-bits, all zeroes] [Power Mode (cmd_code 0x5): 3-bits, LSB aligned: [list] [0x0 - Maximum Efficiency [0x3 - Maximum Power [0x4-0x7 - Manufacturer-specific [list] [AVSBus Status (cmd_code 0xe): 16-bits, encoded per AVS Spec [AVSBus Version (cmd_code 0xf): 4 bits encoded per AVS Spec, LSB-aligned] [For read cmd_code's, this is 0xffff] [list]
2:0	Reserved	-	-	Reserved

Table 580. AVS_READBACK Register

Bits	Field	Access	Reset	Description
31:30	SLAVE_ACK	R	0x0	Indicates whether command executed or not - codes 0x1 and 0x2 trigger automatic retries by hardware: [list] [0x0: Action Performed] [0x1: Good CRC received but resource unavailable - hardware will retry] [0x2: Bad CRC received - hardware will retry] [0x3: Good CRC received but bad data, data type or selector - no hardware retry] [list]
29	CONST0	R	0x0	Always 0
28:24	STATUS_RESPONSE	R	0x0	5-bit field indicating general condition of the slave, per AVS spec StatusReponse field: [list] [bit 4: VDone - AND of all VDone bits of active rails] [bit 3: StatusAlert - Indicates potential status issue, requiring AVSBus Status read cmd for details] [bit 2: AVS_Control - 1 if ABSBus is controlling at least 1 device output, 0 otherwise.] [bit 1: MfrSpcfc_Stts1 - Manufacturer-specific [*]] [bit 0: MfrSpcfc_Stts2 - Manufacturer-specific [list]
23:8	CMD_DATA	R	0x0	Read Cmd Data. Format depends on cmd_code in previous master subframe:[list] [Voltage (cmd_code 0x0): 16-bit unsigned int, 1LSB=1mV] [Transition Rate (cmd_code 0x1): [list] [MSByte: Rise Rate, 8-bit unsigned int, 1LSB=1mV/us] [LSByte: Fall Rate, 8-bit unsigned int, 1LSB=1mV/us] [list] [Current (cmd_code 0x2): 16-bit unsigned, 1LSB=10mA] [Temperature (cmd_code 0x3): 16-bit signed, 1LSB=0.1degC] [Power Mode (cmd_code 0x5): 3-bits, LSB aligned: [list] [0x0 - Maximum Efficiency [0x3 - Maximum Power [0x4-0x7 - Manufacturer-specific [list] [AVSBus Status (cmd_code 0xe): 16-bits, encoded per AVS Spec [AVSBus Version (cmd_code 0xf): 4 bits encoded per AVS Spec, LSB-aligned [*]] [If master issued a write (rw!=0x2) in previous master subframe, this field is 0xffff] [list]
7:3	Reserved	-	-	Reserved
2:0	CRC	R	0x0	The CRC-3 code received from slave

Table 581. AVS_DEBUG_READBACK Register

Bits	Field	Access	Reset	Description
31:0	AVS_SLAVE_SUBFRAME	R	0xFFFFFFFF	The current top-of-fifo AVS slave response sub-frame in the readback fifo

Table 582. AVS_LATEST_SLAVE_SUBFRAME Register

Bits	Field	Access	Reset	Description
31:0	AVS_SLAVE_SUBFRAME	R	0xFFFF	The most recently received AVS slave response sub-frame

Table 583. AVS_NORMAL_STATUS Register

Bits	Field	Access	Reset	Description
31:23	Reserved	-	-	Reserved
22	AVS_SLAVE_IS_IN_RESYNC	R	0x0	Indicates that the AVS master is issuing a slave resync operation (34 clock cycles of holding mdata high)
21	AVS_BUS_IS_IDLE	R	0x0	Indicates that neither AVS master nor slave is currently driving a transaction on the AVS bus
20	READBACK_HAS_DATA	R	0x0	Indicates AVS_READBACK register has data ready to read on APB
19	READBACK_FIFO_FULL	R	0x0	Indicates read-back fifo is full - no new AVS commands will be launched until space is freed by an AVS_READBACK access
18	CMD_FIFO_FULL	R	0x0	Indicates AVS_CMD fifo is full - any additional APB writes to AVS_CMD will be dropped until the AVS bus can accept the next command, thus freeing space in the cmd fifo
17	CMD_FIFO_EMPTY	R	0x0	Indicates AVS_CMD fifo is empty
16	AVS_MASTER_IS_RETRYING	R	0x0	Master is currently in the middle of retrying a failed command
15:0	TOTAL_RETRIES	R	0x0	Total number of retries attempted by the AVS Master

Table 584. AVS_SLAVE_STATUS Register

Bits	Field	Access	Reset	Description
31:18	Reserved	-	-	Reserved
17:16	AVS_SLAVE_ACK	R	0x0	The SlaveAck bits of the most recent slave subframe received: [list] []00: Action Performed []01: Good CRC received by resource unavailable. No action performed (triggers retry) []10: Bad CRC received (triggers retry) []11: Good CRC received bit bad data, data type, or selector. No action performed [/list]
15:5	Reserved	-	-	Reserved
4:0	AVS_SLAVE_STATUS_RESPONSE	R	0x0	The 5-bit Slave StatusResponse of the most recent slave subframe received, indicating the general condition of the slave, per AVS spec StatusReponse field: [list] []bit 4: VDone - AND of all VDone bits of active rails []bit 3: StatusAlert - Indicates potential status issue, requiring AVSBus Status read cmd for details []bit 2: AVS_Control - 1 if ABSBus is controlling at least 1 device output, 0 otherwise. []bit 1: MfrSpfcStts1 - Manufacturer-specific [*]bit 0: MfrSpfcStts2 - Manufacturer-specific [/list]

Table 585. AVS_FIFOS_STATUS Register

Bits	Field	Access	Reset	Description
31:28	Reserved	-	-	Reserved
27:24	READBACK_FIFO_VACANT_SLOTS	R	0x8	Number of vacant slots in the readback FIFO
23:20	Reserved	-	-	Reserved
19:16	READBACK_FIFO_OCCUPIED_SLOTS	R	0x0	Number of occupied slots in the readback FIFO
15:12	Reserved	-	-	Reserved
11:8	CMD_FIFO_VACANT_SLOTS	R	0x8	Number of vacant slots in the command FIFO

Bits	Field	Access	Reset	Description
7:4	Reserved	-	-	Reserved
3:0	CMD_FIFO_OCCUPIED_SLOTS	R	0x0	Number of occupied slots in the command FIFO

Table 586. AVS_INTERRUPT Register

Bits	Field	Access	Reset	Description
31:9	Reserved	-	-	Reserved
8	READBACK_OVERFLOW_INT	R	0x0	Indicates that the response from AVS slave could not be written to readback FIFO due to it being full ⇒ response was dropped
7	READBACK_UNDERFLOW_INT	R	0x0	Indicates that an APB read on the AVS_READBACK reg occurred when the readback fifo was empty (AVS_NORMAL_STATUS:readback_has_data=0)
6	CMD_FIFO_OVERFLOW_INT	R	0x0	Indicates that an APB write to AVS_CMD was attempted when the the command fifo was already full - the command is dropped
5	MAX_RETRIES_ATTEMPTED_INT	R	0x0	Indicates that the AVS Master failed to receive a non-failure acknowledgement from the Slave after a command was retried the maximum allowable number of times (as determined by AVS_CFG_0:max_retries)
4	SLAVE_UNRESPONSIVE_INT	R	0x0	Indicates the AVS Slave device is not responding to command subframes from the Master
3	READBACK_HAS_DATA_INT	R	0x0	Indicates AVS_READBACK register has data ready to read on APB
2	READBACK_FIFO_FULL_INT	R	0x0	Indicates read-back fifo is full - no new AVS commands will be launched until space is freed by an AVS_READBACK access
1	CMD_FIFO_FULL_INT	R	0x0	Indicates AVS_CMD fifo is full - any additional APB writes to AVS_CMD will be dropped until the AVS bus can accept the next command, thus freeing space in the cmd fifo
0	AVS_SLAVE_ISSUED_INTERRUPT	R	0x0	Indicates the AVS slave has signaled an interrupt

Table 587. AVS_INTERRUPT_MASK Register

Bits	Field	Access	Reset	Description
31:9	Reserved	-	-	Reserved
8	DISABLE_READBACK_OVERFLOW_INT	RW	0x1	Disable readback_overflow interrupt
7	DISABLE_READBACK_UNDERFLOW_INT	RW	0x1	Disable readback_underflow interrupt
6	DISABLE_CMD_FIFO_OVERFLOW_INT	RW	0x1	Disable cmd_fifo_overflow interrupt
5	DISABLE_MAX_RETRIES_ATTEMPTED_INT	RW	0x1	Disable max_retries_attempted interrupt
4	DISABLE_SLAVE_UNRESPONSIVE_INT	RW	0x1	Disable slave_unresponsive interrupt
3	DISABLE_READBACK_HAS_DATA_INT	RW	0x1	Disable readback_has_data interrupt
2	DISABLE_READBACK_FIFO_FULL_INT	RW	0x1	Disable readback_fifo_full interrupt
1	DISABLE_CMD_FIFO_FULL_INT	RW	0x1	Disable cmd_fifo_full interrupt
0	DISABLE_AVS_SLAVE_ISSUED_INTERRUPT	RW	0x1	Disable notification of AVS slave interrupt

Table 588. AVS_INTERRUPT_CLEAR Register

Bits	Field	Access	Reset	Description
31:9	Reserved	-	-	Reserved
8	CLEAR_READBACK_OVERFLOW_INT	W	0x0	Clear readback_overflow interrupt
7	CLEAR_READBACK_UNDERFLOW_INT	W	0x0	Clear readback_underflow interrupt
6	CLEAR_CMD_FIFO_OVERFLOW_INT	W	0x0	Clear cmd_fifo_overflow interrupt
5	CLEAR_MAX_RETRIES_ATTEMPTED_INT	W	0x0	Clear max_retries_attempted interrupt
4	CLEAR_SLAVE_UNRESPONSIVE_INT	W	0x0	Clear slave_unresponsive interrupt
3	CLEAR_READBACK_HAS_DATA_INT	W	0x0	Clear readback_has_data interrupt
2	CLEAR_READBACK_FIFO_FULL_INT	W	0x0	Clear readback_fifo_full interrupt
1	CLEAR_CMD_FIFO_FULL_INT	W	0x0	Clear cmd_fifo_full interrupt
0	CLEAR_AVS_SLAVE_ISSUED_INTERRUPT	W	0x0	Clear avs_slave_issued_interrupt

Table 589. AVS_CFG_0 Register

Bits	Field	Access	Reset	Description
31:24	Reserved	-	-	Reserved
23:16	MAX_RETRIES	RW	0x5	Maximum number of times to attempt a retry if an AVS command returns a fail status (either due to targetted resource being busy or CRC failure)
15:0	RESYNC_INTERVAL	RW	0x1000	Max interval to issue slave resync operation on AVS bus, expressed in number of APB clock cycles. NOTE: there is a 3 cycle delay for the resync flag to be seen by the avs controller due to synchronization.

Table 590. AVS_CFG_1 Register

Bits	Field	Access	Reset	Description
31:24	CLK_DIVIDER_DUTY_CYCLE_NUMERATOR	RW	0x0	Determines the desired duty cycle of the divided clock (when avs_clock_select is set to select the divided APB clock or refclk clock). This value will be divided by 256 to determine the high-pulse percentage of the divided clock period. Depending on the clk_divider_value, it may not be possible to precisely achieve the desired duty cycle - in that case, the closest possible fit is used. If this is set to 0, a value of 1 will be assumed. This is set to 0 by default to trick the clock divider into maintaining the HW-default settings (numerator=0x80) by matching the FF default in the resynced version of the signal.
23:16	CLK_DIVIDER_VALUE	RW	0x0	Determines the divisor value used to generate the AVS clock (when avs_clock_select is set to select the divided APB clock or divided refclk clock). The duty cycle of the divided clock depends both on the clk_divider_value together with the clk_divider_duty_cycle_numerator value. A 50% duty cycle is only possible on even clk_divider_values with clk_divider_duty_cycle_numerator set to 128. This field requires a minimum value of 2 - for lower values, 2 will be assumed. This is set to 0 by default to trick the clock divider into maintaining the HW-default settings (divider=0x4) by matching the FF default in the resynced version of the signal.

Bits	Field	Access	Reset	Description
15:11	Reserved	-	-	Reserved
10	TURN_OFF_ALL_PREMUX_CLOCKS	RW	0x0	Gate off all clocks entering the AVS clock mux - do this before changing the mux selection or changing the clock divider settings, then turn back on afterwards
9	FORCE_SLAVE_RESYNC_OPERATION	RW	0x0	Force master to issue a slave resync (34 clock cycles of holding the mdata signal high) at the next possible opportunity. NOTE: there is a 3 cycle delay for the resync flag to be seen by the avs controller due to synchronization.
8	STOP_AVS_CLOCK_ON_IDLE	RW	0x0	When the AVS bus is idle, gate the avs_clock from running. Restart it when there are new commands to be run. The master will always first issue 34 slave-resync cycles when the clock first restarts.
7:2	Reserved	-	-	Reserved
1:0	AVS_CLOCK_SELECT	RW	0x3	Selects which clock to use for AVS Clock: [list] [] 00: Use the APB clock directly as the AVS clock [] 01: Use a divided version of APB clock as the AVS Clock. The clock divider settings are determined by AVS_CFG_1:clk_divider_value and AVS_CFG_1:clk_divider_duty_cycle_numerator [] 10: Use the refclk clock directly as the AVS clock [] 11: Use a divided version of refclk clock as the AVS Clock. The clock divider settings are determined by AVS_CFG_1:clk_divider_value and AVS_CFG_1:clk_divider_duty_cycle_numerator [/list]

Table 591. AVS_CONFIG Register

Bits	Field	Access	Reset	Description
31:1	Reserved	-	-	Reserved
0	AVS_GPIO_ENABLE	RW	0x1	Enable AVSBus connection to GPIOs. The effective sets OE of the GPIOs dedicated to avs_clock and avs_cdata to 1, sets IE of GPIO dedicated to avs_tdata to 1. By default, this is enabled as the AVSBus state machine immediately starts after reset deassertion.

5.10.8. GPIO Peripheral

Overview

The GPIO (General Purpose Input/Output) peripheral provides programmable digital I/O capabilities for the OCH system. It serves as a flexible interface for configuration straps, general-purpose signaling, and bit-banging protocols. The peripheral features dual-mode operation supporting both register-driven control and LSIO (Low Speed Input/Output) interface control with comprehensive access filtering and interrupt generation capabilities.

Description and Features

- **Dual Control Modes:** Register-driven control or LSIO interface selection with programmable direction control
- **Flexible I/O Configuration:** Bidirectional data path with independent RX/TX enable controls
- **Security & Access Control:** Advanced register access filtering with AXI protection level requirements
- **Configuration Strap Support:** Hardware strap capture during system reset for boot-time configuration
- **Comprehensive Interrupt System:** Configurable interrupt generation with multiple trigger modes
- **Programmable Electrical Characteristics:** Software-configurable drive strength and pull resistors

- **Standard Bus Interfaces:** Support for both APB4 and AXI4-Lite register interfaces

Architecture

At the heart of the OCH system's digital I/O lies the GPIO peripheral, a controller designed for both versatility and robust security. Its dual-mode operation allows it to seamlessly switch between register-driven control and direct hardware guidance via the LSIO interface, all while maintaining strict access governance and flexible interrupt handling throughout.

Functional Blocks

The top-level GPIO controller (`gpio.sv`) sits at the center of the architecture, orchestrating the actions of all internal components. It elegantly arbitrates incoming bus transactions, mediates between register and LSIO modes, fuses interrupt sources into a unified signal, and ensures strap values are captured properly when the system starts. Address spaces are kept distinct and the transition between control modes is smooth and automatic.

Security is enforced by the Access Control Filter (`gpio_filter.sv`). True to its name, it inspects every AXI read and write transaction, confirming the proper permission levels before allowing access. The protection requirements are software-programmable, allowing enforcement policies to adapt to changing system needs on the fly.

The filter implements full 3-bit AXI protection matching, comparing the transaction's protection bits (`aw.prot[2:0]` for writes, `ar.prot[2:0]` for reads) against configurable requirement values. The protection bits encode privilege level (bit 0), security domain (bit 1), and transaction type (bit 2). When protection filtering is enabled, only transactions with matching protection attributes are allowed through; unauthorized attempts are blocked and receive an immediate error response, all while keeping performance unaffected for legitimate accesses.

Protection filtering is configured via the `ACCESS_FILTER` register fields: `awprot_requirement` and `arprot_requirement` specify the required protection values, while `write_filter_enable` and `read_filter_enable` control whether filtering is active. See the Programming section for configuration details.

Between the hardware and software is the GPIO Shim (`gpio_shim.sv`). This block translates software-side register programming into model-specific control signals, maintaining a consistent state across the register interface and the underlying physical I/O. The shim also abstracts model-specific details so different pad implementations can coexist under a unified interface.

Finally, the GPIO Model (`gpio_model/`) is where electrical attributes and physical pad behavior live. It offers programmable drive strengths, pull-up and pull-down configuration, signal conditioning to guard against glitches, and fully independent TX/RX enables, making true bidirectional I/O possible.

Data Flow Architecture

In typical register-driven mode, the data path forms a tight, predictable loop: software writes an output value into a `DATA_CTRL` register, the mode selector switches to software control, and the register's value flows to the model, driving the I/O pad. Input values are always readable via a separate field, ensuring software insight into pad state.

When LSIO mode is engaged, control shifts: the LSIO hardware asserts its selection input, and from this moment, it directly governs the pad's output and enable signals. The regular register pathway is bypassed, and the status of LSIO is reflected back to software for monitoring purposes. Thanks to the clear priority scheme, LSIO always wins control if contention arises.

Control Path and Mode Selection

Configuration sources follow a strict ranking: LSI0 commands always take the highest precedence, overriding any register settings when active. If LSI0 is not in control, the system defers to register-directed configuration. If neither is in play, safe hardware defaults ensure the pad remains in a benign state.

The logic for this selection is straightforward but decisive:

```
if (lsio_interface_select_i && !lsio_disable) {  
    // LSI0 mode: hardware takes over  
    pad_control_source = LSI0;  
} else if (interface_enable) {  
    // Register mode: software control  
    pad_control_source = REGISTERS;  
} else {  
    // Defaults: pad is idle and safe  
    pad_control_source = DEFAULTS;  
}
```

Interrupts: Flexible and Responsive

The GPIO's interrupt subsystem is engineered for flexibility. It supports both level and edge detection, allowing software to choose between active-high/low levels or rising/falling edges. Interrupts can be enabled or masked at runtime, and the state is always visible to software.

Here's a look at the internal decision logic:

```
interrupt_trigger = case (interrupt_type)  
    2'b00: pad2core;           // Active-high level  
    2'b01: !pad2core;         // Active-low level  
    2'b10: pad2core_rising_edge; // Rising edge  
    2'b11: pad2core_falling_edge; // Falling edge  
endcase  
  
interrupt_o = interrupt_enable && interrupt_trigger;
```

Clocking and Reset: Safe and Predictable

All GPIO logic is synchronized to a single system clock (clk_i). There are no internal clock domains or generated clocks to complicate timing—everything operates in lockstep, rising edge after rising edge.

Reset is provided as a synchronous active-low input (rst_ni). When asserted, every flip-flop in the design is immediately reset to a known baseline. During the reset period, the pad value is continuously monitored; once reset is released (on a clock edge), the final pad state is latched as a configuration strap. Only after this carefully ordered process does the GPIO begin normal operation, fully configured and secure.

Interface

The GPIO peripheral provides bus interfaces for register access, LSIO integration for hardware protocols, and direct GPIO pad connections.

Configuration Parameters

Parameter	Default	Description
NUM_GPIO	65	Total number of GPIO pins (61 bonded + 4 unbonded)
BUS_PROTOCOL	AXI4-Lite	Register interface protocol (AXI4-Lite or APB4)
ADDR_WIDTH	32	Address bus width
DATA_WIDTH	32	Data bus width

Signal Interface Definitions

Global Interface

Table 592. Global Interface Signals

Signal Name	Width	Direction	Active	Description
clk_i	1	Input	Rising	System clock input. All synchronous operations use rising edge.
rst_ni	1	Input	Low	System reset input. Asynchronous assertion, synchronous deassertion.
test_en_i	1	Input	High	Test enable input for DFT (Design for Test) support.

LSIO Interface

The LSIO (Low Speed Input/Output) interface enables hardware-controlled GPIO operation for protocol processing.

Table 593. LSIO Interface Signals

Signal Name	Width	Direction	Description
lsio_interface_select_i	1	Input	LSIO interface selection control (active high)
lsio_core2pad_en_ni	1	Input	LSIO core-to-pad enable (active low)
lsio_core2pad_data_i	1	Input	LSIO core-to-pad data value
lsio_pad2core_en_ni	1	Input	LSIO pad-to-core enable (active low)
lsio_pad2core_data_o	1	Output	LSIO pad-to-core data value

LSIO Protocol Specifications

- **Selection Priority:** LSIO mode overrides register-driven control when active
- **Enable Logic:** Active-low enables for both transmit and receive paths
- **Data Latency:** Zero-cycle latency for data path when selected
- **Mode Switching:** Immediate effect, no settling time required
- **Status Reflection:** LSIO activity status available in DATA_CTRL.lsio_enable register

GPIO Pad Interface

The GPIO pad interface provides direct connection to external pads with bidirectional capabilities.

Table 594. GPIO Pad Interface Signals

Signal Name	Width	Direction	Description
core2pad_o	1	Output	Data signal from core to external pad
core2pad_en_o	1	Output	Output enable for core-to-pad data path
pad2core_i	1	Input	Data signal from external pad to core
pad2core_en_o	1	Output	Input enable for pad-to-core data path

Interrupt Interface

The interrupt interface provides event notification capabilities with configurable trigger modes.

Table 595. Interrupt Interface Signals

Signal Name	Width	Direction	Description
interrupt_o	1	Output	Interrupt request output (active high)
captured_strap_o	1	Output	Configuration strap capture completion indicator

Interrupt Specifications

- **Polarity:** Active high (asserted when high)
- **Duration:** Level-sensitive, remains asserted based on configuration
- **Trigger Modes:** Level (active-high/low) and edge (rising/falling) detection
- **Clearing:** Automatic clearing based on trigger mode and pad state
- **Latency:** Interrupt assertion within 2-3 clock cycles of triggering event

Memory Map

The GPIO peripheral provides two separate register banks for comprehensive I/O control: the Interface Register Bank (gpio_intf_reg) for high-level GPIO functionality and the Control Register Bank (gpio_ctrl_reg) for physical layer characteristics. Both banks are accessed through 32-bit AXI4-Lite interfaces with independent address spaces.

Address Space Organization

The GPIO peripheral uses a dual-bank register architecture:

- **Interface Register Bank:** High-level GPIO control, data path, and security filtering
- **Control Register Bank:** Physical layer configuration, drive strength, and termination

Each bank operates independently with separate AXI4-Lite interfaces to enable concurrent access and modular functionality.

Reset Behavior

When the system reset (rst_ni) is asserted, the GPIO peripheral initializes each register bank to well-defined, safe starting values. In the Interface Register Bank, the ACCESS_FILTER register is set to 0x00010100, enforcing default security protection levels, while the DATA_CTRL register clears all control bits to 0x00000000, ensuring no data path is active by default. Simultaneously, the Control Register Bank's CONTROL register is initialized to 0x00100002, establishing a 6mA drive strength and enabling strap validity as the hardware default for the physical I/O.

During reset deassertion, the peripheral captures the pad state for configuration strap functionality, updating both the strap_valid and strap_value fields based on the observed pad logic level. At this stage, all GPIO operation modes remain inactive, and explicit software configuration is required to enable any functional path. Throughout reset, the hardware maintains a safe low-current output condition on the pads, minimizing the risk of unintended external drive or contention.

Detailed Register Map

Table 596. Register Map

Address	Name	Access	Description
0x0	DATA_CTRL	RW	-
0x4	DATA_CTRL_ENABLE	RW	-
0x8	ACCESS_FILTER	RW	GPIO Access Filter Register. WARNING: please read back this register to ensure the filter was written correctly. If you do not do so the filter may not update before your next transaction, causing unexpected behavior.

Table 597. DATA_CTRL Register

Bits	Field	Access	Reset	Description
31	pad2core	R	0x0	PAD2SOC Value
30:26	Reserved	-	-	Reserved
25	lsio_enable	R	0x0	When set, this bit indicates that the pad being driving by the LSIO
24:22	Reserved	-	-	Reserved
21:20	interrupt_type	RW	0x0	Interrupt type - 0: active-high level, 1: active-low level, 2: rising edge, 3: falling edge
19	lsio_disable	RW	0x0	When set, LSIO acceses will be blocked from the GPIO interface
18	interrupt_enable	RW	0x0	Interrupt Enable.
17	lsio_select	RW	0x0	Force LSIO interface to be used. interface_enable has higher priority over this
16	interface_enable	RW	0x0	Register Interface Enable. Setting this chooses register values to drive the PAD. This includes core2pad, enable_rx_tx, and pad2core
15:6	Reserved	-	-	Reserved
5:4	enable_rx_tx	RW	0x0	Direction control. 2'b00: Neither RX nor TX enabled. 2'b01: TX enabled. 2'b10: RX enabled. 2'b11: Both enabled
3:1	Reserved	-	-	Reserved
0	core2pad	RW	0x0	Register-driven data to send to the pad.

Table 598. DATA_CTRL_ENABLE Register

Bits	Field	Access	Reset	Description
31:6	Reserved	-	-	Reserved
5	use_reg_rx	RW	0x0	Per-field register override. When set, the RX direction (input enable) is taken from enable_rx_tx[1]. OR'd with interface_enable.
4	use_reg_tx	RW	0x0	Per-field register override. When set, the TX direction (output enable) is taken from enable_rx_tx[0]. OR'd with interface_enable.
3:1	Reserved	-	-	Reserved

Bits	Field	Access	Reset	Description
0	<code>use_reg_core2pad</code>	RW	0x0	Per-field register override. When set, core2pad is register-driven. OR'd with interface_enable.

Table 599. ACCESS_FILTER Register

Bits	Field	Access	Reset	Description
31:19	Reserved	-	-	Reserved
18:16	<code>arprot_requirement</code>	RW	0x1	When read_filter_enable is set, only allow read accesses with this prot value WARNING: please read back this register to ensure the filter was written correctly. If you do not do so the filter may not update before your next transaction, causing unexpected behavior.
15:11	Reserved	-	-	Reserved
10:8	<code>awprot_requirement</code>	RW	0x1	When write_filter_enable is set, only allow write accesses with this prot value WARNING: please read back this register to ensure the filter was written correctly. If you do not do so the filter may not update before your next transaction, causing unexpected behavior.
7:2	Reserved	-	-	Reserved
1	<code>read_filter_enable</code>	RW	0x0	Filter Value - the value that must be written to the filter register to allow access to the GPIO register interface. WARNING: please read back this register to ensure the filter was written correctly. If you do not do so the filter may not update before your next transaction, causing unexpected behavior.
0	<code>write_filter_enable</code>	RW	0x0	Filter Enable - when set, only SEP may access the GPIO register interface. This effectively makes the GPIO a SEP-only interface. WARNING: please read back this register to ensure the filter was written correctly. If you do not do so the filter may not update before your next transaction, causing unexpected behavior.

Programming

Basic GPIO Configuration

Setting up GPIO for simple output operation:

1. Configure physical characteristics in CONTROL register:
 - Set drive_strength to desired value (default 010 = 6mA)
 - Configure pull resistors if needed
 - Enable config_enable = 1
2. Configure interface in DATA_CTRL register:
 - Set interface_enable = 1 for register control
 - Set enable_rx_tx = 01 for output operation
 - Set core2pad to desired output value
3. Monitor operation:
 - Check lsio_enable status for interface conflicts
 - Read pad2core for input feedback

Configuration Strap Capture

Reading configuration straps captured during reset:

1. Wait for system initialization to complete
2. Read `DATA_CTRL.strap_valid` to verify strap capture
3. If valid, read `DATA_CTRL.strap_value` for configuration data
4. Use strap value for system configuration decisions

Note: Strap capture only occurs if GPIO is configured as input by default (`INPUT_BY_DEFAULT` parameter = 1).

Interrupt Configuration

Setting up GPIO interrupt generation:

1. Configure interrupt type in `DATA_CTRL.interrupt_type`:
 - 00: Active-high level interrupt
 - 01: Active-low level interrupt
 - 10: Rising-edge interrupt
 - 11: Falling-edge interrupt
2. Enable interrupts:
 - Set `DATA_CTRL.interrupt_enable = 1`
3. Configure GPIO for input:
 - Set `DATA_CTRL.enable_rx_tx = 10` (RX enabled)
 - Set `DATA_CTRL.interface_enable = 1`
4. Handle interrupts in software interrupt service routine

LSIO Interface Operation

Configuring GPIO for LSIO hardware control:

1. Disable register interface:
 - Set `DATA_CTRL.interface_enable = 0`
2. Enable LSIO interface:
 - Set `DATA_CTRL.lsio_select = 1` (if forcing LSIO)
 - Ensure `DATA_CTRL.lsio_disable = 0`
3. Monitor LSIO operation:
 - Check `DATA_CTRL.lsio_enable` for active status
 - Hardware LSIO interface controls pad directly

4. LSI0 interface has priority over register control when active

Security Configuration

The GPIO controller implements protection-based access filtering to enforce security policies. The filter examines AXI protection bits (**aw.prot** for writes, **ar.prot** for reads) and compares them against configurable requirement values.

Protection Bit Encoding

AXI protection bits are 3-bit fields encoding transaction attributes:

- **Bit [0]:** Privilege level (0=User, 1=Privileged)
- **Bit [1]:** Security domain (0=Secure, 1=Non-Secure)
- **Bit [2]:** Transaction type (0=Data, 1=Instruction)

Common protection values: * **0x0** (000): Data, Secure, User * **0x1** (001): Data, Secure, Privileged (default for GPIO) * **0x2** (010): Data, Non-Secure, User * **0x3** (011): Data, Non-Secure, Privileged

Filter Configuration

Setting up access control filtering:

1. Determine required protection levels:
 - Normal data: PROT = 0x0 (000)
 - Privileged data: PROT = 0x1 (001) - recommended default
 - Secure privileged: PROT = 0x1 (001)
 - Non-secure privileged: PROT = 0x3 (011)
2. Configure protection requirements:
 - Set ACCESS_FILTER.awprot_requirement[2:0] for writes
 - Set ACCESS_FILTER.arprot_requirement[2:0] for reads
 - Default value is 0x1 (Data, Secure, Privileged)
3. Enable filtering:
 - Set ACCESS_FILTER.write_filter_enable = 1
 - Set ACCESS_FILTER.read_filter_enable = 1
4. Test access with different protection levels to verify operation:
 - Transactions with matching protection: allowed
 - Transactions with non-matching protection: blocked (DECERR)

Filter Behavior

When protection filtering is enabled: * Transactions with protection bits matching the requirement are allowed * Transactions with non-matching protection bits are blocked and receive a decode error (DECERR) response * When filtering is disabled, all transactions are allowed regardless of protection bits

The filter operates on every transaction, providing real-time security enforcement without impacting performance for legitimate accesses.

Bidirectional Operation

Configuring GPIO for bidirectional communication:

1. Set initial direction:
 - For output: `enable_rx_tx = 01`
 - For input: `enable_rx_tx = 10`
2. Dynamic direction switching:
 - Write new `enable_rx_tx` value to change direction
 - Direction change takes effect immediately
3. Monitor operation:
 - Always read `pad2core` for current pad state
 - Write `core2pad` when in output mode
4. Handle contention:
 - Ensure external drivers are tristated before switching to output
 - Use pull resistors to define idle state

5.10.9. Telemetry Receiver

Overview

The Telemetry Receiver is a high-performance hardware component designed to decode, buffer, and process telemetry data from telemetry transmitters in multi-chiplet systems. The component implements a specialized receiver for Tenstorrent-specific telemetry message formats while maintaining modularity for adaptation to other telemetry protocols and message structures.

Description and Features

- **32-bit APB4 or AXI4-Lite Register Interface:** Standard bus interfaces for system integration and configuration access
- **8-bit ATB Telemetry Interface:** High-speed telemetry data reception with ATB protocol compliance
- **Configurable Circular Buffer:** Programmable buffer depth (default 8 entries) with threshold-based monitoring
- **Multi-Counter Message Support:** Decodes messages with probe ID and up to 4 counter values per message
- **Flexible Message Format:** Modular design allows easy adaptation to different telemetry message formats
- **Comprehensive Interrupt System:** Event-driven notifications for buffer thresholds and error conditions

Architecture

The Telemetry Receiver leverages a dual-buffer design for robust and continuous telemetry data capture from an ARM Trace Bus (ATB) source. Its architecture brings together configurable register interfaces, full ATB compliance, intelligent

buffering, and efficient message processing—balancing narrative clarity with compact organization.

Functional Blocks

Register Interface Block

The register interface uses an AXI4-Lite bus for access to control, status, and data registers. Control registers handle buffer thresholds and interrupt enables for system optimization. Status registers provide real-time buffer occupancy and error flags. Data registers make probe ID and counter details accessible, while interrupt registers support selective and responsive event handling for seamless system integration.

ATB Interface Block

The ATB interface guarantees protocol-compliant 8-bit telemetry data reception using handshaked ready/valid flows, supporting industrial trace infrastructures. It manages per-source ID tracking for multi-source collection and delivers reliable operation under variable load, with flush signaling to mark stream boundaries and maintain session integrity.

Assembly Buffer

This buffer assembles streams of 8-bit ATB data into 64-bit packets. It ensures data ordering, detects packet boundaries, and supports little-endian (LSB-first) byte alignment. Temporary storage enables smooth handling of irregular traffic and guarantees data integrity throughout packet reconstruction.

Message Decoder

Assembled packets are parsed for probe IDs and up to four counters, each with validity checked. It validates each 9-bit packet block, provides error reporting, and detects last-packet boundaries—crucial for reliable source tracking and efficient telemetry message integrity.

Message Buffer

A circular message buffer persistently holds decoded messages. It offers configurable depth (2–N entries), FIFO access, threshold monitoring with interrupt generation, and full status reporting (occupancy, empty/full).

Data Flow

Reception Phase The ATB interface receives telemetry in 8-bit beats, accumulated into 64-bit packets in the assembly buffer, then validated for structure.

Decoding Phase Packets are parsed for headers (probe ID), broken into 9-bit blocks, counters are extracted and checked, and validity flags assigned.

Buffering Phase Decoded messages are stored FIFO-style; pointers, counters, and flags update atomically. Threshold or error interrupts are generated as needed.

Readout Phase Telemetry is accessed by software via register reads. Buffer pointers advance after each access, and status or error conditions are monitored in real-time.

Control Path

Configuration Control Software configures buffer thresholds, enables interrupts, and clears errors, permitting runtime adaptation and robust error handling.

Status Reporting Occupancy, availability, and error flags are accessible, providing up-to-date telemetry collection health.

Interrupt Handling Interrupts are generated for buffer threshold crossings and validation errors; all can be cleared by software.

Clocking and Reset

Clock Domain The design operates fully synchronous on `clk_i`, avoiding clock crossing complexity and easing verification.

Reset Architecture An asynchronous reset (`rst_ni`), synchronously deasserted, guarantees all storage and state machines revert to known conditions on startup. This approach secures clean initialization and reliable operation from power-on forward.

Interface

The Telemetry Receiver component provides AXI4-Lite register access and ARM Trace Bus (ATB) telemetry data reception interfaces.

Configuration Parameters

Parameter	Default	Description
NUM_BUFFERS	2	Number of telemetry buffers (dual-buffer architecture)
BUFFER_DEPTH	16	Depth of each telemetry buffer (entries)
ATB_DATA_WIDTH	8	ATB interface data width (bits)
PACKET_WIDTH	64	Telemetry packet width (bits)
ADDR_WIDTH	32	Register address width (bits)
DATA_WIDTH	32	Data bus width (bits)

Signal Interfaces

Global Interface

The global interface provides fundamental clock and reset signals required for basic operation.

Table 600. Global Interface Signals

Signal Name	Width	Direction	Active	Description
<code>clk_i</code>	1	Input	Rising	System clock input. All synchronous operations are performed on the rising edge.
<code>rst_ni</code>	1	Input	Low	System reset input. Asynchronous assertion, synchronous deassertion.

Clock and Reset Specifications

- **Clock Frequency:** The component can operate at frequencies up to 200 MHz
- **Clock Jitter:** Maximum 100ps peak-to-peak jitter tolerance
- **Reset Duration:** Minimum 10 clock cycles for complete reset
- **Reset Recovery:** Component ready for operation 2 clock cycles after reset deassertion

ATB Telemetry Interface

The ATB (ARM Trace Bus) interface receives telemetry data using standard ATB protocol.

Table 601. ATB Interface Signals

Signal Name	Width	Direction	Description
atdata_i	8	Input	Telemetry data input (8-bit data beats)
atid_i	7	Input	ATB ID field (trace source identification)
atvalid_i	1	Input	ATB valid signal (data valid indication)
atready_o	1	Output	ATB ready signal (receiver ready indication)
afvalid_o	1	Output	ATB flush valid (flush request indication)
afready_i	1	Input	ATB flush ready (flush acknowledgment)

ATB Protocol Specifications

- **Data Width:** 8 bits per beat
- **ID Width:** 7 bits (128 unique trace sources)
- **Flow Control:** Valid/ready handshaking for reliable data transfer
- **Flush Support:** Bidirectional flush signaling for stream management
- **Back-pressure:** Component can assert **atready_o** low to apply back-pressure
- **Latency:** Zero-cycle latency when ready, immediate acceptance when **atready_o** is high

Interrupt Interface

The interrupt interface provides a single interrupt output for event notification.

Table 602. Interrupt Interface Signals

Signal Name	Width	Direction	Description
irq_o	1	Output	Interrupt request output (active high)

Interrupt Specifications

- **Polarity:** Active high (asserted when high)
- **Duration:** Level-sensitive, remains asserted until cleared
- **Sources:** Buffer threshold and error condition interrupts
- **Clearing:** Cleared by writing appropriate bits in interrupt status registers
- **Latency:** Interrupt assertion within 2 clock cycles of triggering condition

Memory Map

The Telemetry Receiver provides a comprehensive register interface for configuration, status monitoring, and telemetry data access. All registers are accessed through a 32-bit AXI4-Lite interface with 4-byte address alignment.

Address Space Overview

The component uses a 128-byte address space (7-bit address width) with the following organization:

- **Control and Status:** 0x00 - 0x1F (Configuration, status, probe ID, and counter validity registers)
- **Counter Array:** 0x80 - 0xFF (32 × 32-bit counter value registers)

Register Map

Table 603. Complete Register Map

Address	Register Name	Access	Description
0x00	CTRL	RW	Control Register
0x04	STATUS	R	Status Register
0x08	INTR_STATUS	RW	Interrupt Status Register
0x0C	INTR_ENABLE	RW	Interrupt Enable Register
0x10	INTR_TEST	RW	Interrupt Test Register
0x14	TELEMETRY_PROBE_ID	R	Telemetry Probe ID Register
0x18	TELEMETRY_COUNTER_VLDS	R	Telemetry Counter Valid Bits Register
0x80	TELEMETRY_COUNTER[0]	R	Telemetry Counter Register 0
0x84	TELEMETRY_COUNTER[1]	R	Telemetry Counter Register 1
0x88	TELEMETRY_COUNTER[2]	R	Telemetry Counter Register 2
0x8C	TELEMETRY_COUNTER[3]	R	Telemetry Counter Register 3
...	...	...	...
0xFC	TELEMETRY_COUNTER[31]	R	Telemetry Counter Register 31

Reset Behavior

Upon system reset (rst_ni assertion):

- All registers reset to their default values as specified
- Message buffer is cleared and marked empty (STATUS.BUFFER_EMPTY = 1)
- All interrupts are cleared and disabled
- Telemetry data registers reset to zero
- Component enters idle state ready for telemetry reception

The reset operation is asynchronous for assertion and synchronous for deassertion, ensuring clean startup behavior and proper synchronization with the system clock domain.

Detailed Register Map

NOTE	Register Arrays: This register map contains the following register arrays:
	• TELEMETRY_COUNTER[32]: 32 registers ◦ Base Address: 0x80 ◦ Address Increment: 0x4 per register

Table 604. Register Map

Address	Name	Access	Description
0x0	CTRL	RW	Control Register
0x4	STATUS	R	Status Register
0x8	INTR_STATUS	RW	Interrupt Status Register
0xC	INTR_ENABLE	RW	Interrupt Enable Register
0x10	INTR_TEST	RW	Interrupt Test Register
0x14	TELEMETRY_PROBE_ID	R	Telemetry Probe ID Register
0x18	TELEMETRY_COUNTER_VLDS	R	Telemetry Counter Valid Bits Register
0x80 - 0xFC	TELEMETRY_COUNTER[32]	R	Telemetry Counter Register

Table 605. CTRL Register

Bits	Field	Access	Reset	Description
31:24	Reserved	-	-	Reserved
23:12	BUFFER_THRESHOLD	RW	0x0	Telemetry Message Buffer Threshold. The Telemetry Buffer Threshold Interrupt is asserted when the number of entries in the buffer is greater than this value.
11:9	Reserved	-	-	Reserved
8	TELEMETRY_TX_FLUSH	RW	0x0	Telemetry Transmitter Flush. Writing 1 flushes all FIFOs in the telemetry transmitter. The flushing is done via the ATB AF interface. This bit clears itself after the flush completes.
7:5	Reserved	-	-	Reserved
4	TELEMETRY_RX_FLUSH	W	0x0	Telemetry Receiver Flush. Writing 1 flushes the Telemetry Receiver Buffer and all other storage elements in the Telemetry Receiver.
3:1	Reserved	-	-	Reserved
0	BUFFER_POP	W	0x0	Telemetry Message Buffer Pop. Effective only if STATUS.BUFFER_EMPTY = 0. Writing 1 pops the buffer and thus updates the data in the TELEMETRY_PROBE_ID, TELEMETRY_COUNTER_VLDS, and TELEMETRY_COUNTER[32] registers.

Table 606. STATUS Register

Bits	Field	Access	Reset	Description
31:5	Reserved	-	-	Reserved
4	BUFFER_FULL	R	0x0	Telemetry Message Buffer Full.
3:1	Reserved	-	-	Reserved
0	BUFFER_EMPTY	R	0x1	Telemetry Message Buffer Empty.

Table 607. INTR_STATUS Register

Bits	Field	Access	Reset	Description
31:5	Reserved	-	-	Reserved
4	BUFFER_THRESHOLD	R	0x0	Telemetry Message Buffer Threshold Interrupt. Asserted when the number of telemetry messages in the Telemetry Message Buffer is greater than CTRL.BUFFER_THRESHOLD.
3:1	Reserved	-	-	Reserved

Bits	Field	Access	Reset	Description
0	MISSING_LAST	RW	0x0	Last Packet Missing Interrupt. Asserted when the Telemetry Receiver has not received a Last Packet flag when the maximum number of packets a message can consist of has been received.

Table 608. INTR_ENABLE Register

Bits	Field	Access	Reset	Description
31:5	Reserved	-	-	Reserved
4	BUFFER_THRESHOLD	RW	0x0	Telemetry Message Buffer Threshold Interrupt Enable.
3:1	Reserved	-	-	Reserved
0	MISSING_LAST	RW	0x0	Last Packet Missing Interrupt Enable.

Table 609. INTR_TEST Register

Bits	Field	Access	Reset	Description
31:5	Reserved	-	-	Reserved
4	BUFFER_THRESHOLD	RW	0x0	Telemetry Message Buffer Threshold Interrupt Test. Writing 1 forces the interrupt and writing 0 releases it.
3:1	Reserved	-	-	Reserved
0	MISSING_LAST	W	0x0	Last Packet Missing Interrupt Test. Writing 1 forces the interrupt.

Table 610. TELEMETRY_PROBE_ID Register

Bits	Field	Access	Reset	Description
31:5	Reserved	-	-	Reserved
4:0	PROBE_ID	R	0x0	Probe ID. Contains the Probe ID field of the telemetry message at the bottom of the Telemetry Message Buffer.

Table 611. TELEMETRY_COUNTER_VLDS Register

Bits	Field	Access	Reset	Description
31:0	COUNTER_VLDS	R	0x0	Counter Valid Bits. Indicates which of the counter values in the telemetry message at the bottom of the Telemetry Message Buffer are valid. Bit i is for the i-th counter.

Table 612. TELEMETRY_COUNTER[32] Register

Bits	Field	Access	Reset	Description
31:0	COUNTER	R	0x0	Counter Value. Contains one of the 32 counter values in the telemetry message at the bottom of the Telemetry Message Buffer.

5.11. eFuse Shadow Register Scan Protection

DFT scan test and debug scandump share the scan chain infrastructure. Any flop placed on a scan chain can be shifted out, and any flop on a dumpable chain can be observed by a debug user. The SMC eFuse shadow registers hold the LOCKS field, which is the sole enforcement point for read and write access control over the entire SMC fuse map. If those flops were scannable, an attacker could shift a cleared LOCKS value into the shadow array and defeat the access controls without touching the fuse macro itself. This section describes how the LOCKS shadow words are excluded from every scan chain.

The asset classes referenced here (Class 1 / Class 2 / Class 3) are defined authoritatively in the SEP Lifecycle Controller chapter, *Asset and Scan-Chain Protection*. This section records how they apply to the SMC eFuse controller.

5.11.1. SMC eFuse Asset Classification

Table 613. SMC eFuse shadow register classification

Class	SMC content	Scan policy
Class 1a (confidentiality)	<i>none</i>	The SMC fuse map holds no device secrets — it carries identifiers (CHIPLET_ID , PACKAGE_ID , I2C_I3C_ID), memory repair data (BIRA), and topology / clocking / SPI configuration. There are no keys, no UIDs used as key material, and no token registers. Consequently no confidentiality masking is required.
Class 1b (integrity)	LOCKS	Excluded from every scan chain. Never scannable, never dumpable, for any LC state or debug grant.
Class 2 (secret-bearing)	<i>none</i>	The SMC eFuse controller instantiates no cryptographic engine and no key vault, so it contributes no Class 2 scan roots and needs no dedicated secret-bearing chain.
Class 3 (general)	All remaining fuse-map words	Normal scan and scandump handling, subject to the SMC debug policy.

Because SMC has no Class 1a asset, **secure_tm** is not used for secret disconnection in the SMC eFuse controller: **secure_tm_i** is tied to **10b0** and **SECRET_SHADOW_RANGES** is left at **'0**. This differs from SEP, where **secure_tm** disconnects the UID / **CLASS_KEY** sources from their Class 2 consumers.

5.11.2. Implementation

efuse_shadow_regs keeps its shadow storage in two separately named arrays and reconstructs the public register view from both. The array holding Class 1 words is named with an **_n0_scan** suffix, which is the reserved naming handle the synthesis scan-insertion flow uses to identify the flops it must leave off all scan chains:

- **shadow_efuse_values_n0_scan** — Class 1 words, excluded from scan.
- **shadow_efuse_values** — Class 3 words, normal scan handling.

Which words land in the Class 1 array is set by the **CLASS1_SHADOW_RANGES** parameter. **smc_efuse_wrapper** drives it from **smc_efuse_pkg::Class1ShadowRanges**, which derives the range from the generated RDL metadata rather than hard-coding word indices, so the exclusion follows the register map if the map is regenerated:

```
localparam efuse_pkg::shadow_word_range_map_t Class1ShadowRanges = '{
    0: efuse_pkg::make_shadow_word_range(
        LOCKS_REG_OFFSET, $bits(smc_efuse_map_locks_reg_t), NumFuseWordWidth),
    default: '0
};
```

LOCKS sits at fuse-map offset **0x0** and is 64 bits wide, so this resolves to shadow words 0 and 1.

Table 614. SMC shadow word allocation

Quantity	Value
Fuse map size	3 KB (24576 bits, 768 x 32-bit words)
Class 1 words (shadow_efuse_values_n0_scan)	2 (words 0-1, LOCKS)
Class 3 words (shadow_efuse_values)	766

Three `ASSERT_INIT` checks in `efuse_shadow_regs` verify at elaboration that the configured ranges are well formed, non-overlapping, and inside the shadow array, so a bad range map is a build failure rather than a silent loss of protection.

5.11.3. Downstream Lock Path

Excluding the `LOCKS` shadow flops is only sufficient if no scannable flop downstream of them re-derives the lock decision. In the SMC eFuse controller this holds by construction: `efuse_shadow_reg_access_control`, which compares the incoming APB address against the field map and produces the `write_locked` / `read_locked` decisions, is purely combinational. There is no intermediate state element between the Class 1 shadow words and the APB write-enable and read-data paths that they gate.

The one sequential consumer of that logic is `locked_field_access_interrupt_o`, which reports that a locked-field access was attempted. It carries an access-violation event, not lock state, so observing it does not reveal or alter the `LOCKS` value.

5.11.4. Notes for DFT

- The `_n0_scan` suffix is a Class 1 marker and takes precedence over any enclosing scan hierarchy. Do not rename it; the exclusion is carried entirely by the signal name, as there is no separate constraint list in this repository.
- `efuse_shadow_regs` also declares `chiplet_state_change_completed_n0_scan` and `sop_state_change_completed_n0_scan`. These serve the SEP RMA token / LC state transition path. In the SMC instance `HAS_LC_STATE` is `1'b0` and both are driven to a constant `1'b1`, so they hold no SMC state; they are expected to be constant-propagated away.
- Adding a secret or a new integrity-critical field to the SMC fuse map requires extending `Class1ShadowRanges` (and, for a confidentiality asset, `SECRET_SHADOW_RANGES` plus a real `secure_tm_i` source). Adding the register to the RDL alone does not confer scan protection.

5.11.5. Verification

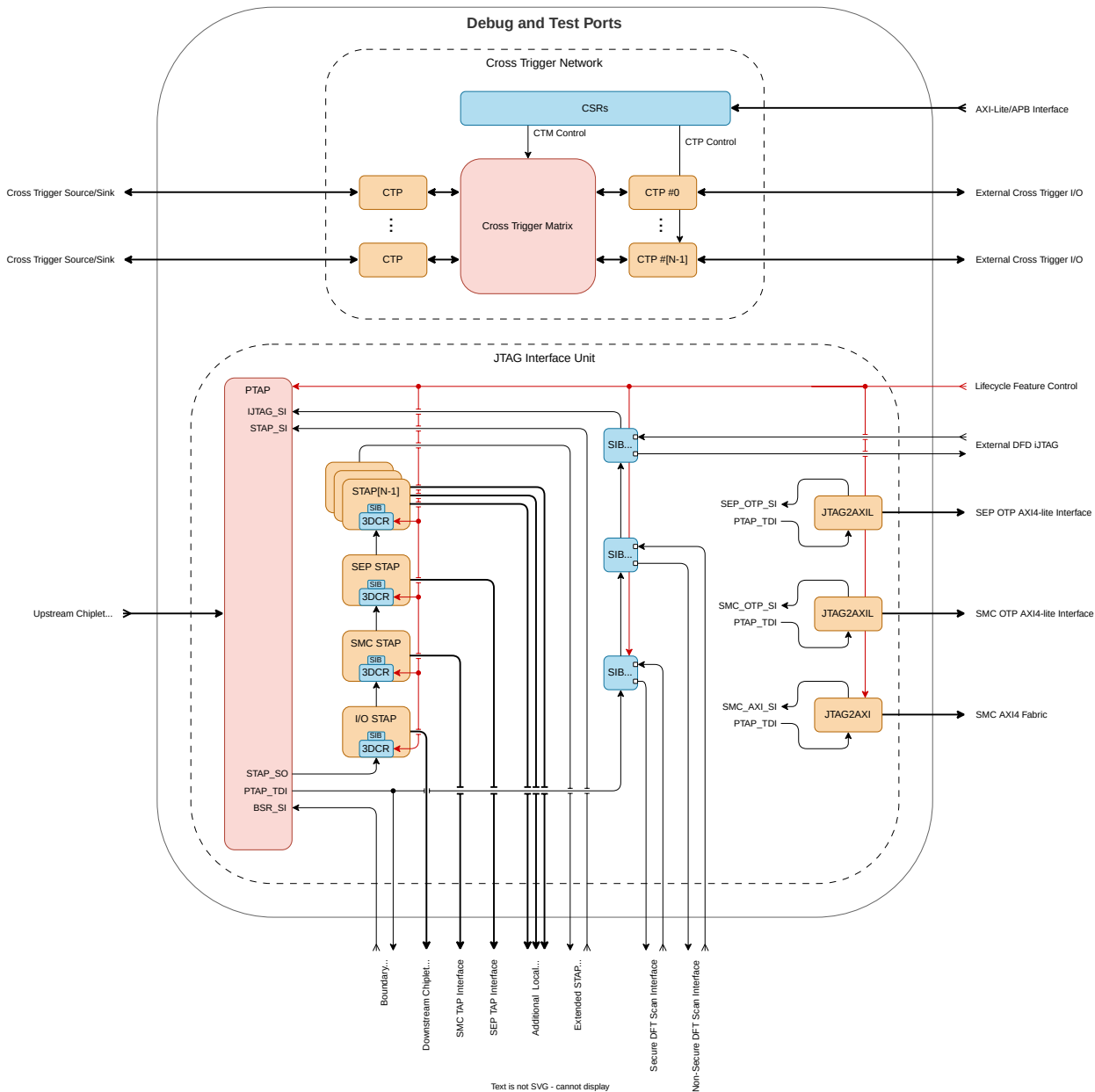
`smc_efuse_shadow_lock_matrix_test` exercises reads and writes across the `LOCKS` matrix and therefore covers the relocated words functionally, confirming that moving `LOCKS` into the Class 1 array does not change access-control behaviour.

Chapter 6. Debug and Test Ports (DTP)

6.1. Overview

The Debug and Test Ports (DTP) module provides the debug and test infrastructure for Tenstorrent chiplets. It implements IEEE 1149.1 (JTAG) and IEEE 1687 (iJTAG) compliant interfaces along with cross-trigger capabilities for multi-chiplet debugging.

The DTP has two major blocks, the JTAG Interface Unit and Cross Trigger Network, shown in the figure below.



6.1.1. Purpose

The DTP serves as the central hub for:

- **Debug Access:** JTAG-based access to on-chip registers and memory

- **Boundary Scan:** IEEE 1149.1 boundary scan for board-level testing
- **Cross Triggering:** Die-to-die and internal trigger coordination
- **Clock Control:** Debug clock-stop controls and CLA clock-stop request aggregation

6.1.2. Key Features

JTAG Interface

- IEEE 1149.1-2013 compliant TAP controller
- Configurable instruction register
- Support for standard instructions (BYPASS, IDCODE, EXTEST, etc.)
- Optional JTAG-to-AXI bridges for register access

Cross Trigger Network

- Configurable number of external Cross Trigger Ports (1-32)
- Configurable number of internal cross triggers (0-32)
- Cross Trigger Matrix for flexible routing
- Wire-OR and Point-to-Point signaling modes
- OR aggregation of incoming CLA clock-stop request sources, OR-combined with JTAG clock stop for clock control gating

ijTAG Support

- IEEE 1687 compliant scan chain management
- Separate chains for BSR, DFD, and DFT
- SIB-based hierarchical access

Secondary TAPs

- I/O STAP for chiplet-to-chiplet JTAG connectivity
- SEP debug STAP for secure enclave access
- Configurable number of extra STAPs

6.1.3. Specifications

Feature	Specification
JTAG Standard	IEEE 1149.1-2013
ijTAG Standard	IEEE 1687-2014
Cross Trigger Protocol	OCH Cross Trigger v1.0
CSR Interface	AXI4-Lite (32-bit)
Debug AXI Interface	AXI4 (configurable width)
Maximum CTPs	32
Maximum Internal CTs	32

6.2. JTAG Interface Unit

6.2.1. Overview

The JTAG Interface Unit serves as the top-level JTAG control module, managing the primary JTAG port for the chiplet and all supporting instrument logic. It contains the following design blocks and features:

- One PTAP
 - Chiplet IDCODE register
 - Boundary scan control
 - BIST control
 - Reset control
- A configurable number of STAPs
 - Downstream chiplet JTAG
 - SMC TAP interface
 - SEP TAP Interface
 - An optional number of additional STAPs
 - Extended STAP scan interface
- An iJTAG network
 - A debug port for the SMC fabric
 - A debug port for the SMC OTP
 - A debug port for the SEP OTP
 - A port for extending the iJTAG network outside the interface unit

The PTAP, STAP and all other supporting logic conform to IEEE standards 1149.1-2013 *Test Access Port and Boundary-Scan Architecture*, 1687-2014 *Access and Control of Instrumentation Embedded within a Semiconductor Device*, and 1838-2019 *Test Access Architecture for Three-Dimensional Stacked Integrated Circuits*.

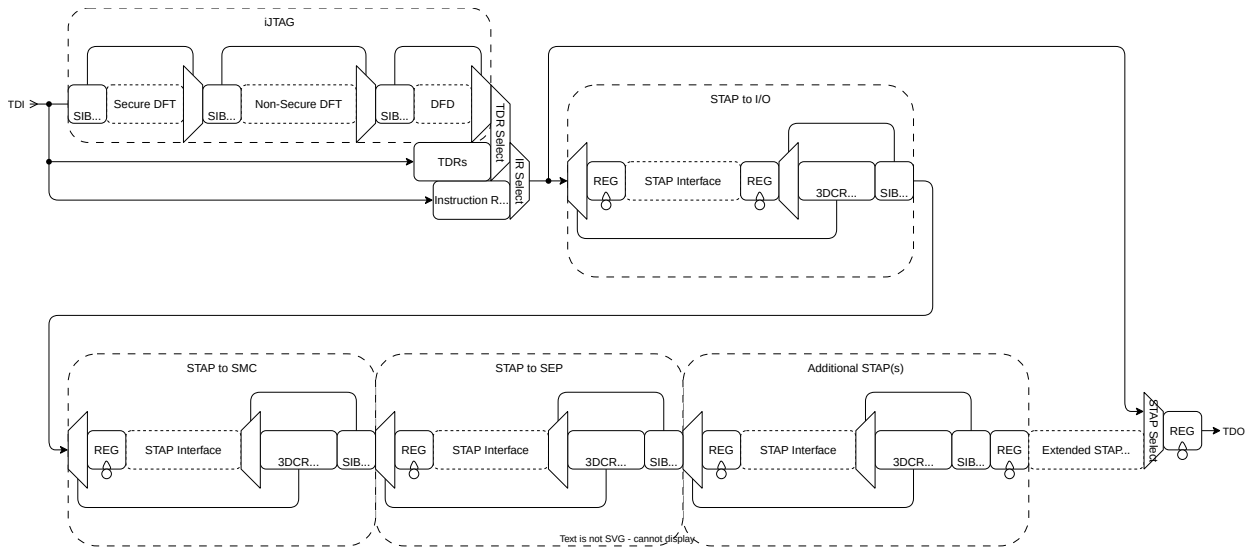
6.2.2. Configuration Parameters

Parameter	Range	Default	Description
NUM_STAP	0-8	4	Number of STAP interfaces
STAP_IO_EN	0/1	1	Enable I/O STAP for chiplet-to-chiplet connectivity
SMC_DBG_EN	0/1	1	Enable SMC debug STAP
SEP_DBG_EN	0/1	1	Enable SEP debug STAP
NUM_XTRA_STAP	0-4	0	Number of additional STAPs

6.2.3. Architecture

The JTAG Interface Unit integrates PTAP, STAP chain, and iJTAG network into a unified debug access point.

Shown below is an abstract diagram of the JTAG scan chain.



Scan Chain Topology

The PTAP is connected to the incoming JTAG interface for the chiplet. The scan chain is organized in three major segments: the Primary TAP processes all instructions, the Secondary TAP chain provides hierarchical TAP access, and the iJTAG network enables instrument-level access to debug blocks.

STAP Chain Hierarchy

The STAP chain implements a four-level hierarchy with configurable lockup latch insertion optimized for different physical routing distances.

Level	STAP Type	TDI_LOCKUP	SCAN_OUT_LOCKUP	Purpose
1	I/O STAP	1	0	Chiplet-to-chiplet connectivity (die boundary crossing)
2	SMC Debug STAP	0	0	On-chip connection to SMC CPU debugger TAP
3	SEP Debug STAP	0	0	On-chip connection to SEP CPU debugger TAP
4	Extra STAPs [N-1:0]	0	$i == N-1 ? 1 : 0$	Local debug ports (scan output lockup on last STAP only)

The I/O STAP enables TDI lockup to handle the longer propagation delays across die boundaries. On-chip STAPs (SMC Debug, SEP Debug) disable TDI lockup since signal routing is local to the chiplet. The final extra STAP in the chain enables scan output lockup when `NUM_EXTRA_STAPS > 0`, providing additional output timing margin before the scan path returns to the PTAP.

Feature Control Synchronization

The lifecycle feature control signals originate in the system clock domain and must be synchronized to the TCK domain before use. The interface unit implements 64 individual 2-FF synchronizer chains, one per bit of the feature control vector. This per-bit synchronization approach breaks up the CDC timing path that would exist if the entire 64-bit vector were synchronized as a single unit.

The synchronized feature control bits determine security access levels for STAP and SIB selection:

Access Level	Feature Control Condition
I/O STAP	sip_debug
SMC Debug STAP	soc_debug ap_debug
SEP Debug STAP	sep_debug soc_debug ap_debug
Extra STAPs	ap_debug
DFT Secure SIB	fuse_test sep_debug soc_debug ap_debug
DFT Non-Secure SIB	soc_debug ap_debug
DFD SIB	ap_debug

ijTAG Network

The ijTAG network is organized into three cascaded Segment Insertion Bit (SIB) chains with progressively restrictive security requirements. Each SIB gates access to a specific set of debug instruments based on the synchronized feature control signals.

DFT Secure Chain

The DFT Secure chain is accessible when any of fuse test, SEP debug, SOC debug, or AP debug features are enabled. This chain is ported out of the DTP for connection to tooling-generated DFT logic such as MBIST controllers and scan compression logic.

DFT Non-Secure Chain

The DFT Non-Secure chain requires either SOC debug or AP debug feature enablement. This chain provides access to DFT instrumentation that does not involve security-sensitive fuse or SEP operations.

DFD Chain

The Debug Forensics Dump (DFD) chain requires AP debug feature enablement, providing the highest security level. This chain grants access to:

- SMC AXI4 system fabric via JTAG2AXI bridge
- SMC OTP controller via AXI-Lite-to-APB bridge
- SEP OTP controller via AXI-Lite-to-APB bridge
- Extension port for integrator-defined debug instruments

JTAG2AXI Interfaces

The interface unit instantiates multiple JTAG2AXI bridges:

- **SMC JTAG2AXI TDRs:** This set of TDRs provide a mechanism to inject AXI transactions onto the SMC AXI local interconnect.
- **SMC Fuse JTAG2AXIL TDRs:** This set of TDRs provide a mechanism to interact directly with the SMC fuse controller. The JTAG2AXIL interface connects to the fuse controller CSRs through an AXI4-lite-to-APB bridge.
- **SEP Fuse JTAG2AXIL TDRs:** This set of TDRs provide a mechanism to interact directly with the SEP fuse controller. The JTAG2AXIL interface connects to the fuse controller CSRs through an AXI4-lite-to-APB bridge.

Security Integration

The JTAG Interface Unit integrates with the chiplet lifecycle controller:

- Security disable signals gate STAP selection
- Lifecycle feature control sync for debug enable/disable
- Per-STAP security lockout capability

6.2.4. Interface

Instruction Encodings

The JTAG interface unit uses 6-bit instruction encodings:

Encoding	Name	Register Size	Security
0x00	BYPASS (IEEE 1149.1, Section 8.4)	1	
0x01	IDCODE (IEEE 1149.1, Section 8.13)	32	
0x02	RUNBIST (IEEE 1149.1, Section 8.10)	BIST	
0x03	SAMPLE_PRELOAD (IEEE 1149.1, Sections 8.6 and 8.7)	BSR	
0x04	EXTEST (IEEE 1149.1, Section 8.8)	BSR	
0x05	EXTEST_TRAIN	BSR	
0x06	EXTEST_PULSE	BSR	
0x07	CLAMP (IEEE 1149.1, Section 8.11)	1	
0x08	HIGHZ (IEEE 1149.1, Section 8.16)	1	
0x09	INTEST (IEEE 1149.1, Section 8.9)	BSR	
0x0A	CLAMP_HOLD (IEEE 1149.1, Section 8.20)	2	
0x0B	CLAMP_RELEASE (IEEE 1149.1, Section 8.20)	2	
0x0C	TMP_STATUS (IEEE 1149.1, Section 8.20)	2	
0x0D	IC_RESET (IEEE 1149.1, Section 8.21)	Variable	
0x0E	TAP_3DCR (IEEE 1838, Section 5.5.1)	2	
0x10-0x17	<i>Reserved for RISC-V</i>		
0x18	DEBUG_CONTROL	5	
0x19	JTAG_CAPS	40	
0x1A	SELECT_IJTAG	Variable	
0x1B	SMC_OTP_JTAG2AXI_CAPS		
0x1C	SMC_OTP_AXI_SINGLE_OP		
0x1D	SMC_OTP_AXI_SERIES_CTRL		
0x1E	SMC_OTP_AXI_SERIES_DATA_INCR		
0x1F	SMC_OTP_AXI_SERIES_DATA_NO_IN CR		
0x20	SMC_OTP_AXI_SERIES_DATA_WITH_ERROR_STATUS		

Encoding	Name	Register Size	Security
0x21	SEP_OTP_JTAG2AXI_CAPS		
0x22	SEP_OTP_AXI_SINGLE_OP		
0x23	SEP_OTP_AXI_SERIES_CTRL		
0x24	SEP_OTP_AXI_SERIES_DATA_INCR		
0x25	SEP_OTP_AXI_SERIES_DATA_NO_INCR		
0x26	SEP_OTP_AXI_SERIES_DATA_WITH_ERROR_STATUS		
0x27	SMC_JTAG2AXI_CAPS		
0x28	SMC_AXI_SINGLE_OP		
0x29	SMC_AXI_SERIES_CTRL		
0x2A	SMC_AXI_SERIES_DATA_INCR		
0x2B	SMC_AXI_SERIES_DATA_NO_INCR		
0x2C	SMC_AXI_SERIES_DATA_WITH_ERROR_STATUS		
0x3D	ZERO_LENGTH_BYPASS	0	
0x3E	INV_BYPASS		
0x3F	BYPASS (IEEE 1149.1, Section 8.4)	1	

NOTE: All instruction encodings *not* explicitly defined in this table map to the BYPASS instruction.

Signal Interface

Signal	Direction	Width	Description
clk_i	Input	1	System clock for AXI interfaces
rst_ni	Input	1	Active-low system reset
pwr_on_rst_ni	Input	1	Power-on reset for JTAG logic
tck_i	Input	1	JTAG TCK
tdi_i	Input	1	JTAG TDI
tdo_o	Output	1	JTAG TDO
tms_i	Input	1	JTAG TMS
trst_ni	Input	1	JTAG TRST
tdo_en_o	Output	1	TDO output enable
jtag_clock_stop_o	Output	1	Clock stop from DEBUG_CONTROL
cla_clock_stop_en_o	Output	1	CLA clock stop enable from DEBUG_CONTROL
cla_clock_stop_i	Input	1	CLA clock stop status for readback
boot_stall_o	Output	1	Boot stall control
boot_stall_ovrd_o	Output	1	Boot stall override enable
security_disable_i	Input	1	Lifecycle security disable
stap_ext_*	Bidirectional		Extended STAP chain interface
ijtag_ext_*	Bidirectional		Extended iJTAG interface
smc_axi_*	Output		SMC AXI master interface
smc_otp_axil_*	Output		SMC OTP AXI-Lite interface

Signal	Direction	Width	Description
sep_otp_axil_*	Output		SEP OTP AXI-Lite interface

Dependencies

The JTAG Interface Unit depends on the following components:

- `hw/ip/jtag_ptap` - Primary TAP
- `hw/ip/jtag_stap` - Secondary TAP
- `hw/common/prim` - Primitive cells

6.3. JTAG PTAP

6.3.1. Overview

The Primary TAP (PTAP) provides the main JTAG access point for the chiplet, implementing IEEE 1149.1-2013 compliance with extensions for IEEE 1687-2014 (iJTAG) and IEEE 1838-2019 (3DIC).

The PTAP design includes:

- Full IEEE 1149.1-2013 JTAG TAP controller
- Test Data Registers (TDRs) for all defined JTAG instructions
- Test Mode Persistence (TMP) support
- Boundary scan support with scan ports and control signals
- BIST support
- 3DIC support with STAP scan ports and control signals
- iJTAG network access
- JTAG2AXI debug interface for memory-mapped access

6.3.2. Configuration Parameters

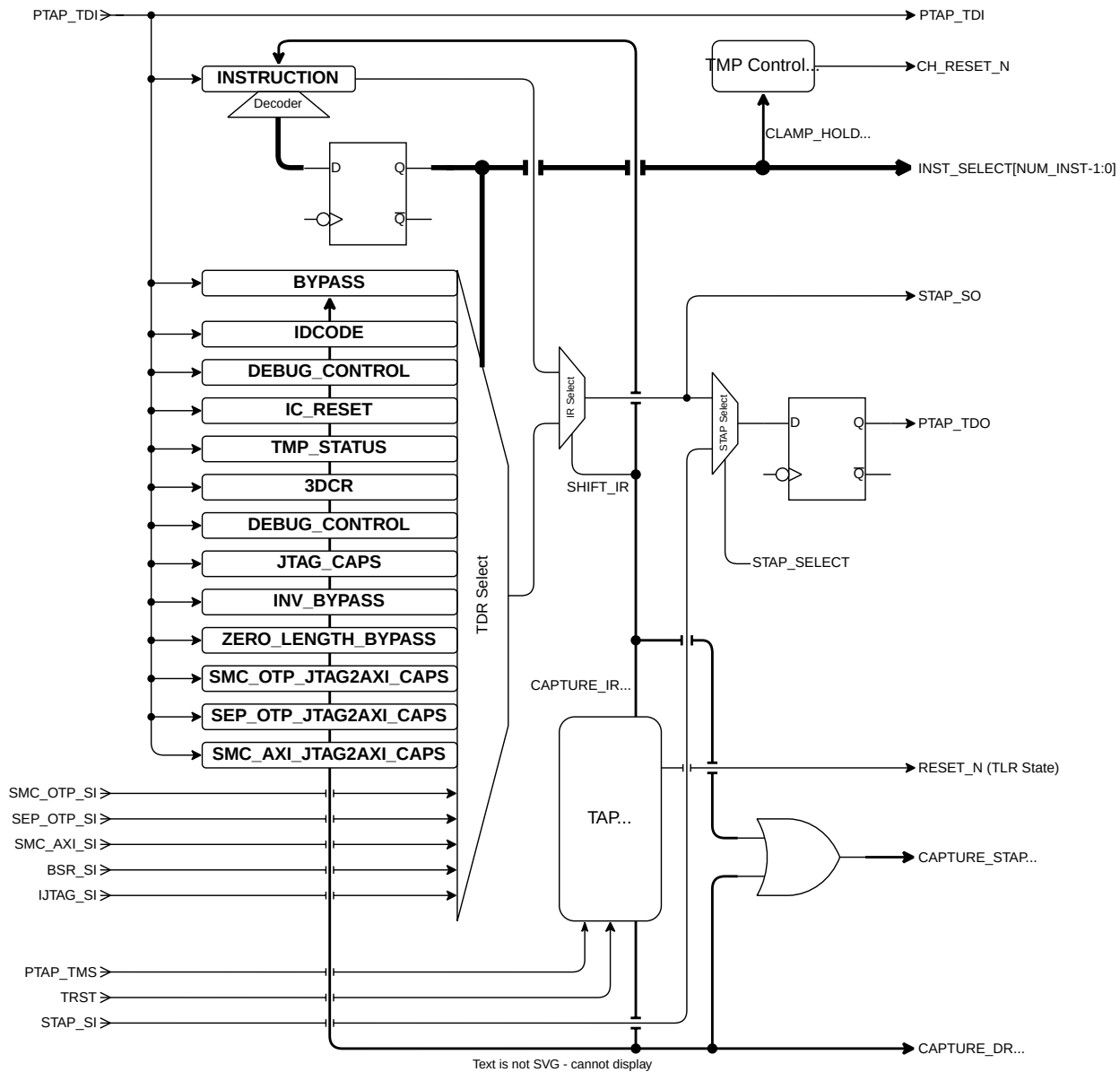
The PTAP is highly parameterizable to support different feature sets and integration requirements.

Parameter	Type	Default	Description
Feature Enable Parameters			
BSR_ENABLE	bit	1	Enables all mandatory JTAG boundary scan instructions
EXTEST_TRAIN_ENABLE	bit	1	Enables optional EXTEST_TRAIN instruction
EXTEST_PULSE_ENABLE	bit	1	Enables optional EXTEST_PULSE instruction
INTEST_ENABLE	bit	1	Enables optional INTEST instruction
CLAMP_ENABLE	bit	1	Enables optional CLAMP instruction
HIGHZ_ENABLE	bit	1	Enables optional HIGHZ instruction
RUNBIST_ENABLE	bit	1	Enables optional RUNBIST instruction
TMP_ENABLE	bit	1	Enables TMP controller functionality and instructions

Parameter	Type	Default	Description
IC_RESET_SMC_ENABLE	bit	0	Enables the SMC slice of the IC_RESET TDR (requires matching <code>ic_reset_smc_t</code>)
IC_RESET_SEP_ENABLE	bit	0	Enables the SEP slice of the IC_RESET TDR (requires matching <code>ic_reset_sep_t</code>)
IC_RESET_EXT_ENABLE	bit	0	Enables the external slice of the IC_RESET TDR (requires matching <code>ic_reset_ext_t</code>)
SMC_DBG_ENABLE	bit	1	Enables JTAG2AXI ports for SMC debug interface
SEP_DBG_ENABLE	bit	1	Enables STAP for SEP debug interface
STAP_IO_ENABLE	bit	1	Enables STAP for chiplet-to-chiplet connectivity
IC_RESET Struct Type Parameters			
<code>ic_reset_smc_t</code>	type	logic	Packed struct type for the SMC IC_RESET slice (typically <code>smc_pkg::jtag_smc_reset_ctrl_t</code>). Must contain <code>.ovrd</code> and <code>.val</code> sub-structs of identical width.
<code>ic_reset_sep_t</code>	type	logic	Packed struct type for the SEP IC_RESET slice (typically <code>sep_pkg::jtag_sep_reset_ctrl_t</code>). Must contain <code>.ovrd</code> and <code>.val</code> sub-structs of identical width.
<code>ic_reset_ext_t</code>	type	logic	Packed struct type for the external IC_RESET slice. Must contain <code>.ovrd</code> and <code>.val</code> sub-structs of identical width.
Size Parameters			
NUM_EXTRA_STAPS	int	0	Number of additional STAPs for local connectivity
NUM_XTRIG_CTP	int	8	Number of cross trigger ports
NUM_XTRIG_INT_CT	int	1	Number of internal cross triggers
IDCODE Parameters			
IDCODE_MFR_ID	logic[10:0]	11'h000	JEDEC manufacturer ID (11 bits)
IDCODE_PART_NUM	logic[15:0]	16'h0000	Chiplet model ID (16 bits)
IDCODE_SI_REV	logic[3:0]	4'h0	Silicon revision (4 bits)
Version Parameter			
OCH_VER	logic[7:0]	8'h00	DTP IP major version number
AXI Pipeline Depth Parameters			
SMC_OTP_RD_PL_DEPTH	logic[1:0]	2'h3	SMC OTP read pipeline depth (0=single outstanding)
SMC_OTP_WR_PL_DEPTH	logic[1:0]	2'h3	SMC OTP write pipeline depth
SEP_OTP_RD_PL_DEPTH	logic[1:0]	2'h3	SEP OTP read pipeline depth
SEP_OTP_WR_PL_DEPTH	logic[1:0]	2'h3	SEP OTP write pipeline depth
SMC_RD_PL_DEPTH	logic[1:0]	2'h3	SMC fabric read pipeline depth
SMC_WR_PL_DEPTH	logic[1:0]	2'h3	SMC fabric write pipeline depth

6.3.3. Architecture

The PTAP is fully compliant with the IEEE JTAG specifications. It implements a modular design with a core TAP controller, dedicated register modules for each instruction type, and optional AXI bridge modules for debug access. The architecture supports Test Mode Persistence (TMP), boundary scan with dedicated scan ports and control signals, BIST functionality, and 3DIC support with STAP scan ports and control signals derived from internal TDRs.



Module Hierarchy

The PTAP implementation instantiates the following submodules:

Module	Purpose	Conditional
<code>jtag_tap_ctrlr</code>	IEEE 1149.1 TAP controller FSM (16 states, one-hot encoded)	Always instantiated
<code>jtag_inst_reg</code>	6-bit instruction register with 64-wide one-hot decoded output	Always instantiated
<code>jtag_byp_reg</code>	1-bit bypass register (BYPASS instruction)	Always instantiated
<code>jtag_inv_byp_reg</code>	1-bit inverted bypass register (INV_BYPASS instruction)	Always instantiated
<code>jtag_idcode_reg</code>	32-bit IDCODE register	Always instantiated
<code>jtag_3dcr_reg</code>	2-bit 3D Control Register for STAP control	When <code>STAP_IO_ENABLE SEP_DBG_ENABLE SMC_DBG_ENABLE NUM_EXTRA_STAPS != 0</code>
<code>jtag_tmp</code>	Test Mode Persistence controller	When <code>TMP_ENABLE=1</code>
<code>jtag_tmp_status_reg</code>	2-bit TMP status register	When <code>TMP_ENABLE=1</code>

Module	Purpose	Conditional
jtag_ic_reset_reg	IC_RESET control register (2*(NUM_SMC_IC_RESET+NUM_SEP_IC_RESET+NUM_EXT_IC_RESET)+1 bits)	When any of IC_RESET_SMC_ENABLE, IC_RESET_SEP_ENABLE, IC_RESET_EXT_ENABLE is 1
jtag_debug_ctrl_reg	5-bit debug control register	Always instantiated
jtag_caps_reg	60-bit capabilities readback register	Always instantiated
prim_jtag2axi (SMC OTP)	JTAG-to-AXI-Lite bridge for SMC OTP controller	When SMC_DBG_ENABLE=1
prim_jtag2axi (SEP OTP)	JTAG-to-AXI-Lite bridge for SEP OTP controller	When SEP_DBG_ENABLE=1
prim_jtag2axi (SMC fabric)	JTAG-to-AXI bridge for SMC system fabric	When SMC_DBG_ENABLE=1

TAP Controller State Machine

The TAP controller implements the IEEE 1149.1 16-state finite state machine using one-hot encoding. Each state is assigned a unique bit position within a 16-bit vector, enabling direct state detection via single-bit comparisons.

State	Encoding	Description
TEST_LOGIC_RESET	16'h0001	State 0: Reset state, instruction register loads IDCODE
RUN_TEST_IDLE	16'h0002	State 1: Idle state between operations
SELECT_DR_SCAN	16'h0004	State 2: Select data register scan path
CAPTURE_DR	16'h0008	State 3: Capture data into selected DR
SHIFT_DR	16'h0010	State 4: Shift data through selected DR
EXIT1_DR	16'h0020	State 5: Exit from shift to either pause or update
PAUSE_DR	16'h0040	State 6: Pause DR shift operation
EXIT2_DR	16'h0080	State 7: Exit from pause back to shift or update
UPDATE_DR	16'h0100	State 8: Update DR from shift register
SELECT_IR_SCAN	16'h0200	State 9: Select instruction register scan path
CAPTURE_IR	16'h0400	State 10: Capture data into IR (fixed pattern 6'b000001)
SHIFT_IR	16'h0800	State 11: Shift data through IR
EXIT1_IR	16'h1000	State 12: Exit from shift to either pause or update
PAUSE_IR	16'h2000	State 13: Pause IR shift operation
EXIT2_IR	16'h4000	State 14: Exit from pause back to shift or update
UPDATE_IR	16'h8000	State 15: Update IR, decode new instruction

The state machine includes an automatic fault recovery mechanism. A 3-bit TMS reset counter tracks consecutive TMS high assertions. When the counter reaches 5, the state machine automatically transitions to TEST_LOGIC_RESET regardless of current state, providing recovery from corrupted or invalid states.

Control signals are generated using a split approach. Update, run-test-idle, and test-logic-reset signals are generated combinatorially as Moore machine outputs. Capture and shift signals are registered on the falling edge of TCK to ensure stable control windows before the rising edge samples data.

Reset Hierarchy

The PTAP employs a three-level reset hierarchy to support power-on initialization, JTAG reset, and conditional register preservation.

Reset Signal	Source	Scope
<code>jtag_trst_n</code>	<code>pwr_on_rst_ni && client_tap_ctrl_i.trst_n</code>	Primary reset for all JTAG logic, combines power-on and external TRST
<code>test_logic_reset</code>	Derived from TAP FSM state (asserted when state == TEST_LOGIC_RESET)	Resets instruction and data registers
<code>chrst_n</code>	<code>TMP_ENABLE ? (!test_logic_reset persistence_mode_i) : !test_logic_reset</code>	Channel reset for TMP-aware registers, preserves state when in persistence mode

The primary reset `jtag_trst_n` is formed by AND-gating the power-on reset input with the external TRST signal. This combined reset feeds all JTAG clock domain logic. The TAP controller derives `test_logic_reset` from the FSM state, which normally resets all registers. When Test Mode Persistence is enabled, the conditional reset `chrst_n` allows boundary scan and configuration registers to survive TEST_LOGIC_RESET state transitions while in persistence mode.

TDR Multiplexer

The PTAP implements a 41-case instruction-dependent data register multiplexer. During IR scan operations, the instruction register connects to the scan path. During DR scan operations, the appropriate data register is selected based on the currently decoded instruction. Each test data register has dedicated scan control logic that enables selection only when the corresponding instruction is active and the required feature is enabled.

The scan path supports zero-length bypass through explicit `prim_stdmux2` multiplexer cells rather than behavioral multiplexing. This implementation prevents glitches on scan path selectors during instruction transitions. The zero-length bypass instruction connects TDI directly to TDO without intermediate register stages, providing minimum scan latency for hierarchical JTAG chains.

STAP selection is implemented as a second-level multiplexer. When the 3DCR register's select bit is asserted, the STAP chain input replaces the TDR multiplexer output. The STAP scan control signals are generated by OR-combining the instruction register and data register scan controls, enabling STAP operation during both IR and DR scan sequences as required by IEEE 1838.

TDO Retiming

The TDO output is retimed on the falling edge of TCK as required by IEEE 1149.1 Section 4.5.2. The retiming register ensures that TDO transitions occur at the same clock edge that samples TDI at the receiving device, maintaining timing margins across board-level JTAG chains. For zero-length bypass operation, the retimer is bypassed using an explicit `prim_stdmux2` cell to minimize scan latency.

ID Code

The IDCODE instruction accesses a TDR containing device identification information.

Bits	Field	Description	Access
31:28	<code>si_rev</code>	Silicon revision	RO
27:12	<code>part_num</code>	Chiplet model ID	RO
11:1	<code>mfr_id</code>	JEDEC-assigned manufacturer ID, encoded into 11 bits	RO
0	<code>marker</code>	Marker bit, always 1	RO

To access the IDCODE TDR:

1. Shift the IDCODE instruction into the instruction register using the IR-Scan sequence.

2. Shift any pattern into the IDCODE TDR on TDI using the DR-Scan sequence, starting with the least significant bit.
3. The bits shifted into the IDCODE TDR will be ignored. The contents of the IDCODE TDR will shift out on TDO.

Debug Control

The DEBUG_CONTROL instruction accesses a TDR defining general control bits for debug and control of the design.

Bits	Field	Description	Access	Reset
4	cla_clock_stop	When set, indicates that a CLA in the chiplet attempted to stop the clocks. Note that the clocks will actually have been stopped only when <i>cla_clock_stop_en</i> is set.	RO	0
3	jtag_clock_stop	When set, stops all clocks in the chiplet. This is wired to the functional clock gates on all of the PLLs in the chiplet. This bit is active high.	R/W	0
2	cla_clock_stop_en	When set, enables the CLAs to stop the clocks in the design. This bit indicates that an external debugger is connected and ready to do debug or scan dump operations after the CLA stops the clocks. This bit is intended to protect against unintentional CLA clock stops in the field due to incorrect CLA programming. This bit is active high.	R/W	0
1	boot_stall_ovrd	When set, overrides the boot stall signal to the internals of the part to use the value from boot_stall in this TDR instead. This bit is active high.	R/W	0
0	boot_stall	When boot_stall_override is set, the value in this bit is used for boot stall. This bit is active high: setting to 1 causes boot stall.	R/W	0

To access the DEBUG_CONTROL TDR:

1. Shift the DEBUG_CONTROL instruction into the instruction register using the IR-Scan sequence.
2. Shift the desired data into the DEBUG_CONTROL TDR on TDI using the DR-Scan sequence, starting with the least significant bit.
3. R/W bits shifted into the IDCODE TDR will be captured by the TDR. RO bits will be ignored. The contents of the DEBUG_CONTROL TDR will shift out on TDO.

JTAG Capabilities

The JTAG_CAPS instruction accesses a read-only TDR reporting the DTP version number and general configuration

parameters for the DTP.

Bits	Field	Description	Access	
59:54	num_xtrig_int_ct	The number of internal cross trigger signals.	RO	
53:48	num_xtrig_ctp	The number of Cross Trigger Ports.	RO	
47:44	num_xtra_stap	The number of additional DTP STAP interfaces.	RO	
43	stap_io_en	The STAP interface for chiplet-to-chiplet connectivity is enabled.	RO	
42	sep_dbg_en	The SEP STAP interface enabled.	RO	
41	smc_dbg_en	The SMC JTAG2AXI interface is enabled.	RO	
40:33	num_smc_ic_rst	The number of SMC IC_RESET override ports.	RO	
32:25	num_sep_ic_rst	The number of SEP IC_RESET override ports.	RO	
24:17	num_ext_ic_rst	The number of external IC_RESET override ports.	RO	
16	ic_rst_inst_en	The IC_RESET instruction is enabled.	RO	
15	tmp_inst_en	The TMP controller functionality and instructions are enabled.	RO	
14	runbist_inst_en	The RUNBIST instruction is enabled.	RO	
13	highz_inst_en	The HIGHZ instruction is enabled.	RO	
12	clamp_inst_en	The CLAMP instruction is enabled.	RO	
11	intest_inst_en	The INTEST instruction is enabled.	RO	
10	extest_pulse_en	The EXTEST_PULSE instruction is enabled.	RO	
9	extest_train_en	The EXTEST_TRAIN instruction is enabled.	RO	
8	bsr_inst_en	All mandatory boundary scan instructions are enabled.	RO	
7:0	och_ver	The MAJOR version number of the DTP IP.	RO	

To access the JTAG_CAPS TDR:

1. Shift the JTAG_CAPS instruction into the instruction register using the IR-Scan sequence.
2. Shift any pattern into the JTAG_CAPS TDR on TDI using the DR-Scan sequence, starting with the least significant bit.
3. The bits shifted into the JTAG_CAPS TDR will be ignored. The contents of the JTAG_CAPS TDR will shift out on TDO.

Boundary Scan Support

All boundary scan and BIST instructions defined in IEEE 1149.1 are optionally supported, but support is not enabled by default. RTL parameters are provided to selectively enable support for each instruction. Any disabled instruction has the same behavior as the BYPASS instruction.

To access the boundary scan registers:

1. Shift any boundary scan instruction that selects the BSRs into the instruction register using the IR-Scan sequence.
2. Shift any pattern into the BSRs on TDI using the DR-Scan sequence, starting with the least significant bit.
3. The contents of the BSRs will shift out on TDO.

BIST Support

The RUNBIST instruction is optionally supported, but support is not enabled by default. An RTL parameter is provided to selectively enable support. Any disabled instruction has the same behavior as the BYPASS instruction.

When the RUNBIST instruction is selected, the iJTAG network is selected as the active scan chain. An iJTAG network control signal indicates that the RUNBIST instruction is active. A designer may also utilize the decoded INST_SELECT signal to trigger the BIST logic for any instructions that they have enabled.

To execute BIST and access the BIST TDR:

1. Shift the RUNBIST instruction into the instruction register using the IR-Scan sequence.
2. Shift 2'b01 into TDI using the DR-Scan sequence to enable the SIB for the external iJTAG interface, starting with the least significant bit.
3. Shift {2'b01, BIST_TDR} into TDI using the DR-Scan sequence to access the BIST TDR, starting with the least significant bit. The BIST_TDR sequence is defined by the user's implementation of the extended iJTAG network. Multiple sequences may be required to configure the external iJTAG network to access the BIST TDR.
4. The contents of the BIST_TDR will shift out on TDO as {2'b01, BIST_TDR}.

Test Mode Persistence (TMP) Support

There is optional support for TMP, but support for TMP is disabled by default. An RTL parameter enables support for TMP.

When TMP support is enabled, a TMP controller is included in the PTAP. The TMP controller manages the active-low *CH_RESET_N* signal. The *CH_RESET_N* signal is the reset signal for any registers that must retain their state across TLR soft resets, mainly the boundary scan cells. *CH_RESET_N* asserts whenever the TAP is in the TLR state *except* when the TMP controller is in the *PERSISTENCE_ON* state. When the TMP controller is in the *PERSISTENCE_ON* state, the only events that will assert *CH_RESET_N* are full power-on-reset (POR) or TRST signal assertion.

When TMP support is disabled, the *CLAMP_HOLD*, *CLAMP_RELEASE*, and *TMP_STATUS* instructions have the same behavior as the *BYPASS* instruction, and the *CH_RESET_N* signal always responds to every POR, TRST, or TLR event, i.e. it is an exact copy of the *RESET_N* signal.

Bit	Description	Reset
Bypass Escape	Whenever this bit is set, the TMP controller will transition to the <i>PERSISTENCE_OFF</i> state when the TAP controller enters the <i>UPDATE_IR</i> state after the <i>BYPASS</i> instruction has been selected.	1

Bit	Description	Reset
TMP Status	This bit is set when the TMP controller is in the Persistence-On state.	0

3DIC Support

There is optional support for STAP interfaces, and support is enabled by default. An RTL parameter selects the number of STAP interfaces, where a value of 0 disables STAP support.

In order to properly support the IEEE 1838 standard, a chain of STAP interfaces must be inserted at the end of each chiplet's scan chain. This is done after the *IR Select* MUX to ensure that both the instruction registers and TDRs from all enabled PTAPs in the scan chain are scannable. As a consequence, the STAPs require dedicated capture, shift, and update control signals that are triggered during both the IR and DR sequences.

The PTAP has a 2-bit *3DCR* register that is accessed with the *TAP_3DCR* instruction. This register provides an option to bypass all STAPs in the scan chain.

Bit	Description	Reset
Config Hold	When this bit is set, the PTAP TLR state does not reset this TDR, and it will hold its current state until TRST is asserted.	0
Select	When this bit is unset, the PTAP's top-level STAP scan interface is completely bypassed.	0

When STAP support is disabled, the *TAP_3DCR* instruction has the same behavior as the *BYPASS* instruction, and the STAP scan chain cannot be selected.

IC_RESET Support

The PTAP provides support for a configurable number of IEEE 1149.1-compliant reset controls. The IC_RESET TDR is composed of three independently-enabled slices:

- **SMC slice** (nearest TDI): driven by the `ic_reset_smc_t` struct type and gated by `IC_RESET_SMC_ENABLE`. Sized from `$bits(ic_reset_smc_t)`.
- **SEP slice**: driven by the `ic_reset_sep_t` struct type and gated by `IC_RESET_SEP_ENABLE`. Sized from `$bits(ic_reset_sep_t)`.
- **External slice** (nearest TDO): driven by the `ic_reset_ext_t` struct type and gated by `IC_RESET_EXT_ENABLE`. Sized from `$bits(ic_reset_ext_t)`.

Each slice's struct must contain `.ovrd` and `.val` sub-structs of identical width (checked via an elaboration-time assertion in `jtag_ptap`). Within `jtag_ic_reset_reg`, the override-enable and control bits remain interleaved as `{enable, control}` pairs per the JTAG specification; `jtag_ptap` packs the three struct slices into that interleaved bus and unpacks the outputs back into structured form. The TDR carries the SMC slice closest to TDI, then SEP, then the external slice, followed by the single `reset_hold` bit.

The total length is $2 * (\text{NUM_SMC_IC_RESET} + \text{NUM_SEP_IC_RESET} + \text{NUM_EXT_IC_RESET}) + 1$ bits, where each `NUM*_IC_RESET` is half the width of its corresponding struct type when the matching enable parameter is set (otherwise 0).

The TDR has the following structure:

Bits	Field	Description	Access	Reset
2*n+2	reset_control	The controlled, active-low reset value driven while <i>reset_enable</i> is 0 (i.e. while JTAG override is active).	R/W	1
2*n+1	reset_enable	Active-LOW per IEEE 1149.1 §17: 1 (default) ⇒ JTAG override DISABLED (upstream reset passes through); 0 ⇒ JTAG override ENABLED (the port is driven by <i>reset_control</i>). The "enable" in the IEEE name refers to "enabling the controlled reset state", not the override mechanism.	R/W	1
0	reset_hold	When 0, TLR will not reset the <i>reset_enable</i> and <i>reset_control</i> bits. TRST/POR will always reset all bits.	R/W	1

IMPORTANT

Polarity convention — the TDR field names come from the IEEE 1149.1 §17 specification and use an active-low *reset_enable*. *jtag_ic_reset_reg* converts this into a conventional **active-high** override signal (*ic_reset_ovrd_o[i] = !reset_enable[i]*) before it leaves the register, so every downstream consumer sees *.ovrd == 1* meaning "JTAG is currently overriding this port". Do not re-invert this signal downstream.

For each of the resets defined in the TDR, the PTAP exposes two logical signals (grouped into the per-slice *jtag*_reset_ctrl_t* structs):

- *ic_reset_ovrd* / *.ovrd* - **Active-high** JTAG-override flag (the bit-wise inversion of *reset_enable*). 1 ⇒ JTAG is driving this reset, 0 ⇒ upstream reset passes through.
- *ic_reset_ctrl_n* / *.val* - Active-low reset value driven when *ic_reset_ovrd* is asserted (identical to the *reset_control* TDR bit).

ijTAG Support

The PTAP has optional support for a IEEE 1687-2014-compliant hierarchical ijTAG network, and support is enabled by default. When the *SELECT_IJTAG* instruction is enabled, the ijTAG network is selected for scan.

JTAG2AXI Support

*_JTAG2AXI_CAPS

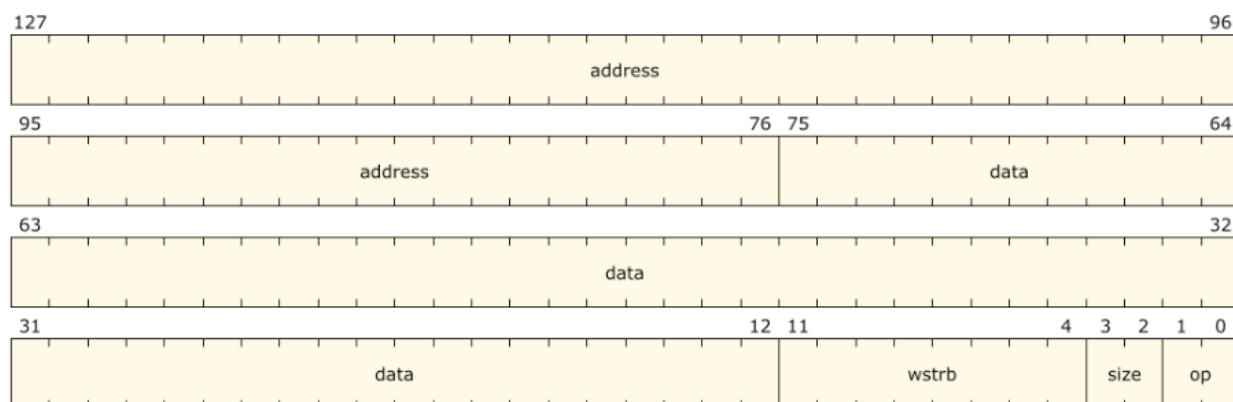
This is a private instruction that accesses a read-only TDR reporting the configuration parameters for the given JTAG2AXI debugging instrument.

Bits	Field	Description	Access	
13:12	rd_pl_depth	The depth of the transaction read queue where 0 indicates only a single outstanding transaction allowed at any given time.	RO	

Bits	Field	Description	Access	
11:10	wr_pl_depth	The depth of the transaction write queue where 0 indicates only a single outstanding transaction allowed at any given time.	RO	
9:7	data_size	The size of AxDATA in powers-of-2 bytes. 0: 1 byte (8 bits) 1: 2 bytes (16 bits) 2: 4 bytes (32 bits) 3: 8 bytes (64 bits) 4: 16 bytes (128 bits) 5: 32 bytes (256 bits) 6: 64 bytes (512 bits) 7: 128 bytes (1024 bits)	RO	
6:1	addr_size	The size of AxADDR in bits.	RO	
0	bus_type	The type of the AXI bus. 0: AXI4 1: AXI4-Lite	RO	

*_AXI_SINGLE_OP

This is a private instruction that generates a single AXI read or write transaction. It provides a mechanism for an external debugger to do sparse reads or writes to nearly any register in the design. Since the entire address must be shifted in for each transaction, this instruction is most useful for sparse accesses. This instruction uses semantics very similar to the RISC-V DTM DMI instruction.



Bits	Field	Description	Access	Reset
(...) +: addr_size	address	Address used for AXI access. In Update-DR the AXI transaction is issued. On a Capture-DR, this field holds the previous value.	R/W	0
(...) +: 8*(2**data_size)	data	The data to write on an AXI write during Update-DR. On a Capture-DR, this field contains the data returned from an AXI read if the previous operation was a read, or holds the write data previously used if the previous operation was a write.	R/W	0

Bits	Field	Description	Access	Reset
5 +: (2**data_size)	wstrb	The byte strobes for a write transaction. This value is driven on WSTRB[7:0] when the transaction is a write. On a Capture-DR, this field holds the previous value.	R/W	0
(...) +: \$bits(size)	size	The number of bytes to read/write. This field is used as AxSIZE for the transaction. 0: 1 byte (8 bits) 1: 2 bytes (16 bits) 2: 4 bytes (32 bits) 3: 8 bytes (64 bits) 4: 16 bytes (128 bits) 5: 32 bytes (256 bits) 6: 64 bytes (512 bits) 7: 128 bytes (1024 bits)	R/W	0
1:0	op	When the debugger writes this field, it has the following meaning: 0: Ignore data and address (nop) Don't send anything over AXI during Update-DR. This operation should never result in a busy or error response. 1: Read from address (read) 2: Write data to address (write) 3: Reserved When the debugger reads this field, it means the following: 0: The previous operation completed successfully and returned an OKAY status 1: The previous operation completed but failed and returned a SLVERR status 2: The previous operation completed but failed and returned a DECERR or other error status. 3: The latest operation was attempted before the previous operation completed; the previous AXI request is still in progress. The operation attempted in this access will be ignored. If a debugger sees this status, it needs to give the target more TCK edges between Update-DR and Capture-DR. The simplest way to do that is to add extra transitions in Run-Test/Idle.	R/W	0

*_AXI_SERIES_CTRL

This is a private instruction that preloads the start address of a region that will be subsequently read or written using the AXI_SERIES_DATA instruction. It is used when an external debugger wants to either:

- Read or write a contiguous block of registers or memory.
- Repeatedly read or write the same location (e.g., polling)

Using the series instructions instead of the AXI_SINGLE_OP instruction is much more time efficient for these use cases.



Bits	Field	Description	Access	Reset
(...) +: 1	reset	Reset the pending error flags reported in op.	R/W	0
(...) +: addr_size	address	Address used for AXI access. In Update-DR the AXI transaction is issued.	R/W	0
(...) +: 2	pl_depth	pipeline_depth For write transactions, this sets the number of pending unissued write transactions that can be queued by this block to pl_depth + 1. Setting a non-zero value allows the debugger to queue additional write transactions before the previous ones have been issued. For read transactions, this field sets the number of read responses that this block will pipeline. This allows the debugger to queue multiple reads to targets with long or variable latency while getting read data responses in a deterministic fashion. The first pl_depth + 1 reads will return DEADBEEF marker data. This field can be set to a value between 0 and (wr/rd)_pl_depth field read from the JTAG_CAPS register. When the debugger reads this field, an indication of whether there are any pending transactions will be returned: 2'b00: There are no pending transactions 2'b01: There is at least one pending transaction	R/W	0
(...) +: \$bits(size)	size	The number of bytes to read/write. This field is used as AxsIZE for the transaction. 0: 1 byte (8 bits) 1: 2 bytes (16 bits) 2: 4 bytes (32 bits) 3: 8 bytes (64 bits) 4: 16 bytes (128 bits) 5: 32 bytes (256 bits) 6: 64 bytes (512 bits) 7: 128 bytes (1024 bits)	R/W	0

Bits	Field	Description	Access	Reset
1:0	op	When the debugger writes this field, it has the following meaning: 0: Ignore data and address (nop). Don't do anything on Update-DR for AXI_SERIES_DATA. 1: Read from address (read). Update-DR for AXI_SERIES_DATA causes an AXI read to be performed. 2: Write data to address (write). Update-DR for AXI_SERIES_DATA causes an AXI write to be performed. 3: Reserved When the debugger reads this field, it means the following: 0: All previous operations completed successfully and returned an OKAY status. 1: At least one previous operation completed but failed and returned a SLVERR status. This status is sticky until cleared by AXI_SERIES_CTRL.reset. 2: At least one previous operation completed but failed and returned a DECERR or other error status. This status is sticky until cleared by AXI_SERIES_CTRL.reset. 3: At least one operation was attempted before the previous operation completed. The operations attempted when there was a pending operation in progress were ignored. If a debugger sees this status, it needs to give the target more TCK edges between Update-DR and Capture-DR. The simplest way to do that is to add extra transitions in Run-Test/Idle. This status is sticky until cleared by AXI_SERIES_CTRL.reset.	R/W	0

*_AXI_SERIES_DATA_INCR

This is a private instruction that does the actual read/write to the address previously specified.

After the access is completed, an internal address register used for the series, which was previously loaded by AXI_SERIES_CTRL, is incremented as follows:

```
AXI_SERIES_CTRL.address += 2**AXI_SERIES_CTRL.size
```

This allows the debugger to load or dump contiguous regions of address space.

Note that this is a variable-length data register. The length is n bits where $n = 8 \cdot (2^{**} \text{AXI_SERIES_CTRL.size})$.

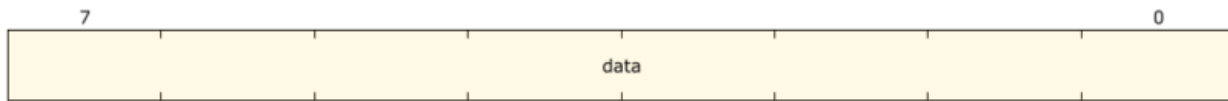


Figure 7. AXISeriesDataIncr when AXISeriesCtrl.size==0

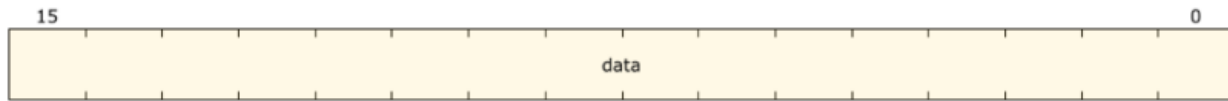


Figure 8. AXISeriesDataIncr when AXISeriesCtrl.size==1

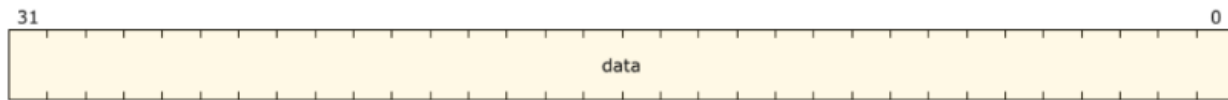


Figure 9. AXISeriesDataIncr when AXISeriesCtrl.size==2



Figure 10. AXISeriesDataIncr when AXISeriesCtrl.size==3

Bits	Field	Description	Access	Reset
n-1:0	data	Data value to read/write. The read data is from the previous read access.	R/W	0

*_AXI_SERIES_DATA_NO_INCR

This is a private instruction that does the actual read/write to the address previously specified.

Unlike AXI_SERIES_DATA_INCR, this instruction does not change the address in AXI_SERIES_CTRL. This instruction allows the debugger to continue to read or write this same address using additional AXI_SERIES_DATA_NO_INCR instructions. This is useful for repeated polling or triggering.

Note that this is a variable-length data register. The length is n bits where $n = 8 \cdot (2 \cdot \text{AXI_SERIES_CTRL.size})$.

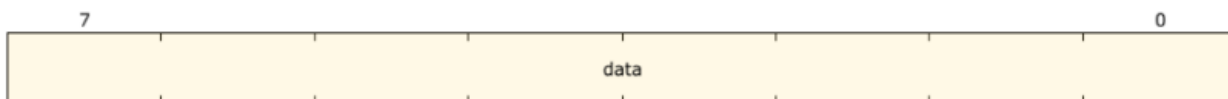


Figure 11. AXISeriesDataNoIncr when AXISeriesCtrl.size==0

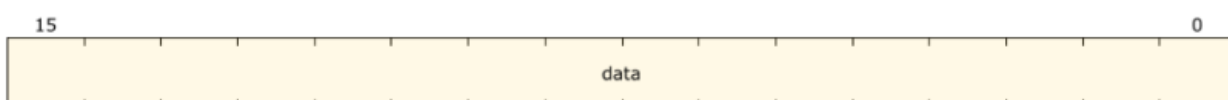


Figure 12. AXISeriesDataNoIncr when AXISeriesCtrl.size==1

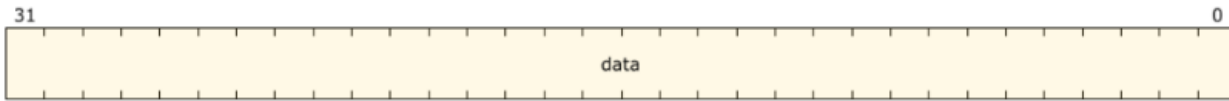


Figure 13. AXISeriesDataNoIncr when AXISeriesCtrl.size==2

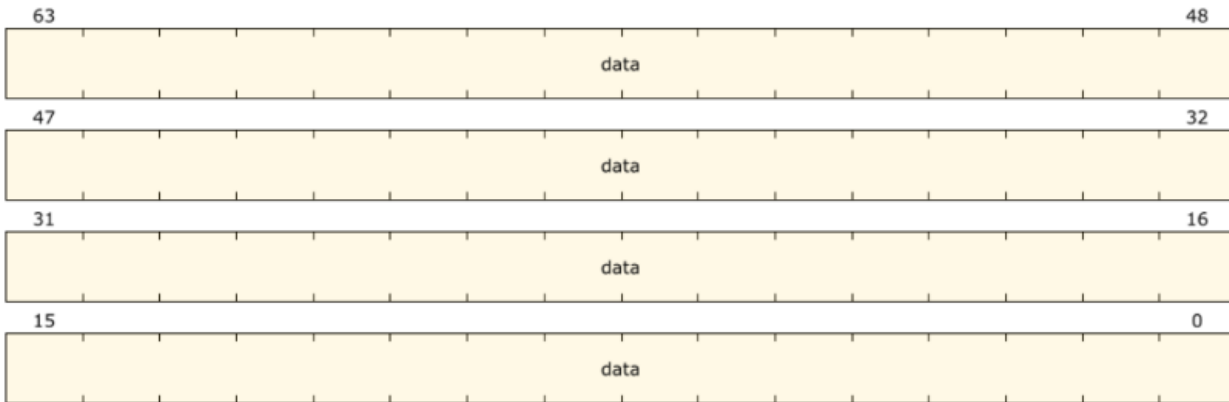


Figure 14. AXISeriesDataNoIncr when AXISeriesCtrl.size==3

Bits	Field	Description	Access	Reset
n-1:0	data	Data value to read/write. The read data is from the previous read access.	R/W	0

*_AXI_SERIES_DATA_WITH_ERROR_STATUS

This is a private instruction that does the actual read/write to the address previously specified.

Unlike AXI_SERIES_DATA_INCR and AXI_SERIES_DATA_NO_INCR, the increment is selectable per transaction and the error status from the previous status is returned. This instruction is used when granular error reporting is desired during a block operation.

Note that this is a variable-length data register. The length is n+1 bits where $n = 8 * (2^{AXI_SERIES_CTRL.size})$.

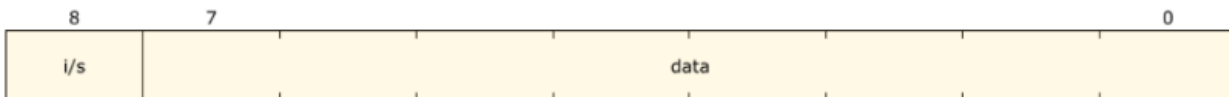


Figure 15. AXISeriesDataWithErrorStatus when AXISeriesCtrl.size==0

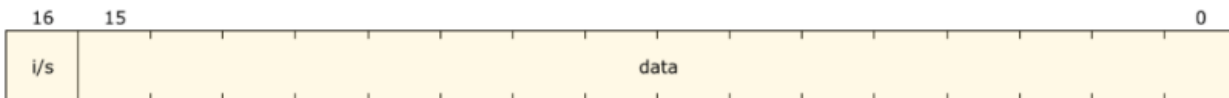


Figure 16. AXISeriesDataWithErrorStatus when AXISeriesCtrl.size==1

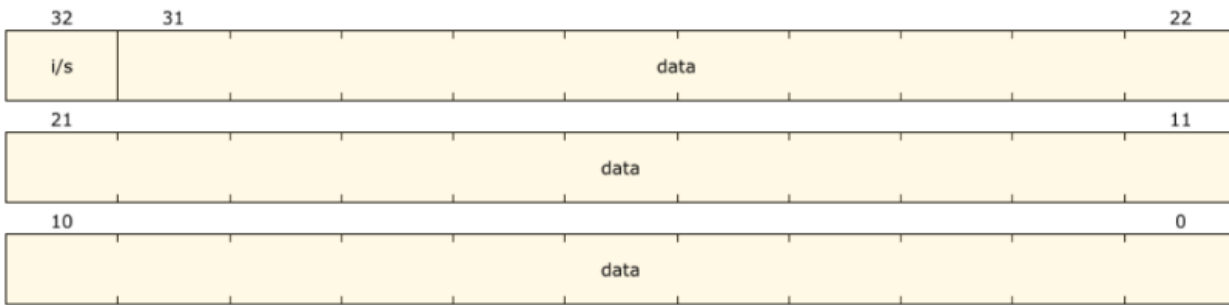


Figure 17. AXISeriesDataWithErrorStatus when AXISeriesCtrl.size=2

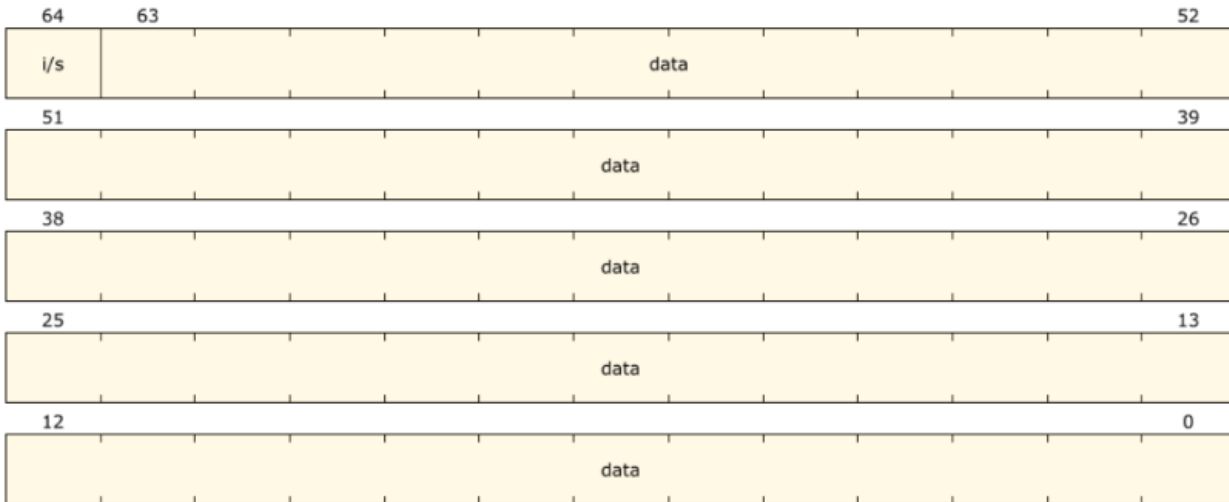


Figure 18. AXISeriesDataWithErrorStatus when AXISeriesCtrl.size=3

Bits	Field	Description	Access	Reset
n	increment/status	The value shifted into this bit indicates whether the address should be incremented after this transaction completes. The value shifted out of this bit indicates whether the previous transaction completed successfully (0 = succeeded, 1 = failed). An AXI result of SLVERR or DECERR as well as attempting a new transaction while the previous was still pending will cause the status to be returned as "failed".	R/W	0
n-1:0	data	Data value to read/write. The read data is from the previous read access.	R/W	0

Security and Lifecycle Control

The PTAP integrates with the chiplet lifecycle controller through the `feat_ctrl_i` input signal to enforce security policies on debug interfaces. This 64-bit feature control vector originates from the SEP eFuse controller and determines which debug features are accessible based on the current lifecycle state. `feat_ctrl_i` is enable-polarity (1 = feature enabled).

The PTAP derives three independent security disable signals from the feature control vector. Each bridge is enabled

only when **all** of its listed enables are asserted, so it is disabled when **any** one of them is deasserted:

Debug Interface	Enabled When Feature Control Satisfies	Security Disable Signal
SMC Fabric JTAG2AXI	<code>feat_ctrl_i.soc_debug & feat_ctrl_i.ap_debug</code>	<code>!feat_ctrl_i.soc_debug !feat_ctrl_i.ap_debug</code>
SMC OTP JTAG2AXIL	<code>feat_ctrl_i.fuse_test & feat_ctrl_i.soc_debug & feat_ctrl_i.ap_debug</code>	<code>!feat_ctrl_i.fuse_test !feat_ctrl_i.soc_debug !feat_ctrl_i.ap_debug</code>
SEP OTP JTAG2AXIL	<code>feat_ctrl_i.fuse_test & feat_ctrl_i.sep_debug & feat_ctrl_i.soc_debug & feat_ctrl_i.ap_debug</code>	<code>!feat_ctrl_i.fuse_test !feat_ctrl_i.sep_debug !feat_ctrl_i.soc_debug !feat_ctrl_i.ap_debug</code>

Each derived security disable signal feeds into the corresponding `prim_jtag2axi` module instance. When a debug interface is disabled by lifecycle state, the JTAG2AXI module blocks all AXI transaction generation, preventing unauthorized access to memory-mapped debug registers and system resources.

6.3.4. Interface

Instruction Encodings

The PTAP uses 6-bit instruction encodings. During the capture phase, the least significant bit (closest to TDO) of the instruction register always presents a 1 to the scan chain, and the next least significant bit presents a 0 as specified by the 1149.1 standard, section 7.2.1. All other bits of the instruction register always return 0s.

Encoding	Name	Register Size	Description
0x00	BYPASS	1	IEEE 1149.1, Section 8.4
0x01	IDCODE	32	IEEE 1149.1, Section 8.13
0x02	RUNBIST	BIST	IEEE 1149.1, Section 8.10
0x03	SAMPLE_PRELOAD	BSR	IEEE 1149.1, Sections 8.6 and 8.7
0x04	EXTEST	BSR	IEEE 1149.1, Section 8.8
0x05	EXTEST_TRAIN	BSR	
0x06	EXTEST_PULSE	BSR	
0x07	CLAMP	1	IEEE 1149.1, Section 8.11
0x08	HIGHZ	1	IEEE 1149.1, Section 8.16
0x09	INTEST	BSR	IEEE 1149.1, Section 8.9
0x0A	CLAMP_HOLD	2	IEEE 1149.1, Section 8.20
0x0B	CLAMP_RELEASE	2	IEEE 1149.1, Section 8.20
0x0C	TMP_STATUS	2	IEEE 1149.1, Section 8.20
0x0D	IC_RESET	Variable	IEEE 1149.1, Section 8.21
0x0E	TAP_3DCR	2	IEEE 1838, Section 5.5.1
0x18	DEBUG_CONTROL	5	Debug and clock stop control
0x19	JTAG_CAPS	40	Capabilities readback
0x1A	SELECT_IJTAG	Variable	ijTAG network selection
0x3D	ZERO_LENGTH_BYPASS	0	
0x3E	INV_BYPASS		
0x3F	BYPASS	1	IEEE 1149.1, Section 8.4

NOTE: All instruction encodings *not* explicitly defined in this table map to the BYPASS instruction.

Signal Interface

Signal	Direction	Width	Description
tck_i	Input	1	JTAG test clock (from client_tap_ctrl_i.tck)
trst_ni	Input	1	JTAG test reset (from client_tap_ctrl_i.trst_n)
tdi_i	Input	1	Test Data In
tdo_o	Output	1	Test Data Out
tms_i	Input	1	Test Mode Select
tdo_en_o	Output	1	TDO output enable
ir_o	Output	6	Current instruction register value
tap_state_o	Output	4	Current TAP controller state
capture_dr_o	Output	1	Capture-DR state indicator
shift_dr_o	Output	1	Shift-DR state indicator
update_dr_o	Output	1	Update-DR state indicator
reset_n_o	Output	1	Combined reset output
ch_reset_n_o	Output	1	Channel reset (TMP-aware)
stap_tdi_o	Output	1	STAP chain TDI output
stap_tdo_i	Input	1	STAP chain TDO input
stap_capture_o	Output	1	STAP capture control
stap_shift_o	Output	1	STAP shift control
stap_update_o	Output	1	STAP update control
ic_reset_ovrd_o	Output	N	IC_RESET override enables
ic_reset_ctrl_n_o	Output	N	IC_RESET control values
ijtag_tdi_o	Output	1	ijTAG network TDI
ijtag_tdo_i	Input	1	ijTAG network TDO
pwr_on_rst_ni	Input	1	Power-on reset (active low), AND-combined with TRST
clk_i	Input	1	System clock for AXI bridge modules
rst_n_i	Input	1	System reset for AXI bridge modules (active low)
feat_ctrl_i	Input	64	Lifecycle feature control vector, enable-polarity (1 = feature enabled); security gating for debug interfaces

Dependencies

The PTAP depends on the following components:

- [hw/common/prim](#) - Primitive cells (synchronizers, clock gating)
- [hw/ip/jtag_stap](#) - Secondary TAP (when 3DIC support enabled)

6.4. JTAG STAP

6.4.1. Overview

The Secondary TAP (STAP) enables hierarchical interconnection of TAP controllers across multiple chiplets as specified in IEEE 1838-2019 (3DIC). Each STAP interface provides an extension of a SiP's primary JTAG interface across internal or external chiplets.

The DTP supports the following STAP types:

- **I/O STAP:** Extends JTAG to the next chiplet in the SiP
- **SMC Debug STAP:** Provides access to the SMC CPU core debugger TAP
- **SEP Debug STAP:** Provides access to the SEP CPU core debugger TAP
- **Additional STAPs:** Provides access to additional TAPs for on or off-chip connectivity
- **Extended STAP Scan Interface:** Port for adding STAPs external to the DTP

6.4.2. Configuration Parameters

The STAP provides three timing-related configuration parameters to optimize performance across different physical topologies.

Parameter	Type	Default	Description
SCAN_IN_PIPE	bit	0	Adds pipeline stage to client interface scan input for timing closure
TDI_LOCKUP	bit	0	Adds lockup latch to STAP TDI input for die boundary crossings
SCAN_OUT_LOCKUP	bit	0	Adds lockup latch to STAP scan output for extended chain timing

6.4.3. Architecture

IEEE 1838-2019 enables hierarchical interconnection of TAP controllers across multiple chiplets. Each STAP interface provides an extension of a SiP's primary JTAG interface across internal chiplets. When enabled, a STAP interface is part of the JTAG scan chain, otherwise the STAP interface is bypassed. In addition, STAPs can also support connection of other TAP interfaces included on the same chiplet as the PTAP.

Module Hierarchy

The STAP implementation consists of three main components integrated within a single module:

Module	Purpose	Conditional
<code>prim_jtag_sib_mux_pre</code>	Segment Insertion Bit multiplexer for TAP control signal routing	Always
<code>prim_jtag_scan_reg</code>	3-bit 3DCR control register (<code>config_hold</code> , <code>stap_sel</code> , <code>tms_hold</code>)	Always
Lockup latches	TDO lockup (always), TDI lockup (optional), scan output lockup (optional)	Per parameter

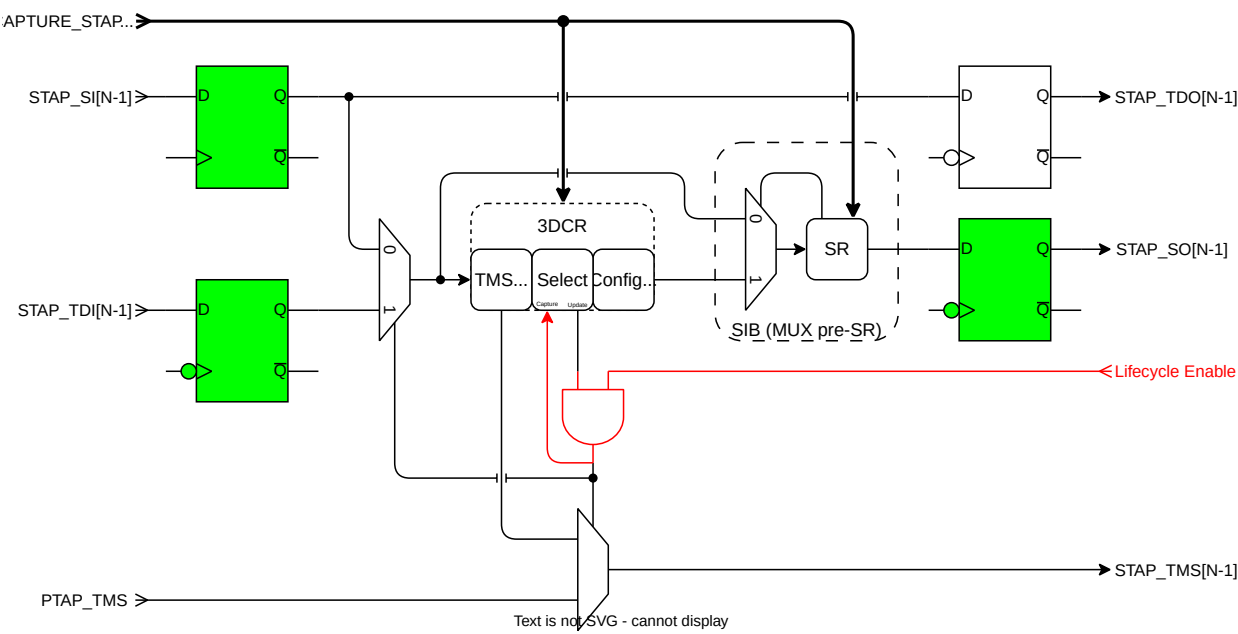
STAP Module

Each STAP has a 3-bit 3DCR TDR that defines the control interface for the STAP. Note that the STAP 3DCRs take a distributed segment approach to improve timing, as opposed to the single monolithic PTAP 3DCR approach detailed in the IEEE 1838 standard. This 3DCR is connected inline within each STAP's scan chain. A SIB is inserted at the end of the

3DCR to exclude it from the scan chain when not needed.

When not selected, the STAP interface TMS signal is tied to the constant value specified by the TMS-Hold bit of the 3DCR, otherwise it follows the PTAP TMS signal.

The following diagram illustrates the STAP architecture and the details of the 3DCR register. The green colored flops are optional, each configured via parameters. The red AND gate and nets implement security access lockout when the chiplet is not in the correct lifecycle state.



3DCR Register

The 3-bit 3DCR control register is implemented using `prim_jtag_scan_reg` with a loopback configuration. The register outputs feed back to the data inputs, allowing the register to retain its configuration across scan operations while still being scannable for updates.

Bit	Field	Description	Reset
2	TMS-Hold	Defines the parked TMS value when the STAP is disabled. When <code>stap_sel=0</code> , the host TMS output is driven with this static value rather than following the client TMS input.	0
1	Select (<code>stap_sel</code>)	Enables the STAP scan interface. When <code>stap_sel=1</code> , the STAP routes scan data through the downstream TAP. When <code>stap_sel=0</code> , the STAP acts as a simple 3-bit TDR and the downstream TAP's TMS is held at the TMS-Hold value.	0
0	Config-Hold	Prevents reset of the 3DCR register when the TAP enters Test-Logic-Reset state. The register is reset by <code>trst_n AND (config_hold OR scan_reset)</code> , protecting configuration when <code>config_hold=1</code> . TRST assertion always resets the register regardless of this bit.	0

STAP Types

The following table lists the STAPs allocated for the DTP JTAG interface. The STAPs are serially chained TDI → TDO according to their order in this table from top to bottom. In addition to the STAPs integrated into the DTP, the STAP scan chain is ported out of the DTP so that the chiplet integrator may add additional STAPs closer to their point of use.

Purpose	Description
STAP to Next Chiplet	This STAP forwards JTAG to the next chiplet in the SiP.

Purpose	Description
SMC Core Debug	Provides access to the internal SMC CPU core debugger TAP.
SEP Core Debug	Provides access to the internal SEP CPU core debugger TAP.
[N-1:0] Additional STAPs	Provides access to additional STAPs for on or off-chip TAP connectivity.
Extended STAP Scan Interface	Provides the ability to add STAPs external to the DTP closer to their point of use.

Security Lockout

The security lockout feature prevents unauthorized access to debug TAPs based on chiplet lifecycle state. The `security_disable_i` input controls three aspects of STAP operation:

Signal	Security Lockout Behavior
<code>stap_sel</code> (internal)	When <code>security_disable_i=0</code> (debug disabled), <code>stap_sel</code> is forced to 0 regardless of the 3DCR Select bit value. This prevents routing scan data through the downstream TAP.
<code>host_tdo_o</code>	When <code>security_disable_i=0</code> , <code>host_tdo_o</code> is forced to 0. This prevents any data from leaking out of the downstream TAP even if the TAP is inadvertently accessed.
<code>client_reg_scan_ctrl.update_en</code>	When <code>security_disable_i=0</code> , the 3DCR register update enable signal is forced to 0. This prevents modification of the STAP configuration when lifecycle state restricts debug access, ensuring the configuration cannot be altered to bypass security controls.

This triple gating approach ensures that STAP selection, TDO output, and configuration updates are all blocked when the chiplet lifecycle state restricts debug access. The gating logic is implemented combinationaly at the module output, preventing any timing-dependent bypass paths.

Lockup Latches

The STAP implements three types of lockup latches to manage timing across die boundaries and long scan chains. These latches are clocked on the falling edge of TCK to capture data during the second half of the clock cycle, providing setup time margin for the next rising edge.

TDO Lockup Latch

The TDO lockup latch is always present and cannot be disabled. This latch captures the scan data propagating from client to host on the falling edge of TCK. The latch ensures that data from the downstream TAP's TDO has sufficient setup time before being sampled by the upstream TAP on the rising edge of TCK.

TDI Lockup Latch

The TDI lockup latch is optional and controlled by the `TDI_LOCKUP` parameter (default = 0). When enabled, this latch captures the host TDI input on the falling edge of TCK before routing it to the client TDI output. This adds one additional cycle of latency but provides timing margin for die boundary crossings where signal delays may be significant. In the DTP integration, TDI lockup is enabled only for the I/O STAP that crosses between chiplets.

Scan Output Lockup Latch

The scan output lockup latch is optional and controlled by the `SCAN_OUT_LOCKUP` parameter (default = 0). This latch is implemented within the `prim_jtag_sib_mux_pre` module and adds a register stage to the client scan output path. This is typically enabled only on the final STAP in an extended chain to provide additional output timing margin.

6.4.4. Interface

Signal Interface

Signal	Direction	Width	Description
clk_i	Input	1	JTAG TCK clock
rst_ni	Input	1	Active-low asynchronous reset (TRST or POR)
tdi_i	Input	1	Test Data In from upstream (PTAP or previous STAP)
tdo_o	Output	1	Test Data Out to downstream (next STAP or PTAP)
capture_i	Input	1	Capture control from PTAP
shift_i	Input	1	Shift control from PTAP
update_i	Input	1	Update control from PTAP
tms_i	Input	1	TMS from PTAP
security_disable_i	Input	1	Security lockout override (lifecycle control)
stap_tdi_o	Output	1	TDI to downstream TAP
stap_tdo_i	Input	1	TDO from downstream TAP
stap_tms_o	Output	1	TMS to downstream TAP (or parked value when disabled)
stap_tck_o	Output	1	TCK to downstream TAP
stap_trst_no	Output	1	TRST to downstream TAP
stap_enabled_o	Output	1	Indicates STAP is selected and enabled

Timing Requirements

The STAP interface timing follows IEEE 1838-2019:

- TDI is sampled on the rising edge of TCK
- TDO is driven on the falling edge of TCK
- TMS is sampled on the rising edge of TCK
- The lockup latch captures data on the falling edge of TCK when transitioning out of Shift-DR/IR states

Dependencies

The STAP depends on the following components:

- `hw/common/pr im` - Primitive cells (clock gating, synchronizers)

6.5. Cross Trigger Network

6.5.1. Overview

OCH supports generating and sinking cross triggers between chiplets in a SiP as specified in the Open Chiplet Cross Triggering (OCCT) specification. A Cross Trigger Port (CTP) module serves as the signaling interface between the chiplet core logic and the I/O pads, while a Cross Trigger Matrix (CTM) module statically routes cross triggers between CTPs and the on-chip resources generating and sinking cross trigger pulses.

The CTPs support two different inter-chiplet cross trigger signalling schemes, wire-OR and point-to-point. Wire-OR supports a wider range of existing chiplet implementations and is very resource efficient, while point-to-point is faster, more reliable, and simpler to configure.

6.5.2. Purpose

The primary purposes of the CTN are:

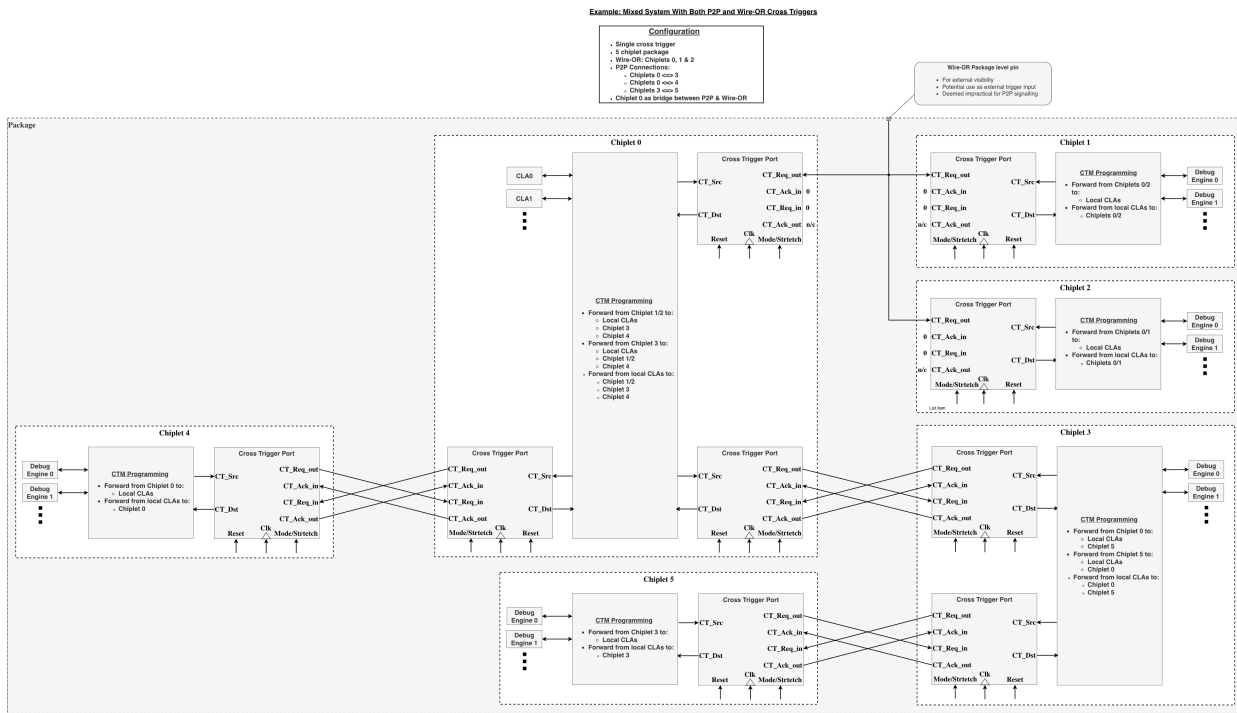
- 1. **Die-to-Die Cross Triggering:** Enable debug triggers to propagate between chiplets in a multi-die package
- 2. **On-Chip Cross Triggering:** Route cross trigger signals between internal debug modules (e.g., CLAs)
- 3. **Unified Management:** Provide a single AXI-Lite interface for configuring all cross trigger functionality
- 4. **Clock Stop Coordination:** Aggregate clock stop requests from multiple sources

6.5.3. Configuration Parameters

Parameter	Range	Default	Description
NUM_CTP	1-32	16	Number of external Cross Trigger Ports
NUM_INT_CT	0-32	10	Number of internal cross triggers
NUM_CLK_STOP_REQ	1-32	9	Number of clock stop request inputs
INT_CT_MODE	bit vector	'0	Mode per internal CTP (0=pulse, 1=handshake)

6.5.4. Architecture

The following diagram is an example of a SiP with a heterogeneous cross triggering scheme utilizing both wire-OR and point-to-point cross triggering interfaces:



Cross Trigger Port (CTP)

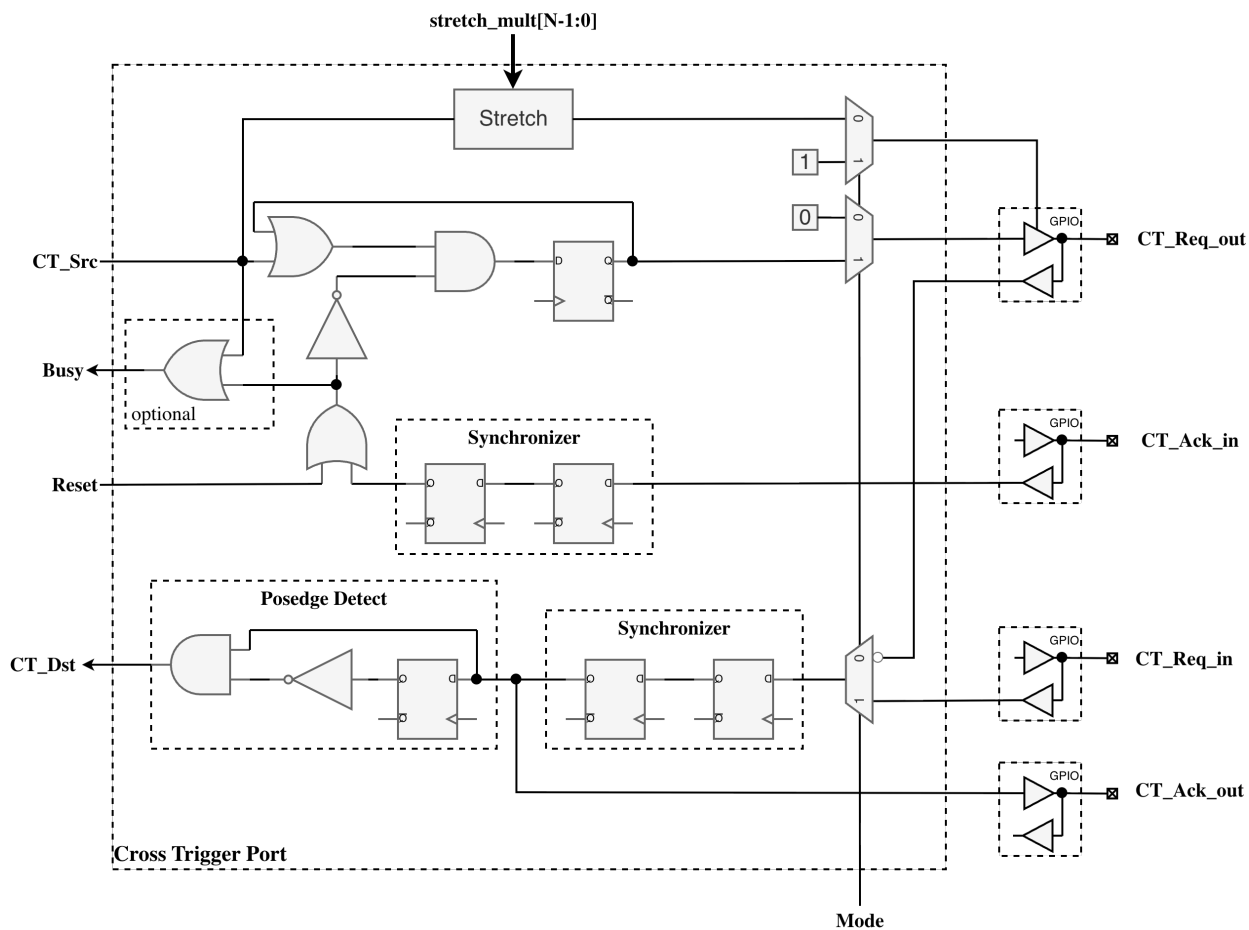
OCH supports a configurable number of CTPs, from 0 to 8, for a total number of up to 32 required GPIOs. These ports support both point-to-point and wire-OR cross trigger signaling modes.

Each CTP connects to four separate I/O pads:

- **CT_Req_out** - The cross trigger request output signal. The CTP asserts this signal to transmit a cross trigger in both point-to-point and wire-OR modes. In point-to-point mode the request is held high until the receiver acknowledges the request. In wire-OR mode, this signal drives the output-enable port of the I/O pad for duration defined by a pulse stretching module, and the pad output is driven static-low. Also in wire-OR mode the I/O pad input signal must be inverted, as the pad signal itself is an open-drain active-low signal.
- **CT_Ack_in** - The cross trigger acknowledge input signal. Exclusive to point-to-point mode. The cross trigger receiver asserts this signal when it acknowledges the CTP's cross trigger request.
- **CT_Req_in** - The cross trigger request input signal. Exclusive to point-to-point mode. This is the external cross trigger request received by the CTP.
- **CT_Ack_out** - The cross trigger acknowledge output signal. Exclusive to point-to-point mode. The CTP asserts this signal after receiving a cross trigger request, and de-asserts this signal after the cross trigger request is released.

When a GPIO pad is configured for cross triggering, the I/O pad assumes the role of one of these four cross trigger signals, and the pad controls and I/O data signals are connected to the associated CTP.

The following diagram and table illustrate the logical behavior and signal interface of a CTP module:



Name	Direction	Description
mode	input	Indicates the operational mode of the CTP. 0 for wire-OR and 1 for peer-to-peer. The value at reset should be 0.
ct_src	input	Core-side cross trigger input pulse.
ct_dst	output	Core-side cross trigger output pulse.
busy	output	Optional signal indicating that an outgoing cross trigger pulse transfer is in progress.
reset	input	Resets the outgoing peer-to-peer handshake logic.
stretch_mult[N-1:0]	input	Configures the pulse duration of the pulse stretcher module for wire-OR mode.
ct_req_out_dout_en	output	Enables the output driver for the <i>CT_Req_out</i> pad. In wire-OR mode this signal is enabled by a cross trigger pulse from the pulse stretcher. In peer-to-peer mode it is always enabled.
ct_req_out_din_en	output	Enables the input buffer for the <i>CT_Req_out</i> pad. In wire-OR mode this signal is enabled and in peer-to-peer mode it is disabled.
ct_req_out_dout	output	The output data to drive on the <i>CT_Req_out</i> pad. In peer-to-peer mode the pulse capture latch controls this signal. In wire-OR mode it is held static-low.
ct_req_out_din	input	The input data that is driven on the <i>CT_Req_out</i> pad. In wire-OR mode this signal triggers a pulse on <i>ct_dst</i> after synchronization. In peer-to-peer mode it is unused.

Name	Direction	Description
ct_req_in_din_en	output	Enables the input buffer for the <i>CT_Req_in</i> pad. In peer-to-peer mode this signal is enabled and in wire-OR mode it is disabled.
ct_req_in_din	input	The input data that is driven on the <i>CT_Req_in</i> pad. In peer-to-peer mode this signal triggers a pulse on <i>ct_dst</i> after synchronization. In wire-OR mode it is unused.
ct_ack_in_din_en	output	Enables the input buffer for the <i>CT_Ack_in</i> pad. In peer-to-peer mode this signal is enabled and in wire-OR mode it is disabled.
ct_ack_in_din	input	The input data that is driven on the <i>CT_Ack_in</i> pad. This signal clears the cross trigger request output in peer-to-peer mode. In wire-OR mode it is unused.
ct_ack_out_dout_en	output	Enables the output driver for the <i>CT_Ack_out</i> pad. In peer-to-peer mode this signal is enabled and in wire-OR mode it is disabled.
ct_ack_out_dout	output	The output data to drive on the <i>CT_Ack_out</i> pad. In peer-to-peer mode the <i>ct_req_in</i> pulse synchronizer controls this signal. In wire-OR mode it is unused.

The register interface for each CTP module is described in the [Memory Map](#) section below.

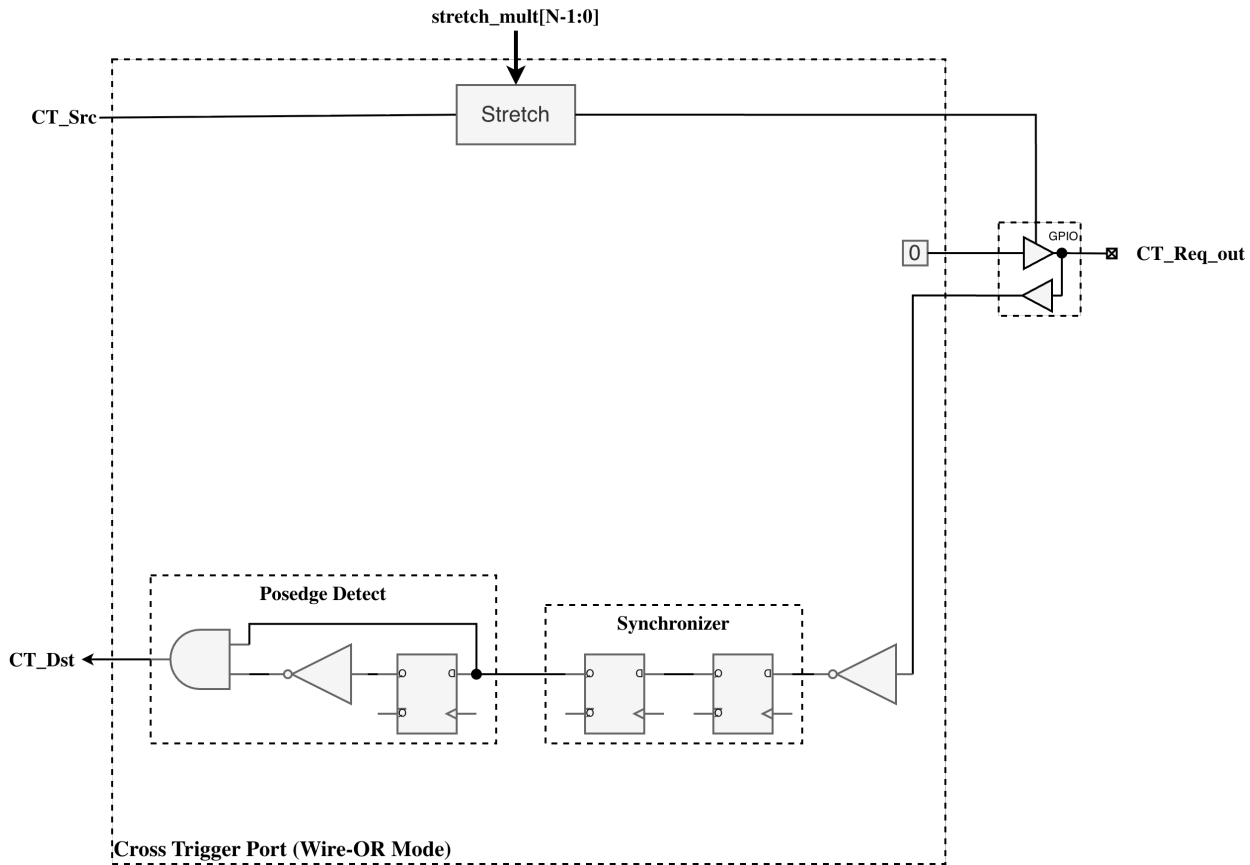
Wire-OR Mode

Wire-OR mode is a 1-wire cross trigger signaling scheme compatible with many existing cross triggering implementations. This scheme is very resource efficient, as it only requires a single pin per chiplet connected to the network. However, the performance of the network decreases as the number of connected chiplets increase. Also, the duration of the cross trigger pulse must be pre-arranged across all connected chiplets.

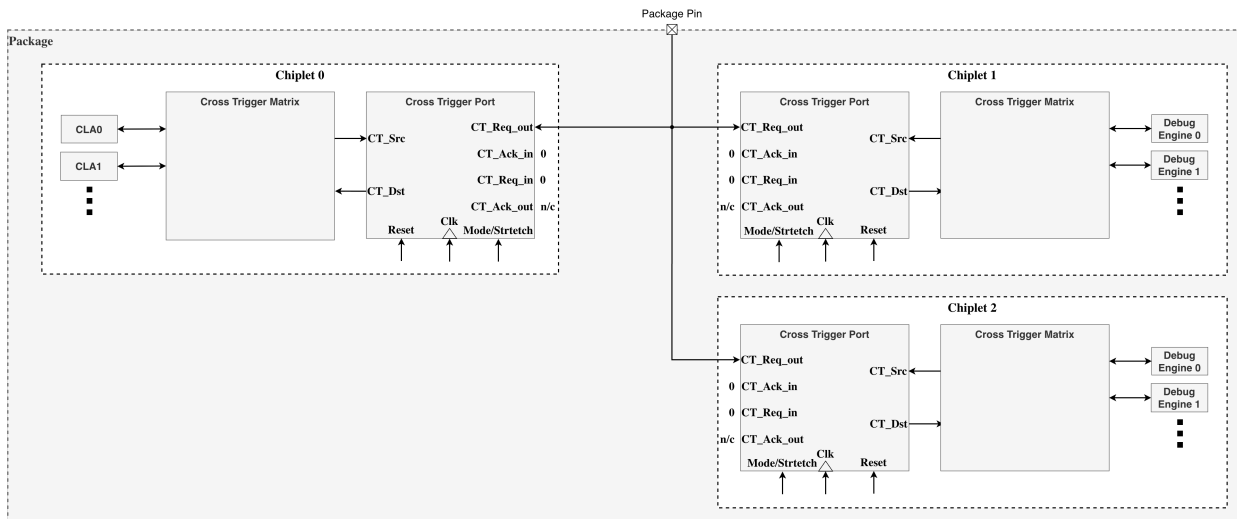
Each chiplet has an open-drain I/O connected to a single shared wire. Any connected chiplet may pull this signal low to assert a cross trigger. Depending on the electrical requirements, one or more internal pad pull-ups or external keepers ensure the cross trigger request signal remains high when no cross trigger is asserted. To ensure no connected chiplet misses a cross trigger pulse, all chiplets transmitting a trigger pulse must stretch the pulse a mutually agreed upon length of time.

Note that in wire-OR mode the *CT_Req_out* signal is connected to an open-drain configured I/O pad. For I/O pads that do have explicit open-drain control, the request data signal should connect to the output-enable control pin of the I/O pad, and the output data signal of the I/O pin should be driven statically low.

The following diagram illustrates the logical behavior of the CTPs when configured in wire-OR mode:



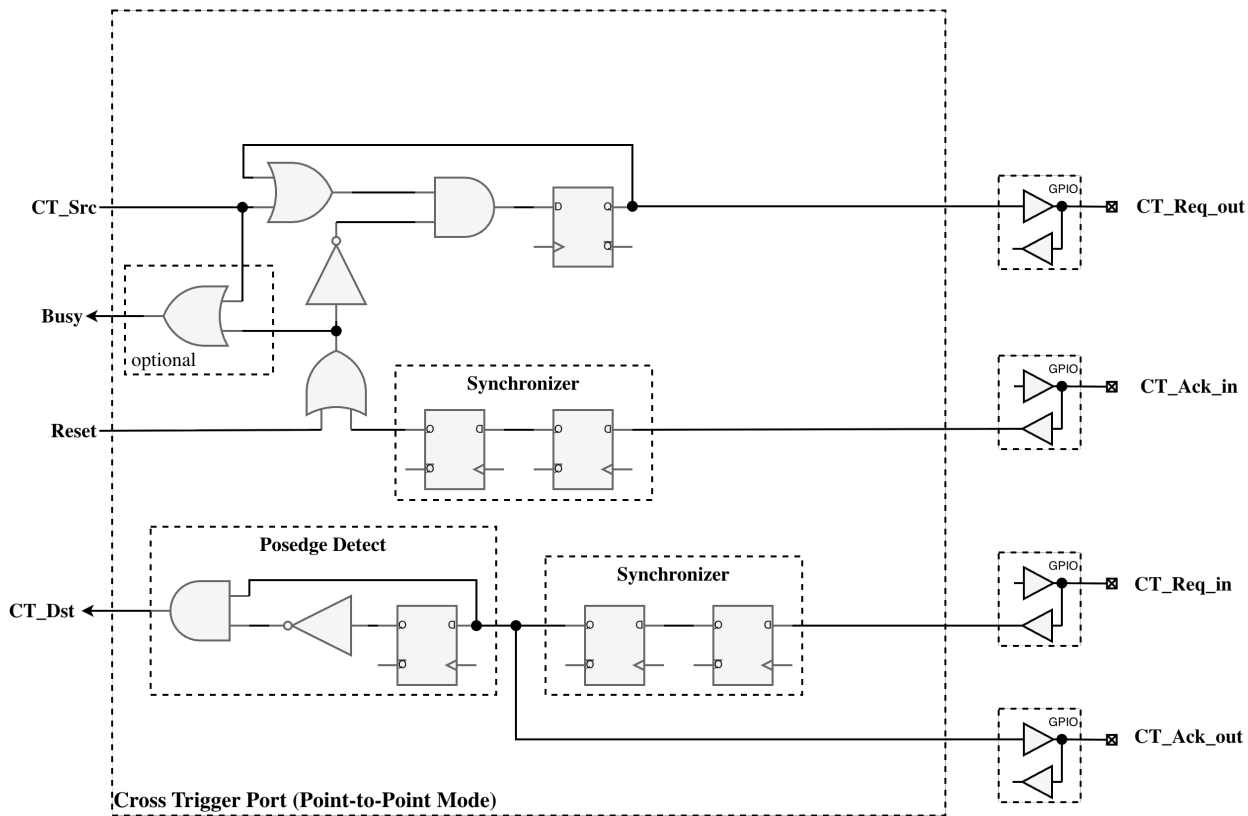
Any number of chiplets may have their *CT_Req_out* pads interconnected as shown in the following example diagram:



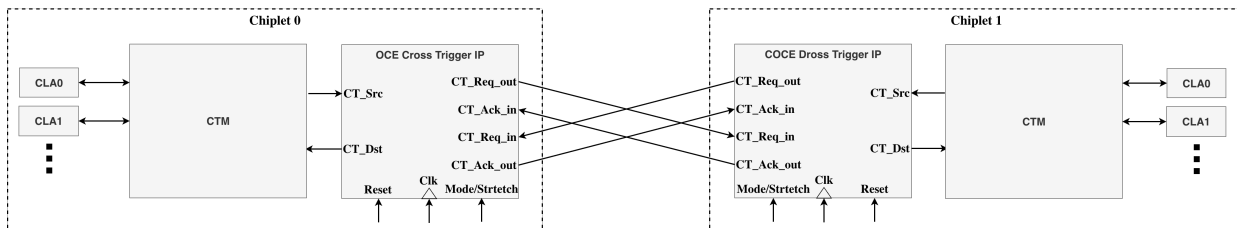
Point-to-Point Mode

Point-to-point mode utilizes a four-phase handshaking scheme to synchronize a cross trigger quickly and reliably between two chiplets without pulse stretching. While this scheme is simple and fast, it requires a dedicated CTP for each chiplet-to-chiplet interconnection, and each CTP requires four I/O pads for bi-directional communication, or two pads for unidirectional communication. All output pads are configured for push-pull operation.

The following diagram illustrates the logical behavior of the CTPs when configured in point-to-point mode:



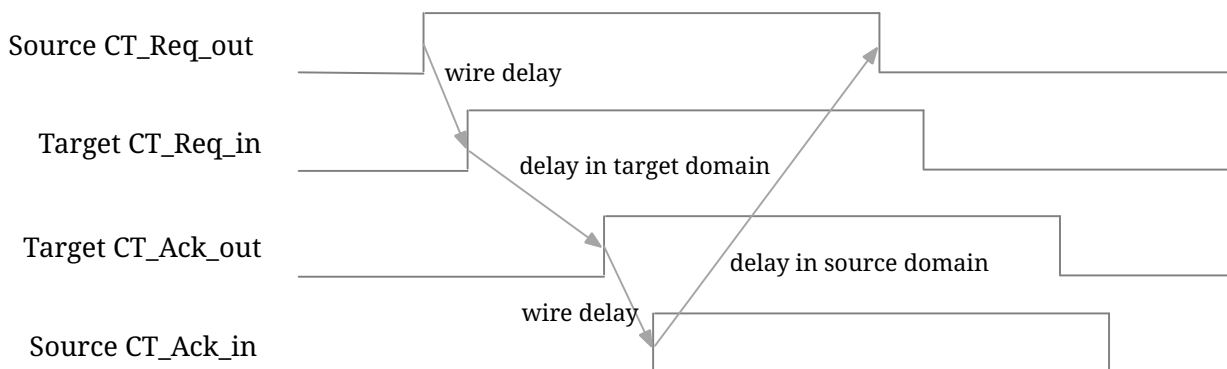
Any two chiplets may have their four cross trigger pads cross-coupled as shown in the following example diagram:



On the sender side, the receiver's **CT_Ack_out** is connected to the sender's **CT_Ack_in** pad. **CT_Req_out** asserts whenever **CT_Src** asserts. **CT_Req_out** will then remain asserted until **CT_Ack_in** asserts.

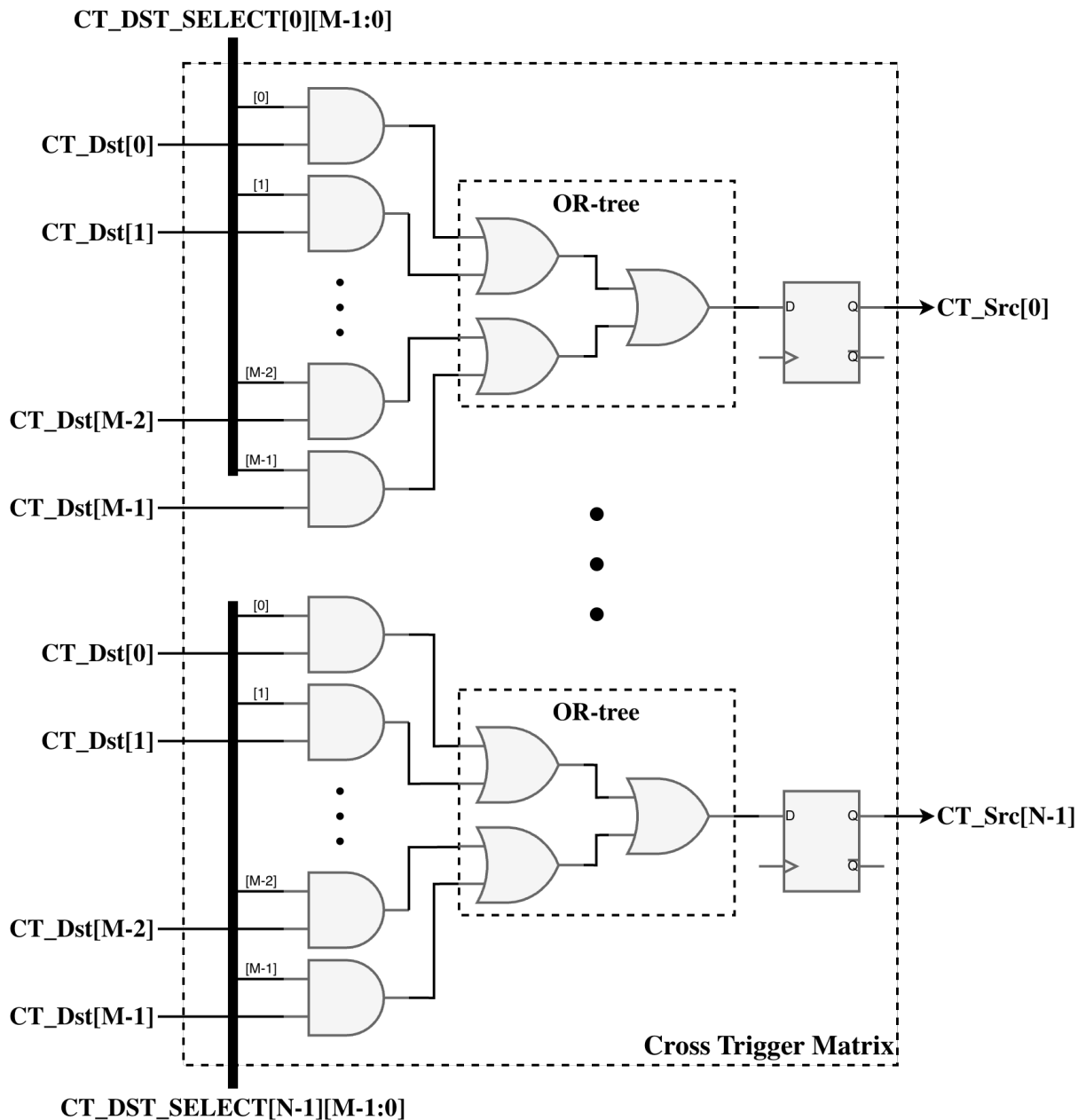
On the receiver side, the sender's **CT_Req_out** is connected to the receiver's **CT_Req_in** pad. On any positive edge of **CT_Req_in**, the receiver generates a pulse on **CT_Dst** and asserts **CT_Ack_out**. Deassertion proceeds in exactly the same sequence.

The timing of the request/acknowledge handshake is shown in the diagram below:



Cross Trigger Matrix (CTM)

The CTM is a crossbar that aggregates and statically routes cross trigger pulses between various sources and sinks. Typical connections to the CTM include Core Logic Analyzers (CLAs) and CTPs, including other miscellaneous devices. The CTM is compatible with CTPs operating in either of the two signaling modes, point-to-point or wire-OR. Typically, each device or logic module connected to the CTM will act as both a source and a sink for cross trigger pulses.



The CTM is programmed at runtime via firmware or debug transactions. Proper configuration of the CTMs in each chiplet will minimize trigger latency and prevent loops in the SiP cross trigger network. Configuration is accomplished via CSRs. The CTM register interface is described in the [Memory Map](#) section below.

Address Map

The CTN provides a unified CSR address space accessed via a single AXI-Lite subordinate port. An internal AXI-Lite crossbar (**axi_lite_xbar**) routes transactions to the CTM and external CTPs based on address decoding.

Component	Address Range	Size	Description
CTM	0x0000 - 0x01FF	512 bytes	Cross Trigger Matrix configuration registers (26 sources × 2 config registers)
CTP[0]	0x0200 - 0x020F	16 bytes	External CTP 0 configuration
CTP[1]	0x0210 - 0x021F	16 bytes	External CTP 1 configuration
...	...	...	...
CTP[15]	0x02F0 - 0x02FF	16 bytes	External CTP 15 configuration
Internal CTPs	No CSR address	N/A	Configured via build-time parameters only

The crossbar is configured with `MaxMstTrans=1` and `MaxSlvTrans=1`, serializing all CSR accesses. This simplification reduces arbitration logic at the cost of concurrent access capability.

Internal vs External CTP Architecture

The CTN instantiates 16 external CTPs using the full `cross_trigger_port` module with CSR interface, and 10 internal CTPs using only the `cross_trigger_port_core` module without registers.

Aspect	External CTP	Internal CTP
Module	<code>cross_trigger_port</code> (full)	<code>cross_trigger_port_core</code> (CSR-less)
CSR Interface	16-byte AXI-Lite address space	None (build-time parameters)
GPIO Interface	4-pad interface to die I/O	Logical signals to internal CLAs
Configuration	Runtime via CSRs (mode, invert, stretch_mult, reset)	Build-time via <code>INT_CT_MODE</code> parameter
Index Range	<code>ctm_ct_src/dst[0:15]</code>	<code>ctm_ct_src/dst[16:25]</code>
Signal Mapping	Standard GPIO pad controls	Repurposed GPIO signals (see table below)

Internal CTP Signal Remapping

Internal CTPs reuse the `cross_trigger_port_core` GPIO interface but connect to logical signals rather than physical pads. The `INT_CT_MODE` parameter bits are inverted before use: `MODE_WIRE_OR = ~INT_CT_MODE[i]`. This means `INT_CT_MODE[i]=0` selects Point-to-Point mode, while `INT_CT_MODE[i]=1` selects Wire-OR mode.

Wire-OR Mode (`INT_CT_MODE[i]=1`, internal `MODE_WIRE_OR=0`):

CTP Core Signal	Internal Connection
<code>ct_req_out_dout_en</code>	→ <code>ctm_src_req_o[i]</code> (stretched pulse to CLA)
<code>ct_req_out_din</code>	← <code>ctm_dst_req_i[i]</code> (stretched pulse from CLA)
All other signals	Unused (tied to constants)

Point-to-Point Mode (`INT_CT_MODE[i]=0`, internal `MODE_WIRE_OR=1`):

CTP Core Signal	Internal Connection
<code>ct_req_out_dout</code>	→ <code>ctm_src_req_o[i]</code> (level-based request to CLA)
<code>ct_req_in_din</code>	← <code>ctm_dst_req_i[i]</code> (level-based request from CLA)
<code>ct_ack_in_din</code>	← <code>ctm_src_ack_i[i]</code> (acknowledge from CLA)
<code>ct_ack_out_dout</code>	→ <code>ctm_dst_ack_o[i]</code> (acknowledge to CLA)

This remapping allows internal CTPs to operate in either mode without physical GPIO pads, enabling on-chip cross-triggering between CLAs and the external cross-trigger network.

Signal Flow

External Cross Trigger Flow

1. **Outgoing Trigger:** CTM ct_src → CTP ct_src_i → GPIO pads
2. **Incoming Trigger:** GPIO pads → CTP ct_dst_o → CTM ct_dst_i

Internal Cross Trigger Flow

1. **To Internal Sink:** CTM ct_src → Internal CTP → ctm_src_req_o
2. **From Internal Source:** ctm_dst_req_i → Internal CTP → CTM ct_dst

Clock Stop Control

The `ctn_clock_stop_ctrl` module aggregates clock stop requests from multiple sources using a two-stage OR-tree architecture. The module separates CLA-only status from the combined functional halt signal to support independent JTAG visibility and chiplet clock gating control.

Signal	Width	Description
<code>clk_stop_req_i</code>	NUM_CLK_STOP_REQ (default=9)	Clock stop requests from individual CLAs or other on-chip sources
<code>jtag_clock_stop_i</code>	1	Direct clock stop from JTAG DEBUG_CONTROL register
<code>cla_clock_stop_o</code>	1	CLA requests only (OR-reduction of <code>clk_stop_req_i</code>), fed back to JTAG for DEBUG_CONTROL status readback
<code>stop_clks_o</code>	1	Registered functional halt (JTAG OR CLA requests), drives chiplet clock gating

The aggregation logic operates as follows:

1. **CLA OR-Tree:** All CLA request inputs are OR-reduced combinationally: `cla_clk_stop_req = |clk_stop_req_i`
2. **Combined Request:** JTAG and CLA requests are OR-combined: `clk_stop_req_combined = jtag_clock_stop_i | cla_clk_stop_req`
3. **Output Registration:** The combined request is registered on the rising edge of `clk_i`: `stop_clks_o <= clk_stop_req_combined`

The registered `stop_clks_o` output prevents glitches from propagating to the chiplet's functional clock gating logic. The combinatorial `cla_clock_stop_o` output provides immediate visibility of CLA activity to the JTAG interface, allowing debuggers to distinguish between JTAG-initiated and CLA-initiated clock stops.

Port Indexing

CTM ports are indexed as follows:

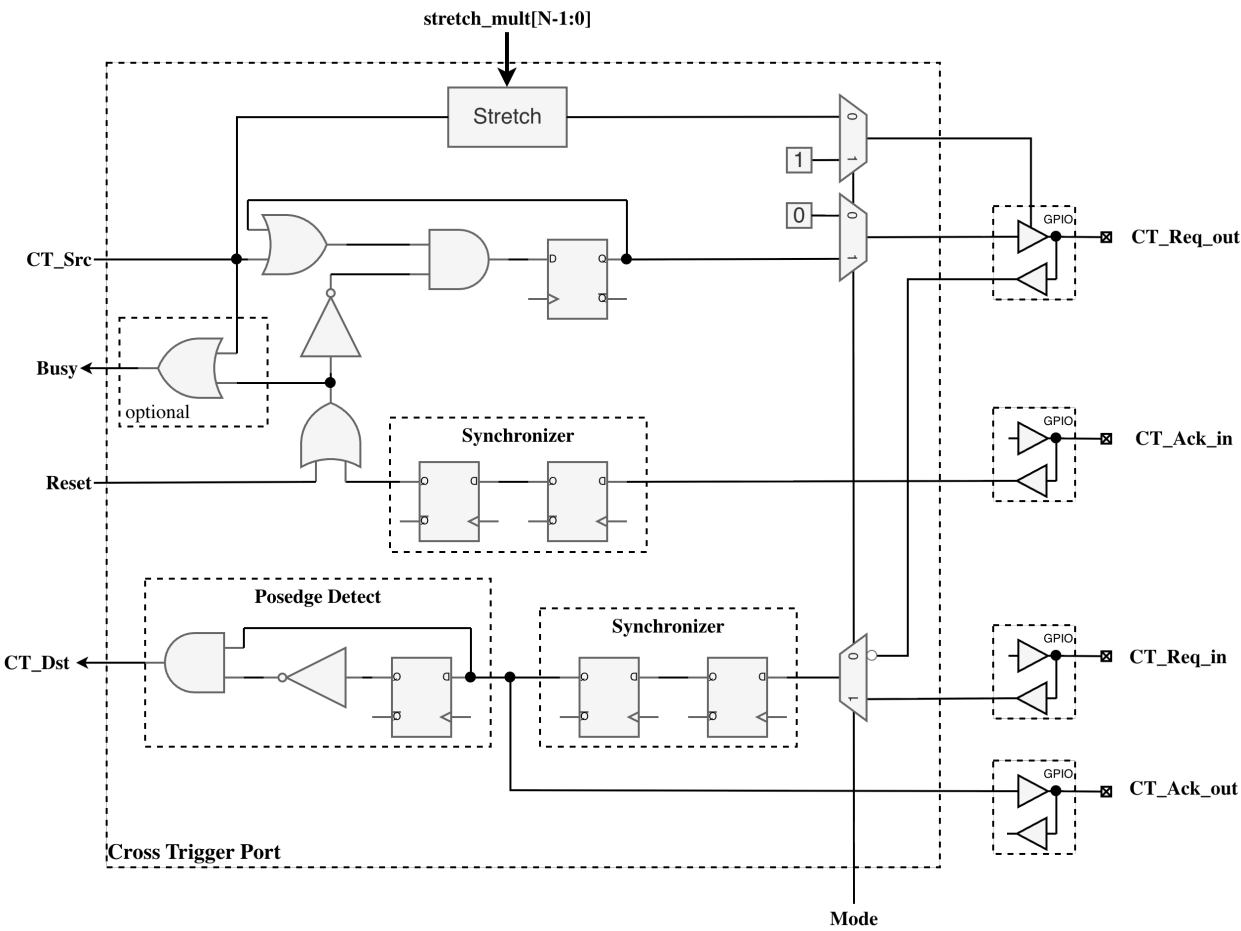
Port Index	Description
0 to N-1	External CTPs (NUM_CTP)
N to N+M-1	Internal CTPs (NUM_INT_CT)

Where N = NUM_CTP and M = NUM_INT_CT. This indexing is used for CTM ct_dst_i and ct_src_o port assignment and CTM routing configuration registers.

6.5.5. Interface

Cross Trigger Port (CTP)

The following diagram and table illustrate the logical behavior and signal interface of a CTP module:



Name	Direction	Description
mode	input	Indicates the operational mode of the CTP. 0 for wire-OR and 1 for peer-to-peer. The value at reset should be 0.
ct_src	input	Core-side cross trigger input pulse.
ct_dst	output	Core-side cross trigger output pulse.
busy	output	Optional signal indicating that an outgoing cross trigger pulse transfer is in progress.
reset	input	Resets the outgoing peer-to-peer handshake logic.
stretch_mult[N-1:0]	input	Configures the pulse duration of the pulse stretcher module for wire-OR mode.
ct_req_out_dout_en	output	Enables the output driver for the <i>CT_Req_out</i> pad. In wire-OR mode this signal is enabled by a cross trigger pulse from the pulse stretcher. In peer-to-peer mode it is always enabled.
ct_req_out_din_en	output	Enables the input buffer for the <i>CT_Req_out</i> pad. In wire-OR mode this signal is enabled and in peer-to-peer mode it is disabled.

Name	Direction	Description
ct_req_out_dout	output	The output data to drive on the <i>CT_Req_out</i> pad. In peer-to-peer mode the pulse capture latch controls this signal. In wire-OR mode it is held static-low.
ct_req_out_din	input	The input data that is driven on the <i>CT_Req_out</i> pad. In wire-OR mode this signal triggers a pulse on <i>ct_dst</i> after synchronization. In peer-to-peer mode it is unused.
ct_req_in_din_en	output	Enables the input buffer for the <i>CT_Req_in</i> pad. In peer-to-peer mode this signal is enabled and in wire-OR mode it is disabled.
ct_req_in_din	input	The input data that is driven on the <i>CT_Req_in</i> pad. In peer-to-peer mode this signal triggers a pulse on <i>ct_dst</i> after synchronization. In wire-OR mode it is unused.
ct_ack_in_din_en	output	Enables the input buffer for the <i>CT_Ack_in</i> pad. In peer-to-peer mode this signal is enabled and in wire-OR mode it is disabled.
ct_ack_in_din	input	The input data that is driven on the <i>CT_Ack_in</i> pad. This signal clears the cross trigger request output in peer-to-peer mode. In wire-OR mode it is unused.
ct_ack_out_dout_en	output	Enables the output driver for the <i>CT_Ack_out</i> pad. In peer-to-peer mode this signal is enabled and in wire-OR mode it is disabled.
ct_ack_out_dout	output	The output data to drive on the <i>CT_Ack_out</i> pad. In peer-to-peer mode the <i>ct_req_in</i> pulse synchronizer controls this signal. In wire-OR mode it is unused.

Cross Trigger Matrix (CTM)

The CTM has the following data ports and logical structure:

- **CT_Src[N-1:0]** - Cross trigger pulse output signals from the CTM to CLAs, CTPs, or other cross trigger sinking devices, where N is the number of these cross trigger sinks.
- **CT_Dst[M-1:0]** - Cross trigger pulse input signals to the CTM from CLAs, CTPs, or other cross trigger generating devices, where M is the number of these cross trigger sources.

Cross Trigger Network Module Interface

The top-level `cross_trigger_network` module has the following interface:

Parameters

- **INT_CT_MODE** - Per-bit mode selection for internal CTPs (0 = Wire-OR, 1 = P2P)
- **axil_req_t, axil_resp_t** - AXI-Lite request/response types

Ports

```
// Global
input logic clk_i
```



```

input  logic rst_ni

// AXI-Lite CSR Interface
input  axil_req_t  axil_req_i
output axil_resp_t axil_resp_o

// Clock Stop Interface
input  logic [NUM_CLK_STOP_REQ-1:0] clk_stop_req_i
input  logic jtag_clock_stop_i
output logic stop_clks_o
output logic cla_clock_stop_o

// Internal Cross Trigger Interface (to CLAs)
output logic [NUM_INT_CT-1:0] ctm_src_req_o
input  logic [NUM_INT_CT-1:0] ctm_src_ack_i
input  logic [NUM_INT_CT-1:0] ctm_dst_req_i
output logic [NUM_INT_CT-1:0] ctm_dst_ack_o

// External CTP GPIO Interface (per CTP)
// CT_Req_out, CT_Req_in, CT_Ack_in, CT_Ack_out signals

```

Dependencies

The CTN depends on the following components:

- `hw/ip/cross_trigger_port` - Cross Trigger Port IP
- `hw/ip/cross_trigger_matrix` - Cross Trigger Matrix IP
- `deps/axi` - AXI infrastructure (`axi_lite_xbar`)

6.5.6. Memory Map

The Cross Trigger Network combines the Cross Trigger Matrix (CTM) and Cross Trigger Port (CTP) registers into a unified address space:

- 0x0000 - 0x01FF: Cross Trigger Matrix (CTM) - 512 bytes
- 0x0200 - 0x02FF: Cross Trigger Ports (CTP[0-15]) - 16 bytes each

6.6. Cross Trigger Port

6.6.1. Overview

The Cross Trigger Port (CTP) module serves as the signaling interface between the chiplet core logic and the I/O pads for cross-trigger communication as specified in the Open Chiplet Cross Triggering (OCCT) specification.

Each CTP connects to four separate I/O pads:

- **CT_Req_out** - The cross trigger request output signal

- **CT_Ack_in** - The cross trigger acknowledge input signal (point-to-point only)
- **CT_Req_in** - The cross trigger request input signal (point-to-point only)
- **CT_Ack_out** - The cross trigger acknowledge output signal (point-to-point only)

The CTPs support two different inter-chiplet cross trigger signalling schemes:

- **Wire-OR**: Supports a wider range of existing chiplet implementations and is very resource efficient
- **Point-to-Point**: Faster, more reliable, and simpler to configure

6.6.2. Configuration

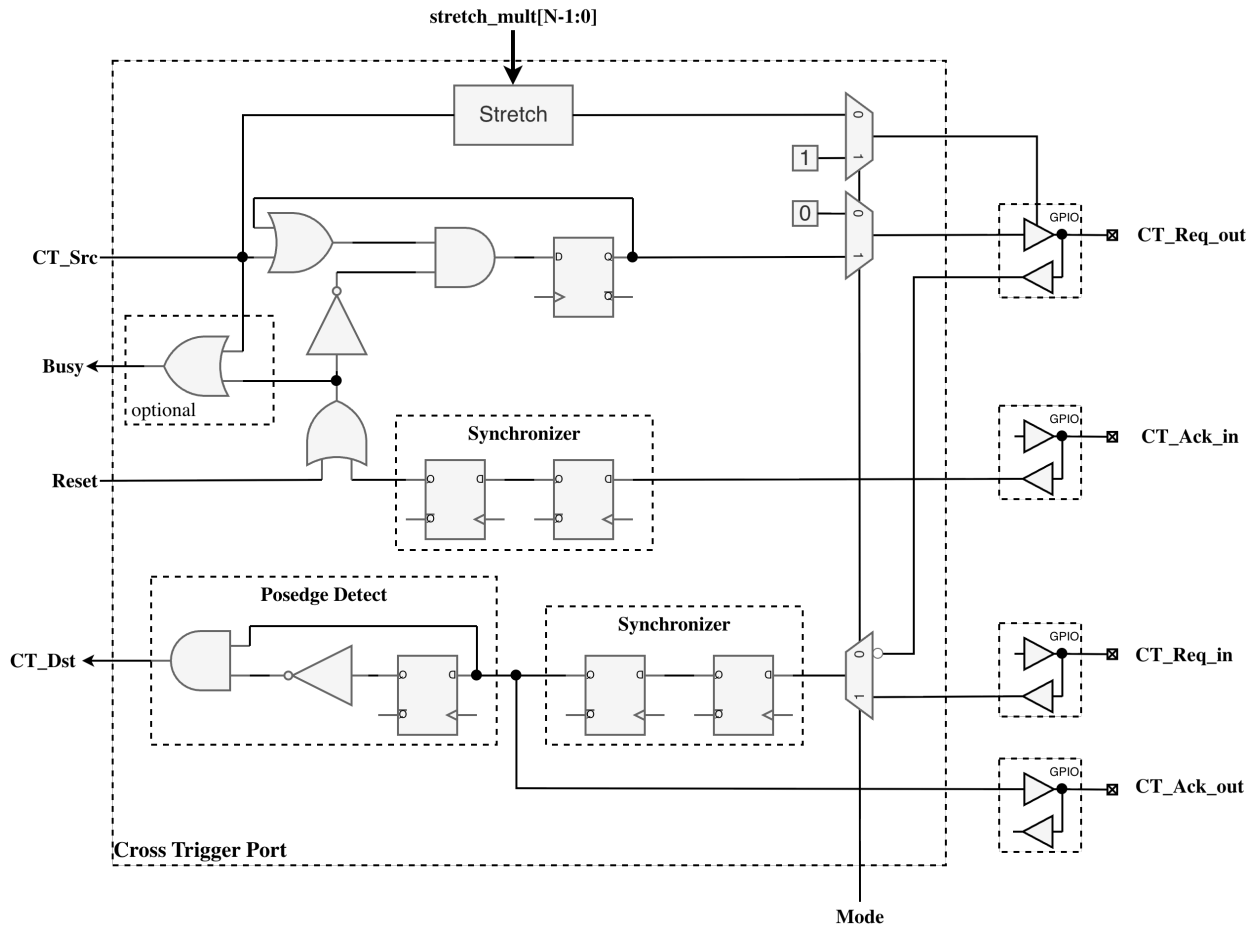
The Cross Trigger Port has no build-time configuration parameters. All operational settings are configured at runtime via CSR registers accessed through the AXI-Lite interface:

- **MODE** - Wire-OR (0) or Point-to-Point (1) mode selection
- **STRETCH_MULT** - 16-bit pulse stretch multiplier for Wire-OR mode
- **INVERT** - Signal polarity inversion for GPIO adaptation
- **RESET** - Handshake state machine reset control

Refer to the register documentation for detailed CSR field descriptions.

6.6.3. Architecture

The Cross Trigger Port architecture separates the register interface from core logic to enable reuse. The full `cross_trigger_port` module includes CSRs for runtime configuration, while the `cross_trigger_port_core` module provides the core functionality without registers, allowing instantiation for internal cross-trigger connections without AXI-Lite overhead.



Submodule Hierarchy

The CTP core instantiates four specialized submodules for different aspects of cross-trigger operation:

Module	Purpose
<code>ctp_synchronizer</code>	Implements 2-FF synchronizers for asynchronous GPIO inputs (<code>ct_req_out_din</code> , <code>ct_req_in_din</code> , <code>ct_ack_in_din</code>) from remote chiplets
<code>ctp_pulse_stretcher</code>	Extends source pulse duration for Wire-OR mode transmission, with configurable stretch multiplier (0-65535 cycles)
<code>ctp_edge_detector</code>	Detects positive edges on synchronized <code>ct_req_in</code> signal for Point-to-Point mode receiver operation
<code>ctp_handshake_ctrl</code>	Implements sender and receiver state machines for Point-to-Point mode four-phase handshaking protocol

Wire-OR Mode

Wire-OR mode is a 1-wire cross trigger signaling scheme compatible with many existing cross triggering implementations. This scheme is very resource efficient, as it only requires a single pin per chiplet connected to the network. However, the performance of the network decreases as the number of connected chiplets increase. Also, the duration of the cross trigger pulse must be pre-arranged across all connected chiplets.

Pulse Stretcher Operation

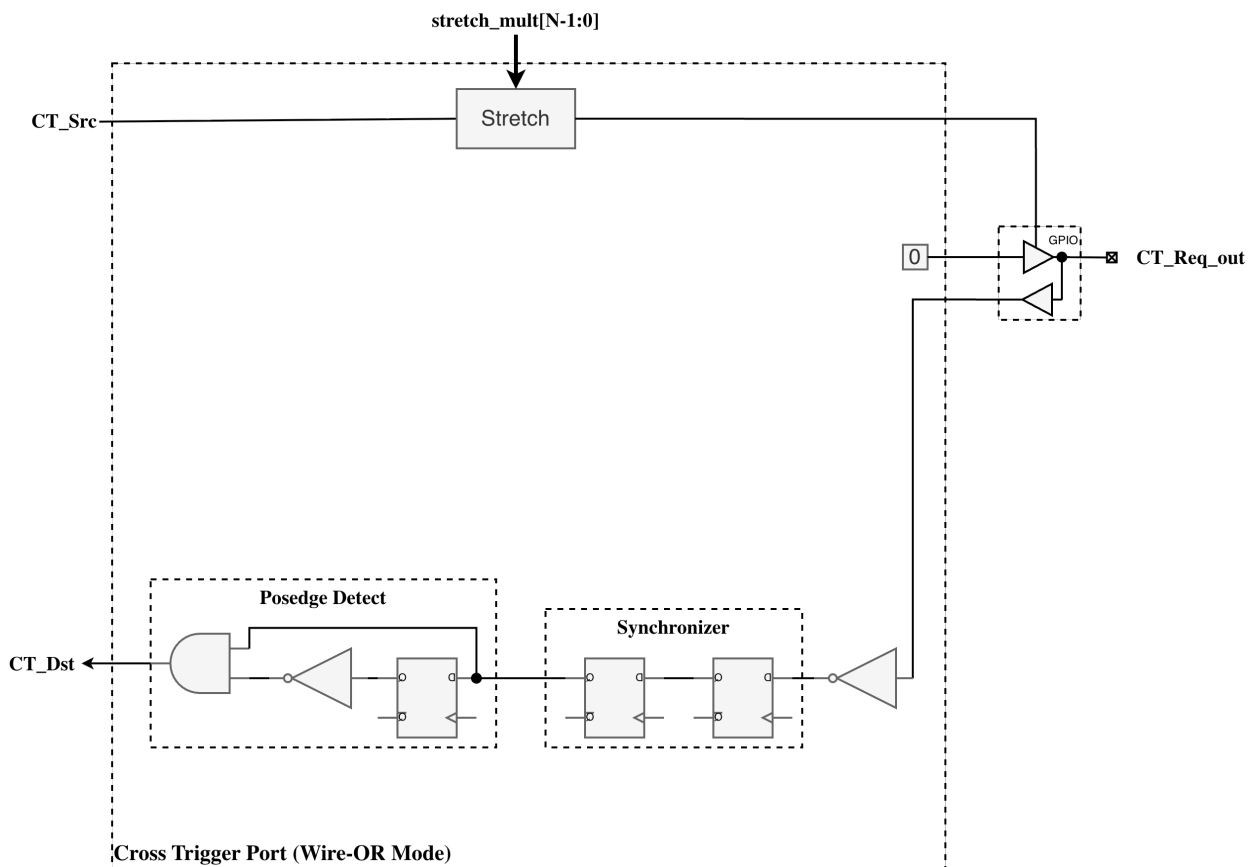
The pulse stretcher extends single-cycle core-side pulses to multi-cycle GPIO assertions lasting `stretch_mult + 1` clock cycles. The module implements a 16-bit countdown counter that loads the stretch multiplier value when a new pulse arrives and decrements to zero. If a new pulse arrives before the counter reaches zero, the counter restarts from the

stretch multiplier value, ensuring that rapid successive pulses merge into an extended assertion.

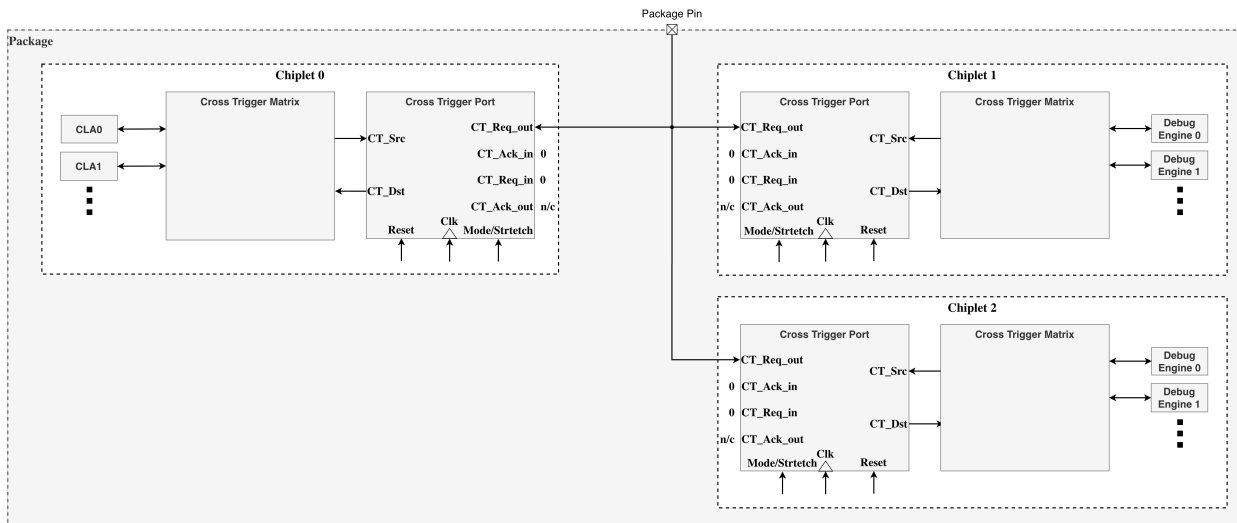
The output `stretched_pulse` remains asserted as long as the counter is non-zero. This stretched pulse drives the output enable control for the open-drain `CT_Req_out` GPIO pad, effectively pulling the shared wire low for the configured duration.

Each chiplet has an open-drain I/O connected to a single shared wire. Any connected chiplet may pull this signal low to assert a cross trigger. Depending on the electrical requirements, one or more internal pad pull-ups or external keepers ensure the cross trigger request signal remains high when no cross trigger is asserted. To ensure no connected chiplet misses a cross trigger pulse, all chiplets transmitting a trigger pulse must stretch the pulse a mutually agreed upon length of time.

Note that in wire-OR mode the `CT_Req_out` signal is connected to an open-drain configured I/O pad. For I/O pads that do have explicit open-drain control, the request data signal should connect to the output-enable control pin of the I/O pad, and the output data signal of the I/O pin should be driven statically low.

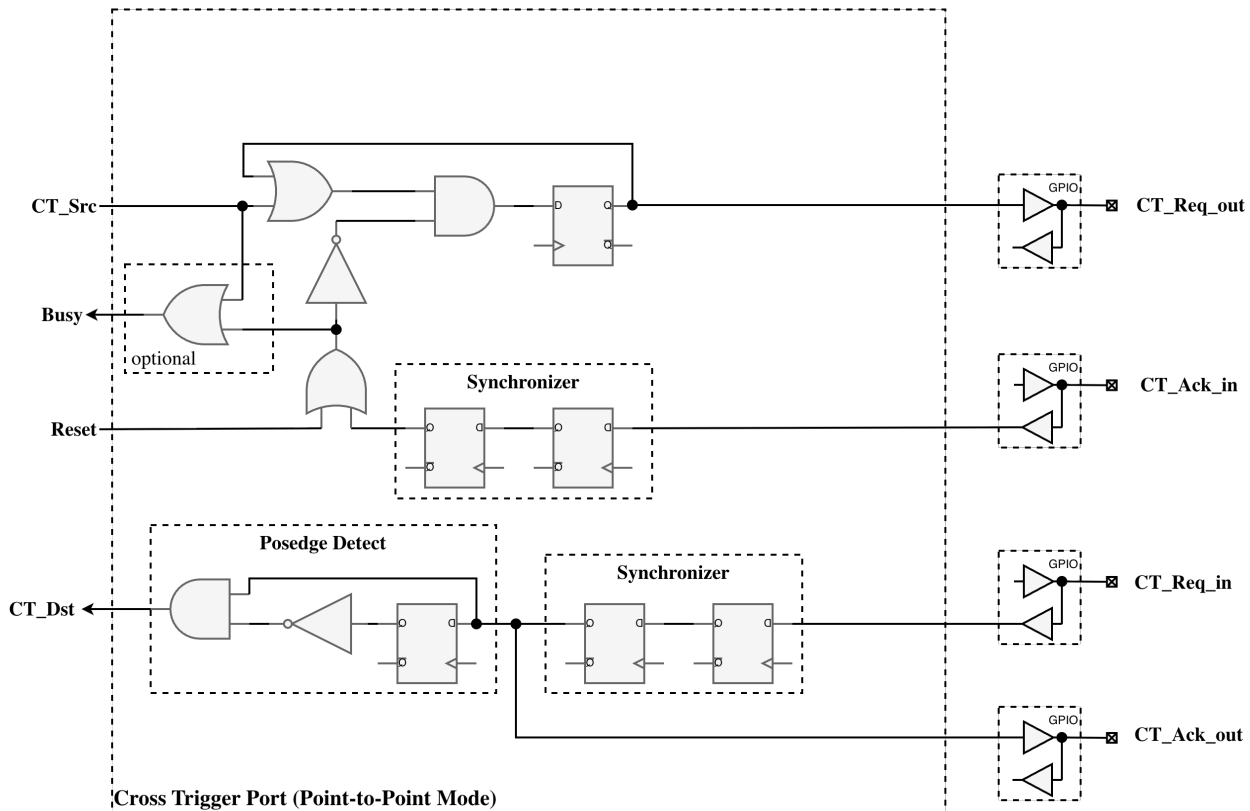


Any number of chiplets may have their `CT_Req_out` pads interconnected as shown in the following example diagram:

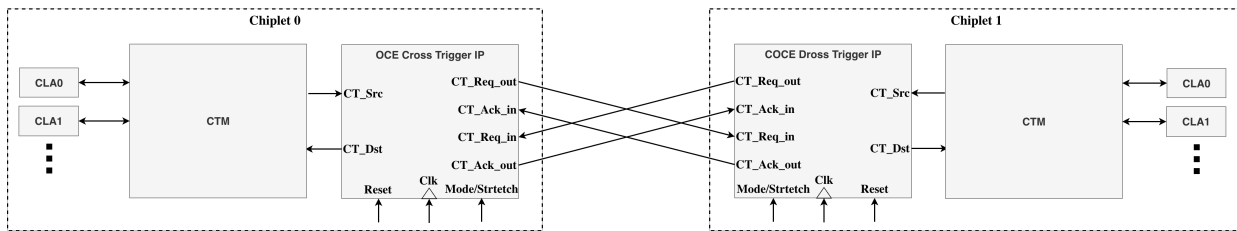


Point-to-Point Mode

Point-to-point mode utilizes a four-phase handshaking scheme to synchronize a cross trigger quickly and reliably between two chiplets without pulse stretching. While this scheme is simple and fast, it requires a dedicated CTP for each chiplet-to-chiplet interconnection, and each CTP requires four I/O pads for bi-directional communication, or two pads for unidirectional communication. All output pads are configured for push-pull operation.



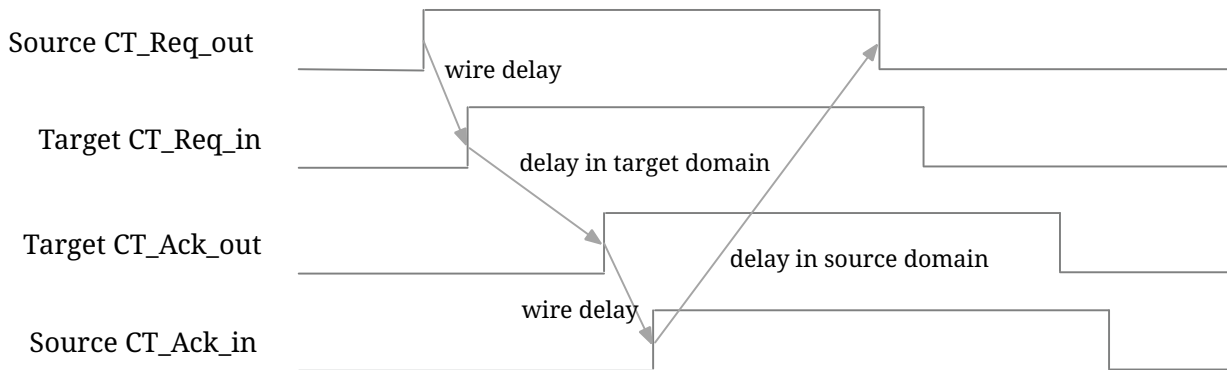
Any two chiplets may have their four cross trigger pads cross-coupled as shown in the following example diagram:



On the sender side, the receiver's *CT_Ack_out* is connected to the sender's *CT_Ack_in* pad. *CT_Req_out* asserts whenever *CT_Src* asserts. *CT_Req_out* will then remain asserted until *CT_Ack_in* asserts.

On the receiver side, the sender's *CT_Req_out* is connected to the receiver's *CT_Req_in* pad. On any positive edge of *CT_Req_in*, the receiver generates a pulse on *CT_Dst* and asserts *CT_Ack_out*. Deassertion proceeds in exactly the same sequence.

The timing of the request/acknowledge handshake is shown in the diagram below:



Synchronization and CDC Handling

All GPIO input signals are asynchronous with respect to the CTP system clock and require synchronization to prevent metastability. The `ctp_synchronizer` module implements three independent 2-FF synchronizer chains using the `prim_flop_2sync` primitive:

Signal	Used In Mode	Synchronization Purpose
<code>ct_req_out_din</code>	Wire-OR	Synchronizes open-drain wire state from remote chiplets before edge detection
<code>ct_req_in_din</code>	Point-to-Point	Synchronizes request signal from remote sender before receiver FSM
<code>ct_ack_in_din</code>	Point-to-Point	Synchronizes acknowledge signal from remote receiver before sender FSM

After synchronization, an optional inversion is applied if the `INVERT` configuration bit is set. This allows adaptation to GPIO pads with different polarity requirements. Edge detection and state machine processing occur only on synchronized (and optionally inverted) signals, ensuring all CTP core logic operates on glitch-free inputs.

Handshake State Machines

Point-to-point mode implements two independent state machines within the `ctp_handshake_ctrl` module, enabling full-duplex cross-triggering. The sender state machine controls outgoing cross-trigger requests, while the receiver state machine handles incoming requests. Both state machines operate synchronously in the system clock domain on synchronized GPIO inputs.

Sender State Machine:

Current State	Condition	Next State	Output Actions
SENDER_IDLE	<code>ct_src_i</code> asserted	SENDER_REQ_ASSERTED	Assert <code>ct_req_out=1</code>
SENDER_REQ_ASSERTED	<code>ct_ack_in_sync</code> asserted	SENDER_WAIT_ACK_DEASSERT	De-assert <code>ct_req_out=0</code>
SENDER_WAIT_ACK_DEASSERT	<code>ct_ack_in_sync</code> de-asserted	SENDER_IDLE	None (remain idle)
Any state	<code>reset_i</code> asserted	SENDER_IDLE	Force <code>ct_req_out=0</code>

Receiver State Machine:

Current State	Condition	Next State	Output Actions
RECEIVER_IDLE	<code>ct_req_in_sync</code> asserted	RECEIVER_ACK_ASSERTED	Assert <code>ct_ack_out=1</code> , generate <code>ct_dst</code> pulse
RECEIVER_ACK_ASSERTED	<code>ct_req_in_sync</code> de-asserted	RECEIVER_WAIT_REQ_DEASSERT	De-assert <code>ct_ack_out=0</code> , clear <code>ct_dst</code>
RECEIVER_WAIT_REQ_DEASSERT	<code>ct_req_in_sync</code> remains low	RECEIVER_IDLE	None
RECEIVER_WAIT_REQ_DEASSERT	<code>ct_req_in_sync</code> goes high	Remain in state	Continue waiting

The `ct_dst` output pulse is generated for a single clock cycle when the receiver state machine transitions from IDLE to ACK_ASSERTED. The busy signal is asserted whenever either state machine is not in its IDLE state, indicating an active cross-trigger transfer.

6.6.4. Interface

Signal Interface

Name	Direction	Description
mode	input	Indicates the operational mode of the CTP. 0 for wire-OR and 1 for peer-to-peer. The value at reset should be 0.
ct_src	input	Core-side cross trigger input pulse.
ct_dst	output	Core-side cross trigger output pulse.
busy	output	Optional signal indicating that an outgoing cross trigger pulse transfer is in progress.
reset	input	Resets the outgoing peer-to-peer handshake logic.
stretch_mult[N-1:0]	input	Configures the pulse duration of the pulse stretcher module for wire-OR mode.
ct_req_out_dout_en	output	Enables the output driver for the <i>CT_Req_out</i> pad. In wire-OR mode this signal is enabled by a cross trigger pulse from the pulse stretcher. In peer-to-peer mode it is always enabled.
ct_req_out_din_en	output	Enables the input buffer for the <i>CT_Req_out</i> pad. In wire-OR mode this signal is enabled and in peer-to-peer mode it is disabled.
ct_req_out_dout	output	The output data to drive on the <i>CT_Req_out</i> pad. In peer-to-peer mode the pulse capture latch controls this signal. In wire-OR mode it is held static-low.

Name	Direction	Description
ct_req_out_din	input	The input data that is driven on the <i>CT_Req_out</i> pad. In wire-OR mode this signal triggers a pulse on <i>ct_dst</i> after synchronization. In peer-to-peer mode it is unused.
ct_req_in_din_en	output	Enables the input buffer for the <i>CT_Req_in</i> pad. In peer-to-peer mode this signal is enabled and in wire-OR mode it is disabled.
ct_req_in_din	input	The input data that is driven on the <i>CT_Req_in</i> pad. In peer-to-peer mode this signal triggers a pulse on <i>ct_dst</i> after synchronization. In wire-OR mode it is unused.
ct_ack_in_din_en	output	Enables the input buffer for the <i>CT_Ack_in</i> pad. In peer-to-peer mode this signal is enabled and in wire-OR mode it is disabled.
ct_ack_in_din	input	The input data that is driven on the <i>CT_Ack_in</i> pad. This signal clears the cross trigger request output in peer-to-peer mode. In wire-OR mode it is unused.
ct_ack_out_dout_en	output	Enables the output driver for the <i>CT_Ack_out</i> pad. In peer-to-peer mode this signal is enabled and in wire-OR mode it is disabled.
ct_ack_out_dout	output	The output data to drive on the <i>CT_Ack_out</i> pad. In peer-to-peer mode the <i>ct_req_in</i> pulse synchronizer controls this signal. In wire-OR mode it is unused.

Dependencies

The CTP depends on the following components:

- [hw/common/pr im](#) - Primitive cells (synchronizers, pulse stretcher)

6.7. Cross Trigger Matrix

6.7.1. Overview

The Cross Trigger Matrix (CTM) is a crossbar that aggregates and statically routes cross trigger pulses between various sources and sinks. Typical connections to the CTM include Core Logic Analyzers (CLAs) and Cross Trigger Ports (CTPs), including other miscellaneous devices.

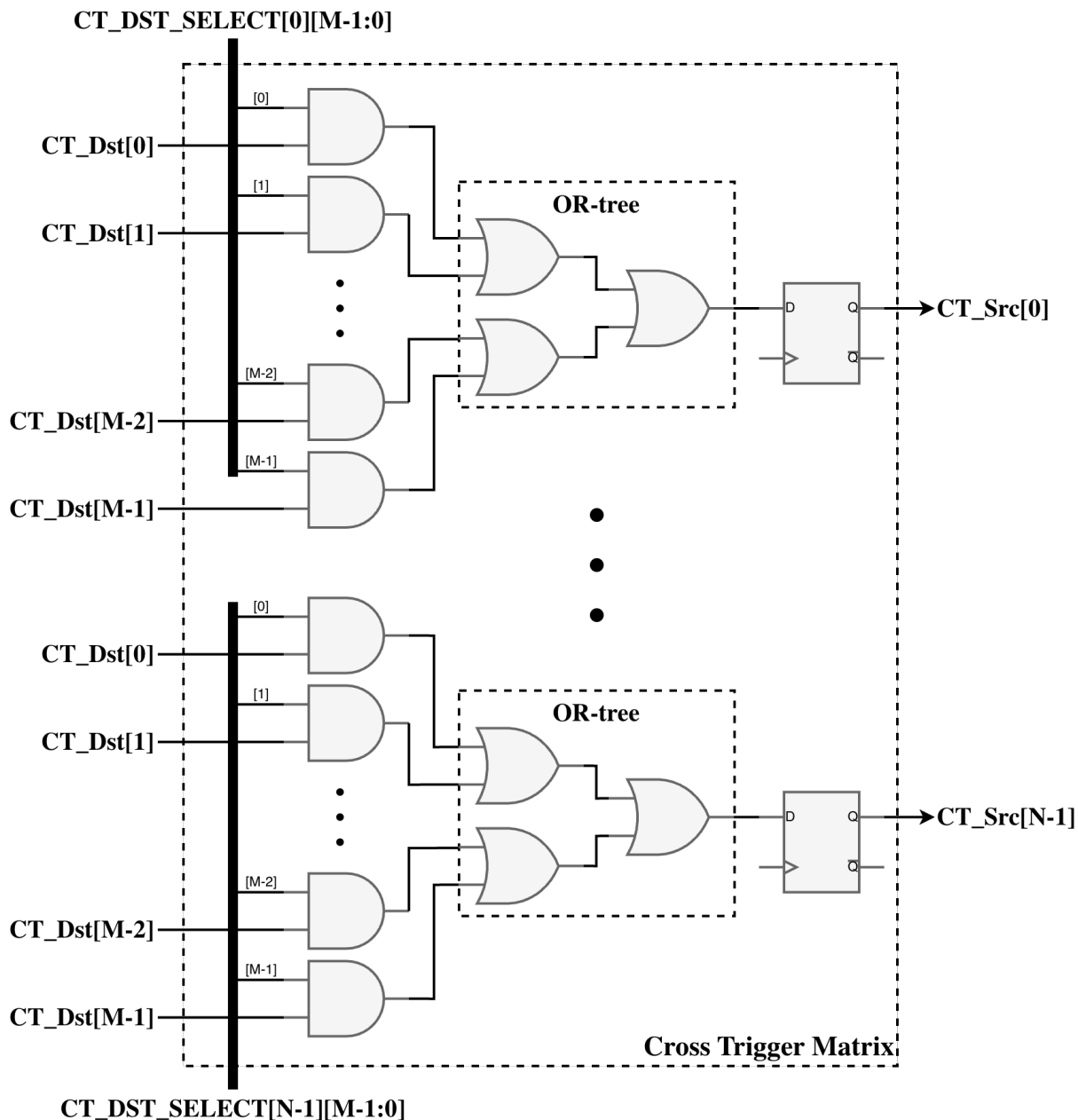
The CTM is compatible with CTPs operating in either of the two signaling modes, point-to-point or wire-OR. Typically, each device or logic module connected to the CTM will act as both a source and a sink for cross trigger pulses.

6.7.2. Configuration Parameters

Parameter	Range	Default	Description
NUM_CTP	1-32	16	Number of external Cross Trigger Ports
NUM_INT_CT	0-32	1	Number of internal cross triggers (to CLAs)

6.7.3. Architecture

The Cross Trigger Matrix is automatically generated by the `generate_ip.py` script with a fixed configuration of 26 source ports and 26 destination ports. The matrix implements a combinatorial AND-OR crossbar with registered outputs to route cross-trigger pulses between CTPs and internal cross-trigger sources.



The CTM is programmed at runtime via firmware or debug transactions. Proper configuration of the CTMs in each chiplet will minimize trigger latency and prevent loops in the SiP cross trigger network.

Routing Logic

The CTM routing implements a two-stage combinatorial path followed by output registration. Each of the 26 `CT_Src` output ports is driven by a dedicated `ctm_src_selector` module that performs the following operation:

Stage	Operation
Selection (AND)	Each CT_Dst[i] input is AND-gated with its corresponding select bit: $\text{selected_pulses}[i] = \text{ct_dst_i}[i] \ \& \ \text{select_mask}[i]$
Aggregation (OR)	All selected pulses are OR-reduced: $\text{ct_src_comb} = \text{selected_pulses}$
Registration	The combinatorial result is registered on clk_i rising edge: $\text{ct_src_o} \leftarrow \text{ct_src_comb}$

This architecture allows each CT_Src output to combine pulses from multiple CT_Dst inputs through OR-aggregation. Multiple CT_Src outputs can source from the same CT_Dst input, enabling broadcast distribution. The register stage prevents glitches on the output when configuration changes or when multiple input pulses arrive simultaneously.

Port Indexing

CTM ports are indexed as follows:

Port Index	Description
0 to N-1	External CTPs (NUM_CTP)
N to N+M-1	Internal CTPs (NUM_INT_CT)

Where N = NUM_CTP and M = NUM_INT_CT. This indexing is used for CTM ct_dst_i and ct_src_o port assignment and CTM routing configuration registers.

6.7.4. Interface

Signal Interface

Signal	Direction	Width	Description
clk_i	Input	1	System clock
rst_ni	Input	1	Active-low asynchronous reset
ct_dst_i	Input	N+M	Cross trigger inputs from CTPs and internal sources
ct_src_o	Output	N+M	Cross trigger outputs to CTPs and internal sinks
ct_src_ack_i	Input	M	Acknowledge from internal sinks (for handshake mode)
ct_dst_ack_o	Output	M	Acknowledge to internal sources (for handshake mode)
route_cfg_i	Input	varies	Routing configuration per output port

Where N = NUM_CTP and M = NUM_INT_CT.

Data Ports

The CTM has the following data ports:

- **CT_Src[N-1:0]** - Cross trigger pulse output signals from the CTM to CLAs, CTPs, or other cross trigger sinking devices, where N is the number of these cross trigger sinks.
- **CT_Dst[M-1:0]** - Cross trigger pulse input signals to the CTM from CLAs, CTPs, or other cross trigger generating devices, where M is the number of these cross trigger sources.

Dependencies

The CTM depends on the following components:

- `hw/common/prim` - Primitive cells

6.8. DTP JTAG Integration

This section describes how JTAG components are integrated within the DTP subsystem. For detailed component documentation, see:

- [JTAG Interface Unit](#) - Top-level JTAG control module
- [JTAG PTAP](#) - Primary TAP implementation
- [JTAG STAP](#) - Secondary TAP for 3DIC support

6.8.1. DTP JTAG Topology

The DTP instantiates a single JTAG Interface Unit that serves as the primary debug access point for the chiplet. The interface unit connects to:

- External JTAG pins (TCK, TDI, TDO, TMS, TRST)
- Cross Trigger Network for clock stop control
- SMC AXI fabric via JTAG2AXI
- SMC and SEP OTP controllers via JTAG2AXIL
- SMC and SEP CPU debug TAPs via STAPs

6.8.2. Clock Domains

The DTP operates in two clock domains:

Domain	Signal	Description
System Clock	clk_i	Main system clock for cross trigger logic and AXI interfaces
JTAG Clock	TCK	JTAG test clock for all TAP controller and scan logic

All cross-trigger signals are synchronized to the system clock domain within the CTPs.

6.8.3. Reset Architecture

- `rst_n_i`: Active-low asynchronous reset for cross trigger logic and AXI interfaces
- `pwr_on_rst_ni`: Asynchronous active-low power-on reset for JTAG logic, combined internally with `TRST_n`
- All registers reset to safe states
- Clock stop output resets to 0 (clocks running)
- JTAG TAP resets to Test-Logic-Reset state on `TRST` or power-on

6.8.4. Security Integration

The DTP integrates with the chiplet lifecycle controller:

- Security disable signals gate STAP selection for debug TAPs
- Lifecycle feature control determines which debug features are accessible
- JTAG2AXI access can be restricted based on lifecycle state

6.9. Clock Stop

This section describes DTP-level clock stop integration. For the clock stop controller implementation details, see [Cross Trigger Network](#).

From the DTP perspective, clock-stop handling combines JTAG DEBUG_CONTROL and CLA request sources into one functional halt path, while keeping CLA-only status separate for DEBUG_CONTROL readback.

- **JTAG `jtag_clock_stop`:** The JTAG Interface Unit captures the DEBUG_CONTROL `jtag_clock_stop` bit and drives it into the Cross Trigger Network (CTN) clock stop controller. It is OR-combined with CLA clock-stop requests to form the registered DTP `stop_clks_o` output that connects to chiplet functional clock gating.
- **Exported `cla_clock_stop_en`:** The DEBUG_CONTROL `cla_clock_stop_en` bit is exported at the DTP top level for downstream use (for example SMC/CLA debug policy). It is not used inside the CTN `stop_clks_o` aggregation.
- **CLA clock-stop indication:** The CTN OR-reduces `xtrig_clk_stop_req_i` to form the `cla_clock_stop` readback status (CLA requests only). This does not include the JTAG `jtag_clock_stop` bit.

6.9.1. Architecture

At a high level, the architecture is:

1. The JTAG Interface Unit captures DEBUG_CONTROL bits. The `jtag_clock_stop` bit is routed to the CTN clock stop controller; `cla_clock_stop_en` is exported at the DTP boundary.
2. The CTN OR-reduces `xtrig_clk_stop_req_i` for `cla_clock_stop` status and combines that aggregate with `jtag_clock_stop` for the registered `stop_clks_o` output.
3. The `cla_clock_stop` status is fed back into the DEBUG_CONTROL readback path so software or a debugger can observe whether any CLA clock-stop source is active.

This keeps CLA clock-stop observability distinct from the JTAG direct-stop path, while both can assert the functional chiplet halt through `stop_clks_o`.

Chapter 7. System Management Network

The SMN components are described in this document.

WARNING

This section is currently under development. More content will be added as the project matures.

Chapter 8. AXI-Over-UCIe Bridge

The AoU components are described in this document.

WARNING

This section is currently under development. More content will be added as the project matures.